

=====

FILE: settings_user_create_admin_guard_test.js

=====

```
/**
 * settings_user_create_admin_guard_test.js
 * ?????? ???? : ?????? ?? ?? ????? ?????? ???? (POST /api/settings/users) ???? ????? Admin ???.
 * Static assertion test ? no DB/HTTP, no PHI. node settings_user_create_admin_guard_test.js
 */
const fs = require('fs');
const src = fs.readFileSync(require('path').join(__dirname, 'server.js'), 'utf8');

let pass = 0, fail = 0;
const ok = (c, m) => { if (c) { pass++; console.log(' PASS', m); } else { fail++; console.log(' FAIL', m); }
};

// isolate the POST /api/settings/users handler body (up to the next app.<verb>)
const start = src.indexOf("app.post('/api/settings/users')");
ok(start > -1, "POST /api/settings/users handler exists");
const rest = src.slice(start);
const next = rest.indexOf("app.put('/api/settings/users/:id')");
const body = rest.slice(0, next > -1 ? next : 4000);

// Batch 2: the inline `role !== 'Admin'` check was unified into the requireTenantAdmin middleware
// (runs before the handler; identity from session). Same Admin-only invariant, centralized + tested.
ok(/requireTenantAdmin\\(\\{[^}]*action:\\s*'BLOCKED_USER_CREATE'\\.test(body), "guard: requireTenantAdmin (Admin-
only) bound to the route");
ok(/BLOCKED_USER_CREATE\\.test(body), "audit action on blocked create is preserved");
// the Admin gate is middleware -> it executes (and 403s) BEFORE user creation runs.
// Batch 4D: the actual INSERT moved into user_provisioning.createSystemUserWithTenantLink; the route
// invokes it only after the guard chain. Assert the guard precedes the creation CALL.
const guardIdx = body.indexOf('requireTenantAdmin');
const createIdx = body.indexOf('createSystemUserWithTenantLink');
ok(guardIdx > -1 && createIdx > -1 && guardIdx < createIdx, "admin guard precedes the user-creation call");
// the real INSERT lives in the helper (invoked only after the guard)
const upSrc = fs.readFileSync(require('path').join(__dirname, 'user_provisioning.js'), 'utf8');
ok(/INSERT INTO system_users\\.test(upSrc), "user creation INSERT is in user_provisioning helper");
// identity must come from session, never req.body (no req.body.role decides authz)
ok(/requireRole\\('settings'\\)\\.test(body), "still layered behind requireRole('settings')");
// the shared guard module is what returns 403 for non-admins (behavioral proof in rbac_guards_test.js)
const guardSrc = fs.readFileSync(require('path').join(__dirname, 'rbac_guards.js'), 'utf8');
ok(/requireTenantAdmin\\.test(guardSrc) && /status\\(403\\)\\.test(guardSrc), "rbac_guards.requireTenantAdmin retu
rns 403");

console.log(`\n${fail === 0 ? 'ALL PASS' : 'FAILURES'}: ${pass} passed, ${fail} failed`);
process.exit(fail === 0 ? 0 : 1);
```

=====

FILE: sms_service.js

=====

```

/**
 * sms_service.js
 * Extensible SMS Gateway Client supporting Twilio, Unifonic, and a Mock provider.
 * Built using Node's native https module to avoid external dependencies.
 */

const https = require('https');
const fs = require('fs');
const path = require('path');
const { pool } = require('./db_postgres');
require('dotenv').config();

const PROVIDER = process.env.SMS_PROVIDER || 'mock';
const LOG_DIR = path.join(__dirname, 'backups');
const LOG_FILE = path.join(LOG_DIR, 'sms_sent.log');

// Ensure log directory exists for the mock provider
if (PROVIDER === 'mock' && !fs.existsSync(LOG_DIR)) {
  fs.mkdirSync(LOG_DIR, { recursive: true });
}

/**
 * Log the SMS sending operation in the audit_trail table
 */
async function logSMSAudit(to, text, eventType, status, errorMsg = '') {
  try {
    const details = `SMS to: ${to} | Event: ${eventType} | Status: ${status}${errorMsg ? ` | Error: ${errorMsg}` : ''} | Msg: "${text.slice(0, 100)}..."`;
    await pool.query(
      'INSERT INTO audit_trail (user_id, username, action, module, details, ip_address) VALUES ($1, $2, $3, $4, $5, $6)',
      [null, 'SYSTEM_SMS', 'SMS_SEND', 'Notifications', details, '127.0.0.1']
    );
  } catch (e) {
    console.error('[SMS SERVICE] Failed to log audit trail:', e.message);
  }
}

/**
 * Send an SMS using Twilio REST API
 */
function sendTwilio(to, text) {
  return new Promise((resolve, reject) => {
    const sid = process.env.SMS_TWILIO_SID;
    const token = process.env.SMS_TWILIO_TOKEN;
    const from = process.env.SMS_TWILIO_FROM;

    if (!sid || !token || !from) {
      return reject(new Error('Missing Twilio credentials (SMS_TWILIO_SID, SMS_TWILIO_TOKEN, SMS_TWILIO_FROM)'));
    }

    const auth = Buffer.from(`${sid}:${token}`).toString('base64');

```

```

const postData = new URLSearchParams({
  To: to,
  From: from,
  Body: text
}).toString();

const options = {
  hostname: 'api.twilio.com',
  port: 443,
  path: `/2010-04-01/Accounts/${sid}/Messages.json`,
  method: 'POST',
  headers: {
    'Authorization': `Basic ${auth}`,
    'Content-Type': 'application/x-www-form-urlencoded',
    'Content-Length': postData.length
  }
};

const req = https.request(options, (res) => {
  let body = '';
  res.on('data', chunk => body += chunk);
  res.on('end', () => {
    if (res.statusCode >= 200 && res.statusCode < 300) {
      resolve(JSON.parse(body));
    } else {
      reject(new Error(`Twilio API returned HTTP ${res.statusCode}: ${body}`));
    }
  });
});

req.on('error', reject);
req.write(postData);
req.end();
});
}

/**
 * Send an SMS using Unifonic REST API (Saudi Arabia compliance)
 */
function sendUnifonic(to, text) {
  return new Promise((resolve, reject) => {
    const apiKey = process.env.SMS_UNIFONIC_API_KEY;
    const sender = process.env.SMS_UNIFONIC_SENDER;

    if (!apiKey || !sender) {
      return reject(new Error('Missing Unifonic credentials (SMS_UNIFONIC_API_KEY, SMS_UNIFONIC_SENDER)'));
    }

    const postData = JSON.stringify({
      AppSid: apiKey,
      Recipient: to.replace('+', ''), // Unifonic expects digits only
      Body: text,
      SenderID: sender
    });
  });
}

```

```

});

const options = {
  hostname: 'api.unifonic.com',
  port: 443,
  path: '/rest/SMS/messages',
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Content-Length': postData.length
  }
};

const req = https.request(options, (res) => {
  let body = '';
  res.on('data', chunk => body += chunk);
  res.on('end', () => {
    if (res.statusCode >= 200 && res.statusCode < 300) {
      resolve(JSON.parse(body));
    } else {
      reject(new Error(`Unifonic API returned HTTP ${res.statusCode}: ${body}`));
    }
  });
});

req.on('error', reject);
req.write(postData);
req.end();
});
}

/**
 * Send an SMS message
 * @param {string} to - Recipient phone number (e.g., '+966500000000')
 * @param {string} text - Message body
 * @param {string} eventType - Event type for audit logging (e.g., 'APPOINTMENT_CONFIRM')
 * @param {object} metadata - Optional metadata
 */
async function sendSMS(to, text, eventType = 'GENERAL', metadata = {}) {
  if (!to || !text) {
    console.warn('[SMS SERVICE] Missing recipient or message body.');
```

return false;

```

  }

  const formattedTo = String(to).trim();
  console.log(`[SMS SERVICE] Sending ${eventType} to ${formattedTo} using ${PROVIDER}...`);

  if (PROVIDER === 'mock') {
    const logEntry = `[${new Date().toISOString()}] EVENT: ${eventType} | TO: ${formattedTo} | MSG: "${text}"\n`;
    try {
      fs.appendFileSync(LOG_FILE, logEntry);
      console.log(`[SMS MOCK LOGGED] To: ${formattedTo} | Msg: ${text}`);
      await logSMSAudit(formattedTo, text, eventType, 'SUCCESS MOCK');
```

```

        return true;
    } catch (e) {
        console.error('[SMS SERVICE] Failed to write to mock log file:', e.message);
        await logSMSAudit(formattedTo, text, eventType, 'FAILED MOCK', e.message);
        return false;
    }
}

try {
    let result;
    if (PROVIDER === 'twilio') {
        result = await sendTwilio(formattedTo, text);
    } else if (PROVIDER === 'unifonic') {
        result = await sendUnifonic(formattedTo, text);
    } else {
        throw new Error(`Unsupported SMS provider: ${PROVIDER}`);
    }

    console.log('[SMS SERVICE] SMS sent successfully via API.');
    await logSMSAudit(formattedTo, text, eventType, 'SUCCESS_API');
    return true;
} catch (err) {
    console.error('[SMS SERVICE] API sending failed:', err.message);
    await logSMSAudit(formattedTo, text, eventType, 'FAILED_API', err.message);
    return false;
}
}

module.exports = { sendSMS };

```

```

=====
FILE: sms_service_test.js
=====

```

```

/**
 * sms_service_test.js
 * Integration test for SMS notifications across appointments, lab results, and radiology reports.
 */

```

```

const { spawn } = require('child_process');
const http = require('http');
const { Pool } = require('pg');
const fs = require('fs');
const path = require('path');
const assert = require('assert');
const bcrypt = require('bcryptjs');
require('dotenv').config();

const DB_HOST = process.env.DB_HOST || 'localhost';
const DB_PORT = process.env.DB_PORT || 5432;
const DB_NAME = process.env.DB_NAME || 'nama_medical_web';
const DB_USER = process.env.DB_USER || 'postgres';

```

```

const DB_PASSWORD = process.env.DB_PASSWORD || 'postgres';

const TEST_PORT = 3006;
const TEST_USERNAME = 'sms_doctor';
const TEST_PASSWORD = 'EXAMPLE_PASSWORD';
const LOG_FILE = path.join(__dirname, 'backups', 'sms_sent.log');
const EMAIL_LOG_FILE = path.join(__dirname, 'backups', 'email_sent.log');

const pool = new Pool({
  host: DB_HOST,
  port: parseInt(DB_PORT),
  database: DB_NAME,
  user: DB_USER,
  password: DB_PASSWORD
});

function makeRequest(method, path, body = null, headers = {}) {
  return new Promise((resolve, reject) => {
    const req = http.request({
      hostname: 'localhost',
      port: TEST_PORT,
      path: path,
      method: method,
      headers: {
        'Content-Type': 'application/json',
        ...headers
      }
    }, (res) => {
      let data = '';
      res.on('data', chunk => data += chunk);
      res.on('end', () => {
        try {
          resolve({
            statusCode: res.statusCode,
            headers: res.headers,
            body: JSON.parse(data)
          });
        } catch (e) {
          resolve({
            statusCode: res.statusCode,
            headers: res.headers,
            body: data
          });
        }
      });
    });
    req.on('error', reject);
    if (body) {
      req.write(JSON.stringify(body));
    }
    req.end();
  });
}

```

```

async function runTest() {
  console.log('=== Running SMS Gateway Integration Test ===');

  // Ensure lab_results has correct columns by running migration
  try {
    const migrationPath = path.join(__dirname, 'migrations', 'e3_02_lab_results_up.sql');
    const sql = fs.readFileSync(migrationPath, 'utf8');
    await pool.query(sql);
    console.log('? Ran migration e3_02_lab_results_up.sql on test database.');
```

```

  } catch (migErr) {
    console.error('Failed to run migration:', migErr.message);
  }

  // 1. Clean up mock log files
  if (fs.existsSync(LOG_FILE)) {
    fs.unlinkSync(LOG_FILE);
  }
  if (fs.existsSync(EMAIL_LOG_FILE)) {
    fs.unlinkSync(EMAIL_LOG_FILE);
  }

  // 2. Insert test data
  const patientId = 9991;
  const doctorId = 9992;
  const orderId = 9993;

  const client = await pool.connect();
  try {
    await client.query("SET app.tenant_id = '1'");

    // Clean up first
    await client.query('DELETE FROM portal_users WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM lab_results WHERE order_id = $1', [orderId]);
    await client.query('DELETE FROM lab_radiology_orders WHERE id = $1', [orderId]);
    await client.query('DELETE FROM appointments WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
    await client.query('DELETE FROM hr_employees WHERE national_id = \'9999999999\'');
    await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorId]);
    await client.query('DELETE FROM system_users WHERE id = $1', [doctorId]);

    // Insert patient
    await client.query(
      'INSERT INTO patients (id, name_en, name_ar, phone, tenant_id) VALUES ($1, $2, $3, $4, 1)',
      [patientId, 'SMS Test Patient', '???? ???? ????', '+966555555555']
    );

    // Insert patient portal user to receive emails
    await client.query(
      'INSERT INTO portal_users (patient_id, username, password_hash, email, phone, tenant_id) VALUES ($1, $2, $3, $4, $5, 1)',
      [patientId, 'sms_patient_portal', 'pwd', 'patient@example.com', '+966555555555']
    );

    // Insert doctor user

```

```

    const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
    await client.query(
      'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
      [doctorId, 'sms_doctor', hashedPassword, 'SMS Test Doctor', 'Doctor', 'General', ['"appointments",
"lab", "radiology"']]
    );

    // Associate doctor with tenant 1
    await client.query('INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)', [d
octorId]);

    // Insert doctor employee record to resolve phone & email
    await client.query(
      'INSERT INTO hr_employees (emp_number, name_en, name_ar, national_id, phone, email, job_title) VAL
UES ($1, $2, $3, $4, $5, $6, $7)',
      ['EMP9992', 'SMS Test Doctor', '????? ?????? ???????', '9999999999', '+966544444444', 'doctor@examp
le.com', 'Doctor']
    );

    // Insert order
    await client.query(
      'INSERT INTO lab_radiology_orders (id, patient_id, doctor_id, order_type, status, tenant_id) VALUE
S ($1, $2, $3, $4, $5, $6)',
      [orderId, patientId, doctorId, 'Lab', 'Requested', 1]
    );
  } finally {
    client.release();
  }

  // 3. Start the server on TEST_PORT
  const server = spawn('node', [path.join(__dirname, 'server.js')], {
    env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'test', SKIP_DB_INIT: 'true' }
  });

  server.stdout.on('data', (data) => console.log(`[Server] ${data.toString().trim()}`));
  server.stderr.on('data', (data) => console.error(`[Server Error] ${data.toString().trim()}`));

  // Wait for server to start
  await new Promise(resolve => setTimeout(resolve, 2500));

  try {
    // --- Step 0: Login to get session cookie ---
    console.log('Logging in to get session cookie...');
    const loginRes = await makeRequest('POST', '/api/auth/login', {
      username: 'sms_doctor',
      password: TEST_PASSWORD
    });
    assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');

    const setCookie = loginRes.headers['set-cookie'];
    assert.ok(setCookie, 'Should receive set-cookie header');
    const cookie = setCookie[0].split(';')[0];
    console.log(`? Logged in successfully. Cookie:', cookie);
  }

```



```

const authHeaders = { 'Cookie': cookie };

// --- Test Case 1: Appointment Confirmation SMS ---
console.log('Testing Case 1: Appointment Confirmation...');
const apptRes = await makeRequest('POST', '/api/appointments', {
  patient_name: 'SMS Test Patient',
  patient_id: patientId,
  doctor_name: 'SMS Test Doctor',
  department: 'General',
  appt_date: '2026-07-01',
  appt_time: '10:00',
  fee: 0
}, authHeaders);

assert.strictEqual(apptRes.statusCode, 200, 'Appointment booking should succeed');

// Verify log file contains appointment confirmation
await new Promise(resolve => setTimeout(resolve, 800)); // wait for async SMS/Email
const logContent1 = fs.readFileSync(LOG_FILE, 'utf8');
assert.ok(logContent1.includes('APPOINTMENT_CONFIRM'), 'Log should contain APPOINTMENT_CONFIRM event')
;

assert.ok(logContent1.includes('+966555555555'), 'Log should contain patient phone number');
assert.ok(logContent1.includes('SMS Test Doctor'), 'Log should contain doctor name');
console.log('? Appointment confirmation SMS successfully sent and logged.');

const emailContent1 = fs.readFileSync(EMAIL_LOG_FILE, 'utf8');
assert.ok(emailContent1.includes('patient@example.com'), 'Email log should contain patient email');
assert.ok(emailContent1.includes('Appointment Confirmation'), 'Email log should contain appointment su
bject');
console.log('? Appointment confirmation email successfully sent and logged.');

// --- Test Case 2: Lab Result Ready SMS (Auto-Verified) ---
console.log('Testing Case 2: Lab Result Ready (Auto-Verified)...');
fs.writeFileSync(LOG_FILE, ''); // Clear log file
fs.writeFileSync(EMAIL_LOG_FILE, ''); // Clear log file

const resultRes = await makeRequest('POST', '/api/lab/results', {
  lab_sample_id: null,
  loinc: '12345-6',
  test_name: 'Glucose',
  value: '95',
  unit: 'mg/dL',
  normal_range: '70-100',
  ref_low: 70,
  ref_high: 100,
  order_id: orderId
}, authHeaders);

assert.strictEqual(resultRes.statusCode, 200, 'Result submission should succeed');

await new Promise(resolve => setTimeout(resolve, 500));
const logContent2 = fs.readFileSync(LOG_FILE, 'utf8');
assert.ok(logContent2.includes('LAB_RESULT_READY'), 'Log should contain LAB_RESULT_READY event');

```

```

    assert.ok(logContent2.includes('+966555555555'), 'Log should contain patient phone');
    console.log('? Lab result ready SMS successfully sent and logged.');
```



```

    const emailContent2 = fs.readFileSync(EMAIL_LOG_FILE, 'utf8');
    assert.ok(emailContent2.includes('patient@example.com'), 'Email log should contain patient email');
    assert.ok(emailContent2.includes('Lab Results Ready'), 'Email log should contain lab result subject');
    console.log('? Lab result ready email successfully sent and logged.');
```



```

    // Verify database order was updated
    const dbOrder = (await pool.query('SELECT sms_sent FROM lab_radiology_orders WHERE id = $1', [orderId]
    )).rows[0];
    assert.strictEqual(dbOrder.sms_sent, 1, 'sms_sent should be set to 1 in the database');
    console.log('? Database sms_sent updated successfully.');
```



```

    // --- Test Case 3: Critical Lab Result Doctor Alert ---
    console.log('Testing Case 3: Critical Lab Result Doctor Alert...');
    fs.writeFileSync(LOG_FILE, ''); // Clear log file
    fs.writeFileSync(EMAIL_LOG_FILE, ''); // Clear log file
```



```

    const criticalRes = await makeRequest('POST', '/api/lab/results', {
      lab_sample_id: null,
      loinc: '12345-6',
      test_name: 'Potassium',
      value: '2.5', // Critical low
      unit: 'mmol/L',
      normal_range: '3.5-5.0',
      ref_low: 3.5,
      ref_high: 5.0,
      order_id: orderId
    }, authHeaders);
```



```

    await new Promise(resolve => setTimeout(resolve, 500));
    const logContent3 = fs.readFileSync(LOG_FILE, 'utf8');
    assert.ok(logContent3.includes('LAB_CRITICAL_ALERT'), 'Log should contain LAB_CRITICAL_ALERT event');
    assert.ok(logContent3.includes('+966544444444'), 'Log should contain doctor phone');
    assert.ok(logContent3.includes('Potassium'), 'Log should contain test name');
    console.log('? Critical lab result doctor alert SMS successfully sent and logged.');
```



```

    const emailContent3 = fs.readFileSync(EMAIL_LOG_FILE, 'utf8');
    assert.ok(emailContent3.includes('doctor@example.com'), 'Email log should contain doctor email');
    assert.ok(emailContent3.includes('Critical Result'), 'Email log should contain critical result subject
    ');

    console.log('? Critical lab result doctor email alert successfully sent and logged.');
```



```

    console.log('? SMS & Email Notification Integration Test passed successfully!');
  } catch (e) {
    console.error('? Test failed:', e);
    process.exitCode = 1;
  } finally {
    // Clean up
    server.kill();
    const cleanClient = await pool.connect();
    try {
      await cleanClient.query("SET app.tenant_id = '1'");
    }
  }
}

```

```

        await cleanClient.query('DELETE FROM portal_users WHERE patient_id = $1', [patientId]);
        await cleanClient.query('DELETE FROM lab_results WHERE order_id = $1', [orderId]);
        await cleanClient.query('DELETE FROM lab_radiology_orders WHERE id = $1', [orderId]);
        await cleanClient.query('DELETE FROM appointments WHERE patient_id = $1', [patientId]);
        await cleanClient.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await cleanClient.query('DELETE FROM hr_employees WHERE national_id = \'9999999999\');
        await cleanClient.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorId]);
        await cleanClient.query('DELETE FROM system_users WHERE id = $1', [doctorId]);
    } finally {
        cleanClient.release();
    }
    await pool.end();
    if (fs.existsSync(LOG_FILE)) {
        fs.unlinkSync(LOG_FILE);
    }
    if (fs.existsSync(EMAIL_LOG_FILE)) {
        fs.unlinkSync(EMAIL_LOG_FILE);
    }
    console.log('? Cleanup complete');
}
}

```

```
runTest();
```

```
=====
```

```
FILE: specialties_next2_integration_test.js
```

```
=====
```

```

/**
 * specialties_next2_integration_test.js ? Integration test for G21, G24, and G25 clinical modules.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3024;
const TEST_USERNAME = 'spec_next2_doctor';
const TEST_PASSWORD = 'SPEC_NEXT2_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
    return new Promise((resolve, reject) => {
        const payload = body ? JSON.stringify(body) : '';
        const req = http.request({
            hostname: 'localhost',

```

```

    port: TEST_PORT,
    path,
    method,
    headers: {
      'Content-Type': 'application/json',
      'Content-Length': Buffer.byteLength(payload),
      ...headers
    }
  }, (res) => {
    let data = '';
    res.on('data', chunk => data += chunk);
    res.on('end', () => {
      try {
        const parsed = data ? JSON.parse(data) : {};
        resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
      } catch (e) {
        resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
      }
    });
  });
  req.on('error', reject);
  if (payload) req.write(payload);
  req.end();
});
}

```

```

async function runTests() {
  console.log('--- STARTING G21, G24, & G25 MODULE INTEGRATION TESTS ---');

  const patientId = 9977;
  const doctorUserId = 9978;
  const client = await pool.connect();

  try {
    console.log('Setting up test data...');
    await client.query("SET app.tenant_id = '1'");

    // Clean up old test data
    await client.query('DELETE FROM obgyn_pregnancies WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM psychiatric_evaluations WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM dermatology_lesions WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
    await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
    await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);

    // Insert patient
    await client.query(
      'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
      [patientId, 'Spec Next2 Patient', '???? ??????? ??????? 2']
    );

    // Insert doctor user
    const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
    await client.query(

```

```

    'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
    [doctorUserId, TEST_USERNAME, hashedPassword, 'Dr. Specialist OBGYN', 'Doctor', 'OBGYN', '["patien
ts", "prescriptions"]']
  );

  // Associate doctor with tenant 1
  await client.query(
    'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
    [doctorUserId]
  );

  console.log('Spawning test server...');
  serverProcess = spawn('node', ['server.js'], {
    env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' },
    stdio: 'pipe'
  });

  serverProcess.stdout.on('data', (data) => {
    console.log(`[SERVER] ${data.toString().trim()}`);
  });

  serverProcess.stderr.on('data', (data) => {
    console.error(`[SERVER ERR] ${data.toString().trim()}`);
  });

  // Wait 6 seconds for server to boot
  await new Promise(resolve => setTimeout(resolve, 6000));

  console.log('Logging in...');
  const loginRes = await makeRequest('POST', '/api/auth/login', {
    username: TEST_USERNAME,
    password: TEST_PASSWORD
  });

  assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
  console.log('? Logged in successfully.');
```

```

  // Extract session cookie
  const setCookie = loginRes.headers['set-cookie'];
  assert.ok(setCookie, 'Should receive session cookie');
  const cookie = setCookie[0].split(';')[0];

  // 1. Test POST /api/obgyn/pregnancies
  console.log('Testing create OBGYN pregnancy record...');
  const obgynRes = await makeRequest('POST', '/api/obgyn/pregnancies', {
    patient_id: patientId,
    gravida: 2,
    para: 1,
    abortions: 0,
    living: 1,
    lmp_date: '2026-01-01',
    gestational_weeks: 25,
    notes: 'Uneventful pregnancy'
  });

```

```

}, { 'Cookie': cookie });

assert.strictEqual(obgynRes.statusCode, 200, 'Should return 200 OK');
assert.strictEqual(obgynRes.body.success, true);
assert.ok(obgynRes.body.id);
const pregId = obgynRes.body.id;
console.log(`? Pregnancy record created. ID: ${pregId}`);

// 2. Test GET /api/obgyn/pregnancies/patient/:patient_id
console.log('Testing get patient pregnancy records...');
const getObgynRes = await makeRequest('GET', `/api/obgyn/pregnancies/patient/${patientId}`, null, { 'Cookie': cookie });
assert.strictEqual(getObgynRes.statusCode, 200);
assert.strictEqual(getObgynRes.body.length, 1);
assert.strictEqual(getObgynRes.body[0].id, pregId);
assert.strictEqual(getObgynRes.body[0].gravida, 2);
assert.strictEqual(getObgynRes.body[0].gestational_weeks, 25); // (June 30 - Jan 1) / 7
console.log('? Patient pregnancy records retrieved.');
```

```

// 3. Test POST /api/psychiatry/evaluations
console.log('Testing create psychiatric evaluation...');
const psychRes = await makeRequest('POST', '/api/psychiatry/evaluations', {
  patient_id: patientId,
  mse_appearance: 'Neat',
  mse_behavior: 'Cooperative',
  mse_speech: 'Normal',
  mse_mood: 'Euthymic',
  mse_affect: 'Congruent',
  mse_insight: 'Good',
  mse_judgment: 'Intact',
  diagnostic_summary: 'No active mental health issues'
}, { 'Cookie': cookie });

assert.strictEqual(psychRes.statusCode, 200);
assert.ok(psychRes.body.id);
const psychId = psychRes.body.id;
console.log(`? Psychiatric evaluation created. ID: ${psychId}`);

// 4. Test GET /api/psychiatry/evaluations/patient/:patient_id
console.log('Testing get patient psychiatric evaluations...');
const getPsychRes = await makeRequest('GET', `/api/psychiatry/evaluations/patient/${patientId}`, null, { 'Cookie': cookie });
assert.strictEqual(getPsychRes.statusCode, 200);
assert.strictEqual(getPsychRes.body.length, 1);
assert.strictEqual(getPsychRes.body[0].id, psychId);
assert.strictEqual(getPsychRes.body[0].mse_appearance, 'Neat');
assert.strictEqual(getPsychRes.body[0].mse_judgment, 'Intact');
console.log('? Patient psychiatric evaluations retrieved.');
```

```

// 5. Test POST /api/dermatology/lesions
console.log('Testing create dermatology lesion...');
const dermRes = await makeRequest('POST', '/api/dermatology/lesions', {
  patient_id: patientId,
  body_site: 'Right arm',

```

```

        lesion_type: 'Papule',
        color: 'Red',
        size_mm: 3.5,
        distribution: 'Isolated',
        biopsy_taken: true,
        notes: 'Slightly itchy'
    }, { 'Cookie': cookie });

    assert.strictEqual(dermRes.statusCode, 200);
    assert.ok(dermRes.body.id);
    const dermId = dermRes.body.id;
    console.log(`? Dermatology lesion created. ID: ${dermId}`);

    // 6. Test GET /api/dermatology/lesions/patient/:patient_id
    console.log('Testing get patient dermatology lesions...');
    const getDermRes = await makeRequest('GET', `/api/dermatology/lesions/patient/${patientId}`, null, { '
Cookie': cookie });
    assert.strictEqual(getDermRes.statusCode, 200);
    assert.strictEqual(getDermRes.body.length, 1);
    assert.strictEqual(getDermRes.body[0].id, dermId);
    assert.strictEqual(getDermRes.body[0].body_site, 'Right arm');
    assert.strictEqual(parseFloat(getDermRes.body[0].size_mm), 3.5);
    assert.strictEqual(getDermRes.body[0].biopsy_taken, true);
    console.log(`? Patient dermatology lesions retrieved.`);

    } finally {
        console.log('Cleaning up test data...');
        await client.query("SET app.tenant_id = '1'");
        await client.query('DELETE FROM obgyn_pregnancies WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM psychiatric_evaluations WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM dermatology_lesions WHERE patient_id = $1', [patientId]);
        await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
        await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
        await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);
        client.release();

        if (serverProcess) {
            console.log('Killing test server...');
            serverProcess.kill();
        }
    }

    console.log(`? G21, G24, & G25 Module Integration Tests passed successfully!\n`);
}

if (require.main === module) {
    runTests().catch(async err => {
        console.error(`? Test failed:`, err);
        // Wait 3 seconds for server logs to flush
        await new Promise(resolve => setTimeout(resolve, 3000));
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    });
}

```

```
module.exports = { runTests };
```

```
=====
FILE: specialties_next_integration_test.js
=====
```

```
/**
 * specialties_next_integration_test.js ? Integration test for G16, G19, and G20 clinical modules.
 */
'use strict';
```

```
process.env.NODE_ENV = 'staging';
```

```
const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');
```

```
const TEST_PORT = 3023;
const TEST_USERNAME = 'spec_next_doctor';
const TEST_PASSWORD = 'SPEC_NEXT_PASSWORD_PLACEHOLDER';
```

```
let serverProcess;
```

```
function makeRequest(method, path, body, headers = {}) {
  return new Promise((resolve, reject) => {
    const payload = body ? JSON.stringify(body) : '';
    const req = http.request({
      hostname: 'localhost',
      port: TEST_PORT,
      path,
      method,
      headers: {
        'Content-Type': 'application/json',
        'Content-Length': Buffer.byteLength(payload),
        ...headers
      }
    }, (res) => {
      let data = '';
      res.on('data', chunk => data += chunk);
      res.on('end', () => {
        try {
          const parsed = data ? JSON.parse(data) : {};
          resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
        } catch (e) {
          resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
        }
      });
    });
    req.on('error', reject);
```



```

    if (payload) req.write(payload);
    req.end();
  });
}

async function runTests() {
  console.log('--- STARTING G16, G19, & G20 MODULE INTEGRATION TESTS ---');

  const patientId = 9987;
  const doctorUserId = 9988;
  const client = await pool.connect();

  try {
    console.log('Setting up test data...');
    await client.query("SET app.tenant_id = '1'");

    // Clean up old test data
    await client.query('DELETE FROM cpb_logs WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM pain_assessments WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM pediatric_growth_records WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
    await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
    await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);

    // Insert patient
    await client.query(
      'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
      [patientId, 'Spec Next Patient', '???? ??????? ???????']
    );

    // Insert doctor user
    const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
    await client.query(
      'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission_s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
      [doctorUserId, TEST_USERNAME, hashedPassword, 'Dr. Specialist Urologist', 'Doctor', 'Anesthesiologist', '["patients", "prescriptions"]']
    );

    // Associate doctor with tenant 1
    await client.query(
      'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
      [doctorUserId]
    );

    console.log('Spawning test server...');
    serverProcess = spawn('node', ['server.js'], {
      env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' },
      stdio: 'pipe'
    });

    serverProcess.stdout.on('data', (data) => {
      console.log(`[SERVER] ${data.toString().trim()}`);
    });
  }
}

```

```

serverProcess.stderr.on('data', (data) => {
  console.error(`[SERVER ERR] ${data.toString().trim()}`);
});

// Wait 6 seconds for server to boot
await new Promise(resolve => setTimeout(resolve, 6000));

console.log('Logging in...');
const loginRes = await makeRequest('POST', '/api/auth/login', {
  username: TEST_USERNAME,
  password: TEST_PASSWORD
});

assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
console.log('? Logged in successfully.');
```

// Extract session cookie

```

const setCookie = loginRes.headers['set-cookie'];
assert.ok(setCookie, 'Should receive session cookie');
const cookie = setCookie[0].split(';')[0];
```

// 1. Test POST /api/surgery/cpb

```

console.log('Testing create CPB log...');
const cpbRes = await makeRequest('POST', '/api/surgery/cpb', {
  patient_id: patientId,
  pump_time: 120,
  cross_clamp_time: 80,
  flow_rate: 2.55,
  min_temp: 28.5,
  notes: 'Uneventful CPB'
}, { 'Cookie': cookie });
```

```

assert.strictEqual(cpbRes.statusCode, 200, 'Should return 200 OK');
assert.strictEqual(cpbRes.body.success, true);
assert.ok(cpbRes.body.id);
const cpbId = cpbRes.body.id;
console.log(`? CPB log created. ID: ${cpbId}`);
```

// 2. Test GET /api/surgery/cpb/patient/:patient_id

```

console.log('Testing get patient CPB logs...');
const getCpbRes = await makeRequest('GET', `/api/surgery/cpb/patient/${patientId}`, null, { 'Cookie':
cookie });
assert.strictEqual(getCpbRes.statusCode, 200);
assert.strictEqual(getCpbRes.body.length, 1);
assert.strictEqual(getCpbRes.body[0].id, cpbId);
assert.strictEqual(getCpbRes.body[0].pump_time, 120);
assert.strictEqual(parseFloat(getCpbRes.body[0].flow_rate), 2.55);
console.log('? Patient CPB logs retrieved.');
```

// 3. Test POST /api/anesthesia/pain

```

console.log('Testing create pain assessment...');
const painRes = await makeRequest('POST', '/api/anesthesia/pain', {
  patient_id: patientId,
```

```

    pain_score_vas: 6,
    pca_pump_used: true,
    pca_demands: 15,
    pca_deliveries: 12,
    notes: 'Epidural functioning'
  }, { 'Cookie': cookie });

assert.strictEqual(painRes.statusCode, 200);
assert.ok(painRes.body.id);
const painId = painRes.body.id;
console.log(`? Pain assessment created. ID: ${painId}`);

// 4. Test GET /api/anesthesia/pain/patient/:patient_id
console.log('Testing get patient pain assessments...');
const getPainRes = await makeRequest('GET', `/api/anesthesia/pain/patient/${patientId}`, null, { 'Cookie': cookie });
assert.strictEqual(getPainRes.statusCode, 200);
assert.strictEqual(getPainRes.body.length, 1);
assert.strictEqual(getPainRes.body[0].id, painId);
assert.strictEqual(getPainRes.body[0].pain_score_vas, 6);
assert.strictEqual(getPainRes.body[0].pca_pump_used, true);
console.log('? Patient pain assessments retrieved.');
```

```

// 5. Test POST /api/pediatrics/growth
console.log('Testing create pediatric growth record...');
const pedRes = await makeRequest('POST', '/api/pediatrics/growth', {
  patient_id: patientId,
  apgar_1min: 8,
  apgar_5min: 9,
  weight_kg: 3.45,
  height_cm: 51.2,
  head_circ_cm: 34.5
}, { 'Cookie': cookie });

assert.strictEqual(pedRes.statusCode, 200);
assert.ok(pedRes.body.id);
const pedId = pedRes.body.id;
console.log(`? Pediatric growth record created. ID: ${pedId}`);

// 6. Test GET /api/pediatrics/growth/patient/:patient_id
console.log('Testing get patient pediatric growth records...');
const getPedRes = await makeRequest('GET', `/api/pediatrics/growth/patient/${patientId}`, null, { 'Cookie': cookie });
assert.strictEqual(getPedRes.statusCode, 200);
assert.strictEqual(getPedRes.body.length, 1);
assert.strictEqual(getPedRes.body[0].id, pedId);
assert.strictEqual(parseFloat(getPedRes.body[0].weight_kg), 3.45);
assert.strictEqual(getPedRes.body[0].apgar_1min, 8);
console.log('? Patient pediatric growth records retrieved.');
```

```

} finally {
  console.log('Cleaning up test data...');
  await client.query("SET app.tenant_id = '1'");
  await client.query('DELETE FROM cpb_logs WHERE patient_id = $1', [patientId]);
}

```

```

    await client.query('DELETE FROM pain_assessments WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM pediatric_growth_records WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
    await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
    await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);
    client.release();

    if (serverProcess) {
        console.log('Killing test server...');
        serverProcess.kill();
    }
}

console.log('? G16, G19, & G20 Module Integration Tests passed successfully!\n');
}

if (require.main === module) {
    runTests().catch(async err => {
        console.error('? Test failed:', err);
        // Wait 3 seconds for server logs to flush
        await new Promise(resolve => setTimeout(resolve, 3000));
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    });
}

module.exports = { runTests };

```

```

=====
FILE: specialty_scores.js
=====

```

```

/**
 * specialty_scores.js ? Gate 1 pure clinical score engine for specialty modules.
 *
 * Server-side authority for clinical scores that were previously trusted from the
 * client (spoof risk): Glasgow Coma Scale (GCS), DAS28 (rheumatology disease
 * activity, ESR & CRP variants), and NIHSS (stroke severity). Mirrors the
 * ob_engine.js / nursing_scores.js pattern: pure functions (no I/O, no DB), and
 * fail-CLOSED on incomplete/invalid critical input ? returning
 * { ok:false, band:'Incomplete', error } or a null score rather than a falsely
 * reassuring default (a missing GCS component must never silently become 15).
 *
 * ??? ???? ?????? ???????? ???????? (???? ?? ??? ?????? ? ?? ??? ??? "score" ?? ??????).
 */
'use strict';

/** Parse an integer that must be finite; returns null on any non-numeric/NaN input. */
function toInt(v) {
    if (v === null || v === undefined || v === '') return null;
    const n = Number(v);
    if (!Number.isFinite(n) || !Number.isInteger(n)) return null;

```

```

    return n;
}

/** Parse a finite number (allows decimals); null on non-numeric/NaN. */
function toNum(v) {
    if (v === null || v === undefined || v === '') return null;
    const n = Number(v);
    return Number.isFinite(n) ? n : null;
}

function round2(n) {
    return Math.round((n + Number.EPSILON) * 100) / 100;
}

// ===== Glasgow Coma Scale (GCS) =====
// Components: eye 1-4, verbal 1-5, motor 1-6. Total 3-15.
// Bands: 3-8 Severe, 9-12 Moderate, 13-15 Mild.
function gcsBand(total) {
    if (total <= 8) return 'Severe';
    if (total <= 12) return 'Moderate';
    return 'Mild';
}

function computeGCS(input) {
    const fail = (error) => ({ ok: false, total: null, band: 'Incomplete', error });
    if (!input || typeof input !== 'object') return fail('GCS input missing');
    const eye = toInt(input.eye);
    const verbal = toInt(input.verbal);
    const motor = toInt(input.motor);
    if (eye === null || verbal === null || motor === null) {
        return fail('GCS requires eye, verbal and motor components');
    }
    if (eye < 1 || eye > 4) return fail('GCS eye out of range (1-4)');
    if (verbal < 1 || verbal > 5) return fail('GCS verbal out of range (1-5)');
    if (motor < 1 || motor > 6) return fail('GCS motor out of range (1-6)');
    const total = eye + verbal + motor;
    return { ok: true, total, band: gcsBand(total), components: { eye, verbal, motor } };
}

// ===== DAS28 (Disease Activity Score, 28 joints) =====
// TJC28/SJC28 in 0-28; GH = patient global health VAS 0-100.
// DAS28-ESR = 0.56*sqrt(TJC28) + 0.28*sqrt(SJC28) + 0.70*ln(ESR) + 0.014*GH (ESR > 0)
// DAS28-CRP = 0.56*sqrt(TJC28) + 0.28*sqrt(SJC28) + 0.36*ln(CRP+1) + 0.014*GH + 0.96 (CRP >= 0)
// Category: <2.6 Remission, 2.6-3.2 Low, >3.2-5.1 Moderate, >5.1 High.
function das28Category(score) {
    if (score < 2.6) return 'Remission';
    if (score <= 3.2) return 'Low';
    if (score <= 5.1) return 'Moderate';
    return 'High';
}

function validateDas28Common(input) {
    if (!input || typeof input !== 'object') return { error: 'DAS28 input missing' };
    const tjc28 = toInt(input.tjc28);

```

```

const sjc28 = toInt(input.sjc28);
const gh = toNum(input.gh);
if (tjc28 === null || sjc28 === null || gh === null) {
    return { error: 'DAS28 requires tjc28, sjc28 and gh (global health)' };
}
if (tjc28 < 0 || tjc28 > 28) return { error: 'TJC28 out of range (0-28)' };
if (sjc28 < 0 || sjc28 > 28) return { error: 'SJC28 out of range (0-28)' };
if (gh < 0 || gh > 100) return { error: 'GH (global health VAS) out of range (0-100)' };
return { tjc28, sjc28, gh };
}

function computedAS28ESR(input) {
    const fail = (error) => ({ ok: false, score: null, category: 'Incomplete', error });
    const base = validateDas28Common(input);
    if (base.error) return fail(base.error);
    const esr = toNum(input.esr);
    if (esr === null) return fail('DAS28-ESR requires esr');
    if (esr <= 0) return fail('ESR must be > 0 (ln undefined otherwise)');
    const raw = 0.56 * Math.sqrt(base.tjc28) + 0.28 * Math.sqrt(base.sjc28)
        + 0.70 * Math.log(esr) + 0.014 * base.gh;
    const score = round2(raw);
    return { ok: true, score, category: das28Category(score), variant: 'ESR' };
}

function computedAS28CRP(input) {
    const fail = (error) => ({ ok: false, score: null, category: 'Incomplete', error });
    const base = validateDas28Common(input);
    if (base.error) return fail(base.error);
    const crp = toNum(input.crp);
    if (crp === null) return fail('DAS28-CRP requires crp');
    if (crp < 0) return fail('CRP must be >= 0');
    const raw = 0.56 * Math.sqrt(base.tjc28) + 0.28 * Math.sqrt(base.sjc28)
        + 0.36 * Math.log(crp + 1) + 0.014 * base.gh + 0.96;
    const score = round2(raw);
    return { ok: true, score, category: das28Category(score), variant: 'CRP' };
}

// ===== NIHSS (Stroke Scale) =====
// 15 standard items with per-item maxima; total 0-42.
// Bands: 0 None, 1-4 Minor, 5-15 Moderate, 16-20 Moderate-Severe, 21-42 Severe.
const NIHSS_ITEM_MAX = {
    loc: 3, loc_questions: 2, loc_commands: 2, gaze: 2, visual: 3, facial: 3,
    arm_left: 4, arm_right: 4, leg_left: 4, leg_right: 4, ataxia: 2, sensory: 2,
    language: 3, dysarthria: 2, extinction: 2,
};

function nihssBand(total) {
    if (total === 0) return 'None';
    if (total <= 4) return 'Minor';
    if (total <= 15) return 'Moderate';
    if (total <= 20) return 'Moderate-Severe';
    return 'Severe';
}

```

```

function computeNIHSS(items) {
  const fail = (error) => ({ ok: false, total: null, band: 'Incomplete', error });
  if (!items || typeof items !== 'object') return fail('NIHSS items missing');
  let total = 0;
  for (const key of Object.keys(NIHSS_ITEM_MAX)) {
    const v = toInt(items[key]);
    if (v === null) return fail(`NIHSS item '${key}' missing or non-integer`);
    if (v < 0 || v > NIHSS_ITEM_MAX[key]) {
      return fail(`NIHSS item '${key}' out of range (0-${NIHSS_ITEM_MAX[key]})`);
    }
    total += v;
  }
  return { ok: true, total, band: nihssBand(total) };
}

/** Range-validate a pre-summed NIHSS total (when item-level data is unavailable). */
function validateNIHSSTotal(v) {
  const fail = (error) => ({ ok: false, total: null, band: 'Incomplete', error });
  const n = toInt(v);
  if (n === null) return fail('NIHSS total non-integer');
  if (n < 0 || n > 42) return fail('NIHSS total out of range (0-42)');
  return { ok: true, total: n, band: nihssBand(n) };
}

// ===== Shared optional-field contract =====
// Distinguishes ABSENT (undefined/null/'') ? allowed, stored NULL ? from INVALID
// (garbage / non-integer / out-of-range) ? which must 422, never silently coerce.
// Kills the lenient parseInt('12abc')=12 path and the NaN-passes-comparison trap.
function parseOptionalInt(v, min, max, field) {
  if (v === undefined || v === null || v === '') {
    return { provided: false, ok: true, value: null };
  }
  const n = toInt(v);
  if (n === null || n < min || n > max) {
    return { provided: true, ok: false, value: null, error: `${field} || 'value' must be an integer in ${min}-${max}` };
  }
  return { provided: true, ok: true, value: n };
}

// ===== GCS component validation (partial-friendly) =====
// Unified semantics for neurology assessments AND emergency trauma: each provided
// component is strictly range-validated; a PARTIAL set is legitimate (e.g. best-motor
// only in rapid trauma) and yields total NULL ? honest, never a reassuring default.
// Any invalid provided component fails the whole validation (fail-closed).
const GCS_COMPONENT_RANGES = { eye: [1, 4], verbal: [1, 5], motor: [1, 6] };

function validateGCSComponents(input) {
  const src = input && typeof input === 'object' ? input : {};
  const components = {};
  for (const key of Object.keys(GCS_COMPONENT_RANGES)) {
    const [lo, hi] = GCS_COMPONENT_RANGES[key];
    const p = parseOptionalInt(src[key], lo, hi, `GCS ${key}`);
    if (!p.ok) return { ok: false, complete: false, total: null, band: 'Incomplete', components: null, err

```

```

or: p.error };
    components[key] = p.value;
  }
  const complete = components.eye !== null && components.verbal !== null && components.motor !== null;
  if (!complete) return { ok: true, complete: false, total: null, band: 'Incomplete', components };
  const total = components.eye + components.verbal + components.motor;
  return { ok: true, complete: true, total, band: gcsBand(total), components };
}

// ===== ICU point totals (server-computed, anti-spoof) =====
// The icu_assessments table stores POINT values per component; the total is therefore a
// server-side sum ? a client-sent sofa_score/apache_ii_score is never trusted. A missing
// component contributes 0 points (cannot fabricate organ failure) but flags incomplete;
// an invalid provided component fails closed.
const SOFA_POINT_FIELDS = { pao2_fio2: 4, platelets: 4, bilirubin: 4, map_vasopressor: 4, gcs: 4, creatinine:
4 };
// APACHE-II simplified point schema: 11 physiologic vars 0-4, creatinine 0-8 (ARF doubling),
// GCS points (15 - GCS) 0-12, age 0-6, chronic health 0-5. Max total = 71.
const APACHE2_POINT_FIELDS = {
  temp: 4, map: 4, hr: 4, rr: 4, pao2: 4, ph: 4, na: 4, k: 4,
  creatinine: 8, hct: 4, wbc: 4, gcs_points: 12, age_points: 6, chronic_points: 5,
};

function sumPointFields(input, fields, label) {
  const src = input && typeof input === 'object' ? input : {};
  let total = 0, providedCount = 0;
  const points = {};
  for (const key of Object.keys(fields)) {
    const p = parseOptionalInt(src[key], 0, fields[key], `${label} ${key}`);
    if (!p.ok) return { ok: false, total: null, complete: false, points: null, error: p.error };
    points[key] = p.value === null ? 0 : p.value;
    if (p.value !== null) { providedCount++; total += p.value; }
  }
  return { ok: true, total, complete: providedCount === Object.keys(fields).length, points };
}

function sumSOFAPoints(input) { return sumPointFields(input, SOFA_POINT_FIELDS, 'SOFA'); }
function sumAPACHE2Points(input) { return sumPointFields(input, APACHE2_POINT_FIELDS, 'APACHE-II'); }

// ===== APGAR total validation =====
function validateAPGARTotal(v) {
  const p = parseOptionalInt(v, 0, 10, 'APGAR');
  if (!p.ok) return { ok: false, total: null, error: p.error };
  return { ok: true, total: p.value };
}

// ===== FEV1/FVC (server-derived ratio) =====
// The ratio is DERIVED ? a client-sent fev1_fvc_ratio is never trusted. Volumes in liters;
// >12 L is implausible for human spirometry. FEV1 > FVC is physiologically inconsistent.
// obstructive_pattern flags ratio < 0.70 (informational, not a diagnosis).
function computeFEV1FVC(fev1, fvc) {
  const fail = (error) => ({ ok: false, ratio: null, error });
  const f1 = toNum(fev1), fv = toNum(fvc);
  if (f1 === null || fv === null) return fail('FEV1/FVC requires both fev1 and fvc');

```



```

    if (f1 <= 0 || fv <= 0) return fail('FEV1 and FVC must be positive');
    if (f1 > 12 || fv > 12) return fail('FEV1/FVC volumes implausible (> 12 L)');
    if (f1 > fv) return fail('FEV1 > FVC is physiologically inconsistent');
    const ratio = round2(f1 / fv);
    return { ok: true, ratio, obstructive_pattern: ratio < 0.70 };
}

```

```

module.exports = {
  computeGCS, gcsBand, validateGCSComponents,
  computeDAS28ESR, computeDAS28CRP, das28Category,
  computeNIHSS, validateNIHSSTotal, nihssBand,
  NIHSS_ITEM_MAX,
  parseOptionalInt,
  sumSOFAPoints, sumAPACHE2Points,
  validateAPGARTotal,
  computeFEV1FVC,
};

```

```

=====
FILE: specialty_scores_test.js
=====

```

```

/**
 * specialty_scores_test.js ? PURE-ENGINE unit test for Gate 1 (server-side specialty scores).
 * Run: node specialty_scores_test.js    (no DB; deterministic)
 *
 * Safety invariant: clinical scores (GCS, DAS28, NIHSS) are computed/validated
 * server-side from raw inputs. Incomplete/invalid critical input must return a
 * fail-CLOSED { ok:false, band:'Incomplete' } ? NEVER a falsely-reassuring default
 * (e.g. a missing GCS component must not silently become a normal 15).
 */
'use strict';
const S = require('./specialty_scores');

const GREEN = '\x1b[32m', RED = '\x1b[31m', RESET = '\x1b[0m', BOLD = '\x1b[1m';
let passed = 0, failed = 0; const fails = [];
function assert(cond, name, det = '') {
  if (cond) { console.log(` ${GREEN}PASS${RESET} ${name}`); passed++; }
  else { console.log(` ${RED}FAIL${RESET} ${name}${det ? ' | ' + det : ''}`); failed++; fails.push(name); }
}
const approx = (a, b, eps = 0.01) => Math.abs(a - b) <= eps;

console.log(`${BOLD}Gate 1 ? Specialty clinical score engine (pure unit test)${RESET}\n`);

// ---- GCS ----
console.log('[1] computeGCS ? sum, bands, fail-closed');
let g = S.computeGCS({ eye: 4, verbal: 5, motor: 6 });
assert(g.ok === true && g.total === 15 && g.band === 'Mild', 'E4V5M6 -> 15 Mild', JSON.stringify(g));
g = S.computeGCS({ eye: 3, verbal: 4, motor: 5 });
assert(g.ok === true && g.total === 12 && g.band === 'Moderate', 'E3V4M5 -> 12 Moderate', JSON.stringify(g));
g = S.computeGCS({ eye: 1, verbal: 1, motor: 1 });
assert(g.ok === true && g.total === 3 && g.band === 'Severe', 'E1V1M1 -> 3 Severe', JSON.stringify(g));

```

```

g = S.computeGCS({ eye: 3, verbal: 3, motor: 3 });
assert(g.ok === true && g.total === 9 && g.band === 'Moderate', 'total 9 -> Moderate (boundary)', JSON.stringify(g));
g = S.computeGCS({ eye: 2, verbal: 3, motor: 3 });
assert(g.ok === true && g.total === 8 && g.band === 'Severe', 'total 8 -> Severe (boundary)', JSON.stringify(g));
// fail-closed: missing motor must NOT become 15
g = S.computeGCS({ eye: 4, verbal: 5 });
assert(g.ok === false && g.total === null && g.band === 'Incomplete', 'missing motor -> Incomplete (not reassuring default)', JSON.stringify(g));
// out-of-range component rejected
g = S.computeGCS({ eye: 9, verbal: 5, motor: 6 });
assert(g.ok === false && g.band === 'Incomplete', 'eye=9 out of range -> Incomplete', JSON.stringify(g));
g = S.computeGCS({ eye: 4, verbal: 0, motor: 6 });
assert(g.ok === false && g.band === 'Incomplete', 'verbal=0 out of range -> Incomplete', JSON.stringify(g));
g = S.computeGCS(null);
assert(g.ok === false && g.band === 'Incomplete', 'null input -> Incomplete', JSON.stringify(g));

// ---- DAS28-ESR ----
console.log('\n[2] computedDAS28ESR ? formula, category, fail-closed');
let d = S.computedDAS28ESR({ tjc28: 4, sjc28: 3, esr: 30, gh: 40 });
assert(d.ok === true && approx(d.score, 4.55), 'TJC4 SJC3 ESR30 GH40 -> 4.55', JSON.stringify(d));
assert(d.category === 'Moderate', '4.55 -> Moderate category', JSON.stringify(d));
d = S.computedDAS28ESR({ tjc28: 0, sjc28: 0, esr: 5, gh: 5 });
assert(d.ok === true && d.score < 2.6 && d.category === 'Remission', 'low activity -> Remission', JSON.stringify(d));
d = S.computedDAS28ESR({ tjc28: 20, sjc28: 18, esr: 80, gh: 90 });
assert(d.ok === true && d.score > 5.1 && d.category === 'High', 'high activity -> High', JSON.stringify(d));
// fail-closed: ESR must be > 0 (ln undefined otherwise)
d = S.computedDAS28ESR({ tjc28: 4, sjc28: 3, esr: 0, gh: 40 });
assert(d.ok === false && d.score === null, 'ESR=0 -> fail-closed (ln undefined)', JSON.stringify(d));
d = S.computedDAS28ESR({ tjc28: 4, sjc28: 3, gh: 40 });
assert(d.ok === false && d.score === null, 'missing ESR -> fail-closed', JSON.stringify(d));
d = S.computedDAS28ESR({ tjc28: 40, sjc28: 3, esr: 30, gh: 40 });
assert(d.ok === false, 'TJC>28 out of range -> fail-closed', JSON.stringify(d));

// ---- DAS28-CRP ----
console.log('\n[3] computedDAS28CRP ? formula, fail-closed');
let c = S.computedDAS28CRP({ tjc28: 4, sjc28: 3, crp: 10, gh: 40 });
assert(c.ok === true && approx(c.score, 3.99), 'TJC4 SJC3 CRP10 GH40 -> 3.99', JSON.stringify(c));
assert(c.category === 'Moderate', '3.99 -> Moderate', JSON.stringify(c));
c = S.computedDAS28CRP({ tjc28: 4, sjc28: 3, crp: -1, gh: 40 });
assert(c.ok === false, 'CRP<0 -> fail-closed', JSON.stringify(c));
// CRP=0 is valid (ln(0+1)=0)
c = S.computedDAS28CRP({ tjc28: 0, sjc28: 0, crp: 0, gh: 0 });
assert(c.ok === true && approx(c.score, 0.96), 'all-zero CRP -> baseline 0.96', JSON.stringify(c));

// ---- NIHSS ----
console.log('\n[4] NIHSS ? item sum, validation, total-guard');
const zeroItems = { loc:0, loc_questions:0, loc_commands:0, gaze:0, visual:0, facial:0,
  arm_left:0, arm_right:0, leg_left:0, leg_right:0, ataxia:0, sensory:0, language:0, dysarthria:0, extinction:0 };
let n = S.computeNIHSS(zeroItems);
assert(n.ok === true && n.total === 0 && n.band === 'None', 'all-zero -> 0 None', JSON.stringify(n));

```

```

const maxItems = { loc:3, loc_questions:2, loc_commands:2, gaze:2, visual:3, facial:3,
  arm_left:4, arm_right:4, leg_left:4, leg_right:4, ataxia:2, sensory:2, language:3, dysarthria:2, extinction:
2 };
n = S.computeNIHSS(maxItems);
assert(n.ok === true && n.total === 42 && n.band === 'Severe', 'all-max -> 42 Severe', JSON.stringify(n));
// item out of range -> fail-closed
n = S.computeNIHSS({ ...zeroItems, arm_left: 9 });
assert(n.ok === false && n.total === null, 'arm_left=9 out of range -> fail-closed', JSON.stringify(n));
// missing item -> fail-closed (no silent 0)
const missing = { ...zeroItems }; delete missing.extinction;
n = S.computeNIHSS(missing);
assert(n.ok === false, 'missing item -> fail-closed', JSON.stringify(n));
// validateNIHSSTotal (when only the total is available from client UI)
let vt = S.validateNIHSSTotal(7);
assert(vt.ok === true && vt.total === 7 && vt.band === 'Moderate', 'total 7 -> Moderate', JSON.stringify(vt));
vt = S.validateNIHSSTotal(43);
assert(vt.ok === false, 'total 43 out of range -> reject', JSON.stringify(vt));
vt = S.validateNIHSSTotal(-1);
assert(vt.ok === false, 'total -1 -> reject', JSON.stringify(vt));
vt = S.validateNIHSSTotal('abc');
assert(vt.ok === false, 'non-numeric total -> reject', JSON.stringify(vt));
vt = S.validateNIHSSTotal(0);
assert(vt.ok === true && vt.band === 'None', 'total 0 -> None', JSON.stringify(vt));

// ---- parseOptionalInt (shared absent-vs-garbage contract) ----
console.log('\n[5] parseOptionalInt ? absent vs invalid vs valid');
let p = S.parseOptionalInt(undefined, 0, 28);
assert(p.provided === false && p.ok === true && p.value === null, 'undefined -> not provided, null', JSON.stringify(p));
p = S.parseOptionalInt('', 0, 28);
assert(p.provided === false && p.ok === true && p.value === null, '' -> not provided, null', JSON.stringify(p));
p = S.parseOptionalInt(null, 0, 28);
assert(p.provided === false && p.ok === true && p.value === null, 'null -> not provided, null', JSON.stringify(p));
p = S.parseOptionalInt('12', 0, 28);
assert(p.provided === true && p.ok === true && p.value === 12, '"12" -> 12', JSON.stringify(p));
p = S.parseOptionalInt(0, 0, 28);
assert(p.provided === true && p.ok === true && p.value === 0, '0 -> 0 (falsy-zero safe)', JSON.stringify(p));
p = S.parseOptionalInt('12abc', 0, 28);
assert(p.provided === true && p.ok === false, '"12abc" -> invalid (NOT lenient parseInt 12)', JSON.stringify(p));
p = S.parseOptionalInt('abc', 0, 28);
assert(p.provided === true && p.ok === false, '"abc" -> invalid (never NaN passthrough)', JSON.stringify(p));
p = S.parseOptionalInt(29, 0, 28);
assert(p.provided === true && p.ok === false, '29 -> out of range invalid', JSON.stringify(p));
p = S.parseOptionalInt(7.5, 0, 42);
assert(p.provided === true && p.ok === false, '7.5 -> non-integer invalid', JSON.stringify(p));

// ---- validateGCSCComponents (unified partial semantics for neurology + trauma) ----
console.log('\n[6] validateGCSCComponents ? complete, partial, invalid');
let v = S.validateGCSCComponents({ eye: 4, verbal: 5, motor: 6 });
assert(v.ok === true && v.complete === true && v.total === 15 && v.band === 'Mild', 'complete E4V5M6 -> total 15', JSON.stringify(v));

```

```

v = S.validateGCSComponents({ motor: 5 });
assert(v.ok === true && v.complete === false && v.total === null && v.components.motor === 5 && v.components.e
ye === null,
    'motor-only partial -> ok, total NULL (rapid trauma)', JSON.stringify(v));
v = S.validateGCSComponents({});
assert(v.ok === true && v.complete === false && v.total === null && v.components.eye === null, 'none provided
-> ok, all NULL (unassessed)', JSON.stringify(v));
v = S.validateGCSComponents({ eye: 'abc' });
assert(v.ok === false, "eye='abc' -> invalid 422 (NaN must not leak to DB)", JSON.stringify(v));
v = S.validateGCSComponents({ eye: 9, verbal: 5, motor: 6 });
assert(v.ok === false, 'eye=9 -> invalid', JSON.stringify(v));
v = S.validateGCSComponents({ verbal: 6 });
assert(v.ok === false, 'verbal=6 partial out of range -> invalid', JSON.stringify(v));

// ---- ICU point totals (server-computed, anti-spoof) ----
console.log('\n[7] sumSOFAPoints / sumAPACHE2Points ? server-side totals');
let sf = S.sumSOFAPoints({ pao2_fio2: 3, platelets: 2, bilirubin: 1, map_vasopressor: 4, gcs: 2, creatinine: 1
});
assert(sf.ok === true && sf.total === 13, 'SOFA points sum -> 13', JSON.stringify(sf));
sf = S.sumSOFAPoints({ pao2_fio2: 4, platelets: 4, bilirubin: 4, map_vasopressor: 4, gcs: 4, creatinine: 4 });
assert(sf.ok === true && sf.total === 24, 'SOFA max -> 24', JSON.stringify(sf));
sf = S.sumSOFAPoints({});
assert(sf.ok === true && sf.total === 0 && sf.complete === false, 'SOFA none provided -> 0, flagged incomplete
', JSON.stringify(sf));
sf = S.sumSOFAPoints({ pao2_fio2: 5 });
assert(sf.ok === false, 'SOFA organ points 5 -> invalid (max 4)', JSON.stringify(sf));
sf = S.sumSOFAPoints({ gcs: 'abc' });
assert(sf.ok === false, "SOFA gcs='abc' -> invalid", JSON.stringify(sf));

let ap = S.sumAPACHE2Points({ temp: 1, map: 2, hr: 3, rr: 0, pao2: 4, ph: 2, na: 1, k: 0, creatinine: 2, hct:
1, wbc: 0, gcs_points: 5, age_points: 3, chronic_points: 2 });
assert(ap.ok === true && ap.total === 26, 'APACHE-II points sum -> 26', JSON.stringify(ap));
ap = S.sumAPACHE2Points({ temp: 4, map: 4, hr: 4, rr: 4, pao2: 4, ph: 4, na: 4, k: 4, creatinine: 8, hct: 4, w
bc: 4, gcs_points: 12, age_points: 6, chronic_points: 5 });
assert(ap.ok === true && ap.total === 71, 'APACHE-II max -> 71', JSON.stringify(ap));
ap = S.sumAPACHE2Points({ temp: 5 });
assert(ap.ok === false, 'APACHE physiologic points 5 -> invalid (max 4)', JSON.stringify(ap));
ap = S.sumAPACHE2Points({ gcs_points: 13 });
assert(ap.ok === false, 'APACHE gcs_points 13 -> invalid (max 12)', JSON.stringify(ap));
ap = S.sumAPACHE2Points({});
assert(ap.ok === true && ap.total === 0 && ap.complete === false, 'APACHE none -> 0, incomplete', JSON.stringi
fy(ap));

// ---- APGAR total validation ----
console.log('\n[8] validateAPGARTotal ? 0-10 guard');
let ag = S.validateAPGARTotal(9);
assert(ag.ok === true && ag.total === 9, 'APGAR 9 -> ok', JSON.stringify(ag));
ag = S.validateAPGARTotal(0);
assert(ag.ok === true && ag.total === 0, 'APGAR 0 -> ok (falsy-zero safe)', JSON.stringify(ag));
ag = S.validateAPGARTotal(11);
assert(ag.ok === false, 'APGAR 11 -> reject', JSON.stringify(ag));
ag = S.validateAPGARTotal('abc');
assert(ag.ok === false, "APGAR 'abc' -> reject", JSON.stringify(ag));

```

```
// ---- FEV1/FVC ratio (server-derived, anti-spoof) ----
console.log('\n[9] computeFEV1FVC ? server-derived ratio, fail-closed');
let f = S.computeFEV1FVC(2.1, 3.0);
assert(f.ok === true && approx(f.ratio, 0.7), 'FEV1 2.1 / FVC 3.0 -> 0.70', JSON.stringify(f));
f = S.computeFEV1FVC(1.1, 2.0);
assert(f.ok === true && approx(f.ratio, 0.55) && f.obstructive_pattern === true, '0.55 -> obstructive flag', JSON.stringify(f));
f = S.computeFEV1FVC(3.0, 3.0);
assert(f.ok === true && approx(f.ratio, 1.0) && f.obstructive_pattern === false, 'ratio 1.0 -> not obstructive', JSON.stringify(f));
f = S.computeFEV1FVC(3.5, 3.0);
assert(f.ok === false, 'FEV1 > FVC -> physiologically inconsistent, reject', JSON.stringify(f));
f = S.computeFEV1FVC(0, 3.0);
assert(f.ok === false, 'FEV1 0 -> reject (non-positive)', JSON.stringify(f));
f = S.computeFEV1FVC(2.1, null);
assert(f.ok === false, 'missing FVC -> fail-closed', JSON.stringify(f));
f = S.computeFEV1FVC(15, 20);
assert(f.ok === false, 'implausible volumes (>12L) -> reject', JSON.stringify(f));

console.log(`\n${BOLD}Result:${RESET} ${passed} passed, ${failed} failed`);
if (failed) { console.log(`${RED}FAILURES:${RESET} ${fails.join(', ')} `); process.exit(1); }
process.exit(0);
```

```
=====
FILE: staging_gaps_remediation_plan_test.js
=====
```

```
/**
 * staging_gaps_remediation_plan_test.js
 * =====
 * Static validation test for Staging Gaps Owner Remediation Plan phase.
 * Verifies that:
 * 1. All 7 remediation planning documents exist in the parent directory.
 * 2. No DDL execution or migration files were generated.
 * 3. isLiveEndpointEnabled and isWriteOperationEnabled remain false.
 * 4. No production secrets or production connection strings are leaked in the new reports.
 */
```

```
const assert = require('assert');
const fs = require('fs');
const path = require('path');
```

```
// Mock browser globals for contract verification
global.window = {};
require('./public/js/enterprise-contracts.js');
```

```
console.log('Running Staging Gaps Remediation Plan Static Tests...');
```

```
// 1. Live actions must remain disabled (always false)
assert.strictEqual(global.window.isLiveEndpointEnabled(), false, 'Live endpoints must remain disabled');
assert.strictEqual(global.window.isWriteOperationEnabled(), false, 'Write operations must remain disabled');
console.log('? Live integration boundary guards verified.');
```

```

// 2. Ensure no SQL migration files exist in the root or new migrations in the migrations folder
const rootFiles = fs.readdirSync(__dirname);
const sqlFilesInRoot = rootFiles.filter(f => f.endsWith('.sql') && f !== 'add_admin.sql' && f !== 'categorize.sql');
assert.strictEqual(sqlFilesInRoot.length, 0, 'No executable SQL files should be created in the root directory');

const migrationsDir = path.join(__dirname, 'migrations');
if (fs.existsSync(migrationsDir)) {
  const migrationFiles = fs.readdirSync(migrationsDir);
  console.log(`? Zero-DDL constraint verified. Found ${migrationFiles.length} existing migration files.`);
}

// 3. Verify existence of all 7 remediation planning documents in the parent directory
const govDir = path.join(__dirname, '..', 'docs', 'governance', 'enterprise-hospital-platform');
const requiredDocs = [
  'PHASE_STAGING_GAPS_REMEDIATION_PREFLIGHT_AR.md',
  'STAGING_BLOCKERS_CONSOLIDATED_AR.md',
  'STAGING_OWNER_DEVOPS_REMEDIATION_PLAN_AR.md',
  'STAGING_ENVIRONMENT_OWNER_CHECKLIST_AR.md',
  'STAGING_SAFE_VERIFICATION_COMMANDS_DRAFT_AR.md',
  'STAGING_GO_NO_GO_DECISION_TEMPLATE_AR.md',
  'STAGING_API_PROTOTYPE_UNLOCK_CRITERIA_AR.md'
];

requiredDocs.forEach(doc => {
  const docPath = path.join(govDir, doc);
  assert.ok(fs.existsSync(docPath), `Remediation document ${doc} must exist at ${docPath}`);

  // Verify UTF-8 encoding by reading the file
  const content = fs.readFileSync(docPath, 'utf8');
  assert.ok(content.length > 0, `Document ${doc} should not be empty`);

  // Verify no production IP or secrets are leaked (split passwords to avoid secret scanner false positives)
  assert.ok(!content.includes('DB_' + 'PASS' + 'WORD=') || content.includes('__CHANGE_ME__'), `Document ${doc} should not contain real credentials`);
});
console.log(`? All 7 Remediation Planning documents verified (present, non-empty, UTF-8 compliant).`);

console.log('All Staging Gaps Remediation Plan Static Tests Passed!');
process.exit(0);

```

```

=====
FILE: staging_provisioning_evidence_test.js
=====

```

```

/**
 * staging_provisioning_evidence_test.js
 * =====
 * Static validation test for Staging Provisioning & Evidence Capture phase.
 * Verifies that:

```

```

* 1. OWNER_STAGING_RETURN_EVIDENCE_TEMPLATE_AR.md does not contain secrets, passwords, or connection strings.
* 2. If sign-off is pending, the system does not allow a ready status.
* 3. If the decision is ready, the evidence must be complete.
*/

const assert = require('assert');
const fs = require('fs');
const path = require('path');

console.log('Running Staging Provisioning Evidence Safety Tests...');

const govDir = path.join(__dirname, '..', 'docs', 'governance', 'enterprise-hospital-platform');
const evidenceFile = path.join(govDir, 'OWNER_STAGING_RETURN_EVIDENCE_TEMPLATE_AR.md');
const signoffFile = path.join(govDir, 'STAGING_OWNER_SIGNOFF_FORM_AR.md');

// 1. Verify files exist
assert.ok(fs.existsSync(evidenceFile), 'Evidence template file must exist');
assert.ok(fs.existsSync(signoffFile), 'Signoff form file must exist');

// 2. Read and analyze the evidence template
const evidenceContent = fs.readFileSync(evidenceFile, 'utf8');

// Ensure no secrets or connection strings are in the template
const forbiddenPatterns = [
  'DATABASE_URL',
  'postgres://',
  'postgresql://',
  'PRIVATE KEY',
  'API_KEY'
];

forbiddenPatterns.forEach(pattern => {
  assert.ok(!evidenceContent.includes(pattern), `Evidence template must not contain forbidden pattern: ${pattern}`);
});

// Split the word PASSWORD to avoid secret scanner false positives
const splitPasswordPattern = 'DB_' + 'PASS' + 'WORD=';
assert.ok(!evidenceContent.includes(splitPasswordPattern) || evidenceContent.includes('<STAGING_DB_PASS' + 'WORD>'), 'Evidence template must not contain real passwords');
console.log('? Evidence template verified free of secrets, passwords, and connection strings.');
```

```

// 3. Read and analyze the signoff form
const signoffContent = fs.readFileSync(signoffFile, 'utf8');

// Verify that if sign-off has empty signatures/checkboxes, it does not allow a ready status
const isOwnerSigned = signoffContent.includes('[x] ?????? ??????');
const isDevOpsSigned = signoffContent.includes('DevOps: confirmed');

if (!isOwnerSigned && !isDevOpsSigned) {
  console.log('? Verified: Signoff is currently pending or unsigned.');
```

```

} else {
  // If signed, verify that all checkboxes in the evidence are filled with [x]
  const allChecked = evidenceContent.includes('[x] ??? (YES)') && !evidenceContent.includes('[ ] ??? (YES)');
```

```

    assert.ok(allChecked, 'If signoff is complete, all evidence checkmarks must be checked [x]');
    console.log('? Verified: Completed sign-off has complete evidence.');
```

}

// 4. Staging Env Loading Safety Tests

```

console.log('Running Staging Environment Loading Safety Tests...');
const { execSync } = require('child_process');
```

try {

```

    // Test A: NODE_ENV=staging and missing .env.staging should throw error (fail closed)
    const originalEnvPath = path.join(__dirname, '.env.staging');
    const tempEnvPath = path.join(__dirname, '.env.staging.bak');
```

let envStaged = false;

```

if (fs.existsSync(originalEnvPath)) {
    fs.renameSync(originalEnvPath, tempEnvPath);
    envStaged = true;
}
```

try {

```

    execSync('node -e "require(\'../db_postgres\')"', {
        env: { ...process.env, NODE_ENV: 'staging' },
        stdio: 'pipe'
    });
    assert.fail('Should have failed when .env.staging is missing');
```

} catch (err) {

```

    assert.ok(err.message.includes('CRITICAL: .env.staging file is missing'), 'Expected missing file error');
    console.log(' ? Test A passed: Missing .env.staging fails closed.');
```

} finally {

```

    if (envStaged) {
        fs.renameSync(tempEnvPath, originalEnvPath);
    }
}
```

} catch (e) {

```

    console.error('Failed Staging Env Safety Test A:', e);
    process.exit(1);
}
```

try {

```

    // Test B: NODE_ENV=staging and DB_NAME=nama_medical_web should throw error
    const tempEnvPath = path.join(__dirname, '.env.staging.temp');
    fs.writeFileSync(tempEnvPath, 'DB_NAME=nama_medical_web\nNODE_ENV=staging\n');
```

const originalEnvPath = path.join(__dirname, '.env.staging');

```

const backupEnvPath = path.join(__dirname, '.env.staging.real.bak');
let hasReal = false;
if (fs.existsSync(originalEnvPath)) {
    fs.renameSync(originalEnvPath, backupEnvPath);
    hasReal = true;
}
```

```

fs.renameSync(tempEnvPath, originalEnvPath);
```

try {

```

    execSync('node -e "require(\'../db_postgres\')"', {
```



```

    env: { ...process.env, NODE_ENV: 'staging' },
    stdio: 'pipe'
  });
  assert.fail('Should have failed when DB_NAME is nama_medical_web');
} catch (err) {
  assert.ok(err.message.includes('Must not connect to production database'), 'Expected invalid DB_NAME error
');
  console.log(' ? Test B passed: DB_NAME=nama_medical_web is rejected.');
} finally {
  if (fs.existsSync(originalEnvPath)) {
    fs.unlinkSync(originalEnvPath);
  }
  if (hasReal) {
    fs.renameSync(backupEnvPath, originalEnvPath);
  }
}
} catch (e) {
  console.error('Failed Staging Env Safety Test B:', e);
  process.exit(1);
}

console.log('All Staging Provisioning Evidence Safety Tests Passed!');
process.exit(0);

```

```

=====
FILE: staging_readiness_design_test.js
=====

```

```

/**
 * staging_readiness_design_test.js
 * =====
 * Static design validation test for Staging Readiness & Environment Gap phase.
 * Verifies that:
 * 1. All 11 staging readiness documents exist and are populated.
 * 2. No DDL execution or migration files were generated.
 * 3. isLiveEndpointEnabled and isWriteOperationEnabled remain false.
 * 4. No production secrets or production URLs/IPs are leaked in the new reports.
 */

```

```

const assert = require('assert');
const fs = require('fs');
const path = require('path');

```

```

// Mock browser globals for contract verification
global.window = {};
require('./public/js/enterprise-contracts.js');

```

```

console.log('Running Staging Readiness Design Static Tests...');

```

```

// 1. Live actions must remain disabled (always false)
assert.strictEqual(global.window.isLiveEndpointEnabled(), false, 'Live endpoints must remain disabled');
assert.strictEqual(global.window.isWriteOperationEnabled(), false, 'Write operations must remain disabled');

```

```

console.log('? Live integration boundary guards verified (isLiveEndpointEnabled & isWriteOperationEnabled are false).');

// 2. Ensure no SQL migration files exist in the root or new migrations in the migrations folder
const rootFiles = fs.readdirSync(__dirname);
const sqlFilesInRoot = rootFiles.filter(f => f.endsWith('.sql') && f !== 'add_admin.sql' && f !== 'categorize.sql');
assert.strictEqual(sqlFilesInRoot.length, 0, 'No executable SQL files should be created in the root directory during this phase');

// Check that no new migrations were added to migrations folder (there should be a fixed set of migrations)
const migrationsDir = path.join(__dirname, 'migrations');
if (fs.existsSync(migrationsDir)) {
  const migrationFiles = fs.readdirSync(migrationsDir);
  // We just ensure no DDL is executed or written during this phase.
  console.log(`? Zero-DDL constraint verified. Found ${migrationFiles.length} existing migration files.`);
}

// 3. Verify existence of all 11 staging readiness documents
const govDir = path.join(__dirname, 'docs', 'governance', 'enterprise-hospital-platform');
const requiredDocs = [
  'PHASE_STAGING_READINESS_PREFLIGHT_AR.md',
  'STAGING_ENVIRONMENT_DISCOVERY_AR.md',
  'STAGING_ISOLATION_CHECKLIST_AR.md',
  'STAGING_READONLY_RUNTIME_VERIFICATION_PLAN_AR.md',
  'STAGING_API_PROTOTYPE_SCOPE_AR.md',
  'STAGING_BACKEND_RLS_GAP_REPORT_AR.md',
  'STAGING_READINESS_DECISION_REPORT_AR.md',
  'PHASE_STAGING_READINESS_IMPLEMENTATION_AR.md',
  'STAGING_SECURITY_BOUNDARY_REPORT_AR.md',
  'STAGING_READINESS_TEST_REPORT_AR.md',
  'PHASE_STAGING_READINESS_CLOSEOUT_AR.md'
];

requiredDocs.forEach(doc => {
  const docPath = path.join(govDir, doc);
  assert.ok(fs.existsSync(docPath), `Staging readiness document ${doc} must exist`);

  // Verify UTF-8 encoding by reading the file
  const content = fs.readFileSync(docPath, 'utf8');
  assert.ok(content.length > 0, `Document ${doc} should not be empty`);

  // Verify no production IP or domain is mentioned in a way that leaks credentials
  // We can mention them as examples, but we must verify no passwords or production connection strings are leaked.
  assert.ok(!content.includes('DB_' + 'PASS' + 'WORD=') || content.includes('__CHANGE_ME__'), `Document ${doc} should not contain real credentials`);
});
console.log('? All 11 Staging Readiness documents verified (present, non-empty, UTF-8 compliant).');

console.log('All Staging Readiness Design Static Tests Passed!');
process.exit(0);

```

=====

FILE: super_admin.js

=====

```
/**
 * super_admin.js ? Jumanasoft SaaS Tenant Control Center (Batch 1).
 *
 * Platform-level Super Admin to manage TENANTS (list / details / suspend / reactivate). Additive +
 * flag-gated (`SUPER_ADMIN_ENABLED`): mounting an empty router when disabled = zero behavior change.
 *
 * Security model (no tenant breakout):
 *   - Super Admin identity = an explicit ENV allowlist `SUPER_ADMIN_USERS` (comma-separated usernames).
 *     A tenant 'Admin' is NOT a Super Admin ? prevents privilege escalation. Fail-closed.
 *   - The tenants table has no RLS (top-level), so listing reads it directly.
 *   - Per-tenant stats (users/facilities/last activity) are read INSIDE that tenant's RLS context
 *     (runWithTenant) ? never a bulk cross-tenant read.
 *   - Every state change writes an audit event. No delete, no payments, no impersonation (this batch).
 *
 * Pure core (isSuperAdmin / parseAllowlist / canTransition / parseTenantFilters / deriveTenantSummary)
 * is unit-tested with no DB. makeSuperAdminRouter(deps) builds the Express router.
 */
'use strict';

const ALLOWED_STATUSES = ['active', 'suspended', 'trial', 'cancelled'];

// ENV allowlist -> Set of usernames (trimmed, non-empty).
function parseAllowlist(envValue) {
  return new Set(String(envValue || '')
    .split(',')
    .map(s => s.trim())
    .filter(Boolean));
}

// Super Admin iff the user is active AND explicitly listed. Tenant role is irrelevant (no escalation).
function isSuperAdmin(user, allowlist) {
  if (!user || !user.username) return false;
  if (user.is_active === 0 || user.is_active === false) return false;
  const set = allowlist instanceof Set ? allowlist : parseAllowlist(allowlist);
  return set.has(String(user.username));
}

// Allowed status transitions (conservative). cancelled is terminal here (reactivation deferred).
const TRANSITIONS = {
  active:    ['suspended'],
  suspended: ['active'],
  trial:     ['active', 'suspended'],
  cancelled: []
};

function canTransition(from, to) {
  if (!ALLOWED_STATUSES.includes(to)) return false;
  const allowed = TRANSITIONS[from] || [];
  return allowed.includes(to);
}

// Validate/normalize list filters from the query string (enum + bounded text). Never throws.
```

```

function parseTenantFilters(query = {}) {
  const out = {};
  if (query.status && ALLOWED_STATUSES.includes(String(query.status))) out.status = String(query.status);
  if (query.plan && /^[^w-]{1,50}$/.test(String(query.plan))) out.plan = String(query.plan);
  if (query.q) out.q = String(query.q).trim().slice(0, 100);
  return out;
}

// Build a stable display summary for one tenant row (+ optional computed stats).
function deriveTenantSummary(row, stats = {}) {
  if (!row) return null;
  return {
    id: row.id,
    name: row.name || '',
    subdomain: row.subdomain || '',
    status: ALLOWED_STATUSES.includes(row.status) ? row.status : 'active',
    plan: row.plan_type || 'standard',
    created_at: row.created_at || null,
    users: stats.users == null ? null : Number(stats.users),
    facilities: stats.facilities == null ? null : Number(stats.facilities),
    last_activity: stats.last_activity || null
  };
}

// ---- Express router factory ----
// deps: { pool, getActor(req)->user, runWithTenant(tenantId,fn), logAudit(...), allowlist, enabled,
//         requireAuth (middleware) }
function makeSuperAdminRouter(deps = {}) {
  const express = require('express');
  const router = express.Router();
  const { pool, getActor, runWithTenant, logAudit, requireAuth } = deps;
  const enabled = deps.enabled !== false; // default on when constructed; server gates by env
  const allowlist = deps.allowlist instanceof Set ? deps.allowlist : parseAllowlist(deps.allowlist);

  if (!enabled) return router; // inert: no routes registered

  function requireSuperAdmin(req, res, next) {
    const user = getActor ? getActor(req) : (req.session && req.session.user);
    if (!isSuperAdmin(user, allowlist)) {
      try { logAudit && logAudit(user && user.id, user && user.display_name, 'SUPER_ADMIN_DENY', 'SuperAdmin', `denied ${req.method} ${req.originalUrl || req.url}`, req.ip); } catch (_) {}
      return res.status(403).json({ error: 'Super Admin access required', code: 'SUPER_ADMIN_REQUIRED' });
    }
    return next();
  }

  if (requireAuth) router.use(requireAuth);
  router.use(requireSuperAdmin);

  const posInt = (v) => { const n = Number(v); return Number.isInteger(n) && n >= 1 ? n : null; };

  // STATS ? Aggregated platform metrics (Phase 3)
  router.get('/stats', async (req, res) => {
    try {

```

```

// 1. Total revenue collected
const revQuery = await pool.query('SELECT COALESCE(SUM(total), 0) AS total FROM invoices WHERE paid = 1');

const totalRevenue = parseFloat(revQuery.rows[0].total || 0);

// 2. Total tenants count
const tenQuery = await pool.query('SELECT COUNT(*) AS count FROM tenants');
const totalTenants = parseInt(tenQuery.rows[0].count || 0, 10);

// 3. Active subscriptions count
let activeSubscriptions = 0;
try {
  const subQuery = await pool.query('SELECT COUNT(*) AS count FROM tenant_plan_assignments WHERE effective_to IS NULL');
  activeSubscriptions = parseInt(subQuery.rows[0].count || 0, 10);
} catch (_) {}

// 4. Plans breakdown
let plansBreakdown = [];
try {
  const breakdownQuery = await pool.query(`
    SELECT plan_key, COUNT(*) AS count
    FROM tenant_plan_assignments
    WHERE effective_to IS NULL
    GROUP BY plan_key
  `);
  plansBreakdown = breakdownQuery.rows.map(r => ({
    plan_key: r.plan_key,
    count: parseInt(r.count || 0, 10)
  }));
} catch (_) {}

res.json({
  total_revenue: totalRevenue,
  total_tenants: totalTenants,
  active_subscriptions: activeSubscriptions,
  plans_breakdown: plansBreakdown
});
} catch (e) {
  res.status(500).json({ error: 'Server error' });
}
});

// LIST ? reads `tenants` directly (no RLS); filters applied in SQL.
router.get('/tenants', async (req, res) => {
  try {
    const f = parseTenantFilters(req.query);
    const where = [], params = [];
    if (f.status) { params.push(f.status); where.push(`status = ${params.length}`); }
    if (f.plan) { params.push(f.plan); where.push(`plan_type = ${params.length}`); }
    if (f.q) { params.push('%' + f.q + '%'); where.push(`(name ILIKE ${params.length} OR subdomain ILIKE ${params.length})`); }
    const sql = `SELECT id, name, subdomain, status, plan_type, created_at FROM tenants
      ${where.length ? 'WHERE ' + where.join(' AND ') : ''}`;
  }

```

```

        ORDER BY id DESC LIMIT 500`;
        const rows = (await pool.query(sql, params)).rows;
        res.json({ tenants: rows.map(r => deriveTenantSummary(r)), filters: f, count: rows.length });
    } catch (e) { res.status(500).json({ error: 'Server error' }); }
});

// DETAILS ? core row (no RLS) + per-tenant stats read INSIDE that tenant's RLS context.
router.get('/tenants/:id', async (req, res) => {
    const id = posInt(req.params.id);
    if (!id) return res.status(400).json({ error: 'Invalid tenant id' });
    try {
        const row = (await pool.query('SELECT id, name, subdomain, status, plan_type, created_at FROM tenants WHERE id=$1', [id])).rows[0];
        if (!row) return res.status(404).json({ error: 'Tenant not found' });
        let stats = { users: null, facilities: null, last_activity: null };
        const readStats = async () => {
            const u = (await pool.query('SELECT COUNT(*)::int AS c FROM user_tenants WHERE tenant_id=$1 AND is_active=true', [id])).rows[0];
            const fac = (await pool.query('SELECT COUNT(*)::int AS c FROM facilities WHERE tenant_id=$1', [id])).rows[0];
            const la = (await pool.query('SELECT MAX(created_at) AS m FROM audit_trail WHERE tenant_id=$1', [id])).rows[0];
            stats = { users: u && u.c, facilities: fac && fac.c, last_activity: la && la.m };
        };
        if (runWithTenant) { await runWithTenant(id, readStats); } else { await readStats(); }
        res.json({ tenant: deriveTenantSummary(row, stats) });
    } catch (e) { res.status(500).json({ error: 'Server error' }); }
});

async function changeStatus(req, res, to, action) {
    const id = posInt(req.params.id);
    if (!id) return res.status(400).json({ error: 'Invalid tenant id' });
    try {
        const row = (await pool.query('SELECT id, name, status FROM tenants WHERE id=$1', [id])).rows[0];
        if (!row) return res.status(404).json({ error: 'Tenant not found' });
        if (row.status === to) return res.json({ tenant: deriveTenantSummary(row), unchanged: true });
        if (!canTransition(row.status, to)) {
            return res.status(409).json({ error: `Cannot change status from ${row.status} to ${to}`, code: 'INVALID_TRANSITION' });
        }
        const updated = (await pool.query('UPDATE tenants SET status=$1 WHERE id=$2 RETURNING id, name, subdomain, status, plan_type, created_at', [to, id])).rows[0];
        const actor = getActor ? getActor(req) : (req.session && req.session.user);
        try { logAudit && logAudit(actor && actor.id, actor && actor.display_name, action, 'SuperAdmin', `tenant#${id} (${row.name}) ${row.status} -> ${to}`, req.ip); } catch (_) {}
        res.json({ tenant: deriveTenantSummary(updated) });
    } catch (e) { res.status(500).json({ error: 'Server error' }); }
}

router.post('/tenants/:id/suspend', (req, res) => changeStatus(req, res, 'suspended', 'TENANT_SUSPEND'));
router.post('/tenants/:id/reactivate', (req, res) => changeStatus(req, res, 'active', 'TENANT_REACTIVATE'));

return router;
}

```

```

module.exports = {
  ALLOWED_STATUSES, TRANSITIONS,
  parseAllowlist, isSuperAdmin, canTransition, parseTenantFilters, deriveTenantSummary,
  makeSuperAdminRouter
};

```

```

=====
FILE: super_admin_test.js
=====

```

```

/**
 * super_admin_test.js ? Tenant Control Center tests.
 *   Part A: pure unit tests (no DB, no server).
 *   Part B: in-process HTTP integration (express + mocked pool/audit) for permissions, audit, no-leak.
 * Run: node super_admin_test.js   (exit 0 = all pass)
 */
'use strict';
const http = require('http');
const express = require('express');
const SA = require('./super_admin');

let pass = 0, fail = 0;
function ok(name, cond) { if (cond) pass++; else { fail++; console.error(' FAIL:', name); } }

// ----- Part A: pure -----
ok('parseAllowlist splits/trims', (() => { const s = SA.parseAllowlist(' op , boss , '); return s.has('op') &&
s.has('boss') && s.size === 2; }));
ok('isSuperAdmin allows listed active', SA.isSuperAdmin({ username: 'op', is_active: 1 }, SA.parseAllowlist('o
p')) === true);
ok('isSuperAdmin denies unlisted', SA.isSuperAdmin({ username: 'x', is_active: 1 }, SA.parseAllowlist('op')) =
== false);
ok('isSuperAdmin denies inactive', SA.isSuperAdmin({ username: 'op', is_active: 0 }, SA.parseAllowlist('op'))
=== false);
ok('isSuperAdmin denies tenant Admin (no escalation)', SA.isSuperAdmin({ username: 'tadmin', role: 'Admin', is
_active: 1 }, SA.parseAllowlist('op')) === false);
ok('isSuperAdmin denies null', SA.isSuperAdmin(null, SA.parseAllowlist('op')) === false);

ok('canTransition active->suspended', SA.canTransition('active', 'suspended') === true);
ok('canTransition suspended->active', SA.canTransition('suspended', 'active') === true);
ok('canTransition trial->active', SA.canTransition('trial', 'active') === true);
ok('canTransition rejects cancelled->active', SA.canTransition('cancelled', 'active') === false);
ok('canTransition rejects unknown status', SA.canTransition('active', 'bogus') === false);

(() => {
  const f = SA.parseTenantFilters({ status: 'active', plan: 'standard', q: ' jum ' });
  ok('filters status', f.status === 'active');
  ok('filters plan', f.plan === 'standard');
  ok('filters q trimmed', f.q === 'jum');
  ok('filters drop bad status', SA.parseTenantFilters({ status: 'hax' }).status === undefined);
})();

```

```

(() => {
  const s = SA.deriveTenantSummary({ id: 3, name: 'X', subdomain: 'x', status: 'active', plan_type: 'pro', created_at: 'd' }, { users: 5, facilities: 2, last_activity: 'la' });
  ok('summary maps fields', s.id === 3 && s.plan === 'pro' && s.users === 5 && s.facilities === 2 && s.last_activity === 'la');
  ok('summary defaults bad status to active', SA.deriveTenantSummary({ id: 1, status: 'weird' }).status === 'active');
  ok('summary null-safe', SA.deriveTenantSummary(null) === null);
})();

// ----- Part B: HTTP integration -----
function fakePool(audit) {
  const calls = [];
  return {
    calls,
    query(sql, params) {
      calls.push({ sql: sql.replace(/\s+/g, ' ').trim(), params });
      const s = sql.replace(/\s+/g, ' ');
      if (/FROM tenants WHERE id=\$1/.test(s)) {
        if (params[0] === 99) return Promise.resolve({ rows: [] }); // not found
        return Promise.resolve({ rows: [{ id: params[0], name: 'Acme', subdomain: 'acme', status: 'active', plan_type: 'standard', created_at: '2026-01-01' }] });
      }
      if (/UPDATE tenants SET status=\$1/.test(s)) return Promise.resolve({ rows: [{ id: params[1], name: 'Acme', subdomain: 'acme', status: params[0], plan_type: 'standard', created_at: '2026-01-01' }] });
      if (/FROM tenants/.test(s)) return Promise.resolve({ rows: [{ id: 1, name: 'Acme', subdomain: 'acme', status: 'active', plan_type: 'standard', created_at: '2026-01-01' }] });
      if (/FROM user_tenants/.test(s)) return Promise.resolve({ rows: [{ c: 7 }] });
      if (/FROM facilities/.test(s)) return Promise.resolve({ rows: [{ c: 2 }] });
      if (/FROM audit_trail/.test(s)) return Promise.resolve({ rows: [{ m: '2026-06-30' }] });
      if (/SUM\(total\)/.test(s)) return Promise.resolve({ rows: [{ total: 12500 }] });
      if (/FROM tenant_plan_assignments/.test(s)) {
        if (/GROUP BY/.test(s)) return Promise.resolve({ rows: [{ plan_key: 'free_trial', count: 5 }, { plan_key: 'basic', count: 4 }] });
        return Promise.resolve({ rows: [{ count: 9 }] });
      }
      return Promise.resolve({ rows: [] });
    }
  };
}

function buildApp(currentUserRef, auditLog, tenantBindLog) {
  const app = express();
  app.use(express.json());
  app.use((req, res, next) => { req.session = { user: currentUserRef.user }; next(); });
  const pool = fakePool();
  app._pool = pool;
  app.use('/api/super-admin', SA.makeSuperAdminRouter({
    pool,
    getActor: (req) => req.session.user,
    runWithTenant: (tenantId, fn) => { tenantBindLog.push(tenantId); return fn(); },
    logAudit: (...a) => auditLog.push(a),
    allowlist: 'op',
    enabled: true
  }));
}

```



```

    }));
    return app;
}

function req(port, method, path) {
    return new Promise((resolve) => {
        const r = http.request({ host: '127.0.0.1', port, method, path }, (res) => {
            let body = ''; res.on('data', d => body += d); res.on('end', () => {
                let json; try { json = JSON.parse(body); } catch { json = {}; }
                resolve({ status: res.statusCode, json });
            });
        });
        r.end();
    });
}

(async () => {
    const userRef = { user: { id: 1, username: 'op', display_name: 'Operator', is_active: 1 } };
    const audit = [], binds = [];
    const app = buildApp(userRef, audit, binds);
    const srv = app.listen(0);
    await new Promise(r => srv.once('listening', r));
    const port = srv.address().port;

    // super admin can get stats
    let rStats = await req(port, 'GET', '/api/super-admin/stats');
    ok('super admin gets stats 200', rStats.status === 200 && rStats.json.total_revenue === 12500 && rStats.json
.active_subscriptions === 9 && rStats.json.plans_breakdown.length === 2);

    // super admin can list
    let r1 = await req(port, 'GET', '/api/super-admin/tenants');
    ok('super admin lists tenants 200', r1.status === 200 && Array.isArray(r1.json.tenants));

    // non-super (unlisted) denied
    userRef.user = { id: 2, username: 'nobody', role: 'Admin', is_active: 1 };
    let r2 = await req(port, 'GET', '/api/super-admin/tenants');
    ok('tenant Admin denied 403', r2.status === 403 && r2.json.code === 'SUPER_ADMIN_REQUIRED');

    // unauthenticated denied
    userRef.user = null;
    let r3 = await req(port, 'GET', '/api/super-admin/tenants');
    ok('no session denied 403', r3.status === 403);

    // back to super admin: details reads inside tenant RLS context (runWithTenant called with id)
    userRef.user = { id: 1, username: 'op', display_name: 'Operator', is_active: 1 };
    binds.length = 0;
    let r4 = await req(port, 'GET', '/api/super-admin/tenants/5');
    ok('details 200 with stats', r4.status === 200 && r4.json.tenant.users === 7 && r4.json.tenant.facilities ==
= 2);
    ok('no cross-tenant leak: runWithTenant bound to id 5 only', binds.length === 1 && binds[0] === 5);

    // details 404 for missing
    let r5 = await req(port, 'GET', '/api/super-admin/tenants/99');
    ok('details 404 when missing', r5.status === 404);

```

```

// suspend writes audit + changes status
audit.length = 0;
let r6 = await req(port, 'POST', '/api/super-admin/tenants/5/suspend');
ok('suspend 200', r6.status === 200 && r6.json.tenant.status === 'suspended');
ok('suspend wrote audit (TENANT_SUSPEND)', audit.length === 1 && audit[0][2] === 'TENANT_SUSPEND');

// invalid id rejected
let r7 = await req(port, 'GET', '/api/super-admin/tenants/abc');
ok('invalid id 400', r7.status === 400);

srv.close();

// disabled router mounts nothing
const app2 = express(); app2.use((req, res, next) => { req.session = { user: userRef.user }; next(); });
app2.use('/api/super-admin', SA.makeSuperAdminRouter({ enabled: false }));
app2.use((req, res) => res.status(404).json({ fell_through: true }));
const srv2 = app2.listen(0); await new Promise(r => srv2.once('listening', r));
let r8 = await req(srv2.address().port, 'GET', '/api/super-admin/tenants');
ok('disabled router is inert (falls through 404)', r8.status === 404 && r8.json.fell_through === true);
srv2.close();

console.log(`super_admin_test: ${pass} passed, ${fail} failed`);
process.exit(fail === 0 ? 0 : 1);
})();

```

```

=====
FILE: surgery_urology_integration_test.js
=====

```

```

/**
 * surgery_urology_integration_test.js ? Integration test for General Surgery and Urology clinical modules.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3022;
const TEST_USERNAME = 'surg_uro_doctor';
const TEST_PASSWORD = 'SURG_URO_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
  return new Promise((resolve, reject) => {
    const payload = body ? JSON.stringify(body) : '';

```

```

const req = http.request({
  hostname: 'localhost',
  port: TEST_PORT,
  path,
  method,
  headers: {
    'Content-Type': 'application/json',
    'Content-Length': Buffer.byteLength(payload),
    ...headers
  }
}, (res) => {
  let data = '';
  res.on('data', chunk => data += chunk);
  res.on('end', () => {
    try {
      const parsed = data ? JSON.parse(data) : {};
      resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
    } catch (e) {
      resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
    }
  });
});
req.on('error', reject);
if (payload) req.write(payload);
req.end();
});
}

```

```

async function runTests() {
  console.log('--- STARTING SURGERY & UROLOGY MODULE INTEGRATION TESTS ---');

  const patientId = 9977;
  const doctorUserId = 9978;
  const client = await pool.connect();

  try {
    console.log('Setting up test data...');
    await client.query("SET app.tenant_id = '1'");

    // Clean up old test data
    await client.query('DELETE FROM surgical_checklists WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM surgical_time_logs WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM urodynamic_studies WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
    await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
    await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);

    // Insert patient
    await client.query(
      'INSERT INTO patients (id, name_en, name_ar, tenant_id) VALUES ($1, $2, $3, 1)',
      [patientId, 'Surg Uro Patient', '???? ?????? ?????????']
    );

    // Insert doctor user

```

```

const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
await client.query(
  'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
  [doctorUserId, TEST_USERNAME, hashedPassword, 'Dr. Surgeon Urologist', 'Doctor', 'General Surgery'
, ['patients', 'prescriptions']]
);

// Associate doctor with tenant 1
await client.query(
  'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
  [doctorUserId]
);

console.log('Spawning test server...');
serverProcess = spawn('node', ['server.js'], {
  env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' },
  stdio: 'pipe'
});

serverProcess.stdout.on('data', (data) => {
  console.log(`[SERVER] ${data.toString().trim()}`);
});

serverProcess.stderr.on('data', (data) => {
  console.error(`[SERVER ERR] ${data.toString().trim()}`);
});

// Wait 6 seconds for server to boot
await new Promise(resolve => setTimeout(resolve, 6000));

console.log('Logging in...');
const loginRes = await makeRequest('POST', '/api/auth/login', {
  username: TEST_USERNAME,
  password: TEST_PASSWORD
});

assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
console.log('? Logged in successfully.');

// Extract session cookie
const setCookie = loginRes.headers['set-cookie'];
assert.ok(setCookie, 'Should receive session cookie');
const cookie = setCookie[0].split(';')[0];

// 1. Test POST /api/surgery/checklists
console.log('Testing create surgical checklist...');
const checklistRes = await makeRequest('POST', '/api/surgery/checklists', {
  patient_id: patientId,
  procedure_name: 'Laparoscopic Cholecystectomy',
  sign_in_confirmed: true,
  time_out_confirmed: true,
  sign_out_confirmed: false,
  notes: 'Uneventful checklist prep'
});

```

```

}, { 'Cookie': cookie });

assert.strictEqual(checklistRes.statusCode, 200, 'Should return 200 OK');
assert.strictEqual(checklistRes.body.success, true, 'Should return success true');
assert.ok(checklistRes.body.id, 'Should return record ID');
const checklistId = checklistRes.body.id;
console.log(`? Surgical checklist created. ID: ${checklistId}`);

// 2. Test GET /api/surgery/checklists/patient/:patient_id
console.log('Testing get patient surgical checklists...');
const getChecklistRes = await makeRequest('GET', `/api/surgery/checklists/patient/${patientId}`, null,
{ 'Cookie': cookie });
assert.strictEqual(getChecklistRes.statusCode, 200);
assert.strictEqual(getChecklistRes.body.length, 1);
assert.strictEqual(getChecklistRes.body[0].id, checklistId);
assert.strictEqual(getChecklistRes.body[0].sign_in_confirmed, true);
assert.strictEqual(getChecklistRes.body[0].sign_out_confirmed, false);
console.log('? Patient surgical checklists retrieved.');
```

```

// 3. Test POST /api/surgery/timelogs
console.log('Testing create surgical time log...');
const now = new Date().toISOString();
const timelogRes = await makeRequest('POST', '/api/surgery/timelogs', {
  patient_id: patientId,
  procedure_name: 'Laparoscopic Cholecystectomy',
  anesthesia_start_time: now,
  incision_time: now,
  closure_time: null,
  anesthesia_end_time: null
}, { 'Cookie': cookie });

assert.strictEqual(timelogRes.statusCode, 200);
assert.ok(timelogRes.body.id);
const timelogId = timelogRes.body.id;
console.log(`? Surgical time log created. ID: ${timelogId}`);

// 4. Test GET /api/surgery/timelogs/patient/:patient_id
console.log('Testing get patient surgical time logs...');
const getTimelogRes = await makeRequest('GET', `/api/surgery/timelogs/patient/${patientId}`, null, { '
Cookie': cookie });
assert.strictEqual(getTimelogRes.statusCode, 200);
assert.strictEqual(getTimelogRes.body.length, 1);
assert.strictEqual(getTimelogRes.body[0].id, timelogId);
console.log('? Patient surgical time logs retrieved.');
```

```

// 5. Test POST /api/urology/urodynamics
console.log('Testing create urodynamic study...');
const uroRes = await makeRequest('POST', '/api/urology/urodynamics', {
  patient_id: patientId,
  max_flow_rate: 12.5,
  voided_volume: 300,
  post_void_residual: 65,
  detrusor_pressure: 45,
  interpretation: 'Bladder Outlet Obstruction suspected'
}, { 'Cookie': cookie });

```

```

    }, { 'Cookie': cookie });

    assert.strictEqual(uroRes.statusCode, 200);
    assert.ok(uroRes.body.id);
    const uroId = uroRes.body.id;
    console.log(`? Urodynamic study created. ID: ${uroId}`);

    // 6. Test GET /api/urology/urodynamics/patient/:patient_id
    console.log('Testing get patient urodynamic studies...');
    const getUroRes = await makeRequest('GET', `/api/urology/urodynamics/patient/${patientId}`, null, { 'Cookie': cookie });
    assert.strictEqual(getUroRes.statusCode, 200);
    assert.strictEqual(getUroRes.body.length, 1);
    assert.strictEqual(getUroRes.body[0].id, uroId);
    assert.strictEqual(parseFloat(getUroRes.body[0].max_flow_rate), 12.5);
    assert.strictEqual(parseFloat(getUroRes.body[0].post_void_residual), 65);
    console.log('? Patient urodynamic studies retrieved.');
```

```

} finally {
    console.log('Cleaning up test data...');
    await client.query("SET app.tenant_id = '1'");
    await client.query('DELETE FROM surgical_checklists WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM surgical_time_logs WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM urodynamic_studies WHERE patient_id = $1', [patientId]);
    await client.query('DELETE FROM patients WHERE id = $1', [patientId]);
    await client.query('DELETE FROM user_tenants WHERE user_id = $1', [doctorUserId]);
    await client.query('DELETE FROM system_users WHERE id = $1', [doctorUserId]);
    client.release();

    if (serverProcess) {
        console.log('Killing test server...');
        serverProcess.kill();
    }
}

console.log('? General Surgery and Urology Module Integration Tests passed successfully!\n');
}

if (require.main === module) {
    runTests().catch(async err => {
        console.error('? Test failed:', err);
        // Wait 3 seconds for server logs to flush
        await new Promise(resolve => setTimeout(resolve, 3000));
        if (serverProcess) serverProcess.kill();
        process.exit(1);
    });
}

module.exports = { runTests };

=====
FILE: surgical_count_test.js

```

=====

```
/**
 * surgical_count_test.js ? Integration test for the Surgical Count Sheets API.
 */
'use strict';

process.env.NODE_ENV = 'staging';

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3024;
const TEST_USERNAME = 'surg_tester';
const TEST_PASSWORD = 'SURG_PASSWORD_PLACEHOLDER';

let serverProcess;

function makeRequest(method, path, body, headers = {}) {
  return new Promise((resolve, reject) => {
    const payload = body ? JSON.stringify(body) : '';
    const req = http.request({
      hostname: 'localhost',
      port: TEST_PORT,
      path,
      method,
      headers: {
        'Content-Type': 'application/json',
        'Content-Length': Buffer.byteLength(payload),
        ...headers
      }
    }, (res) => {
      let data = '';
      res.on('data', chunk => data += chunk);
      res.on('end', () => {
        try {
          const parsed = data ? JSON.parse(data) : {};
          resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
        } catch (e) {
          resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
        }
      });
    });
    req.on('error', reject);
    if (payload) req.write(payload);
    req.end();
  });
}

async function runTests() {
  console.log('--- STARTING SURGICAL COUNT SHEETS INTEGRATION TESTS ---');
```

```

const surgeryId = 8871;
const staffUserId = 8872;
const client = await pool.connect();

try {
  console.log('Setting up test data...');
  await client.query("SET app.tenant_id = '1'");

  // Clean up old test data
  await client.query('DELETE FROM surgery_count_sheets WHERE surgery_id = $1', [surgeryId]);
  await client.query('DELETE FROM user_tenants WHERE user_id = $1', [staffUserId]);
  await client.query('DELETE FROM system_users WHERE id = $1', [staffUserId]);

  // Insert doctor user
  const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
  await client.query(
    "INSERT INTO system_users (id, username, password_hash, display_name, role, permissions, is_active"
  ) VALUES ($1, $2, $3, $4, $5, $6, 1)",
    [staffUserId, TEST_USERNAME, hashedPassword, 'Dr. Sarah Surgeon', 'Doctor', ['"patients"']]
  );

  // Associate doctor with tenant 1
  await client.query(
    'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)',
    [staffUserId]
  );

  console.log('Spawning test server...');
  serverProcess = spawn('node', ['server.js'], {
    env: { ...process.env, PORT: TEST_PORT, NODE_ENV: 'staging', SKIP_DB_INIT: 'true' }
  });

  serverProcess.stdout.on('data', (data) => {
    if (process.env.DEBUG_TESTS) console.log(`[Server STDOUT] ${data.toString().trim()}`);
  });

  // Wait for server to start
  await new Promise(resolve => setTimeout(resolve, 6000));

  console.log('Logging in...');
  const loginRes = await makeRequest('POST', '/api/auth/login', {
    username: TEST_USERNAME,
    password: TEST_PASSWORD
  });
  assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
  const cookie = loginRes.headers['set-cookie'][0];

  // Test 1: Create matching count sheet
  console.log('Testing create matching surgical count sheet...');
  const createRes1 = await makeRequest('POST', '/api/surgery/count-sheet', {
    surgery_id: surgeryId,
    sponge_count_initial: 10,
    sponge_count_final: 10,
  });

```



```

        needle_count_initial: 15,
        needle_count_final: 15,
        instrument_count_initial: 24,
        instrument_count_final: 24,
        witness1_name: 'Nurse Aisha',
        witness2_name: 'Dr. Sarah',
        notes: 'All items accounted for.'
    }, { Cookie: cookie });

    assert.strictEqual(createRes1.statusCode, 200);
    assert.strictEqual(createRes1.body.counts_match, true, 'Counts should match');
    console.log(`? Matching count sheet created successfully. ID: ${createRes1.body.id}`);

    // Test 2: Create mismatching count sheet
    console.log('Testing create mismatching surgical count sheet...');
    const createRes2 = await makeRequest('POST', '/api/surgery/count-sheet', {
        surgery_id: surgeryId,
        sponge_count_initial: 10,
        sponge_count_final: 9, // mismatch
        needle_count_initial: 15,
        needle_count_final: 15,
        instrument_count_initial: 24,
        instrument_count_final: 24,
        witness1_name: 'Nurse Aisha',
        witness2_name: 'Dr. Sarah',
        notes: 'Sponges missing!'
    }, { Cookie: cookie });

    assert.strictEqual(createRes2.statusCode, 200);
    assert.strictEqual(createRes2.body.counts_match, false, 'Counts should mismatch');
    console.log(`? Mismatching count sheet created successfully. ID: ${createRes2.body.id}`);

    // Test 3: Get Surgery Count Sheets
    console.log('Testing get surgical count sheets...');
    const getRes = await makeRequest('GET', `/api/surgery/count-sheet/${surgeryId}`, null, { Cookie: cookie });

    assert.strictEqual(getRes.statusCode, 200);
    assert.strictEqual(getRes.body.length, 2);
    console.log('? Count sheets list retrieved successfully.');

    console.log('Cleaning up test data...');
    await client.query('DELETE FROM surgery_count_sheets WHERE surgery_id = $1', [surgeryId]);
    await client.query('DELETE FROM user_tenants WHERE user_id = $1', [staffUserId]);
    await client.query('DELETE FROM system_users WHERE id = $1', [staffUserId]);

    console.log('Killing test server...');
    serverProcess.kill();
    console.log('? Surgical Count Sheets Integration Tests passed successfully!');
    process.exit(0);

} catch (e) {
    console.error('? Test failed:', e);
    if (serverProcess) serverProcess.kill();
    process.exit(1);
}

```

```

    } finally {
        client.release();
    }
}

```

```
runTests();
```

```

=====
FILE: surgical_wave2_engine.js
=====

```

```

// =====
// Surgical ? Wave 2 PURE ENGINE (no DB, no I/O)
// -----
// Converts Enterprise_Blueprint_2026 Wave 2 (Surgical) C-classified department
// specs into real, unit-testable decision-support gates. Same conventions as
// the Wave 1/10 engines: pure functions, fail-closed, each rule is the EXACT
// "Critical Alert"/"Business Logic" rule stated in the department's blueprint.
// =====

```

```
'use strict';
```

```

// 1) Surgical Oncology ? tumor board scheduling gate.
// Blueprint rule: auto-schedule tumor board review for any "Borderline Resectable" case
// before OR booking.

```

```

function checkTumorBoardSchedulingGate({ resectabilityStatus } = {}) {
    const requiresTumorBoard = resectabilityStatus === 'borderline_resectable';
    return {
        tumorBoardRequiredBeforeOr: requiresTumorBoard,
        orBookingAllowed: !requiresTumorBoard || false,
        reason: requiresTumorBoard
            ? 'borderline resectable ? tumor board review required before OR booking'
            : 'resectability status does not require pre-OR tumor board review'
    };
}

```

```

// 2) Endocrine Surgery ? intraoperative nerve monitoring alert.
// Blueprint rule: critical alert if intraoperative nerve signal is lost (immediate
// surgical team notification).

```

```

function checkNerveMonitoringAlert({ nerveSignalLost } = {}) {
    return {
        criticalAlert: nerveSignalLost === true,
        reason: nerveSignalLost === true
            ? 'intraoperative nerve signal lost ? immediate surgical team notification'
            : 'nerve signal intact'
    };
}

```

```

// 3) Robotic Surgery ? docking angle safety alert.
// Blueprint rule: alert if docking angle deviates significantly from the procedure-specific
// optimal range (risk of arm collision).

```

```
function checkDockingAngleSafety({ dockingAngleDegrees, optimalAngleDegrees, toleranceDegrees = 15 } = {}) {
```

```

    if (dockingAngleDegrees === undefined || dockingAngleDegrees === null || optimalAngleDegrees === undefined
|| optimalAngleDegrees === null) {
        return { alert: false, reason: 'incomplete docking-angle data ? cannot assess' };
    }
    const deviation = Math.abs(Number(dockingAngleDegrees) - Number(optimalAngleDegrees));
    const alert = deviation > toleranceDegrees;
    return {
        armCollisionRiskAlert: alert,
        deviationDegrees: deviation,
        reason: alert
            ? `docking angle deviates ${deviation.toFixed(1)}deg from optimal (>${toleranceDegrees}deg toleran
ce) ? arm collision risk`
            : 'docking angle within safe tolerance'
    };
}

// 4) Bariatric Surgery ? OR scheduling clearance gate.
// Blueprint rule: hard-block OR scheduling if psych/nutrition clearance is missing.
function checkBariatricOrSchedulingGate({ psychClearance, nutritionClearance } = {}) {
    const cleared = psychClearance === true && nutritionClearance === true;
    return {
        orSchedulingAllowed: cleared,
        reason: cleared
            ? 'psych and nutrition clearance both documented'
            : 'BLOCKED ? OR scheduling requires both psych and nutrition clearance'
    };
}

// 5) Breast Surgery ? imaging-pathology discordance flag.
// Blueprint rule: auto-flag any discordant imaging-pathology result for mandatory tumor board discussion.
function checkBreastImagingPathologyConcordance({ imagingSuspicionLevel, pathologyResult } = {}) {
    if (!imagingSuspicionLevel || !pathologyResult) {
        return { discordant: false, tumorBoardRequired: false, reason: 'incomplete imaging/pathology data ? ca
nnot assess concordance' };
    }
    const highSuspicionImaging = imagingSuspicionLevel === 'high' || imagingSuspicionLevel === 'BI-RADS 4' ||
imagingSuspicionLevel === 'BI-RADS 5';
    const benignPathology = pathologyResult === 'benign';
    const discordant = highSuspicionImaging && benignPathology;
    return {
        discordant,
        tumorBoardRequired: discordant,
        reason: discordant
            ? 'high-suspicion imaging with benign pathology ? discordant, mandatory tumor board review'
            : 'imaging and pathology results are concordant'
    };
}

// 6) Trauma Surgery ? Massive Transfusion Protocol activation.
// Blueprint rule: auto-activate Massive Transfusion Protocol alert to Blood Bank if criteria met.
function checkMassiveTransfusionProtocol({ systolicBp, heartRate, suspectedActiveHemorrhage, positiveFast } =
{}) {
    const shockPhysiology = (systolicBp !== undefined && systolicBp !== null && Number(systolicBp) < 90)
        && (heartRate !== undefined && heartRate !== null && Number(heartRate) > 120);

```

```

const mtpCriteria = (shockPhysiology && positiveFast === true) || suspectedActiveHemorrhage === true;
return {
  mtpActivated: mtpCriteria,
  reason: mtpCriteria ? 'MTP criteria met ? urgent Blood Bank activation for massive transfusion' : 'MTP
criteria not met'
};
}

// 7) Colorectal Surgery ? diverting ostomy recommendation.
// Blueprint rule: auto-suggest diverting loop ileostomy if leak risk score exceeds threshold.
function checkAnastomoticLeakRisk({ leakRiskScore, leakRiskThreshold = 7 } = {}) {
  if (leakRiskScore === undefined || leakRiskScore === null || !Number.isFinite(Number(leakRiskScore))) {
    return { divertingOstomyRecommended: false, reason: 'leak risk score missing ? cannot assess' };
  }
  const recommend = Number(leakRiskScore) > leakRiskThreshold;
  return {
    divertingOstomyRecommended: recommend,
    reason: recommend
      ? `leak risk score ${leakRiskScore} exceeds threshold ${leakRiskThreshold} ? diverting loop ileost
omy suggested`
      : 'leak risk within acceptable range'
  };
}

// 8) Neurosurgery ? intraoperative neuromonitoring (IONM) requirement flag.
// Blueprint rule: auto-flag cases requiring IONM/awake mapping based on lesion-eloquent-cortex proximity.
function checkIonmRequirement({ eloquentCortexProximityMm, proximityThresholdMm = 10 } = {}) {
  if (eloquentCortexProximityMm === undefined || eloquentCortexProximityMm === null) {
    return { ionmRequired: false, reason: 'lesion-cortex proximity data missing ? cannot assess' };
  }
  const required = Number(eloquentCortexProximityMm) <= proximityThresholdMm;
  return {
    ionmOrAwakeMappingRequired: required,
    reason: required
      ? `lesion within ${eloquentCortexProximityMm}mm of eloquent cortex (<=${proximityThresholdMm}mm) ?
IONM/awake mapping required`
      : 'lesion sufficiently distant from eloquent cortex'
  };
}

// 9) Spine Surgery ? Cauda Equina Syndrome emergency trigger.
// Blueprint rule: critical alert if Cauda Equina Syndrome criteria present (immediate OR
// activation, bypass elective queue).
function checkCaudaEquinaEmergency({ saddleAnesthesia, bowelBladderDysfunction, bilateralLegWeakness } = {}) {
  const emergency = saddleAnesthesia === true && (bowelBladderDysfunction === true || bilateralLegWeakness =
== true);
  return {
    emergencyOrActivation: emergency,
    reason: emergency
      ? 'Cauda Equina Syndrome criteria present ? immediate OR activation, bypass elective queue'
      : 'Cauda Equina Syndrome criteria not met'
  };
}

```

```

// 10) Maxillofacial Surgery ? airway compromise alert.
// Blueprint rule: critical alert for panfacial trauma with airway compromise risk (immediate
// ENT/Anesthesia co-management).
function checkAirwayCompromiseAlert({ panfacialTrauma, airwayCompromiseSigns } = {}) {
  const alert = panfacialTrauma === true && airwayCompromiseSigns === true;
  return {
    immediateEntAnesthesiaCoManagement: alert,
    reason: alert
      ? 'panfacial trauma with airway compromise signs ? immediate ENT/Anesthesia co-management required
    ,
      : 'no airway compromise criteria met'
  };
}

module.exports = {
  checkTumorBoardSchedulingGate,
  checkNerveMonitoringAlert,
  checkDockingAngleSafety,
  checkBariatricOrSchedulingGate,
  checkBreastImagingPathologyConcordance,
  checkMassiveTransfusionProtocol,
  checkAnastomoticLeakRisk,
  checkIonmRequirement,
  checkCaudaEquinaEmergency,
  checkAirwayCompromiseAlert
};

```

```

=====
FILE: surgical_wave2_engine_unit_test.js
=====

```

```

// =====
// surgical_wave2_engine_unit_test.js ? DB-free unit tests for the 10 Wave 2
// (Surgical) safety/alert gates.
// =====
'use strict';
const assert = require('assert');
const eng = require('./surgical_wave2_engine');

function runTests() {
  // 1) Surgical Oncology ? borderline resectable must require tumor board before OR.
  assert.strictEqual(eng.checkTumorBoardSchedulingGate({ resectabilityStatus: 'borderline_resectable' }).tumorBoardRequiredBeforeOr, true);
  assert.strictEqual(eng.checkTumorBoardSchedulingGate({ resectabilityStatus: 'resectable' }).tumorBoardRequiredBeforeOr, false);

  // 2) Endocrine Surgery ? nerve signal loss must trigger critical alert.
  assert.strictEqual(eng.checkNerveMonitoringAlert({ nerveSignalLost: true }).criticalAlert, true);
  assert.strictEqual(eng.checkNerveMonitoringAlert({ nerveSignalLost: false }).criticalAlert, false);

  // 3) Robotic Surgery ? large docking angle deviation must trigger alert.
  assert.strictEqual(eng.checkDockingAngleSafety({ dockingAngleDegrees: 60, optimalAngleDegrees: 30 }).armCo

```

```

llisionRiskAlert, true);
    assert.strictEqual(eng.checkDockingAngleSafety({ dockingAngleDegrees: 32, optimalAngleDegrees: 30 }).armCollisionRiskAlert, false);

    // 4) Bariatric Surgery ? missing clearance must block OR scheduling.
    assert.strictEqual(eng.checkBariatricOrSchedulingGate({ psychClearance: true, nutritionClearance: false }).orSchedulingAllowed, false);
    assert.strictEqual(eng.checkBariatricOrSchedulingGate({ psychClearance: true, nutritionClearance: true }).orSchedulingAllowed, true);

    // 5) Breast Surgery ? high-suspicion imaging + benign pathology must flag discordance.
    assert.strictEqual(eng.checkBreastImagingPathologyConcordance({ imagingSuspicionLevel: 'high', pathologyResult: 'benign' }).tumorBoardRequired, true);
    assert.strictEqual(eng.checkBreastImagingPathologyConcordance({ imagingSuspicionLevel: 'high', pathologyResult: 'malignant' }).tumorBoardRequired, false);

    // 6) Trauma Surgery ? shock physiology + positive FAST must activate MTP.
    assert.strictEqual(eng.checkMassiveTransfusionProtocol({ systolicBp: 80, heartRate: 130, positiveFast: true }).mtpActivated, true);
    assert.strictEqual(eng.checkMassiveTransfusionProtocol({ systolicBp: 120, heartRate: 80 }).mtpActivated, false);

    // 7) Colorectal Surgery ? high leak risk score must suggest diverting ostomy.
    assert.strictEqual(eng.checkAnastomoticLeakRisk({ leakRiskScore: 9 }).divertingOstomyRecommended, true);
    assert.strictEqual(eng.checkAnastomoticLeakRisk({ leakRiskScore: 3 }).divertingOstomyRecommended, false);

    // 8) Neurosurgery ? lesion near eloquent cortex must require IONM.
    assert.strictEqual(eng.checkIonmRequirement({ eloquentCortexProximityMm: 5 }).ionmOrAwakeMappingRequired, true);
    assert.strictEqual(eng.checkIonmRequirement({ eloquentCortexProximityMm: 25 }).ionmOrAwakeMappingRequired, false);

    // 9) Spine Surgery ? Cauda Equina criteria must trigger emergency OR activation.
    assert.strictEqual(eng.checkCaudaEquinaEmergency({ saddleAnesthesia: true, bowelBladderDysfunction: true }).emergencyOrActivation, true);
    assert.strictEqual(eng.checkCaudaEquinaEmergency({ saddleAnesthesia: false }).emergencyOrActivation, false);

    // 10) Maxillofacial ? panfacial trauma + airway compromise must trigger alert.
    assert.strictEqual(eng.checkAirwayCompromiseAlert({ panfacialTrauma: true, airwayCompromiseSigns: true }).immediateEntAnesthesiaCoManagement, true);
    assert.strictEqual(eng.checkAirwayCompromiseAlert({ panfacialTrauma: true, airwayCompromiseSigns: false }).immediateEntAnesthesiaCoManagement, false);

    console.log('surgical_wave2_engine_unit_test: all 10 department gates verified');
}

if (require.main === module) {
    runTests();
}

module.exports = { runTests };

```

```

=====
FILE: surgical_wave2_routes_static_test.js
=====

/**
 * surgical_wave2_routes_static_test.js ? DB-free static check verifying the 10
 * Wave 2 (Surgical) stateless routes are wired correctly in server.js.
 */
'use strict';
const fs = require('fs');
const path = require('path');
const assert = require('assert');

const src = fs.readFileSync(path.join(__dirname, 'server.js'), 'utf8');

const ROUTES = [
  { route: '/api/surg-onc/resectability-assessment', fn: 'checkTumorBoardSchedulingGate' },
  { route: '/api/endo-surg/nerve-monitoring-log', fn: 'checkNerveMonitoringAlert' },
  { route: '/api/robotic/docking-log', fn: 'checkDockingAngleSafety' },
  { route: '/api/bariatric/eligibility-check', fn: 'checkBariatricOrSchedulingGate' },
  { route: '/api/breast-surg/concordance-check', fn: 'checkBreastImagingPathologyConcordance' },
  { route: '/api/trauma/mtp-activation', fn: 'checkMassiveTransfusionProtocol' },
  { route: '/api/colorectal/leak-risk', fn: 'checkAnastomoticLeakRisk' },
  { route: '/api/neurosurg/elloquent-risk', fn: 'checkIonmRequirement' },
  { route: '/api/spine/instability-status', fn: 'checkCaudaEquinaEmergency' },
  { route: '/api/maxfax/airway-risk', fn: 'checkAirwayCompromiseAlert' },
];

function runTests() {
  assert.ok(src.includes("require('./surgical_wave2_engine')"), 'server.js must require surgical_wave2_engine');

  for (const r of ROUTES) {
    const declStr = `app.post('${r.route}')`;
    const routeIdx = src.indexOf(declStr);
    assert.ok(routeIdx !== -1, `route must exist: ${r.route}`);

    const routeEnd = src.indexOf('});', routeIdx);
    const block = src.slice(routeIdx, routeEnd === -1 ? routeIdx + 300 : routeEnd);
    assert.ok(block.includes('requireAuth'), `${r.route} must require auth`);
    assert.ok(block.includes("requireRole('patients', 'prescriptions')"), `${r.route} must require patients/prescriptions role`);
    assert.ok(block.includes(`surgWave2Engine.${r.fn}()`), `${r.route} must call surgWave2Engine.${r.fn}`);
  }

  console.log(`surgical_wave2_routes_static_test: all ${ROUTES.length} routes verified`);
}

if (require.main === module) {
  runTests();
}

module.exports = { runTests };

```

```
=====
FILE: tailwind.config.js
=====
```

```
module.exports = {
  content: [
    './public/**/*.html',
    './public/**/*.js'
  ],
  darkMode: "class",
  theme: {
    extend: {
      colors: {
        "surface-dim": "#d9dadb",
        "secondary-container": "#7af1fc",
        "inverse-on-surface": "#f0f1f2",
        "on-tertiary": "#ffffff",
        "primary": "#001629",
        "primary-fixed": "#cfe5ff",
        "on-secondary-fixed-variant": "#004f54",
        "surface-container-lowest": "#ffffff",
        "secondary-fixed": "#7df4ff",
        "surface-bright": "#f8f9fa",
        "on-tertiary-fixed": "#251a00",
        "on-primary-fixed": "#001d34",
        "surface-container": "#edeef",
        "outline": "#73777e",
        "on-surface-variant": "#42474d",
        "error-container": "#ffdad6",
        "surface-container-high": "#e7e8e9",
        "surface-tint": "#406182",
        "surface-variant": "#e1e3e4",
        "on-error-container": "#93000a",
        "surface-container-low": "#f3f4f5",
        "on-primary-container": "#7293b6",
        "inverse-primary": "#a8caef",
        "background": "#f8f9fa",
        "secondary": "#006970",
        "secondary-fixed-dim": "#5dd8e2",
        "on-background": "#191c1d",
        "on-secondary": "#ffffff",
        "primary-container": "#002b49",
        "tertiary-fixed": "#ffdf9d",
        "on-tertiary-container": "#ac8c44",
        "inverse-surface": "#2e3132",
        "on-secondary-container": "#006e75",
        "on-primary": "#ffffff",
        "surface-container-highest": "#e1e3e4",
        "primary-fixed-dim": "#a8caef",
        "error": "#ba1a1a",
        "surface": "#f8f9fa",
```



```

    "outline-variant": "#c3c7ce",
    "on-primary-fixed-variant": "#274969",
    "tertiary-fixed-dim": "#e6c274",
    "tertiary-container": "#362700",
    "on-surface": "#191c1d",
    "on-secondary-fixed": "#002022",
    "on-tertiary-fixed-variant": "#5b4300",
    "tertiary": "#1d1300",
    "on-error": "#ffffff"
  },
  borderRadius: {
    "DEFAULT": "0.25rem",
    "lg": "0.5rem",
    "xl": "0.75rem",
    "full": "9999px"
  },
  spacing: {
    "gutter": "24px",
    "md": "24px",
    "xl": "64px",
    "base": "8px",
    "margin-desktop": "48px",
    "margin-mobile": "16px",
    "xs": "4px",
    "lg": "40px",
    "sm": "12px"
  },
  fontFamily: {
    "display-lg": ["IBM Plex Sans Arabic", "IBM Plex Sans", "sans-serif"],
    "headline-lg": ["IBM Plex Sans Arabic", "IBM Plex Sans", "sans-serif"],
    "headline-md": ["IBM Plex Sans Arabic", "IBM Plex Sans", "sans-serif"],
    "title-lg": ["IBM Plex Sans Arabic", "IBM Plex Sans", "sans-serif"],
    "body-lg": ["IBM Plex Sans Arabic", "IBM Plex Sans", "sans-serif"],
    "body-md": ["IBM Plex Sans Arabic", "IBM Plex Sans", "sans-serif"],
    "label-md": ["IBM Plex Sans Arabic", "IBM Plex Sans", "sans-serif"],
    "caption": ["IBM Plex Sans Arabic", "IBM Plex Sans", "sans-serif"]
  }
}
},
plugins: [
  require('@tailwindcss/forms'),
  require('@tailwindcss/container-queries')
]
};

```

```

=====
FILE: task.md
=====

```

```

# ? ????? ???? ?????? ???????? ?????? ?????? (HOS_PRO)

- `[x]` ????? ?????? ?????? HOS_PRO ?????? ???? ??????.

```

- `[x]` ??? ?????? ???????? ???????? (Sidebar) ?? `namaweb/public/js/app.js` ??? ?????? (44 ????? ???????? 0
????? ???????? 0 submenu ??? ????).
- `[x]` ????? ?????? ???????? ???????? (Global Benchmark & Gap Analysis) ?????? 25 ????? ?????? ???
4 ????? ?????? P0-P3.
- `[x]` ????? ???????? ???????? ???????? (Master Department Catalog) ?????? 73 ??? ? 10 ?????? ???????.
- `[x]` ????? ??? ???????? ?????? ???????? (Requirements Blueprint) ?????? ???????? ?????? ???????? ???
???.
- `[x]` ????? ?????? ???????? (RBAC Matrix) ?????? ?? ?????? ???????? (PDPL & CBAHI).
- `[x]` ????? ?????? ???????? ???????? ???????? ?????? ?????? ?????? (Safety Gates).
- `[x]` ????? ?????? ???????? ?????? ?????? ?????? ?????? ?????? (UI/UX Blueprint).
- `[x]` ????? ?????? ?????? ???????? ???????? ?????? ?????? ?????? APIs ???????? (HL7/FHIR/ZAT
CA/NPHIES).
- `[x]` ????? ???????? ??? ?????? ???????? (End-to-End Clinical Workflows).
- `[x]` ????? ??? ???????? ???????? ???????? ??? ??? Mojibake (QA & Acceptance Testing).
- `[x]` ????? ??? ?????? ???????? ? ?????? `.ai-brain/` ????? UTF-8 ????.
- `[x]` ????? ?????? ???????? (`AI\_PROJECT\_MEMORY.md`) ??? ?????? ?????? (`walkthrough.md`).
- `[x]` ????? ?????? ?????? ??? ??? ? ? ? ? ? (Final No-Missing Department Audit).

=====
FILE: tenant_context.js
=====

```
const { AsyncLocalStorage } = require('async_hooks');
const tenantStore = new AsyncLocalStorage();

function runWithTenant(context, fn) {
  return tenantStore.run(context || {}, fn);
}

function getCurrentTenantId() {
  const s = tenantStore.getStore();
  return s && s.tenantId ? s.tenantId : null;
}

module.exports = {
  tenantStore,
  runWithTenant,
  getCurrentTenantId
};
```

=====
FILE: tenant_context_pg_session.js
=====

```
/**
 * tenant_context_pg_session.js
 *
 * Helper module for managing tenant session context securely in PostgreSQL transactions.
 * Utilizes `set_config` inside transactions to prevent connection pool pollution.
```

```

*/

/**
 * Executes database operations within a scoped transaction,
 * setting PostgreSQL session variables for tenant isolation.
 *
 * @param {Object} pool - PostgreSQL connection pool.
 * @param {Object} ctx - The tenant context containing tenant_id, facility_id (optional), branch_id (optional)
 *
 * @param {Function} handler - Async callback: (client, ctx) => Promise<any>
 * @returns {Promise<any>}
 */
async function withTenantTransaction(pool, ctx, handler) {
  if (!pool) {
    throw new Error('Database pool is required');
  }
  if (!ctx) {
    throw new Error('Tenant context is required');
  }
  if (ctx.tenant_id === undefined || ctx.tenant_id === null) {
    throw new Error('tenant_id is mandatory');
  }
  if (typeof handler !== 'function') {
    throw new Error('Handler must be a function');
  }

  const client = await pool.connect();
  try {
    await client.query('BEGIN');

    // Set mandatory tenant_id (cast to string for set_config compatibility)
    await client.query("SELECT set_config('app.tenant_id', $1, true)", [ctx.tenant_id.toString()]);

    // Set optional facility_id if provided
    if (ctx.facility_id !== undefined && ctx.facility_id !== null) {
      await client.query("SELECT set_config('app.facility_id', $1, true)", [ctx.facility_id.toString()]);
    }

    // Set optional branch_id if provided
    if (ctx.branch_id !== undefined && ctx.branch_id !== null) {
      await client.query("SELECT set_config('app.branch_id', $1, true)", [ctx.branch_id.toString()]);
    }

    const result = await handler(client, ctx);

    await client.query('COMMIT');
    return result;
  } catch (error) {
    try {
      await client.query('ROLLBACK');
    } catch (rollbackErr) {
      console.error('Failed to rollback transaction:', rollbackErr.message);
    }
  }
}

```

```

        throw error;
    } finally {
        client.release();
    }
}

```

```

module.exports = {
    withTenantTransaction
};

```

```

=====
FILE: tenant_context_pg_session_test.js
=====

```

```

const { pool } = require('./db_postgres');
const { withTenantTransaction } = require('./tenant_context_pg_session');
const fs = require('fs');
const path = require('path');

```

```

const RED = '\x1b[31m';
const GREEN = '\x1b[32m';
const YELLOW = '\x1b[33m';
const BLUE = '\x1b[34m';
const RESET = '\x1b[0m';
const BOLD = '\x1b[1m';

```

```

console.log(`\n${BOLD}${BLUE}===== ${RESET}`);
console.log(`${BOLD}${BLUE} ????? ?????? ??? Tenant Context PG Session Prototype${RESET}`);
console.log(`${BOLD}${BLUE}===== ${RESET}\n`);

```

```

async function runTests() {
    let failed = false;

    // Test 1: Missing tenant fails
    try {
        console.log(`[Test 1] ????? ?? ??? ????? ??? ??? tenant_id...`);
        await withTenantTransaction(pool, {}, async (client, ctx) => {
            // Should not reach here
        });
        console.log(` ${RED}? ??? : ?? ??? ???? tenant_id!${RESET}`);
        failed = true;
    } catch (error) {
        if (error.message.includes('tenant_id is mandatory')) {
            console.log(` ${GREEN}? ??? : ?? ??? ????? ??????: ${error.message}${RESET}`);
        } else {
            console.log(` ${RED}? ??? : ??? ??? ???? ?????: ${error.message}${RESET}`);
            failed = true;
        }
    }
}

// Test 2: SET LOCAL works inside transaction
try {

```

```

console.log(`\n[Test 2] ?????? ?? ????? SET LOCAL ???? ??? Transaction...`);
const result = await withTenantTransaction(pool, { tenant_id: 123, facility_id: 456, branch_id: 789 },
async (client, ctx) => {
    const resTenant = await client.query("SELECT current_setting('app.tenant_id', true) AS tenant");
    const resFacility = await client.query("SELECT current_setting('app.facility_id', true) AS facilit
y");

    const resBranch = await client.query("SELECT current_setting('app.branch_id', true) AS branch");
    return {
        tenant: resTenant.rows[0].tenant,
        facility: resFacility.rows[0].facility,
        branch: resBranch.rows[0].branch
    };
});

if (result.tenant === '123' && result.facility === '456' && result.branch === '789') {
    console.log(` ${GREEN}? ????: ?? ?????? ?????????? ???? ?????????? ?????? (tenant: 123, facility: 456,
branch: 789){RESET}`);
} else {
    console.log(` ${RED}? ????: ??? ?????????? ??? ??????????: ${JSON.stringify(result)}{RESET}`);
    failed = true;
}
} catch (error) {
    console.log(` ${RED}? ????: ??? ??? ?????? ?????? ??????????: ${error.message}{RESET}`);
    failed = true;
}

// Test 3: COMMIT cleans up context
try {
    console.log(`\n[Test 3] ?????? ?? ?????? ?????? (COMMIT cleanup) ??? ?????? ?????????? ??????...`);
    // We will acquire a client manually, execute the helper context flow, commit, and then query the same
    client.
    const client = await pool.connect();
    try {
        // Simulating withTenantTransaction manually on this connection to ensure we query the exact same
        physical client.
        await client.query('BEGIN');
        await client.query("SELECT set_config('app.tenant_id', $1, true)", ['999']);
        await client.query('COMMIT');

        // Querying the setting after commit on the SAME client
        const checkRes = await client.query("SELECT current_setting('app.tenant_id', true) AS tenant");
        const tenantVal = checkRes.rows[0].tenant;

        if (tenantVal === '' || tenantVal === null) {
            console.log(` ${GREEN}? ????: ?? ??? ?????????? ?????????? ??? ??? COMMIT (??????? ??????????: "${tenant
tVal}")${RESET}`);
        } else {
            console.log(` ${RED}? ????: ?????????? ?? ?????? ??? ??? COMMIT! (????????: "${tenantVal}")${RESET}`)
;

            failed = true;
        }
    } finally {
        client.release();
    }
}

```

```

} catch (error) {
  console.log(` ${RED}? ??? : ??? ??? : ${error.message}${RESET}`);
  failed = true;
}

// Test 4: ROLLBACK cleans up context
try {
  console.log(`\n[Test 4] ?????? ?? ????? ?????? (ROLLBACK cleanup) ??? ??? ?????????? ??????????...`);
  const client = await pool.connect();
  try {
    try {
      await client.query('BEGIN');
      await client.query("SELECT set_config('app.tenant_id', $1, true)", ['888']);
      throw new Error('Simulated failure');
    } catch (err) {
      await client.query('ROLLBACK');
    }

    // Querying the setting after rollback on the SAME client
    const checkRes = await client.query("SELECT current_setting('app.tenant_id', true) AS tenant");
    const tenantVal = checkRes.rows[0].tenant;

    if (tenantVal === '' || tenantVal === null) {
      console.log(` ${GREEN}? ??? : ?? ??? ?????????? ?????????? ??? ??? ROLLBACK (?????? ?????????? : "${tenantVal}")${RESET}`);
    } else {
      console.log(` ${RED}? ??? : ????????? ?? ????? ??? ??? ROLLBACK! (??????? : "${tenantVal}")${RESET}`);
    }

    failed = true;
  }
} finally {
  client.release();
}
} catch (error) {
  console.log(` ${RED}? ??? : ??? ??? : ${error.message}${RESET}`);
  failed = true;
}

// Test 5: Tenant 1 does not leave trace on Tenant 2 (Concurrency/sequential leakage test)
try {
  console.log(`\n[Test 5] ?????? ?? ??? ?????? ?????????? ????? Context ??? ?????? ?????...`);

  // Execute Tenant 1
  await withTenantTransaction(pool, { tenant_id: 1 }, async (client, ctx) => {
    const tenantVal = (await client.query("SELECT current_setting('app.tenant_id', true) AS tenant")).rows[0].tenant;
    if (tenantVal !== '1') throw new Error(`Expected 1, got ${tenantVal}`);
  });

  // Execute Tenant 2
  await withTenantTransaction(pool, { tenant_id: 2 }, async (client, ctx) => {
    const tenantVal = (await client.query("SELECT current_setting('app.tenant_id', true) AS tenant")).rows[0].tenant;
    if (tenantVal !== '2') throw new Error(`Expected 2, got ${tenantVal}`);
  });
}

```

```

});

    console.log(` ${GREEN}? ??? : ?? ????? ?????????? ?????????? ?????? ?? ?????? ?? ?????? ${RESET}`);
} catch (error) {
    console.log(` ${RED}? ??? : ??? ??? ?????? ?? ??????: ${error.message}${RESET}`);
    failed = true;
}

// Test 6: Verify no plain SET statements in helper file
try {
    console.log(`\n[Test 6] ??? ?????? ?????????? ?? helper ?????? ??? ?????????? SET ?????????...`);
    const helperContent = fs.readFileSync(path.join(__dirname, 'tenant_context_pg_session.js'), 'utf8');

    // Regex checking for client.query("SET app.tenant_id...") or similar plain SET statements
    const hasPlainSet = /client\.query\(\s*['"]SET\s+/i.test(helperContent);

    if (!hasPlainSet) {
        console.log(` ${GREEN}? ??? : ?? ?????? ?? ???????????? SET ?????? ?????????? ?? ?????? ?????????? ?? helper. ${RESET}`);
    } else {
        console.log(` ${RED}? ??? : ?? ?????????? ??? ?????????? SET ?????? ?? ?????? ?????????! ${RESET}`);
        failed = true;
    }
} catch (error) {
    console.log(` ${RED}? ??? : ??? ??? ?????? ?? ??????: ${error.message}${RESET}`);
    failed = true;
}

// Test 7: Connection leak check (checking pool status)
try {
    console.log(`\n[Test 7] ?????? ?? ??? ?????? ?????? ?? ?????????? (Connection Leaks Check)...`);

    const activeCountBefore = pool.totalCount - pool.idleCount;

    await withTenantTransaction(pool, { tenant_id: 100 }, async (client, ctx) => {
        return 1;
    });

    const activeCountAfter = pool.totalCount - pool.idleCount;

    if (activeCountAfter <= activeCountBefore + 1) {
        console.log(` ${GREEN}? ??? : ?? ?????? ?? ?????? ?? ???????????? ??? ?????? ??? middleware (??????????? ???
        ??? ??????????: ${activeCountAfter}) ${RESET}`);
    } else {
        console.log(` ${RED}? ??? : ???????????? ?????????? ?????? ?????? ??? ??????! (??? : ${activeCountBefore}? ??? :
        ${activeCountAfter}) ${RESET}`);
        failed = true;
    }
} catch (error) {
    console.log(` ${RED}? ??? : ??? ??? : ${error.message}${RESET}`);
    failed = true;
}

console.log(`\n${BOLD}${BLUE}===== ${RESET}`);

```

```

    if (failed) {
      console.log(`${BOLD}${RED}? ????? ??????? ???? Prototype: ??? ??? ??????????!!${RESET}`);
      process.exit(1);
    } else {
      console.log(`${BOLD}${GREEN}? ????? ??????? ???? Prototype: ??? ???? ?????????? 100%${RESET}`);
    }
    console.log(`${BOLD}${BLUE}===== ${RESET}\n`);
  }
}

```

```

runTests().catch(err => {
  console.error(err);
  process.exit(1);
});

```

```

=====
FILE: tenant_middleware.js
=====

```

```

const { runWithTenant } = require('./tenant_context');

function tenantMiddleware(req, res, next) {
  // Extract tenant ID from request headers
  let tenantId = req.headers['x-tenant-id'] || req.headers['x-tenant-id-key'];

  // Fallback to host-based subdomain if header is not present
  if (!tenantId && req.headers.host) {
    const hostParts = req.headers.host.split('.');
    if (hostParts.length > 2) {
      const subdomain = hostParts[0];
      // If subdomain is not 'www' or a common system subdomain, check if it maps to a tenant ID
      // For development, if subdomain is a number, treat it as the tenant ID.
      if (subdomain !== 'www' && !isNaN(subdomain)) {
        tenantId = parseInt(subdomain);
      }
    }
  }

  // Parse to integer or default to 1 (development fallback)
  const parsedTenantId = parseInt(tenantId) || 1;

  // Wrap request execution in the AsyncLocalStorage context
  runWithTenant({ tenantId: parsedTenantId }, () => {
    next();
  });
}

module.exports = tenantMiddleware;

```

```

=====
FILE: tenant_resolve.js

```


=====

```
/**
 * tenant_resolve.js ? Gate 4 pure tenant-context resolution (RLS binding source).
 *
 * SECURITY (anti cross-tenant spoof): the tenant of a tenant-BOUND authenticated user is
 * taken from the trusted SESSION and can NEVER be overridden by a client-supplied
 * x-tenant-id header. The previous logic read the header FIRST, so a user authenticated
 * for tenant 5 could send `x-tenant-id: 7` and bind the RLS connection var app.tenant_id
 * to tenant 7 (RLS was then the only, defeated, backstop).
 *
 * Precedence:
 * 1. Session user WITH a valid tenantId -> session tenant (header IGNORED). [the fix]
 * 2. Session user WITHOUT a tenantId (tenant-unbound super admin) -> header if valid.
 * 3. No session -> dev fallback tenant 1 in non-production only; null in production.
 * Any tenant id must be a positive integer; invalid values are treated as absent and are
 * NEVER silently coerced to 1.
 *
 * ??? ??? ?????: ????? ???? ???? ?????? ?????? ?????? (??? ??????).
 */
'use strict';

function positiveInt(v) {
  if (v === null || v === undefined || v === '') return null;
  const n = Number(v);
  if (!Number.isInteger(n) || n <= 0) return null;
  return n;
}

function resolveTenantContext({ headerTenant, sessionUser, isProduction } = {}) {
  const sessionTenant = sessionUser ? positiveInt(sessionUser.tenantId) : null;

  // 1. Tenant-bound authenticated user ? trusted session wins, header ignored.
  if (sessionTenant !== null) {
    const facilityId = sessionUser && sessionUser.facilityId !== null ? sessionUser.facilityId : null;
    return { tenantId: sessionTenant, facilityId, isProduction: !!isProduction, source: 'session' };
  }

  // 2. Authenticated but tenant-unbound (e.g. super admin) ? may scope via header.
  if (sessionUser) {
    const headerT = positiveInt(headerTenant);
    if (headerT !== null) {
      return { tenantId: headerT, facilityId: null, isProduction: !!isProduction, source: 'header-privileged' };
    }
    return { tenantId: null, facilityId: null, isProduction: !!isProduction, source: 'none' };
  }

  // 3. Unauthenticated ? dev fallback only, never honor a header, never in production.
  if (!isProduction) {
    return { tenantId: 1, facilityId: 1, isProduction: false, source: 'dev-fallback' };
  }
  return { tenantId: null, facilityId: null, isProduction: true, source: 'none' };
}
```

```
module.exports = { resolveTenantContext, positiveInt };
```

```
=====
```

```
FILE: tenant_resolve_test.js
```

```
=====
```

```
/**
 * tenant_resolve_test.js ? PURE unit test for Gate 4 (tenant-context resolution precedence).
 * Run: node tenant_resolve_test.js    (no DB; deterministic)
 *
 * SECURITY invariant: a tenant-BOUND authenticated user's tenant comes from the SESSION and
 * can NEVER be overridden by a client-supplied x-tenant-id header (the old code read the
 * header first, letting a user for tenant 5 bind RLS context to tenant 7). The header is
 * honored ONLY for a tenant-UNBOUND privileged session (super admin) or dev fallback.
 */
'use strict';
const T = require('./tenant_resolve');

const GREEN = '\x1b[32m', RED = '\x1b[31m', RESET = '\x1b[0m', BOLD = '\x1b[1m';
let passed = 0, failed = 0; const fails = [];
function assert(cond, name, det = '') {
  if (cond) { console.log(` ${GREEN}PASS${RESET} ${name}`); passed++; }
  else { console.log(` ${RED}FAIL${RESET} ${name}${det ? ' | ' + det : ''}`); failed++; fails.push(name); }
}

console.log(`${BOLD}Gate 4 ? tenant context resolution (pure unit test)${RESET}\n`);

// ---- THE FIX: session wins over spoofed header ----
console.log('[1] tenant-bound session ignores x-tenant-id header (anti-spoof)');
let r = T.resolveTenantContext({ headerTenant: '7', sessionUser: { tenantId: 5, facilityId: 2 }, isProduction: true });
assert(r.tenantId === 5 && r.source === 'session', 'session tenant 5 + header 7 -> 5 (header ignored)', JSON.stringify(r));
r = T.resolveTenantContext({ headerTenant: '99', sessionUser: { tenantId: 5, facilityId: 2 }, isProduction: false });
assert(r.tenantId === 5, 'even in dev, bound session beats header', JSON.stringify(r));
r = T.resolveTenantContext({ headerTenant: 5, sessionUser: { tenantId: 5, facilityId: 2 }, isProduction: true });
assert(r.tenantId === 5 && r.facilityId === 2, 'facility from session', JSON.stringify(r));

// ---- privileged tenant-unbound session may use header ----
console.log('\n[2] tenant-unbound (super-admin) session may scope via header');
r = T.resolveTenantContext({ headerTenant: '3', sessionUser: { tenantId: null, role: 'SuperAdmin' }, isProduction: true });
assert(r.tenantId === 3 && r.source === 'header-privileged', 'unbound session + header 3 -> 3', JSON.stringify(r));
r = T.resolveTenantContext({ headerTenant: null, sessionUser: { tenantId: null, role: 'SuperAdmin' }, isProduction: true });
assert(r.tenantId === null && r.source === 'none', 'unbound session no header -> null', JSON.stringify(r));
r = T.resolveTenantContext({ headerTenant: 'abc', sessionUser: { tenantId: null }, isProduction: true });
assert(r.tenantId === null, 'garbage header -> null (never coerced to 1)', JSON.stringify(r));
```

```

r = T.resolveTenantContext({ headerTenant: '0', sessionUser: { tenantId: null }, isProduction: true });
assert(r.tenantId === null, 'header 0 -> null (must be positive int)', JSON.stringify(r));
r = T.resolveTenantContext({ headerTenant: '-2', sessionUser: { tenantId: null }, isProduction: true });
assert(r.tenantId === null, 'negative header -> null', JSON.stringify(r));

// ---- no session ----
console.log('\n[3] unauthenticated ? dev fallback only, never header, never prod');
r = T.resolveTenantContext({ headerTenant: '9', sessionUser: null, isProduction: true });
assert(r.tenantId === null && r.source === 'none', 'no session + header in PROD -> null (no leak)', JSON.stringify(r));
r = T.resolveTenantContext({ headerTenant: '9', sessionUser: null, isProduction: false });
assert(r.tenantId === 1 && r.source === 'dev-fallback', 'no session in DEV -> tenant 1 fallback (header ignore d)', JSON.stringify(r));
r = T.resolveTenantContext({ headerTenant: null, sessionUser: null, isProduction: false });
assert(r.tenantId === 1, 'no session no header in DEV -> 1', JSON.stringify(r));
r = T.resolveTenantContext({ headerTenant: null, sessionUser: null, isProduction: true });
assert(r.tenantId === null, 'no session no header in PROD -> null', JSON.stringify(r));

// ---- invalid session tenant treated as unbound (not coerced) ----
console.log('\n[4] invalid session tenantId is not silently coerced');
r = T.resolveTenantContext({ headerTenant: null, sessionUser: { tenantId: 0 }, isProduction: true });
assert(r.tenantId === null, 'session tenantId 0 -> treated as unbound null', JSON.stringify(r));
r = T.resolveTenantContext({ headerTenant: '4', sessionUser: { tenantId: 'x' }, isProduction: true });
assert(r.tenantId === 4 && r.source === 'header-privileged', 'invalid session tenant + header -> unbound path uses header', JSON.stringify(r));

console.log(`\n${BOLD}Result:${RESET} ${passed} passed, ${failed} failed`);
if (failed) { console.log(`${RED}FAILURES:${RESET} ${fails.join(', ')} `); process.exit(1); }
process.exit(0);

```

```

=====
FILE: tracked_secret_redaction_test.js
=====

```

```

/**
 * tracked_secret_redaction_test.js
 * =====
 * PHASE 1 C-1B ? current-tree secret guard across ALL tracked files in BOTH repos
 * (namaweb + parent NamaMedical): scripts, docs, project_brain, markdown, txt, json, sh, js.
 *
 * Fails if any tracked file contains a hardcoded secret VALUE (password / connection string /
 * JWT or SESSION secret / API key). Detection is by SHAPE ? this test embeds NO secret of its own.
 *
 * Per-MATCH allow-list (NOT whole-file exclusions): env-var expansions ($VAR / ${VAR}),
 * obvious placeholders (__CHANGE_ME__, <...>, [REDACTED...], CHANGE_ME, EXAMPLE/placeholder),
 * and the in-source dev-only fallbacks. *_test.js files and *.example files are skipped because
 * they legitimately contain detection patterns / placeholders.
 *
 * node tracked_secret_redaction_test.js
 */
const { execSync } = require('child_process');
const fs = require('fs');

```

```

const path = require('path');

const RED = '\x1b[31m', GREEN = '\x1b[32m', BLUE = '\x1b[34m', RESET = '\x1b[0m', BOLD = '\x1b[1m';
let passed = 0, failed = 0; const failures = [];
function assert(cond, name, details = '') {
  if (cond) { console.log(` ${GREEN}PASS${RESET} ? ${name}`); passed++; }
  else { console.log(` ${RED}FAIL${RESET} ? ${name}${details ? ' | ' + details : ''}`); failed++; failures.
push({ name, details }); }
}

const repos = [
  { name: 'namaweb', root: __dirname },
  { name: 'parent', root: path.join(__dirname, '..') },
];

// A matched value is acceptable (not a real secret) when it is a var / placeholder / redacted / dev-only.
function isAllowed(v) {
  if (!v) return true;
  const s = String(v).trim().replace(/^["]|["$"]$/g, '');
  if (s === '') return true;
  if (s.length <= 2) return true; // 1?2 chars cannot be a real secret (ellipsi
s/fragments)
  if (s.startsWith('$')) return true; // $VAR / ${VAR} / ${ secrets.X }
  if (/^[?~-]/.test(s)) return true; // ${VAR:?msg} / ${VAR:-default} expansion re
mainder
  if (/^__.*__$/.test(s)) return true; // __CHANGE_ME__
  if (/^<[()]/.test(s)) return true; // <placeholder> / [REDACTED...] / (code expr
  if (/REDACTED|CHANGE_ME|PLACEHOLDER|EXAMPLE|YOUR_|XXXX|\.\.\.$/.test(s)) return true;
  if (s === 'dev-only-insecure-secret-change-me') return true; // in-source dev fallback (guarded in prod)
  if (/^postgres:postgres@localhost/.test(s)) return true; // dev localhost default
  if (s === 'postgres') return true; // bare default dev value
  return false;
}

// Detection rules ? each returns the captured secret VALUE (group) to test against isAllowed().
const RULES = [
  { what: 'connection-string password', re: /\b(?:postgres(?:ql)?|mysql|mongodb(?:\+srv)?|redis|amqp):\\\/[^\
:\/\s"]+:([@\s"'`]+)@/ig, group: 1 },
  { what: 'SQL WITH PASSWORD literal', re: /\b(?:WITH PASSWORD|IDENTIFIED BY)\s+'([^\']+)'/ig, group: 1 },
  { what: 'sqlcmd -P password literal', re: /-P\s+"([^\"]+)"/g, group: 1 },
  { what: 'JWT/SESSION/API secret assignment', re: /\b(?:JWT_SECRET|SESSION_SECRET|API_KEY|SECRET_KEY|ACCESS
_KEY|PRIVATE_KEY|MSSQL_SA_PASSWORD)\b\s*[:=]\s*"([^\s"#`]+)"/ig, group: 1 },
  { what: 'generic *PASSWORD literal', re: /\b[A-Za-z_]*PASSWORD\b\s*[:=]\s*"([^\']+)"/ig, group: 1 },
];

const SKIP_EXT = new Set(['.bak', '.db', '.png', '.jpg', '.jpeg', '.gif', '.bmp', '.webp', '.ico', '.pdf', '.z
ip', '.gz', '.exe', '.msi', '.dll', '.lock']);
function shouldSkip(rel) {
  if (rel.includes('node_modules/')) return true;
  if (/\.example($|\.)/.test(rel) || /\.env\.example/.test(rel)) return true; // placeholder templates
  if (SKIP_EXT.has(path.extname(rel).toLowerCase())) return true;
  return false;
  // NOTE: *_test.js files are NOT skipped ? a real password literal in a test file MUST fail.
  // (Detection-regex source lines are tolerated via isAllowed + isRegexDefLine, never real values.)

```

```

}

// A line that is a detection-rule definition (regex source / pattern keyword list), not a real secret.
function isRegexDefLine(line) {
    return /(\\s[*+]|\\[\\^|new RegExp|re:s*\\|\\.|.test\\(|\\.exec\\(|\\.replace\\(|RULES|LITERAL_RULES|\\.\\.\\.RULES)/.
test(line);
}

console.log(`\n${BOLD}${BLUE}=== Tracked-Tree Secret Redaction Guard (C-1B) ===${RESET}\n`);

let scanned = 0, totalViolations = 0; const violationDetail = [];
for (const repo of repos) {
    let files = [];
    try { files = execSync('git ls-files', { cwd: repo.root, encoding: 'utf8' }).split(/\r?\n/).filter(Boolean
); }
    catch (e) { console.log(` (skip ${repo.name}: not a git repo)`); continue; }
    for (const rel of files) {
        if (shouldSkip(rel)) continue;
        const isTest = /^(^|\/)[^\/]*_test\.js$/.test(rel);
        const abs = path.join(repo.root, rel);
        let st; try { st = fs.statSync(abs); } catch (e) { continue; }
        if (st.size > 2 * 1024 * 1024) continue; // skip very large data files
        let content; try { content = fs.readFileSync(abs, 'utf8'); } catch (e) { continue; }
        scanned++;
        const lines = content.split(/\r?\n/);
        lines.forEach((line, i) => {
            // skip secret-GENERATION snippets (they derive a value, they don't hardcode one)
            if (/randomBytes|crypto\.|openssl\s+rand|node\s+-e/.test(line)) return;
            // skip detection-rule definition lines (regex sources / pattern keyword lists are not secrets)
            if (isRegexDefLine(line)) return;
            for (const rule of RULES) {
                rule.re.lastIndex = 0;
                let m;
                while ((m = rule.re.exec(line)) !== null) {
                    const val = m[rule.group];
                    if (isAllowed(val)) continue;
                    // In *_test.js, short fixtures (wrong-password / validation inputs, <16 chars) are
                    // legitimate test DATA, NOT secrets ? but credential-grade values (>=16 chars, e.g. a
                    // real DB password / session secret / API key) STILL fail even inside a test file.
                    const clean = String(val).trim().replace(/^['"]|['"]$/g, '');
                    if (isTest && clean.length < 16) continue;
                    totalViolations++;
                    violationDetail.push(`${repo.name}:${rel}:${i + 1} (${rule.what})`);
                }
            }
        });
    }
}

assert(scanned > 0, 'tracked files were scanned across both repos', `scanned ${scanned} files`);
if (totalViolations > 0) {
    console.log(` ${RED}Violations:${RESET}`);
    violationDetail.slice(0, 50).forEach(v => console.log(` - ${v}`));
}

```

```
assert(totalViolations === 0, 'ZERO hardcoded secret values in the tracked tree', `${totalViolations} violations(s)`);
```

```
console.log(`\n${BOLD}Result: ${passed} passed, ${failed} failed (scanned ${scanned} tracked files){RESET}`);
if (failed > 0) { process.exit(1); }
process.exit(0);
```

```
=====
FILE: user_provisioning.js
=====
```

```
/**
 * user_provisioning.js ? Jumanasoft atomic system-user creation with tenant linkage (Batch 4D).
 *
 * Closes the max_users count gap: a tenant user created via POST /api/settings/users is linked into
 * user_tenants in the SAME transaction as the system_users insert, so entitlements.countTenantUsers
 * (which reads user_tenants) reflects it. No orphan: a link failure rolls back the whole creation.
 *
 * Security: tenantId is supplied by the CALLER from the session (never the request body). app.tenant_id
 * is bound (transaction-local) for the RLS-protected user_tenants insert. Duplicate link is a no-op
 * (ON CONFLICT (user_id, tenant_id) DO NOTHING via uq_user_tenant).
 *
 * The caller owns the client lifecycle (pool.connect / client.release).
 */
'use strict';

const SELECT_USER_COLS = 'id, username, display_name, role, speciality, permissions, commission_type, commission_value, is_active, created_at';

/**
 * createSystemUserWithTenantLink(client, opts) -> created user row.
 * opts: { user: {username,password_hash,display_name,role,speciality,permissions,commission_type,commission_value}, tenantId }
 * Runs BEGIN..COMMIT; throws (after ROLLBACK) on any failure so no partial/orphan row remains.
 */
async function createSystemUserWithTenantLink(client, opts = {}) {
  const u = opts.user || {};
  const tenantId = opts.tenantId;
  await client.query('BEGIN');
  try {
    // bind tenant context for the RLS-protected user_tenants insert (transaction-local)
    if (tenantId) await client.query("SELECT set_config('app.tenant_id', $1, true)", [String(tenantId)]);
    const ins = await client.query(
      `INSERT INTO system_users (username, password_hash, display_name, role, speciality, permissions, commission_type, commission_value) VALUES ($1,$2,$3,$4,$5,$6,$7,$8) RETURNING id`,
      [u.username, u.password_hash, u.display_name || '', u.role || 'Reception', u.speciality || '', u.permissions || '', u.commission_type || 'percentage', parseFloat(u.commission_value) || 0]
    );
    const newId = ins.rows[0].id;
    // link to the actor's tenant (idempotent). tenantId is from the session, NEVER the request body.
    if (tenantId) {
      await client.query(
```

```

        'INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1,$2,true) ON CONFLICT (user_id, tenant_id) DO NOTHING',
        [newId, tenantId]
    );
}
const row = (await client.query('SELECT ' + SELECT_USER_COLS + ' FROM system_users WHERE id=$1', [newId])).rows[0];
await client.query('COMMIT');
return row;
} catch (e) {
    try { await client.query('ROLLBACK'); } catch (_) { /* best-effort */ }
    throw e;
}
}
}

```

```

module.exports = { createSystemUserWithTenantLink, SELECT_USER_COLS };

```

```

=====

```

```

FILE: user_tenant_linkage_test.js

```

```

=====

```

```

/**
 * user_tenant_linkage_test.js ? Batch 4D tests.
 *   Part A: behavioral ? createSystemUserWithTenantLink against a fake pg client + in-memory store,
 *   proving atomic create+link, count growth (via entitlements.countTenantUsers), rollback, etc.
 *   Part B: static ? server.js route wires the helper, uses a transaction, reads tenantId from session
 *   (not body), keeps the 4C guard chain.
 * Run: node user_tenant_linkage_test.js (exit 0 = all pass)
 */
'use strict';
const fs = require('fs');
const path = require('path');
const UP = require('./user_provisioning');
const E = require('./entitlements');

let pass = 0, fail = 0;
function ok(name, cond) { if (cond) pass++; else { fail++; console.error(' FAIL:', name); } }

// In-memory store shared by the fake client (writes) and fake pool (countTenantUsers reads).
function makeStore() { return { users: [], links: [], seq: 100, log: [], appTenant: null, committed: false, rolledback: false }; }
function fakeClient(store, opts = {}) {
    return {
        query(sql, params) {
            const s = sql.replace(/\s+/g, ' ').trim();
            if (/^BEGIN/.test(s)) { store.log.push('BEGIN'); return Promise.resolve({ rows: [] }); }
            if (/^COMMIT/.test(s)) { store.committed = true; store.log.push('COMMIT'); return Promise.resolve({ rows: [] }); }
            if (/^ROLLBACK/.test(s)) { store.rolledback = true; store.log.push('ROLLBACK'); return Promise.resolve({ rows: [] }); }
            if (/set_config\(\'app\.tenant_id\'/.test(s)) { store.appTenant = params[0]; store.log.push('SET_TENANT:' + params[0]); return Promise.resolve({ rows: [] }); }

```

```

    if (/INSERT INTO system_users/.test(s)) {
        store.log.push('INS_USER');
        const id = ++store.seq;
        store.users.push({ id, username: params[0], display_name: params[2], role: params[3], speciality: params[4], permissions: params[5], commission_type: params[6], commission_value: params[7], is_active: 1, created_at: 'd' });
        return Promise.resolve({ rows: [{ id }] });
    }
    if (/INSERT INTO user_tenants/.test(s)) {
        store.log.push('INS_LINK');
        if (opts.failLink) throw new Error('RLS denied / link failure');
        const [uid, tid] = params;
        if (!store.links.some(l => l.user_id === uid && l.tenant_id === tid)) store.links.push({ user_id: uid, tenant_id: tid, is_active: true });
        return Promise.resolve({ rows: [] });
    }
    if (/SELECT .* FROM system_users WHERE id=\$1/.test(s)) {
        const u = store.users.find(x => x.id === params[0]);
        return Promise.resolve({ rows: u ? [u] : [] });
    }
    return Promise.resolve({ rows: [] });
}
};
}
function fakePool(store) {
    return {
        query(sql, params) {
            const s = sql.replace(/\s+/g, ' ');
            if (/COUNT(\*\)::int AS c FROM user_tenants WHERE tenant_id=\$1 AND is_active=true/.test(s)) {
                const c = store.links.filter(l => l.tenant_id === params[0] && l.is_active).length;
                return Promise.resolve({ rows: [{ c }] });
            }
            return Promise.resolve({ rows: [] });
        }
    };
}
const baseUser = { username: 'newdoc', password_hash: '$2bxxx', display_name: 'New Doc', role: 'Doctor' };

(async () => {
    // 1) create + link atomically (tenant from caller/session)
    let store = makeStore();
    let row = await UP.createSystemUserWithTenantLink(fakeClient(store), { user: baseUser, tenantId: 7 });
    ok('returns created user row', row && row.username === 'newdoc');
    ok('system_users row created', store.users.length === 1);
    ok('user_tenants link created to tenant 7', store.links.length === 1 && store.links[0].tenant_id === 7 && store.links[0].user_id === row.id);
    ok('app.tenant_id bound to 7', store.appTenant === '7');
    ok('committed, not rolled back', store.committed === true && store.rolledback === false);
    ok('transaction order BEGIN<set<insUser<insLink<COMMIT',
        store.log.indexOf('BEGIN') === 0 &&
        store.log.indexOf('SET_TENANT:7') < store.log.indexOf('INS_USER') &&
        store.log.indexOf('INS_USER') < store.log.indexOf('INS_LINK') &&
        store.log.indexOf('INS_LINK') < store.log.indexOf('COMMIT'));

```



```

// 2) countTenantUsers grows after creation (same store, real countTenantUsers)
store = makeStore();
const pool = fakePool(store);
const before = await E.countTenantUsers(pool, 7);
await UP.createSystemUserWithTenantLink(fakeClient(store), { user: baseUser, tenantId: 7 });
const after = await E.countTenantUsers(pool, 7);
ok('countTenantUsers grows by 1 after create', before === 0 && after === 1);

// 3) no tenantId -> user created, NO link, NO set_config, no throw (edge: non-tenant context)
store = makeStore();
row = await UP.createSystemUserWithTenantLink(fakeClient(store), { user: baseUser, tenantId: null });
ok('no tenant: user created', store.users.length === 1);
ok('no tenant: no link', store.links.length === 0);
ok('no tenant: app.tenant_id not bound', store.appTenant === null);
ok('no tenant: committed', store.committed === true);

// 4) link failure -> ROLLBACK, no commit, throws (no orphan persisted in a real DB)
store = makeStore();
let threw = false;
try { await UP.createSystemUserWithTenantLink(fakeClient(store), { failLink: true }, { user: baseUser, tenantId: 7 }); }
catch (e) { threw = true; }
ok('link failure throws', threw);
ok('link failure rolled back (not committed)', store.rolledback === true && store.committed === false);

// 5) duplicate link -> ON CONFLICT DO NOTHING (no dup, no throw) ? simulate same pair pre-seeded
store = makeStore();
store.links.push({ user_id: 101, tenant_id: 7, is_active: true }); // seq starts at 100 -> next id 101
row = await UP.createSystemUserWithTenantLink(fakeClient(store), { user: baseUser, tenantId: 7 });
ok('duplicate link: no throw + no duplicate row', store.links.filter(l => l.user_id === 101 && l.tenant_id === 7).length === 1 && store.committed === true);

// 6) global user (no link) is NOT counted in a tenant
store = makeStore();
store.links.push({ user_id: 500, tenant_id: 9, is_active: true }); // member of tenant 9 only
ok('global/non-member not counted for tenant 7', (await E.countTenantUsers(fakePool(store), 7)) === 0);

// ----- Part B: static wiring checks on server.js -----
const src = fs.readFileSync(path.join(__dirname, 'server.js'), 'utf8');
const start = src.indexOf("app.post('/api/settings/users'");
const body = src.slice(start, src.indexOf("app.put('/api/settings/users/:id'"));
ok('route uses createSystemUserWithTenantLink', /createSystemUserWithTenantLink\/.test(body));
ok('route uses a dedicated transaction client', /pool\.connect\(\)\.test(body) && /client\.release\(\)\.test(body));
ok('tenantId from session, not body', /req\.session\.user && req\.session\.user\.tenantId\/.test(body) && !/req\.body\.tenant_id\/.test(body));
ok('4C guard chain preserved', /requireTenantAdmin\([^\)]*\), userLimitGuard\/.test(body));
ok('helper module binds app.tenant_id + ON CONFLICT', /set_config\('app\.tenant_id'\/.test(fs.readFileSync(path.join(__dirname, 'user_provisioning.js'), 'utf8')) && /ON CONFLICT \((user_id, tenant_id\) DO NOTHING\/.test(fs.readFileSync(path.join(__dirname, 'user_provisioning.js'), 'utf8'))));

console.log(`user_tenant_linkage_test: ${pass} passed, ${fail} failed`);
process.exit(fail === 0 ? 0 : 1);
})();

```

=====

FILE: validation.js

=====

```
/**
 * validation.js ? central, dependency-free input validation (GATE3-H1 remediation).
 *
 * Why: input validation was scattered inline across ~538 routes with no single source of truth, so
 * non-money fields (required strings, ids, enums, dates, lengths, Saudi national-id/phone) were
 * validated inconsistently or not at all. Money fields are already robustly validated by
 * ./billing_integrity (parseMoney / assertAmountWithinCap, fail-closed) ? this module complements that
 * for the NON-money fields and is the building block for a gradual per-route rollout.
 *
 * Design (mirrors ./billing_integrity):
 *   - Pure, side-effect-free, no external dependency.
 *   - FAIL-CLOSED: every validator throws a ValidationError on bad input (never silently coerces).
 *   - Errors carry .statusCode = 400 so the existing sendBillingError(res, e) path (server.js) maps
 *     them to HTTP 400 automatically, and `.field` for client-friendly messages.
 *   - validateBody(schema) returns an Express middleware; validate(obj, schema) is the pure core.
 *
 * IMPORTANT: this module is ADDITIVE. Importing it has ZERO runtime effect until a route opts in via
 * validate()/validateBody(). It does not modify or replace any existing inline checks.
 */
'use strict';

class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = 'ValidationError';
    this.statusCode = 400;
    if (field) this.field = field;
  }
}

function fail(field, msg) { throw new ValidationError(`${field}: ${msg}`, field); }

// ---- primitive validators (each returns the cleaned value or throws) ----

function str(value, { field = 'value', required = true, min = 0, max = 10000, trim = true } = {}) {
  if (value === undefined || value === null || value === '') {
    if (required) fail(field, 'is required');
    return '';
  }
  if (typeof value !== 'string') fail(field, 'must be a string');
  const v = trim ? value.trim() : value;
  if (required && v.length === 0) fail(field, 'must not be blank');
  if (v.length < min) fail(field, `must be at least ${min} characters`);
  if (v.length > max) fail(field, `must be at most ${max} characters`);
  return v;
}
```

```

function int(value, { field = 'value', required = true, min = -Infinity, max = Infinity } = {}) {
  if (value === undefined || value === null || value === '') {
    if (required) fail(field, 'is required');
    return null;
  }
  const n = Number(value);
  if (!Number.isInteger(n)) fail(field, 'must be an integer');
  if (n < min) fail(field, `must be >= ${min}`);
  if (n > max) fail(field, `must be <= ${max}`);
  return n;
}

// positive integer primary-key / foreign-key id
function id(value, { field = 'id', required = true } = {}) {
  return int(value, { field, required, min: 1, max: Number.MAX_SAFE_INTEGER });
}

function enumOf(value, allowed, { field = 'value', required = true } = {}) {
  if (value === undefined || value === null || value === '') {
    if (required) fail(field, 'is required');
    return '';
  }
  if (!Array.isArray(allowed) || !allowed.includes(value)) {
    fail(field, `must be one of: ${allowed || []}.join(', ')`);
  }
  return value;
}

function bool(value, { field = 'value', required = false } = {}) {
  if (value === undefined || value === null || value === '') {
    if (required) fail(field, 'is required');
    return false;
  }
  if (value === true || value === 1 || value === '1' || value === 'true') return true;
  if (value === false || value === 0 || value === '0' || value === 'false') return false;
  fail(field, 'must be a boolean');
}

// accepts a parseable date (ISO 8601 preferred); returns the original string after validity check
function dateStr(value, { field = 'date', required = true } = {}) {
  if (value === undefined || value === null || value === '') {
    if (required) fail(field, 'is required');
    return '';
  }
  if (typeof value !== 'string') fail(field, 'must be a date string');
  const t = Date.parse(value);
  if (Number.isNaN(t)) fail(field, 'must be a valid date');
  return value;
}

// Saudi national id / iqama: exactly 10 digits (optional unless required)
function nationalId(value, { field = 'national_id', required = false } = {}) {
  if (value === undefined || value === null || value === '') {
    if (required) fail(field, 'is required');
  }

```

```

        return '';
    }
    const v = String(value).trim();
    if (!/^d{10}$/.test(v)) fail(field, 'must be 10 digits');
    return v;
}

// phone: 7..15 chars, digits with optional leading + and spaces/dashes
function phone(value, { field = 'phone', required = false } = {}) {
    if (value === undefined || value === null || value === '') {
        if (required) fail(field, 'is required');
        return '';
    }
    const v = String(value).trim();
    if (!/^+?\d{s-}{7,15}$/.test(v)) fail(field, 'must be a valid phone number');
    return v;
}

// ---- schema runner ----
// schema: { fieldName: (value, ctx) => cleaned } where each fn is one of the validators above bound
// with options, OR a plain options object { type, ...opts }. Returns a NEW object of cleaned values.
const TYPES = { str, int, id, enumOf, bool, dateStr, nationalId, phone };

function validate(obj, schema) {
    if (!obj || typeof obj !== 'object') throw new ValidationError('body: must be an object', 'body');
    const out = {};
    for (const field of Object.keys(schema)) {
        const spec = schema[field];
        if (typeof spec === 'function') {
            out[field] = spec(obj[field], field);
        } else if (spec && typeof spec === 'object' && spec.type) {
            const fn = TYPES[spec.type];
            if (!fn) throw new Error(`validation: unknown type '${spec.type}' for field '${field}'`);
            const { type, allowed, ...opts } = spec;
            out[field] = type === 'enumOf'
                ? fn(obj[field], allowed, { field, ...opts })
                : fn(obj[field], { field, ...opts });
        } else {
            throw new Error(`validation: invalid spec for field '${field}'`);
        }
    }
    return out;
}

// Express middleware: validates req.body against schema, attaches cleaned values to req.validated,
// and on failure responds 400 (fail-closed) without leaking internals.
function validateBody(schema) {
    return function (req, res, next) {
        try {
            req.validated = validate(req.body || {}, schema);
            return next();
        } catch (e) {
            if (e instanceof ValidationError) {
                return res.status(400).json({ error: e.message, field: e.field });
            }
        }
    };
}

```

```

    }
    console.error('validation middleware error:', e.message);
    return res.status(400).json({ error: 'Invalid request' });
  }
};
}

```

```

module.exports = {
  ValidationError,
  str, int, id, enumOf, bool, dateStr, nationalId, phone,
  validate, validateBody
};

```

```

=====
FILE: validation_test.js
=====

```

```

/**
 * validation_test.js ? pure unit tests for ./validation (no DB, no network).
 * Run: node validation_test.js    (exit 0 = all pass)
 */
'use strict';
const v = require('./validation');

let pass = 0, fail = 0;
function ok(name, cond) { if (cond) { pass++; } else { fail++; console.error('  FAIL:', name); } }
function throws(name, fn) {
  try { fn(); ok(name + ' (should throw)', false); }
  catch (e) { ok(name, e instanceof v.ValidationError && e.statusCode === 400); }
}
function noThrow(name, fn) { try { fn(); ok(name, true); } catch (e) { ok(name + ' (' + e.message + ')', false); } }

// str
noThrow('str accepts normal', () => v.str('Ahmad', { field: 'name' }));
ok('str trims', v.str('  x  ', { field: 'n' }) === 'x');
throws('str required blank', () => v.str(' ', { field: 'n' }));
throws('str required undefined', () => v.str(undefined, { field: 'n' }));
throws('str max length', () => v.str('abcdef', { field: 'n', max: 3 }));
throws('str min length', () => v.str('ab', { field: 'n', min: 3 }));
throws('str non-string', () => v.str(123, { field: 'n' }));
ok('str optional empty -> ""', v.str('', { field: 'n', required: false }) === '');

// int / id
ok('int parses', v.int('42', { field: 'x' }) === 42);
throws('int non-integer', () => v.int('4.5', { field: 'x' }));
throws('int below min', () => v.int(0, { field: 'x', min: 1 }));
throws('int above max', () => v.int(11, { field: 'x', max: 10 }));
ok('id accepts positive', v.id(7) === 7);
throws('id rejects zero', () => v.id(0));
throws('id rejects negative', () => v.id(-3));
throws('id rejects non-int', () => v.id('abc'));

```

```

// enumOf
ok('enum accepts allowed', v.enumOf('Paid', ['Paid', 'Pending'])) === 'Paid');
throws('enum rejects other', () => v.enumOf('Hacked', ['Paid', 'Pending']));
throws('enum required missing', () => v.enumOf('', ['A'], { required: true }));

// bool
ok('bool true variants', v.bool('1') === true && v.bool(true) === true && v.bool('true') === true);
ok('bool false variants', v.bool('0') === false && v.bool(false) === false);
throws('bool invalid', () => v.bool('maybe'));

// dateStr
noThrow('date ISO ok', () => v.dateStr('2026-06-30'));
throws('date invalid', () => v.dateStr('not-a-date'));
throws('date required missing', () => v.dateStr('', { required: true }));

// nationalId
ok('nationalId 10 digits', v.nationalId('1234567890') === '1234567890');
throws('nationalId 9 digits', () => v.nationalId('123456789', { required: true }));
throws('nationalId letters', () => v.nationalId('12345abcde', { required: true }));
ok('nationalId optional empty', v.nationalId('', { required: false }) === '');

// phone
noThrow('phone ok', () => v.phone('+966 50-123-4567'));
throws('phone too short', () => v.phone('12', { required: true }));

// validate() schema runner
noThrow('validate schema ok', () => v.validate(
  { name: 'Sara', age: '30', status: 'Active' },
  { name: { type: 'str', max: 50 }, age: { type: 'int', min: 0, max: 150 }, status: { type: 'enumOf', allowe
d: ['Active', 'Inactive'] } }
));
ok('validate returns cleaned', v.validate({ age: '30' }, { age: { type: 'int' } }).age === 30);
throws('validate fails on bad field', () => v.validate(
  { patient_id: 'x' }, { patient_id: { type: 'id' } }
));
throws('validate rejects non-object', () => v.validate(null, { a: { type: 'str' } }));

// validateBody middleware (fake req/res)
(function testMiddleware() {
  const mw = v.validateBody({ name: { type: 'str', max: 10 } });
  // success path
  let nextCalled = false; const req1 = { body: { name: 'ok' } };
  mw(req1, { status: () => ({ json: () => {} }) }, () => { nextCalled = true; });
  ok('middleware next on valid', nextCalled === true && req1.validated.name === 'ok');
  // failure path
  let status = 0, payload = null;
  const res2 = { status: (s) => { status = s; return { json: (p) => { payload = p; } } } };
  mw({ body: { name: 'this-is-way-too-long' } }, res2, () => { ok('middleware should NOT call next on invali
d', false); });
  ok('middleware 400 on invalid', status === 400 && payload && /name/.test(payload.error));
})();

console.log(`validation_test: ${pass} passed, ${fail} failed`);

```

```
process.exit(fail === 0 ? 0 : 1);
```

```
=====
FILE: waiting_queue_acuity_test.js
=====
```

```
/**
 * waiting_queue_acuity_test.js
 * Integration test for waiting_queue acuity sorting, status mapping, and triage APIs.
 */

const { spawn } = require('child_process');
const http = require('http');
const assert = require('assert');
const bcrypt = require('bcryptjs');
const { pool } = require('./db_postgres');

const TEST_PORT = 3012;
const TEST_USERNAME = 'queue_admin';
const TEST_PASSWORD = 'QUEUE_PASSWORD_PLACEHOLDER';

let server;

function makeRequest(method, path, body, headers = {}) {
  return new Promise((resolve, reject) => {
    const payload = body ? JSON.stringify(body) : '';
    const req = http.request({
      hostname: 'localhost',
      port: TEST_PORT,
      path,
      method,
      headers: {
        'Content-Type': 'application/json',
        'Content-Length': Buffer.byteLength(payload),
        ...headers
      }
    }, (res) => {
      let data = '';
      res.on('data', chunk => data += chunk);
      res.on('end', () => {
        try {
          const parsed = data ? JSON.parse(data) : {};
          resolve({ statusCode: res.statusCode, headers: res.headers, body: parsed });
        } catch (e) {
          resolve({ statusCode: res.statusCode, headers: res.headers, rawBody: data });
        }
      });
    });
    req.on('error', reject);
    if (payload) req.write(payload);
    req.end();
  });
}
```

```
}
```

```
async function runTests() {
  console.log('--- STARTING WAITING QUEUE ACUITY INTEGRATION TESTS ---');

  const patient1Id = 9901;
  const patient2Id = 9902;
  const adminUserId = 9903;
  const client = await pool.connect();

  try {
    await client.query("SET app.tenant_id = '1'");

    // Clean up
    await client.query('DELETE FROM waiting_queue WHERE patient_id IN ($1, $2)', [patient1Id, patient2Id])
;

    await client.query('DELETE FROM patients WHERE id IN ($1, $2)', [patient1Id, patient2Id]);
    await client.query('DELETE FROM user_tenants WHERE user_id = $1', [adminUserId]);
    await client.query('DELETE FROM system_users WHERE id = $1', [adminUserId]);

    // Insert patients
    await client.query(
      'INSERT INTO patients (id, name_en, name_ar, phone, tenant_id) VALUES ($1, $2, $3, $4, 1)',
      [patient1Id, 'Acuity Patient A', '???? ?', '+966555555501']
    );
    await client.query(
      'INSERT INTO patients (id, name_en, name_ar, phone, tenant_id) VALUES ($1, $2, $3, $4, 1)',
      [patient2Id, 'Acuity Patient B', '???? ?', '+966555555502']
    );

    // Insert admin user
    const hashedPassword = await bcrypt.hash(TEST_PASSWORD, 10);
    await client.query(
      'INSERT INTO system_users (id, username, password_hash, display_name, role, speciality, permission
s, is_active) VALUES ($1, $2, $3, $4, $5, $6, $7, 1)',
      [adminUserId, TEST_USERNAME, hashedPassword, 'Queue Admin', 'Admin', 'General', '["patients", "set
tings"]']
    );

    // Associate user with tenant 1
    await client.query('INSERT INTO user_tenants (user_id, tenant_id, is_active) VALUES ($1, 1, true)', [a
dminUserId]);

  } finally {
    client.release();
  }

  // Start server
  server = spawn('node', ['server.js'], {
    env: { ...process.env, PORT: TEST_PORT, SKIP_DB_INIT: '1' }
  });

  server.stderr.on('data', (data) => {
    console.error('SERVER ERR:', data.toString());
  });
}
```



```

});

// Wait for server to boot
await new Promise(resolve => setTimeout(resolve, 3000));

try {
  // 1. Login to get session
  const loginRes = await makeRequest('POST', '/api/auth/login', { username: TEST_USERNAME, password: TEST_PASSWORD });
  assert.strictEqual(loginRes.statusCode, 200, 'Login should succeed');
  const cookie = loginRes.headers['set-cookie'][0];

  // 2. Check-in Patient A (default triage 5)
  const checkinARes = await makeRequest('POST', '/api/queue/checkin', {
    patient_id: patient1Id,
    doctor: 'Dr. Sameer',
    department: 'General Medicine',
    triage_level: 5,
    exam_room_id: 'Room-101',
    acuity_notes: 'Cold symptoms'
  }, { Cookie: cookie });
  assert.strictEqual(checkinARes.statusCode, 201, 'Patient A check-in should succeed');
  assert.strictEqual(checkinARes.body.patient_name, '???? ?', 'Patient A name matches');

  // 3. Check-in Patient B (higher priority triage 2)
  const checkinBRes = await makeRequest('POST', '/api/queue/checkin', {
    patient_id: patient2Id,
    doctor: 'Dr. Sameer',
    department: 'General Medicine',
    triage_level: 2,
    exam_room_id: 'Room-102',
    acuity_notes: 'Chest pain'
  }, { Cookie: cookie });
  assert.strictEqual(checkinBRes.statusCode, 201, 'Patient B check-in should succeed');

  // 4. Retrieve queue and verify sorting (Patient B triage_level 2 must be before Patient A triage_level 5)
  const queueRes = await makeRequest('GET', '/api/queue/patients', null, { Cookie: cookie });
  assert.strictEqual(queueRes.statusCode, 200, 'Fetching queue should succeed');
  assert.ok(Array.isArray(queueRes.body), 'Queue response should be an array');

  // Find index of Patient A and Patient B in the retrieved array
  const indexA = queueRes.body.findIndex(p => p.patient_id === patient1Id);
  const indexB = queueRes.body.findIndex(p => p.patient_id === patient2Id);

  assert.ok(indexA !== -1, 'Patient A should be in the queue');
  assert.ok(indexB !== -1, 'Patient B should be in the queue');
  assert.ok(indexB < indexA, 'Patient B (ESI-2) must be prioritized before Patient A (ESI-5)');

  // 5. Test triage update API
  const triageRes = await makeRequest('PUT', `/api/queue/patients/${checkinARes.body.id}/triage`, {
    triage_level: 1,
    acuity_notes: 'Condition deteriorated - cardiac arrest'
  }, { Cookie: cookie });

```

```

    assert.strictEqual(triageRes.statusCode, 200, 'Updating triage should succeed');
    assert.strictEqual(triageRes.body.triage_level, 1, 'Triage level updated correctly');

    // 6. Test status update API
    const statusRes = await makeRequest('PUT', `/api/queue/patients/${checkinARes.body.id}/status`, {
      status: 'InConsultation'
    }, { Cookie: cookie });
    assert.strictEqual(statusRes.statusCode, 200, 'Updating status should succeed');
    assert.strictEqual(statusRes.body.status, 'InConsultation', 'Status updated correctly');

    // 7. Test patient calling API
    const callRes = await makeRequest('PUT', `/api/queue/patients/${checkinARes.body.id}/call`, null, { Cookie: cookie });
    assert.strictEqual(callRes.statusCode, 200, 'Calling patient should succeed');
    assert.strictEqual(callRes.body.success, true, 'Call broadcast successful');

    console.log('? ALL WAITING QUEUE ACUITY INTEGRATION TESTS PASSED!');
  } catch (e) {
    console.error('? TEST FAILED:', e.message);
    process.exitCode = 1;
  } finally {
    // Cleanup database
    const cleanupClient = await pool.connect();
    try {
      await cleanupClient.query("SET app.tenant_id = '1'");
      await cleanupClient.query('DELETE FROM waiting_queue WHERE patient_id IN ($1, $2)', [patient1Id, patient2Id]);
      await cleanupClient.query('DELETE FROM patients WHERE id IN ($1, $2)', [patient1Id, patient2Id]);
      await cleanupClient.query('DELETE FROM user_tenants WHERE user_id = $1', [adminUserId]);
      await cleanupClient.query('DELETE FROM system_users WHERE id = $1', [adminUserId]);
    } catch (e) {}
    cleanupClient.release();

    // Stop server
    if (server) {
      server.kill();
    }
  }
}

```

```
runTests();
```

```

=====
FILE: walkthrough.md
=====

```

```
# ????? ????? ?????: ??? ??????? ?????? ??????? ?????? (Wave 0 ? Walkthrough)
```

```

?? ?????? ?? `WAVE_0_FOUNDATION_IMPLEMENTATION_AUTOPILOT` ?????? ?????? ??????? ??? 48 ??????? ?????? ??? ??????
?? ????????? ?????? ?????? ?? ????????? ??????? ??????? ?????? ??? ??????? ?? ?????? ??????????.

```

```
---
```

```
* **??? ?????? ???????? ?????? (Master Catalog)**: 48 ?????.
```

```
* **??? ????? ???????? ????????*: 49 ??? ????? ????? (???? ????? 44 ?????? ?????? ???????? ??????).
```

```
* **???? ?????? ????????*: ??? ???? ??? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? UI/UX.
```

```
* **???? ????????*: ?????? ?????? ?? ??????: [departments](file:///c:/Users/ice/Desktop/NamaMedical/.ai-brain/departments/).
```

```
## 2. ???????? ???????? ??? Wave 0 ?? ??? AI Brain
```

```
?? ????? ?????? ?????????? ???????? ???????? ???????? ???????? (UTF-8):
1. **[??? ?????? ?????????? ???????? Wave 0 Plan](file:///c:/Users/ice/Desktop/NamaMedical/.ai-brain/hospital-wave-0-
foundation-plan-ar.md)**: ?????? ?????? ?????? ?????? ???????? ?????? RBac ?????????? ???????? ?????????? ???
???? ?????? ??????????.
2. **[????? ?????????? ???????????? ??????????](file:///c:/Users/ice/Desktop/NamaMedical/.ai-brain/hospital-final-audit-
report-ar.md)**: ?????? ??? ?????????????? ?????????? ?????? ?????????? ?????? ???????????.
3. **[????? ?? ?????? ?? ??? (No Missing Audit)](file:///c:/Users/ice/Desktop/NamaMedical/.ai-brain/hospital-n
o-missing-department-audit-ar.md)**: ?????? ?????????? ?????? ??? ?????????? ?? ??????????.
```

3. ??????? ??????? ??????? (Safety Guardrails)

```

* **Production Touched**: **NO** (?? ??? ??? ??? ??? ??? ??? ??? ??? ???).
* **DB Changed**: **NO** (?? ??? ? ??? ??? DDL ? ??? ??? ? ??? ??? ??? ???).
* **Secrets Exposed**: **NO** (??? ??? ??? ??? ? ??? ??? ??? ??? ??? ??? ???).
* **Git Tree Status**: Clean (??? ??? ??? ? ??? ??? ??? ??? ??? ???).
* **Arabic UTF-8 Audit**: **PASS** (??? ??? ??? ??? ??? ??? ??? Mojibake).

```

```
=====
FILE: wave0_doctor_station_ui_static_test.js
=====

const fs = require('fs');
const path = require('path');

const root = __dirname;
const doctorJs = fs.readFileSync(path.join(root, 'public/js/doctor-station.js'), 'utf8');
const css = fs.readFileSync(path.join(root, 'public/css/styles.css'), 'utf8');

const checks = [
  {
    name: 'Doctor Station keeps one active orders section',
    pass: doctorJs.includes('activeOrderSection') &&
      doctorJs.includes('window.dsApplyOrdersAccordion') &&
      doctorJs.includes("document.querySelectorAll('#dsOrdersContent .ds-order-section')"),
  },
  {
    name: 'Orders panel has consent section and stable data-section hooks',
  }
];
```

```

    pass: doctorJs.includes('data-section="diagnosis"') &&
    doctorJs.includes('data-section="consent"') &&
    doctorJs.includes('window.dsCreateConsentFromStation'),
  },
  {
    name: 'Doctor chart includes medical consents tab',
    pass: doctorJs.includes("id: 'consents'") &&
    doctorJs.includes("case 'consents':") &&
    doctorJs.includes('window.dsTabConsents(content)'),
  },
  {
    name: 'Consent workflow supports create, list, sign modal, and signature submit',
    pass: doctorJs.includes('window.dsTabConsents') &&
    doctorJs.includes('window.dsOpenConsentSignModal') &&
    doctorJs.includes('window.dsInitConsentSignatureCanvas') &&
    doctorJs.includes('window.dsSignConsentFromStation') &&
    doctorJs.includes('/api/consent-forms/templates/list') &&
    doctorJs.includes('/api/consent-forms'),
  },
  {
    name: 'Orders accordion and consent modal CSS are present',
    pass: css.includes('.ds-order-section.collapsed .ds-order-section-body') &&
    css.includes('.ds-consent-modal') &&
    css.includes('.ds-consent-card') &&
    css.includes('grid-template-columns: minmax(260px, 300px) minmax(0, 1fr) minmax(320px, 360px)'),
  },
];

```

```

const failed = checks.filter(c => !c.pass);
if (failed.length) {
  console.error('Doctor Station UI static checks failed:');
  failed.forEach(c => console.error(`- ${c.name}`));
  process.exit(1);
}

```

```

console.log(`Doctor Station UI static checks passed (${checks.length}/${checks.length}).`);

```

```

=====
FILE: wave0_remaining_workflows_static_test.js
=====

```

```

const fs = require('fs');
const path = require('path');

const root = __dirname;
const appJs = fs.readFileSync(path.join(root, 'public/js/app.js'), 'utf8');
const serverJs = fs.readFileSync(path.join(root, 'server.js'), 'utf8');

const patientAccountsBlock = appJs.slice(
  appJs.indexOf('async function renderPatientAccounts'),
  appJs.indexOf('window.loadPatientAccount')
);

```

```

const checks = [
  {
    name: 'Patient Accounts defines search before invoking it',
    pass: patientAccountsBlock.indexOf('window.searchPatientAccounts = async') > -1 &&
      patientAccountsBlock.indexOf('await window.searchPatientAccounts();') >
        patientAccountsBlock.indexOf('window.searchPatientAccounts = async') &&
        !patientAccountsBlock.includes('\n  searchPatientAccounts();'),
  },
  {
    name: 'Doctor wait queue uses current exam_rooms columns',
    pass: serverJs.includes('COALESCE(r.name_ar, r.name_en, r.room_number) as exam_room_name') &&
      serverJs.includes('COALESCE(r.name_ar, r.name_en, r.room_number) AS room_name') &&
      serverJs.includes('p.insurance_policy_number AS insurance_number'),
  },
  {
    name: 'Optional lab/RIS/pharmacy/HR/CSSD reads degrade to empty arrays',
    pass: [
      '/api/lab/samples',
      '/api/lab/qc',
      '/api/radiology/worklist',
      '/api/pharmacy/batches',
      '/api/hr/leave-requests',
      '/api/cssd/trays',
    ].every(route => {
      const i = serverJs.indexOf(`app.get('${route}')`);
      const block = serverJs.slice(i, i + 1200);
      return i >= 0 && block.includes("e.code === '42P01'") && block.includes('return res.json([])');
    }),
  },
  {
    name: 'Legacy internal messages without tenant_id can still load sent mail',
    pass: serverJs.includes("app.get('/api/users'") &&
      serverJs.includes('im.sender_id AS from_user_id') &&
      serverJs.includes('im.receiver_id AS to_user_id') &&
      serverJs.includes('req.body.receiver_id || req.body.to_user_id') &&
      serverJs.includes("app.get('/api/messages/sent'") &&
      serverJs.includes("if (e.code === '42703'") &&
      serverJs.includes('WHERE m.sender_id=$1 ORDER BY m.created_at DESC'),
  },
];

const failed = checks.filter(c => !c.pass);
if (failed.length) {
  console.error('Remaining workflow static checks failed:');
  failed.forEach(c => console.error(`- ${c.name}`));
  process.exit(1);
}

console.log(`Remaining workflow static checks passed (${checks.length}/${checks.length}).`);

```

=====

```
FILE: wave0_server_rbac_static_test.js
=====
```

```
const fs = require('fs');
const path = require('path');

const serverPath = path.join(__dirname, 'server.js');
const source = fs.readFileSync(serverPath, 'utf8');

function assert(condition, message) {
  if (!condition) {
    throw new Error(message);
  }
}

assert(/function normalizeRoleName\(role\)\/.test(source), 'normalizeRoleName helper is missing');
assert(/function canReviewOvr\(user\)\[s\S]*Quality Manager\[s\S]*Infection Control\[s\S]*Director\/.test(source), 'OVR reviewer role guard is incomplete');
assert(/function canViewAdminAuditTrail\(user\)\[s\S]*Admin\[s\S]*IT\[s\S]*Quality Manager\[s\S]*Director\/.test(source), 'Audit Trail role guard is incomplete');

assert(/app\.get\('\/api\/incidents\/ovr'\[s\S]*const isPrivileged = canReviewOvr\(user\)\/.test(source), 'OVR list route does not use canReviewOvr');
assert(/app\.put\('\/api\/incidents\/ovr\/:id'\[s\S]*?!canReviewOvr\(user\)\/.test(source), 'OVR update route does not use canReviewOvr');
assert(/const sacValue = sac_classification \|\| req\.body\.severity \|\| 'SAC4'\/.test(source), 'OVR post route does not preserve UI severity alias');

assert(/app\.get\('\/api\/admin\/audit-log'\[s\S]*?!canViewAdminAuditTrail\(user\)\/.test(source), 'Admin audit-log route does not use audit trail guard');
assert(/app\.get\('\/api\/admin\/audit-trail\/modules'\[s\S]*canViewAdminAuditTrail\(req\.session\.user\)\/.test(source), 'Audit Trail modules route is missing or unguarded');
assert(/app\.get\('\/api\/admin\/audit-trail'\[s\S]*?!canViewAdminAuditTrail\(req\.session\.user\)\/.test(source), 'Admin audit-trail route does not use audit trail guard');

console.log('wave0_server_rbac_static_test: PASS');
```

```
=====
FILE: wave0_sidebar_rbac_static_test.js
=====
```

```
const fs = require('fs');
const path = require('path');

const appJsPath = path.join(__dirname, 'public', 'js', 'app.js');
const source = fs.readFileSync(appJsPath, 'utf8');

function assert(condition, message) {
  if (!condition) {
    throw new Error(message);
  }
}
```

```

const navItemsMatch = source.match(/const NAV_ITEMS = \[([\\s\\S]*?)\\];/);
assert(navItemsMatch, 'NAV_ITEMS array was not found');

const navItemsBody = navItemsMatch[1];
const navItemCount = (navItemsBody.match(/\\s*icon:/g) || []).length;
assert(navItemCount === 48, `Expected 48 NAV_ITEMS entries, found ${navItemCount}`);

assert(/44:\\s*\\[\\s\\S]*?clinical\\.specialties\\.view\\[\\s\\S]*?\\]/.test(source), 'Specialties access rule is missing');
assert(/45:\\s*\\[\\s\\S]*?quality\\.ovr\\.view\\[\\s\\S]*?Quality Manager\\[\\s\\S]*?\\]/.test(source), 'OVR access rule is missing quality governance permissions');
assert(/46:\\s*\\[\\s\\S]*?audit\\.read\\[\\s\\S]*?IT\\[\\s\\S]*?\\]/.test(source), 'Audit Trail access rule is missing audit permissions');

assert(/function canAccessNavItem\\(index, options = \\{\\}\\)/.test(source), 'canAccessNavItem guard was not found');
assert(/function buildNav\\(\\)[\\s\\S]*?canAccessNavItem\\(i\\)/.test(source), 'buildNav does not use canAccessNavItem');
assert(/async function navigateTo\\(page\\)[\\s\\S]*?canAccessNavItem\\(page, \\{ allowHidden: true \\}\\)/.test(source), 'navigateTo does not guard direct navigation');
assert(/const isHiddenDirectAccess = item\\.hidden && options\\.allowHidden/.test(source), 'Hidden direct access handling is missing');
assert(/en:\\s*'Specialties'[\\s\\S]*?hidden:\\s*true/.test(source), 'Specialties must remain hidden from the sidebar');

console.log('wave0_sidebar_rbac_static_test: PASS');

```

```

=====

```

```

FILE: wave0_workflow_repairs_static_test.js

```

```

=====

```

```

const fs = require('fs');
const path = require('path');

const appJs = fs.readFileSync(path.join(__dirname, 'public', 'js', 'app.js'), 'utf8');
const ordersJs = fs.readFileSync(path.join(__dirname, 'orders.js'), 'utf8');

function assert(condition, message) {
  if (!condition) throw new Error(message);
}

assert(!appJs.includes('/ api / consent - forms'), 'Broken spaced consent API URL is still present');
assert(/async function renderConsentForms\\(el\\)[\\s\\S]*?\\/api\\/consent-forms\\/templates\\/list/.test(appJs), 'Consent page does not load real templates');
assert(/id="cfTemplate"[\\s\\S]*?loadConsentTemplate\\(\\)/.test(appJs), 'Consent template selector is missing');
assert(/id="cfSignSelect"[\\s\\S]*?loadConsentForSign\\(\\)/.test(appJs), 'Consent signing selector is missing');
assert(/API\\.get\\(\\`\\/api\\/consent-forms\\/\\$\\{fid}\\`\\)/.test(appJs), 'Consent load URL is not fixed');
assert(/API\\.put\\(\\`\\/api\\/consent-forms\\/\\$\\{fid}\\\\/sign`/.test(appJs), 'Consent sign URL is not fixed');

assert(/async function renderBloodBank\\(el\\)[\\s\\S]*?Promise\\.allSettled/.test(appJs), 'Blood Bank does not use resilient loading');

```

```

assert(/bbLoadErrors/.test(appJs), 'Blood Bank missing partial-load error notice');

assert(/async function renderPatientAccounts\(el\)\s*\{\s*const content = el;/.test(appJs), 'Patient Accounts
content binding is missing');

assert(/const orderTypeMeta = \{[\s\S]*?lab:[\s\S]*?rad:[\s\S]*?med:[\s\S]*?consult:/.test(appJs), 'CPOE order
type metadata is missing');
assert(/const statusRank = \{ active: 1, pending: 2, completed: 3, cancelled: 4 \}/.test(appJs), 'CPOE status
ordering is missing');
assert(/item_summary/.test(appJs), 'CPOE UI does not render item_summary');
assert(/Order placed using legacy workflow/.test(appJs), 'CPOE legacy workflow fallback is missing');

assert(/STRING_AGG\(NULLIF\(oi.catalog_ref, ''\), ', ' ORDER BY oi.id\)/.test(ordersJs), 'Orders API does no
t aggregate item_summary');
assert(/COUNT\(oi.id\), 0\)::int AS item_count/.test(ordersJs), 'Orders API does not expose item_count');
assert(/function isMissingRelationError\(e\)/.test(ordersJs), 'Orders API missing relation fallback guard');
assert(/async function listLegacyOrders/.test(ordersJs), 'Orders API missing legacy order fallback');

console.log('wave0_workflow_repairs_static_test: PASS');

```

```

=====
FILE: zatca_phase2.js
=====

```

```

/**
 * zatca_phase2.js ? ZATCA (Fatoora) Phase-2 cryptographic core + Fatoora API client (GATE-INT: ZATCA).
 *
 * Phase-1 artifacts (UBL 2.1 XML, TLV QR tags 1-5, document SHA-256) already live in ./finance_engine.
 * This module adds everything Phase-2 needs UP TO the credentialed network call, so the system becomes
 * "CSID-ready" ? the org plugs in its ZATCA-issued certificate (CSID) and private key and it goes live.
 *
 * Built WITHOUT any ZATCA credential (Node stdlib `crypto` only, no external deps):
 *   - secp256k1 EC key generation (the keypair the org owns; its public part is certified by ZATCA).
 *   - invoiceHash(canonicalXml) -> base64 SHA-256 (Phase-2 QR tag 6 + the PIH chain value).
 *   - signHashECDSA(hashB64, privPem) -> base64 ECDSA signature (QR tag 7).
 *   - buildPhase2QR(fields) -> base64 of TLV tags 1..9 (5 Phase-1 + hash/signature/publicKey/caStamp).
 *   - PIH chaining helpers (each invoice carries the previous invoice's hash -> tamper-evident chain).
 *   - generateCsrConfig(...) -> the exact ZATCA openssl CSR config (CSR creation uses openssl, since
 *     PKCS#10 with ZATCA's custom SAN cannot be produced by Node stdlib alone).
 *   - FatooraClient: compliance / production-CSID / clearance(standard) / reporting(simplified) calls,
 *     GATED ? it refuses to transmit unless a real CSID + base URL are configured.
 *
 * CREDENTIAL BOUNDARY (cannot be done without the org): the compliance/production CSID is issued by
 * ZATCA after submitting the CSR with a one-time OTP from the taxpayer's Fatoora portal. Everything in
 * this file works before that; only FatooraClient.* actually transmits, and only once configured.
 */
'use strict';
const crypto = require('crypto');

// ---- TLV (tag-length-value), bytes, ZATCA QR -----
// length is a single byte per ZATCA spec (values are short); value is UTF-8 (or raw bytes for crypto tags).
function tlv(tag, value) {

```



```

    const v = Buffer.isBuffer(value) ? value : Buffer.from(String(value == null ? '' : value), 'utf8');
    return Buffer.concat([Buffer.from([tag & 0xff]), Buffer.from([v.length & 0xff]), v]);
}

// ---- EC keypair (secp256k1) -----
// Returns PEM strings. The PUBLIC key (DER, base64) becomes QR tag 8; the PRIVATE key signs (tag 7).
function generateKeyPair() {
    const { publicKey, privateKey } = crypto.generateKeyPairSync('ec', {
        namedCurve: 'secp256k1',
        publicKeyEncoding: { type: 'spki', format: 'pem' },
        privateKeyEncoding: { type: 'pkcs8', format: 'pem' }
    });
    return { publicKeyPem: publicKey, privateKeyPem: privateKey };
}

// raw DER bytes of an SPKI public key PEM (ZATCA QR tag 8 carries the DER public key)
function publicKeyDer(publicKeyPem) {
    return crypto.createPublicKey(publicKeyPem).export({ type: 'spki', format: 'der' });
}

// ---- hashing + PIH chain -----
// invoiceHash: base64( SHA-256( canonical invoice XML ) ). Caller supplies already-canonicalized XML
// (Phase-2 mandates C14N over the UBL; finance_engine.buildUBLInvoice emits deterministic XML).
function invoiceHash(canonicalXml) {
    return crypto.createHash('sha256').update(Buffer.from(canonicalXml, 'utf8')).digest('base64');
}

// PIH for the very first invoice in a chain is the base64 of SHA-256("0") per ZATCA spec.
const GENESIS_PIH = crypto.createHash('sha256').update(Buffer.from('0', 'utf8')).digest('base64');
function nextPih(previousInvoiceHashB64) {
    return previousInvoiceHashB64 && previousInvoiceHashB64.length ? previousInvoiceHashB64 : GENESIS_PIH;
}

// ---- ECDSA signing / verification -----
// Signs the raw SHA-256 *digest bytes* of the invoice (ZATCA signs the invoice hash). Returns base64.
function signHashECDSA(invoiceHashB64, privateKeyPem) {
    const digest = Buffer.from(invoiceHashB64, 'base64'); // 32-byte SHA-256 digest
    const sign = crypto.createSign('SHA256'); // sign the digest representation
    sign.update(digest); sign.end();
    return sign.sign(privateKeyPem).toString('base64'); // DER ECDSA signature, base64
}

function verifyHashECDSA(invoiceHashB64, signatureB64, publicKeyPem) {
    const digest = Buffer.from(invoiceHashB64, 'base64');
    const verify = crypto.createVerify('SHA256');
    verify.update(digest); verify.end();
    return verify.verify(publicKeyPem, Buffer.from(signatureB64, 'base64'));
}

// ---- Phase-2 QR (9 TLV tags, base64) -----
// fields: { sellerName, sellerVat, timestamp, total, vat, invoiceHashB64, signatureB64,
//           publicKeyDerBuf, caSignatureB64? } (tag 9 CA signature optional for simplified invoices)
function buildPhase2QR(f) {
    const tags = [
        tlv(1, f.sellerName || ''),

```

```

        tlv(2, f.sellerVat || ''),
        tlv(3, f.timestamp || ''),
        tlv(4, f.total == null ? '0.00' : String(f.total)),
        tlv(5, f.vat == null ? '0.00' : String(f.vat)),
        tlv(6, f.invoiceHashB64 || ''),
        tlv(7, f.signatureB64 || ''),
        tlv(8, Buffer.isBuffer(f.publicKeyDerBuf) ? f.publicKeyDerBuf : Buffer.from(f.publicKeyDerBuf || '', '
utf8'))
    ];
    if (f.caSignatureB64) tags.push(tlv(9, Buffer.from(f.caSignatureB64, 'base64')));
    return Buffer.concat(tags).toString('base64');
}

// decode a base64 ZATCA QR back into { tag: value(Buffer) } ? used by tests + diagnostics.
function decodeQR(b64) {
    const buf = Buffer.from(b64, 'base64');
    const out = {};
    let i = 0;
    while (i + 2 <= buf.length) {
        const tag = buf[i], len = buf[i + 1];
        out[tag] = buf.subarray(i + 2, i + 2 + len);
        i += 2 + len;
    }
    return out;
}

// ---- CSR config (openssl drives the actual PKCS#10 with ZATCA's custom SAN) -----
// ZATCA requires a CSR whose subjectAltName/dirName carries the EGS unit identity. Node stdlib can't
// emit that, so we generate the canonical openssl config + the two commands the operator runs.
function generateCsrConfig({
    commonName, serialNumber, organizationIdentifier, organizationUnitName,
    organizationName, countryName = 'SA', invoiceType = '1100',
    location = '', industry = 'Medical', environment = 'sandbox'
} = {}) {
    // template.asn1 oid for ZATCA: 1.3.6.1.4.1.311.20.2 (Microsoft certificateTemplateName) per spec.
    const titleByEnv = { sandbox: 'TSTZATCA-Code-Signing', simulation: 'PREZATCA-Code-Signing', production: 'Z
ATCA-Code-Signing' };
    const config = [
        '[req]', 'default_bits = 2048', 'emailAddress = ', 'req_extensions = v3_req',
        'x509_extensions = v3_req', 'prompt = no', 'distinguished_name = dn', '',
        '[dn]', `CN = ${commonName || 'EGS-Unit'}`, '', '[v3_req]',
        `1.3.6.1.4.1.311.20.2 = ASN1:UTF8String:${titleByEnv[environment] || titleByEnv.sandbox}`,
        'subjectAltName = dirName:alt_names', '', '[alt_names]',
        `SN = ${serialNumber || ''}`, // EGS serial (1-Software|2-Model|3-SerialNo)
        `UID = ${organizationIdentifier || ''}`, // VAT / org identifier
        `title = ${invoiceType}`, // 1100 = standard+simplified
        `registeredAddress = ${location}`,
        `businessCategory = ${industry}`,
        `OU = ${organizationUnitName || ''}`,
        `O = ${organizationName || ''}`,
        `C = ${countryName}`
    ].join('\n');
    const commands = [
        'openssl ecparam -name secp256k1 -genkey -noout -out ec-private.pem',

```

```

        'openssl req -new -sha256 -key ec-private.pem -extensions v3_req -config csr.cnf -out generated.csr',
        '# then base64 the CSR and POST it to the compliance endpoint with the portal OTP'
    ];
    return { config, commands };
}

// ---- Fatoora API client (GATED ? never transmits without a configured CSID + base URL) -----
const FATOORA_BASE = {
    sandbox: 'https://gw-fatoora.zatca.gov.sa/e-invoicing/developer-portal',
    simulation: 'https://gw-fatoora.zatca.gov.sa/e-invoicing/simulation',
    production: 'https://gw-fatoora.zatca.gov.sa/e-invoicing/core'
};

class FatooraClient {
    constructor({ environment = 'sandbox', complianceCsid = null, complianceSecret = null,
        productionCsid = null, productionSecret = null, fetchImpl = null, enabled = false } = {}) {
        this.base = FATOORA_BASE[environment] || FATOORA_BASE.sandbox;
        this.environment = environment;
        this.complianceCsid = complianceCsid; this.complianceSecret = complianceSecret;
        this.productionCsid = productionCsid; this.productionSecret = productionSecret;
        this.enabled = enabled;
        this._fetch = fetchImpl || (typeof fetch === 'function' ? fetch : null);
    }

    _auth(csid, secret) { return 'Basic ' + Buffer.from(`${csid}:${secret}`).toString('base64'); }
    _assertReady(csid, secret) {
        if (!this.enabled) { const e = new Error('ZATCA disabled (set ZATCA_ENABLED=true to transmit)'); e.statusCode = 503; e.code = 'ZATCA_GATED'; throw e; }
        if (!csid || !secret) { const e = new Error('ZATCA CSID/secret not configured'); e.statusCode = 503; e.code = 'ZATCA_NO_CSID'; throw e; }
        if (!this._fetch) { const e = new Error('no fetch implementation available'); e.statusCode = 500; throw e; }
    }

    async _post(path, body, csid, secret, extraHeaders = {}) {
        this._assertReady(csid, secret);
        const res = await this._fetch(this.base + path, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'Accept-Version': 'V2', 'Authorization': this._auth(csid, secret), ...extraHeaders },
            body: JSON.stringify(body)
        });
        const text = await res.text();
        let json; try { json = JSON.parse(text); } catch { json = { raw: text }; }
        return { status: res.status, ok: res.ok, body: json };
    }

    // POST /compliance ? submit CSR (+OTP) to obtain the compliance CSID
    submitCsr(csrBase64, otp) {
        this._assertReady('csr', 'csr'); // only needs enabled + fetch; OTP/CSR are the creds here
        return this._fetch(this.base + '/compliance', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'Accept-Version': 'V2', 'OTP': otp },
            body: JSON.stringify({ csr: csrBase64 })
        }).then(async r => ({ status: r.status, ok: r.ok, body: await r.json().catch(() => ({})) }));
    }

    // clearance = standard (B2B) invoices ; reporting = simplified (B2C) invoices
    clearInvoice(payload) { return this._post('/invoices/clearance/single', payload, this.productionCsid, this

```

```
.productionSecret, { 'Clearance-Status': '1' }); }

    reportInvoice(payload) { return this._post('/invoices/reporting/single', payload, this.productionCsid, this.productionSecret, { 'Clearance-Status': '0' }); }

}

module.exports = {
    tlv, generateKeyPair, publicKeyDer,
    invoiceHash, GENESIS_PIH, nextPih,
    signHashECDSA, verifyHashECDSA,
    buildPhase2QR, decodeQR,
    generateCsrConfig, FatooraClient, FATOORA_BASE
};
```

```
=====
FILE: zatca_phase2_integration_test.js
=====
```

```
/**
 * zatca_phase2_integration_test.js
 * Integration tests for ZATCA Phase-2 cryptographic signing, QR codes generation,
 * and transmission logic via mock Express server.
 */
'use strict';
```

```
const Database = require('better-sqlite3');
const express = require('express');
const http = require('http');
const Z = require('./zatca_phase2');
const fe = require('./finance_engine');
```

```
let pass = 0, fail = 0;

function ok(name, cond) {
  if (cond) {
    pass++;
    console.log(` PASS: ${name}`);
  } else {
    fail++;
    console.error(` FAIL: ${name}`);
  }
}
```

```
// Mock pool wrapper similar to server.js
function buildMockPool(db) {
  return {
    query(sql, params = []) {
      let sqliteSql = sql;
      if (params.length > 0) {
        sqliteSql = sql.replace(/\$(\d+)/g, '?');
      }
      try {
        if (sql.trim().toLowerCase().startsWith('select')) {
          const stmt = db.prepare(sqliteSql);
```

```

        const rows = stmt.all(...params);
        return Promise.resolve({ rows });
    } else {
        const stmt = db.prepare(sqliteSql);
        const info = stmt.run(...params);
        return Promise.resolve({ rows: [], lastInsertRowid: info.lastInsertRowid, affectedRows: info.changes });
    }
} catch (err) {
    return Promise.reject(err);
}
};
}
}

```

```

async function req(port, method, path, body = null, headers = {}) {
    return new Promise((resolve) => {
        const options = {
            host: '127.0.0.1',
            port,
            method,
            path,
            headers: {
                'Content-Type': 'application/json',
                ...headers
            }
        };
        const r = http.request(options, (res) => {
            let resBody = '';
            res.on('data', d => resBody += d);
            res.on('end', () => {
                let json;
                try { json = JSON.parse(resBody); } catch { json = {}; }
                resolve({ status: res.statusCode, json });
            });
        });
        if (body) {
            r.write(JSON.stringify(body));
        }
        r.end();
    });
}
}

```

```

(async () => {
    console.log('Running ZATCA Phase-2 Integration Tests...');

    // 1. Setup in-memory SQLite DB
    const db = new Database(':memory:');

    db.exec(`
        CREATE TABLE IF NOT EXISTS integration_settings (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            tenant_id INTEGER,
            integration_name TEXT,

```

```

        provider TEXT,
        api_key TEXT,
        api_secret TEXT,
        endpoint_url TEXT,
        is_enabled INTEGER DEFAULT 0,
        config_json TEXT DEFAULT '{}',
        last_sync TEXT
    )
`);

```

```

db.exec(`
    CREATE TABLE IF NOT EXISTS zatca_invoices (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        invoice_id INTEGER,
        invoice_number TEXT DEFAULT '',
        invoice_type TEXT DEFAULT 'Standard',
        seller_name TEXT DEFAULT '',
        seller_vat TEXT DEFAULT '',
        buyer_name TEXT DEFAULT '',
        buyer_vat TEXT DEFAULT '',
        total_before_vat REAL DEFAULT 0,
        vat_amount REAL DEFAULT 0,
        total_with_vat REAL DEFAULT 0,
        qr_code TEXT DEFAULT '',
        qr_tlv TEXT DEFAULT '',
        ubl_xml TEXT DEFAULT '',
        xml_hash TEXT DEFAULT '',
        digital_stamp TEXT DEFAULT '',
        submission_status TEXT DEFAULT 'Pending',
        clearance_status TEXT DEFAULT 'NOT_SUBMITTED',
        submission_date TEXT DEFAULT '',
        zatca_response TEXT DEFAULT '',
        tenant_id INTEGER
    )
`);

```

```

db.exec(`
    CREATE TABLE IF NOT EXISTS invoices (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        invoice_number TEXT,
        total REAL,
        patient_name TEXT,
        tenant_id INTEGER
    )
`);

```

// Insert dummy invoice

```

db.prepare("INSERT INTO invoices (invoice_number, total, patient_name, tenant_id) VALUES (?, ?, ?, ?)").run('INV-00000001', 115.00, 'Ahmad', 1);

```

// 2. Setup Express application

```

const app = express();
app.use(express.json());

```

```

// Mock authentication and tenant context middlewares
const requireAuth = (req, res, next) => {
  req.session = { user: { id: 10, display_name: 'Test Admin', role: 'Admin' } };
  next();
};

const requireTenantScope = (req, res, next) => {
  req.tenantId = 1;
  next();
};

const e10RequireTenant = (req) => req.tenantId;
const e10IntId = (v) => parseInt(v, 10);
const e10Err = (res, e) => res.status(500).json({ error: e.message });
const logAudit = () => {};
const e10ZatcaEnabled = () => true;

const pool = buildMockPool(db);

// Generate Route (from server.js)
app.post('/api/zatca/generate', requireAuth, requireTenantScope, async (req, res) => {
  try {
    const tenantId = e10RequireTenant(req);
    const invoiceId = e10IntId(req.body.invoice_id);
    if (!invoiceId) return res.status(422).json({ error: 'Invalid invoice_id' });

    const inv = (await pool.query('SELECT * FROM invoices WHERE id=$1 AND tenant_id=$2', [invoiceId, tenantId])).rows[0];
    if (!inv) return res.status(404).json({ error: 'Invoice not found' });

    const vatBreak = fe.vatFromInclusive(inv.total);
    const issueDate = '2026-07-02';
    const issueTime = '12:00:00';
    const sellerName = 'Test Hospital';
    const sellerVat = '300000000000003';
    const invNumber = inv.invoice_number;

    const qrTlv = fe.buildZatcaQR({
      sellerName, sellerVat, timestamp: issueDate + 'T' + issueTime + 'Z',
      total: vatBreak.total_incl, vat: vatBreak.vat_amount
    });

    const ubl = fe.buildUBLInvoice({
      invoiceNumber: invNumber, issueDate, issueTime, invoiceTypeCode: '388',
      sellerName, sellerVat, buyerName: inv.patient_name,
      buyerVat: '', baseExcl: vatBreak.base_excl, vat: vatBreak.vat_amount, total: vatBreak.total_incl,
      currency: 'SAR', stampPlaceholder: 'UNSIGNED-NO-CSID'
    });
    const xmlHash = fe.ublHash(ubl);
    const stampPlaceholder = 'PLACEHOLDER:' + xmlHash.slice(0, 32);

    // Clean previous entry if exists to avoid UNIQUE constraint issue in sqlite test
    await pool.query('DELETE FROM zatca_invoices WHERE invoice_id=$1 AND tenant_id=$2', [invoiceId, tenantId]);
  } catch (e) {
    return e10Err(res, e);
  }
});

```

```

const result = await pool.query(
  `INSERT INTO zatca_invoices
    (invoice_id, invoice_number, seller_name, seller_vat, buyer_name, total_before_vat, vat_amo
unt, total_with_vat,
    qr_code, qr_tlv, ubl_xml, xml_hash, digital_stamp, submission_status, clearance_status, te
nant_id)
    VALUES ($1,$2,$3,$4,$5,$6,$7,$8,'',$9,$10,$11,$12,'Generated','NOT_SUBMITTED',$13)`,
  [invoiceId, invNumber, sellerName, sellerVat, inv.patient_name,
    vatBreak.base_excl, vatBreak.vat_amount, vatBreak.total_incl, qrTlv, ubl, xmlHash, stampPlaceh
older, tenantId]);

const inserted = (await pool.query('SELECT * FROM zatca_invoices WHERE invoice_id=$1 AND tenant_id
=$2', [invoiceId, tenantId])).rows[0];
res.json(inserted);
} catch (e) { e10Err(res, e); }
});

// Submit Route (from server.js, modified)
app.post('/api/zatca/submit', requireAuth, requireTenantScope, async (req, res) => {
  try {
    const tenantId = e10RequireTenant(req);
    const invoiceId = e10IntId(req.body.invoice_id);
    if (!invoiceId) return res.status(422).json({ error: 'Invalid invoice_id' });

    const z = (await pool.query('SELECT * FROM zatca_invoices WHERE invoice_id=$1 AND tenant_id=$2', [
invoiceId, tenantId])).rows[0];
    if (!z) return res.status(404).json({ error: 'E-invoice not generated yet' });

    const settings = (await pool.query('SELECT * FROM integration_settings WHERE tenant_id=$1 AND inte
gration_name=$2', [tenantId, 'ZATCA'])).rows[0];
    const isZatcaEnabled = e10ZatcaEnabled() && settings && settings.is_enabled === 1 && settings.api_
key && settings.api_secret;

    if (!isZatcaEnabled) {
      await pool.query(
        `UPDATE zatca_invoices
          SET clearance_status=$1, submission_status=$2, submission_date=$3, zatca_response=$4
          WHERE invoice_id=$5 AND tenant_id=$6`,
        ['RECORDED', 'Submitted_Mock', new Date().toISOString(), JSON.stringify({ message: 'ZATCA
clearance simulated or disabled' }), invoiceId, tenantId]
      );
      return res.json({
        success: true,
        clearance_status: 'RECORDED',
        submission_status: 'Submitted_Mock',
        message: 'ZATCA clearance simulated or disabled (sandbox/mock mode)'
      });
    }
  }

  const zatcaPhase2 = require('./zatca_phase2');
  let configJson = {};
  try {
    configJson = JSON.parse(settings.config_json || '{}');
  }

```



```

    } catch (_) {}

    const privKey = configJson.private_key_pem;
    const pubKey = configJson.public_key_pem;
    const environment = configJson.environment || 'sandbox';

    if (!privKey || !pubKey) {
        return res.status(422).json({ error: 'ZATCA private/public keys are missing in config_json' })
    }

    const xmlHash = zatcaPhase2.invoiceHash(z.ubl_xml);
    const signatureB64 = zatcaPhase2.signHashECDSA(xmlHash, privKey);
    const pubDer = zatcaPhase2.publicKeyDer(pubKey);

    const qr = zatcaPhase2.buildPhase2QR({
        sellerName: z.seller_name,
        sellerVat: z.seller_vat,
        timestamp: new Date(z.created_at || Date.now()).toISOString().slice(0, 19) + 'Z',
        total: z.total_with_vat,
        vat: z.vat_amount,
        invoiceHashB64: xmlHash,
        signatureB64,
        publicKeyDerBuf: pubDer
    });

    const payload = {
        invoiceHash: xmlHash,
        uuid: z.xml_hash ? z.xml_hash.slice(0, 36) : 'dummy-uuid',
        invoice: Buffer.from(z.ubl_xml, 'utf8').toString('base64')
    };

    // Mock fetch implementation for test
    const mockFetch = async (url, opts) => {
        return {
            ok: true,
            status: 200,
            text: async () => JSON.stringify({ clearanceStatus: 'CLEARED', warnings: [] })
        };
    };

    const client = new zatcaPhase2.FatooraClient({
        environment: environment,
        productionCsid: settings.api_key,
        productionSecret: settings.api_secret,
        enabled: true,
        fetchImpl: mockFetch
    });

    let result;
    const isSimplified = z.invoice_type && z.invoice_type.toLowerCase() === 'simplified';
    if (isSimplified) {
        result = await client.reportInvoice(payload);
    } else {

```

```

        result = await client.clearInvoice(payload);
    }

    const clearanceStatus = result.ok ? 'CLEARED' : 'FAILED';
    const submissionStatus = result.ok ? 'Reported' : 'Failed';
    const responseText = JSON.stringify(result.body);

    await pool.query(
        `UPDATE zatca_invoices
        SET qr_code=$1, xml_hash=$2, digital_stamp=$3, submission_status=$4, clearance_status=$5, submission_date=$6, zatca_response=$7
        WHERE invoice_id=$8 AND tenant_id=$9`,
        [qr, xmlHash, signatureB64, submissionStatus, clearanceStatus, new Date().toISOString(), responseText, invoiceId, tenantId]
    );

    res.json({
        success: result.ok,
        clearance_status: clearanceStatus,
        submission_status: submissionStatus,
        zatca_response: result.body
    });
} catch (e) { e10Err(res, e); }
});

// Start HTTP server
const srv = app.listen(0);
await new Promise(r => srv.once('listening', r));
const port = srv.address().port;

// Test 1: Generate E-Invoice XML
let resGen = await req(port, 'POST', '/api/zatca/generate', { invoice_id: 1 });
ok('E-invoice generation returns 200', resGen.status === 200);
ok('Generated UBL XML is not empty', resGen.json.ubl_xml && resGen.json.ubl_xml.length > 0);

// Test 2: Submit with disabled settings (Mock fallback)
let resSubmitMock = await req(port, 'POST', '/api/zatca/submit', { invoice_id: 1 });
ok('Submit returns 200 with mock fallback', resSubmitMock.status === 200);
ok('Clearance status is RECORDED', resSubmitMock.json.clearance_status === 'RECORDED');

// Test 3: Submit with active sandbox credentials (Real signing & crypto)
const keys = Z.generateKeyPair();
const config_json = JSON.stringify({
    private_key_pem: keys.privateKeyPem,
    public_key_pem: keys.publicKeyPem,
    environment: 'sandbox'
});

// Seed active ZATCA settings
db.prepare(`
    INSERT INTO integration_settings (tenant_id, integration_name, provider, api_key, api_secret, is_enabled, config_json)
    VALUES (?, ?, ?, ?, ?, ?, ?)
`).run(1, 'ZATCA', 'ZATCA Sandbox', 'sandbox-csid', 'sandbox-secret', 1, config_json);

```

```

let resSubmitReal = await req(port, 'POST', '/api/zatca/submit', { invoice_id: 1 });
ok('Submit returns 200 with active credentials', resSubmitReal.status === 200);
ok('Clearance status is CLEARED', resSubmitReal.json.clearance_status === 'CLEARED');

// Retrieve database record
const updatedInvoice = db.prepare("SELECT * FROM zatca_invoices WHERE invoice_id = 1").get();
ok('QR code generated and saved', updatedInvoice.qr_code.length > 50);
ok('Digital stamp (ECDSA signature) saved', updatedInvoice.digital_stamp.length > 20);

srv.close();
console.log(`ZATCA Phase-2 Integration Tests: ${pass} passed, ${fail} failed`);
process.exit(fail === 0 ? 0 : 1);
})();

```

```

=====
FILE: ZATCA_PHASE2_READINESS.md
=====

```

ZATCA Phase-2 ? credential-ready status

`zatca\_phase2.js` (+ 33/33 tests) completes the Phase-2 cryptographic pipeline **up to the credentialed network call**. Phase-1 artifacts (UBL XML, QR tags 1-5, doc hash) remain in `finance\_engine.js`.

Built now (no ZATCA credential required)

- secp256k1 EC keypair generation (`generateKeyPair`) ? the org owns this key.
- `invoiceHash` (base64 SHA-256 over canonical UBL) + PIH chain (`GENESIS\_PIH`, `nextPih`).
- `signHashECDSA` / `verifyHashECDSA` ? ECDSA over the invoice hash (QR tag 7), round-trip tested.
- `buildPhase2QR` ? full 9-tag TLV QR (1-5 + 6 hash, 7 signature, 8 public key DER, 9 CA stamp).
- `generateCsrConfig` ? exact ZATCA openssl CSR config (template OID 1.3.6.1.4.1.311.20.2 + SAN dirName) + the two openssl commands (Node stdlib cannot emit a PKCS#10 with ZATCA's custom SAN).
- `FatooraClient` ? compliance / clearance(standard) / reporting(simplified) calls, **GATED**: throws `ZATCA\_GATED` unless `enabled`, and `ZATCA\_NO\_CSID` unless a CSID+secret are configured. It transmits nothing by default.

Credential boundary ? needs the taxpayer's Fatoora portal (cannot be done for you)

1. Generate key + CSR with the emitted config:

```

`openssl ecparam -name secp256k1 -genkey -noout -out ec-private.pem`
`openssl req -new -sha256 -key ec-private.pem -extensions v3_req -config csr.cnf -out generated.csr`

```

2. In the org's Fatoora portal: get a one-time **OTP**, then `FatooraClient.submitCsr(base64(csr), otp)` ? receive the **compliance CSID**.

3. Run the compliance checks, then request the **production CSID**.

4. Set env: `ZATCA\_ENABLED=true`, `ZATCA\_ENV=production`, `ZATCA\_PROD\_CSID`, `ZATCA\_PROD\_SECRET`, and the private key path. Wire `clearInvoice`/`reportInvoice` into `/api/zatca/submit` (the route already records intent + fail-closes today).

Go-live wiring (one focused change, once CSID exists)

In `/api/zatca/submit`: build UBL ? `invoiceHash` ? `signHashECDSA` ? `buildPhase2QR` ? persist QR/hash ? `FatooraClient.clearInvoice|reportInvoice`. Store `clearance\_status` + the returned cleared XML/QR. Until then the module is INERT (imported by nothing), so it carries zero runtime risk.

NOT deployed yet

Module lives in the repo only. Deploy + wiring happen together once the org has (at least sandbox) CSID, so we can prove a real cleared invoice before flipping any flag on the live hospital.

```
=====
FILE: zatca_phase2_test.js
=====

/**
 * zatca_phase2_test.js ? unit tests for ./zatca_phase2 (Node stdlib crypto only, no network, no creds).
 * Run: node zatca_phase2_test.js    (exit 0 = all pass)
 */
'use strict';
const Z = require('./zatca_phase2');

let pass = 0, fail = 0;
function ok(name, cond) { if (cond) pass++; else { fail++; console.error(' FAIL:', name); } }

// ---- key generation (secp256k1) ----
const kp = Z.generateKeyPair();
ok('keypair has private PEM', /BEGIN PRIVATE KEY/.test(kp.privateKeyPem));
ok('keypair has public PEM', /BEGIN PUBLIC KEY/.test(kp.publicKeyPem));
const pubDer = Z.publicKeyDer(kp.publicKeyPem);
ok('public DER is a Buffer', Buffer.isBuffer(pubDer) && pubDer.length > 0);

// ---- invoice hash + PIH chain ----
const h1 = Z.invoiceHash('<Invoice>one</Invoice>');
const h1b = Z.invoiceHash('<Invoice>one</Invoice>');
const h2 = Z.invoiceHash('<Invoice>two</Invoice>');
ok('hash is deterministic', h1 === h1b);
ok('hash differs by content', h1 !== h2);
ok('hash is base64 32-byte', Buffer.from(h1, 'base64').length === 32);
ok('genesis PIH = sha256("0") b64', Z.GENESIS_PIH === require('crypto').createHash('sha256').update('0').digest('base64'));
ok('nextPih falls back to genesis when empty', Z.nextPih('') === Z.GENESIS_PIH);
ok('nextPih passes through a real prior hash', Z.nextPih(h1) === h1);

// ---- ECDSA sign / verify round-trip ----
const sig = Z.signHashECDSA(h1, kp.privateKeyPem);
ok('signature is base64 non-empty', typeof sig === 'string' && Buffer.from(sig, 'base64').length > 8);
ok('verify accepts a valid signature', Z.verifyHashECDSA(h1, sig, kp.publicKeyPem) === true);
ok('verify rejects a tampered hash', Z.verifyHashECDSA(h2, sig, kp.publicKeyPem) === false);
ok('verify rejects a wrong key', Z.verifyHashECDSA(h1, sig, Z.generateKeyPair().publicKeyPem) === false);

// ---- Phase-2 QR: 9 tags, round-trip decode ----
const qr = Z.buildPhase2QR({
  sellerName: 'Jumana Medical', sellerVat: '3000000000000003', timestamp: '2026-06-30T10:00:00Z',
  total: '115.00', vat: '15.00', invoiceHashB64: h1, signatureB64: sig, publicKeyDerBuf: pubDer,
  caSignatureB64: Buffer.from('ca-stamp-bytes').toString('base64')
});
const dec = Z.decodeQR(qr);
ok('QR is base64', Buffer.from(qr, 'base64').toString('base64') === qr);
ok('QR tag1 sellerName', dec[1].toString('utf8') === 'Jumana Medical');
```

```

ok('QR tag2 vat', dec[2].toString('utf8') === '3000000000000003');
ok('QR tag4 total', dec[4].toString('utf8') === '115.00');
ok('QR tag5 vat amount', dec[5].toString('utf8') === '15.00');
ok('QR tag6 = invoice hash', dec[6].toString('utf8') === h1);
ok('QR tag7 = signature', dec[7].toString('utf8') === sig);
ok('QR tag8 = public key DER bytes', Buffer.compare(dec[8], pubDer) === 0);
ok('QR tag9 present (CA stamp)', dec[9] && dec[9].length > 0);

// simplified invoice (no CA stamp) => tag 9 absent
const qrSimple = Z.buildPhase2QR({ sellerName: 'X', sellerVat: '3', timestamp: 't', total: '1', vat: '0', invoiceHashB64: h1, signatureB64: sig, publicKeyDerBuf: pubDer });
ok('simplified QR omits tag 9', Z.decodeQR(qrSimple)[9] === undefined);

// ---- CSR config ----
const csr = Z.generateCsrConfig({ commonName: 'EGS1', serialNumber: '1-Nama|2-ERP|3-001', organizationIdentifier: '3000000000000003', organizationName: 'Jumana', organizationUnitName: 'Riyadh', environment: 'sandbox' });
ok('csr config has req section', /\[req\]/.test(csr.config));
ok('csr config has ZATCA template OID', /1\3\6\1\4\1\311\20\2/.test(csr.config));
ok('csr config carries sandbox title', /TSTZATCA-Code-Signing/.test(csr.config));
ok('csr config carries UID (org id)', /UID = 3000000000000003/.test(csr.config));
ok('csr commands include openssl genkey', csr.commands.some(c => /ecparam -name secp256k1/.test(c)));

// ---- FatooraClient gating (no creds => never transmits) ----
(async () => {
  const c = new Z.FatooraClient({ environment: 'sandbox', enabled: false });
  let gated = false;
  try { await c.clearInvoice({}); } catch (e) { gated = e.code === 'ZATCA_GATED'; }
  ok('client refuses when disabled (ZATCA_GATED)', gated);

  const c2 = new Z.FatooraClient({ environment: 'production', enabled: true, fetchImpl: async () => ({ ok: true, status: 200, text: async () => '{}'} ) });
  let noCsid = false;
  try { await c2.reportInvoice({}); } catch (e) { noCsid = e.code === 'ZATCA_NO_CSID'; }
  ok('client refuses without CSID (ZATCA_NO_CSID)', noCsid);

  // configured => transmits via injected fetch
  let called = null;
  const c3 = new Z.FatooraClient({
    environment: 'production', enabled: true, productionCsid: 'cid', productionSecret: 'sec',
    fetchImpl: async (url, opts) => { called = { url, opts }; return { ok: true, status: 200, text: async () => '{"clearanceStatus":"CLEARED"}' }; }
  });
  const r = await c3.clearInvoice({ invoiceHash: 'x' });
  ok('configured client transmits', called && /invoices\/clearance\/single/.test(called.url));
  ok('configured client sends Basic auth', /^Basic /.test(called.opts.headers['Authorization']));
  ok('configured client parses response', r.ok && r.body.clearanceStatus === 'CLEARED');

  console.log(`zatca_phase2_test: ${pass} passed, ${fail} failed`);
  process.exit(fail === 0 ? 0 : 1);
})();

```

=====

FILE: _fix.js

=====

// Quick fix script - diagnose and fix PostgreSQL connection

const { Pool } = require('pg');

async function fix() {

console.log('=== Nama Medical - Quick Fix ===\n');

// Step 1: Try connecting to default 'postgres' database

console.log('1. Testing PostgreSQL connection...');

const pool = new Pool({

host: 'localhost', port: 5432,

database: 'postgres', user: 'postgres', password: 'postgres'

});

try {

const res = await pool.query('SELECT version()');

console.log(' ? PostgreSQL connected!');

console.log(' Version:', res.rows[0].version.split(',')[0]);

} catch (e) {

console.log(' ? Connection failed:', e.message);

if (e.message.includes('password')) {

console.log('\n ? Password issue. Trying without password...');

const pool2 = new Pool({ host: 'localhost', port: 5432, database: 'postgres', user: 'postgres', password: '' });

try {

await pool2.query('SELECT 1');

console.log(' ? Connected with empty password! Update your .env: DB_PASSWORD=');

} catch (e2) {

console.log(' ? Also failed. You may need to reset PostgreSQL password.');

console.log(' Fix: Edit pg_hba.conf to use "trust" auth, restart service, then set password.

');

}

await pool2.end();

}

await pool.end();

process.exit(1);

}

// Step 2: Check if database exists, create if not

console.log('\n2. Checking database "nama_medical_web"...');

try {

const dbCheck = await pool.query("SELECT 1 FROM pg_database WHERE datname='nama_medical_web'");

if (dbCheck.rows.length === 0) {

console.log(' ?? Database not found. Creating...');

await pool.query('CREATE DATABASE nama_medical_web');

console.log(' ? Database created!');

} else {

console.log(' ? Database exists!');

}

} catch (e) {

console.log(' ? Error:', e.message);

}

```

    await pool.end();

    // Step 3: Test the actual app connection
    console.log('\n3. Testing app database connection...');
    const appPool = new Pool({
      host: 'localhost', port: 5432,
      database: 'nama_medical_web', user: 'postgres', password: 'postgres'
    });
    try {
      await appPool.query('SELECT 1');
      console.log('    ? App database connection works!\n');
      console.log('=== Everything is ready! Run: node server.js ===');
    } catch (e) {
      console.log('    ? App connection failed:', e.message);
    }
    await appPool.end();
  }

  fix().catch(e => { console.error('Fatal:', e.message); process.exit(1); });

```

```

=====
FILE: .github\PULL_REQUEST_TEMPLATE.md
=====

```

<!-- NamaMedical PR checklist ? PHI/financial system: review carefully. -->

What & why

Checklist

- [] `node run\_safe\_tests.js` passes locally (DB-free unit tests)
- [] Integration suite run on an **isolated/staging** DB (never production) if DB-affecting
- [] No secrets/PHI in code, logs, or test fixtures
- [] Tenant-scoped: any new query respects RLS / `app.tenant\_id` (no cross-tenant leak)
- [] AuthZ added for new routes (`requireAuth` + role/permission)
- [] Input validated (`validation.js` / engine fail-closed) for new mutations
- [] DDL changes shipped as `migrations/\*\_{up,down,validate}.sql` (owner-run, not auto-applied)
- [] Money columns use `NUMERIC`, not floating `REAL`

Risk / rollback

```

=====
FILE: docs\BACKUP_RESTORE_DR_RUNBOOK_AR.md
=====

```

??? ???? ????????? ???? ????? ? ???? (Backup / Restore / DR Runbook)

????: NamaMedical / ????? ? ???? PostgreSQL.

— — —

```
- ????? ????????? ???? `POST /api/admin/backup` (Admin ???) ????? **???????? ?????????** (AES?256?GCM ??? gzip).
- **???? ?????? ????? ???? PHI ??? ?????**?: ??? ?? ?????? `BACKUP_ENCRYPTION_KEY` (?16 ??????) ??? `400 BACKUP_KEY_REQUIRED`.
- ?? ??????? shell ??? ????????? ?????? ?? ??????: `pg_dump` ??????? ??? `spawnSync` ?????? ?? ?????? (`DATABASE_URL` ?? `DB_*` ?? ????? ???? ?????? ??? `PGPASSWORD`).
- ?????? ?????? ??????: `nama_backup<ts>.sql.gz.enc`.
- ??? ???? ???? ??????: `[salt(16)][iv(12)][authTag(16)][ciphertext]? ???????? = `scrypt(BACKUP_ENCRYPTION_KEY, salt, 32)`.
```

```
| ??????? | ????? |
|---|---|
| `BACKUP_ENCRYPTION_KEY` | ????? ????? ?????? (16 ?????). **????? ?? ???? ????? ??????? ?? ?????.** ?????? =
????? ??????? ??? ??????????. |
| `DATABASE_URL` ?? (`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`) | ????? `pg_dump`. |
```

```
- ?? ??????? (Admin): ?? ????? ?????????? ? ?????? ??? `.sql.gz.enc` ?????? ?? `backups/` (?????? `gitignored`)  
.  
- ??????? ?????? ?? `audit_trail` ?? `DATABASE_BACKUP`.
```

> ?? ??????? ?????? ??????? ?????? ??????, ??????? ??? ??????/????? ??????? ??????? ?????????? ?????? ??? ?????????.

```

```bash
BACKUP_ENCRYPTION_KEY=*** node restore_backup.js backups/nama_backup-<ts>.sql.gz.enc /tmp/restore.sql
```
- ????? ???? ?? ??? ?????? ?? ? ???? ?? ??????? (GCM auth) ??????.

```

```

```bash
createdb nama_restore_test

??? ???? ???? ? ? PGPASSWORD ? ? ~/.pgpass ? ? ???? ? ? ????
PGPASSWORD="$DB_PASSWORD" psql -h HOST -p 5432 -U USER -d nama_restore_test -f /tmp/restore.sql
```

```

6. ??? ` /tmp/restore.sql` (??? PHI ??? ???) ??? ?????????.

```
- `backups/` ?????? ?? ???? ? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? (??? 30/90 ?????) ?? ???????.  
- ????? ??????? ????? **???? DR*: ???? ????? `enc` ??? ????? ?????/???? ????? (???? ????? ??????? ?? ????  
???? ???? ????? ??).  
- **????* ????? `BACKUP_ENCRYPTION_KEY` ?? ????? ????? (???? ????????? ?? ???).
```

- ????? ?????? ????????? ????? (??? ????? ??? ??????) ??? ????? ????????? ?????? RT0/RPO ??????????.

- ????? ?? ?? ????? ??????? ??????? ??? ???? ???? ????? (???? ?????? ??? ????? ?????????).

7. ?????? ????

- ????? ?????????? ????? ??? ?? ???? ?????? ?????? ?????? ?????? ?????? superuser ?? ???.
- ?? ?????? ?? ??????? ?? ????????? ?? ?? ??? ?????.
- ????? ??????? ?????? `sql` (?? ??????) ?????????? ?? `backups/` ??? **?????? ?? ?????** ? ??? PHI ??? ???.

=====
FILE: docs\PHASE1_AUTOPILOT_REMEDIATION_REPORT_AR.md
=====

????? ?????? ?????? ? ??????? ??????? ??????? ?????? (P0/P1)

?????: NamaMedical / ????? (HIS/EMR + ERP ???).
????: `audit/p0p1-remediation-autopilot` (????? ?? `319c4a5`).
?????: ??? ??? «???? ? ??? ? ????? ? ??????» ? ?????/???? ??? ??? ??? push? ??? ????? DB? ??? DDL ??? ?
?????
?????: **13 commit**? **92/92** ?????? ??? ???.

1. ?? ?????? ??? ?????? ??? (Code ? ???????)

| # | ?????? | ?????:????? | ?????? |
|---|---|---|---|
| 1 | **XSS ?????** ?? render ????????? ? ????? JSON ??? `</script>` | `server.js:5530` | ?????? ?????? ??? ?
??? ????????? |
| 2 | **eMAR ?????** ?? ??? ?????? ?????? ? ????? «?? ?????» ??? ?????????? ??? ?????? ?????? | `server.js:8490` |
e6 5-rights 43/43 |
| 3 | **CDS ??? ??? ?????** ?? ?????? ?????? ??? | `clinical\_cpoe.js:360` | CDS 9/41/60 + ?????? ???-2 |
| 4 | **????????? ?????? ?????? ack** (read-back) ??? ??????? | `server.js:2229` | e3 11/11 (+ ???? ack=0 ??????)
|
| 5 | **?????/????? ?????? ?????? = ???? ???** (?? ? ? ??????) | `server.js:1549/1567` | emr guard 10/10 |
| 6 | **????? ?????? ??? ???? ?????????** + ?????? IDOR ?????????? ????????? (portal) | `server.js:4546/11276/8193`
| e12 49+26? cross-tenant 100% |
| 7 | **??? ?????? ?????????** ?????????? + **rbac fail-closed** ?????????? | `server.js:9357`, `rbac.js:64` | rba
c 18/18? order guards 41/60 |
| 8 | **????? ??? ????** (regex ?????? + mimetype + ??? ???) | `server.js:43-52` | a3a 10/10 + 7 ?????? |
| 9 | **??? ??? ?????** (SELECT 1) + ?????? ???? `/api/health` ?? DEPLOY_RUN | `server.js:687` | syntax + ??? ?
????? |
| 10 | **??? ??????? ?????** AES-256-GCM (fail-closed) + `restore\_backup.js` + runbook DR | `server.js:12392`?
`restore\_backup.js` | round-trip + ??? ?????? ??? |
| 12 | **????????? ?????** ?????? (aria/landmarks/skip link/aria-expanded) | `public/index.html`, `app.js:213` |
parse + emr_ui 8/8 |

?????? ?????? ??? ?????:
- **????? ?????????** ?????? ???? `BACKUP\_ENCRYPTION\_KEY` (16) ??? ???? `400` (?? ???? PHI ??? ???). ??? `s
ql` ?????? ?? `backups/` ??? ???????/?????
- **????? ?????????? ?????????**: ?????? ??????? ??????? ??????? ??????? ??? ?? ?????? `ack` ? `consent_over
ride_reason` ??? ?????? (?????? ?????????).
- **????? ??? portal/consent**: ?????? ??? JOIN ??? `patients` (?????? ?????? ??? tenant_id).

```

????? migration ???? up/down/validate (idempotent? ?????? ?????? ?????????)? **?? ????????:
- `migrations/p1_01_legacy_core_rls_*` ? FORCE RLS + tenant_id NOT NULL + FK ??: `patients, invoices, appointments, medical_records` (???? ???? ?? ??????). `audit_trail` ?????? ????? (???? ????? `logAudit` ????? tenant_id ?????? ?????? ?? ?????? ?????).
- `migrations/p1_02_gl_posting_idempotency_*` ? `idempotency_key` + ???? ???? ???? ??? `finance_journal_entries` ???? ?????? ???????? ??? ?????? ???????? ??????????.

????????: ??? ?????? DEPLOY_RUN DDL (up ?? validate) ???????? ???????? ??? ??? `nama_medical_app` (????? ?????? ?? ???? ????/????? RLS).

```

```
- **????? ????? RLS ??? ???????** (????? ??? superuser/bypassrls) ? ????? ????? DB ??? (?????).
- **RLS ?? audit_trail** ? ????? ??????? ?? `logAudit` (??? ????????? + ????? ????? ??????).
- **???????? ?????? ?????** (NPHIES/ZATCA/HL7/DICOM) ? ????? ???/?/????? ?????? (XL).
- **CSP ?????? + ?????? `unsafe-inline`** *rate-limit ?????* **CI/CD + observability** ? ??? ???????.
- **a1ly ??? ??????? ??44** (????/?/????) ? ????? ??? ??????? ??? ??? ?????? ??????? ????.
- **GET /api/settings** ???? ??????? (?????? UI ??? ??? ?????/?/????)? **DELETE /api/patients/:id** Admin-only
y ?????? ? ?????? ?? ???.
```

```

...
BRANCH: audit/p0p1-remediation-autopilot (off 319c4a5)
COMMITTS: 13 | SAFE_TEST_SUITE: 92/92 PASS
PUSH_DONE: NO | GITLINK_TOUCHED: NO | DB_TOUCHED: NO | DDL_RUN: NO
PRODUCTION_TOUCHED: NO | DEPLOY_RUN: NO
SECRETS_PRINTED: NO | tracked-secret scanner: PASS
...

```

```
## 5. ?????? ??????? ?????????
1. ?????? ?????? ??? 13 commit ??? ????? (??? push).
2. ????? DDL: ????? `p1_01` `p1_02` (up?validate) ??? ??? ?????? ?? ??????? ????????.
3. ??? `BACKUP_ENCRYPTION_KEY` ??????/??? ??? `.sql` ????????.
4. ????? ?????? ??????? (?????????? observability? ally ??????) ?? ????? ??????.
```

```
# ????? ?????? ?? ??????? ????? B (?????)

**???? ???????**

- ?? ?????? ?? ?? ????? (npm run build:css, `node --check` ???????) ????? ?? ??????.
- ?? ?????? ??????? ?????? ?? ?????? ?? ?????? ?????? ?????? ?????? ?????? ???????.
- ?? ?????? ?????? ?????? ?????????? ?? ?????? ?????????? ?????????? ?????????? ?????? ?????? RTL ?????????? ??????? ??????.
- ?? ?????? ?? ?????? ?????? 500 ?? ?????? ?????? (?????? ?? ?? ?????? `date` ? `doctor` ?? ????? `appointments` ??
  ?????? ??????? PostgreSQL) ?? ?????? ?????? ?????? ?????? ?????????? ?? ?????? ?????? `server.js` ?? ?? ???????.
- ?? ??????? ?? ?? ?????? ??????? ??????? ?? ????????? (Console Errors: 0).
```

```
=====
FILE: docs\STITCH_BATCH_B_DB_SCOPE_VIOLATION_AR.md
=====

# ????? ?????? ?????? ??? ?????? ????????? ?? Batch B (?????)
(STITCH BATCH B DATABASE SCOPE VIOLATION REPORT - RESOLVED)

## ? ??? ???? ?????? ?? ????????? | Post-Remediation Summary
- **DB currently clean**: YES (?? - ????? ????????? ?????? ??????)
- **Temporary local DB touched then rolled back**: YES (?? - ?? ??????? ?????? ??????? ?? ????????)
- **Direct DB schema change retained**: NO (?? - ?? ?? ???? ??????? ?? ?????? ?? ???? ?????? ??????????)
- **????? ?????? ?????? ??????? ??????? (server.js updated for query compatibility)**: ?? (YES)
- **?????? ?????????? ?????? Batch B**: ?????? ??????? ????????? (BATCH_B_COMPLETED_BROWSER_VALIDATED)

---

## ?? ?? ?????????? ????? ?? ????????? | Executed Remediation Plan
????? ?????? ?????? ?????? ?????????? ??????? ?????? ?????????? ?? ?????? ?????????? ?????????:

### 1. ?????? ?????????? ??????? ?? `server.js`
?? ?????? ?????????? SQL ?????? ??? ???? ?? `date` ? `doctor` ?? ???? ?????????? ?????????? ?????????? ??????????
?? ?????? ?????? ??????????????:
- **?????? ??????????**: ??????? `NULLIF(appt_date, '')::DATE` ?????????? ?????????? ?? ?????????? ???? ??????.
- **??? ??????????**: ??????? `doctor_name as doctor` ?????????? ?????? ?????????? JSON ?????????? ?????????? ??? ?????? ????
?????.
- **??? ??????????**: ?????? ?????????? ??????? ?? ?????????? ?????????? (/api/appointments/check-duplicate) ?????? `appt_da
te` ? `doctor_name` ??????.

### 2. ?????????? ??????? ?? ?????????? ?????? ?????????? (Rollback SQL)
?? ?????? ?????????? ?????????? ?????????? ??????? ??????? ?????????? ?????????? ??????????:
```sql
ALTER TABLE appointments DROP COLUMN IF EXISTS date;
ALTER TABLE appointments DROP COLUMN IF EXISTS doctor;
DROP FUNCTION IF EXISTS text_to_date_immutable(text);
```

---

## ? ?????? ??????? ?? ?????????? | Validation Results
- ?? ?????? ?????? ?????? ??????????.
- ?????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????? ?????????? ?????? `200 OK` ??????
?? JSON ??????.
- ?? ?? ??????? ?? ???? ?????? ?????????? ??????? ?? ?? ?????? (Console Errors: 0).
- ???? ?????? ?????????? ???? ??????? ??????? ?????????? ?????????? ?????????? ?? `db_postgres.js`.

=====
FILE: docs\STITCH_BATCH_C_SUPPLY_CHAIN_REPORT_AR.md
```

=====

????? ????? ?????? ??? ????? ??????? ??????? (Batch C)

???? ??? ?????? ??????? ??????? ?????? ?????? ?????? ?????? ?????? ?????? ?? *????? ??????? ???????
? (Batch C)** ?? ????? *??? ????? (Nama Medical Web)** ????? ??? ?????? ?????? **Stitch Premium**.

1. ?????? ??????? ?????? ?????? (Scope of Redesign)

?? ?????? ?????? ?????? ??????? ?????? ?????? ?????? ?????? "?????" ??????? ?? ?????? ?????? ???
??? (SPA):

- * **????? ?????? (Inventory Items)**: ?????? ??????? ?????????? ?????????? ??????.
- * **????? ?????????? (Procurement / POs)**: ??? ?????? ??????? ??????? ?????? ?????? ??????.
- * **????? ?????????? (Suppliers)**: ?????? ?????????? ?????????? ?????????? ??????.
- * **????????? ?????????? ?????? ?????????? (Logistics)**: ?????? ??? ??????? ?????????? ?????? ??????.
- * **????? ?????????? ?????????? (AI Command Center)**: ??? ?????? ??????? ??????? ?????? ??????? ??????.

2. ??? ?????? ??????? ??????? (Stitch Visual Theme & Tokens)

- * **?????? ?????????**:
 - **????? ?????? ?????? (Primary Deep Navy)**: `#001629` - ?????? ?? ?????? ??????? ??????? ??????? ??????
 - **????? ?????? (Secondary Medical Teal)**: `#006970` - ?????? ?? ??????? ?????????? ?????????? ??????? ??????
 - **????? ?????? (Tertiary Gold)**: `#ac8c44` - ?????? ?? ??????? ??????? ??????? ??????.
- * **?????? ???????**:
 - ?????? ?????? *?????? ??????? (Glassmorphism)** ??? ?????? `.glass-card` (????? ?????? ?????? 80% ?????? ?????? `backdrop-blur` ?????? `12px` ?????? ??? ?????? ??????).
 - ??????? ?????? **Bento Grid** ?????? ?????????? ??????? (KPI Cards) ?? ??? ?????? ?????????? ?????? (Hover effects) ?????? ?????? ?????? ?????? ?????? ?????????.
 - * **????????**: ?????? ??????? ?????? RTL ??????? 100% ??? ?????? ??????? ??????? ?????????? ?????????.

3. ?????? ??????? ?????????? ?????? (Technical Implementation)

- * **????? ??????? (Inventory Management)**:
 - ??? ?????? ?? ?????? ?????? PostgreSQL ??????? ??? ?????????? API (`GET /api/inventory` ? `POST /api/inventory` ? `PUT /api/inventory/:id` ? `DELETE /api/inventory/:id`).
 - ?????? ?????? ?????? (Modal Form) ?????? ?????? ?????? ??? ??????? ?? ?????? ?????? ?????????.
 - ??? ?????? ??????? ??? ??????? ??? ?????? ??????? ??????? ?????? ?????? ?????? ??????.
- * **??? ?????? ??????? (Connected PO Workflow)**:
 - ??? ?????? ??? ?????? (PO) ?????? ??? ?????? ?????? ?? ?????? ??????.
 - ??? ?????? ??? ?????? "????? ??????"? ??? ?????? ?????????? ?????? ?????? ?????? ?? ?????? ?????????? ?????????? ?? ?????? `PUT /api/inventory/:id` ??????? ?????????? ?? ??????? ?????????? ?????? ?????? ??? `POST /api/inventory`.
- * **????? ?????????? ?????????? (Automated Replenishment)**:
 - ??? ?????? ?????? ??????? ?????? ?????? ??????? ??? ?????? ??????? ?? ?????? ??????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ??????.
 - ??? ?????????? ?????? ??? "????? ?????? ??????????" ?????? ??????? ?????? ??? ?????? ?????? ??????? ?????? ?????? ?????? ??????.

```
## 4. ?????? ????????? (Verification & Validation)
?? ?? ?????? ??????? ??????? ??????? ?? ??????? ??????? ?? ??????? ???????:
1. **????? ??????? (Syntax Validation)**:
  - ?? ?????? `node --check` ??? ??? ??????? ????????? ?????? ?????? ?????? ?????????? ??? ???? ?????? ?? ?????? ??
  ?????? ?? ??????????:
    - `public/js/app.js` (???? 100%)
    - `public/js/login.js` (???? 100%)
    - `public/js/admin.js` (???? 100%)
    - `public/js/api.js` (???? 100%)
    - `server.js` (???? 100%)
2. **??? ???? ?????? ??????? (Tailwind CDN Check)**:
  - ?? ??????? Tailwind CDN ??????? ??????? ?????????? ??????? ?? ??????? ??????? ??????? ??? `npm run build:cs
  s` ?????? `public/css/tailwind-compiled.css`.
3. **????????? ?????? ??????? (Browser Automated Testing)**:
  - ??????? ??? ?????? ?????? ?????? ??????? ?????? ??? ?????? ??? (`safe test credentials`).
  - ?????????? ??? ??????? ?????? ?????? ??????? ?????? (????????? ??????????? ?????????? ?????????? ?????
  ??????).
  - ?????? ?? ??? ?????? ?????? ?????????? ?????????? ?? ??? ??????.
  - ?????? ?? ??? ??????? ?????? ?? ???? ?????? ?? ??? ?????????? (Console Errors: 0).

---

## 5. ?????? ??????? ????????? (Security & Credentials Policy)
- ?? ?????????? ?????? ??? ?????? ?? ?????? ??? ?? ?????? ??? ?? ?????? ?????? ?????? (??? ?????? ?????????? ???????
) ??? ?? ?????????? ?? ??? ?????? ?????? ?????????? ???.
- ??? ?????? ?????? ?????????? ?? ?????????? ?? ?????? ?????????? ???.

---

## 6. ?????? ?????? ????????? (Data Mutation Audit)
* **????????? ???????/?????????**: ?? ?????? ?? ?????? ?????? `????? ?????? ?? 11` ??????? 150 ????????? 85.67 ??????
  ??????? `???? ?????? ??????? ???????` ?? ??? ????????? (`inventory`) ?? ?????? ?????? ?? ??????? `PO-2024-002`
  .
* **??? ??????????**: ?????? ?? ??????? ??????? ?????? (`test data`).
* **????? ??????? (Rollback)**: ?? ??? ?????????? ?? ??? ?????????? ?? ??? ?????? ?? ?????? ?????? ?????????? ???????
  ??????? ?????????? (`nama_medical_web`) ?????? ?? ??? ?????????? ?????? ?????????? ?????? ?? ??? ?????????? ?????????? ??
  ?????? ?????? ??????????.
* **?????????**: ??????? ?? ??????? ?? ?????? ?? ?? ?????? ?????????? ?? ?????? ?????????? ??????.

=====
FILE: docs\STITCH_BATCH_D_FINANCE_HR_COMPLIANCE_REPORT_AR.md
=====

# ?????? ?????? ?????? ??: ??????? ?????????? ??????? ?????????? (Batch D)
(Finance, HR / Payroll, Compliance / Audit Report)

???? ???? ?????????? ?????????? ??????? ??????? ?????? ?????????? ?? **?????? ?** ?? ?????? **??? ?????? (Nama Medical Web)*
* ?????? ??? ?????????? ?????????? **Stitch Premium RTL**? ?? ??? ?????????? ?????????? ?????????? ??? ???? ?????????? ???
?? ?????? ??????????.

---
```

2. **??? ????? ??? ?????????**:

```
?? ????? ??????? ??????? ?? JavaScript ????????:
```bash
node --check public/js/app.js public/js/login.js public/js/admin.js public/js/api.js server.js
```
?????? ???? ??????? ?????? ?????? ???? ?????? ?? ?????? ?? ????????.
```

```
### 2. ?????? ?? ??????? ???????? (Browser Validation)
?? ?????? ?????? ?????? ?????? ?? ?????? `3000` ???????? ??? ?????? ??????? ???????? ?????? ??????? ?????????:
1. **????? ???????*: ?? ?????? ?????? ?????? ?????? ?? ?????? ?????????.
2. **????????*: ?????? ?? ??? ?????? Bento ?????? ???????? ?????? ?????? ?? ?????? ?????? ????????? ??????????
?????????.
3. **??????? ???????*: ?????? ?? ??? ?????????? ?????? ?? ?????? ?????????? ?????? ?????????? ?????????? ??????
?.
4. **????????? ???????*: ?????? ?? ?????? ???????? ?????? ?????? (Checklist)? ?? ?? ?????? ?????? ?????? ?? ??????
????????? ?????? ?????? ?? ?????? ?????? ???????? ?????????? ?????????? ?????????? ?? 40% ?????? ??????. ?? ??
????? ?? ?? ??
?? ?? ?????? ??????.
5. **????? ?????????*: ?? ?? ??? ?????????? (Console Logs) ?? ?????? ???????? ?? ?? ??? ?? ?? ?????? ?? ???
? ?????? ?????? ????????? (Console Errors: 0).

---
```

```
## 3. ????? ??????????? ??? ?????? (Sanitization Audit)

?? ????? ????????? ?? ?? ?? ???? ???? ???? ???? :
- **????? ?????***: ? ???? ????????? ???? ???? ???? ???? ???? ???? (?? ????????? ?? `C:\Users`).
```

```
* **???? ?????? ?????? ?** : **????? ???????? ??????? ????? 100%**
* **??????? ?????????? ???????? ???** : **BATCH_D_COMPLETED_CLEAN**

=====
FILE: docs\STITCH_BATCH_E_GOVERNANCE_FACILITY_ANALYTICS_REPORT_AR.md
=====
```

```
# ????? ????? ?????? ?? (????? ??????????? ?????????? ??????????) - Stitch RTL

???? ??? ??????? ????????? ?????? ??????????? ?????? ?? (Batch E) ?? ????? ?????? ?????? ?????? **??? ????? (Nama
Medical Web)** ?????????? ??? ????????? ?????????? **Stitch Premium RTL** ?????? ????????? ?????? ?? ????????? ?????????
?????????.
```

```
## 1. ?????? ?????????? ??????????

### ?. ??????? ?????? ?????????? (Cybersecurity & Governance)

?? ??? ?? ?????? ?????????? ??? ??? ?????????? ?????????? (`renderSettings`) ?? ??????? ??????? ?????? ?????? ??????
```

? ?????? ???????:

* **???? ?????? (Threat Level Gauge)**: ?????? ?????? ?????? ?????? ?????? SVG ?????? ?????? ?????? ?
????? (30% Low / Secure) ?? ?????? ?????? ?????? ?????? ?????? ?????? ??????.

* **???? ?????? (Data Encryption Status)**: ??? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?
????? (AES-256 ? RSA-4096) ?????? ?????? ?????? (SSL/TLS 1.3).

* **????????? ?????? ?????? (National Authority Compliance)**: ?????? ?????? ?????? ?? ?????? ?????? ?????? ?
????? ?????? ?????? (SDAIA)? ?????? ?????? ?????? (PDPL)? ?????? ?????? ?????? ?????? ??????

.

* **????? ?????? ?????? (Database Backup Infrastructure)**: ??? ??? ?????? ?????? ?????? (`nama_me
dical_web`) ?????? ?????? ?? ?? ??? ?????? ??? pg_dump ?????? ?????? ?????? ?????? ?????? ??????

?? ?????? `.sql` ?? `.bak`.

* **??? ?????? ?????? ?????? (Security Audit Trail)**: ??? ?????? ??? ??? ?????? ?????? ?????? ?
? ?????? (??? ?????? ?????? ?????? ?????? ?????? IP ?????? ??????) ??? ?????? ?????? ?? ??? `audit\_trail`
` ?????? ??????.

?. ?????? ?????? ?????? ?????? (Biomedical & Facility Maintenance)

?? ?????? ?????? ?????? ?????? ?????? ?????? ?????? (`renderMaintenance`) ?????? ??? ?????? ?????? ??????:

1. **????? ?????? (Maintenance Dashboard)**:

* ?????? Bento Grid ??? ?????? ????: ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?
????? ?????? ?????? ??????.

* ?????? ?????? (Preventive Maintenance Calendar) ?????? ?????? ??? CSS ?????? ??? ?????? ??????

? ??????.

* ??? ???? ?????? ?????? ?????? ??? ?????? ?????? ?????? ?? ?????? ?????? (CBAHI).

2. **????? ?????? ?????? (Work Orders & Tickets)**:

* ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?? ?????? ?? ?????? `maintenance`
_orders` ? `maintenance_work_orders`.

* ?????? ?????? ?????? ?????? ?? ?? ?????? ?????? ?????? ?????? ?????? ?????? ??????.

3. **??? ?????? ?????? (Medical Assets Registry)**:

* ?????? ?????? ?????? ??? ?????? ?????? ?????? ?????? ??????.

* ??? ???? ?????? ?????? ?????? ?? ?????? `maintenance\_equipment` ?? ?????? ?????? ?? ?????? ??????

?? ??? ????.

?. ?????? ?????? ?????? (Executive Analytics)

????? ??? ?????? ?????? ?????? (`renderDashboard`) ?????? ??? ?????? ?????? ?????? (Command Center):

* **????? ?????? 2030**?: ??? ??? ?????? ?????? ?????? (94%)? ?????? ?????? (82%)? ??? ?
????? (89%) ?? ?????? ??? ???.

* **????? ?????? (Departmental Performance)**: ?????? ?????? ??? ?????? (??? ?????? ?????? ?
? ??????)? ?????? (????? ?????? ?????? ??????)? ?????? ?????? (??? ?????? ?????? ??????)?
?????.

* **????? ?????? (Chart.js charts)**:

* ??? ?????? ??? ?????? ?????? ?????? ??????.

* ??? ?????? (Pie & Doughnut charts) ?????? ??? ?????? ?????? ?????? ??? ???? ??????

?????.

2. ?????? ??????

?? ?????? ?????? 100%:

1. **????? (Tailwind build)**: ?? ??? ?????? ?????? ??? ???? CDN ??????.

2. **??? (Syntax Node Check)**: ?? ?????? ??? ?????? ?????? ?????? ?????? ?????? ?
???:

- `node --check public/js/app.js` (???)

- `node --check server.js` (???)

3. ****??? ?????? ??????? (Browser Verification)**:**

- ?? ?????? ?? ??? ?????? ?????? ??????? ??????? ?????????? ??????????.
- ?? ?? ??????? ?????????? ?????????? ?????? ?????????? ?? ??????? ??????? (stats SQL cast error) ??? ?? ?????? ?? ?????? ?? `next\_due` ?????? ?????? ?????????? ?? PostgreSQL ?? ?????? ?????????? ??????? (`next\_due < CURRENT\_DATE::text`).
- ??? ?????? ?????????? ??????? ?? ?? ?????? ?????? ??????? ?? ?????? ??????.

3. **????? ?????? ?????????? ??????? ???????**

*** **????????? ?????????? (Data Mutations)**:** ?? ??? ?????? ?? ??? ?? ?????? ??????? ??? ?????? ?? ?????? ?????????? ?????? ?????? ?????????? ?????? ?????????? ?????????? ?????????? ?????????? ?????? ?????? ??????.

*** **?????? ?????????? ?????????? (Sanitization)**:** ?????? ?????????? ?????????? ?????????? ?? ?? ?????????? ?????? ?????????? ?????? ?????? ?????????? ?? ?????????? ?????????? ??????????.

*** **????????? ??????????**: **BATCH_E_COMPLETED_CLEAN****

=====

FILE: docs\STITCH_DESIGN_IMPLEMENTATION_REPORT_AR.md

=====

????? ?????? ??? ?????????? ?????????? ??? ?????? (Stitch RTL)

????? ?? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????? ?? ****Stitch**** ?????????? ?????? ?? ?????????? ?? ?????????? ?? ?????????? (RTL) ?? ?????? ****??? ?????? (Nama Medical Web)****. ?? ?????????? ??? ?????? ?????????? ?? ?????????? ?????????? ?????????? ?????????? ?????????? ?????? ?? ?????? ?????????? ?????????? (Backend) ??? ?? ?????? ?? ?????? ??? ?????? ?? ?????????? ?????????? ?????????? ?????? C++/Desktop.

1. **????? ?????? ?????????? (Design System Tokens)**

?? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?? ?????? ?????? Stitch ?????????? ?????????? CSS ?????? ? ? ??? [styles.css](namaweb/public/css/styles.css) ?????? ?????? ?????????? ??????:

```
```css
:root {
 /* ?????? ?????????? ?????????? */
 --color-primary: #001629; /* ?????? ?????? (Deep Navy) */
 --color-primary-container: #002b49;
 --color-secondary: #006970; /* ?????? ??? (Medical Teal) */
 --color-secondary-container: #7af1fc;
 --color-tertiary: #ac8c44; /* ?????? ?????? (Gold) */

 /* ?????? ?????????? ?????????? */
 --color-background: #f8f9fa; /* ?????? ?????? ?????? */
 --color-surface-lowest: #ffffff; /* ??? ?????? ?????? */
 --color-surface-low: #f3f4f5;
 --color-surface: #edeeef;
 --color-surface-high: #e7e8e9;
```

```
/* ?????? ??????? */
--color-text-primary: #191c1d;
--color-text-muted: #42474d;
--color-outline: #73777e;
--color-outline-variant: #c3c7ce;
--color-error: #ba1a1a;
}
...

* **????? ??????????*: ?? **IBM Plex Sans Arabic** ?????? ???? ?????? ?????? ??????????.
* **????? ??????????*: ?????? ?????? ?????? ???????? (Glassmorphism) ?????? ?? ???? ?????????? ?????? ?????? ??? ?????? ?
??? ???????? (Hover micro-animations) ??? ?????????? ?????????? ???????? ?????? ?????? ?????????? ?????????? ??????????????.

2. ?????? ?????? ???? ??????? ??????? ??????? (`login.html`)

??? ?????? ?????? ???? ?????? ??????? ????????? ??????? ?????? `saudihealth_premium_landing_page_rtl`:

* **????????? ??????????*: ?? ??? ?????? ?????????? ??????? (????????? ?????? ?????????? ?????????? ?????? ?????????? "????????? ???????
? ?? ?????????? ??????????"? ?????????? ?????????? ??????????).
* **??????? ?????? ??????????*: ?? ?????? ?????? ?????? ?????? ?????????? ??? **?????? ??????? ?????????? (Modal)** ?????? ??? ??
??? ??? ?? "????? ?????????? / Portal Login".
* **????????? ??????????*: ?? ??????? ??? ?????????? ??????? (IDs) ?????????? (`username`, `password`, `loginBtn`, `login
Error`) ??? ??? ??? ??? ?????????? ??? ?????? ?????????? ?????????? ?? `login.js` ?????? ??? ?????? ?????????? ??????????? ?? ?????
???.

3. ?????? ?????? ???? ???? ??????? ??????? ?????? ?????? ??????????

?? ?????? ?????????? ??????? ??? ?????????? ?????????? ?????????? ?? ?????? `public/index.html` ? `public/js/app.js`:

?. ?????? ?????? ?????????? (App Shell)
* **????????? ?????????? (Sidebar)**: ?? ?????????? ?????????? ?????????? ?????????? ??????? ?????? ?????? ??????? ??????? ??????? ??
????? (`#001629`) ??? ?? ??????? ??????? (????? ??? ?????????? RTL)? ?????? ??????????? ?????????? ?????????? ?????????? ??????????
?? ?????????? ?????? ?????? ?????????? ?????????? ?????????? ??????????.
* **????????? ??????? (Topbar)**: ??? ?????????? ?????????? (????????? ?????? ?????????? ?????? ??????????) ?? ?????? ?????? ?????? (??
??/? ????????) ?????? ?????????????? ?? ?????? ??????? (RTL/LTR).

?. ??? ???? ???? ?????????? ?????????????????? (Dashboard & Commands)
?? ??? ??????? `saudihealth_premium_hospital_operations_dashboard_rtl` ? `saudihealth_premium_strategic_executi
ve_command_center_rtl` ??????? ???? ???? bento-grid ?????? ?????? ???:
1. **????????? ??????? ??????? ?????? (KPI Cards)**:
 * **?????? ?????? ??????? (Bed Occupancy)**: ?????? ?????????? ?? ??????? ?? ?????? ??? ??????.
 * **????????? ??? ?????????? (Average Wait Time)**: ?????? ?????? ??????.
 * **?????? ?????????? ?????????? (Compliance Index)**: ?????? 98.5% ?? ?????????? ?????? ?? ??????? ??????? ?????? (CHI)
 ?????? ?????? (MOH).
 * **????????? ?????????? ?????????? (Emergency Cases)**: ?????? ?????? ?????????? ????????.
2. **????????? ?????????? ?????????????? (Chart.js)**:
 * ??? ?????? ??? ?????????? ?????? ?????????????? ?????????? ?????? ?????????? ??????????????.
 * ??????? ?????? ?????? ??????? ??????? ??????? ?????????? ?????????? ??????? ?????? ??????.
 * **????????? ??????*: ?? ?? ?????? ?????? ?????? ?????????? ??? ??????? ??? ?????????? ?? ?????? ?????? ?????? ?????? ?????? ?
```

????????? ??????? `Chart.getChart(canvasId)?.destroy()` ??? ????? ??????.

3. **\*\*\*?????? ??????? (Compliance Alerts Feed)\*\*:** ??? ??????? ??? ?????? ?????? ??????? ?????????? ??????? ?  
????? ?????????? ???????????.

4. **\*\*\*????? ?????? ?????? ?????? 2030\*\*:** ????? ??? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ??????  
?????.

---

## 4. ?????? ?????????? ?????? ?????? (Validation & Production Hardening)

?? ?????? ?????? ??? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ??????  
??? ?????????? (CDNs):

### ?. ?????? ?????? ??????????? ?????????? (External CDNs)

?? ??? ?????? ??? ?????? ?????????? ?? ?????? ?????????:

1. **\*\*\*????? ?? ??????? (TAILWIND\_CDN\_REMOVE)\*\*:**

- ?? ?????????? ?? ????? CDN ?????? ?? Tailwind CSS (`https://cdn.tailwindcss.com`) ?? ????? `index.html` ? `login.html`.
- ?? ?????? ??? Tailwind CSS ?????? ?????????? ?????? (`devDependencies`) ?????? ??? ?????????? ?????????? ?????????? ?  
????????? ?????? ?? ?????? `/css/tailwind-compiled.css` ????? 33.7 ?????????? ???.

2. **\*\*\*????? ?? ??????? ?????? ?????? ?????????? ?????????? (ICON\_FONT & GOOGLE\_FONT)\*\*:**

- ?? **\*\*IBM Plex Sans Arabic\*\*** ??? Google Fonts ?????? ??? ?????? ??????.
- ??????? **\*\*Material Symbols Outlined\*\*** ??? ?????????? ?????????? ?? ?????? ?????????? ?????? ??????.

3. **\*\*\*?????? ??????? ?????????? (REQUIRED\_EXISTING\_LIBRARY)\*\*:**

- ?????? **\*\*Chart.js\*\*** ?????? ?????????? ??????????.
- ?????? **\*\*JsBarcode\*\*** ?????? ?????? ?????? ?????????? ??????.
- ?????? **\*\*Flatpickr\*\*** ??????????? ?????????? ?? ?????????? ?????????? ?????????? ?????????? ??????????????.

### ?. ?????? ?????? ?????????? (Validation Commands)

1. **\*\*\*??? ?????? ??? ?????????? (Syntax Checks)\*\*:**

- ?? ?????? ?? ?????? ??? ?????? ??? ?????? ?????? ?????? ?????? ?????? ?????? ?????? `node --check` ??? ?????? ??  
??? ??? ?? ?????? ?? ?????????? ?????????:
- `public/js/app.js` (???? 100%)
- `public/js/login.js` (???? 100%)
- `public/js/admin.js` (???? 100%)
- `public/js/api.js` (???? 100%)
- `server.js` (???? 100%)

2. **\*\*\*????? ?? ?????????? ?????? ?????? (Scripts Check)\*\*:**

- ?? ?????? ?? ??? ??? ?????????? ?????? ?????? (`npm test`) ?? ??? ??? (`npm run lint`) ?????? ?? `package.j  
son` ?????????? ??? ?????????? ?????? ?????????? ?????? ?????????? ??????.

### ?. ??? ??????? ?????? ?????????? (Browser Verification)

?? ?????? ??? ?????? ?????? ??? ?????? `3000` ?????????? ??? ??? ?????? ?????? ?????? ?????? ?????? ??????:

1. ??? ?????? ?????? (`login.html`) ?????????? ?? ??? ?????????? ?????????? (Modal) ??????.

2. ?????? ?????? ?????? ?????????? ?????? ?????? ?? ??? ?????? ?????????? ?????????? ?????????? ?????? ?????? ??? ?? ?  
???

3. ?????? ?? ?????? ?????????? ?????????? ??? ?????? CDN ?????????? ?????? ?? ?????????? ?????????? ?????? Stitch.

4. ?????? ?????????? ?????? (????/????????)? ??? ?? ?????? ?? ?????? ?????? ?????? ?????????? ??? ?????? ?????? ??????  
????????? ??? LTR ??? ?????? ?????? ?????????????? ?????????? ?????? RTL ??? ?????? ?????????? ??????.

5. ?????? ?? ??? ?????? ?????? ?????? ?? ?? ?????? ?????? ?? ??? ?????????? (Console log errors: 0).

---

## 5. ??? ?????? ?????????? ?????????? ?????????? (Production Readiness & Risks)

```
- ?? ??? ?????? ??? ???????? ???????? ???????? ???????? ???????? ???????? ????????
- ?? ??????? ?????? ?? ?????? ?????? ???????? ???????? ???????? ?????? ?????????? ?? `safe test credentials`.
- ?? ?? ??? ?????? ???????? ???????? ?????? ?????? ???????? ???????? ???????? ?????? ?????? ??? ?????? ?????????? (?
?? `file:///C:/Users/...`) ?? ??? ?????? ????????
- ?? ?????? ?? ?? ?????? ?? ???????? ?????? `env`.
```

```
- ?? ?????? ?? ??? ????? HTML ?????? ?? ????? Tailwind CDN.
- ????????????? ?????????? ?????????? ?????????? ??? ?????????? ?????????? ???
 - ?? **IBM Plex Sans Arabic** ?????????? **Material Symbols Outlined** ??? ?????? Google ?????? ?????? ?????? ??
????? ???????????.
 - ?????? **Chart.js** ?????? ?????????? **JsBarcode** ?????? ?????????? **Flatpickr** ?????????? ?????? ??????.
```

```
- ?? ?????? ?? ???? ???? ??? `tailwind-compiled.css` ?????? ?????? ???? (33.7 ?????????).
- ?? ??? ?????? ?????? ???? `index.html` ? `login.html`.
- ?? ??????? ?? ??? ?????? ??? ?????????? ?????????? ?? ?????? ????????? ?????????? ?? ?????? ?????????? `gitignore`.
```

```
- ????????? ?????? ?????? ??? ?????? ?????? ?????????? ?????????? ?????? ????????? ?? ????? `namaweb`:
- ????? `package.json` ? `package-lock.json` ?????? ?????? ?????????? ??????????.
- ????? `public/index.html` ? `public/login.html` ? `public/css/styles.css` ? `public/js/app.js` ?????? ??????
?? ????????? ?????????? ??????????.
- ?????? ?????? ?????????? ?????????? `tailwind.config.js` ? `tailwind-input.css` ???? ?????????? ?????????? `public/css/tailwind-compiled.css`.
- ?? ?? ???? ?? ?????? ??? ??? ?????????? ?????????? (Backend business logic) ?? ?????? ??????????.
```

```
?? ????? ????? ????? Stitch Premium RTL ??? ??????? ?????? ????????? ????????:
* **???????? (Laboratory)**: ????? ?????? ????? ??? ?????? ?????????? ?????? ????????? ?? ?????? ??? ???? ???? ???? ????
? ????????? ?????? ?????? ?????????.
* **??????? (Radiology)**: ????? ?????? ?????? ??? ?????? ??? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????
* **????????? (Pharmacy)**: ????? ??? ???? ????????? ??????? ????????? ?? ?????? ?????? ?????????? ?????????? ??????????.
```

?? ????? ????? ????? Stitch Premium RTL ?????? ?????? ?????????? ?????? ?????? ?????????? ?????????? ?????????? ???  
 ??? ?????????? ??????????:

1. **\*\*????????? ?????? (Medical Inventory)\*\***: ???? Bento Grid ?????? ?????? ?????????? ?????? ?????????? ?????????? ?????  
 ?????? ?????????? ?????????? ?????? ?? ?????? ?????????? ?????????? ?????? ?? Modals ?????????? ?????????? ?????? ?? ?????? ?  
 ?????????.
2. **\*\*?????? ??????? ?????????????? (Procurement & POs)\*\***: ??? ??? ??????? ??????? ??????? ??????? ?????????? ?? ??? ?????  
 ?????????? ?????????? ??? ?????? ?????? "???????" ??????? ?????? ?????????? ?????????? ?? ?????? ??????????.
3. **\*\*????? ?????????? (Suppliers Directory)\*\***: ????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????  
 ?????.
4. **\*\*????????? ?????????????? (Logistics Tracking)\*\***: ?????????? ?????? ?????? ?????????? ?????????? ?????????? ?????????? ?????? ?????? ?  
 ?????????? ?????????? ?????????? ?????????? ?????? ??? ?????? ?????? ?????? ?????? ??.
5. **\*\*????? ?????????? ?????????????? (AI Command Center)\*\***: ????? ?????????? ?????? ??????? ?????????? ?????????? ?????????? ?????????? ??  
 ?????????.

---

## ## 8. ??????? ??????? ??????? (Validation & Test Commands)

- ??? ???? ???? ?????? (`npm run build:css`).

- ?????? ?????? ?????????? ?????????? ?????????? ?????????? ?????????? (`node --check`).

- ?????? ?????????? ?????????? ?????????? ??? ???? ?????? ?????? ?????? 100% ??????? ?????? ?????????? ?????? ?? ??? ?  
??? ?????? ?????? ?? ??????? ?? ?????????? (Console Errors: 0).

\* \*\*????? ?????? ?????????\*: ??? ?????? ?????????? ?????????? ?????? ??????.

\* \*\*?????? ?????????? ?????????? (Final Status)\*\*: \*\*BATCH\_C\_COMPLETED\_CLEAN\*\*

---

## ## 9. ?????? ? : ?????? ??????? ??? ?????????? ?????????????? (Batch D)

?? ?????? ?????? ?????? Stitch Premium RTL ??? ?????? ?????? ?????????????? ?????????? ?????:

### 1. \*\*????????? ?????????? (Finance & Accounting)\*\*:

\* ??? ???? ???? Bento Grid ?????? ?????? ?????????????? ?????????? ?????????? ?????? ??????????.

\* ?????? ?????? ?? ?????? ?????????? ?????????? (Revenue Chart) ?????????? ?????? ?????? ?????????.

\* ??? ?????? ?????????? ?????????? ?????? ?????????? (Chart of Accounts)? ??? ???? ?????????? ?????? ?????? ?????? ?????  
????????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?? ?????????? ?????? (VAT Reconciliation).

### 2. \*\*????????? ?????????? ?????????? (HR & Payroll)\*\*:

\* ??? ?????? ?????? ?????????? ?????????? ?????????? ??????????.

\* ?????? ?????????? ?????? ?????? ?????? ?????????? (Salary Slips) ?????????? ?? ???? ?????? ?????? (WPS).

\* ?????? ??? ?????? ?????????? ?????? ?????????? ?????????? ?????????? ?????????? ??????????.

### 3. \*\*????????? ?????????? ?????????? (Quality & Compliance)\*\*:

\* ??? ?????????? ?????? ?????? ?????? ?????????????? ?????????? ?????????? ?????? ??????

\* ?????? ?????? ?????????? ?????? ?????? ?????? (CBAHI)? ?????? ?????? ?????????? ?????????? (ZATCA) ?????? ?????? ?????  
??? ?????????? ?????? ?????? (MOH)? ??? ??? ?????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????? ?????? ??  
????? ?????? ??????.

\* ?????? ?????? ?????? ?????? (Audit Trail Viewer) ?????????? ?????????? ?????????? ?????????? (?????? ?????? ?????????? ??  
? ?????????? ?????? ?????????? ??????) ?? ?????????? ?????????? ?????? ?????????.

---

## ## 10. ?????????? ?????????? ?????? ? (Validation & Test Commands - Batch D)

- ??? ???? ???? ?????? (`npm run build:css`).

- ?????? ?????? ?????????? ??? ???? ?????? ?????????? (`node --check`).

- ?????? ?????????? ?????????? ?????????? ?????? 100% ?????? ?? ?????? ?????? ?????????? ?? ?????????? ?????????? ?????? ?????? ?  
????????? ?????????? ?????? ?????? ?????????? ?????????? ?????????? ??? ?? ?????? ?? ?????????? (Console Errors: 0).

\* \*\*????? ?????? ?????????? ?????? ?\*: ??? ?????? ?????? ?????? ?????????? ??????.

\* \*\*????????? ?????????? ?????????? ?????? ?\*: \*\*BATCH\_D\_COMPLETED\_CLEAN\*\*

---

## ## 11. ?????? ? : ?????????? ?????????? ?????????? ?????????????? ?????????????? (Batch E)

?? ?????? ?????? ?????? Stitch Premium RTL ??? ?????? ?????? ?????? ?????????:

### 1. \*\*????????? ?????????? ?????????? (Cybersecurity & Governance)\*\*:

\* ?? ??? ???? ?????? ?????????? ?????????? ?????? ?????? ?????????????? (Settings - Tab 3) ??? ???? ?????? ?????? ??????

?? ?????????????? ?????????? ?????????? ?????? (AES-256 ? RSA-4096).

\* ?????? ???? ?????? ?????? ?????????????? ?????????? ?????????????? ??? ???? ??? ?????? ?????????? ?????? ?????? ?????? ?????? ?  
????????????? ?????? (Postgres Backup via pg\_dump) ?????? ?????? ?????? ?? ?????????.

\* ??? ??? ?????????? ?????????? ?????? ?????? (Security Audit Trail) ?????????? ?? ??????.

```
2. **??????? ?????? ?????? (Biomedical & Facility Maintenance)**:
 * ??? ???? bento-grid ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ??????
 ?????? ?????? ?????? ?????? ??????.
 * ????? ?????? ?????? ?????? ?????? ?????? ?????? ?? ???? ?????? ?????? ?????? ??????.
 * ????? ?????? ?????? ?????? ?????? ???? ?????? ?????? queues ?????? ?? ?????? ?????? ??? ?????? ???.
 * ???? ?????? ?????? ?????? ?????? ?????? ?????? (Asset Registry) ?? ?????? ?????? ?????? ?????? ??
 ????? ??????.
3. **????????? ?????????? ?????? ?????? (Executive Analytics)**:
 * ???? ?????? ?????????? ?????????? (Operational Command Center) ?????? ?????? ??????.
 * ?????? ?????????? ?????? ?????? ?? ???? ?????? 2030 (????? ?????? ?????? ?????? ??? ??????????).
 * ????? ?????? ?? ?????? ?????????? ?????? ?????????? ?????? ?????? ?????? ?????????? ?????? ?????? ??????.
```

```
12. ????????? ????????? ?????? ?? (Validation & Test Commands - Batch E)
- ??? ???? ???? ??????? (`npm run build:css`).
- ?????? ?????? ????????? ?????? ??????? ????? JavaScript ? Node.js (`node --check`).
- ?????? ????????? ????????? ?????????? ??? ???? ?????? ?????? 100% ?????? ?? ???? ??? 500 ?? ??????? ????????? (status error resolved) ???? ???? ????????? ????????? ?????? ?????? ?????????? ?????????? ???? ?? ?????? ?? ????????? (Console Errors: 0).

* **????? ??????? ????????? ??????? ??*: ???? ??????? ?????????? ?????? ???? ?????????? ??????.
* **??????? ?????????? ????????? ??????? ??*: **BATCH_E_COMPLETED_CLEAN**
```

FILE: docs\STITCH\_FULL\_DESIGN\_CLOSEOUT\_REPORT\_AR.md

```
?????? ?????? ?????? ?????? ??? ?????? ?????? ??? ?????? (Stitch RTL)

???? ?? ??????? ??????? ??????? ??????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? *??? ????? (Nama Medical Web)* ??????? ?????? ??????? ??????? **Stitch Premium RTL** ?????? ??????? ??????? ?? ??????? ??????? ?? ???????.
```

```
1. ?????? ?????? ?????? (Completed Phases)

?? ????? ?????? ??????? ?????? ?? ??????? ??????? (APIs) ??????? ?? ???? ?? ?????? ?????? ??????? ??
??:

1. **????? ??????? ??????? ?????? (Foundation & App Shell)**:
 * ????? ?????? ??? ?????? ?????? ?????? (`login.html`) ?????? ??? ?????? ?????? (Modal).
 * ?????? ??????? ?????? (Dark Medical Navy Sidebar) ?? ?????? ?????? ??????? ?? ?????? (RTL).
 * ??? ???? ?????? ?????? (????/??????) ?? ?????? ?????? ??? ?????? ?????? ?????? ?????? ?????? (RTL/LTR) ??
 ?????? ?? ?????? ??????.
 * ????? ?????? ?????? ?????? ??????? ??????? ?? Stitch.

2. **????? ? (Batch B: ?????? ?????? ???????)**:
 * **?????? (Laboratory)**: ????? ?????? ?????? ??????? ?????? ?????? ?????? ?????? ??????.
 * **?????? (Radiology)**: ????? ?????? ?? ???? ?????? ?????? ?????? ??????? ??????.
 * **???????? (Pharmacy)**: ????? ?? ???? ??????? ?????? ??????? ?? ????? ?????? ??????? ???????.
```

3. \*\*????? ? (Batch C: ????? ????? ??????? ????????)\*\*:  
 \* \*\*??????? ?????? (Medical Inventory)\*\*: ??? Bento Grid ?????? ?????? ?????????? ?????????? ??  
 ?????? ??????? ??????? ??????.  
 \* \*\*?????????? ?????? ?????? (Procurement & POs)\*\*: ??? ?????? ??? ?????? ??? "?????" ?????? ??? ?????? ?????  
 ??? ?????????? ?? ?????? ??????????.  
 \* \*\*?????????? (Suppliers)\*\*: ??? ?????? ?????????? ?????????? ??? ?????????? ??????????.  
 \* \*\*????? ?????????? ??????????? (AI Command Center & Logistics)\*\*: ?????????? ??? ?????? ?????????? ?????????? ?  
 ?????? ?????? ?????? ?????????? ?????????? ?????????? ??????????.

4. \*\*??????? ? (Batch D: ?????????? ?????????????? ??????????)\*\*:  
 \* \*\*????????? ?????????? (Finance & Accounting)\*\*: ??? ?????????? (COA)? ?????? ?????? ?????? ??? ?????????? ???  
 ?????? ?????????? ?????? ?????? ?????????? ?????????? (VAT Reconciliation).  
 \* \*\*????????? ?????????? ?????????? (HR & Payroll)\*\*: ??? ?????????? ?????? ?????????? ?????????? (Salary Slips) ?????????  
 ?? ?? ??? ?????? ?????? (WPS)? ?????? ?????????? ??????????.  
 \* \*\*????????? ?????????? (Quality & Compliance)\*\*: ??? ?????? ?????????? ?????? ?????????? ?????? ?????? ??????????  
 (CBAHI, ZATCA, MOH) ??? ?????? ?????????? ?????? ??????.

5. \*\*??????? ? (Batch E: ?????????? ?????????? ?????????? ?????????? ??????????)\*\*:  
 \* \*\*??????? ?????????? ?????????? (Cybersecurity & Governance)\*\*: ??? ?????? ?????????? ?????????? ?????? ?????????? ??  
 ?? ?????? ?????????? ?????? ?????????? (pg\_dump) ?? ?????? ??? ?????? ??? ?????????? ?????? (Security Audit Trail).  
 \* \*\*????????? ?????????? ?????????? (Biomedical & Facility Maintenance)\*\*: ??? Bento Grid? ?????? ?????????? ?????????? ?  
 ?????????? (PM Calendar)? ?????? ?????? ?????? ?????? ??? ?? ??? ?????????? ?????????? ?????? ?????? ?????????? ?????????? (Asset  
 Registry).  
 \* \*\*????????????? ?????????? (Executive Analytics)\*\*: ??? ?????????? ?????????? (Command Center) ?????????? ?????? ??????  
 ?? 2030 (?????? ?????????? ?????? ?????????? ??? ?????????????) ?? ?????? ?????? ??? (Chart.js).





\* \*\*???????? ???? (Local HEAD)\*\*: `28b30c12e2c01997d91d95393c8375e81d11ff31`  
\* \*\*???? (origin/main HEAD)\*\*: `04b5e52a48173a373ac4f3bdaf9bd7ac3288bf1`  
\* \*\*??? (Ahead/Behind)\*\*: ??? ???? ???? ?? 11 ?????? (0 behind, 11 ahead).  
\* \*\*??? ???? (Failed Push Reason)\*\*: ??? ???? \*\*403 Permission Denied\*\* (?? ???? ???? ???? ???? ????  
`iceman18ice-sketch` ??? ???? `ialqrashi62/namawab.git` ).  
\* \*\*???? ???? UAT ?? ???? ???? (UAT Executed)\*\*: ?? (NO).  
\* \*\*???? ???? ???? ???? ???? (Staging Touched)\*\*: ?? (NO).  
\* \*\*???? ???? ???? (Production Touched)\*\*: ?? (NO).  
\* \*\*???? ???? ???? ???? (DB Changed)\*\*: ?? (NO).  
\* \*\*???? ???? ???? ???? (Migrations Run)\*\*: ?? (NO).  
\* \*\*??? ???? ???? ???? (Prisma db push)\*\*: ?? (NO).  
\* \*\*???? ???? ???? (Env Changed)\*\*: ?? (NO).

---

## ## 2. ???? ???? ???? (Reasons Blocked)

1. \*\*??? ???? ???? GitHub (GitHub 403 Push Blocker)\*\*: ?? ???? ???? ???? ???? ???? `iceman18ice-sketch` ????  
???? ???? ???? ???? ???? ???? ???? `ialqrashi62/namawab.git` .
2. \*\*??? ???? UAT ???? (Missing Required UAT Scratch Tooling)\*\*: ??? ???? ???? ???? ???? (`scratch/tes  
t\_posting\_flows.ts` ? `scratch/run\_posting\_flows.js`) ?? ???? ???? ???? .
3. \*\*??? ???? ???? ???? ???? ???? (Working Tree with Modified Blocker Docs)\*\*: ??? ???? ???? ????  
??? ???? ???? ???? ???? .

---

## ## 3. ???? ???? ???? (Required Human Action)

???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? :

\* \*\*???? \*\*?: ??? ???? (Write Access) ???? `ialqrashi62/namawab.git` ???? GitHub ???? (`ic  
eman18ice-sketch` ).  
\* \*\*???? \*\*?: ???? ???? ???? ???? Git ???? ???? ???? ???? ???? ???? ???? ????  
???? .  
\* \*\*???? \*\*?: ???? ???? ???? (Remote origin) ??? ???? (Fork) ???? ???? ???? ?  
???? ???? ???? ???? ???? (Pull Request).  
\* \*\*???? \*\*?: ??? ???? Nama Invest ERP ???? ???? ???? ???? ???? `scratch/` ???? ???? `sca  
tch/test\_posting\_flows.ts` ? `scratch/run\_posting\_flows.js` ???? ???? UAT ???? ???? .

---

\* \*\*???? ???? \*\*?: \*\*GIT\_PUSH\_BLOCKED\_PERMISSION\*\*

=====  
FILE: docs\STITCH\_GIT\_PUSH\_REPORT\_AR.md  
=====

## # ???? ???? ???? Git (Stitch Git Push Report)

???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? \*\*Stitch  
Premium RTL\*\* ?? \*\*??? ????\*\* ??? ???? ???? ???? ???? .

---

## 1. ?????? ?????? ???????? (Push Details)

\* \*\*????? ?????? (Pushed Branch)\*\*: `main`  
\* \*\*????? ?????????? ?????? (Pushed Commit SHA)\*\*: `866705db16c7e5688ce9890e9e40e94caf8c4cf8`  
\* \*\*????? ?????????? ?????? (Remote Sync Status)\*\*: ?????? ???????? (SUCCESSFULLY SYNCED). ?????? ???????? ???????? ???  
????? ???????? (0 ahead, 0 behind).  
\* \*\*????? ?????? ?????????? (Deployment Status)\*\*: ?? ??? ?????? (NOT DEPLOYED).  
\* \*\*??????? ?????? ?????????? (Database Touched)\*\*: ?? (NO).  
\* \*\*??????? ?????????? (Migrations Run)\*\*: ?? (NO).  
\* \*\*??????? ?????? ?????? (Push Result)\*\*: \*\*??????? ?????????? (SUCCESS)\*\*.

---

## 2. ???????? ???????? ?????????? ?????????? (Success Context)

?? ?????? ??? ?????????? ?????????? (Write Access) ???????? `iceman18ice-sketch` ??? ???????? `ialqrashi62/namawab.git` ?  
? ?????? ?????? ??????????. ?? ??? ??? ?????? ??? ?????? ??????:  
````  
To https://github.com/ialqrashi62/namawab.git
04b5e52..866705d HEAD -> main
````

---

## 3. ???????? ?????????? ??????????

????? ?????????????? ?????????? ???????? ?????? UAT ?????? ?????????? ?????????? ?????????? ?? ?????? ???????? ?????????? ??? ?????? ??????  
?? (`main`) ??? GitHub. ?????????? ???????? ?????? ?????????? ?????????? ?????? ?????? 100% ?? ?????? ?????? ?????????.

=====  
FILE: docs\STITCH\_MODULE\_BATCH\_PROGRESS\_AR.md  
=====

# ??? ??? ???? ???? ?????????? ???????? ?? Stitch  
(Stitch Module Batch Progress)

????? ??? ?????? ???? ?????? ?????????? ?????????? ?????????? ?? Stitch ??? ??? ???????? ?????? ??? 5 ????????? ?  
?? (Batches) ???????? ?????????? ?????? ?????????? ?????????.

---

## 1. ??? ???? ?????????? ?????? ??????

??? ????????	?????????? ???????? ??????????????	????????	?????? ??????????
\*\*Batch A\*\*	?????????? ?????????? ???????? ???????? (Reception, Appointments, Patient Portal)	`PENDING`	-
\*\*Batch B\*\*	?????????? ???????? ??????????: ?????????? ?????????? ?????????? (Laboratory, Radiology, Pharmacy)	`COMPLE	
TED`	2026-06-14		
\*\*Batch C\*\*	?????? ?????????? ?????????? ?????????? (Inventory, Supply Chain, Suppliers)	`COMPLETED`	2026-06-
15			
\*\*Batch D\*\*	?????????? ??????????????????: ?????????? ?????????? ?????????? ???????? (Finance, HR, Quality/Compliance)	`C	

```
| **Batch E** | ??????? ???????? ????????? ?????????? ?????????? (Cybersecurity, Maintenance, Analytics) | `COMPL
ETED` | 2026-06-15 |
```

```
2. ?????? ?????? ?????? ?? ?????
```

```
- **????????? ?????? ????????*: ??????? (Laboratory)? ?????? (Radiology)? ???????? (Pharmacy).
- **?????????*: ?????? ?????? Stitch Premium RTL ?? ??????? ???? ??????? ??????? ??????? ??????? ???? Tailwi
nd CDN ???????.
```

```
- **????????? ?????? ???????*. ??????? ????????? ????? (Inventory)? ?????? ????????? (Suppliers)? ?????? ?????? ?
????????? (Procurement)? ????????? ?????? ????????? ????????? ?????????? (Logistics & AI Command Center).
- **?????????*: ?????? ??? ?????????? ????????? (Bento Grid) ?????? ????????? ????????? ?????????? (Multi-tab layout)
????????? ?????? ?????? ?????????? ????????????? ?????????? ?????????? (RTL) ?????? ?????? ?? ??? ?????? ?????????????
????????? (?????????
?? ??????? ?????????).
```

```
- **???????? ???? ????**?: ?????? ??????? (Finance & Accounting)? ?????? ?????? (HR & Payroll)? ?????? ??????? ?????? ?????? (Quality, Compliance & Audit).
- **??????**:
 - **??????**?: ??? Bento Grid ?????? ?????? ?? ?????? ??????? ?????? ?????? ?????? ?????? ??????
 ?????? ?????? ?????? ???????.
 - **?????? ?????**?: ??? ?????? ?????? ?????? ?????? ?????? ?????? WPS? ?????? ?????? ???
 ??? ???????.
 - **????? ?????**?: ??? ??? ?????? ?????? ?? ?????? ?????? ?????? ?????? ?????? ?? ?????
 ??? ?????? ??? ?????? ?????? ?????? ?????? ?? ?? ?????? ??? ?????? ?????? (Audit Trail) ???
 ??? ??????.
```

```
- **???????? ?????? ???????*: ????? ????????? ????????? (Cybersecurity & Governance)? ?????? ?????? ?????????
(Biomedical & Facility Maintenance)? ?????????? ?????????? ?????? ????????? ?????? (Executive Analytics & Strategic
Dashboard).

- **????????*:

 - **???????? ??????*: ??? ?????? ?????? ?????????? ??? ?????? ?????????? ?????? ??? ???? ?????? ??????????
 ?????? ??? ?????? ?????????? ?????????? ??? ?????? ?????????????? ?????????? ?? ????????? ?????? pg_dump ?????????
 ?????????? ?????? ?????????? (Security Audit Trail).

 - **????????? ?????? ??????*: ?????? ?????? ?????????? ?????????? ?????????? ?????????? ??????????: ???
 ?????????? ?????????? ?? ??????
 ? ?????????? ?????????? ?????????? ?????????? ?????? ?????? ?????????? ?????????? ?????? ?????????? ??????
 ?????? ?????????? ?????????? ??????????????.

 - **????????????? ?????????????*: ?????? ?????????? ?????????? (Command Center) ?????????? ?? ?????????
 ?????????? ??????????
 ?????????? ?????? ?????? ?????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????? ??????????
 2030 (????????? ???
 ?????? ?????? ?????????? ??? ??????????????).
```

```
=====
FILE: docs\governance\enterprise-hospital-platform\AI_RAG_LANGCHAIN_VECTOR_ARCHITECTURE_AR.md
=====
```

# ???????? ??????? ???????????? ????? ???????????? RAG

????? ??? RAG ?????? ?????? ?????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\AI\_RAG\_SAFETY\_TEST\_PLAN\_AR.md

=====

# ??? ?????? ??? ?????? ?????? ??????

????? ?? ??? ?????? ??? PHI ??? ?????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\AI\_SAFETY\_AND\_HUMAN\_APPROVAL\_POLICY\_AR.md

=====

# ?????? ??? ?????? ?????????? ?????????? ??????

??? ?????? ??? ?????? ?????? ?????? ?????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\API\_CONTRACT\_PREPARATION\_REPORT\_AR.md

=====

# ?????? ?????? ?????? ??? ?????????? ?????????? (API Contracts)

????? ??? ?????????? ?????? ??? ?????????? ?????????? ?????? ??? DTos ??? OpenAPI ?????????? ?????????.

## ?????? ?????? ??????????

1. \*\*????? ??????\*: ?? ?????? ?????????? ?????? ?????????? (`1.0.0-draft`).

2. \*\*??? DTos ?????????\*: ?????? ?????? ?????????? ?????? ?????????? ?????????? (????????? ?????????? ?????????? ?????????? ?? ??????).

3. \*\*???????? OpenAPI\*: ?????? ?????????? ?????? ?????? ?????????? ?????????? ?????? YAML ?????????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\API\_SECURITY\_MATRIX\_AR.md

=====

# ?????? ??? ?????? ?????? ?????????? ??????

????????? ?????????? ?????????? ?????????? ?? ?????????? ??? ?????????.

=====

FILE: docs\governance\enterprise-hospital-platform\API\_SPECIFICATIONS\_OPENAPI\_AR.md

=====

# ?????????? ?????????? ?????????? ?????????? OpenAPI

????????? ??? API ?????? OpenAPI ?????????? ?????? ?????? ?????????.

FILE: docs\governance\enterprise-hospital-platform\API\_TO\_RLS\_GUARD\_MATRIX\_AR.md

???? ??? ???????? ?? ???? ?????? ???? ?????? ?????? ???? RLS ???????? ?? ???????? ???????.

```
| ?????? ??????? | ?????? ??????? | ?????? ??? RLS ??????? | ?????? ??????? ????????? PHI | ??? ???? |
| :--- | :--- | :--- | :---: | :---: |
| `/api/v1/facilities` | ??? ???? | `facility_view_policy` | ???? | ??? |
| `/api/v1/queue` | ??? / ??? | `queue_department_policy` | ???? | ??? |
| `/api/v1/clinical-orders` | ??? / ???? | `orders_encounter_policy` | ???? | ??? |
| `/api/v1/clinical-billing` | ??? ???? | `billing_tenant_policy` | ???? | ??? |
```

FILE: docs\governance\enterprise-hospital-platform\APPOINTMENTS\_QUEUE\_SECURITY\_AND\_SAFETY\_REVIEW\_AR.md

???? ??? ?????? ??????? ??????? ??????? ?????? ??? ??????? ?????????? ??????? ??????? ?????????.

1. **\*\*??? ??????? ?????? ?????? ?? ????? ?????????\*\***: **???? ??????? ?????? ???? Check-in ????? ??????? (Read-only P review) ???? ????? ?????????**.
2. **\*\*??? ?????? ??????? PHI\*\***: **??? ??????? ?????????? ??????? ?????????? ?????????? ?? ?? ????????? ?????? ??????? ??????? ??? ???? ????????? ?????? ?????? ?????????? ????????? (PDPL)**.
3. **\*\*???????? ????????? ????????? (Urgent Badges)\*\***: **?????? ?????????? ?????? ?????? ?????????? ?????????? ?????? ?????? ?????????? ? ?????????? ?????????? ?????????? ??????**.

FILE: docs\governance\enterprise-hospital-platform\APPOINTMENTS\_QUEUE\_TEST\_REPORT\_AR.md

???? ???? ????? ???? ???? ???? ???? ???? ???? ???? ????.

1. **\*\*?????? ???? ???? ???????\***: ?????? ????? ?? ??? ??? ?????? ??????? ??????? ?? ??????? ??????? ??????? ???????  
??.
2. **\*\*?????? ?????? ?????????\*\***: ?????? ?? ?????? ??????? ????????? ??????? ??????? ?????? (????? ?????) ??????
3. **\*\*?????? ?????? ?????? ?????\*\***: ?????? ?? ?? ?????? ?????????? ?????? ????????? ??????? ??????? ????????? ???????  
?????? ?????????.

FILE: docs\governance\enterprise-hospital-platform\APPOINTMENTS\_QUEUE\_WORKFLOW\_DESIGN\_AR.md

# ????? ?????? ??? ????????? ??????? ?????? ??????

???? ??? ????????? ??????? ????????? ?????? ??? ????????? ?????? ??????? ??????? ??????.

## ??? ???? ?????????? (Appointment Booking Flow)

1. \*\*?????? ??????? ???????\*: ??? ????????? ??????? ??????? ?????? ????????? ???????.
2. \*\*????? ??? ?????????\*: ?????? ??? ?????? ???? ???? (Walk-in)? ?? ?????? ???????.
3. \*\*??? ?????????? ?????????\*: ?????? ????????? ?????????? ?????????? ??????? ?????? ?????? ?? ??? ?? ?????? ?? ?????? ??????????.

## ??? ?????? ?????????? ??????? (Patient Queue & Triage Flow)

1. \*\*????? ??????? (Check-in)\*: ?????? ??? ?????? ?????????? ??????? ?????????? ???????.
2. \*\*????? ?????? (Triage)\*: ?????? ?????? ?????? ?????????? ?????????? ??????? ??????? ??????? (????? ?????? ?????? ?????? ?????????? ?? ?????? ??????).
3. \*\*??? ?????????? (Encounter Launch)\*: ?????????? ??????? ?????? ?????????? ?????? ?????????? ?????????? ???????.

=====

FILE: docs\governance\enterprise-hospital-platform\APPOINTMENT\_BOOKING\_PREVIEW\_UI\_REPORT\_AR.md

=====

# ?????? ?????? ??????? ??? ?????????? (Appointment Booking Preview)

????? ??????? ?????? ??????? ?????? ??? ?????????? ?????????.

## ?????? ?????????? ?????????

1. \*\*?????? ??????? ???????\*: ?????? ??? ??????? ?? ??????? ??????? ??? ?????????? ?????????? ??????? ?????????.
2. \*\*?????? ?????? ??????????\*: ?????? ?????? ?????????? ??????? ?????? ?? ??? ?????? ??? ??????? (Draft).

=====

FILE: docs\governance\enterprise-hospital-platform\APP\_JS\_GROWTH\_AND\_REFACTOR\_RISK\_REVIEW\_AR.md

=====

# ?????? ??????? ??? ?????? ??? app.js ??? ?????? ?????????? (Refactor Risk Review)

????? ??? ?????????? ?????? ??? ?????? ?????? ?????????? ?????????? `public/js/app.js` ??????? ??? ??? ??????? ?????? ??????????.

## ?????? ??????? ??????

1. \*\*??? ??????? ???????\*: ??????? ?????? ??????? 14,500 ???.
2. \*\*?????? ?????????\*: ?? ?????? ??????? ?????? (?????? ??????? ?????????? ??????? ???????) ??? ?????????? ?????????.
3. \*\*?????? ??????? ???????\*:
  - \* \*\*??? ???????\*: ?????????? ??? ?????? ??????? ??????? (`Keep as-is`) ?????? ?????????? ?????????? ?????????????.
  - \* \*\*??? ???????\*: ?????? ??????? ?? ?????????? ?????????? ?? ?????? ??? ?????? ?????????? ??????? ??????? (??? `app-billi ng.js`? `app-pharmacy.js`) ??????? ?????????? ??????? ?????????.

=====

FILE: docs\governance\enterprise-hospital-platform\APP\_JS\_MODULE\_BOUNDARY\_REVIEW\_AR.md

=====

# ?????? ??????? ??? ?????????????? ?????? ??? app.js ???????

```
???? ??? ?????? ?????? ?????? ??? ?????????? ?????? ?????? ??? ??? ??????.

????? ????????? ???????
1. **??? ??? ????? ?????**?: ?????? ?????? ?????? 14,500 ???.
2. **????? ?? ?????**?: ?? ?????? ??? ?????????? ??????? ?????? ?? ?????? ?????? (?? ?????????? ?????????? ?????????
?).
3. **???????? ?????????**?: ??????? ?? ?????? ????????? ?????????? ?? ?????? ?????? ?????? ?????? ?????? (Adapters)
?????? ??????????.

=====
FILE: docs\governance\enterprise-hospital-platform\ARCHITECTURE_DOCUMENT_AR.md
=====

????? ?????? ?????????? ?????? ??????
???? ?????? ??????? ??????? ?????? ?????????? ??????? ??????? ??????.

=====
FILE: docs\governance\enterprise-hospital-platform\AUDIT_PERSISTENCE_DESIGN_NO_DDL_AR.md
=====

????? ?????? ?????????? ?????? ?????????? ?????? (Audit Persistence)

???? ?? ?????????? ??????? ?????????? ?????????? ?????? ?????? ?????????? ??????? ?????????? ?????????? ?????????? ??????????.

??? ??? ?????? ??????? (audit_events)
1. **????? ?????? (event_id)**: ?????? ?????? ???.
2. **????????? ?????? (actor_id, actor_role)**: ?????? ??? ?????????? ??? ??????????.
3. **????? ??????? (facility_id, department_id)**: ?????? ??? ??? ??????????.
4. **????????? ??????? ??? ?????? (action, risk_level, status)**: ?????? ?? ?????????? ??????? ?????????? ?????????? (??
?????? ?? ??????).

=====
FILE: docs\governance\enterprise-hospital-platform\AUDIT_PREVIEW_UI_REPORT_AR.md
=====

????? ?????? ??????? ??????? ?? ?????????? (Audit Preview UI)

???? ??????? ?????? ??????? ??? ?????? ?? ?????????? ??????? ??????????.

????? ?????????? ?????????
1. **?????? ??????? ?????? ?????**?: ?????? ?????????? ??? ?????? ?????? ?????????? ?????????? (????? ??????? ??? ??????).
2. **????? ?? ?? ?????????**?: ?????? ??????? ??? ??????? ?????? ??????? ??????? ?????? JSON ?????? ?? ??????? ?????? ??
?????.

=====
FILE: docs\governance\enterprise-hospital-platform\BACKEND_API_RUNTIME_READINESS_CHECKLIST_AR.md
=====

????? ??????? ??? ?????????? ??????? ?????????? ?????????? (Runtime Readiness)
```

???? ???? ?????? ????? ????????? ?????? ????????? ????????? ??? ????? ?? ????? ?????? ?????????.

## ????? ?????????

1. \*\*????? ?????? ????????? ?????????\*\*?: ?????? ?????? ????????? ????????? SSL/TLS.
2. \*\*??????? ?????? ?????? ?????????\*\*?: ?????? ????????? ????????? ????????? ?????????? ?? ????????? ?????????.
3. \*\*????? ?????????? ?????? ?????????\*\*?: ?????? ?????????? ?????????? ??? ?????? ?????? RLS ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\BACKEND\_ERD\_DESIGN\_NO\_DDL\_AR.md

=====

# ?????? ?????? ?????? ?????????? ?????????? ?????????? (ERD Design)

???? ??? ?????????? ?????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ??????????.

## ?????????? ?????????? ??????????

1. \*\*health\_networks & facilities\*\*?: ?????? ?????????? ?????????? ?????????? ?????????? ???.
2. \*\*appointments & queue\_items\*\*?: ?????? ?????? ?????????? ?????? ??????.
3. \*\*encounters & clinical\_orders & clinical\_results\*\*?: ?????? ?????????? ?????????? ?????? ?????? ?????????? ?????? ???.
4. \*\*billing\_charge\_previews & claim\_drafts\*\*?: ?????? ?????? ?????? ?????????? ?????????? ?????????? ???.

=====

FILE: docs\governance\enterprise-hospital-platform\BACKEND\_READINESS\_MATRIX\_AR.md

=====

# ?????? ?????? ?????????? ?????????? (Backend Readiness Matrix)

???? ??? ?????????? ?????? ?????????? ?????? ??? ?????? ??? ?????? ?????? ?????????? ?????????? ??????????.

## ?????? ??????????

??????????   ?????????? ??????????   ?????? ??????   ??????????????   ??? ????????? (DTO)   ?????? ?????? (RBAC)   ?????? ?????? ??????
:---   :---:   :---:   :---:   :---:   :---:   :---:
**????????? ?????????**   ???   ??   ???   ???   ???   ?????? (???)
**????????? ?????????**   ???   ??   ???   ???   ???   ?????? (???)
**????????? ?????????**   ???   ??   ???   ???   ???   ?????? (???)
**????????? ?????? FEFO**   ???   ??   ???   ???   ???   ?????? (???)
**????????? ?????????**   ???   ??   ???   ???   ???   ?????? (???)

=====

FILE: docs\governance\enterprise-hospital-platform\BACKEND\_SCHEMA\_RLS\_DESIGN\_TEST\_REPORT\_AR.md

=====

# ?????? ?????????? ?????? ?????? ?????????? ?????? RLS

???? ?????? ?????? ?????????? ?????????? ?????? ?????? ?????????? ?????? ?????? ?? ?? ?????? ??????.

## ?????? ?????????????

1. \*\*????????? ??? ?????? ?? ?????????? ?????????\*\*?: ?????? ?????? ?? ??? ?????? ?? ?????? SQL ?? ?????? ?????? ??????????.



```
2. **?????? ???? ???????? ????????*: ?????? ?? ???????? ??? ????????? ?????????? ?? ???? ????????? ?????????? (`PLANNE
D_DISABLED`).
```

FILE: docs\governance\enterprise-hospital-platform\BACKEND\_SECURITY\_THREAT\_REVIEW\_AR.md

???? ??? ?????? ?????? ?????????? ???????? ?????????? ???????? ?????????? ?????????? ?????????? ?????????? ???? ???.

```
????????? ???? ?????
```

1. **\*????? ?????? ?????????? (Tenant Leakage)\*:** ???? ???? ??? ??? ?????? RLS ?????? ??? ???? ??????? ?????? ??  
??????.

3. \*\*?????? ?? ???? ?? ?????? ?????? (PHI Exposure)\*\*: ???? ?????? ????????? ??????? ?????? ?? ????? ??????  
?? ?????????? ??????.

FILE: docs\governance\enterprise-hospital-platform\BUSINESS\_FLOWS\_MASTER\_AR.md

?????? ???? ???? ?????? ?????????? ???? ?????????? ?????????? ?????????.

FILE: docs\governance\enterprise-hospital-platform\BUTTONS AND ACTIONS\_CATALOG\_AR.md

??? ?????? ?????????? ?????????? ?????????? ??? ?? ?? ?????? ???????.

FILE: docs\governance\enterprise-hospital-platform\CBAHI\_EVIDENCE\_MATRIX\_AR.md

????? ?????? ???????? (OVR) ???? ???????? ???????? ??????.

FILE: docs\governance\enterprise-hospital-platform\CHARGE\_CAPTURE\_PREVIEW\_UI\_REPORT\_AR.md

???? ?????? ?????? ?????? ???? ?????? ?????? ?????? ??????.

## ?????? ??????? ???????

1. \*\*????? ??? ???????: ??? ?????? ??????? ?????? ??????? ??????? (??? ??? ?????????? ?????? ???).
2. \*\*????????? ??? ???????: ?????? ??????? ?????? ?????????? ?? ??? ??? ?????? ??? ?????? ?????? ?????? ??????.

=====  
FILE: docs\governance\enterprise-hospital-platform\CI\_CD\_PIPELINE\_PLAN\_AR.md  
=====

# ??? ?? ?????? ??????? ?????? ??????

????? ?????? ??????? ?????????????? ?????? ?????????? ??? ??????.

=====  
FILE: docs\governance\enterprise-hospital-platform\CLAIM\_DRAFT\_PREVIEW\_UI\_REPORT\_AR.md  
=====

# ?????? ?????? ??????? ??????? ?????????? (Claim Draft Preview)

????? ??????? ?????? ??????? ???? ??????? ?????????? ?????????? ??????????.

## ?????? ??????? ???????

1. \*\*??? ??????? ??????????: ??? ?????? ?????????? ?????????? ???? ??????? ??????? ??? ??????.
2. \*\*????????? ?????? ?? ??????????: ?????? ???? ?????????? ?????????? ?????????? ???? ?????? ?????????? (??? ??? ?????????? ?? ?????????? ?????????).

=====  
FILE: docs\governance\enterprise-hospital-platform\CLINICAL\_BILLING\_CLAIMS\_WORKFLOW\_DESIGN\_AR.md  
=====

# ?????? ??????? ???? ?????????? ?????????? ??????? ??????? ?????

????? ??? ?????????? ??????? ?????????? ?????? ?????? ??????? ?????????? ?? ?????????? ??????? ??????????.

## ???? ?????? ??????? (Charge Capture Flow)

1. \*\*?????? ???????: ??? ?????? ?????????? ?? ?????????? ??? ?????? ??????? ?????????? ??????.
2. \*\*??? ??????? ???????: ?????? ?? ?????? ?????????? ??????? SOAP ??????? ?????????? ?????? ?? ??? ??? ??????.
3. \*\*?????? ???????: ?????? ?????? ?? ???? ?????????? ???? ?????? ?????????? ?????????? ???? ?????????? ??????????.

## ???? ?????? ?????????? (Insurance & NPHIES Flow)

1. \*\*??? ???????: ?????????? ?????????? ?? ?????? ?????????? ???? ??????? ??????.
2. \*\*?????? ?????? ??????????: ?????? ??????? ?????????? ?????????? ?? ??? ?????????? ??????? ?????????? ??? ?????? ??????.
3. \*\*??? ??????? ???????: ??? ?????? ?????????? ???? ??? ?????????? ??????? ?????? ???? ?? ??? ?????????? ??????????.

=====  
FILE: docs\governance\enterprise-hospital-platform\CLINICAL\_BILLING\_SECURITY\_AND\_SAFETY\_REVIEW\_AR.md  
=====

# ????? ?????? ?????? ?????? ???????? ???????? ??????????

???? ??? ???????? ???????? ???????? ???????? ?????? ?????? ???????? ???????? ???????? ??????????.

## ???????? ???????? ????????

1. \*\*??? ?????? ???????? ???????? ???????? ????????\*\*?: ??? ?????? ?????? ???????? ???????? ?????? ?????? ?????? ?? ?????? ???????? (Read-only Preview) ?????? ??? ??? ?????? ?????? ???????.
2. \*\*??? ?????? ?????????? ?????? ?????? (NPHIES Submissions)\*\*?: ??? ?????? ?????????? ???????? (Draft Preview) ??? ?? ?????? ?????? ???????? ???????? ??? ?????? ??????.
3. \*\*??? ?????? ?????? PHI ?????? ???????? ?????????\*\*?: ?? ?????? ?????? ???????? ?????????? ?? ?? ?????? ?????? ?? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\CLINICAL\_BILLING\_TEST\_REPORT\_AR.md

=====

# ????? ???????? ?????? ???????? ???????? ???????? ????

???? ??? ?????? ???????? ???????? ?????? ???????? ???????? ?????????? ???????.

## ?????? ????????????

1. \*\*?????? ?? ?????????? ?????????\*\*?: ?????? ?????? ?? ?? ?????? ???????? ?????????? ???????? ?????? ?????????? ?????? ?????? `false` ??????.
2. \*\*?????? ???????? ?? ?????????\*\*?: ?????? ?? ??? ???????? ?? ???????? ???????? ?? ?? ?? ?????? ?????????? ?????? ??????.
3. \*\*?????? ???????? ??????\*\*?: ?????? ?? ?? ???????? ?????? ?????????? ?????? ???????? ?????????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\CLINICAL\_DOCUMENTATION\_UI\_REPORT\_AR.md

=====

# ????? ?????? ?????? ???????? ???????? (Clinical Documentation)

???? ?????? ?????? ?????? ??? ???????? ???????? ?????? ?????????? ??????.

## ?????? ???????? ????????

1. \*\*?????? SOAP ?????????\*\*?: ?????? ?????? ?????? ?????????? ???????? ?????????? ?????????? ?????? ?????????? ??????.
2. \*\*??? ???????? ?????????? ?????????\*\*?: ?????? ?????? ?????????? ?????????? ?????????? ???????? ?????? ??? ?? ?????? ??? ?????? ?? ??? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\CLINICAL\_ORDERS\_SECURITY\_AND\_SAFETY\_REVIEW\_AR.md

=====

# ????? ?????? ?????? ???????? ???????? ???????? ???????

???? ??? ???????? ?????????? ?????????? ???????? ?????? ?????? ?????? ??? ???????? ??????.

## ?????????? ?????????? ????????

1. \*\*??? ?????????? ?????????? (Allergy Check)\*\*?: ?????? ???????? ?????? ??? ?????? ?? ??? ?????? ?????? ?????? ?????? ? ?????? ??????.

2. **\*\*???? ?????? (Consent Validation)\*\***: ?????? ?????? ?? ?????? ?????? ?????? ?????? ??? ?????? ?? ?  
????? ?? ?????? ?????? (??? ?????? ??????).
3. **\*\*??? ?????????? ?????????\*\***: ??? ?????? ?????????? ?????? ??? ?????? ?????????? (Draft Only) ?????? ??? ?????????? ?? ??????  
????????? ??? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\CLINICAL\_ORDERS\_TEST\_REPORT\_AR.md

=====

# ?????? ?????????? ?????? ?????????? ?????? ?????????? ??????????

????? ???? ?????? ?????????? ?????????? ?????? ?????????? ?????????? ?????????? ??????????.

## ?????? ?????????????

1. **\*\*????????? ??? ?????????? ?????????? (Health Unit)\*\***: ?????? ?????? ?? ??? ??? ?????????????? ?????????? ?????????? ?????????? ?  
? ?????????? ??????????.
2. **\*\*????????? ??? ?????????? (Polyclinic)\*\***: ?????? ?????? ?? ??? ?????????? ??? ?????? ?????????? ?????????? ?????? ?????? ???  
????? ??????????.
3. **\*\*????????? ?????????????? ?????????\*\***: ?????? ?? ?????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????? ?????????? ??  
???

=====

FILE: docs\governance\enterprise-hospital-platform\CLINICAL\_ORDERS\_WORKFLOW\_DESIGN\_AR.md

=====

# ?????? ?????? ???? ?????????? ?????? ??????????

????? ??? ?????????? ?????? ?????????? ?????? ??? ?????????? ?????? ?????????? (????????? ?????????? ?????????? ?????????????? ??????  
?????).

- ## ???? ?????? ?????????? ?????????? (Lab & Radiology Flow)
1. **\*\*????? ?????????\*\***: ?????? ?????? ?????? ?????????? ?? ?????????? ?????? ?????? ??????.
2. **\*\*??? ?????????? ?????????\*\***: ?????? ?? ?????? ?????? ?????? ??? ?????????? ??????????.
3. **\*\*????????? ?????????\*\***: ?????? ?????? ?????? ?????????? ?????????? ?? ?????? ?????? ?? ?????? ??????????.

- ## ???? ?????????? ?????????? ?????????? (Medications & Procedures Flow)
1. **\*\*????? ?????? / ?????????\*\***: ?????? ?????? ?? ?????????? ??????????.
2. **\*\*??? ?????????? ?????????\*\***: ??? ?????? ?????? ?? ??? ?????? ?????????? ?????? ?? ?????? ??? ??????.
3. **\*\*??? ?????? ?????????\*\***: ??? ?????? ?????? ??? ?????? ?????????? ?????? ?????????? ?????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\CLINICAL\_PHARMACY\_FEOF\_WORKFLOW\_DESIGN\_AR.md

=====

# ?????? ?????? ???? ?????????? ?????????? ?????? FEFO

????? ??? ?????????? ?????? ?????????? ?????? ?????????? ?????????? ?????? ?????????? ?????? ??????.

## ???? ?????????? ?????????? (Clinical Pharmacy Flow)

1. **\*\*??? ?????????\*\***: ???? ?????? ???? ?????? ??????.
2. **\*\*????????? ?????????? (Review Queue)\*\***: ??? ?????? ?????? ?????? ?????????? ?????? ?? ?????? ?????????? ??????????.

?? ????? ???? ?? ???? ?? ????? ????????? ?? 10 ????? DTO ??????? ?? ????? ?????? ?????? ????????:

```
| ?????? (Resource) | ??? ???? ????? (DTO Name) | ??????? ???? ??? ?????? ???? (Verified Fields) |
| :--- | :--- | :--- |
| **facilities** | `FacilityDTO` | `id`, `name_en`, `name_ar`, `type` |
| **departments** | `DepartmentDTO` | `id`, `name_en`, `name_ar`, `facilityType` |
| **appointments** | `AppointmentDTO` | `id`, `type`, `preferredDate` |
| **queue** | `QueueItemDTO` | `queueNo`, `type`, `status`, `priority`, `waitTimeMin`, `deptPage` |
| **encounters** | `EncounterDTO` | `id`, `patientId`, `encounterTypeId` |
| **clinical-orders** | `ClinicalOrderDTO` | `id`, `type`, `itemCode`, `status` |
| **clinical-results** | `ClinicalResultDTO` | `id`, `code`, `value`, `abnormal` |
| **clinical-pharmacy** | `PharmacyReviewDTO` | `id`, `medicationCode`, `status` |
| **clinical-billing** | `BillingPreviewDTO` | `encounterId`, `totalAmount`, `copayPercent` |
| **audit-events** | `AuditPreviewDTO` | `timestamp`, `role`, `action`, `status` |
```

---

## ## 2. ????? ?????? ?????? ??????? (Runtime Validation Mechanism)

??? ?????? ??? ?????????? ?????????? ??? ??? ?? ????? ??? ????? ??????????:

1. ??? ?????? ??? ??? DTO ?????????? ?????????? ?????????? ??? ?????????? ??????????.
2. ??? ?????????? ???? ?????????? ??????????:  
`window.validateDtoShapePreview(dtoName, dataItem)`
3. ????? ?????????? ?????????? ???? ???? ?????????? ?????????? ???? ?????????? ?????? ???.
4. ?? ??? ????? ?? ??? ?? ?????????? ?? ????? ?????????????? ??? ??? ?????????????? ?????? ?????????? ?????? ??? ?????? ??? ??????????:  
``json  
{  
 "status": "CONTRACT\_VIOLATION",  
 "reason": "DTO\_SHAPE\_MISMATCH",  
 "message": "Mock data item in [Resource] does not match the contract DTO: [DTOName]"  
}  
``

---

## ## 3. ?????? ??? ?????????? (Validation Results)

\* \*\*????? ??? ?????????? ??? 10:\*\* \*\*????? ?????? 100%\*\* ?.

\* ?? ?????????? ?????????? ??? ??? ?????????????? ?????????? ?? ?? ?????? ?????????? ?????????? ?????????? ?? `MOCK\_DATA\_STORE` ?????????? ??  
????? ?? ?????? ?????????????? ?????????? ??? ?????? ??? ?? ?????? ?????????? ?? ?????? ?? ?????? ?????? ??? ?????????? ?????????? ??????????  
?????????.

```
=====
FILE: docs\governance\enterprise-hospital-platform\CONTRACT_TO_DATA_MAPPING_REVIEW_AR.md
=====
```

# ?????? ?????????? ???? ?????????? ?? ?????? ?????????? ?????????? ??????????

????? ??? ?????????? ?????? ?????????? ?????? ?????????? ?????????? ?????????? ?????????? ?????????? ?? ??? ?????????? ?? ?????? ?????? ???  
?????? ?????????????.

## ## ?????? ??????????

1. \*\*????????? ??????????\*\*: ?????? ?????? ??? DTOs ?????????? ?? ?????????? ?????????? ?????????? ?????????? ?????????? ??????????.



=====

FILE: docs\governance\enterprise-hospital-platform\CURRENT\_WORKFLOWS\_INVENTORY\_AR.md

=====

# ??? ?????? ?????? ?????????? ??????????

????? ?????????? ?????????? ?????? ??????? ?????????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\DASHBOARD\_REQUIREMENTS\_AR.md

=====

# ?????????? ?????? ?????????? ??? ?????? ????????

????? ?????? ?????? ?????????? ?????????? ?????????? (Facility Switcher) ?????????? ?????????? ?????? Widgets ?????? ??? ?????????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\DATABASE\_ERD\_MASTER\_AR.md

=====

# ?????????? ?????????? ?????? ?????????? ?????????? ??????????

????????? ??? ERD ?????? ?????????? ?????????? ??? ?????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\DATA\_CLASSIFICATION\_AND\_SCOPE\_MATRIX\_AR.md

=====

# ?????????? ?????? ?????????? ?????? ?????????? (Data Classification Matrix)

????? ??? ?????????? ?????????? ?????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ??? ??????????.

## ?????? ??????????

1. \*\*????????? ?????? ?????? ?????????? (High PHI)\*\*: ?????? ?????? ?????????? ?????????????? ?????????? ?????????? ?????????? (?????? ?????? ? RLS ?????? ?????????? ??????????).
2. \*\*????????? ?????? ?????? (Financial Data)\*\*: ?????? ?????????? ?????????? ?????????? ?????????? ?????? ?????????? (?????? ??? ?????? ?????????? ??????????).
3. \*\*????????? ?????????? ?????? (Operational Data)\*\*: ?????? ?????? ?????????? ?????????? ?????????? ?????????? (?????? ?????? ?????????? ?????????? ??????????).

=====

FILE: docs\governance\enterprise-hospital-platform\DATA\_FLOW\_MASTER\_AR.md

=====

# ?????????? ?????? ?????? ?????????? ?????????? ??????????

????? ?????????? ?????????? ?????????? ?????????????? ?????? RAG ?????? ??????????.



=====

FILE: docs\governance\enterprise-hospital-platform\DATA\_MODEL\_DRAFT\_NO\_DDL\_AR.md

=====

# ????? ????? ????????? ??????? ??????? ?????????? ??????? (???? DDL)

???? ??? ??????? ????????? ????????? ????????? ????????? ????????? ?????????? ?????????? ??? ????? ?? ????? ??? ??? ?????

????????? ?????????.

## ????????? ?????????? ?????????

1. \*\*???? ??????? (facilities)\*\*: ?????? ??????? ????????????? ?????????? ??? ?????????.

2. \*\*???? ????????? (encounters)\*\*: ??? ?????? ????????? ??????? ?? ????????? ?????? ???????.

3. \*\*???? ?????? ????????? (inventory\_batches)\*\*: ?????? ?????? ????????? ????????? ????????? ?????????? ??????? FEFO.

4. \*\*???? ????????? ????????? (claim\_drafts)\*\*: ?????? ??????? ?????????? ????????????? ?????????? ?????? ?? ?????.

=====

FILE: docs\governance\enterprise-hospital-platform\DEPARTMENT\_CATALOG\_SEED\_PLAN\_AR.md

=====

# ??? ?????? ?????????? ?????????? ?????????

??? ?????? ??????? ?????????? ?????????????? ??????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\DEPARTMENT\_FACILITY\_APPLICABILITY\_MATRIX\_AR.md

=====

# ??????? ??????? ?????????? ?????????? ???????

????? ?????? ?????? ?????????? ?????????? ?????? ??? ?????????? (????????? ?????? ?????? ??????).

=====

FILE: docs\governance\enterprise-hospital-platform\DEPARTMENT\_PERMISSION\_MATRIX\_AR.md

=====

# ??????? ?????????? ??????? ?????????? ?????????? ?????????

????? ?????????????? ?????????????? ?????????????? ??? ??? ??? ?????? ??? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\DEPARTMENT\_SCREEN\_REQUIREMENTS\_MATRIX\_AR.md

=====

# ??????? ?????? ?????????? ?????????? ?????????? ?????????

????? ?????????? ?????????? ?????????????? ?????????????? ??? ?????? ???.

=====

FILE: docs\governance\enterprise-hospital-platform\DEPLOYMENT\_PLAN\_AR.md

=====

# ??? ????? ???????? ????? ???????

????? ?????? ???????? ??? ???????? ???????? ?? ???????? ??? PM2.

=====

FILE: docs\governance\enterprise-hospital-platform\E2E\_SMOKE\_TEST\_PLAN\_AR.md

=====

# ??? ???????? ???????? ??? ???????? ????????

?????? ???????????? ??? ???????? ???????? ???????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\ELIGIBILITY\_PREVIEW\_UI\_REPORT\_AR.md

=====

# ????? ?????? ?????? ?????? ?? ?????? (Eligibility Preview)

???? ?????? ?????? ?????? ??? ?????? ?? ?????? ???????? ??????.

## ?????? ???????? ????????

1. \*\*????? ?????? ?? ????????\*: ?????? ??? ?????? ?? ???????? ???????? ???????? ?? ??? ??????.
2. \*\*??????? ???????? ????????\*: ??? ??? ?????? ?????? ?????? ?????? ???????? ??? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\ENCOUNTER\_SECURITY\_AND\_CLINICAL\_SAFETY\_REVIEW\_AR.md

=====

# ?????? ?????? ?????? ?????? ?????? ???????? ???????? ??????

???? ??? ???????? ???????? ???????? ?????????? ?????????? ?????????? ???????? ?????? ?????? ??????.

## ???????? ????????

1. \*\*??? ?????? ?????? PHI\*: ??? ?????? ?????? ?????? ?? ??? ?? ?????? ??? ?????? ?????? ?? ?????????? ???????? ?? ?????? ???????? ?????? ?????? ?????? ?????? (PDPL).
2. \*\*??? ?????????? ??? ??? ????????\*: ?????? ?????? ?????????? ?????? ?????? ??? ?????? ???????? ???????? ?? ?????????? ?? ?????? (Polyclinics) ???????? ???????? (Health Units).
3. \*\*????????? ???????? ???????? (High-Risk Badges)\*: ?????? ???????? ???????? ?????? ??? ?????? ?????? ?? ???????? ??? ?????? ??? ?????????? ?????????? ???????? ?????? ???????? ???????? ????????.

=====

FILE: docs\governance\enterprise-hospital-platform\ENCOUNTER\_WORKFLOW\_TEST\_REPORT\_AR.md

=====

# ?????? ?????????? ?????? ?????????? ?????? ??????????

???? ???? ?????? ?????????? ?????????? ?????? ?????????? ?????????? ???????? ??????????.

1. \*\*?????? ???? ?????? ?????? (Health Unit)\*\*: ?????? ?????? ?? ??? ??? ?????? ??????? ??????? ?? ??????? ????

=====

???? ??? ?????? ?????? ?????????? ?????????? ?????? ?? ???? ?????? ?????? ?????????? ?????????.

1. \*\*???? ???? (Facility Scope)\*\*: ??? ????? ?? ????? ???? ???? ?????? ??????? ???? ???? ????.
2. \*\*???? ???? (Department Scope)\*\*: ????? ?? ????? ???? ????? ?? ????? ???? ????? ???? ?????.
3. \*\*???? ????? (Encounter Scope)\*\*: ??? ?????? ?????? ?????? ?????? ???? ???? ?????? ?? ????? ??????  
??? ????? ?????? ?????.

=====

???? ???? ????? ??????? ????????? ???? ????????????? ??????? ?????? ???????????.

```
1. **?????? ? ???? ????*?: ????? ???? ? ? ???? ???? ???? ???? ???? ???? `false` ?????.
```

```
2. **????? ???? ?????? (DTO Validation)*?: ????? ? ? ???? ???? ???? ???? ???? ???? ??
```

```
?,
```

```
3. **????? ???? ?????*?: ????? ? ???? ? ? OpenAPI ????? ???? ???? ????.
```

=====

???? ??? ?????? ?????? ?????? ???????? ?????????? ?????????? ?????? ?????? ?????????? ?????? ?????????? ??????????

1. **\*\*?????? ?? ?????? ??????? (Final Buttons)\*\*:** ??? ?????? ??????? ??????? (?? ????? ?? ????? ????? ???????) ?? ?????? ?????? ??????? ??????? ??????.
2. **\*\*????? ????????? ????????? (PHI)\*\*:** ?? ????????? ??????? ?? ?? ?????? ??? ?????? ?????? ?????? ?????? ?????? ???????.
3. **\*\*??????? ?????????? (Guards)\*\*:** ?????? ??? ?????? ??? ?????????? ?????????? ?????? ?? ?????? ?????????? ?? ?????? ? ??????? ?????????.

FILE: docs\governance\enterprise-hospital-platform\ENTERPRISE\_RBAC\_ACTION\_MATRIX\_AR.md

???? ??? ???????? ?????????? ???????? ??? ??? ?????? ?????????? ?????????? ?????????? ??? ?????????? ??????????

```
| ????? ?????? | ?????? ??????? | ?????? ?????????? | ?????? ?????????? | ??????? ???????????? | ?????? ??????? |
| :--- | :---: | :---: | :---: | :---: | :---: |
| **???? (Physician)** | ?????? | ?????? (?????) | ?????? | ?????? | ?????? |
| **????? (Pharmacist)** | ?????? | ?????? | ?????? (??????) | ?????? | ?????? |
| **???? ????? (Billing)** | ?????? | ?????? | ?????? | ?????? (??????) | ?????? |
| **???? ?????? (Admin)** | ?????? | ?????? (?????) | ?????? (??????) | ?????? (??????) | ?????? |
| **???? (Auditor)** | ?????? | ?????? | ?????? (??????) | ?????? (??????) | ?????? |
```

FILE: docs\governance\enterprise-hospital-platform\EXISTING\_APPOINTMENTS\_QUEUE\_CAPABILITY\_REVIEW\_AR.md

???? ?? ?????? ????? ????????? ?????????? ?????????? ?????? ??? ????????? ?????? ??? ?????? ?????? ?????? ???  
?? ????? ???????.

1. \*\*???? ???? ??????? (Appointment Booking)\*\*: ??? ???? ???? ?????? ????? ???? ??????? ???? ??????? ??????  
? ??????.
2. \*\*????? ?????? ?????? (Patient Queue Board)\*\*: ??? ?????? ???? ????? ?????? ????????? ??????? ??????  
???? ?????????? ??????
3. \*\*????? ?????? ????????? (Triage & Priority)\*\*: ?? ???? ???? ????? ?????? (???? ??) ?????? ?????? ??  
?? ?????????.
4. \*\*???????? ???? ??????? (Department Routing)\*\*: ?????? ????? ?????? ?????? ??? ?????? ?????? ?????? ??????  
.

FILE: docs\governance\enterprise-hospital-platform\EXISTING BILLING CLAIMS CAPABILITY REVIEW AR.md

???? ?? ?????? ????? ????????? ?????????? ?????????? ?????????? ?????????? ?? ?????? ?????????? ?????????? ???  
?? ?????????? ????

1. **\*\*????? ?????? ???????? (Charge Capture)\*\*:** ??? ???? ???? ??? ?????? ?????? ?????? ?????? (?? ????????? ? ?????????) ??? ????? ??????? ???????.
2. **\*\*??????? ?? ?????? ???????? (Insurance Eligibility)\*\*:** ?????? ??? ?????? ?????? ???????? ?????? ???????? ??? ???

3. \*\*????? ????????? (Claim Draft)\*\*: ????? ?????? ?????? ?????? ?????? ?????? ?? ?????? ?????????? ??? ?? ?????? ???  
?? ?????.

4. \*\*????????? ??????? (Financial Compliance)\*\*: ??? ?????? ??????? ?????? ?????????? ?????????? ?????????? ?????? ?????  
????? ?????????.

```
?????? ?????? ???????? ???????? ???????? ??????? ??????????? ????????????
```

```
??????? ???????? ?? ???????
```

1. **\*\*????? ?????? (Lab Orders)\*\*:** ???? ????? ????? ?????? ????? ????????? ????????? ???? ????? ????????? ????????? ?????? ??????.
2. **\*\*????? ?????? (Radiology Orders)\*\*:** ???? ?????? ??? ????????? ????????? ????? ??? ?????? ??? ?????? ??????.
3. **\*\*???????? ????????? (Prescriptions)\*\*:** ???? ????????? ????????? ????? ?????? ????????? ????????? ?????????? ??? ?? ???.
4. **\*\*????????? ?????????? (Procedures & Consultations)\*\*:** ???? ?????? ????? ?????? ?????????? ??? ?????? ?????? ?????? ?????? ?????? ??? ???????.

```
?????? ?????? ???????? ???????? ???????? ???????? ???????????? ??????????
```

???? ??? ?????? ?????? ?????????? ?????????? ?????????? ?????????? ??????? ??????? ??????? ?????????? ???????.

```
?????? ?????? ?? ??????
```

1. \*\*????? ???? ???? (Patient Demographics)\*\*: ??? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? (MRN) ??? ? ????.
2. \*\*???? ???? ???? (Nursing Vitals)\*\*: ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????.
3. \*\*???? ???? ???? (SOAP Notes)\*\*: ??? ???? ???? ???? ???? ???? ???? SOAP ???? ???? ???? ????.
4. \*\*??? ???? ???? (Lab & Radiology Orders)\*\*: ???? ???? ???? ???? ???? ???? ???? ???? ????.

```
????? ?????? ??????? ??????? ?????????? ?????????? ??????
```

???? ?? ?????? ???? ??????? ????????? ????????? ??????? ????????? ????????? ????????? ?????????  
 ???????????.

## ?????? ?????? ?? ?????

1. \*\*????? ?????? ??????? (Prescriptions)\*\*: ?????? ?????? ??????? ??????? ?????? ??????? ???????  
? ?????? ??????.
2. \*\*????????? ?????? ?????? (Medication Safety)\*\*: ??? ?????? ??????? ??? ?????? ?????? ??????? ?? ??????? ?????  
?? ?? ??????.
3. \*\*????? ?????? ??????? (FEFO Inventory)\*\*: ?????? ?????? ?????? ??????? ??????? ?????? ?? ?????? ?????? ???????  
? ??????.
4. \*\*????? ?????????? ?????? (Dispense Preview)\*\*: ?????? ?????? ??? ??? ??????? ?????? ??????? ??????? ?????? ?? ??  
???? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\EXISTING\_RESULTS\_DOCUMENTATION\_CAPABILITY\_REVIEW\_AR.md

=====

# ?????? ?????? ??????? ??????? ??????? ??????? ???????

???? ??? ?????? ?????? ??????? ??????? ??????? ??????? ??? ?????? ??????? ?????? ??????? ??????? ??????  
????????? ??????.

## ??????? ??????? ?? ?????

1. \*\*?????? ?????? ??????? (Results Inbox)\*\*: ??? ?????? ??? ?????? ?????? ??????? ??????? ??????? ??????  
.
2. \*\*????????? ?????? ????????? (SOAP Notes)\*\*: ?????? ?????????? ?????????? ?????? ?????? ?????? ??????  
.
3. \*\*????????? ??????? ?? ????????? (Abnormal Findings)\*\*: ??? ?????? ?????? ??????? ?????? ??????? ?? ?????? ???  
???? ?????? ?????? ??????.
4. \*\*?????? ?????????? ????????? (Medical Records)\*\*: ??? ?????????? ??????? ?????????? ?????? ?????? ??? ?? ??????  
.

=====

FILE: docs\governance\enterprise-hospital-platform\FACILITY\_API\_OPENAPI\_EXTENSION\_AR.md

=====

# ?????? ??????? ?????? ??????? ??????? ???????

????? ?????? ?????? ??? API ??????? ?????? ?????? ?????? ?????????? ?????? ??????? ?????????.

=====

FILE: docs\governance\enterprise-hospital-platform\FACILITY\_CUSTOMIZATION\_ENGINE\_AR.md

=====

# ??? ?????? ?????? ??????? ?? ???????

???? ?????? ?????? ?? ?????? ??? ??? ?????? ?????? ??? ?????? ?????? ?????? ?????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\FACILITY\_DEPARTMENT\_TEMPLATE\_MATRIX\_AR.md

=====

# ?????? ?????? ??????? ??????? ???????

???? ?????????? ?????? ??????????? ?????????? ?????????????? ??? ??? ?????? (????: ?????? ?????? ??????? ?????? ?????? ??  
??????).

=====

FILE: docs\governance\enterprise-hospital-platform\FACILITY\_ERD\_EXTENSION\_AR.md

=====

# ?????? ???? ????? ??????? ??????? ???????

????? ?????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ??????? ???????.

=====

FILE: docs\governance\enterprise-hospital-platform\FACILITY\_SELECTOR\_TEST\_REPORT\_AR.md

=====

# ?????? ??????? ?????? ?????? ??????

????? ?????? ??????? ??????? ?? ??? ??????? ??????? ??????? ?????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\FACILITY\_SWITCHER\_UI\_IMPLEMENTATION\_REPORT\_AR.md

=====

# ?????? ?????? ???? ??????? ?? ???? ??????

????? ?????? ???? ?????? (Facility Switcher) ?? ?????? ?????? ?????? ?????? ??????? ?????? ???? ??? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\FACILITY\_TEST\_CASES\_AR.md

=====

# ?????? ??????? ?????? ?????? ??????

??? ??????? ?????? ?? ??? ??????? ??????? ?????? ?????? ?????? Switcher ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\FACILITY\_TYPE\_MODEL\_AR.md

=====

# ?????? ?????? ?????? ?????? ??????

????? ??????? ??????? (????? ?????? ?????? ???? ?????? ?????? ???? ?????? ???? ?????? ???? ????)  
.

=====

FILE: docs\governance\enterprise-hospital-platform\FEFO\_INVENTORY\_PREVIEW\_UI\_REPORT\_AR.md

=====

# ?????? ?????? ?????? ?????? ?????? FEFO (FEFO Inventory Preview)

???? ?????? ????? ?????? ???? ?????? ?????? ??????? ????????

## ?????? ??????? ???????

- 1. \*\*????? ?????? ???????\*\*?: ??? ??? ??????? ?????? ??????? ?????? ??????? ??????? (FEFO Rank).
- 2. \*\*????? ??????? ?????? ???????\*\*?: ????? ?? ??? ??????? ??????? ??? ????? ?? ?????? ??????? ??????
- .

=====
FILE: docs\governance\enterprise-hospital-platform\FINAL\_ACTION\_GUARD\_SWEEP\_REPORT\_AR.md
=====

# ????? ??? ??? ?????? ??????? ???????

???? ?????? ??????? ?????? ?????? ??????? ??????? ?????? ??????? ???????

## ?????? ?????? ???????

- 1. \*\*????? ???????\*\*?: ?????? ?? ?????? ??????? ??????? ?? ??? ?????? ?? ?????? ?????? ??????? ????????
- 2. \*\*????? ???????\*\*?: ?????? ?? ?????? ??????? ??????? ??????? ?????? ?? ?? ?????? ?????? ????
- 3. \*\*????? ??????? ???????\*\*?: ?????? ??????? ??????? ??????? ?????? ??????? ?? ??? ?????? ???.

=====
FILE: docs\governance\enterprise-hospital-platform\FINAL\_EXECUTION\_ROADMAP\_AR.md
=====

# ????? ?????? ??????? ??????? ???????

?????? ??????? ?????? ??? ?????? ??????? ??????? ???????

=====
FILE: docs\governance\enterprise-hospital-platform\GLOBAL\_DEPARTMENT\_CATALOG.json
=====

{
"description": "Hierarchical clinical catalog of all 100+ subspecialties."
}

=====
FILE: docs\governance\enterprise-hospital-platform\GLOBAL\_DEPARTMENT\_CATALOG\_AR.md
=====

# ?????? ?????? ?????? ????????

?????? ?????? ?????? ?????? ??? 100 ????????

=====
FILE: docs\governance\enterprise-hospital-platform\GLOBAL\_HOSPITAL\_BENCHMARK\_AR.md
=====

# ??????? ??????? ????????



????? ???? ???? ?????????? ?????????? ??? Epic ? Cerner.

=====

FILE: docs\governance\enterprise-hospital-platform\GLOBAL\_REQUIREMENTS\_GAP\_MATRIX\_AR.md

=====

# ?????? ?????? ?????????? ?????? ??????????

????? ?????????? ?????????? ?????????? ?????????? ?????????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\I18N\_TRANSLATION\_PLAN\_AR.md

=====

# ??? ?????????? ?????? ??????? ??????? i18n

????? ?????? ?????????? ?????????? ??? ???? RTL/LTR ?????????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\INPATIENT\_ENCOUNTER\_UI\_REPORT\_AR.md

=====

# ?????? ?????? ??????? ?????????? ?????????? (Inpatient)

????? ?????????? ?????? ?????????? ?????????? ?????????? ?????????? ?????????? ??????????.

## ?????? ?????????? ??????????

1. \*\*????? ?????????? ?????????? ??????????\*\*: ?????????? ?????? ?????? ?????????????? ??? ?????????? ?????????? ?????????? ??????????.

2. \*\*????? ?????????? ?????????????? ??????????????\*\*: ?????? ?????? ?????? ?????????????? ?????????????? ?????????????? ??????????.

3. \*\*????? ?????????? (Inpatient Timeline)\*\*: ?????? ?????? ?????? ?????????? ?? ??? ?????????? ??? ?????????? ?????????? ?? ??????????

?.

=====

FILE: docs\governance\enterprise-hospital-platform\INTEGRATION\_TEST\_PLAN\_AR.md

=====

# ??? ?????????? ?????????? ??? ??????????????

????????? ?? ?????? ?????????????? ?????????????? ?????????? ?????????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\LAB\_ORDERS\_UI\_REPORT\_AR.md

=====

# ?????? ?????? ?????????? ?????? ?????????? (Lab)

????? ?????????? ?????? ?????????? ?????? ?????????? ??????????.

## ????? ?????? ??????

1. \*\*????? ?????? ??????\*: ????? ?????? ??? ?????? ?????? ?????? ?????? ?????? (?? ???? ??? ?????? ?????? ?????).
2. \*\*???? ?????\*: ????? ?????? ?? ????? ?????? ?????? ?????? ??? ?????? ?? ?????? ?? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\LAB\_RESULTS\_UI\_REPORT\_AR.md

=====

# ????? ?????? ?????? ?????? ?????? (Lab Results)

???? ?????? ?????? ?????? ??? ?????? ?????? ??????.

## ????? ?????? ??????

1. \*\*??? ??????\*: ??? ?????? ??? ?????? ?????? ?????? ?????? (?? ????????????? ?????? ?????).
2. \*\*??????? ?????? ?? ??????\*: ????? ??? ?????? ?????? ?????? ?? ?????? ?????? (?? ALT).

=====

FILE: docs\governance\enterprise-hospital-platform\LEGAL\_AND\_COMPLIANCE\_DOCS\_AR.md

=====

# ?????? ?????????? ?????????? ?????????? ??????????

??????? ?????????? ?????????? ?????? ?????????? ?????????? ?????????.

=====

FILE: docs\governance\enterprise-hospital-platform\LIVE\_INTEGRATION\_BLOCKER\_CHECKLIST\_AR.md

=====

# ????? ?????? ?????????? ?????? ?????????? ?????? (Live Integration Blockers)

???? ??? ?????? ?????? ?????????? ?????? ?????????? ??? ??? ?????? ??? ?????? ??? ?????? ?? ??? ?????? ?? ?????? ?? ?? ??????.

## ????? ?????????? ?????????? (Blockers Checklist)

1. \*\*????? ?????? ?????????? ?????? ?????? ?????? (RLS)\*: ?????? ?? ??? ?????? ?????? ?????????.
2. \*\*????? ?????? ?????????? ?????? ??????\*: ?????? ?????? ?????? ?????? ?????? ?????? ?????? ??????.
3. \*\*????? ?????? ?????? ?????????? (Staging)\*: ?????? ?????????? ?????????? ?????????? ?????? ?????????? ?????? ?.

=====

FILE: docs\governance\enterprise-hospital-platform\LOCAL MOCK\_API\_ARCHITECTURE\_AR.md

=====

# ??? ?????? ?????? ?????????? ?????? (Local Mock API Architecture)

\* \*\*?????:\* ??? ??? ?????? (NamaMedical ERP)

\* \*\*?????:\* ?????? ?????? ?????????? ?????????? ?????????? (PHASE\_LOCAL MOCK\_API\_RUNTIME\_PROTOTYPE\_NO\_S  
TAGING\_NO\_PRODUCTION)

\* \*\*?????:\* ??? ?????? ?????? ?????? ?????????? ?????? ?????? ?????? ?????? ?????? ?????? ?

```
?????.

1. ??? ???? ???? ???? (Data Flow Architecture)

???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? (In-Memory Sandbox). ??? ???? ????
?? ???? ???? ???? ?????:

```mermaid
sequenceDiagram
    participant UI as ??? ???? ????
    participant Runtime as ??? ???? (Runtime)
    participant Contracts as ??? ???? (Contracts)
    participant Safety as ??? ???? (Safety Guards)

    UI->>Runtime: ?????? getMockApiResponse(resource, params)

    rect rgb(30, 41, 59)
        Note over Runtime, Safety: ??? ???? ????
        Runtime->>Safety: ??? assertNoLiveEndpoint(endpoint)
        Safety-->>Runtime: ?????? ???
        Runtime->>Safety: ??? assertNoPhiPayload(payload)
        Safety-->>Runtime: ?????? ??
        Runtime->>Safety: ??? assertNoWriteOperation(action)
        Note over Safety: ??? ???? ???? (POST) -> ???
    end

    alt ????? (GET)
        Runtime->>Contracts: ?????? ??? DTO_SCHEMAS
        Contracts-->>Runtime: ?????? ????? 100% ???
        Runtime->>Runtime: ??? buildReadOnlyResponse()
        Runtime-->>UI: ????? ????
    else ????? (POST)
        Runtime->>Runtime: ??? buildErrorResponse()
        Runtime-->>UI: ????? ???? (BLOCKED)
    end
```

2. ????? ???? ???? (Core Architectural Components)

???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ?????:

?. ??? ???? ???? (Anonymized Mock Store)
* ????? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????
* ?? ???? ???? ???? ???? ???? ???? ???? ???? ???? DTO ????? ???? ?????.

?. ??? ???? ???? (Assertion Engine)
* **`assertNoWriteOperation`**:?? ???? ????
* **`assertNoLiveEndpoint`**:?? ???? ???? ???? ???? ???? ???? ????
* **`assertNoPhiPayload`**:?? ???? ???? ???? ???? ???? ???? ????.
```

```
?. ????? ?????? (Contract Enforcer)
* ????? ?????? [enterprise-contracts.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/enterprise-contracts.js).
* ????? ?????? ?? ????? ?????? ?? ????? ?? ?????? ?? 10 ?????? ?? ?????? ?????? ??? DTOs ?????? ??????
?? ????? ?????? ?????? ?? ?????? ?????? ??????.

?. ????? ?????? ?????? (Simulated Audit Generator)
* **`getMockAuditPreview`:** ????? ?????? ????? ?????? ????? ?????? ?????? ?????? ?????? (???
?? ?? ??????) ?????? ?????? ?????? ?????? CBAHI ?? ?????? ?? ??????.

?????: ????? ?? ?????? ?????? ?????? ?????? ?? ?????? ?? ?????? ?? ?????? ?????? ?????? ?????? ?
????? ?? ?????? ?????? ?????? ?????? ??????.

=====
FILE: docs\governance\enterprise-hospital-platform\LOCAL MOCK API SECURITY REVIEW_AR.md
=====

????? ?????? ?????? ?????? ?????? ?????? (Local Mock API Security Review)

* **?????:** ????? ?? ?????? (NamaMedical ERP)
* **?????:** ?????? ?????? ?????? ?????? ?????? ?????? (PHASE_LOCAL MOCK API_RUNTIME_PROTOTYPE_NO_S
TAGING_NO_PRODUCTION)
* **?????:** ?? ?????? ?????? ?????? ?? ?? ?????? ?? ?? ?????? (Approved - Zero Risk)

1. ????? ?????? ?????? (Active Security Controls)

?? ?????? ?????? ?? ?????? ?????? ?????? ?????? [mock-api-runtime.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/mock-api-runtime.js) ?????? ?? ?????? ?????? ??????:

?. ?????? ?????? (Environment Control Flags)
* ?? ??? ?????? ?????? ?????? ?????? ?? ??????:
 * `mockApiRuntimeEnabled = true` (????? ?? ??????).
 * `liveApiRuntimeEnabled = false` (????? ?? ?????? ??????).
 * `writeRuntimeEnabled = false` (????? ?? ??????).
* ??? ?????? ?????? [enterprise-contracts.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/enterprise-contracts.js) ??????:
 * `isLiveEndpointEnabled() = false`
 * `isWriteOperationEnabled() = false`

?. ??? ?????? ?????? (Blocking Write Operations)
* ?? ?????? ?????? `assertNoWriteOperation(action)` ??? ??????.
* ?? ??? ?????? ?????? ?? ?????? ?? ?? `POST`, `PUT`, `DELETE`, `PATCH` ?? ?? ?????? ?????? ??? `FINAL_SIGNATURE`
`?? `FINAL_DISPENSE` ?? `SUBMIT_CLAIM` ?????? ?????? ?????? ?????? ?????? ??????:
 ``json
 {
 "status": "BLOCKED",
 "reason": "WRITE_OPERATION_DISABLED",
 "message": "WRITE_OPERATION_DISABLED: Cannot perform write/final actions in local mock runtime."
 }
```

[illegible]

namaweb/local\_mock\_api\_runtime\_test.js) ?????? ????????? ?? ??????? ??????? ??????? ??????? ???????.

### ?????? ?????? ?????? ??????????:

- 1. **\*\*\*\*? ???? ?????????? ??????? (No Network / Zero-Fetch Check):\*\***  
\* ?? ???? ???? ????????? [mock-api-runtime.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/mock-api-runtime.js) ? [local-api-preview-ui.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/local-api-preview-ui.js) ??????? ??????? ?????? ?? ?????????? ?????????? `fetch` ?? `XMLHttpRequest`.  
\* **\*\*\*\*?????:\*\*** ????? (PASS) ?.
- 2. **\*\*\*\*? ???? ??????? ??????? (Write Blocking Check):\*\***  
\* ?? ????????? ????? ?????????? ??????? ?????? `POST` ??????? ????? ??????? ?????????? ??????? ??? ?????? `WRITE\_OPERATION\_DISABLED`.  
\* **\*\*\*\*?????:\*\*** ????? (PASS) ?.
- 3. **\*\*\*\*? ?????? ??????? ?????????? ????? Staging (Environment Flags Check):\*\***  
\* ?? ??????? ?? ????? ?????????? `liveApiRuntimeEnabled` ? `writeRuntimeEnabled` ? `isLiveEndpointEnabled()` ? `isWriteOperationEnabled()` ??????? ?????????????? **\*\*`false`\*\***.  
\* **\*\*\*\*?????:\*\*** ????? (PASS) ?.
- 4. **\*\*\*\*? ??????? ?????????? ??? 10 ?????? ?????????? (DTO Compliance Check):\*\***  
\* ?? ?????????? ?? ????? ?? ?????????? ??? 10 ?? ??? ?????????? (`GET`) ??????????? ?? ??????? DTO ??????? ??? ?? ??? ??? ????? ?????????? ??????? ?????? ?????????????? ???????.  
\* **\*\*\*\*?????:\*\*** ????? (PASS) ?.
- 5. **\*\*\*\*? ?????? ??????? ??????? ????? PHI (Secrets & PHI Prevention Check):\*\***  
\* ??????? ?? ??? ??????? ?????????? ?????????? ?? ?? ?????? ????? ?? ??????? ??????? ?? ?????????? ????? ???????.  
\* **\*\*\*\*?????:\*\*** ????? (PASS) ?.

---

## 2. ??? ??????? ?????? ?????????? ??????? (Test Console Output)

```
```text
Running Local Mock API Runtime Integration Tests...
? Core runtime flags and contract safety guards verified.
? Network-free constraints verified (No fetch/XHR).
? Security boundaries verified (No external URLs, secrets, or PHI).
? Write blocking safety guards verified.
? Contract-backed mock responses verified for all 10 resources.
All Local Mock API Runtime Integration Tests Passed!
```

****?????:** ?? ?????? ??? ??????? ??????? ?????????? ?????????? ?????????? ?????????? ??????? ?????????????? ??? ?????? ????? ?
????? ?? ?? ?????? ?????????? ??? ?????????? ?? ??? Staging.
```

=====

FILE: docs\governance\enterprise-hospital-platform\MEDICAL\_CITY\_SELECTOR\_FLOW\_AR.md

=====

# ?????? ?????????? ?????????? ??????????

????? ?????? ??????? ?????????? ??? ?????????? ?????????? ?????????? ?? ??????????? ?? ?????????? ??????? ?????????? ?????????? ?????????? ???????

??????.

=====

FILE: docs\governance\enterprise-hospital-platform\MEDICATION\_ORDERS\_UI\_REPORT\_AR.md

=====

# ????? ????? ????? ????? ?????? ???????? (Medication)

???? ?????? ?????? ?????? ???????? ???????? ???????.

## ????? ???????? ????????

1. \*\*????? ???????? ????????\*\*: ?????? ?????? ???????? ?? ???????? ?????? ???????.
2. \*\*??? ?????? ?????????\*\*: ?????? ???????? ?????? ??? ???????? ?? ?????? ?? ??? ?????? ?????? ???????.

=====

FILE: docs\governance\enterprise-hospital-platform\MIGRATION\_SCRIPTS\_PLAN\_AR.md

=====

# ??? ???????? ??? ??? ??? ????????

???? ??? ?????? ?????????? ???????? ???????? ?? ?????? ???.

=====

FILE: docs\governance\enterprise-hospital-platform\MIGRATION\_STRATEGY\_NO\_DDL\_AR.md

=====

# ?????????? ?????? ?????? ?????????? ?????????? ???????? ???????

???? ??? ?????????????? ???????? ?????????? ?????????? ?????? ?????? ?????????? ???????? ???????? RLS ??? ???????? ??? ?  
??????? ???????? ???????.

## ?????? ???????? ??????

1. \*\*????? ?????????? ?? ??? ???????? (Dry-run)\*\*: ?????? ?????????? ?? ?????? ???????? ???????? ???????? ?? ?????? ???????  
????????? ???????.
2. \*\*????????? ?????????? (Rollback Strategy)\*\*: ?????? ??? ???? ???? ?????????? ???????? ?? ??? ??? ? ? ? ???? ?  
??? ???????.
3. \*\*????? ??? ???????? (Lock Mitigation)\*\*: ?????? ?????????? ??? ?????? ?????? ?????? ?????????? ??? ???? ???? ????  
?? ?????????? ???????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\MOCK\_ADAPTER\_LAYER\_REPORT\_AR.md

=====

# ?????? ??? ?????????? ???????????? (Mock Adapter Layer)

???? ?????? ?????? ?????? ??? ?????????? ???????? ?????????? ?????????? ??????????.

## ?????? ???????? ????????

1. \*\*????? ???????????? (Mock Responses)\*\*: ?????? ?????????? ???????? ?????????? ?? ???????? OpenAPI ???????.
2. \*\*??? ???????? ????????\*\*: ?????? ?? ?????? ???????? ?????????? ?????????? ?????????? ?????????? ?? ??? ?????? ???  
???

=====

FILE: docs\governance\enterprise-hospital-platform\MODULE\_CONTRACT\_INDEX\_REPORT\_AR.md

=====

# ????? ???? ???? ???????? ?????? (Module Contract Index)

???? ?????? ???? ???????? ???????? ???????? ???????? ???????? ????????.

## ????? ?????? ???????

1. \*\*???? ????? ???????? (DTOs)\*\*: ????? ???? ????? ???????? ???????? ???????? ???????? ????????.
2. \*\*???? ????? ????????\*\*: ????? ???? ????? ?????? ?? ????? ???????? ???????? ?????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\NPHIES\_READINESS\_CHECKLIST\_REPORT\_AR.md

=====

# ????? ?????? ????? ???????? ????? ???? (NPHIES Readiness)

???? ???????? ???????? ???????? ???????? ?????? ???????? ???????? ?????? ?? ?????? ???????? ???????? ?????? (????)  
.

## ?????? ???????? ???????

1. \*\*????? ?????? ????????\*\*?: ????? ???? ????? ?????? ?? ????? ???? ?????? ?????? ?????? ??? ?????? ??????
2. \*\*??? ?????? ????????\*\*?: ????? ?? ????? ?????? ?????? ?? ???????? ???????? ??? ?????? ??? ???????? ??? ? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\NPHIES\_READINESS\_PLAN\_AR.md

=====

# ??? ?????? ???????? ?? ???? ???? ???????

????? ???????? ?????????? ???????? ?????????? ?????????? ???????.

=====

FILE: docs\governance\enterprise-hospital-platform\OPD\_ENCOUNTER\_UI\_REPORT\_AR.md

=====

# ????? ?????? ?????? ?????? ???????? ???????? (OPD)

???? ?????? ?????? ?????? ?????? ???????? ???????.

## ????? ?????? ???????

1. \*\*??????? ????????\*\*?: ????? ???? ???????? ?????? ?????? ?????? ?? ?????? ?????? ???????.
2. \*\*??????? ?????? (SOAP)\*\*: ?????? ?????? ???????? ???????? ???????? ??????.
3. \*\*????? ???????? (OPD Timeline)\*\*: ??? ???? ???? ???? ?????? ??? ?????? ???????? ??????.

=====



=====

???? ??? ?????? ?????? ?????? ?????? ?????? ????????? (OPD) ?????? ????????? ????????? (Inpatient).

1. **\*\*????? ?????? / ??? ???????\***: ??? ??? ? ???? ???? ???? ???? ????.
2. **\*\*????? ??????\***: ???? ???? ???? ???? ???? ???? ??
3. **\*\*????? ?????????\***: ???? ???? ???? ? ???? ???? ???? ??

1. \*\*??? ??????\*: ?? ????? ???? ???? ????? ?? ?? ????? ?????.
2. \*\*???? ??????: ????? ???? ???? ???? ???? ???? ???? ????.
3. \*\*???? ???? ???? ???? ????\*: ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ??  
?
4. \*\*??? ??????: ????? ??? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????.

=====

=====

????? ???? ?????? ?????????? ??????????? ??????????? ?? ?????????? ?????????????.

=====

=====

???? ???? ???? ???? ???? ???? ???? ???? ???? ????.

1. **\*\*???? ???? ???????\*\***; ??? ??? ????????? ???? ????????? ?????? ?????? ???? ????????? ??????? ???????.
2. **\*\*????? ???? ???? ???????\*\***; ????? ?????? ????? ???? ???? ????????? (??????? ?????? ??????? ???????) ?????? ?  
????? ???????.

=====

=====

????? ?????? ?????? ??????????? ???????? ???????? ?? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHARMACY\_INVENTORY\_TEST\_REPORT\_AR.md

=====

# ????? ???????? ?????? ???????? ?????? FEFO

???? ???? ?????? ???????? ???????? ?????? ???????? ???????? ???????? FEFO.

## ?????? ????????????

1. \*\*?????? ???? ?????? ????????\*\* : ?????? ?????? ?? ?? ????? `canFinalizeDispense` ????? ?????? `false` ?????? ?? ?  
??? ???????.
2. \*\*??????? ?????? FEFO\*\* : ?????? ?? ??? ?????? ?????? ???????? ???????? ?????? ?????? ?????? ??????  
??.
3. \*\*??????? ???? ???????? ???????? (Health Unit)\*\* : ?????? ?? ??? ???????? ???????? ???????? ???????? ???????? ??????  
? FEFO ?????? ???????? ???????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHARMACY\_REVIEW\_QUEUE\_UI\_REPORT\_AR.md

=====

# ????? ?????? ?????? ?????? ?????? ???????? (Pharmacy Review Queue)

???? ?????? ?????? ?????? ?????? ?????? ???????? ???????.

## ?????? ???????? ????????

1. \*\*?????? ???????? ????????\*\* : ??? ?????? ???????? ???????? ???????? ???????? ???????? ????????.
2. \*\*?????? ???????? ????????\*\* : ?????? ?????? ??? ??? ?????? ???????? ???????? ?? ?????? ???????? ????????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_APPOINTMENTS\_QUEUE\_CLOSEOUT\_AR.md

=====

# ????? ?????? ?????? ?????? ???????? ???????? ???????

???? ???? ?????? ???? ???????? ???????? ???? ???????? ???????? ???????? ???????? ???????.

## ???? ????????

1. \*\*??????? ???? ????????\*\* : ?? ???????? ?????? ?? ??? ?????? ?????? ??? ???????? ???????? ?????? ?????? ?????? ?  
???? ???????.
2. \*\*?????? ?????? ?????? ?????? ????????\*\* : ?? ?????? ?????? ?????? ?????? ?????? ?????? ?? ?? ?????? ?????? ?????? (P  
HI) ?? ?????? ?????? ??????.
3. \*\*??????? ???????? ???????? ????????\*\* : ?????? ?????? ?????? ?????? ?????? ?? ?????? ?????? ???????? ???????? ??????  
??????? ???????? ???????? (Clinical Billing & Claims Core).

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_APPOINTMENTS\_QUEUE\_IMPLEMENTATION\_AR.md

=====

# ??? ?????? ?????? ???????? ???????? ???????

??? ??? ???? ?????? ?????? ??? ???????? ???????? ???????? ?? ???????? ???????.

## ?????? ????????

- 1. **\*\*????? ??????? ???????\*\***: ????? ???? ??????? ?????? ??????? ???? ????? `public/js/appointments-queue.js` .
- 2. **\*\*????? ??????? ???????\*\***: ??? ???? `Patient Queue Board` ?????? ?????? ??? ?????? ?????? ?????? ?????? ?????? ? ????????
- 3. **\*\*????? ??????? ???????\*\***: ??? ?????? ?????? ??????? ??????? (Inpatient Admissions) ?? ?????? ??????? ??????? ?????? ?????? ??????? ??????? ?????? ????????

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_APPOINTMENTS\_QUEUE\_PREFLIGHT\_AR.md

=====

# ????? ?????? ??????? ?????? ??????? ??????? (Appointments & Patient Queue)

???? ???? ??? ?????? ?????? ??????? ?? ?????? ??????? ??????? ??? ?????? ??????? ???????.

## ????? ??????

- 1. **\*\*????? ?????? (Git Status)\*\***: ??? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????.
- 2. **\*\*??? ???? Diff\*\***: ?? ?????? ?? ??? ???? ?????? ?????? ?? ???????.
- 3. **\*\*????? ?????? ???????\*\***: ??? ??????? ?????? ?????? UTF-8 ?????? ?? ?? ?????? (Mojibake).
- 4. **\*\*????? ?????????\*\***: ?? ?????? ?????????? ??? 105 ?????????? ?????? 100%.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_BACKEND\_SCHEMA\_RLS\_DESIGN\_CLOSEOUT\_AR.md

=====

# ????? ?????? ?????? ?????? ?????? ??????? ?????? RLS ???????

???? ???? ?????? ???? ??????? ??????? ???? ??????? ??????? ??????? ??????? ???????.

## ???? ???????

- 1. **\*\*?????? ??????? ???????\*\***: ?? ??????? ???? ???? ???? ???? ???? ???? ???? RLS ???? ???? ERD ???????.
- 2. **\*\*????? ?????? ?????? ?????? ???????\*\***: ?? ?????? ???? ?????? ?????? ???? ???? ???? ???? ???? ???? ???? ???? (SDAIA).
- 3. **\*\*????? ??????? ??????? ???????\*\***: ?????? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_BACKEND\_SCHEMA\_RLS\_DESIGN\_IMPLEMENTATION\_AR.md

=====

# ??? ?????? ?????? ?????? ??????? ?????? ?????? RLS

??? ??? ?????? ?????? ?????? ??????? ??????? ??? RLS ?????? ??????? ???????.

## ?????? ???????

- 1. **\*\*????? ??? ERD ?????\*\***: ??? ?????? ??????? ???? ?????? ?????? ???????.
- 2. **\*\*????? ?????? ?????\*\***: ?????? ???? ??? RLS ?????? ??????? ??????? ?? ?????? ?????? (CBAHI).
- 3. **\*\*????? ???? ?????\*\***: ??? ?????? ?? ?????? ?????? ?? ?????? ???? ???? ??????? ??????? ??????? ???? ???? ???? ????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_BACKEND\_SCHEMA\_RLS\_PREFLIGHT\_AR.md

=====

# ????? ????? ????????? ?????? ????? ?????? ?????????? ????? RLS (Backend Schema & RLS Design)

???? ???? ??? ?????? ??????? ????????? ?? ?????? ?????????? ?????????? ??? ???? ????????? ?????????.

## ????? ??????

1. \*\*????? ?????? (Git Status)\*\*: ???? ??????? ??? ?????? ?????? ??? ??????? ?? ?????? ???? ??? ???.
2. \*\*????? ??? Diff\*\*?: ?? ??????? ?? ??? ?????? ??????? ?????? ?? ?????????.
3. \*\*??????? ?????? ?????????\*\*?: ???? ?????????? ??????? ??????? UTF-8 ?????? ?? ?? ?????? (Mojibake).
4. \*\*????? ?????????????\*\*?: ?? ?????? ?????????????? ??? 109 ?????????? ?????? 100%.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_CLINICAL\_BILLING\_CLOSEOUT\_AR.md

=====

# ?????? ?????? ?????? ?????? ?????????? ?????????? ?????????? ?????

????? ???? ???? ???? ???? ?????? ?????????? ???? ???? ?????? ?????????? ?????????? ?????????? ??????????

## ???? ?????????

1. \*\*????????? ???? ???? ?????????\*\*?: ?? ?????????? ?????? ?? ??? ?????? ???? ?????????? ?????????? ?????????? ?????????? ?????? ?? ?????.
2. \*\*????????? ??????? ?????? ??????? ?????????\*\*?: ?? ??????? ???? ?????? ?????? ?????? ?????? ?? ?? ?????? ???? ?????? (PHI) ?? ?????? ???? ??????
3. \*\*????????? ?????????? ?????????? ?????????\*\*?: ??????? ?????? ??????? ??????? ?????? ?? ?????? ??????? ?????????? ?????????? ??? ??? ??????? ???? ?????????? ??????? ??? ??????? ???? ?????????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_CLINICAL\_BILLING\_IMPLEMENTATION\_AR.md

=====

# ??? ?????? ??????? ?????????? ?????????? ?????????? ?????????? ?????

??? ??? ??? ??????? ??????? ??? ?????????? ?????????????? ?????????? ?? ?????????????? ??????????

## ??????? ?????????

1. \*\*????????? ?????????? ?????????\*\*?: ?????? ???? ?????????? ?????????? ??????? ?????????? ???? ???? ?????? `public/js/clinical-billing.js`.
2. \*\*????? ???? ???? ?????\*\*?: ??? ?????? `Clinical Billing & Claims` ??????? ?????? ??? ?????????? ?????????? ??????? ??????? ???? ???? ????.
3. \*\*????????? ??????? ?????????\*\*?: ?????? ?????? ??????? ???? (NPHIES Readiness Checklist) ??????? ?? ??????? ?????????? ? ?????????? ?????????? ??? ?????? ?????????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_CLINICAL\_BILLING\_PREFLIGHT\_AR.md

=====

# ????? ????? ????????? ?????? ??????? ????????? ?????????? (Clinical Billing & Claims)

???? ???? ??? ?????? ?????? ??????? ?? ????? ??????? ????????? ??? ?????? ??????? ???????.

## ????? ?????

1. \*\*????? ?????? (Git Status)\*\*: ??? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????.
2. \*\*??? ???? Diff\*\*?: ?? ?????? ?? ??? ???? ?????? ?????? ?? ???????.
3. \*\*?????? ?????? ?????????\*\*?: ??? ??????? ?????? ?????? UTF-8 ????? ?? ?? ????? (Mojibake).
4. \*\*????? ???????????\*\*?: ?? ????? ??????????? ???? 106 ?????????? ????? 100%.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_CLINICAL\_ORDERS\_CLOSEOUT\_AR.md

=====

# ????? ????? ????? ????? ??????? ?????? ?????????

???? ???? ?????? ??? ??????? ??????? ????? ??????? ?????? ??????? ????????? ???????.

## ??? ???????

1. \*\*?????? ????? ?????????\*\*?: ?? ??????? ?????? ?? ??? ?????? ???? ???? ??????? ?????? ??????? ?????? ??????.
2. \*\*?????? ??????? ?????? ?????? ?????????\*\*?: ?? ?????? ???? ???? ?????? ?????? ?????? ?? ?? ?????? ???? ?????? (PHI) ?? ????? ???? ?????.
3. \*\*?????? ?????????? ?????????? ?????????\*\*?: ?????? ?????? ?????? ?????? ?????? ?? ??? ?????? ??????? ????????? ??????? (LIS/RIS Integration) ???????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_CLINICAL\_ORDERS\_IMPLEMENTATION\_AR.md

=====

# ??? ????? ??????? ??????? ??????? ?????????

??? ??? ??? ?????? ?????? ??? ??????? ?????? ??????? ?? ?????????? ?????????.

## ?????? ???????

1. \*\*?????? ?????????? ?????????\*\*?: ?????? ???? ??????? ?????????? ?????? ???? ?????? `public/js/clinical-orders.js`.
2. \*\*??? ???????????\*\*?: ?????? ?? ??? ??????? ??????? ???? ???? ?????????? ?????????? ?? ??????? ??????? ?????? ?????? ? ???????.
3. \*\*??? ?????????\*\*?: ??? ???? ???? ??????? ??????? ?????? ?????? ???? ?????? ?????????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_CLINICAL\_ORDERS\_PREFLIGHT\_AR.md

=====

# ????? ????? ?????????? ?????? ??????? ?????? ????????? (Clinical Orders)

???? ???? ??? ?????? ?????? ??????? ?? ????? ??????? ?????????? ??? ?????? ??????? ???????.

## ????? ?????

1. \*\*????? ?????? (Git Status)\*\*: ??? ?????? ??? ???? ???? ???? ???? ???? ???? ???? ???? ????.
2. \*\*????? ??? Diff\*\*?: ?? ?????? ?? ??? ???? ?????? ?????? ?? ???????.
3. \*\*?????? ?????? ?????????\*\*?: ??? ??????? ?????? ?????? UTF-8 ????? ?? ?? ????? (Mojibake).

4. \*\*???? ??????????\*: ?? ????? ?????????? ??? 102 ?????????? ????? 100%.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_CLINICAL\_PHARMACY\_CLOSEOUT\_AR.md

=====

# ????? ?????? ?????? ?????? ?????????? ?????????? ??????? FEFO

???? ?????? ?????? ?????? ?????????? ?????????? ?????? ?????????? ?????????? ?????????? ??????????.

## ?????? ?????????

1. \*\*????????? ?????? ??????????\*: ?? ?????????? ?????? ?? ??? ?????? ?????? ?????????? ?????????? ??????? FEFO ?????? ??????.
2. \*\*?????? ??????? ?????? ??????? ??????????\*: ?? ??????? ?????? ?????? ??????? ?????? ?????? ?? ?? ??????? ?????? ?????? (PHI) ?? ?????? ?????? ??????.
3. \*\*????????? ?????????? ?????????? ??????????\*: ??????? ?????? ??????? ?????????? ?????? ?? ?????? ??????? ?????????? ?????????? ?????? ?????????? ??????? (Clinical Billing & Claims Core).

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_CLINICAL\_PHARMACY\_IMPLEMENTATION\_AR.md

=====

# ??? ?????? ??????? ?????????? ?????????? ?????????? FEFO

??? ??? ??? ??????? ??????? ??? ?????????? ?????????? ?????? ?????????? ?? ?????????? ??????????.

## ??????? ?????????

1. \*\*?????? ?????????? ??????????\*: ?????? ?????? ?????????? ?????????? ?????????? ?????? ?????? `public/js/clinical-pharmacy.js`.
2. \*\*????? ?????? ??????????\*: ??? ?????? `Clinical Pharmacy & FEFO` ??????? ?????? ??? ?????????? ?????????? ??????? ??????.
3. \*\*?????? ?????????? ??????????\*: ?????? ?????????? ?????? ?????????? ??? ?????? ?????????? ??????? ?? ?????? ?????????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_CLINICAL\_PHARMACY\_PREFLIGHT\_AR.md

=====

# ?????? ?????? ?????????? ??????? ?????????? ?????????? ??????? FEFO (Clinical Pharmacy & FEFO Inventory)

????? ?????? ??? ??????? ??????? ?????????? ?? ?????? ?????????? ?????????? ??? ??????? ?????????? ??????????.

## ?????? ??????

1. \*\*?????? ?????? (Git Status)\*: ??? ??????? ??? ?????? ?????? ??? ??????? ?? ?????? ?????? ??? ???.
2. \*\*????? ??? Diff\*: ?? ??????? ?? ??? ?????? ??????? ?????? ?? ?????????.
3. \*\*?????? ?????? ??????????\*: ???? ?????????? ??????? ??????? UTF-8 ?????? ?? ?? ?????? (Mojibake).
4. \*\*?????? ??????????????\*: ?? ?????? ?????????????? ??? 104 ?????????????? ?????? 100%.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_LOCAL MOCK\_API\_CLOSEOUT\_AR.md

=====

# ?????? ?????? ?????? ?????????? ?????????? ?????????????? (Phase Local Mock API Closeout)

\* \*\*?????:\* ??? ???? (NamaMedical ERP)  
\* \*\*?????:\* ?????? ?????? ?????? ?????? ?????? (PHASE\_LOCAL MOCK\_API\_RUNTIME\_PROTOTYPE\_NO\_S  
TAGING\_NO\_PRODUCTION)  
\* \*\*???? ??????:\* 30 ???? 2026  
\* \*\*???? ??????:\* ?????? ?????? ?????? (PASS) ?

---

## 1. ??? ?????? ?????? (Deliverables Summary)

?? ????? ??? ?????? ?????? ?????? ?????? ?????? ?? ????? ?????? ??????:

1. [PHASE\_LOCAL MOCK\_API\_PREFLIGHT\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/PHASE\_LOCAL MOCK\_API\_PREFLIGHT\_AR.md) - ????? ?????.
2. [LOCAL MOCK\_API\_ARCHITECTURE\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/LOCAL MOCK\_API\_ARCHITECTURE\_AR.md) - ????? ?????? ?????? ?????.
3. [CONTRACT\_BACKED MOCK\_RESPONSES\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/CONTRACT\_BACKED MOCK\_RESPONSES\_AR.md) - ????? ?????? DT0.
4. [LOCAL MOCK\_API\_SECURITY\_REVIEW\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/LOCAL MOCK\_API\_SECURITY\_REVIEW\_AR.md) - ?????? ?????? ?????? ?????.
5. [LOCAL MOCK\_API\_TEST\_REPORT\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/LOCAL MOCK\_API\_TEST\_REPORT\_AR.md) - ????? ?????? ?????? ?????.
6. [PHASE\_LOCAL MOCK\_API\_CLOSEOUT\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/PHASE\_LOCAL MOCK\_API\_CLOSEOUT\_AR.md) - (?? ?????) ?????? ?????? ?????? ??
7. [mock-api-runtime.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/mock-api-runtime.js) - ?? ? ?????? ?????.
8. [local-api-preview-ui.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/local-api-preview-ui.js) - ?????? ?????? ?????.
9. [local\_mock\_api\_runtime\_test.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/local\_mock\_api\_runtime\_test.js) - ?????? ?????? ?????.

---

## 2. ??? ?????? ?????? (Security & Testing Summary)

\* \*\*????? ??? Zero-DDL:\* ?? ?????? ?????? 100%? ?? ?? ?????? ?? ?????? ?? ?????? ?????? ??????  
\* \*\*??? ?????? ??????:\* ?? ?????? ?? ?????? ?????? ?????? ?? ?????? `fetch` ?? ?????? ?????? ?  
????? (PHI).  
\* \*\*????????? ??????:\* ?? ?????? ?????? ?????? ?????? ?????? ?? ?????? ?????? ?????.

---

## 3. ?????? ?????? (Next Phase Roadmap)

??? ??? Staging ?????? ?????? ??? ?????? ?????? ?????? ?????? ??????. ??? ??? ?????? ??? ??? ?????? ?  
????? ?????? ?????? ??????

\*\*`PHASE\_STAGING\_API\_PROTOTYPE\_INTEGRATION`\*\* (?? ???? ?????? ??? ?????? ?????? ???? ??????)  
\* \*\*?????:\* ??? ?? ?????? ?????? ??? (GET Mocks) ??? ??? Staging ?????? ?????? ?????? ??? ??? ?????? ?  
????? ?? ?????? ?????? ??? ?????? ?? ???? ????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_LOCAL MOCK\_API\_IMPLEMENTATION\_AR.md

=====

# ????? ????? ????? ??????? ??????? ????????? (Local Mock API Implementation Report)

\* \*\*?????:\* ??? ??? ????? (NamaMedical ERP)

\* \*\*?????:\* ?????? ?????? ????????? ????????? ????????? (PHASE\_LOCAL MOCK\_API\_RUNTIME\_PROTOTYPE\_NO\_S  
TAGING\_NO\_PRODUCTION)

\* \*\*???? ??????:\* ????? (Completed)

---

## 1. ?????? ?????????? ?????????? ????????? (Code Changes)

?? ?????? ??????? ?????????? ?????????? ?????????? ?????????? ?????? ??? ????? ?? ?????????? ?????????? ??? ?????????? ?????  
??? ?????? ?????????? ?????????:

1. \*\*????? ??? ??? ?????????? ????????? (Mock API Runtime):\*\*

\* ?? ?????? ?????? [mock-api-runtime.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/mock-api-  
runtime.js) ?? ??? ?????????? ?????????.

\* ?????? ??? ?????????? ?????????? ?????????? ?????? ?????????? ?????? ?????????? ??? 10.

2. \*\*????? ?????? ?????????? ?????????? (Interactive Preview UI):\*\*

\* ?? ?????? ?????? [local-api-preview-ui.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/local-  
api-preview-ui.js).

\* ??? ??? ??? ?????????? ?????????? ??? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ??????????

3. \*\*????? ?????? ?????????? ?????????? (HTML Integration):\*\*

\* ?? ?????? ?????? [index.html](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/index.html) ?????? ??  
????????? ?????????? ?????????? ?????????? ??? ?????? ??? ?? ?????????? ?????????? ?? ??? `app.js` ?????????? ?????????.

4. \*\*????? ??? ?????? ?????????? (Automated Test):\*\*

\* ?? ?????? ??? ?????? ?????????? [local\_mock\_api\_runtime\_test.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/local\_mock\_api\_runtime\_test.js) ?????????? ?????????? ?? ?????????? ?????????.

---

## 2. ??? ?????????? ?????????? ?????????? ?? ?????????? ??????

???? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ??????????

| ?????? (Resource) | ?????? ?????????? (GET) | ?????? ?????????? (POST) | ?????? ?????????? |

| :--- | :--- | :--- | :---: |

| \*\*????????? (Facilities)\*\* | `/api/v1/facilities` | `/api/v1/actions/finalize` | ?????? ?????? ?????????? / ??? ??  
?????? ?? |

| \*\*????????? (Departments)\*\* | `/api/v1/departments` | `/api/v1/actions/finalize` | ?????? ?????? ?????????? / ???  
????????? ?? |

| \*\*????????? (Appointments)\*\* | `/api/v1/appointments` | `/api/v1/actions/finalize` | ?????? ?????? ?????????? / ???  
????????? ?? |

| \*\*?????? (Queue)\*\* | `/api/v1/queue` | `/api/v1/actions/finalize` | ?????? ?????? ?????????? / ??? ?????????? ?? |

| \*\*????????? (Encounters)\*\* | `/api/v1/encounters` | `/api/v1/actions/finalize` | ?????? ?????? ?????????? / ??? ?  
????????? ?? |

| \*\*????????? (Orders)\*\* | `/api/v1/clinical-orders` | `/api/v1/actions/finalize` | ?????? ?????? ?????????? / ??? ??  
?????? ?? |





?????

---

## 3. ??? ?????? ?????? ??????

- ????? ?? ?????? ?????? ?????? ??????:
1. ?? ????? ?? ????? ?? ????? DDL ?? ????? ?????? (Zero-DDL/Zero-Migrations).
  2. ??? ????? ?????? ?????? ?????? ?????? (`isLiveEndpointEnabled = false` ? `isWriteOperationEnabled = false`)
  3. ?? ????? ?????? ????? Staging ????? ?????? ?????? ?????? ?????? ?????? (Mock/Read-Only) ??? ?? ???

---

\*\*?????:\*\* ?????? ?? ?????? ?????? ?????? ?????? ?????? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_MASTER\_DISCOVERY\_AND\_CATALOG\_CLOSEOUT\_AR.md

=====

# ????? ?????? ?????? ?????????? ?????????? ??????

???? ?????? ?????? ?????????? ?????????? ?????? ?? ?????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_MASTER\_DISCOVERY\_PREFLIGHT\_AR.md

=====

# ????? ?????? ?????????? ??? ?????? ??????

???? ?????? ?? ?????? ?????? ?????????? ?? ?????? ?????????? ?????????? ?????????? ??????

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_MODULE\_REFACTOR\_API\_CONTRACT\_CLOSEOUT\_AR.md

=====

# ????? ?????? ?????? ?????? ?????????????? ?????? ?????????? ??????????

???? ?????? ?????? ?????? ?????????? ?????????? ?????? ?????????? ?????????? ?????????? ??????????

- ## ??? ??????
1. \*\*????? ?????? ?????????? ??????????\*\*: ?? ?????????? ?????? ?? ?? ?????????? ?????? ?????????? ?????????? ?????????? ??????
  2. \*\*????? ?????? ?????? ?????? ??????????\*\*: ?? ?????? ?????? ?????? ?????? ?????? ?????? ?? ?? ?????? ?????? ?????? (PHI) ?? ?????? ?????? ??????
  3. \*\*????????? ?????????? ?????????? ??????????\*\*: ?????? ?????? ?????? ?????? ?????? ?? ?????? ?????? ?????? ?????? ?????????? ??????????

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_MODULE\_REFACTOR\_API\_CONTRACT\_IMPLEMENTATION\_AR.md  
=====

# ??? ????? ????? ?????????? ????? ?????????? ??????????

??? ??? ?????? ??? ?????????? ?????????? ?? ?????? ?????? ?????????? ?????????? ?? ??? ?????????????.

- ## ?????? ????????
1. **\*\*????? ??????? ???????\*\***: ?????? ?????????? ?????????? ?????? ?????? `public/js/enterprise-contracts.js`.
  2. **\*\*????????? OpenAPI\*\***: ??? ?????? ?????? ?????????? ?????????? ?????????? ?????? ?? ??? ?????????? ???????.
  3. **\*\*????? ??????? ???????\*\***: ??? ?????? ?? ?????? ??? ?? ?????? ?????? ?????????? ?????????? ?????????? ??????????

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_MODULE\_REFACTOR\_API\_CONTRACT\_PREFLIGHT\_AR.md

=====

# ?????? ?????? ?????????? ??????? ?????? ?????????????? ??????? ?????? ?????????? (Module Refactor & API Contract)

????? ?????? ??? ??????? ??????? ?????????? ?? ?????? ?????????? ?????????? ??? ?????? ?????????? ??????????

- ## ?????? ??????
1. **\*\*????? ?????? (Git Status)\*\***: ?????? ??????? ??? ?????? ?????? ??? ??????? ?? ?????? ?????? ??? ???.
  2. **\*\*????? ??? Diff\*\***: ?? ??????? ?? ??? ?????? ??????? ?????? ?? ??????????
  3. **\*\*?????? ?????? ?????????\*\***: ?????? ?????????? ?????????? ??????? UTF-8 ?????? ?? ?? ?????? (Mojibake).
  4. **\*\*????? ?????????????\*\***: ?? ?????? ?????????????? ??? 108 ?????????? ?????? 100%.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_OPD\_INPATIENT\_ENCOUNTER\_CLOSEOUT\_AR.md

=====

# ?????? ?????? ?????? ?????? ??????? ?????????? ??????? ??????????

????? ?????? ?????? ??? ?????????? ?????????? ?????? ?????????? ?????????? ?????????? ?????????? ??????????

- ## ??? ???? ???????
1. **\*\*????????? ??? ???? ?????????\*\***: ?? ?????????? ?????? ?? ??? ??????? ?????? ?????????? ??????? ?????????? ?????????? ?? ??? ???????.
  2. **\*\*?????? ??????? ?????? ??????? ?????????\*\***: ?? ??????? ?????? ?????? ??????? ?????? ??????? ?? ?? ??????? ?????? ?????? (PHI) ?? ?????? ?????? ??????
  3. **\*\*????????? ?????????? ?????????? ?????????\*\***: ??????? ?????? ??????? ??????? ?????? ?? ??? ?????????? ??????? ??????? ?????????? ?????????? ?????????? ??????????

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_OPD\_INPATIENT\_ENCOUNTER\_IMPLEMENTATION\_AR.md

=====

# ??? ?????? ??????? ?????????? ?????????? ??????????

??? ??? ??? ??????? ??? ?????? ?????????? ?????????? ?????????? ?????????? ??????????

## ?????? ???????

1. \*\*????? ??????? ?????????\*\*: ?????? ??? ???? ???? ???? ???? ???? ???? `public/js/encounter-workflow.js`  
.
2. \*\*??? ?????????\*\*: ?????? ?? ?????? ??????? ??????? ??? ????? ?? ????? ????? ?????? ?????? ?????? ?? ??  
????? ??????????.
3. \*\*????? ??????? ?????\*\*?: ??? ???? ??????? ????????? `renderEncounterWorkspace` ?????? ?????? ??? ???? ?? ??  
??? ?????? ???????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_OPD\_INPATIENT\_ENCOUNTER\_PREFLIGHT\_AR.md

=====

# ?????? ?????? ?????????? ??????? ?????????? ??????? ?????????? (OPD & Inpatient)

????? ???? ???? ??????? ??????? ?????????? ?? ?????? ?????????? ?????????? ??? ??????? ?????????? ??????????.

## ?????? ??????

1. \*\*????? ?????? (Git Status)\*\*: ??? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????.
2. \*\*??? ??? Diff\*\*?: ?? ?????? ?? ??? ???? ?????? ?????? ?? ?????????.
3. \*\*?????? ?????? ?????????\*\*?: ??? ???? ???? ???? ???? UTF-8 ?????? ?? ?? ?????? (Mojibake).
4. \*\*????? ?????????????\*\*?: ?? ?????? ?????????????? ??? 101 ?????????? ?????? 100%.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_PUBLIC\_FACILITY\_SWITCHER\_CLOSEOUT\_AR.md

=====

# ?????? ?????? ?????? ?????? ??????? ?????????? ?????? ?????????

????? ???? ???? ???? ??????? ?????????? ?????? ?????????? ??????? ?????????? ?????????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_PUBLIC\_FACILITY\_SWITCHER\_PREFLIGHT\_AR.md

=====

# ?????? ?????? ?????????? ??????? ??????? ?????????

????? ???? ???? ??????? ??????? ?????????? ?? ?????? ?????????? ?????????? ??? ??????? ?????????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_PUBLIC\_HOMEPAGE\_AND\_FACILITY\_SWITCHER\_IMPLEMENTATION\_AR.md

=====

# ??? ?????? ??????? ?????????? ?????? ?????????

??? ??? ???? ??????? ?????????? ?????????? ?????????? ?????????? ?????????? ??????? ??????? ??????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_RESULTS\_DOCUMENTATION\_CLOSEOUT\_AR.md

=====

# ????? ????? ????? ????? ????????? ????????? ???????

???? ????? ????? ????? ????????? ????????? ????? ????????? ????????? ????????? ?????????.

## ????? ???????

1. \*\*????? ???? ?????????\*\*?: ?? ????????? ????? ?? ??? ?????? ?????? ????????? ?????? ????????? ????????? ?????? ??????.
2. \*\*????? ?????? ?????? ?????? ?????????\*\*?: ?? ?????? ???? ?????? ?????? ?????? ?????? ?? ?? ?????? ???? ?????? (PHI) ?? ?????? ???? ??????.
3. \*\*????? ?????????? ?????????? ?????????\*\*?: ?????? ?????? ?????? ?????? ?????? ?? ?????? ?????? ?????????? ?????????? ??? ??? ?????????? ?????? ???? ?????????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_RESULTS\_DOCUMENTATION\_IMPLEMENTATION\_AR.md

=====

# ??? ????? ??????? ?????? ?????????? ?????????? ???????

??? ??? ??? ?????? ?????? ??? ?????? ?????????? ?????????? ?????????? ?????????? ?? ?????????? ?????????.

## ?????? ???????

1. \*\*????? ?????????? ?????????\*\*?: ?????? ???? ?????????? ?????? ?????????? ???? ?????? `public/js/clinical-results.js`.
2. \*\*????? ?????????? ?????????\*\*?: ??? ???? `Results Inbox` ?????? ?????????? ?????? ?????? ?????? ?????? ???? ? ?????? (????? ???? ?????????? ?????????).
3. \*\*????????? ?????????\*\*?: ?????? ???? ?????? ?????? SOAP ?????? ?????? ??? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_RESULTS\_DOCUMENTATION\_PREFLIGHT\_AR.md

=====

# ????? ?????? ?????????? ?????? ?????? ?????????? ?????????? ????????? (Results & Documentation)

???? ????? ??? ?????? ?????? ?????????? ?? ?????? ?????????? ?????????? ??? ?????? ?????????? ?????????.

## ?????? ??????

1. \*\*????? ?????? (Git Status)\*\*: ??? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????.
2. \*\*????? ??? Diff\*\*?: ?? ?????? ?? ??? ???? ?????? ?????? ?? ?????????.
3. \*\*????? ?????? ?????????\*\*?: ??? ?????????? ?????? ?????? UTF-8 ?????? ?? ?? ?????? (Mojibake).
4. \*\*????? ?????????????\*\*?: ?? ?????? ?????????????? ??? 103 ?????????? ?????? 100%.

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE\_SECURITY\_RBAC\_AUDIT\_CLOSEOUT\_AR.md

=====

# ????? ?????? ?????? ?????? ?????? ?????????????? ?????? ?????????? ?????????

???? ????? ?????? ?????? ?????????? ?????????? ?????? ?????????? ?????????? ?????????? ?????????.

## ?????? ???????



1. [PHASE\_STAGING\_READINESS\_PREFLIGHT\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/PHASE\_STAGING\_READINESS\_PREFLIGHT\_AR.md) - ??? ????? ??????? ??????.

2. [STAGING\_ENVIRONMENT\_DISCOVERY\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/STAGING\_ENVIRONMENT\_DISCOVERY\_AR.md) - ????? ??????? ??? ??????.

3. [STAGING\_ISOLATION\_CHECKLIST\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/STAGING\_ISOLATION\_CHECKLIST\_AR.md) - ????? ?????? ??? ??? ????????

4. [STAGING\_READONLY\_RUNTIME\_VERIFICATION\_PLAN\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/STAGING\_READONLY\_RUNTIME\_VERIFICATION\_PLAN\_AR.md) - ??? ?????? ?????? ? ??.

5. [STAGING\_API\_PROTOTYPE\_SCOPE\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/STAGING\_API\_PROTOTYPE\_SCOPE\_AR.md) - ????? ?????? ??????? ??????? ?????? ??? APIs.

6. [STAGING\_BACKEND\_RLS\_GAP\_REPORT\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/STAGING\_BACKEND\_RLS\_GAP\_REPORT\_AR.md) - ????? ??????? ??????? ??????? ?????? ??????? ?.

7. [STAGING\_READINESS\_DECISION\_REPORT\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/STAGING\_READINESS\_DECISION\_REPORT\_AR.md) - ??????? ??????? ??????? ??????? ??????? ?.

8. [PHASE\_STAGING\_READINESS\_IMPLEMENTATION\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/PHASE\_STAGING\_READINESS\_IMPLEMENTATION\_AR.md) - ????? ?????? ????? ????????

9. [STAGING\_SECURITY\_BOUNDARY\_REPORT\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/STAGING\_SECURITY\_BOUNDARY\_REPORT\_AR.md) - ????? ??? ?????? ????????

10. [STAGING\_READINESS\_TEST\_REPORT\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/STAGING\_READINESS\_TEST\_REPORT\_AR.md) - ????? ?????? ??????? ??????? ????????

11. [PHASE\_STAGING\_READINESS\_CLOSEOUT\_AR.md](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/docs/governance/enterprise-hospital-platform/PHASE\_STAGING\_READINESS\_CLOSEOUT\_AR.md) - (??? ?????) ????? ??????? ????????

---

## ## 2. ??? ?????? ??????? ??????? (Security & Testing Summary)

\* \*\*????? ??????? ????????:\*\* ?? ?????? ??? (PASS). ?? ?????? ??????? [staging\_readiness\_design\_test.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/staging\_readiness\_design\_test.js) ??? ?? ?????? ?????? ???????.

\* \*\*????????? ??? Zero-DDL:\*\* ?? ?????? ?????? 100%? ?? ??? ?????? ?? ??????? ?? ?????? ?????? ?? ?????? ?? ?????? ??? ???????.

\* \*\*????? ?????? ??????:\*\* ?? ?????? ?? ??? ?????? ??????? ??????? ?? ?? ?????? ?????? ?? ?????? ???.

---

## ## 3. ??? ?? ??????? ?????????? ??????? ??????? (Next Phase Roadmap)

????? ??? ?????? ??????? ?????? ??? Staging ?????? ?????? ?????? ?????????? ?????????? `nama\_medical\_staging` ?????? ?????? ?????? ?????????? ?????? ?????????? ?????????? ??????????

```mermaid

sequenceDiagram

participant Owner as ??? / ???

participant DevOps as ??? CI/CD

participant Backend as ???

Owner->>DevOps: ????? ??? Staging ?????? ?????? Staging ???????

DevOps->>DevOps: ????? ?? ?????? ?????? ??????? ?? Staging

DevOps->>Backend: ????? ?? ?????? ?? ????? Staging

Backend->>Backend: ????? ??????? ??????? ??????? ??????? ?? (GET Mocks)

Backend->>Backend: ????? ??????? ?????? ?? ??? ?????? RLS

...

*??? ?????? ??????? ?????? ?? ??????? ??????.**

??? ??????? ??????? - ??? ???? ??????

=====

FILE: docs\governance\enterprise-hospital-platform\PHASE_STAGING_READINESS_IMPLEMENTATION_AR.md

=====

????? ?????? ?????? ?????? ?????? - ??? ???? ?????? (Phase Staging Readiness Implementation Report)

* **?????:** ??? ???? (NamaMedical ERP)

* **?????:** ?????? ???? ??????? ?????? ?????? (PHASE_STAGING_READINESS_AND_ENVIRONMENT_GAP_REPORT)

* **???? ??????:** ????? (Completed)

1. ?????? ?????? ?? ??? ?????? (Accomplished Tasks)

?? ????? ?? ?????? ?????? ?????? ?????? ?????? ?? ??????? ?? ??????? ??????? ?????? ?????? (Zero-DDL/Zero-DB-Writes). ?????? ?????? ??:

1. **????? ?????? ?????? (Gate 0 - Preflight):**

* ?????? ?? ???? ?????? Git ???? Commit ??? `aa46f53`.

* ?????? ?? ?????? ???? ???? ?????? ??????? ??????? ??????? ??????? UTF-8.

2. **?????? ?????? ?????? (Gate 1 - Environment Discovery):**

* ?????? ?????? ??????? ?????????? ?? ?????? ?????? ?? ?????? ??? Staging.

* ??? ???? ?? ?????? ?? ??????? ?????? ?????? ?????? ?????? ??????.

3. **?????? ?????? ?????? ?????? ?????? ??????? (Gate 2 - Staging Isolation Checklist):**

* ?????? 12 ?????? ???? ???? ??????? ?? ???? ??????? ?????? ??????? ??????? ??????? ???????.

4. **??? ?? ?????? ??????? ??? (Gate 3 - Read-only Runtime Verification Plan):**

* ?????? ??????? ?? ?????? ?????? ?? ??????? ?? ??????? ?? ?????? ?? ?????? ?? ?????? ?? ??????.

5. **????? ?????? ??????? ?????? ??????? (Gate 4 - API Prototype Scope):**

* ??? ?????? ??????? ??????? ?????? ??????? ??????? ?????? ?????? ??? ??????? ??????? (Mock Data) ?? ?????? ???.

6. **????? ?????? ??????? (Gate 5 - Backend/RLS Gap Report):**

* ?????? ?????? 9 ?????? ?????? ??????? ?????? ?????? ?????? ??????? ??? RLS.

7. **????? ?????? ?????? ??????? (Gate 6 - Decision Report):**

* ?????? ???? ?????? ?????? ?????? ??????? `STAGING\_BLOCKED\_GAPS\_FOUND` ?????? ??????? ??????? ?? ???? ??? ???.

8. **????? ?? ??????? ?????? ?????? ??????? (Gate 7 - Static Test):**

* ?????? ?? ?????? ??????? ?????? [staging_readiness_design_test.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/staging_readiness_design_test.js) ?????? ??????? ?? ??? ?????? ?????? ??????.

2. ??????? ?????? ?????? ?????? ?????? (Security Compliance & Constraints)

?? ?????? ?????? ??????? ??????? ??????? ?????? ?????? ?????? ??? ??????;

* **????? ?????? ????????? (DDL/Migrations):** **?? ???? (NO)**. ?? ??? ?????? ?? ?????? ?? ?????? SQL ?? ???????? ?
?????.
* **????????? ?? ?????? ?????????? (DB Writes):** **?? ???? (NO)**. ?? ?????? ?? ?????? ?????? ?? ?????? ??? ?? ????.
* **?????? ???? ????????? (Production Touched):** **?? ???? (NO)**. ?????? ?????????? ?????????? ?????????? ?????? ?? ??? ???
????? ??? ?? ?????????.
* **????????????? ?????????? ?????????? (PHI):** **?? ???? (NO)**. ?? ??? ??? ?? ?????? ?? ?????????? ?????? ?????????.
* **?????? ????????? (Secrets):** **?? ???? (NO)**. ?? ??? ?????? ?? ?????????? ?? ?????? ?????? ?? ?????????? ?????? ?????????.
* **????? ?????????? (Endpoints Status):** **????????? ?????????? ??? (PLANNED_DISABLED)**.
* **????????? ?????????? ??????:** `isLiveEndpointEnabled = false` ? `isWriteOperationEnabled = false`.

?????????: ?? ?????? ?????? ?????? ??? Staging ?????????? ?????????? ?????? ?????? ??? ??? ?????????? ??? ?????????? ?? ??????
????? ?????????? ?????????? ?????????? ?????????? ?????????? ??? ?????? ?????? ?????????? ?????????? ?????????? ??????????.

=====
FILE: docs\governance\enterprise-hospital-platform\PHASE_STAGING_READINESS_PREFLIGHT_AR.md
=====

?????? ?????????? ????????? - ?????? ?????????? ?????? ?????????? (Preflight Readiness Report)

* **?????????:** ?????? ??? ????????? (NamaMedical ERP)
* **????????? ??????????:** ?????????? ?????? ?????????? ?????????? ?????????? (PHASE_STAGING_READINESS_AND_ENVIRONMENT_GAP_REPORT)
* **????? ??????????:** ?????? (PASS)

1. ??? ?????? ????????? Git (Git Status & Log)

?? ??? ?????? ?????????? ?????????? ?? ?????????? ?????????? ?????? commit ?????????? ??????????:
* **Commit ?????????? ?????????? ??????????:** `aa46f53` (????? ???? ???? ???? ?????????? ?????????? ?????????? RLS ???? DDL).
* **????? ??????????:** ?????? (Clean) ?????? ?? ?? ?????????? ?? ?????? ???.
* **????? ??????????:** ?? ?????????? ??? `git diff --check` ?????????? ??? ?????????? ?? ?? ?????????? ?????? ?????? ?? ?????? ??????
?? ?? ?????? ??????.

2. ?????????? ?? ?????? ?????? ?????????? ?????????? ??????????

?? ??? ?????????? ?????? ?????????? ?????????? ?????????? ?? ?????????? ?????????? ?????? ?????? ??????????:

| ??? ?????? | ????????? | ????????? |
| :--- | :--- | :---: |
| `BACKEND\_ERD\_DESIGN\_NO\_DDL.mmd` | `docs/governance/enterprise-hospital-platform/BACKEND\_ERD\_DESIGN\_NO\_DDL.mmd` |
| `RLS\_POLICY\_DESIGN\_NO\_EXECUTION\_AR.md` | `docs/governance/enterprise-hospital-platform/RLS\_POLICY\_DESIGN\_NO\_EXECUTION\_AR.md` |
| `API\_TO\_RLS\_GUARD\_MATRIX\_AR.md` | `docs/governance/enterprise-hospital-platform/API\_TO\_RLS\_GUARD\_MATRIX\_AR.md` |
| `STAGING\_API\_PROTOTYPE\_PREREQUISITES\_AR.md` | `docs/governance/enterprise-hospital-platform/STAGING\_API\_PROTOTYPE\_PREREQUISITES\_AR.md` |
| `BACKEND\_API\_RUNTIME\_READINESS\_CHECKLIST\_AR.md` | `docs/governance/enterprise-hospital-platform/BACKEND\_API\_RUNTIME\_READINESS\_CHECKLIST\_AR.md` |

???? ??? ??????? ?????? ???.

=====
FILE: docs\governance\enterprise-hospital-platform\PROMPT_LIBRARY_SYSTEM_PROMPTS_AR.md
=====

????? ?????? ?????? ?????? ??????? ??????
??????? ??????? ??????? ?????????? ???? Guardrails ?????? ???????.

=====
FILE: docs\governance\enterprise-hospital-platform\PUBLIC_AND_DASHBOARD_WIREFRAMES_AR.md
=====

??? ??????? ?????? ?????? ?????????? ?????? ??????
????? ??????? ???? Wireframes ?????? ?????????? ?????? ?????? ??????? ??????? ???? Navigation Items.

=====
FILE: docs\governance\enterprise-hospital-platform\PUBLIC_FACILITY_SWITCHER_UI_DISCOVERY_AR.md
=====

????? ??????? ?????? ?????? ?????????? ???????
????? ?????? ??????? ?????????? ?????????? ?????? ??? SPA ?????? ??????? ???? ??????????.

=====
FILE: docs\governance\enterprise-hospital-platform\PUBLIC_HOMEPAGE_REQUIREMENTS_AR.md
=====

??????? ?????? ?????????? ?????? ??????
??? ??? ??????? ??????? ?????? ?????????? ?????????? ?????????? ??????? ?????????? ??????? Hero Section ?????? ?? ???
????? ?????? ?????????? ?????????? ???????.

=====
FILE: docs\governance\enterprise-hospital-platform\PUBLIC_HOMEPAGE_UI_REPORT_AR.md
=====

????? ?????? ??????? ?????????? ?????? ??????
????? ?????? ??????? ?????????? ?????????? ?????? ?????? ?????????? ??????? ?????????? ?????????? ??????????.

=====
FILE: docs\governance\enterprise-hospital-platform\RADIOLOGY_ORDERS_UI_REPORT_AR.md
=====

????? ?????? ??????? ?????? ?????? (Radiology)

???? ?????? ?????? ?????? ?????? ?????????? ?????????? ?????????.

?????? ?????????? ??????????

- 1. **????????? ?????????? ??????????**: ?????? ?????????? ?????????? ?????????? ?????? (??? ??????? ?????????? ?????????? ??????????????).
- 2. **????????? ?????????? ??????????????**: ?????? ?????????? ?????????? ?????????? ?????? ?????? ?????? ?????? ?? ?????? ??????.

=====

FILE: docs\governance\enterprise-hospital-platform\RADIOLOGY_RESULTS_UI_REPORT_AR.md

=====

?????? ?????? ?????????? ?????????? ?????????? (Radiology Reports)

????? ?????????? ?????? ?????????? ?????? ?????????? ?????????? ??????????.

?????? ?????????? ??????????

- 1. **????? ?????? ??????????**: ?????? ?????? ?????? ?????????? ?????????? ?????????? ?????? ?????????? (??? ?????????? ?????????? ?????? ?????? ?? ?????????? ??????).
- 2. **????????? ?????????? ??????????????**: ?????? ?????? ?????? ?????????? (?????? ?????????? ?? ?? ??????????) ?? ?????? ?????? ?????? ?????? ?????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\RESULTS_AND_DOCUMENTATION_WORKFLOW_DESIGN_AR.md

=====

?????? ?????????? ?????? ?????? ?????????? ?????????? ??????????

????? ??? ?????????? ?????????? ?????????? ?????? ?????? ?????????? ?????????? ?????????? ?????????? ??????????.

?????? ?????? ?????????? (Results Flow)

- 1. **????????? ??????????**: ??? ??? ?????????? ?????????? ?? ?????????? ?? ?????????? ?? ?????? ?????????? (Results Inbox).
- 2. **????????? ?????????? ??????????**: ??? ?????? ?????????? ?????????? ?? ??? ?????? ?????????? ??? ?????????? (Abnormal) ?????? ?????? ??? ??????.
- 3. **????????? ??????????**: ?????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????? ?????? ?? ?????? ?????? ?? ??? ??? ?????? ??????.

?????? ?????????? ?????????? (Clinical Documentation Flow)

- 1. **????? ??? ??????????**: ??? ?????? ?????????? ?????????? ?????????? (Clinical Note).
- 2. **????????? ?????? SOAP**: ?????? ?????????? ?????????? (????????? ?????????? ?????????? ??????).
- 3. **????? ??????????**: ??? ?????????? ?????????? ?????????? ??? ?????????? (Draft Only) ??? ?????????? ?????????? ?????????? ?????? ?????? ?????? ?????????? ?????????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\RESULTS_DOCUMENTATION_SECURITY_AND_SAFETY_REVIEW_AR.md

=====

?????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ??????????

????? ??? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ?????????? ??????????.

?????????? ?????????? ??????????

1. **PHI (Protected Health Information) (PDPL).
2. ** ()
3. ** ()

=====

FILE: docs\governance\enterprise-hospital-platform\RESULTS_DOCUMENTATION_TEST_REPORT_AR.md

=====

#

##

1. ** (Health Unit):
2. ** (Polyclinic):
3. **:

=====

FILE: docs\governance\enterprise-hospital-platform\RLS_POLICY_DESIGN_NO_EXECUTION_AR.md

=====

Row-Level Security (RLS)

(RLS Policies)

1. ** (Tenant Isolation):
``sql
CREATE POLICY tenant_isolation_policy ON encounters
USING (tenant_id = current_setting('app.current_tenant_id'));
``
2. ** (Facility & Department Scope):
``sql
CREATE POLICY facility_scope_policy ON clinical_orders
USING (facility_id = current_setting('app.current_facility_id')
AND department_id = ANY(current_setting('app.current_user_departments')::int[]));
``
3. **:

=====

FILE: docs\governance\enterprise-hospital-platform\SAMPLE_DATA_SEEDERS_PLAN_AR.md

=====

??? ??????? ????????? ????????? ?????????

??? ????? ??????? ????????? ??????? ??????????? ??????? ?????????.

=====
FILE: docs\governance\enterprise-hospital-platform\SECURITY_PLAN_AR.md
=====

??? ????? ?????????? ??????? ?????????? ???????

?????? ??????? ??????? ??? PHI ????????? ??????? ??????? ???????.

=====
FILE: docs\governance\enterprise-hospital-platform\SECURITY_RBAC_AUDIT_TEST_REPORT_AR.md
=====

????? ?????????? ?????? ?????? ??????????? ?????? ????????

???? ????? ?????? ????????? ?????????? ??????????? ?????? ?????????? ?????? ?????????? ???????.

?????? ????????????

1. **??????? ??? ?????????? ??????????*: ?????? ?????? ?? ?? ????? `canPerformFinalAction` ????? ??????? `false` ?????? ? ?????? ????????? ??????????.
2. **??????? ?????? ??? ?????????*: ?????? ?? ?????? ??????? ??? ?????????? ?????????? ?????? JSON ?????? ??????? ?? ?? ?????? ?? PHI.
3. **??????? ?????? ??????????*: ?????? ?? ?????????? ??? ?????????? ?????? ??????? ?? ?????? ?????????? ??????? ?????????.

=====
FILE: docs\governance\enterprise-hospital-platform\STAFF_JOURNEY_FLOW_AR.md
=====

???? ???? ??????? ?????? ??????????? ??????????

??? ?????? ????????? ??????????? ??????????? ?? ??????? ?? ??????? ?????????.

=====
FILE: docs\governance\enterprise-hospital-platform\STAGING_API_PROTOTYPE_PREREQUISITES_AR.md
=====

???????? ??????????? ?????? ?????? ?????????? ?????????? (Staging Prerequisites)

????? ??? ??????????? ??????? ??????? ?????????? ??????? ?????? ?????? Staging ??????? ?????? ?????????? ?????????.

???????? ?????? ??????

1. **??????? ?????? ?????? ?????? ??????*: ?????? ??????? ?? ?????? ?????????? ???????.
2. **??????? ??????? ??? ??????????*: ?????? ??? ??????? ?????????? ?????? ??????? ?? ?? ?????????? ??????? ???????.
3. **????????? ?? ?????? ??? ?????? (NPHIES Sandbox)*: ?????? ?????????? ?????? ?????????? ??????????? ???????????.

=====

```
* **???????:** ???? ?? ?????? (NamaMedical ERP)
* **???????:** ?????? ???? ????????? ?????? ?????? (PHASE_STAGING_READINESS_AND_ENVIRONMENT_GAP_REPORT)
* **?????:** ?????? ?? ?? ?????? ??? ?? ?????? ?????? ?? ????????? ?????? ?????? ?????? ?????? (API Prototype) ???
?? ?????? ?????? ???? ???? ?????????? ??? ?????? ?????????? ??????????.
```

— — —

??? ???? ??????? ?????? ?????? ?????????? ?????????? (API Prototype)? ????? ??? ?????????? ?????????:

```
* **????? ???? ???? (GET/Read-Only Placeholder Endpoints):**
* ????? ?????? ?????? ????? ?? ?????? ?? ????? ????? ??????.
* ?????: ??? ????? ?????? (`/api/v1/facilities`) ?????? ?????? (`/api/v1/departments`) ???????? ??? ??????
? ????? (Static Arrays) ????? ??????.
* **????????? ????? ????? (Static/Mock-Backed Responses):**
* ????? ?????? ?????? ????? ??? ??? ????????? DTO ?????? ?? ??? [enterprise-contracts.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/enterprise-contracts.js).
* ?? ?? ????? ?? ?????????? ????? ?? ?????? ?? ?????? ??????????
* **??? ?????? ??? (Auth Guard Placeholders):**
* ?????? ?? ??? ?????????? (??? ?????? ?????? ?????) ??? ????? ?????? ?????????? ??? ?????? ??? ?? ??? ?????????? ???
??? ??? ?? ?????? ?? ?????????? ?????????? ??????.
* **?????? ?????? ?????? (Audit Preview Only):**
* ????? ?????? ?????????? ?????????? ?????????? ?? ??? ?????? (Console Logs) ?? ?????? ?? ?????? ?????? ?? ?????????? (
In-Memory) ??? ?? ?????? ?????? ?????????? ?? ?????????? ?? ?????? ?????????? ?????? ??????????.
```

— — —

????? ?????? ?????? ?????? ?????? ?? ?? ?????????? ?????????? ?? ?????????? ??????????:

```
* **?????? ???? ??? (POST/PUT/DELETE Real Writes):**  
    * ????? ?? ????? ?? ?? ?? ????? ?? ????? ?????? ?????? ?????? ?????? ?? ????? ??????  
    ???.  
  
* **????? ????????? ?????? ?????? (Billing Submission):**  
    * ????? ????? ????? ????? ?? ????? ?? ????? ????? ?? ????? ?????? ?? ????? ??????.  
  
* **????? ?????? ?????? (Pharmacy Dispense):**  
    * ????? ????? ????? ?????? ?????? ?? ????? ?? ?????? ?? ????? ??????.  
  
* **??????? ?????? ?? ?????? ?????? (Final Signature):**  
    * ????? ????? ????? ????? ?????? (SOAP Notes Lock) ?? ????? ?? ????? ?????? ?????? ????? ?????.  
  
* **??????? ?????? ????? NPHIES (Real NPHIES Calls):**  
    * ????? ????? ?? ????? ????? ?????? ?? ????? ??? ????? (NPHIES) ??????.  
  
* **??????? ?????? ?????? ?????? (Production DB Connection):**  
    * ????? ????? ?? ?????? ?? ?????? ?????? ?????? ?????? ?????? ??????
```

— — —

3. ?????? ???? ??? ????????? (API Scope Matrix)

```
??????? ???????? (Endpoint)	?????? ??????? ?? ?? Staging	?????? ????????	?????? ????????
`/api/v1/facilities`	??????? ???????? ????????	?????? ?? ?????? ???	?????? ?? ?????? ?????? ?? ??????.
`/api/v1/departments`	???????? ???????? ????????	?????? ????????	?????? ?????? ?? ?????????.
`/api/v1/queue`	??? ?? ?????? ??????????	?????? ?????? ?? ???????? ???	??? ?????? ?????? ?????????? ?? ???
????? ????.			
`/api/v1/encounters`	??? ?????????? ?????? ??????????	?????? ??? ??? ?? ??????	?????? ?????? (Mock Data)
?????? ?? DTO.			
`/api/v1/clinical-orders`	??? ???????? ?????? ??????????	?????? ??? ??? ????????/??????	??? ??? ????????
?????? ?? ??????????.			
`/api/v1/clinical-results`	??? ?????? ??????????	?????? ?????? ?????? ??????	?????? ?????? ?????? ??????.
`/api/v1/clinical-pharmacy`	??? ???????? ?????????? ??????????	?????? ?????? ??????	?????? ??? ???????? ??????
? ?? ???????? ????.			
`/api/v1/clinical-billing`	??? ?????? ?????? ??????????	?????? ?????? ?? ?????? ????????	?????? ??? ??????
? ?????????? ?? ???????? ?????? ??????. |
```

```
---
**????????:** ?????? ???????? ???????? ?????? ?????? ???????? ???????? ???????? ??? ?????????? ???????? (Mocking)? ???
???? ??? ?????? ??? ?????? ?? ????????.
```

```
=====
FILE: docs\governance\enterprise-hospital-platform\STAGING_BACKEND_RLS_GAP_REPORT_AR.md
=====
```

```
# ?????? ???????? ?????? ?????????? ?RTL/RLS - ????? ??? ?????? (Staging & RLS Gap Report)
```

```
* **????????:** ????? ??? ?????? (NamaMedical ERP)
* **????????:** ?????? ??? ?????????? ?????? ?????? (PHASE_STAGING_READINESS_AND_ENVIRONMENT_GAP_REPORT)
* **????????:** ?? ???????? ?????? 9 ?????? ?????? ??? ?????? ??? APIs ??? RLS ?????? ?? ??? ??????????.
```

```
## 1. ?????? ???????? ?????????? ?????????? (Gap Analysis)
```

```
??? ?????? ???????? ?????? ?????????? ?????????? ?????????? ?????? ?????? ?????????? ??? RLS ?? ?????? ?????? ?????????? ???
??? ?????????? ??????????:
```

```
### ??????: ??? ?????? (Environment Gap)
```

```
* **????? ??????:** ?? ??? ??? ?????? ?????? Staging. ??? ?????????? ??? ??? ?????????? ?????? ?? ??? ?????????? ??
???? ??????.
* **?????:** ?? ?????????? ?????????? ?? ??? ??? ?????? ?????? ?????? ?????????? ?????? ?? ??????????.
* **????????:** ?????? ??? ?????? ?????? ?????? ?????? ?????? ?????????? ??????????.
```

```
### ??????: ??? ??? ?????? ?????????? (Database Isolation Gap)
```

```
* **????? ??????:** ?? ??? ?????? ?????? ?????? ?????? ?? Staging ??? ?????? ??????. ?????????? ?????????? ??? ???
??? `nama_medical_web` ?? `namasoft`.
* **?????:** ?? ??? ?????????? ?? ?????????? ?????? ?????? ?????????? (Migrations) ??? ??? ??? ??????????.
* **????????:** ?????? ?????? ?????? ?????? ?????? `nama_medical_staging` ?????? ?????? ??? ??? ?????????? ?????????.
```

```
### ??????: ??? ?????? ?????????? (Secret Management Gap)
```

```
* **????? ??????:** ??? ?????? ?????????? (??? `SESSION_SECRET` ? `JWT_SECRET`) ?? ?????? `.env` ?????? ?????? ??? ??
```



```
* **?????:** ?????? ?????? ??????? ?? ??????? ??? ?????? ??????? ?? ????? ?????????.
```

```

* **????? ??????:* ?? ?????? ?????? RLS ?????? ?? ?????? ?????? ??? ?? ??? ?????? (Zero-DDL). ????? ?????
??? ?????? ?? ????? ?? ?????? ?? ?????? ????? ??????.
* **?????:* ??? ?????? ?????? ?????? ?????? ?????? ?????? ?????? (Tenants) ?????? ?? ????? ??????.
* **?????:* ?????? ?????? ?????? ?? ?????? ?????? ?????? ??? Staging ?????? ??? ??????.

```

```
* **????? ??????:** ????? ??? [enterprise-contracts.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/enterprise-contracts.js) ??? ????? ?????? ?? ????? ?????? ?????? ?????? ?????? ?????? ??????.
```

```
* **?????:** ???????? ???????? ?? ????? ???????? ???????? ?????? ?? ?????? ???????? ???????? ?????? ???
```

```
????.
```

```
* **????????:** ????? ?????? ?????? ??? (Mock Auth / Session-based) ???????? ??????.
```

* **????? ?????:** ?? ?????? ????? ??????? ????? ?? ??????? ?? ?? ??????? ??????? ?? ??????? ??????? ??????? ???????.

* **?????:** ?? ??????? ??????? ?????? ??????? ??????? ??????? ??????? ?? ??????? ?? ??????? ??????? ?????? ?? ?? ???????.

* **?????????:** ?????? ?????? ?????? ?????? ?? ??????? ?? ?????? ??????? ??????? ??????? ???????.

```
* **????? ??????:** ????????? ??? E2E ??????? (`e2e_local_smoke_test.js`) ????? ?????? ?????? ??? ?????? ?????? ?
??? ????? ????? ????? ?? ????? ??????, ?? ????? ?????????? ????? ?????? ?????? Staging ??????????.
* **?????:** ??? ??????? ??? ?????? ??? ?????? ????? ??? ?? ?????? ??? ?????? ?? ?????? ??????????.
* **?????????:** ?????? ?????? ????????? ????? ?????? ?? Staging ?? ?????? ?????? ?????? ?????.
```

```
* **????? ??????:* ?????? ????? ??? ??? ??????? ?? ????? (`deploy_web.sh` ? `redeploy.sh`) ??? ????? ????? ???
?? CI/CD ?????? (??? GitHub Actions) ??? ????? Staging ??? ????? ??????.
* **?????:* ????? ?????????? ??? ????? ?????? ?????? ?????? ??????.
* **?????????:* ????? ??? ?????? ??? ????? ??????? ??????????? ??? ????? Staging ??? ?????.
```

```
* **????? ??????:* ????? ??????? ?? ?????? ?????? ?????? ?????? ?? ????????? ??????.
* **?????:* ?? ????????? ?? ?? ??? ?? ?????? ?????? ?????? ?????? ?????? ??????.
* **?????????:* ????? ?????? ??????? ?????? (Automated Rollback) ??????? ?? ??? ????????? ?????? ?????? ??????
? ??? ?? ?? ????????? ??????.
```

```
??? ?????	??????	??????	??????	??????
**??? ????? ???????**	???? ????	??????	????? ????? ?????? `nama_medical_staging`	????? ????? ??????
?? (DBA)				
**???????? ??????**	????	??????	??? ???? ???? ?? ???? ???? ????	????? ????????? (DevOps)
**?????? ?? RLS**	?????	??????	????? ?? DDL ?????????? ?? ???? ??????????	????? ?????? ?????? (Backend)
```



```
`pm2 restart nama-medical-erp --update-env`
* **????? Docker:**
* ?? ???? ????? `Dockerfile` ?? `docker-compose.yml` ????? ?????? ?? ??? Staging.

---

## 4. ?????? ????? ????????? (Deployment Scripts)

?? ????? ?????????? ??????? ?? ??? ?????????:
* **????? `deploy_web.sh`:**
* ???? ?????? Node.js ? PostgreSQL ? Nginx ? PM2 ??? ??????.
* ???? ?????? ?????? ?????? ???? `namasoft` ??????? ???? `namasoft`.
* **????? `redeploy.sh`:**
* ???? ?????? ?????? ???? `namaweb` ?????? ?? PM2.
* ???? ?????? ?????? ?????? `nama_medical_web` ????????? `postgres`.
* ???? ?? ?????? ??? ???? ?????? ?????? ?? `46.224.178.153`.
* **?????:** ?????????? ?????????? ?????? ?????????? ?????????? ?? ?????? ?????? ??? ?????????? ?????????? ?????????? ??? ???? ???
?? ?????????? ?? Staging ???? ???? ?????????? ??? ?????? ????.

---
```

```
## 5. ?????? ????????? ?????? ????????? (Database Connection Abstraction)

* **????? `database.js` ? `db_postgres.js`:**
* ??? ?????? ?????????? ?????????? ??? ?????? ?????? `DATABASE_URL` ?? ?? ???? ?????????? ?????????? (`DB_HOST`, `DB_USER`, ???).
* ??? ?????????? ???? ?????????? ?????????? ??? ?????? ?????????? ?????????? ?????????? ?????? ?????? ??? `.env` ??? ?????? ?????? ?????
???? ???? ???? ?????? ???? Staging.

---
```

```
## 6. ?????? ?????? ?? ????? (Health Endpoint)

* ?????? ?????????? ??? ?????? ???? ?? ?????? ?? ?????? `/api/health` (??? ?? ???? ???? ?? ?????? ?????????? ?????????? ??????
?).
* ?????? ??? ?????????? ?????? `{"status":"UP"}` ?????? ?????????? 200 ???? ???? ???? ???? ???? ???? ???? ???? ?????.

---
```

```
## 7. ?????? ?????????

????? ?????? ?????? ?????? ?????????????? ?????? ?????? ??? ?????? ?????????? ?????? ???? **?? ???? ???? ?????? Staging ?
???? ?????? ???? ?????? ?????** (?? ???? ???? Staging ???? ?? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????
????????? ?? Staging). ?????? ?????? ?????????? ??? ?????????? ??? ?????????? ?????????? ?????? `.env`.
```

```
=====
FILE: docs\governance\enterprise-hospital-platform\STAGING_ISOLATION_CHECKLIST_AR.md
=====
```

```
# ?????? ??? ???? ?????????? - ???? ??? ?????? (Staging Isolation Checklist)

* **?????????:** ???? ??? ?????? (NamaMedical ERP)
```

```
* **???????;* ?????? ???? ???????? ?????? ?????? (PHASE_STAGING_READINESS_AND_ENVIRONMENT_GAP_REPORT)
* **?????;* ?????? ?? ?????? ?????? ???????? ?????? ?????? ???????? ?????? ?????? APIs ???? ?? ?????? ?? ?????? ????
????? ???????? ?? ???????? ?? ?????? ?????? ????????.
```

1. ???? ?????? ?? ?????? (Isolation Verification Table)

```
| :--- | :--- | :--- | :--- |  
| **?????? ???? (Separate Host)** | ????? ???? Staging ??? ???? ???? ?? ???? ???? ???? ???? (`2  
04.168.144.74`). | ??????: ??? (????).<br>?????: ?? ???? ???? ???? ???? ?????. | **???? (GAP)** ?? | ???  
???? ???? ???? ?? ???? ???? (Port) ????? ???? ???? ?????. |  
| **???? ???? ???? (Separate DB)** | ?????? ???? ???? ???? ???? (????: `nama_medical_staging`) ???  
?? ?? ???? ???? ?????. | ??????: ???.<br>?????: ?? ???? ???? ???? Staging ????? ?? ?????. | **???  
? (GAP)** ?? | ????? ???? ???? `nama_medical_staging` ??????. |  
| **???? ???? ???? ???? (Separate DB User)** | ?????? ???? ???? ???? ???? ???? Staging ?????  
?? ???? ?????. | ?? ???? ???? ???? ???? Staging. | **???? (GAP)** ?? | ????? ???? `nama_staging_user` ??  
???? ???? ???? Staging ????. |  
| **??? ???? ???? (Separate .env File)** | ????? ???? `.env` ????? ???? ???? ???? ???? ???? ?????. | ???  
? ??? ???? `.env` ???? ???? ?????. | **???? ????** ? | ????? ???? `.env.staging` ????? ???? ?????. |  
| **???? ???? (Separate Secrets)** | ????? ???? ???? ???? ???? `SESSION_SECRET` ? `JWT_SECRET` ???? Stagi  
ng. | ?? ???? ???? ???? ???? ???? ???? ???? ?????. | **???? ????** ? | ????? ???? ???? ???? ???? ??  
Staging ???? ???? ?????. |  
| **???? PM2 ???? (Separate PM2 Process)** | ????? ???? ???? ???? ???? ???? ???? PM2 (????: `nama-medical-  
staging`). | ????? ???? ???? `nama-medical-erp`. ?? ???? Staging ?????. | **???? (GAP)** ?? | ????? ????  
?? ?? PM2 ???? ???? ???? ???? ???? `nama-medical-staging`. |  
| **???? ???? (Separate Domain)** | ?????? ???? ???? ???? (????: `staging.jumanasoft.com`). | ?? ???? ???? ?  
??? ???? ???? ?????. | **???? (GAP)** ?? | ????? ???? DNS ????? ???? ???? ???? SSL ?????. |  
| **???? ???? (Separate Logs)** | ??? ???? ???? ???? ???? ???? ???? ???? ???? ???? ?????. | ?  
?? ???? ???? ???? ???? ???? PM2 ? Node.js. | **???? ????** ? | ?? ???? ???? ???? PM2 ???? ???? ??  
logs ???? ???? Staging. |  
| **??? ???? ???? (Separate Backups)** | ??? ???? ???? ???? ???? ???? ???? ???? ???? ???? ??  
? ?????. | ?? ???? ???? ???? ???? ???? ???? ???? ?????. | **???? ????** ? | ?? ???? ???? ???? ???? ??  
? ???? ???? `/var/backups/staging/`. |  
| **??? ???? ???? (No Production Writes)** | ??? ???? ???? ???? ???? ???? ???? ???? Staging ?????  
?? ???? ???? ?????. | ??? ???? ???? ???? ???? ???? ???? ?????. | **???? ????** ? | ????? ????  
? ?? ???? ???? `.env` ???? ???? Staging ?? ???? ???? ?????. |  
| **??? ???? ???? (No Production Migrations)** | ??? ???? ???? ???? ???? ???? ???? (Migrations) ??  
???? Staging ???? ???? ?????. | ??? ???? ???? ???? ???? ???? ???? ?????. | **???? ????** ? | ?  
???? ???? ???? ???? ???? ???? ???? ???? ?????. |  
| **??? ???? ???? (Rollback Plan)** | ?????? ???? ???? ???? ???? ???? ???? ???? ???? ????  
??? ?????. | ????? ???? ???? ???? ???? Git ????? ???? ???? ???? ?????. | **????** ? | ????? ???? ???? ????  
??? ?????. |
```

```
## 2.  ???  ??????  ??????  ????? (Security Boundary Summary)
```

— — —

=====

=====

```
* **??????:**  ???? ??? ?????? (NamaMedical ERP)
```

```
* **????? ?????? ??????:** **???? ???????? ?????? - ???? ????? ?????? (STAGING_BLOCKED_GAPS_FOUND)**
```

— — —

— — —

* **?????:** ?? ???? ???? ?????? ?????? ????????? ???? ???? ???? ???? ???? ???? ???? ???? ???? ????
 ???? ???? ????.

```
* **?????:** ?? ???? ????? ?????? ?????? ???? `nama_medical_staging`. ????? ?? ??????? (Migrations) ?? ????
?? ?? ?? ????? ??????? ?? ?? ???? ??????.
```

* **?????:** ??? ??????? ?????? ?????????? ??????? ?? ??? ?????? ????????? ??? ??? ????????? ??? ?? ????? ???????
? ?? ??? ?????????? ????????? ??????? ?????????? ??????????.

```
* **?????:* ?? ???? ???? ??? `staging.jumanasoft.com` ????? ?????? ?????? ?????????? ?????? ?????? ?????? ??????
???????? ?? ??????? ??????? ??? NPHIES.
```

— — —

```
* [ ] **??? ????? ?????? ?????*? ?? ?????? ????? ????????? (VPS) ?????? ????????? ?????? Staging.
```



```
* ?????? ?? ???? ???? ????? ?????? ?????? ???? 11 ?? ?????? ?????? ?????? ?????? ?? ??????? ??????? ??????? ?
? ?????? ??????? UTF-8.
```

??? ??? ??????? ?????? ??? ??? ???? ?????? ??? ?????? ?????? ?????? ?????? ?????:

- 1. **????? ?? ??? ????? ?????? (App Boot Cleanliness):**
 - * ????? ?????? ?????? ????? PM2 ????? ?????? ?? ??? ????? ?????? ?????? ?? ??? ?????:
 - `pm2 logs nama-medical-staging --lines 50`
 - * ?????? ?? ??? ????? ?????? ?????? ?????? ?????? ?? ?????? ?????? ??????.
- 2. **??? ??? ?????? ?????? (Health Endpoint Verification):**
 - * ?????? ?????? ?????? ?? ?????? ?? ??? ?????? ?? ??????:
 - `curl -i http://localhost:3000/api/health`
 - * **????? ??????:** ??? ?????? `200 OK` ?????? JSON ?????? ????: `{"status":"UP"}`.
- 3. **????? ?????? ?????? ?????? (Static UI Modules Load):**
 - * ??? ?????? ?? ?????? ?????? ?????? (??? ??? ?????? ?????? ?????? ??????).
 - * ??? ?????? ?????? (F12 Console) ?????? ?? ??? ?????? ?? ?????? JavaScript ?????? ?? ?????? ?????? ??????.

2. ?????? ?? ?????? ?????? ?????? ?????? (Security Guards & Contracts)

??? ?????? ?? ?????? ?????? ?????? ?? ?????? ?????? ?????? ?? ??? ?????? ??? ?????? ?? ?????? ???:

- 1. **??? ?????? ?????? ?? ?????? (Contracts Flags Verification):**
 - * ??? ??? ?????? ?? ?????? ?????? ?????? ?????? ?????? ?? ?????? ??????:
 - * `window.isLiveEndpointEnabled()` ??? ?? ?????? **`false`**.
 - * `window.isWriteOperationEnabled()` ??? ?? ?????? **`false`**.
 - * ?????? ?????? ?????? ?????? ?????? ?? ?????? ?? ?????? ?????? ?????? ?????.
- 2. **????? ?????? ?????? OpenAPI (OpenAPI Endpoints Status):**
 - * ?????? ?????? ?? ?????? ?? ?????? ?????? ?????? ?????? ??? (??? `/api/v1/clinical-orders`).
 - * **????? ??????:** ?????? ?? ?????? `404 Not Found` ?? ?????? ?????? ??? ?????? ?????? ??? ?
 - ?? (`PLANNED\_DISABLED`).

3. ??? ?????? ?????? ?????? (No Database Writes & Final Actions)

????? ?????? ?????? ?????? ?????? ?????? ??? ??? ?????? ??????:

- * **??? ?????? ?????? (No DDL/No Migrations):**
 - * ??? ?????? ?? ?????? `.sql` ?? ?????? ?????? ?????? ?? ?????? Staging ?????? ?????? ?????? ?? ?????? ??????.
- * **??? ?????? ?????? ?????? (No PHI Leakage):**
 - * ?????? ?? ??? ?????? ?? ??? ?? ?????? ?????? ??????. ?????? ?????? ?????? (Mock Data) ?????? ?????? ??????.
- * **??? ?????? ?????? ?????? ?????? (No Final Actions):**
 - * ?????? ?? ??? ?????? ?????? ?????? ?????? (No Pharmacy Dispense).
 - * ?????? ?? ??? ?????? ?????? ?? ?????? ?????? NPHIES (No NPHIES Calls).
 - * ?????? ?? ??? ?????? ?????? ?????? ?????? (No Clinical Signatures).

```

?????? ?? ???? Staging ???? ??????? ??? ????? ?????????? ????? ??????:

```

```
sequenceDiagram
```

```

--
**??????:* ???? ??? ????? ???? ??? ???? ???? ?????? ???? ?????? ???? ???? ???? ???? ????
? ??? ?????? ??????? ? ???? ?????????,

```

```
=====
FILE: docs\governance\enterprise-hospital-platform\STAGING_SECURITY_BOUNDARY_REPORT_AR.md
=====
```

```
* **???????:* ???? ?? ?????? (NamaMedical ERP)
* **???????:* ?????? ???? ???????? ?????? ?????? (PHASE_STAGING_READINESS_AND_ENVIRONMENT_GAP_REPORT)
* **?????:* ?????? ?????? ???????? ???????? ?????? ??? ?????? ???????? ??? ?????? ???????? ???????? (Staging)? ???
????? (Production) ???????? ???????? ???????? ???????? ?????? ???????? ?????? ????????.
```

— — —

?? ????? ????? ????? ????? ????? ????? ??? ??????? ????? ??????? ?? ??????? ??? ????? ???????:

```
* **????? ?????? ?????? (Self-Enforcing Contracts):**
* ?? ????? [enterprise-contracts.js](file:///c:/Users/ice/Desktop/NamaMedical/namaweb/public/js/enterprise-
ontracts.js)? ?? ????? ??????:
* `window.isLiveEndpointEnabled() { return false; }`
* `window.isWriteOperationEnabled() { return false; }`
* ??? ?????? ??? ??????? ??? ?????? (Kill Switches)? ??? ?? ??? ?????? ??????? ?????????? ??? ??? ?????? ???
?? ??? ?????? ??????? ?????????? ?????????? ??? ?????? ??? ????????? `false`.
* **????? ?????? ?????? ????????? (OpenAPI Route Guards):**
* ????? ??????? ?????????? ?????????? ??????? ?? `PLANNED_DISABLED`. ?? ????? ??????? ????? ?????? ?????? ?
???? ?????????? ??????.
```

2. ????? ???? ?????? ?????? ???????? (Network & Database Boundaries)

????? ?????? ?????? ?????? ??? ?????? ???????? ???????? ???????? ??? ?????? Staging:

```
```mermaid
graph TD
 subgraph ???? ???????? ???????? (Production Site)
 ProdApp[????? ???????? PM2] -->|????? ????| ProdDB[(????? ?????? ????????)]
 end

 subgraph ???? ???????? ???????? (Staging Site)
 StgApp[????? ???????? PM2] -->|????? ????| StgDB[(????? ?????? ????????)]
 end

 ProdApp -. ->|????? ?????? ???? ????????| StgDB
 StgApp -. ->|????? ?????? ???? ????????| ProdDB
 StgApp -. ->|??????? ??? ????????| NPHIES[????? ???? ????????]
```

- \* \*\*???? ?????? (Network Isolation):\*\*
  - \* ??? ?????? ???? ?????? (Firewall) ???? ???? Staging ?????? ?? ?????? ??? ?????? `5432` ?????? ?????? ???????.
- \* \*\*???? ?????? ???????? (Database Isolation):\*\*
  - \* ??? ???????? ?????? ?????? ?????? ?????? ???????? ???????? ?? ???????.
  - \* ????? ?????? ?????? ???????? ?????? ?????? ???????? ?????? ?????? ?? ??? `.env` ?????? Staging.

---

## 3. ???????? ???????? ???????? ???????? (Saudi Regulatory Compliance)

??????? ?????? ???? ?????? ?? ???????? ???????? ????????:

- \* \*\*????? ?????? ???????? ???????? (PDPL):\*\*
  - \* ?????? ??? ?? ?????? ?????? ?????? ?? ???? ???????? ??? ???? Staging ?????? ?????? ?? ???????.
  - \* ??? ???????? ?????? ?????? ?? ?????? (Mock Data/Anonymized Data) ???????? ?????? ??? ?????? ?? ?????? ???????? ??? ?????.
- \* \*\*??????? ???? ?????? ???? ???????? ???????? (CBAHI):\*\*
  - \* ?????? ?????? ???????? (Audit Logs). ?? ???? ???????? ???? ?????? ?????? ?????? ?????? ?????? ?????? ?? ???? ???????? ???? ?????? ???? ?????? ?????? ?????? ?????? ??????.
- \* \*\*??????? ???? ?????? ???????? ???????? (ZATCA Phase 2):\*\*
  - \* ?????? ?????? ?????? ???????? ??? ???? ?????? ?????? ??? ???? ???? ?????? ???????? (ZATCA Integration) ?????? ?? ?? ???? Staging ?? ?????? ?? ??? ???????? ?????? (Dry-Run Mode) ?????? ?? ?????? ???????.
- \* \*\*??????? ???? ?????? ???????? (NPHIES):\*\*
  - \* ??? ??? ?????? ?? ?????? ???? ?? ???? ?????? ??? ?????? ?????. ???? ???????? ???????? ???????? ?? ??? Staging ?????? ?????? ?????? ???? ???? ???????? ???????.

---

## 4. ?????? ???????? ???????? (Secrets & Encryption)

- \* \*\*????? ???????? (Secrets Isolation):\*\*
  - \* ??? ???????? ?????? ???? ?????? `SESSION\_SECRET` ?????? ?????? ?????? `JWT\_SECRET` ?????? Staging.
  - \* ???????? ??? ???????? ?? ???????? ?????? ?????? ???? ???? ?????? ?????? ?????? ?????? ??????.

```
* **????? ?????????? ??????????:**
* ??? ?????? ????????? ?? ?????? ?????? Staging ??? ?????? ????????? ?????????? ?????? ?? ?????? ?????? ?????? ??????
?? ?????? ??? ?????? ?? ?????? ?????????.

=====
FILE: docs\governance\enterprise-hospital-platform\STYLE_GUIDE_DESIGN_SYSTEM_AR.md
=====

??? ????????? ?????? ?????? ?????????? ?????????

????? ?????? ????????? ?????????? ?????????? ?????? ?????????? (RTL/LTR).

=====
FILE: docs\governance\enterprise-hospital-platform\TEST_CASES_AND_TEST_PLAN_AR.md
=====

??? ?????? ?????????? ?????? ?????????

????????????? ?????????????? ?????????? ?????? ??? ?????????? ??? ??????????.

=====
FILE: docs\governance\enterprise-hospital-platform\TRAINING_VIDEOS_SCRIPT_AR.md
=====

?????????????? ?????????? ?????????? ?????????? ?????????

????? ?????????????? ?????????????? ?????????????? ?????????????? ?????? ??????????.

=====
FILE: docs\governance\enterprise-hospital-platform\TRIAGE_AND_ROUTING_GUARDS_REPORT_AR.md
=====

?????? ?????? ?????? ?????????? ??? ?????????? ??????????

????? ?????????? ?????????? ?????????? ?????????? ?????? ??? ?????????? ?????????? ?????????? ?????????? ??????????.

?????? ?????????? ??????????
1. **??? ?????????? ?? ?????????**?: ??? ?????? ?? ??? ?????????? ?????????? ?????????? ?? ?????????? ?????????? ?????????? ??????????.
2. **????????? ?????? ?????? ?????????**?: ?????????? ?????????? ?????????? ?????????? ?????????? ?????? ??? ?????? ?????? ?????????? ?????? ??
?????.

=====
FILE: docs\governance\enterprise-hospital-platform\UNIT_TEST_PLAN_AR.md
=====

??? ?????????? ?????????? ?????????? ??????????
```

=====

FILE: docs\governance\enterprise-hospital-platform\USER\_MANUAL\_AR.md

=====

# ??? ???? ???? ??????? ???? ?

?? ???? ???? ??????? ?? ??????? ??????? ??????? ???????.

=====

FILE: docs\governance\enterprise-hospital-platform\USER\_STORIES\_ACCEPTANCE\_CRITERIA\_AR.md

=====

# ?? ????????? ??????? ?????? ????????

?????????? ???? ?????????? ??????? ?????????? ??????????.

=====

FILE: docs\governance\enterprise-hospital-platform\WIREFRAMES\_AND MOCKUPS\_AR.md

=====

# ????? ?????? ????????? ?????? ???????

???? ????????? ?????????? ????????? ?????? ???? Mega Menu.

=====

FILE: docs\governance\enterprise-hospital-platform\WIREFRAMES\_AND MOCKUPS\_MASTER\_AR.md

=====

# ????? ?????? ????????? ?????? ????????? ?

???? ?????????? ?????????? ????????? ?????? ???? Mega Menu ???? Switcher.

=====

FILE: docs\migrations\blood\_bank\01\_blood\_bank\_e13\_down.sql

=====

-- =====

-- E13 BLOOD BANK ? candidate migration (DOWN / reverse)

-- =====

-- Reverses ONLY what 01\_...\_up.sql added. It DROPS the genuinely new table

-- (blood\_bank\_transfusion\_reactions) FIRST, then removes the policies, the

-- RLS FORCE, the FK constraints, the indexes, and the columns added to the

-- four PRE-EXISTING tables. It NEVER drops units/donors/crossmatch/

-- transfusions ? those existed before E13 and must survive a rollback.

BEGIN;

-- 1) drop the new table first (and its dependents via CASCADE-safe order)

DROP TABLE IF EXISTS blood\_bank\_transfusion\_reactions CASCADE;

```
-- 2) blood_bank_transfusions ? reverse additions only
DROP POLICY IF EXISTS rls_blood_bank_transfusions_tenant_isolation ON blood_bank_transfusions;
ALTER TABLE blood_bank_transfusions NO FORCE ROW LEVEL SECURITY;
ALTER TABLE blood_bank_transfusions DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_blood_bank_transfusions_tenant;
DROP INDEX IF EXISTS idx_blood_bank_transfusions_tenant_unit;
ALTER TABLE blood_bank_transfusions DROP CONSTRAINT IF EXISTS fk_blood_bank_transfusions_tenant;
ALTER TABLE blood_bank_transfusions DROP COLUMN IF EXISTS tenant_id;
ALTER TABLE blood_bank_transfusions DROP COLUMN IF EXISTS facility_id;
ALTER TABLE blood_bank_transfusions DROP COLUMN IF EXISTS crossmatch_id;
ALTER TABLE blood_bank_transfusions DROP COLUMN IF EXISTS created_by;
ALTER TABLE blood_bank_transfusions DROP COLUMN IF EXISTS updated_at;
```

```
-- 3) blood_bank_crossmatch ? reverse additions only
DROP POLICY IF EXISTS rls_blood_bank_crossmatch_tenant_isolation ON blood_bank_crossmatch;
ALTER TABLE blood_bank_crossmatch NO FORCE ROW LEVEL SECURITY;
ALTER TABLE blood_bank_crossmatch DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_blood_bank_crossmatch_tenant;
DROP INDEX IF EXISTS idx_blood_bank_crossmatch_tenant_patient;
ALTER TABLE blood_bank_crossmatch DROP CONSTRAINT IF EXISTS fk_blood_bank_crossmatch_tenant;
ALTER TABLE blood_bank_crossmatch DROP COLUMN IF EXISTS tenant_id;
ALTER TABLE blood_bank_crossmatch DROP COLUMN IF EXISTS created_by;
ALTER TABLE blood_bank_crossmatch DROP COLUMN IF EXISTS updated_at;
```

```
-- 4) blood_bank_donors ? reverse additions only
DROP POLICY IF EXISTS rls_blood_bank_donors_tenant_isolation ON blood_bank_donors;
ALTER TABLE blood_bank_donors NO FORCE ROW LEVEL SECURITY;
ALTER TABLE blood_bank_donors DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_blood_bank_donors_tenant;
ALTER TABLE blood_bank_donors DROP CONSTRAINT IF EXISTS fk_blood_bank_donors_tenant;
ALTER TABLE blood_bank_donors DROP COLUMN IF EXISTS tenant_id;
ALTER TABLE blood_bank_donors DROP COLUMN IF EXISTS created_by;
ALTER TABLE blood_bank_donors DROP COLUMN IF EXISTS updated_at;
```

```
-- 5) blood_bank_units ? reverse additions only
DROP POLICY IF EXISTS rls_blood_bank_units_tenant_isolation ON blood_bank_units;
ALTER TABLE blood_bank_units NO FORCE ROW LEVEL SECURITY;
ALTER TABLE blood_bank_units DISABLE ROW LEVEL SECURITY;
DROP INDEX IF EXISTS idx_blood_bank_units_tenant;
DROP INDEX IF EXISTS idx_blood_bank_units_tenant_expiry;
ALTER TABLE blood_bank_units DROP CONSTRAINT IF EXISTS fk_blood_bank_units_tenant;
ALTER TABLE blood_bank_units DROP COLUMN IF EXISTS tenant_id;
ALTER TABLE blood_bank_units DROP COLUMN IF EXISTS facility_id;
ALTER TABLE blood_bank_units DROP COLUMN IF EXISTS created_by;
ALTER TABLE blood_bank_units DROP COLUMN IF EXISTS updated_at;
```

```
COMMIT;
```

```
=====
FILE: docs\migrations\blood_bank\01_blood_bank_e13_up.sql
=====
```

```

-- =====
-- E13 BLOOD BANK ? candidate migration (UP) [PLAN ONLY ? do not auto-execute]
-- =====
-- Brings the four pre-existing blood_bank_* tables under tenant isolation
-- (tenant_id NOT NULL REFERENCES tenants(id) + FORCE RLS + canonical policy),
-- adds audit/state columns, and creates the NEW blood_bank_transfusion_reactions
-- table for reaction reporting + recall/lookback.
--
-- Idempotent: re-runnable. Adds columns via ADD COLUMN IF NOT EXISTS; policies
-- via DROP POLICY IF EXISTS then CREATE. NOT NULL on tenant_id is applied only
-- after backfill, and guarded so a second run does not fail.
--
-- NOTE: this migration ALTERS pre-existing tables (units/donors/crossmatch/
-- transfusions) ? it never DROPS them. The DOWN reverses ONLY these additions
-- and drops ONLY the genuinely new reactions table.
BEGIN;

-- -----
-- 1) blood_bank_units
-- -----

ALTER TABLE blood_bank_units ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
ALTER TABLE blood_bank_units ADD COLUMN IF NOT EXISTS facility_id INTEGER;
ALTER TABLE blood_bank_units ADD COLUMN IF NOT EXISTS created_by INTEGER;
ALTER TABLE blood_bank_units ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
-- Backfill legacy rows to tenant 1 (single-tenant legacy data) before NOT NULL.
UPDATE blood_bank_units SET tenant_id = 1 WHERE tenant_id IS NULL;
DO $$ BEGIN
 ALTER TABLE blood_bank_units ALTER COLUMN tenant_id SET NOT NULL;
EXCEPTION WHEN others THEN NULL; END $$;
DO $$ BEGIN
 ALTER TABLE blood_bank_units
 ADD CONSTRAINT fk_blood_bank_units_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE
;
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
CREATE INDEX IF NOT EXISTS idx_blood_bank_units_tenant ON blood_bank_units (tenant_id);
CREATE INDEX IF NOT EXISTS idx_blood_bank_units_tenant_expiry ON blood_bank_units (tenant_id, expiry_date, sta
tus);
ALTER TABLE blood_bank_units ENABLE ROW LEVEL SECURITY;
ALTER TABLE blood_bank_units FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_blood_bank_units_tenant_isolation ON blood_bank_units;
CREATE POLICY rls_blood_bank_units_tenant_isolation ON blood_bank_units
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), ' '::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), ' '::integer);

-- -----
-- 2) blood_bank_donors
-- -----

ALTER TABLE blood_bank_donors ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
ALTER TABLE blood_bank_donors ADD COLUMN IF NOT EXISTS created_by INTEGER;
ALTER TABLE blood_bank_donors ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
UPDATE blood_bank_donors SET tenant_id = 1 WHERE tenant_id IS NULL;
DO $$ BEGIN

```

```

ALTER TABLE blood_bank_donors ALTER COLUMN tenant_id SET NOT NULL;
EXCEPTION WHEN others THEN NULL; END $$;
DO $$ BEGIN
 ALTER TABLE blood_bank_donors
 ADD CONSTRAINT fk_blood_bank_donors_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
CREATE INDEX IF NOT EXISTS idx_blood_bank_donors_tenant ON blood_bank_donors (tenant_id);
ALTER TABLE blood_bank_donors ENABLE ROW LEVEL SECURITY;
ALTER TABLE blood_bank_donors FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_blood_bank_donors_tenant_isolation ON blood_bank_donors;
CREATE POLICY rls_blood_bank_donors_tenant_isolation ON blood_bank_donors
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- -----
-- 3) blood_bank_crossmatch
-- -----
ALTER TABLE blood_bank_crossmatch ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
ALTER TABLE blood_bank_crossmatch ADD COLUMN IF NOT EXISTS created_by INTEGER;
ALTER TABLE blood_bank_crossmatch ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
UPDATE blood_bank_crossmatch SET tenant_id = 1 WHERE tenant_id IS NULL;
DO $$ BEGIN
 ALTER TABLE blood_bank_crossmatch ALTER COLUMN tenant_id SET NOT NULL;
EXCEPTION WHEN others THEN NULL; END $$;
DO $$ BEGIN
 ALTER TABLE blood_bank_crossmatch
 ADD CONSTRAINT fk_blood_bank_crossmatch_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
CREATE INDEX IF NOT EXISTS idx_blood_bank_crossmatch_tenant ON blood_bank_crossmatch (tenant_id);
CREATE INDEX IF NOT EXISTS idx_blood_bank_crossmatch_tenant_patient ON blood_bank_crossmatch (tenant_id, patient_id);
ALTER TABLE blood_bank_crossmatch ENABLE ROW LEVEL SECURITY;
ALTER TABLE blood_bank_crossmatch FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_blood_bank_crossmatch_tenant_isolation ON blood_bank_crossmatch;
CREATE POLICY rls_blood_bank_crossmatch_tenant_isolation ON blood_bank_crossmatch
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- -----
-- 4) blood_bank_transfusions
-- -----
ALTER TABLE blood_bank_transfusions ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
ALTER TABLE blood_bank_transfusions ADD COLUMN IF NOT EXISTS facility_id INTEGER;
ALTER TABLE blood_bank_transfusions ADD COLUMN IF NOT EXISTS crossmatch_id INTEGER;
ALTER TABLE blood_bank_transfusions ADD COLUMN IF NOT EXISTS created_by INTEGER;
ALTER TABLE blood_bank_transfusions ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
;
UPDATE blood_bank_transfusions SET tenant_id = 1 WHERE tenant_id IS NULL;
DO $$ BEGIN
 ALTER TABLE blood_bank_transfusions ALTER COLUMN tenant_id SET NOT NULL;

```

```

EXCEPTION WHEN others THEN NULL; END $$;
DO $$ BEGIN
 ALTER TABLE blood_bank_transfusions
 ADD CONSTRAINT fk_blood_bank_transfusions_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE
CASCADE;
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
CREATE INDEX IF NOT EXISTS idx_blood_bank_transfusions_tenant ON blood_bank_transfusions (tenant_id);
CREATE INDEX IF NOT EXISTS idx_blood_bank_transfusions_tenant_unit ON blood_bank_transfusions (tenant_id, unit
_id);
ALTER TABLE blood_bank_transfusions ENABLE ROW LEVEL SECURITY;
ALTER TABLE blood_bank_transfusions FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_blood_bank_transfusions_tenant_isolation ON blood_bank_transfusions;
CREATE POLICY rls_blood_bank_transfusions_tenant_isolation ON blood_bank_transfusions
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- -----
-- 5) blood_bank_transfusion_reactions (NEW TABLE ? created by E13)
-- -----

CREATE TABLE IF NOT EXISTS blood_bank_transfusion_reactions (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 transfusion_id INTEGER NOT NULL REFERENCES blood_bank_transfusions(id) ON DELETE CASCADE,
 unit_id INTEGER,
 patient_id INTEGER,
 reaction_type TEXT DEFAULT '',
 severity TEXT DEFAULT 'Mild',
 reaction_details TEXT DEFAULT '',
 vital_signs_after TEXT DEFAULT '',
 action_taken TEXT DEFAULT '',
 reported_by TEXT DEFAULT '',
 created_by INTEGER,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX IF NOT EXISTS idx_bb_reactions_tenant ON blood_bank_transfusion_reactions (tenant_id);
CREATE INDEX IF NOT EXISTS idx_bb_reactions_tenant_unit ON blood_bank_transfusion_reactions (tenant_id, unit_i
d);
ALTER TABLE blood_bank_transfusion_reactions ENABLE ROW LEVEL SECURITY;
ALTER TABLE blood_bank_transfusion_reactions FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_blood_bank_transfusion_reactions_tenant_isolation ON blood_bank_transfusion_reaction
s;
CREATE POLICY rls_blood_bank_transfusion_reactions_tenant_isolation ON blood_bank_transfusion_reactions
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: docs\migrations\blood_bank\01_blood_bank_e13_validate.sql

```



=====

```
-- =====
-- E13 BLOOD BANK ? candidate migration (VALIDATE)
-- =====
-- Emits a row for every UNMET invariant. A clean run returns ZERO rows.
-- Asserts, for each E13 table: tenant_id exists + NOT NULL, FK to tenants(id),
-- relrowsecurity (RLS enabled) AND relforcerowsecurity (FORCE), and the
-- canonical tenant-isolation policy is present.
```

```
WITH expected(tbl, policy) AS (
 VALUES
 ('blood_bank_units', 'rls_blood_bank_units_tenant_isolation'),
 ('blood_bank_donors', 'rls_blood_bank_donors_tenant_isolation'),
 ('blood_bank_crossmatch', 'rls_blood_bank_crossmatch_tenant_isolation'),
 ('blood_bank_transfusions', 'rls_blood_bank_transfusions_tenant_isolation'),
 ('blood_bank_transfusion_reactions', 'rls_blood_bank_transfusion_reactions_tenant_isolation')
)
-- 1) tenant_id column present + NOT NULL
SELECT e.tbl AS table_name, 'tenant_id missing or NULLABLE' AS violation
FROM expected e
WHERE NOT EXISTS (
 SELECT 1 FROM information_schema.columns c
 WHERE c.table_name = e.tbl AND c.column_name = 'tenant_id' AND c.is_nullable = 'NO'
)
UNION ALL
-- 2) FK from tenant_id -> tenants(id)
SELECT e.tbl, 'tenant_id FK to tenants(id) missing'
FROM expected e
WHERE NOT EXISTS (
 SELECT 1
 FROM pg_constraint con
 JOIN pg_class rel ON rel.oid = con.conrelid
 JOIN pg_class frel ON frel.oid = con.confrelid
 WHERE con.contype = 'f' AND rel.relname = e.tbl AND frel.relname = 'tenants'
)
UNION ALL
-- 3) RLS enabled AND forced
SELECT e.tbl, 'RLS not ENABLED+FORCED'
FROM expected e
WHERE NOT EXISTS (
 SELECT 1 FROM pg_class rel
 WHERE rel.relname = e.tbl AND rel.relrowsecurity = true AND rel.relforcerowsecurity = true
)
UNION ALL
-- 4) canonical tenant-isolation policy present
SELECT e.tbl, 'canonical tenant-isolation policy missing'
FROM expected e
WHERE NOT EXISTS (
 SELECT 1 FROM pg_policies p
 WHERE p.tablename = e.tbl AND p.policyname = e.policy
);
```

=====

FILE: docs\sql\surgery\_or\_rls\_up.sql

=====

```
-- =====
-- Surgery / Operating Room domain ? RLS candidate DDL (UP).
-- Idempotent. CANDIDATE ONLY ? not auto-executed by the app bootstrap.
-- Applies FORCE Row Level Security + canonical tenant-isolation policy to the
-- pre-existing surgery-domain tables. (E12 NEW tables ship in migrations/e12_001_*.)
-- Canonical policy predicate: tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer
-- =====

BEGIN;

-- surgeries
ALTER TABLE surgeries ENABLE ROW LEVEL SECURITY;
ALTER TABLE surgeries FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_surgeries_tenant_isolation ON surgeries;
CREATE POLICY rls_surgeries_tenant_isolation ON surgeries
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- surgery_preop_assessments
ALTER TABLE surgery_preop_assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE surgery_preop_assessments FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_surgery_preop_assessments_tenant_isolation ON surgery_preop_assessments;
CREATE POLICY rls_surgery_preop_assessments_tenant_isolation ON surgery_preop_assessments
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- surgery_preop_tests
ALTER TABLE surgery_preop_tests ENABLE ROW LEVEL SECURITY;
ALTER TABLE surgery_preop_tests FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_surgery_preop_tests_tenant_isolation ON surgery_preop_tests;
CREATE POLICY rls_surgery_preop_tests_tenant_isolation ON surgery_preop_tests
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- surgery_anesthesia_records
ALTER TABLE surgery_anesthesia_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE surgery_anesthesia_records FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_surgery_anesthesia_records_tenant_isolation ON surgery_anesthesia_records;
CREATE POLICY rls_surgery_anesthesia_records_tenant_isolation ON surgery_anesthesia_records
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- operating_rooms
ALTER TABLE operating_rooms ENABLE ROW LEVEL SECURITY;
ALTER TABLE operating_rooms FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_operating_rooms_tenant_isolation ON operating_rooms;
CREATE POLICY rls_operating_rooms_tenant_isolation ON operating_rooms
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
-- consent_forms
ALTER TABLE consent_forms ENABLE ROW LEVEL SECURITY;
ALTER TABLE consent_forms FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_consent_forms_tenant_isolation ON consent_forms;
CREATE POLICY rls_consent_forms_tenant_isolation ON consent_forms
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;
```

```
=====
FILE: engines\cardiology_engine.js
=====
```

```
/**
 * Cardiology Engine - General Cardiology Logic
 * Part of the NamaMedical Enterprise Platform
 *
 * This engine handles clinical calculations, risk stratification,
 * and data validation for General Cardiology.
 */

const cardiologyEngine = {
 /**
 * Calculate Heart Failure Risk based on EF% and comorbidities
 * @param {number} efPercentage - Ejection Fraction percentage
 * @param {Object} comorbidities - Patient comorbidities (diabetes, hypertension, etc.)
 * @returns {Object} Risk assessment
 */
 calculateHFRisk: (efPercentage, comorbidities = {}) => {
 let riskLevel = 'Low';
 let recommendation = 'Routine monitoring';

 if (efPercentage <= 40) {
 riskLevel = 'High (HFrEF)';
 recommendation = 'Initiate GDMT (Guideline-Directed Medical Therapy)';
 } else if (efPercentage <= 50) {
 riskLevel = 'Moderate (HFpEF/HFmrEF)';
 recommendation = 'Optimize blood pressure and fluid management';
 }

 if (comorbidities.diabetes || comorbidities.renalFailure) {
 riskLevel += ' - Complex';
 recommendation += ' - Close multidisciplinary follow-up required';
 }

 return {
 riskLevel,
 recommendation,
 timestamp: new Date().toISOString()
 };
 },
};
```

```

/**
 * Validate ECG Interpretation against common patterns
 * @param {string} interpretation - The AI or Doctor's interpretation
 * @returns {boolean} Validity status
 */
validateECGInterpretation: (interpretation) => {
 if (!interpretation || interpretation.length < 5) return false;
 // Basic check for medical keywords to ensure it's not junk data
 const keywords = ['sinus', 'rhythm', 'axis', 'st-elevation', 'block', 'tachycardia', 'bradycardia'];
 return keywords.some(word => interpretation.toLowerCase().includes(word));
},

/**
 * Calculate Contrast Volume Limit based on GFR (Glomerular Filtration Rate)
 * To prevent Contrast-Induced Nephropathy (CIN)
 * @param {number} gfr - Patient's GFR value
 * @returns {Object} Limit and warning
 */
calculateContrastLimit: (gfr) => {
 if (gfr >= 60) return { limit: 'Standard', warning: 'No restriction' };
 if (gfr >= 30 && gfr < 60) return { limit: 'Moderate', warning: 'Limit contrast to < 100ml, ensure hydration' };
 if (gfr < 30) return { limit: 'Strict', warning: 'High risk of CIN. Consider alternative imaging or aggressive hydration' };
 return { limit: 'Unknown', warning: 'GFR missing. Do not proceed without manual review' };
}
};

module.exports = cardiologyEngine;

```

```

=====
FILE: engines\ep_engine.js
=====

```

```

/**
 * Electrophysiology (EP) Engine - Cardiac Rhythm & Device Logic
 * Part of the NamaMedical Enterprise Platform
 *
 * Specialized logic for arrhythmia analysis, ablation planning,
 * and cardiac device (Pacemaker/ICD) management.
 */

const epEngine = {
 /**
 * Analyze Cardiac Rhythm and suggest ablation target
 * @param {Object} ecgData - Signal data from the EP study
 * @returns {Object} Analysis and suggested target
 */
 analyzeRhythm: (ecgData) => {
 const { heartRate, pWaveMorphology, qrsDuration, prInterval } = ecgData;
 let diagnosis = 'Normal Sinus Rhythm';

```

```

let targetSite = 'N/A';
let confidence = '100%';

if (heartRate > 150 && pWaveLacks) {
 diagnosis = 'Atrial Fibrillation (AFib)';
 targetSite = 'Pulmonary Vein Ostium';
 confidence = '92%';
} else if (qrsDuration > 120 && prInterval > 200) {
 diagnosis = 'First-Degree AV Block';
 targetSite = 'AV Node';
 confidence = '88%';
}

return {
 diagnosis,
 suggestedTarget: targetSite,
 confidence,
 timestamp: new Date().toISOString()
};
},

/**
 * Validate Device Parameters (Pacemaker/ICD)
 * @param {Object} params - Device settings (Sensitivity, Output, Rate)
 * @returns {Object} Validation result
 */
validateDeviceParams: (params) => {
 const { sensitivity, outputVoltage, lowerRateLimit } = params;
 let status = 'Optimal';
 let warnings = [];

 if (sensitivity < 0.5 || sensitivity > 2.5) {
 status = 'Sub-optimal';
 warnings.push('Sensitivity outside recommended range (0.5-2.5mV)');
 }
 if (outputVoltage < 1.0) {
 status = 'Critical';
 warnings.push('Low output voltage: Risk of failure to capture');
 }

 return {
 status,
 warnings,
 isSafe: status === 'Optimal'
 };
},

/**
 * Calculate Ablation Energy Safety
 * @param {number} joules - Energy delivered
 * @param {string} modality - RF, Cryo, or Laser
 * @returns {Object} Safety check
 */
checkAblationSafety: (joules, modality) => {

```

```

 const limits = { 'RF': 40, 'Cryo': 100, 'Laser': 20 };
 const limit = limits[modality] || 30;

 return {
 isSafe: joules <= limit,
 alert: joules > limit ? `Energy exceeds ${modality} safety threshold of ${limit}J` : null
 };
}
};

```

```

module.exports = epEngine;

```

```

=====
FILE: engines\gastro_engine.js
=====

```

```

/**
 * Gastroenterology Clinical Engine
 * Gold Standard Implementation: S-MODE
 * Handles: Endoscopy Analysis, Child-Pugh Scoring, and Hepatic Risk Assessment
 */

```

```

class GastroEngine {
 constructor() {
 this.CHILD_PUGH_LEVELS = {
 A: 'Compensated',
 B: 'Significant Functional Impairment',
 C: 'Decompensated'
 };
 }
}

```

```

/**
 * Calculate Child-Pugh Score for Liver Cirrhosis
 * @param {Object} data { bilirubin, albumin, ascites, encephalopathy, prothrombinTime }
 * @returns {Object} { score: number, class: string, description: string }
 */

```

```

calculateChildPugh(data) {
 let score = 0;

 // Bilirubin (mg/dL)
 if (data.bilirubin < 2) score += 1;
 else if (data.bilirubin < 3) score += 2;
 else score += 3;

 // Albumin (g/dL)
 if (data.albumin > 3.5) score += 1;
 else if (data.albumin > 2.8) score += 2;
 else score += 3;

 // Ascites
 if (data.ascites === 'none') score += 1;
 else if (data.ascites === 'mild') score += 2;

```

```

else score += 3;

// Encephalopathy
if (data.encephalopathy === 'none') score += 1;
else if (data.encephalopathy === 'grade1-2') score += 2;
else score += 3;

// Prothrombin Time (INR)
if (data.inr < 1.7) score += 1;
else if (data.inr < 2.3) score += 2;
else score += 3;

let cls = 'C';
if (score <= 6) cls = 'A';
else if (score <= 7 || score <= 9) cls = 'B'; // Simplified logic

return {
 score,
 class: cls,
 description: this.CHILD_PUGH_LEVELS[cls]
};
}

/**
 * Analyze Endoscopy Findings (S-MODE)
 * @param {Object} findings { type, bostonBowelScore, biopsyTaken }
 */
analyzeEndoscopy(findings) {
 let risk = 'Low';
 if (findings.bostonBowelScore && findings.bostonBowelScore < 6) {
 risk = 'High (Poor Preparation)';
 }
 if (findings.biopsyTaken) {
 risk = 'Pending Histopathology';
 }
 return { risk, recommendation: 'Follow-up based on biopsy results' };
}

/**
 * Hepatic Encephalopathy Risk Bridge
 * @param {Object} data { ammoniaLevel, asterixisPresent, confusionLevel }
 */
assessHepaticRisk(data) {
 if (data.ammoniaLevel > 100 && data.asterixisPresent) {
 return {
 risk: 'CRITICAL',
 finding: 'Acute Hepatic Encephalopathy suspected.',
 action: 'Immediate administration of Lactulose and Rifaximin. Monitor airway.'
 };
 }
 return { risk: 'STABLE', finding: 'No acute hepatic encephalopathy signs.' };
}
}

```

```
module.exports = new GastroEngine();
```

```
=====
```

```
FILE: engines\interventional_cardiology_engine.js
```

```
=====
```

```
/**
 * Interventional Cardiology Engine - Cath Lab Logic
 * Part of the NamaMedical Enterprise Platform
 *
 * Specialized logic for PCI, Stenting, and Hemodynamic monitoring.
 */

const interventionalCardiologyEngine = {
 /**
 * Calculate Door-to-Balloon (D2B) Time
 * @param {Date} doorTime - Time of arrival at hospital
 * @param {Date} balloonTime - Time of first balloon inflation
 * @returns {Object} Time delta and performance category
 */
 calculateD2BTime: (doorTime, balloonTime) => {
 const diffMs = new Date(balloonTime) - new Date(doorTime);
 const diffMin = Math.floor(diffMs / 60000);

 let category = 'Optimal';
 if (diffMin > 90) category = 'Sub-optimal (Exceeds 90min Guideline)';
 else if (diffMin > 60) category = 'Acceptable';

 return {
 minutes: diffMin,
 category,
 guideline: 'ACC/AHA STEMI Guidelines'
 };
 },

 /**
 * Contrast-Induced Nephropathy (CIN) Risk Assessment
 * @param {number} gfr - Glomerular Filtration Rate
 * @param {number} volumeMl - Planned contrast volume
 * @returns {Object} Risk level and hydration recommendation
 */
 assessCINRisk: (gfr, volumeMl) => {
 let risk = 'Low';
 let action = 'Standard hydration';

 if (gfr < 30) {
 risk = 'Critical';
 action = 'Aggressive pre- and post-procedure hydration; consider alternative imaging';
 } else if (gfr < 45 && volumeMl > 100) {
 risk = 'Moderate';
 action = 'Limit contrast volume; ensure adequate hydration';
 }
 }
}
```



```

 return { risk, action };
 },

 /**
 * Stent Pressure Analysis (Post-Dilation)
 * @param {number} pressure - Measured pressure in mmHg
 * @returns {Object} Success status
 */
 analyzeStentPressure: (pressure) => {
 const isOptimal = pressure >= 14; // Standard target for optimal stent expansion
 return {
 isOptimal,
 status: isOptimal ? 'Optimal Expansion' : 'Under-expansion Risk',
 recommendation: isOptimal ? 'Proceed to closure' : 'Consider further dilation'
 };
 }
};

module.exports = interventionalCardiologyEngine;

```

```

=====
FILE: engines\obgyn_engine.js
=====

```

```

/**
 * OBGYN & Pediatrics Clinical Engine
 * Gold Standard Implementation: S-MODE
 * Handles: EDD Calculation, APGAR Scoring, and Gestational Age Tracking
 */

class ObgynEngine {
 constructor() {}

 /**
 * Calculate Estimated Date of Delivery (EDD) using Naegele's Rule
 * @param {string} lmp - Last Menstrual Period (YYYY-MM-DD)
 * @returns {string} EDD (YYYY-MM-DD)
 */
 calculateEDD(lmp) {
 if (!lmp) throw new Error('LMP date is required for EDD calculation');
 const date = new Date(lmp);
 if (isNaN(date.getTime())) throw new Error('Invalid LMP date format');

 // Naegele's Rule: LMP + 9 months + 7 days
 date.setMonth(date.getMonth() + 9);
 date.setDate(date.getDate() + 7);

 return date.toISOString().split('T')[0];
 }
}

/**

```

```

* Calculate Gestational Age in Weeks and Days
* @param {string} lmp - Last Menstrual Period (YYYY-MM-DD)
* @param {string} today - Current Date (YYYY-MM-DD)
*/
calculateGestationalAge(lmp, today = new Date().toISOString().split('T')[0]) {
 const start = new Date(lmp);
 const end = new Date(today);
 if (isNaN(start.getTime()) || isNaN(end.getTime())) throw new Error('Invalid dates provided');

 const diffTime = Math.abs(end - start);
 const diffDays = Math.ceil(diffTime / (1000 * 60 * 60 * 24));

 const weeks = Math.floor(diffDays / 7);
 const days = diffDays % 7;

 return { weeks, days, formatted: `${weeks}w ${days}d` };
}

/**
* Calculate APGAR Score
* @param {Object} data { appearance, pulse, grimace, activity, respiration }
* Each value should be 0, 1, or 2
*/
calculateAPGAR(data) {
 const score = (data.appearance || 0) +
 (data.pulse || 0) +
 (data.grimace || 0) +
 (data.activity || 0) +
 (data.respiration || 0);

 let interpretation = 'Normal';
 if (score <= 3) interpretation = 'Severely Depressed';
 else if (score <= 6) interpretation = 'Moderately Depressed';

 return { score, interpretation };
}

/**
* Preeclampsia Risk Assessment (S-MODE)
* @param {Object} data { bpSystolic, bpDiastolic, proteinuria }
*/
assessPreeclampsiaRisk(data) {
 const isHypertensive = data.bpSystolic >= 140 || data.bpDiastolic >= 90;
 const hasProteinuria = data.proteinuria === 'positive' || data.proteinuria === '+++';

 if (isHypertensive && hasProteinuria) {
 return {
 risk: 'CRITICAL',
 finding: 'Strong evidence of Preeclampsia.',
 action: 'Immediate stabilization, Magnesium Sulfate protocol, and urgent delivery evaluation.'
 };
 }

 if (isHypertensive) {
 return { risk: 'MODERATE', finding: 'Gestational Hypertension detected.', action: 'Close monitorin

```

```

g of BP and fetal well-being.' }];
 }
 return { risk: 'LOW', finding: 'No immediate signs of preeclampsia.' };
 }
}

```

```

module.exports = new ObgynEngine();

```

```

=====
FILE: engines\pulmonology_engine.js
=====

```

```

/**
 * Pulmonology Clinical Engine
 * Gold Standard Implementation: S-MODE
 * Handles: PFT Analysis, GOLD Criteria, Sleep Apnea Scoring, and Cardiology Bridge
 */

const {
 // Assume these are available in the global scope or imported from a core utility
 // In a real implementation, we would import from ../utils/validation.js etc.
} = require('../utils/clinical_utils');

class PulmonologyEngine {
 constructor() {
 this.GOLD_CRITERIA = {
 STAGE_1: { minFev1: 80, label: 'Mild' },
 STAGE_2: { minFev1: 50, label: 'Moderate' },
 STAGE_3: { minFev1: 30, label: 'Severe' },
 STAGE_4: { minFev1: 0, label: 'Very Severe' }
 };
 }

 /**
 * Calculate GOLD Stage for COPD based on FEV1 % Predicted
 * @param {number} fev1Actual
 * @param {number} fev1Predicted
 * @returns {Object} { stage: string, label: string }
 */
 calculateGOLDStage(fev1Actual, fev1Predicted) {
 if (!fev1Actual || !fev1Predicted) throw new Error('Missing FEV1 data for GOLD calculation');

 const percentPredicted = (fev1Actual / fev1Predicted) * 100;

 if (percentPredicted >= 80) return { stage: 'GOLD 1', label: 'Mild' };
 if (percentPredicted >= 50) return { stage: 'GOLD 2', label: 'Moderate' };
 if (percentPredicted >= 30) return { stage: 'GOLD 3', label: 'Severe' };
 return { stage: 'GOLD 4', label: 'Very Severe' };
 }

 /**
 * Analyze PFT Results (Spirometry)

```

```

* @param {Object} data { fev1Actual, fvcActual, fev1Predicted, fvcPredicted }
*/
analyzeSpirometry(data) {
 const { fev1Actual, fvcActual, fev1Predicted, fvcPredicted } = data;
 const ratio = (fev1Actual / fvcActual);
 const percentPredicted = (fev1Actual / fev1Predicted) * 100;

 let interpretation = '';
 if (ratio < 0.7) {
 interpretation = 'Obstructive Pattern';
 const gold = this.calculateGOLDStage(fev1Actual, fev1Predicted);
 interpretation += ` (${gold.stage}: ${gold.label})`;
 } else if (percentPredicted < 80) {
 interpretation = 'Restrictive Pattern';
 } else {
 interpretation = 'Normal Spirometry';
 }

 return {
 ratio: ratio.toFixed(2),
 percentPredicted: percentPredicted.toFixed(2),
 interpretation,
 goldStage: ratio < 0.7 ? this.calculateGOLDStage(fev1Actual, fev1Predicted).stage : 'N/A'
 };
}

/**
 * Sleep Apnea Severity (AHI Index)
 * @param {number} ahi
 */
calculateSleepApneaSeverity(ahi) {
 if (ahi < 5) return 'Normal';
 if (ahi < 15) return 'Mild OSA';
 if (ahi < 30) return 'Moderate OSA';
 return 'Severe OSA';
}

/**
 * Cardiology Bridge: Cor Pulmonale Risk Assessment
 * Checks if pulmonary hypertension is likely causing right heart failure
 * @param {Object} pulmData { fev1, oxygenSat }
 * @param {Object} cardioData { rightVentriclePressure, ecgFindings }
 */
assessCorPulmonaleRisk(pulmData, cardioData) {
 const isSeverePulm = pulmData.fev1 < 30; // Severe obstruction
 const isRightHeartStrained = cardioData.rightVentriclePressure > 25; // mmHg

 if (isSeverePulm && isRightHeartStrained) {
 return {
 risk: 'HIGH',
 finding: 'Strong evidence of Cor Pulmonale due to chronic pulmonary disease.',
 action: 'Immediate Cardiology Consultation for Right Heart Failure management.'
 };
 }
}

```

```

 return { risk: 'LOW', finding: 'No immediate evidence of Cor Pulmonale.' };
 }
}

```

```

module.exports = new PulmonologyEngine();

```

```

=====
FILE: engines\surgery_engine.js
=====

```

```

/**
 * General Surgery Clinical Engine
 * Gold Standard Implementation: S-MODE
 * Handles: Surgical Risk Assessment, Wound Healing Tracking, and Implant Registry
 */

class SurgeryEngine {
 constructor() {
 this.ASA_PHYSICAL_STATUS = {
 1: 'Healthy patient',
 2: 'Patient with mild systemic disease',
 3: 'Patient with severe systemic disease',
 4: 'Patient with severe systemic disease that is a constant threat to life',
 5: 'Moribund patient who is not expected to survive without the operation'
 };
 }

 /**
 * Calculate Surgical Risk based on ASA Score and Comorbidities
 * @param {Object} data { asaScore, age, comorbiditiesCount, emergencyStatus }
 */
 calculateSurgicalRisk(data) {
 let riskScore = 0;

 // ASA Score weight
 riskScore += (data.asaScore || 1) * 2;

 // Age weight
 if (data.age > 65) riskScore += 2;
 if (data.age > 80) riskScore += 3;

 // Comorbidities
 riskScore += (data.comorbiditiesCount || 0) * 1.5;

 // Emergency status
 if (data.emergencyStatus === 'Emergency') riskScore += 5;

 let category = 'Low';
 if (riskScore > 15) category = 'Very High';
 else if (riskScore > 10) category = 'High';
 else if (riskScore > 5) category = 'Moderate';
 }
}

```

```

 return {
 score: riskScore,
 category,
 recommendation: category === 'High' || category === 'Very High'
 ? 'Pre-operative cardiac clearance and ICU bed reservation strongly recommended.'
 : 'Standard pre-operative protocol.'
 };
 }

 /**
 * Analyze Wound Status
 * @param {Object} data { woundStatus, drainageAmount, drainageType }
 */
 analyzeWound(data) {
 if (data.woundStatus === 'Infected') {
 return {
 status: 'CRITICAL',
 finding: 'Surgical Site Infection (SSI) detected.',
 action: 'Immediate wound culture, swab, and surgical review for possible debridement.'
 };
 }
 if (data.drainageAmount > 100) {
 return {
 status: 'WARNING',
 finding: 'Excessive drainage detected.',
 action: 'Monitor for hematoma or lymphatic leak.'
 };
 }
 return { status: 'STABLE', finding: 'Wound healing as expected.' };
 }
}

```

```

module.exports = new SurgeryEngine();

```

```

=====

```

```

FILE: migrations\e0_01_tenants_archetype_down.sql

```

```

=====

```

```

-- e0_01_tenants_archetype_down.sql (rollback of e0_01_tenants_archetype_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ??? ???? archetype ?????. idempotent (IF EXISTS). ?????: ??? ?????? ????? ????????.
BEGIN;

```

```

ALTER TABLE tenants DROP CONSTRAINT IF EXISTS chk_tenants_archetype;
ALTER TABLE tenants DROP COLUMN IF EXISTS archetype;
ALTER TABLE tenants DROP COLUMN IF EXISTS moh_license;
ALTER TABLE tenants DROP COLUMN IF EXISTS cr_no;
ALTER TABLE tenants DROP COLUMN IF EXISTS vat_no;

```

```

COMMIT;

```

```

=====
FILE: migrations\e0_01_tenants_archetype_up.sql
=====

-- =====
-- e0_01_tenants_archetype_up.sql
-- E0 Facility Onboarding Wizard ? migration 1 of 3.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? ???? archetype ??? tenants (??? ??????: medical_city / large_hospital /
-- general_hospital / polyclinic / health_center). ????? facilityType/FACILITY_ALLOWED ?? ??????.
--
-- idempotent: ADD COLUMN IF NOT EXISTS ? ??? ?????? ??????. ?? RLS ??? tenants (????? ???????
-- ?????? ?????? ?????????? ??????? super-admin? RLS ??? tenants ????? = ??? ?????). ?? ?????? ???.
-- =====
BEGIN;

ALTER TABLE tenants ADD COLUMN IF NOT EXISTS archetype VARCHAR(50);

-- ?????? ?????? ?????? (??? ???): ?????? ?????? ?????? ?????? ?????????? ?????? ??????.
-- idempotent: ADD COLUMN IF NOT EXISTS. ?????? ?? ?????? (????????).
ALTER TABLE tenants ADD COLUMN IF NOT EXISTS moh_license TEXT;
ALTER TABLE tenants ADD COLUMN IF NOT EXISTS cr_no TEXT;
ALTER TABLE tenants ADD COLUMN IF NOT EXISTS vat_no TEXT;

-- ??? ??? ?????? ?????? ?????????? (idempotent: ?????? ?? ?????). NULL ?????? (????????? ??????).
ALTER TABLE tenants DROP CONSTRAINT IF EXISTS chk_tenants_archetype;
ALTER TABLE tenants ADD CONSTRAINT chk_tenants_archetype
 CHECK (archetype IS NULL OR archetype IN
 ('medical_city','large_hospital','general_hospital','polyclinic','health_center'));

COMMIT;

```

```

=====
FILE: migrations\e0_01_tenants_archetype_validate.sql
=====

-- e0_01_tenants_archetype_validate.sql (run AFTER up.sql; read-only)
-- PASS = tenants has archetype column + CHECK constraint present.
SELECT
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='tenants' AND column_name='archetype') AS has_archetype_col, -- expect 1
 (SELECT count(*) FROM pg_constraint
 WHERE conname='chk_tenants_archetype') AS has_check_constraint, -- expect 1
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='tenants'
 AND column_name IN ('moh_license','cr_no','vat_no')) AS has_reg_id_cols; -- expect 3
-- Verify constraint rejects bad values and accepts valid ones (sanity, no row written):
-- INSERT ... archetype='bogus' must fail with check_violation; 'polyclinic' must pass.

```

=====

FILE: migrations\e0\_02\_facilities\_extend\_down.sql

=====

```
-- e0_02_facilities_extend_down.sql (rollback of e0_02_facilities_extend_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ??? facilities ??? ??? ??? (id, tenant_id, name, tax_number, created_at).
-- ????: ?? ??? RLS ?????? ?? ??? facilities ??? RLS ?? ?? ??????. ?? ??? ?? ?????? ???.
-- idempotent (IF EXISTS). ??? ?????? ?????? ??????.
BEGIN;

DROP POLICY IF EXISTS rls_facilities_tenant_isolation ON facilities;
-- ????? facilities ??? ??? ??? RLS (???? ??? ? initDatabase ?? ? ????).
ALTER TABLE facilities NO FORCE ROW LEVEL SECURITY;
ALTER TABLE facilities DISABLE ROW LEVEL SECURITY;

DROP INDEX IF EXISTS idx_facilities_parent;
DROP INDEX IF EXISTS idx_facilities_tenant;

ALTER TABLE facilities DROP CONSTRAINT IF EXISTS chk_facilities_type;
ALTER TABLE facilities DROP CONSTRAINT IF EXISTS fk_facilities_parent;

ALTER TABLE facilities DROP COLUMN IF EXISTS parent_facility_id;
ALTER TABLE facilities DROP COLUMN IF EXISTS timezone;
ALTER TABLE facilities DROP COLUMN IF EXISTS currency;
ALTER TABLE facilities DROP COLUMN IF EXISTS beds;
ALTER TABLE facilities DROP COLUMN IF EXISTS type;

COMMIT;
```

=====

FILE: migrations\e0\_02\_facilities\_extend\_up.sql

=====

```
-- =====
-- e0_02_facilities_extend_up.sql
-- E0 Facility Onboarding Wizard ? migration 2 of 3.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ????: ????? facilities ????? (id, tenant_id FK?tenants, name, tax_number, created_at)
-- ????? ??????: type/archetype ?????? ?? ?????? ?????? ?????? ?????? ?
-- parent_facility_id (self-FK) ??? "???? ??" (???? ???? ?? ????? ??).
-- ?? ????? FORCE RLS ??? tenant_id (?? ????? ????? ?? 150 ?????) + ??? tenant_id.
--
-- idempotent: ADD COLUMN IF NOT EXISTS + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS.
-- ??????: facilities ????? ?? tenant_id? ?? ?????? ?? ????? ?? ?????? (nama_medical_app)
-- ?????/???? ?? ?? ??? app.tenant_id. ????? ?????? ?? ?????? ??? ??? ?????? ???
-- app.tenant_id ?? ?????? ?????? ?? INSERT (???? ???? /api/admin/facilities/provision).
-- =====
BEGIN;
```



```

ALTER TABLE facilities ADD COLUMN IF NOT EXISTS type VARCHAR(50);
ALTER TABLE facilities ADD COLUMN IF NOT EXISTS beds INTEGER NOT NULL DEFAULT 0;
ALTER TABLE facilities ADD COLUMN IF NOT EXISTS currency VARCHAR(10) NOT NULL DEFAULT 'SAR';
ALTER TABLE facilities ADD COLUMN IF NOT EXISTS timezone VARCHAR(64) NOT NULL DEFAULT 'Asia/Riyadh';
ALTER TABLE facilities ADD COLUMN IF NOT EXISTS parent_facility_id INTEGER;

-- self-FK ?????? ???? (????? ??? ? ?????? ??????). idempotent ??? ??? pg_constraint.
DO $$
BEGIN
 IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname = 'fk_facilities_parent') THEN
 ALTER TABLE facilities
 ADD CONSTRAINT fk_facilities_parent
 FOREIGN KEY (parent_facility_id) REFERENCES facilities(id) ON DELETE SET NULL;
 END IF;
END $$;

-- ??? ??? ??? ???????? (??? ?????? ????????). NULL ?????? ?????? ?????????.
ALTER TABLE facilities DROP CONSTRAINT IF EXISTS chk_facilities_type;
ALTER TABLE facilities ADD CONSTRAINT chk_facilities_type
 CHECK (type IS NULL OR type IN
 ('medical_city', 'large_hospital', 'general_hospital', 'polyclinic', 'health_center'));

-- ??? tenant_id (??? ??? ???????? ????????).
CREATE INDEX IF NOT EXISTS idx_facilities_tenant ON facilities (tenant_id);
CREATE INDEX IF NOT EXISTS idx_facilities_parent ON facilities (parent_facility_id);

-- FORCE RLS ??? tenant_id (???? ??? ?????? ?? ??? tenant_*_overrides ??????).
ALTER TABLE facilities ENABLE ROW LEVEL SECURITY;
ALTER TABLE facilities FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_facilities_tenant_isolation ON facilities;
CREATE POLICY rls_facilities_tenant_isolation ON facilities
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: migrations\e0_02_facilities_extend_validate.sql
=====

-- e0_02_facilities_extend_validate.sql (run AFTER up.sql; read-only)
-- PASS = facilities has all 5 new columns + parent self-FK + FORCE RLS + isolation policy + indexes.
SELECT
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='facilities'
 AND column_name IN ('type', 'beds', 'currency', 'timezone', 'parent_facility_id')) AS new_cols, -- e
xpect 5
 (SELECT count(*) FROM pg_constraint WHERE conname='fk_facilities_parent') AS has_parent_fk, -- e
xpect 1
 (SELECT count(*) FROM pg_constraint WHERE conname='chk_facilities_type') AS has_type_check, -- e
xpect 1

```

```

 (SELECT relforcerowsecurity FROM pg_class WHERE relname='facilities') AS force_rls, -- e
expect t
 (SELECT count(*) FROM pg_policies
 WHERE tablename='facilities' AND policyname='rls_facilities_tenant_isolation') AS has_policy, --
expect 1
 (SELECT count(*) FROM pg_indexes
 WHERE tablename='facilities' AND indexname IN ('idx_facilities_tenant','idx_facilities_parent')) AS idx_c
nt; -- expect 2
SELECT rolname, rolsuper, rolbypassrls FROM pg_roles WHERE rolname='nama_medical_app'; -- expect false/false
(app role unchanged)

```

=====

FILE: migrations\e0\_03\_facility\_modules\_down.sql

=====

```

-- e0_03_facility_modules_down.sql (rollback of e0_03_facility_modules_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ???? facility_modules ??????? (??????? ??????). idempotent (IF EXISTS).
BEGIN;

```

```

DROP POLICY IF EXISTS rls_facility_modules_tenant_isolation ON facility_modules;
DROP TABLE IF EXISTS facility_modules;

```

```

COMMIT;

```

=====

FILE: migrations\e0\_03\_facility\_modules\_up.sql

=====

```

-- =====
-- e0_03_facility_modules_up.sql
-- E0 Facility Onboarding Wizard ? migration 3 of 3.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? ???? facility_modules (??????? ?????????? ??? ??????) ????? ??????????? ?????/????
-- ??? FACILITY_ALLOWED ?? app.js. module_index = ??? NAV_ITEMS (0..42).
-- FORCE RLS ??? tenant_id (??? ??? 150 ?????) + ??? tenant_id + UNIQUE(tenant_id, module_index).
--
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS.
-- =====
BEGIN;

```

```

CREATE TABLE IF NOT EXISTS facility_modules (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 module_index INTEGER NOT NULL,
 enabled BOOLEAN NOT NULL DEFAULT TRUE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT uq_facility_module UNIQUE (tenant_id, module_index),
 CONSTRAINT chk_facility_module_index CHECK (module_index >= 0 AND module_index <= 42)

```

```
);
```

```
CREATE INDEX IF NOT EXISTS idx_facility_modules_tenant ON facility_modules (tenant_id);
```

```
ALTER TABLE facility_modules ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE facility_modules FORCE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_facility_modules_tenant_isolation ON facility_modules;
```

```
CREATE POLICY rls_facility_modules_tenant_isolation ON facility_modules
```

```
FOR ALL
```

```
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
```

```
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e0_03_facility_modules_validate.sql
```

```
=====
```

```
-- e0_03_facility_modules_validate.sql (run AFTER up.sql; read-only)
```

```
-- PASS = facility_modules exists with tenant_id FK + FORCE RLS + isolation policy + unique + index + check.
```

```
SELECT
```

```
(SELECT count(*) FROM information_schema.tables
```

```
WHERE table_name='facility_modules') AS table_exists,
```

```
-- expect 1
```

```
(SELECT count(*) FROM information_schema.columns
```

```
WHERE table_name='facility_modules'
```

```
AND column_name IN ('tenant_id','module_index','enabled')) AS cols,
```

```
-- expect 3
```

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='facility_modules') AS force_rls, -- expect t
```

```
(SELECT count(*) FROM pg_policies
```

```
WHERE tablename='facility_modules'
```

```
AND policyname='rls_facility_modules_tenant_isolation') AS has_policy,
```

```
-- expect 1
```

```
(SELECT count(*) FROM pg_constraint WHERE conname='uq_facility_module') AS has_unique,
```

```
-- expect 1
```

```
(SELECT count(*) FROM pg_constraint WHERE conname='chk_facility_module_index') AS has_check, -- expect 1
```

```
(SELECT count(*) FROM pg_indexes
```

```
WHERE tablename='facility_modules' AND indexname='idx_facility_modules_tenant') AS has_idx; -- expect 1
```

```
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
```

```
WHERE conrelid='facility_modules'::regclass AND confrelid='tenants'::regclass AND contype='f'; -- expect 1
```

```
=====
```

```
FILE: migrations\e0_04_integration_settings_rls_down.sql
```

```
=====
```

```
-- e0_04_integration_settings_rls_down.sql (rollback of e0_04_integration_settings_rls_up.sql)
```

```
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
```

```
-- ????? ????? ????? ??????? FORCE/ENABLE RLS ??? integration_settings. idempotent (IF EXISTS).
```

```
-- ?????? ?????: ??????? ????? ????? ? ?? ??????? ??? ??? ?????? ????? ????? E0.
```

```
BEGIN;
```

```
DROP POLICY IF EXISTS rls_integration_settings_tenant_isolation ON integration_settings;
```

```
ALTER TABLE integration_settings NO FORCE ROW LEVEL SECURITY;
```

```
ALTER TABLE integration_settings DISABLE ROW LEVEL SECURITY;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e0_04_integration_settings_rls_up.sql
```

```
=====
```

```
-- =====
```

```
-- e0_04_integration_settings_rls_up.sql
```

```
-- E0 Facility Onboarding Wizard ? migration 4 (RLS hardening).
```

```
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
```

```
--
```

```
-- ?????: ????? FORCE RLS ??? tenant_id ??? integration_settings (??? ??? 150 ????? ???????)
```

```
-- ??? ????? ????? ?????????? ?????????? ?????? ??????? ?????? ??? ??????. ??? tenant_id ?????
```

```
-- ?????? (????? ?? db_postgres.js) ? ?? ????? ?? DDL ??? ??????.
```

```
--
```

```
-- idempotent: DROP POLICY IF EXISTS ??? CREATE? ENABLE/FORCE ??? ?????? ???????.
```

```
-- =====
```

```
BEGIN;
```

```
ALTER TABLE integration_settings ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE integration_settings FORCE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_integration_settings_tenant_isolation ON integration_settings;
```

```
CREATE POLICY rls_integration_settings_tenant_isolation ON integration_settings
```

```
FOR ALL
```

```
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
```

```
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e0_04_integration_settings_rls_validate.sql
```

```
=====
```

```
-- e0_04_integration_settings_rls_validate.sql (run AFTER up.sql; read-only)
```

```
-- PASS = integration_settings has tenant_id column + FORCE RLS + isolation policy.
```

```
SELECT
```

```
(SELECT count(*) FROM information_schema.columns
```

```
WHERE table_name='integration_settings' AND column_name='tenant_id') AS has_tenant_col, -- expect
```

```
1
```

```
(SELECT relrowsecurity FROM pg_class WHERE relname='integration_settings') AS rls_enabled, -- expect
```

```
t
```

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='integration_settings') AS force_rls, -- expect
```

```
t
```

```
(SELECT count(*) FROM pg_policies
```

```
WHERE tablename='integration_settings'
```

```
AND policyname='rls_integration_settings_tenant_isolation') AS has_policy; -- expect
```

```
1
```

=====

FILE: migrations\e100\_cardiology\_general\_down.sql

=====

-- Migration: e100\_cardiology\_general\_down.sql  
-- Description: Rollback tables for General Cardiology module

BEGIN;

DROP TABLE IF EXISTS cardiac\_medications;

DROP TABLE IF EXISTS ecg\_reports;

DROP TABLE IF EXISTS cardiology\_visits;

COMMIT;

=====

FILE: migrations\e100\_cardiology\_general\_up.sql

=====

-- Migration: e100\_cardiology\_general\_up.sql  
-- Description: Create tables for General Cardiology module with Tenant Isolation (RLS)

BEGIN;

-- 1. Cardiology Visits Table

CREATE TABLE IF NOT EXISTS cardiology\_visits (  
 id UUID PRIMARY KEY DEFAULT gen\_random\_uuid(),  
 tenant\_id UUID NOT NULL,  
 patient\_id UUID NOT NULL,  
 doctor\_id UUID NOT NULL,  
 bp\_systolic INTEGER,  
 bp\_diastolic INTEGER,  
 heart\_rate INTEGER,  
 ef\_percentage DECIMAL(5,2),  
 notes TEXT,  
 created\_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT\_TIMESTAMP,  
 updated\_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT\_TIMESTAMP  
);

-- 2. ECG Reports Table

CREATE TABLE IF NOT EXISTS ecg\_reports (  
 id UUID PRIMARY KEY DEFAULT gen\_random\_uuid(),  
 tenant\_id UUID NOT NULL,  
 visit\_id UUID REFERENCES cardiology\_visits(id) ON DELETE CASCADE,  
 patient\_id UUID NOT NULL,  
 ecg\_type VARCHAR(50), -- e.g., Resting, Stress, Holter  
 interpretation TEXT,  
 rhythm VARCHAR(100),  
 axis VARCHAR(50),  
 conclusion TEXT,  
 ecg\_image\_url TEXT,

```

 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Cardiac Medications Table
CREATE TABLE IF NOT EXISTS cardiac_medications (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 drug_name VARCHAR(255) NOT NULL,
 dosage VARCHAR(100),
 frequency VARCHAR(100),
 start_date DATE,
 end_date DATE,
 is_active BOOLEAN DEFAULT TRUE,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable Row Level Security (RLS)
ALTER TABLE cardiology_visits ENABLE ROW LEVEL SECURITY;
ALTER TABLE ecg_reports ENABLE ROW LEVEL SECURITY;
ALTER TABLE cardiac_medications ENABLE ROW LEVEL SECURITY;

-- Create RLS Policies (Tenant Isolation)
CREATE POLICY tenant_isolation_policy ON cardiology_visits
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON ecg_reports
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON cardiac_medications
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

-- Indexes for Performance
CREATE INDEX idx_cardio_visits_patient ON cardiology_visits(patient_id);
CREATE INDEX idx_ecg_reports_visit ON ecg_reports(visit_id);
CREATE INDEX idx_cardio_meds_patient ON cardiac_medications(patient_id);

COMMIT;

=====
FILE: migrations\e100_pulmonology_core_down.sql
=====

-- Migration: e100_pulmonology_core_down.sql
-- Description: Revert Pulmonology Core Schema

BEGIN;

DROP TABLE IF EXISTS pulmonology_bronchoscopy;
DROP TABLE IF EXISTS pulmonology_sleep_studies;
DROP TABLE IF EXISTS pulmonology_pft_results;
DROP TABLE IF EXISTS pulmonology_encounters;

```

COMMIT;

=====

FILE: migrations\e100\_pulmonology\_core\_up.sql

=====

-- Migration: e100\_pulmonology\_core\_up.sql  
-- Description: Core schema for Pulmonology Department (S-MODE Gold Standard)  
-- Compliance: RLS Enabled, Tenant Isolation, Clinical Audit Trail

BEGIN;

-- 1. Pulmonology Encounters (Main clinical record)

```
CREATE TABLE IF NOT EXISTS pulmonology_encounters (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 doctor_id UUID NOT NULL,
 encounter_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 chief_complaint TEXT,
 respiratory_history TEXT,
 smoking_status VARCHAR(50), -- e.g., Never, Former, Current, Pack-Years
 pack_years NUMERIC(5,2),
 physical_exam_findings JSONB, -- Breath sounds, percussion, etc.
 diagnosis_code VARCHAR(20), -- ICD-10
 treatment_plan TEXT,
 status VARCHAR(20) DEFAULT 'active', -- active, completed, cancelled
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-- 2. Pulmonary Function Tests (PFT) - The Gold Standard Data

```
CREATE TABLE IF NOT EXISTS pulmonology_pft_results (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 encounter_id UUID NOT NULL REFERENCES pulmonology_encounters(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 test_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 fev1_predicted NUMERIC(5,2),
 fev1_actual NUMERIC(5,2),
 fvc_predicted NUMERIC(5,2),
 fvc_actual NUMERIC(5,2),
 fev1_fvc_ratio NUMERIC(5,2),
 dlco_predicted NUMERIC(5,2),
 dlco_actual NUMERIC(5,2),
 interpretation TEXT,
 gold_stage VARCHAR(20), -- GOLD 1, 2, 3, 4
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-- 3. Sleep Studies (Polysomnography)

```
CREATE TABLE IF NOT EXISTS pulmonology_sleep_studies (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 study_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 study_type VARCHAR(50), -- e.g., CPAP, BiPAP, Sleep Apnea Study
 apnea_hypopnea_index NUMERIC(5,2),
 oxygen_saturation_min NUMERIC(5,2),
 arousal_index NUMERIC(5,2),
 sleep_efficiency NUMERIC(5,2),
 total_sleep_time NUMERIC(5,2),
 total_wake_time NUMERIC(5,2),
 total_apnea_hypopnea_time NUMERIC(5,2),
 total_arousal_time NUMERIC(5,2),
 total_oxygen_desaturation_time NUMERIC(5,2),
 interpretation TEXT,
 gold_stage VARCHAR(20), -- GOLD 1, 2, 3, 4
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
encounter_id UUID NOT NULL REFERENCES pulmonology_encounters(id) ON DELETE CASCADE,
tenant_id UUID NOT NULL,
study_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
ahi_index NUMERIC(5,2), -- Apnea-Hypopnea Index
lowest_oxygen_saturation NUMERIC(5,2),
sleep_efficiency NUMERIC(5,2),
diagnosis VARCHAR(100), -- Obstructive Sleep Apnea, Central Sleep Apnea, etc.
recommendation TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Bronchoscopy Reports
CREATE TABLE IF NOT EXISTS pulmonology_bronchoscopy (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 encounter_id UUID NOT NULL REFERENCES pulmonology_encounters(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 procedure_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 findings TEXT,
 biopsy_taken BOOLEAN DEFAULT FALSE,
 specimen_details TEXT,
 complications TEXT,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- ENABLE RLS (Row Level Security) - NON-NEGOTIABLE
ALTER TABLE pulmonology_encounters ENABLE ROW LEVEL SECURITY;
ALTER TABLE pulmonology_pft_results ENABLE ROW LEVEL SECURITY;
ALTER TABLE pulmonology_sleep_studies ENABLE ROW LEVEL SECURITY;
ALTER TABLE pulmonology_bronchoscopy ENABLE ROW LEVEL SECURITY;

-- RLS Policies: Only allow access if tenant_id matches the session context
CREATE POLICY pulmonology_encounters_tenant_policy ON pulmonology_encounters
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY pulmonology_pft_results_tenant_policy ON pulmonology_pft_results
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY pulmonology_sleep_studies_tenant_policy ON pulmonology_sleep_studies
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY pulmonology_bronchoscopy_tenant_policy ON pulmonology_bronchoscopy
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

-- Indexes for Performance
CREATE INDEX idx_pulm_enc_patient ON pulmonology_encounters(patient_id);
CREATE INDEX idx_pulm_enc_tenant ON pulmonology_encounters(tenant_id);
CREATE INDEX idx_pulm_pft_enc ON pulmonology_pft_results(encounter_id);
CREATE INDEX idx_pulm_sleep_enc ON pulmonology_sleep_studies(encounter_id);

COMMIT;

```



```
=====
FILE: migrations\e101_gastroenterology_core_down.sql
=====
```

```
-- Migration: e101_gastroenterology_core_down.sql
-- Description: Revert Gastroenterology Core Schema
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS gastro_hepatic_markers;
DROP TABLE IF EXISTS gastro_endoscopy_reports;
DROP TABLE IF EXISTS gastro_encounters;
```

```
COMMIT;
```

```
=====
FILE: migrations\e101_gastroenterology_core_up.sql
=====
```

```
-- Migration: e101_gastroenterology_core_up.sql
-- Description: Core schema for Gastroenterology Department (S-MODE Gold Standard)
-- Compliance: RLS Enabled, Tenant Isolation, Clinical Audit Trail
```

```
BEGIN;
```

```
-- 1. Gastroenterology Encounters
```

```
CREATE TABLE IF NOT EXISTS gastro_encounters (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 doctor_id UUID NOT NULL,
 encounter_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 chief_complaint TEXT,
 gi_history TEXT,
 dietary_habits TEXT,
 physical_exam_findings JSONB, -- Abdominal palpation, bowel sounds, etc.
 diagnosis_code VARCHAR(20), -- ICD-10
 treatment_plan TEXT,
 status VARCHAR(20) DEFAULT 'active',
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. Endoscopy Reports (Gastrosocopy, Colonoscopy, ERCP)
```

```
CREATE TABLE IF NOT EXISTS gastro_endoscopy_reports (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 encounter_id UUID NOT NULL REFERENCES gastro_encounters(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 procedure_type VARCHAR(50), -- Gastrosocopy, Colonoscopy, ERCP, PEG
 procedure_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 sedation_used VARCHAR(100),
 findings TEXT,
```

```

 biopsy_taken BOOLEAN DEFAULT FALSE,
 biopsy_sites TEXT,
 boston_bowel_score INT, -- For Colonoscopy quality
 recommendation TEXT,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Liver Function & Hepatic Markers (Specialized Tracking)
CREATE TABLE IF NOT EXISTS gastro_hepatic_markers (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 encounter_id UUID NOT NULL REFERENCES gastro_encounters(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 bilirubin_total NUMERIC(5,2),
 bilirubin_direct NUMERIC(5,2),
 alt_level NUMERIC(5,2),
 ast_level NUMERIC(5,2),
 alk_phos NUMERIC(5,2),
 albumin NUMERIC(5,2),
 child_pugh_score VARCHAR(20), -- A, B, C
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- ENABLE RLS
ALTER TABLE gastro_encounters ENABLE ROW LEVEL SECURITY;
ALTER TABLE gastro_endoscopy_reports ENABLE ROW LEVEL SECURITY;
ALTER TABLE gastro_hepatic_markers ENABLE ROW LEVEL SECURITY;

CREATE POLICY gastro_encounters_tenant_policy ON gastro_encounters
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY gastro_endoscopy_tenant_policy ON gastro_endoscopy_reports
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY gastro_hepatic_tenant_policy ON gastro_hepatic_markers
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE INDEX idx_gastro_enc_patient ON gastro_encounters(patient_id);
CREATE INDEX idx_gastro_enc_tenant ON gastro_encounters(tenant_id);

COMMIT;

=====
FILE: migrations\e101_interventional_cardiology_down.sql
=====

-- Migration: e101_interventional_cardiology_down.sql
BEGIN;
DROP TABLE IF EXISTS pci_hemodynamics;
DROP TABLE IF EXISTS stent_registry;
DROP TABLE IF EXISTS pci_sessions;
COMMIT;

```

=====

FILE: migrations\e101\_interventional\_cardiology\_up.sql

=====

-- Migration: e101\_interventional\_cardiology\_up.sql

-- Description: Specialized tables for Interventional Cardiology (Cath Lab) with Tenant Isolation

BEGIN;

-- 1. PCI Sessions Table

```
CREATE TABLE IF NOT EXISTS pci_sessions (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 doctor_id UUID NOT NULL,
 access_site VARCHAR(50), -- Radial, Femoral, etc.
 contrast_volume_ml DECIMAL(10,2),
 fluoroscopy_time_min DECIMAL(10,2),
 door_to_balloon_min INTEGER,
 status VARCHAR(50), -- Scheduled, In-Progress, Completed, Complication
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-- 2. Stent Registry Table

```
CREATE TABLE IF NOT EXISTS stent_registry (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 session_id UUID REFERENCES pci_sessions(id) ON DELETE CASCADE,
 vessel_segment VARCHAR(100), -- LAD, LCx, RCA
 stent_brand VARCHAR(100),
 diameter_mm DECIMAL(4,2),
 length_mm DECIMAL(4,2),
 material VARCHAR(50), -- DES, BMS, Bioresorbable
 pressure_post_dilation DECIMAL(5,2),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-- 3. Hemodynamics Log (High Frequency)

```
CREATE TABLE IF NOT EXISTS pci_hemodynamics (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 session_id UUID REFERENCES pci_sessions(id) ON DELETE CASCADE,
 systolic_bp INTEGER,
 diastolic_bp INTEGER,
 heart_rate INTEGER,
 mean_ap INTEGER,
 timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-- Enable RLS

ALTER TABLE pci\_sessions ENABLE ROW LEVEL SECURITY;

ALTER TABLE stent\_registry ENABLE ROW LEVEL SECURITY;

```
ALTER TABLE pci_hemodynamics ENABLE ROW LEVEL SECURITY;
```

```
-- Tenant Isolation Policies
```

```
CREATE POLICY tenant_isolation_pci ON pci_sessions USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
CREATE POLICY tenant_isolation_stents ON stent_registry USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
CREATE POLICY tenant_isolation_hemo ON pci_hemodynamics USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
CREATE INDEX idx_pci_patient ON pci_sessions(patient_id);
```

```
CREATE INDEX idx_stent_session ON stent_registry(session_id);
```

```
CREATE INDEX idx_hemo_session ON pci_hemodynamics(session_id);
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e102_general_surgery_core_down.sql
```

```
=====
```

```
-- Migration: e102_general_surgery_core_down.sql
```

```
-- Description: Revert General Surgery Core Schema
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS surgery_implants;
```

```
DROP TABLE IF EXISTS surgery_wound_logs;
```

```
DROP TABLE IF EXISTS surgery_encounters;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e102_general_surgery_core_up.sql
```

```
=====
```

```
-- Migration: e102_general_surgery_core_up.sql
```

```
-- Description: Core schema for General Surgery Department (S-MODE Gold Standard)
```

```
-- Compliance: RLS Enabled, Tenant Isolation, Clinical Audit Trail
```

```
BEGIN;
```

```
-- 1. Surgery Encounters (Pre-op, Intra-op, Post-op)
```

```
CREATE TABLE IF NOT EXISTS surgery_encounters (
```

```
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
 tenant_id UUID NOT NULL,
```

```
 patient_id UUID NOT NULL,
```

```
 surgeon_id UUID NOT NULL,
```

```
 encounter_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
 surgical_indication TEXT,
```

```
 pre_op_diagnosis TEXT,
```

```

 post_op_diagnosis TEXT,
 surgical_approach VARCHAR(50), -- Open, Laparoscopic, Robotic, Combined
 anesthesia_type VARCHAR(50),
 operation_time_min INT,
 blood_loss_ml INT,
 complications TEXT,
 status VARCHAR(20) DEFAULT 'scheduled', -- scheduled, in-progress, completed, cancelled
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- 2. Surgical Site/Wound Tracking

```

CREATE TABLE IF NOT EXISTS surgery_wound_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 encounter_id UUID NOT NULL REFERENCES surgery_encounters(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 check_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 wound_status VARCHAR(50), -- Healing, Infected, Dehiscence, Seroma
 drainage_amount_ml NUMERIC(5,2),
 drainage_type VARCHAR(50),
 intervention_taken TEXT,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- 3. Surgical Implant Registry

```

CREATE TABLE IF NOT EXISTS surgery_implants (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 encounter_id UUID NOT NULL REFERENCES surgery_encounters(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 implant_type VARCHAR(100), -- Mesh, Stapler, Clip, Plate
 brand_model VARCHAR(100),
 serial_number VARCHAR(100),
 lot_number VARCHAR(100),
 position_details TEXT,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- ENABLE RLS

```

ALTER TABLE surgery_encounters ENABLE ROW LEVEL SECURITY;
ALTER TABLE surgery_wound_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE surgery_implants ENABLE ROW LEVEL SECURITY;

```

```

CREATE POLICY surgery_encounters_tenant_policy ON surgery_encounters
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

CREATE POLICY surgery_wound_tenant_policy ON surgery_wound_logs
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

CREATE POLICY surgery_implants_tenant_policy ON surgery_implants
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

CREATE INDEX idx_surg_enc_patient ON surgery_encounters(patient_id);
CREATE INDEX idx_surg_enc_tenant ON surgery_encounters(tenant_id);

```

COMMIT;

=====

FILE: migrations\e103\_obgyn\_core\_down.sql

=====

-- Migration: e103\_obgyn\_core\_down.sql

-- Description: Revert OBGYN Core Schema

BEGIN;

DROP TABLE IF EXISTS obgyn\_delivery\_records;

DROP TABLE IF EXISTS obgyn\_anc\_tracking;

DROP TABLE IF EXISTS obgyn\_encounters;

COMMIT;

=====

FILE: migrations\e103\_obgyn\_core\_up.sql

=====

-- Migration: e103\_obgyn\_core\_up.sql

-- Description: Core schema for OBGYN & Pediatrics (S-MODE Gold Standard)

-- Compliance: RLS Enabled, Tenant Isolation, Clinical Audit Trail

BEGIN;

-- 1. OBGYN Encounters (Prenatal, Postnatal, Gynae)

CREATE TABLE IF NOT EXISTS obgyn\_encounters (  
    id UUID PRIMARY KEY DEFAULT gen\_random\_uuid(),  
    tenant\_id UUID NOT NULL,  
    patient\_id UUID NOT NULL,  
    doctor\_id UUID NOT NULL,  
    encounter\_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT\_TIMESTAMP,  
    visit\_type VARCHAR(50), -- Prenatal, Postnatal, Gynae, Pediatric  
    gravida INT,  
    para INT,  
    lmp DATE, -- Last Menstrual Period  
    edd DATE, -- Estimated Date of Delivery  
    gestational\_age\_weeks NUMERIC(4,1),  
    chief\_complaint TEXT,  
    physical\_exam\_findings JSONB,  
    diagnosis\_code VARCHAR(20),  
    treatment\_plan TEXT,  
    status VARCHAR(20) DEFAULT 'active',  
    created\_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT\_TIMESTAMP,  
    updated\_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT\_TIMESTAMP  
);

-- 2. Antenatal Care (ANC) Tracking

```

CREATE TABLE IF NOT EXISTS obgyn_anc_tracking (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 encounter_id UUID NOT NULL REFERENCES obgyn_encounters(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 fundal_height_cm NUMERIC(4,1),
 fetal_heart_rate INT,
 fetal_presentation VARCHAR(50),
 maternal_bp VARCHAR(20),
 maternal_weight_kg NUMERIC(5,2),
 edema_grade VARCHAR(20),
 urine_protein VARCHAR(20),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- 3. Delivery & Neonatal Records

```

CREATE TABLE IF NOT EXISTS obgyn_delivery_records (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 encounter_id UUID NOT NULL REFERENCES obgyn_encounters(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 delivery_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 delivery_mode VARCHAR(50), -- Vaginal, C-Section, Vacuum, Forceps
 baby_gender VARCHAR(10),
 baby_weight_g INT,
 apgar_1min INT,
 apgar_5min INT,
 complications TEXT,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- ENABLE RLS

```

ALTER TABLE obgyn_encounters ENABLE ROW LEVEL SECURITY;
ALTER TABLE obgyn_anc_tracking ENABLE ROW LEVEL SECURITY;
ALTER TABLE obgyn_delivery_records ENABLE ROW LEVEL SECURITY;

```

```

CREATE POLICY obgyn_encounters_tenant_policy ON obgyn_encounters
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

CREATE POLICY obgyn_anc_tenant_policy ON obgyn_anc_tracking
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

CREATE POLICY obgyn_delivery_tenant_policy ON obgyn_delivery_records
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

```

```

CREATE INDEX idx_obgyn_enc_patient ON obgyn_encounters(patient_id);
CREATE INDEX idx_obgyn_enc_tenant ON obgyn_encounters(tenant_id);

```

```

COMMIT;

```

```

=====
FILE: migrations\e10_01_gl_structure_down.sql
=====

```

```

-- e10_01_gl_structure_down.sql (rollback of e10_01_gl_structure_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: ?????? ?????? (chart_of_accounts/journal_entries/journal_lines) ????? ?????? (?????
-- bootstrap) ? ???? ?? ???????. ???? ??? ?? ?????? ??? ??????: ?????????? ??????? ?????????
-- ?????????? ??????????. ????? tenant_id ???? ?????? ?? bootstrap ?????? ?????????? (????? ??? NOT NULL/FK
-- ???? ?????????). ????? REAL<-NUMERIC ??? ?????? ????????? ????????? (NUMERIC ???) ??????. idempotent.
BEGIN;

-- policies
DROP POLICY IF EXISTS rls_coa_tenant_isolation ON finance_chart_of_accounts;
DROP POLICY IF EXISTS rls_je_tenant_isolation ON finance_journal_entries;
DROP POLICY IF EXISTS rls_jl_tenant_isolation ON finance_journal_lines;

-- constraints added by this migration
ALTER TABLE finance_chart_of_accounts DROP CONSTRAINT IF EXISTS chk_coa_class;
ALTER TABLE finance_chart_of_accounts DROP CONSTRAINT IF EXISTS fk_coa_tenant;
ALTER TABLE finance_journal_entries DROP CONSTRAINT IF EXISTS chk_je_posting_status;
ALTER TABLE finance_journal_entries DROP CONSTRAINT IF EXISTS chk_je_source_type;
ALTER TABLE finance_journal_entries DROP CONSTRAINT IF EXISTS fk_je_tenant;
ALTER TABLE finance_journal_entries DROP CONSTRAINT IF EXISTS fk_je_reversal_of; -- L-1
ALTER TABLE finance_journal_lines DROP CONSTRAINT IF EXISTS chk_jl_sides;
ALTER TABLE finance_journal_lines DROP CONSTRAINT IF EXISTS fk_jl_entry;
ALTER TABLE finance_journal_lines DROP CONSTRAINT IF EXISTS fk_jl_account;
ALTER TABLE finance_journal_lines DROP CONSTRAINT IF EXISTS fk_jl_tenant;

-- indexes added by this migration
DROP INDEX IF EXISTS uq_coa_tenant_code;
DROP INDEX IF EXISTS idx_coa_tenant_id;
DROP INDEX IF EXISTS idx_je_tenant_id;
DROP INDEX IF EXISTS idx_je_tenant_status;
DROP INDEX IF EXISTS idx_jl_tenant_id;
DROP INDEX IF EXISTS idx_jl_entry;

-- columns added by this migration (tenant_id/facility_id/branch_id pre-existed in bootstrap -> kept)
ALTER TABLE finance_chart_of_accounts DROP COLUMN IF EXISTS account_class;
ALTER TABLE finance_chart_of_accounts DROP COLUMN IF EXISTS opening_balance;
ALTER TABLE finance_journal_entries DROP COLUMN IF EXISTS posting_status;
ALTER TABLE finance_journal_entries DROP COLUMN IF EXISTS source_type;
ALTER TABLE finance_journal_entries DROP COLUMN IF EXISTS posted_by;
ALTER TABLE finance_journal_entries DROP COLUMN IF EXISTS posted_at;
ALTER TABLE finance_journal_entries DROP COLUMN IF EXISTS balanced_at;
ALTER TABLE finance_journal_entries DROP COLUMN IF EXISTS reversal_of;

-- relax NOT NULL we added (column itself pre-existed in bootstrap)
ALTER TABLE finance_chart_of_accounts ALTER COLUMN tenant_id DROP NOT NULL;
ALTER TABLE finance_journal_entries ALTER COLUMN tenant_id DROP NOT NULL;
ALTER TABLE finance_journal_lines ALTER COLUMN tenant_id DROP NOT NULL;

COMMIT;

```

```
=====
```



FILE: migrations\e10\_01\_gl\_structure\_up.sql

=====

-- =====

-- e10\_01\_gl\_structure\_up.sql

-- E10 FINANCE / GENERAL LEDGER ? harden the EXISTING finance\_chart\_of\_accounts,  
-- finance\_journal\_entries, finance\_journal\_lines to a tenant-isolated, double-entry-safe shape.  
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).  
--

-- ?????: ??? ???? ?????? ????? (E10). ?????? ?????? ?????? ?????? (????? bootstrap) ????? ??? ???  
-- FORCE RLS? ???? ???? ?????? ?????? REAL (????? ???). ??????:  
-- - tenant\_id NOT NULL + FK -> tenants(id) + FORCE RLS ?????? ?????? ??? ?????? ???????.  
-- - finance\_journal\_lines: ????? debit/credit ??? NUMERIC(14,2) + FK account\_id -> CoA +  
-- FK entry\_id -> journal\_entries (ON DELETE CASCADE) + ??? CHECK ???? ?????? ???????.  
-- - finance\_journal\_entries: posting\_status enum-CHECK (DRAFT/POSTED/REVERSED) + source\_type +  
-- posted\_by/posted\_at/balanced\_at + reversal\_of (???? ?????? ??????).  
-- - finance\_chart\_of\_accounts: account\_class CHECK (Asset/Liability/Equity/Revenue/Expense) +  
-- opening\_balance NUMERIC(14,2) + ???? (tenant\_id, account\_code).  
-- ??? ?????? ?????? ?????? => down ?? ?????? ?? ??? ?????? ???.  
-- ?? ?????? ?????? ?????? ?? bootstrap ?? db\_postgres.js ??? ??? ?????? (???? ?????? ???).  
--

-- idempotent: ADD COLUMN IF NOT EXISTS + backfill + SET NOT NULL + DROP/ADD CONSTRAINT IF EXISTS  
-- + ALTER TYPE USING + CREATE INDEX IF NOT EXISTS + DROP/CREATE POLICY. wrapped BEGIN; ? COMMIT;  
-- =====

BEGIN;

-- ===== finance\_chart\_of\_accounts =====

ALTER TABLE finance\_chart\_of\_accounts ADD COLUMN IF NOT EXISTS tenant\_id INTEGER;  
UPDATE finance\_chart\_of\_accounts SET tenant\_id = 1 WHERE tenant\_id IS NULL;  
ALTER TABLE finance\_chart\_of\_accounts ALTER COLUMN tenant\_id SET NOT NULL;  
ALTER TABLE finance\_chart\_of\_accounts DROP CONSTRAINT IF EXISTS fk\_coa\_tenant;  
ALTER TABLE finance\_chart\_of\_accounts ADD CONSTRAINT fk\_coa\_tenant  
FOREIGN KEY (tenant\_id) REFERENCES tenants(id) ON DELETE CASCADE;  
ALTER TABLE finance\_chart\_of\_accounts ADD COLUMN IF NOT EXISTS account\_class TEXT;  
-- backfill account\_class from the existing free-text account\_type where recognisable  
UPDATE finance\_chart\_of\_accounts  
SET account\_class = CASE  
WHEN account\_type ILIKE 'asset%' THEN 'Asset'  
WHEN account\_type ILIKE 'liabilit%' THEN 'Liability'  
WHEN account\_type ILIKE 'equity%' THEN 'Equity'  
WHEN account\_type ILIKE 'revenue%' THEN 'Revenue'  
WHEN account\_type ILIKE 'income%' THEN 'Revenue'  
WHEN account\_type ILIKE 'expense%' THEN 'Expense'  
ELSE 'Asset' END  
WHERE account\_class IS NULL;  
ALTER TABLE finance\_chart\_of\_accounts DROP CONSTRAINT IF EXISTS chk\_coa\_class;  
ALTER TABLE finance\_chart\_of\_accounts ADD CONSTRAINT chk\_coa\_class  
CHECK (account\_class IN ('Asset','Liability','Equity','Revenue','Expense'));  
ALTER TABLE finance\_chart\_of\_accounts ADD COLUMN IF NOT EXISTS opening\_balance NUMERIC(14,2) DEFAULT 0;  
CREATE INDEX IF NOT EXISTS idx\_coa\_tenant\_id ON finance\_chart\_of\_accounts (tenant\_id);  
CREATE UNIQUE INDEX IF NOT EXISTS uq\_coa\_tenant\_code ON finance\_chart\_of\_accounts (tenant\_id, account\_code);  
ALTER TABLE finance\_chart\_of\_accounts ENABLE ROW LEVEL SECURITY;  
ALTER TABLE finance\_chart\_of\_accounts FORCE ROW LEVEL SECURITY;  
DROP POLICY IF EXISTS rls\_coa\_tenant\_isolation ON finance\_chart\_of\_accounts;

```

CREATE POLICY rls_coa_tenant_isolation ON finance_chart_of_accounts
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== finance_journal_entries =====
ALTER TABLE finance_journal_entries ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE finance_journal_entries SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE finance_journal_entries ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE finance_journal_entries DROP CONSTRAINT IF EXISTS fk_je_tenant;
ALTER TABLE finance_journal_entries ADD CONSTRAINT fk_je_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
ALTER TABLE finance_journal_entries ADD COLUMN IF NOT EXISTS posting_status TEXT DEFAULT 'DRAFT';
-- migrate the legacy is_posted boolean/int into the new state machine
UPDATE finance_journal_entries SET posting_status = 'POSTED'
 WHERE posting_status IS NULL AND is_posted = 1;
UPDATE finance_journal_entries SET posting_status = 'DRAFT' WHERE posting_status IS NULL;
ALTER TABLE finance_journal_entries DROP CONSTRAINT IF EXISTS chk_je_posting_status;
ALTER TABLE finance_journal_entries ADD CONSTRAINT chk_je_posting_status
 CHECK (posting_status IN ('DRAFT', 'POSTED', 'REVERSED'));
ALTER TABLE finance_journal_entries ADD COLUMN IF NOT EXISTS source_type TEXT DEFAULT 'MANUAL';
ALTER TABLE finance_journal_entries DROP CONSTRAINT IF EXISTS chk_je_source_type;
ALTER TABLE finance_journal_entries ADD CONSTRAINT chk_je_source_type
 CHECK (source_type IN ('MANUAL', 'INVOICE', 'SYSTEM', 'REVERSAL'));
ALTER TABLE finance_journal_entries ADD COLUMN IF NOT EXISTS posted_by INTEGER;
ALTER TABLE finance_journal_entries ADD COLUMN IF NOT EXISTS posted_at TIMESTAMP;
ALTER TABLE finance_journal_entries ADD COLUMN IF NOT EXISTS balanced_at TIMESTAMP;
ALTER TABLE finance_journal_entries ADD COLUMN IF NOT EXISTS reversal_of INTEGER;
-- L-1 fix: FK so reversal_of references a real journal entry (self-referential; SET NULL on delete)
ALTER TABLE finance_journal_entries DROP CONSTRAINT IF EXISTS fk_je_reversal_of;
ALTER TABLE finance_journal_entries ADD CONSTRAINT fk_je_reversal_of
 FOREIGN KEY (reversal_of) REFERENCES finance_journal_entries(id) ON DELETE SET NULL;
CREATE INDEX IF NOT EXISTS idx_je_tenant_id ON finance_journal_entries (tenant_id);
CREATE INDEX IF NOT EXISTS idx_je_tenant_status ON finance_journal_entries (tenant_id, posting_status);
ALTER TABLE finance_journal_entries ENABLE ROW LEVEL SECURITY;
ALTER TABLE finance_journal_entries FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_je_tenant_isolation ON finance_journal_entries;
CREATE POLICY rls_je_tenant_isolation ON finance_journal_entries
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== finance_journal_lines =====
ALTER TABLE finance_journal_lines ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE finance_journal_lines SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE finance_journal_lines ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE finance_journal_lines DROP CONSTRAINT IF EXISTS fk_jl_tenant;
ALTER TABLE finance_journal_lines ADD CONSTRAINT fk_jl_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
-- precision: REAL -> NUMERIC(14,2) (USING preserves existing values)
ALTER TABLE finance_journal_lines ALTER COLUMN debit TYPE NUMERIC(14,2) USING ROUND(debit::numeric, 2);
ALTER TABLE finance_journal_lines ALTER COLUMN credit TYPE NUMERIC(14,2) USING ROUND(credit::numeric, 2);
ALTER TABLE finance_journal_lines ALTER COLUMN debit SET DEFAULT 0;
ALTER TABLE finance_journal_lines ALTER COLUMN credit SET DEFAULT 0;

```

```

ALTER TABLE finance_journal_lines DROP CONSTRAINT IF EXISTS fk_jl_entry;
ALTER TABLE finance_journal_lines ADD CONSTRAINT fk_jl_entry
 FOREIGN KEY (entry_id) REFERENCES finance_journal_entries(id) ON DELETE CASCADE;
ALTER TABLE finance_journal_lines DROP CONSTRAINT IF EXISTS fk_jl_account;
ALTER TABLE finance_journal_lines ADD CONSTRAINT fk_jl_account
 FOREIGN KEY (account_id) REFERENCES finance_chart_of_accounts(id);
-- line invariant: no negatives, never both sides non-zero (double-entry hygiene)
ALTER TABLE finance_journal_lines DROP CONSTRAINT IF EXISTS chk_jl_sides;
ALTER TABLE finance_journal_lines ADD CONSTRAINT chk_jl_sides
 CHECK (debit >= 0 AND credit >= 0 AND NOT (debit > 0 AND credit > 0));
ALTER TABLE finance_journal_lines ADD COLUMN IF NOT EXISTS created_at TIMESTAMP DEFAULT now();
CREATE INDEX IF NOT EXISTS idx_jl_tenant_id ON finance_journal_lines (tenant_id);
CREATE INDEX IF NOT EXISTS idx_jl_entry ON finance_journal_lines (tenant_id, entry_id);
ALTER TABLE finance_journal_lines ENABLE ROW LEVEL SECURITY;
ALTER TABLE finance_journal_lines FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_jl_tenant_isolation ON finance_journal_lines;
CREATE POLICY rls_jl_tenant_isolation ON finance_journal_lines
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

```

```

COMMIT;

```

```

=====

```

```

FILE: migrations\e10_01_gl_structure_validate.sql

```

```

=====

```

```

-- e10_01_gl_structure_validate.sql (run AFTER e10_01_gl_structure_up.sql; read-only)
-- PASS = three GL tables tenant-isolated (FORCE RLS + policy + tenant_id NOT NULL FK), lines NUMERIC
-- with FK to entries+accounts + side CHECK, entries have posting_status CHECK.

```

```

SELECT
 -- chart_of_accounts
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='finance_chart_of_accounts')
S coa_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='finance_chart_of_accounts'
 AND policyname='rls_coa_tenant_isolation')
S coa_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='finance_chart_of_accounts'
 AND column_name='tenant_id' AND is_nullable='NO')
S coa_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='finance_chart_of_accounts'::regclass AND confrelid='tenants'::regclass AND contype='f')
AS coa_tenant_fk, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='finance_chart_of_accounts'::regclass AND conname='chk_coa_class')
AS coa_class_check, -- expect 1
 -- journal_entries
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='finance_journal_entries')
S je_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='finance_journal_entries'
 AND policyname='rls_je_tenant_isolation')

```

```

S je_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='finance_journal_entries'
 AND column_name='tenant_id' AND is_nullable='NO')
A
S je_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='finance_journal_entries'::regclass AND conname='chk_je_posting_status')
A
S je_status_check, -- expect 1
 -- journal_lines
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='finance_journal_lines')
A
S jl_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='finance_journal_lines'
 AND policyname='rls_jl_tenant_isolation')
A
S jl_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='finance_journal_lines'
 AND column_name='tenant_id' AND is_nullable='NO')
A
S jl_tenant_not_null, -- expect 1
 (SELECT data_type FROM information_schema.columns WHERE table_name='finance_journal_lines'
 AND column_name='debit')
A
S jl_debit_type, -- expect numeric
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='finance_journal_lines'::regclass
 AND confrelid='finance_journal_entries'::regclass AND contype='f')
A
S jl_entry_fk, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='finance_journal_lines'::regclass
 AND confrelid='finance_chart_of_accounts'::regclass AND contype='f')
A
S jl_account_fk, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='finance_journal_lines'::regclass AND conname='chk_jl_sides')
A
S jl_sides_check, -- expect 1
 -- L-1: reversal_of self-referential FK
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='finance_journal_entries'::regclass AND conname='fk_je_reversal_of')
A
S je_reversal_of_fk; -- expect 1

```

=====

FILE: migrations\e10\_02\_cost\_centers\_down.sql

=====

```

-- e10_02_cost_centers_down.sql (rollback of e10_02_cost_centers_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: finance_cost_centers ??? ???? => ?? ??????. ??? ??? ???????. idempotent.
BEGIN;

ALTER TABLE finance_journal_lines DROP CONSTRAINT IF EXISTS fk_jl_cost_center;

DROP POLICY IF EXISTS rls_cc_tenant_isolation ON finance_cost_centers;
ALTER TABLE finance_cost_centers DROP CONSTRAINT IF EXISTS fk_cc_tenant;
DROP INDEX IF EXISTS idx_cc_tenant_id;
ALTER TABLE finance_cost_centers DROP COLUMN IF EXISTS budget_amount;
-- facility_id may be shared by bootstrap conventions elsewhere; this migration added it -> drop.

```

```
ALTER TABLE finance_cost_centers DROP COLUMN IF EXISTS facility_id;
ALTER TABLE finance_cost_centers ALTER COLUMN tenant_id DROP NOT NULL;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e10_02_cost_centers_up.sql
```

```
=====
```

```
-- =====
-- e10_02_cost_centers_up.sql
-- E10 FINANCE ? harden the EXISTING finance_cost_centers to tenant isolation + FORCE RLS.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: finance_cost_centers ???? ????? (bootstrap) ????? clinic_id ??? ??? tenant_id ??? ???.
-- ??? tenant_id NOT NULL + FK -> tenants(id) + facility_id + budget_amount + FORCE RLS ?????
-- ??????. ????? clinic_id ??? ?? (???? ????? ?? ????? ??????? ?? "down ??? ??????? ??").
-- ??? ??? ????? => down ?? ??????. ?? ????? ????? ????? bootstrap ??? ??? ??????.
-- idempotent.
-- =====
```

```
BEGIN;
```

```
ALTER TABLE finance_cost_centers ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE finance_cost_centers SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE finance_cost_centers ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE finance_cost_centers DROP CONSTRAINT IF EXISTS fk_cc_tenant;
ALTER TABLE finance_cost_centers ADD CONSTRAINT fk_cc_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
ALTER TABLE finance_cost_centers ADD COLUMN IF NOT EXISTS facility_id INTEGER;
ALTER TABLE finance_cost_centers ADD COLUMN IF NOT EXISTS budget_amount NUMERIC(14,2) DEFAULT 0;
```

```
CREATE INDEX IF NOT EXISTS idx_cc_tenant_id ON finance_cost_centers (tenant_id);
```

```
ALTER TABLE finance_cost_centers ENABLE ROW LEVEL SECURITY;
ALTER TABLE finance_cost_centers FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_cc_tenant_isolation ON finance_cost_centers;
CREATE POLICY rls_cc_tenant_isolation ON finance_cost_centers
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
-- finance_journal_lines.cost_center_id may reference a cost center; add the FK now that CC is hardened.
ALTER TABLE finance_journal_lines DROP CONSTRAINT IF EXISTS fk_jl_cost_center;
-- legacy rows may carry cost_center_id=0 (sentinel "none") -> normalise to NULL before adding FK
UPDATE finance_journal_lines SET cost_center_id = NULL WHERE cost_center_id = 0;
ALTER TABLE finance_journal_lines ADD CONSTRAINT fk_jl_cost_center
 FOREIGN KEY (cost_center_id) REFERENCES finance_cost_centers(id);
```

```
COMMIT;
```

=====

FILE: migrations\e10\_02\_cost\_centers\_validate.sql

=====

```
-- e10_02_cost_centers_validate.sql (run AFTER e10_02_cost_centers_up.sql; read-only)
-- PASS = finance_cost_centers tenant-isolated (FORCE RLS + policy + tenant_id NOT NULL FK) +
-- journal_lines has cost_center FK.
```

SELECT

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='finance_cost_centers') A
S cc_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='finance_cost_centers'
 AND policyname='rls_cc_tenant_isolation') A
S cc_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='finance_cost_centers'
 AND column_name='tenant_id' AND is_nullable='NO') A
S cc_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='finance_cost_centers'::regclass AND confrelid='tenants'::regclass AND contype='f') A
S cc_tenant_fk, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='finance_cost_centers'
 AND column_name='budget_amount') A
S cc_budget_col, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='finance_journal_lines'::regclass
 AND confrelid='finance_cost_centers'::regclass AND contype='f') A
S jl_cc_fk; -- expect 1
```

=====

FILE: migrations\e10\_03\_zatca\_invoices\_down.sql

=====

```
-- e10_03_zatca_invoices_down.sql (rollback of e10_03_zatca_invoices_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: zatca_invoices ???? ?????? => ?? ??????. ??? ?????. tenant_id/facility_id/branch_id
-- ?? bootstrap ?????? ?????? (????? NOT NULL/FK ???). idempotent.
```

BEGIN;

```
DROP POLICY IF EXISTS rls_zatca_tenant_isolation ON zatca_invoices;
ALTER TABLE zatca_invoices DROP CONSTRAINT IF EXISTS chk_zatca_clearance;
ALTER TABLE zatca_invoices DROP CONSTRAINT IF EXISTS fk_zatca_invoice;
ALTER TABLE zatca_invoices DROP CONSTRAINT IF EXISTS fk_zatca_tenant;
DROP INDEX IF EXISTS uq_zatca_tenant_invoice;
DROP INDEX IF EXISTS idx_zatca_tenant_id;
ALTER TABLE zatca_invoices DROP COLUMN IF EXISTS ubl_xml;
ALTER TABLE zatca_invoices DROP COLUMN IF EXISTS digital_stamp;
ALTER TABLE zatca_invoices DROP COLUMN IF EXISTS qr_tlv;
ALTER TABLE zatca_invoices DROP COLUMN IF EXISTS clearance_status;
ALTER TABLE zatca_invoices ALTER COLUMN tenant_id DROP NOT NULL;
```

COMMIT;

```

=====
FILE: migrations\e10_03_zatca_invoices_up.sql
=====

-- =====
-- e10_03_zatca_invoices_up.sql
-- E10 ZATCA PHASE-2 E-INVOICE ? harden EXISTING zatca_invoices for UBL/stamp/QR + tenant isolation.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: zatca_invoices ???? ????? (bootstrap? ???? tenant_id/facility_id/branch_id ??? NOT NULL/FK
-- ??? ???). ?????:
-- - tenant_id NOT NULL + FK -> tenants(id) + FK invoice_id -> invoices(id) + FORCE RLS.
-- - ubl_xml TEXT (UBL 2.1) + digital_stamp TEXT (placeholder ??? ????? CSID) + xml_hash (?????)
-- + qr_tlv TEXT (TLV base64) + clearance_status enum-CHECK (NOT_SUBMITTED/RECORDED/CLEARED/REJECTED).
-- - ???? (tenant_id, invoice_id).
-- ???? ???? ????? => down ?? ??????. ?? ????? ????? ????? bootstrap ??? ??? ??????. idempotent.
-- =====
BEGIN;

ALTER TABLE zatca_invoices ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE zatca_invoices SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE zatca_invoices ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE zatca_invoices DROP CONSTRAINT IF EXISTS fk_zatca_tenant;
ALTER TABLE zatca_invoices ADD CONSTRAINT fk_zatca_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
ALTER TABLE zatca_invoices DROP CONSTRAINT IF EXISTS fk_zatca_invoice;
ALTER TABLE zatca_invoices ADD CONSTRAINT fk_zatca_invoice
 FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE CASCADE;

ALTER TABLE zatca_invoices ADD COLUMN IF NOT EXISTS ubl_xml TEXT DEFAULT '';
ALTER TABLE zatca_invoices ADD COLUMN IF NOT EXISTS digital_stamp TEXT DEFAULT '';
ALTER TABLE zatca_invoices ADD COLUMN IF NOT EXISTS qr_tlv TEXT DEFAULT '';
ALTER TABLE zatca_invoices ADD COLUMN IF NOT EXISTS clearance_status TEXT DEFAULT 'NOT_SUBMITTED';
UPDATE zatca_invoices SET clearance_status = 'NOT_SUBMITTED' WHERE clearance_status IS NULL;
ALTER TABLE zatca_invoices DROP CONSTRAINT IF EXISTS chk_zatca_clearance;
ALTER TABLE zatca_invoices ADD CONSTRAINT chk_zatca_clearance
 CHECK (clearance_status IN ('NOT_SUBMITTED', 'RECORDED', 'CLEARED', 'REJECTED'));

-- one e-invoice artifact per source invoice per tenant (idempotent generate -> upsert)
CREATE UNIQUE INDEX IF NOT EXISTS uq_zatca_tenant_invoice ON zatca_invoices (tenant_id, invoice_id);
CREATE INDEX IF NOT EXISTS idx_zatca_tenant_id ON zatca_invoices (tenant_id);

ALTER TABLE zatca_invoices ENABLE ROW LEVEL SECURITY;
ALTER TABLE zatca_invoices FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_zatca_tenant_isolation ON zatca_invoices;
CREATE POLICY rls_zatca_tenant_isolation ON zatca_invoices
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

=====

FILE: migrations\e10\_03\_zatca\_invoices\_validate.sql

=====

```
-- e10_03_zatca_invoices_validate.sql (run AFTER e10_03_zatca_invoices_up.sql; read-only)
-- PASS = zatca_invoices tenant-isolated (FORCE RLS + policy + tenant_id NOT NULL FK + invoice FK) +
-- ubl_xml/digital_stamp/qr_tlv columns + clearance_status CHECK.
```

SELECT

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='zatca_invoices') A
S z_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='zatca_invoices'
 AND policyname='rls_zatca_tenant_isolation') A
S z_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='zatca_invoices'
 AND column_name='tenant_id' AND is_nullable='NO') A
S z_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='zatca_invoices'::regclass AND confrelid='tenants'::regclass AND contype='f') A
S z_tenant_fk, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='zatca_invoices'::regclass AND confrelid='invoices'::regclass AND contype='f') A
S z_invoice_fk, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='zatca_invoices'
 AND column_name IN ('ubl_xml','digital_stamp','qr_tlv'))
AS z_artifact_cols, -- expect 3
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='zatca_invoices'::regclass AND conname='chk_zatca_clearance') A
S z_clearance_check; -- expect 1
```

=====

FILE: migrations\e10\_04\_invoice\_gl\_link\_down.sql

=====

```
-- e10_04_invoice_gl_link_down.sql (rollback of e10_04_invoice_gl_link_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: invoices ???? ???? ?????? (?????) => ?? ??????? ????? ???? ???????/?????? ???? ??????? ???
-- ??????. idempotent.
BEGIN;
```

```
ALTER TABLE invoices DROP CONSTRAINT IF EXISTS fk_invoice_journal_entry;
ALTER TABLE invoices DROP CONSTRAINT IF EXISTS chk_invoice_posting_status;
DROP INDEX IF EXISTS idx_invoices_journal_entry;
ALTER TABLE invoices DROP COLUMN IF EXISTS journal_entry_id;
ALTER TABLE invoices DROP COLUMN IF EXISTS accounting_posting_status;

COMMIT;
```



=====

FILE: migrations\e10\_04\_invoice\_gl\_link\_up.sql

=====

-- =====

-- e10\_04\_invoice\_gl\_link\_up.sql

-- E10 FINANCE ? link EXISTING invoices to the GL (journal\_entry\_id) + posting status.

-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).

--

-- ?????: ??? ????? (L2 ?????) ??? ????? ??? ??????. ?? ????? ??? invoices ????? (???? ?

-- ????? ???? ? ???? ??:

-- - journal\_entry\_id INTEGER + FK -> finance\_journal\_entries(id) (NULL = ?? ????? ??)

-- - accounting\_posting\_status TEXT DEFAULT 'UNPOSTED' + CHECK (UNPOSTED/POSTED)

-- ??? ????????? ????????? ???? ????? ????? ACCOUNTING\_POSTING\_ENABLED=OFF.

-- ????? ???? ????? => down ????? ????????? ??. ?? ????? ????? ????? bootstrap. idempotent.

-- =====

BEGIN;

ALTER TABLE invoices ADD COLUMN IF NOT EXISTS journal\_entry\_id INTEGER;

ALTER TABLE invoices DROP CONSTRAINT IF EXISTS fk\_invoice\_journal\_entry;

ALTER TABLE invoices ADD CONSTRAINT fk\_invoice\_journal\_entry

FOREIGN KEY (journal\_entry\_id) REFERENCES finance\_journal\_entries(id);

ALTER TABLE invoices ADD COLUMN IF NOT EXISTS accounting\_posting\_status TEXT DEFAULT 'UNPOSTED';

UPDATE invoices SET accounting\_posting\_status = 'UNPOSTED' WHERE accounting\_posting\_status IS NULL;

ALTER TABLE invoices DROP CONSTRAINT IF EXISTS chk\_invoice\_posting\_status;

ALTER TABLE invoices ADD CONSTRAINT chk\_invoice\_posting\_status

CHECK (accounting\_posting\_status IN ('UNPOSTED','POSTED'));

CREATE INDEX IF NOT EXISTS idx\_invoices\_journal\_entry ON invoices (journal\_entry\_id);

COMMIT;

=====

FILE: migrations\e10\_04\_invoice\_gl\_link\_validate.sql

=====

-- e10\_04\_invoice\_gl\_link\_validate.sql (run AFTER e10\_04\_invoice\_gl\_link\_up.sql; read-only)

-- PASS = invoices has journal\_entry\_id FK -> journal\_entries + accounting\_posting\_status CHECK.

SELECT

(SELECT count(\*)::int FROM information\_schema.columns WHERE table\_name='invoices'  
AND column\_name='journal\_entry\_id')

S inv\_je\_col, -- expect 1

(SELECT count(\*)::int FROM pg\_constraint  
WHERE conrelid='invoices'::regclass  
AND confrelid='finance\_journal\_entries'::regclass AND contype='f')

S inv\_je\_fk, -- expect 1

(SELECT count(\*)::int FROM information\_schema.columns WHERE table\_name='invoices'  
AND column\_name='accounting\_posting\_status')

S inv\_status\_col, -- expect 1

A

A

A

```
(SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='invoices'::regclass AND conname='chk_invoice_posting_status')
S inv_status_check; -- expect 1
```

A

```
=====
```

```
FILE: migrations\e10_05_daily_close_rls_down.sql
```

```
=====
```

```
-- e10_05_daily_close_rls_down.sql (rollback of e10_05_daily_close_rls_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: daily_close ???? ?????? => ?? ??????. ????? ??? ?? ?????? ??? ?????? (???????/FK/??????
-- ???? NOT NULL). ????? ?????? tenant_id ??? ?????? ?? ???? ?????? (Phase 178). idempotent.
```

```
BEGIN;
```

```
DROP POLICY IF EXISTS rls_daily_close_tenant_isolation ON daily_close;
ALTER TABLE daily_close DROP CONSTRAINT IF EXISTS fk_daily_close_tenant;
DROP INDEX IF EXISTS idx_daily_close_tenant_id;
ALTER TABLE daily_close ALTER COLUMN tenant_id DROP NOT NULL;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e10_05_daily_close_rls_up.sql
```

```
=====
```

```
-- =====
-- e10_05_daily_close_rls_up.sql
-- E10 FINANCE ? ensure daily_close is tenant-isolated (tenant_id NOT NULL FK + FORCE RLS).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: route ?????? ?????? ???? ?????? tenant ???? ?????? daily_close.tenant_id. ?????? ???? ??????
-- (bootstrap ??? tenant_id ?? ??? ?????? ???????? ???? FORCE RLS ?????? ?? Phase 178). ???? ????
-- tenant_id NOT NULL + FK -> tenants(id) + FORCE RLS ?????? ???????? (idempotent? ??? ?? ??? ????????).
-- ???? ???? ?????? => down ?? ??????.
```

```
-- =====
```

```
BEGIN;
```

```
ALTER TABLE daily_close ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE daily_close SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE daily_close ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE daily_close DROP CONSTRAINT IF EXISTS fk_daily_close_tenant;
ALTER TABLE daily_close ADD CONSTRAINT fk_daily_close_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_daily_close_tenant_id ON daily_close (tenant_id);
```

```
ALTER TABLE daily_close ENABLE ROW LEVEL SECURITY;
ALTER TABLE daily_close FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_daily_close_tenant_isolation ON daily_close;
CREATE POLICY rls_daily_close_tenant_isolation ON daily_close
```

```

FOR ALL
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

```

=====
FILE: migrations\e10_05_daily_close_rls_validate.sql
=====

```

```

-- e10_05_daily_close_rls_validate.sql (run AFTER e10_05_daily_close_rls_up.sql; read-only)
-- PASS = daily_close tenant-isolated (FORCE RLS + policy + tenant_id NOT NULL FK).

```

```

SELECT
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='daily_close')
S dc_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='daily_close'
 AND polycyname='rls_daily_close_tenant_isolation')
S dc_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='daily_close'
 AND column_name='tenant_id' AND is_nullable='NO')
S dc_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='daily_close'::regclass AND confrelid='tenants'::regclass AND contype='f')
S dc_tenant_fk; -- expect 1

```

```

=====
FILE: migrations\e11_01_claims_lifecycle_down.sql
=====

```

```

-- e11_01_claims_lifecycle_down.sql (rollback of e11_01_claims_lifecycle_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: insurance_claims ???? ?????? => ?? ??????. ??? ???? ???????. tenant_id/facility_id/branch_id
-- ?? bootstrap ?????? ?????? (????? NOT NULL/FK + ?????? ?????? ???). idempotent.

```

```

BEGIN;

```

```

DROP POLICY IF EXISTS rls_claims_tenant_isolation ON insurance_claims;
DROP INDEX IF EXISTS idx_ins_claims_tenant_id;
DROP INDEX IF EXISTS idx_ins_claims_tenant_lifecycle;
DROP INDEX IF EXISTS idx_ins_claims_invoice;

```

```

ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS chk_claims_amounts;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS chk_claims_lifecycle;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS fk_claims_company;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS fk_claims_invoice;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS fk_claims_patient;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS fk_claims_tenant;

```

```

ALTER TABLE insurance_claims DROP COLUMN IF EXISTS created_by;

```

```

ALTER TABLE insurance_claims DROP COLUMN IF EXISTS adjudicated_at;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS submitted_at;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS nphies_response_json;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS nphies_request_json;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS nphies_claim_ref;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS patient_share;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS paid_amount;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS approved_amount;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS lifecycle_status;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS pre_auth_id;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS insurance_company_id;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS admission_id;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS invoice_id;
ALTER TABLE insurance_claims DROP COLUMN IF EXISTS patient_id;
ALTER TABLE insurance_claims ALTER COLUMN tenant_id DROP NOT NULL;

```

```

COMMIT;

```

```

=====
FILE: migrations\e11_01_claims_lifecycle_up.sql
=====

```

```

-- =====
-- e11_01_claims_lifecycle_up.sql
-- E11 INSURANCE / NPHIES ? harden the EXISTING insurance_claims into a tenant-isolated,
-- server-authoritative claims-lifecycle table linked to invoices/patients/companies.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: insurance_claims ???? ????? (bootstrap? ???? tenant_id/facility_id/branch_id ??? NOT NULL/FK
-- ??? ??? RLS? status ?? ????? ??? ??? ?????????/??????/??????? ??? ??? ???? ???? ????). ?????:
-- - tenant_id NOT NULL + FK -> tenants(id) + FORCE RLS ?????? ????????.
-- - patient_id / invoice_id / admission_id / insurance_company_id / pre_auth_id ????? ???
-- + FK (ON DELETE SET NULL ?????? ?????????? CASCADE ???????).
-- - lifecycle_status enum-CHECK (draft/submitted/adjudicated/remittance_posted/denied/appealed)
-- ? ??? ?????? ???????? ?? ?????? status ?????? (Pending/Approved/Rejected) ??? ?????? ??????.
-- - ????? NUMERIC(14,2): claim_amount / approved_amount / paid_amount / patient_share.
-- - nphies_request_json / nphies_response_json / nphies_claim_ref + ????? submitted/adjudicated.
-- ????? ?????? => down ?? ?????? ??? ???????? ???.
-- ?? ?????? ?????? ?????? ?? bootstrap ?? db_postgres.js ??? ??? ?????? (candidate-only).
--
-- idempotent: ADD COLUMN IF NOT EXISTS + backfill + SET NOT NULL + DROP/ADD CONSTRAINT IF EXISTS
-- + ALTER TYPE USING + CREATE INDEX IF NOT EXISTS + DROP/CREATE POLICY. wrapped BEGIN; ? COMMIT;
-- =====
BEGIN;

-- ----- tenant isolation -----
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE insurance_claims SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE insurance_claims ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS fk_claims_tenant;
ALTER TABLE insurance_claims ADD CONSTRAINT fk_claims_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;

```

```

-- ----- entity links -----
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS patient_id INTEGER;
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS invoice_id INTEGER;
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS admission_id INTEGER;
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS insurance_company_id INTEGER;
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS pre_auth_id INTEGER;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS fk_claims_patient;
ALTER TABLE insurance_claims ADD CONSTRAINT fk_claims_patient
 FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE SET NULL;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS fk_claims_invoice;
ALTER TABLE insurance_claims ADD CONSTRAINT fk_claims_invoice
 FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE SET NULL;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS fk_claims_company;
ALTER TABLE insurance_claims ADD CONSTRAINT fk_claims_company
 FOREIGN KEY (insurance_company_id) REFERENCES insurance_companies(id) ON DELETE SET NULL;

-- ----- lifecycle state machine (server-authoritative) -----
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS lifecycle_status TEXT DEFAULT 'draft';
-- map legacy free-text status into the new lifecycle where recognisable
UPDATE insurance_claims SET lifecycle_status = CASE
 WHEN status ILIKE 'approved%' THEN 'adjudicated'
 WHEN status ILIKE 'rejected%' THEN 'denied'
 ELSE 'draft' END
WHERE lifecycle_status IS NULL;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS chk_claims_lifecycle;
ALTER TABLE insurance_claims ADD CONSTRAINT chk_claims_lifecycle
 CHECK (lifecycle_status IN ('draft', 'submitted', 'adjudicated', 'remittance_posted', 'denied', 'appealed'));

-- ----- monetary precision (REAL -> NUMERIC) + adjudication amounts -----
ALTER TABLE insurance_claims ALTER COLUMN claim_amount TYPE NUMERIC(14,2) USING ROUND(claim_amount::numeric, 2
);
ALTER TABLE insurance_claims ALTER COLUMN claim_amount SET DEFAULT 0;
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS approved_amount NUMERIC(14,2) DEFAULT 0;
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS paid_amount NUMERIC(14,2) DEFAULT 0;
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS patient_share NUMERIC(14,2) DEFAULT 0;
ALTER TABLE insurance_claims DROP CONSTRAINT IF EXISTS chk_claims_amounts;
ALTER TABLE insurance_claims ADD CONSTRAINT chk_claims_amounts
 CHECK (claim_amount >= 0 AND approved_amount >= 0 AND paid_amount >= 0 AND patient_share >= 0);

-- ----- NPHIES envelope + lifecycle timestamps + actor -----
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS nphies_claim_ref TEXT DEFAULT '';
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS nphies_request_json TEXT DEFAULT '';
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS nphies_response_json TEXT DEFAULT '';
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS submitted_at TIMESTAMP;
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS adjudicated_at TIMESTAMP;
ALTER TABLE insurance_claims ADD COLUMN IF NOT EXISTS created_by INTEGER;

CREATE INDEX IF NOT EXISTS idx_ins_claims_tenant_id ON insurance_claims (tenant_id);
CREATE INDEX IF NOT EXISTS idx_ins_claims_tenant_lifecycle ON insurance_claims (tenant_id, lifecycle_status);
CREATE INDEX IF NOT EXISTS idx_ins_claims_invoice ON insurance_claims (tenant_id, invoice_id);

ALTER TABLE insurance_claims ENABLE ROW LEVEL SECURITY;
ALTER TABLE insurance_claims FORCE ROW LEVEL SECURITY;

```

```

DROP POLICY IF EXISTS rls_claims_tenant_isolation ON insurance_claims;
CREATE POLICY rls_claims_tenant_isolation ON insurance_claims
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

```

=====
FILE: migrations\e11_01_claims_lifecycle_validate.sql
=====

```

```

-- e11_01_claims_lifecycle_validate.sql (run AFTER e11_01_claims_lifecycle_up.sql; read-only)
-- PASS = insurance_claims tenant-isolated (FORCE RLS + policy + tenant_id NOT NULL FK),
-- linked to invoices/patients/companies, lifecycle_status CHECK, amounts NUMERIC + non-negative CHECK.

```

```

SELECT
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='insurance_claims')
S c_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='insurance_claims'
 AND policyname='rls_claims_tenant_isolation')
S c_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='insurance_claims'
 AND column_name='tenant_id' AND is_nullable='NO')
S c_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_claims'::regclass AND confrelid='tenants'::regclass AND contype='f')
S c_tenant_fk, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_claims'::regclass AND confrelid='invoices'::regclass AND contype='f')
S c_invoice_fk, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_claims'::regclass AND confrelid='patients'::regclass AND contype='f')
S c_patient_fk, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_claims'::regclass AND confrelid='insurance_companies'::regclass AND contype='f') AS c_company_fk, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_claims'::regclass AND conname='chk_claims_lifecycle')
S c_lifecycle_check, -- expect 1
 (SELECT data_type FROM information_schema.columns WHERE table_name='insurance_claims'
 AND column_name='claim_amount')
S c_amount_type, -- expect numeric
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_claims'::regclass AND conname='chk_claims_amounts')
S c_amounts_check; -- expect 1

```

```

=====
FILE: migrations\e11_02_companies_policies_down.sql
=====

```

```
-- e11_02_companies_policies_down.sql (rollback of e11_02_companies_policies_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: ?????? ?????? ?????? ?????? => ?? ??????. ??? ?????? ???. idempotent.
BEGIN;
```

```
DROP POLICY IF EXISTS rls_inscompany_tenant_isolation ON insurance_companies;
DROP INDEX IF EXISTS idx_inscompany_tenant_id;
ALTER TABLE insurance_companies DROP CONSTRAINT IF EXISTS fk_inscompany_tenant;
ALTER TABLE insurance_companies ALTER COLUMN tenant_id DROP NOT NULL;
```

```
DROP POLICY IF EXISTS rls_inspolicy_tenant_isolation ON insurance_policies;
DROP INDEX IF EXISTS idx_inspolicy_tenant_id;
ALTER TABLE insurance_policies DROP CONSTRAINT IF EXISTS fk_inspolicy_tenant;
ALTER TABLE insurance_policies ALTER COLUMN tenant_id DROP NOT NULL;
```

```
DROP POLICY IF EXISTS rls_inscontract_tenant_isolation ON insurance_contracts;
DROP INDEX IF EXISTS idx_inscontract_tenant_id;
ALTER TABLE insurance_contracts DROP CONSTRAINT IF EXISTS fk_inscontract_tenant;
ALTER TABLE insurance_contracts ALTER COLUMN tenant_id DROP NOT NULL;
```

```
COMMIT;
```

```
=====
FILE: migrations\e11_02_companies_policies_up.sql
=====
```

```
-- =====
-- e11_02_companies_policies_up.sql
-- E11 INSURANCE / NPHIES ? tenant-isolate the EXISTING insurance_companies + insurance_policies
-- + insurance_contracts (payer master data) so a payer/policy cannot leak across tenants.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ?????? ?????? ?????? ?????? (bootstrap) ??? tenant_id ??? ???. ??? tenant_id NOT NULL + FK
-- -> tenants(id) + FORCE RLS ?????? ?????? + ??? (tenant_id). ????? ?????? ?????? => down ???
-- ?????? ??? ??? ??????. idempotent.
-- =====
BEGIN;
```

```
-- ----- insurance_companies -----
ALTER TABLE insurance_companies ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE insurance_companies SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE insurance_companies ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE insurance_companies DROP CONSTRAINT IF EXISTS fk_inscompany_tenant;
ALTER TABLE insurance_companies ADD CONSTRAINT fk_inscompany_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_inscompany_tenant_id ON insurance_companies (tenant_id);
ALTER TABLE insurance_companies ENABLE ROW LEVEL SECURITY;
ALTER TABLE insurance_companies FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_inscompany_tenant_isolation ON insurance_companies;
CREATE POLICY rls_inscompany_tenant_isolation ON insurance_companies
```

```

FOR ALL
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ----- insurance_policies -----
ALTER TABLE insurance_policies ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE insurance_policies SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE insurance_policies ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE insurance_policies DROP CONSTRAINT IF EXISTS fk_inspolicy_tenant;
ALTER TABLE insurance_policies ADD CONSTRAINT fk_inspolicy_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_inspolicy_tenant_id ON insurance_policies (tenant_id);
ALTER TABLE insurance_policies ENABLE ROW LEVEL SECURITY;
ALTER TABLE insurance_policies FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_inspolicy_tenant_isolation ON insurance_policies;
CREATE POLICY rls_inspolicy_tenant_isolation ON insurance_policies
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ----- insurance_contracts -----
ALTER TABLE insurance_contracts ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE insurance_contracts SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE insurance_contracts ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE insurance_contracts DROP CONSTRAINT IF EXISTS fk_inscontract_tenant;
ALTER TABLE insurance_contracts ADD CONSTRAINT fk_inscontract_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_inscontract_tenant_id ON insurance_contracts (tenant_id);
ALTER TABLE insurance_contracts ENABLE ROW LEVEL SECURITY;
ALTER TABLE insurance_contracts FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_inscontract_tenant_isolation ON insurance_contracts;
CREATE POLICY rls_inscontract_tenant_isolation ON insurance_contracts
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: migrations\e11_02_companies_policies_validate.sql
=====

-- e11_02_companies_policies_validate.sql (run AFTER e11_02_companies_policies_up.sql; read-only)
-- PASS = companies/policies/contracts tenant-isolated (FORCE RLS + policy + tenant_id NOT NULL FK).

SELECT
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='insurance_companies')
S co_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='insurance_companies'
 AND policyname='rls_inscompany_tenant_isolation')
S co_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='insurance_companies'

```

A

A



```

 AND column_name='tenant_id' AND is_nullable='NO')
A
S co_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_companies'::regclass AND confrelid='tenants'::regclass AND contype='f')
A
S co_tenant_fk, -- expect 1
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='insurance_policies')
A
S po_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='insurance_policies'
 AND policyname='rls_inspolicy_tenant_isolation')
A
S po_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='insurance_policies'
 AND column_name='tenant_id' AND is_nullable='NO')
A
S po_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_policies'::regclass AND confrelid='tenants'::regclass AND contype='f')
A
S po_tenant_fk, -- expect 1
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='insurance_contracts')
A
S ct_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='insurance_contracts'
 AND policyname='rls_inscontract_tenant_isolation')
A
S ct_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='insurance_contracts'
 AND column_name='tenant_id' AND is_nullable='NO')
A
S ct_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_contracts'::regclass AND confrelid='tenants'::regclass AND contype='f')
A
S ct_tenant_fk; -- expect 1

```

=====

FILE: migrations\e11\_03\_nphies\_lifecycle\_tables\_down.sql

=====

```

-- e11_03_nphies_lifecycle_tables_down.sql (rollback of e11_03_nphies_lifecycle_tables_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ????? ????? ??????? => ??????? (????????? ???? ????? ?? DROP TABLE). ????? ???? ????????????
-- (lines/denials ????? claims ????? ??CASCADE? ????? ??????? ??????? ???? ??? ????? ?? ????? ?????). idempotent
.
BEGIN;

```

```

DROP TABLE IF EXISTS insurance_payer_pricing;
DROP TABLE IF EXISTS insurance_claim_denials;
DROP TABLE IF EXISTS insurance_claim_lines;
DROP TABLE IF EXISTS insurance_pre_authorizations;
DROP TABLE IF EXISTS insurance_eligibility_checks;

COMMIT;

```

=====

FILE: migrations\e11\_03\_nphies\_lifecycle\_tables\_up.sql

=====

-- =====

-- e11\_03\_nphies\_lifecycle\_tables\_up.sql

-- E11 INSURANCE / NPHIES ? NEW tenant-isolated lifecycle tables: eligibility checks,

-- pre-authorizations, claim lines, claim denials, payer pricing tiers.

-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).

--

-- ?????: ????? ????? ????? ???? ??????? ???? tenant\_id NOT NULL + FK -> tenants(id) + FK ????? ????

-- + FORCE RLS ?????? ??????? + ???? ???? ???? CHECK. ??? ????? ????? => down ?????? ?????.

-- ????? ??????? ???? bootstrap ?? db\_postgres.js (candidate-only? ???? ???????). idempotent.

-- =====

BEGIN;

-- ===== insurance\_eligibility\_checks (NPHIES eligibility intent/response) =====

CREATE TABLE IF NOT EXISTS insurance\_eligibility\_checks (

id SERIAL PRIMARY KEY,

tenant\_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,

patient\_id INTEGER REFERENCES patients(id) ON DELETE SET NULL,

insurance\_company\_id INTEGER REFERENCES insurance\_companies(id) ON DELETE SET NULL,

policy\_number TEXT DEFAULT '',

-- status: pending=intent recorded, NPHIES gated; eligible/ineligible/partial after response

status TEXT NOT NULL DEFAULT 'pending',

coverage\_amount NUMERIC(14,2) DEFAULT 0,

nphies\_request\_json TEXT DEFAULT '',

nphies\_response\_json TEXT DEFAULT '',

checked\_by INTEGER,

created\_at TIMESTAMP DEFAULT now()

);

ALTER TABLE insurance\_eligibility\_checks DROP CONSTRAINT IF EXISTS chk\_elig\_status;

ALTER TABLE insurance\_eligibility\_checks ADD CONSTRAINT chk\_elig\_status

CHECK (status IN ('pending','eligible','ineligible','partial'));

CREATE INDEX IF NOT EXISTS idx\_elig\_tenant\_id ON insurance\_eligibility\_checks (tenant\_id);

CREATE INDEX IF NOT EXISTS idx\_elig\_tenant\_patient ON insurance\_eligibility\_checks (tenant\_id, patient\_id);

ALTER TABLE insurance\_eligibility\_checks ENABLE ROW LEVEL SECURITY;

ALTER TABLE insurance\_eligibility\_checks FORCE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS rls\_elig\_tenant\_isolation ON insurance\_eligibility\_checks;

CREATE POLICY rls\_elig\_tenant\_isolation ON insurance\_eligibility\_checks

FOR ALL

USING (tenant\_id = NULLIF(current\_setting('app.tenant\_id', true), '')::integer)

WITH CHECK (tenant\_id = NULLIF(current\_setting('app.tenant\_id', true), '')::integer);

-- ===== insurance\_pre\_authorizations (request -> approved/denied/partial) =====

CREATE TABLE IF NOT EXISTS insurance\_pre\_authorizations (

id SERIAL PRIMARY KEY,

tenant\_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,

patient\_id INTEGER REFERENCES patients(id) ON DELETE SET NULL,

admission\_id INTEGER REFERENCES admissions(id) ON DELETE SET NULL,

insurance\_company\_id INTEGER REFERENCES insurance\_companies(id) ON DELETE SET NULL,

requested\_amount NUMERIC(14,2) DEFAULT 0,

approved\_amount NUMERIC(14,2) DEFAULT 0,

auth\_status TEXT NOT NULL DEFAULT 'requested',

auth\_number TEXT DEFAULT '',

clinical\_justification TEXT DEFAULT '',

```

nphies_request_json TEXT DEFAULT '',
nphies_response_json TEXT DEFAULT '',
requested_by INTEGER,
decided_at TIMESTAMP,
created_at TIMESTAMP DEFAULT now()
);
ALTER TABLE insurance_pre_authorizations DROP CONSTRAINT IF EXISTS chk_preauth_status;
ALTER TABLE insurance_pre_authorizations ADD CONSTRAINT chk_preauth_status
 CHECK (auth_status IN ('requested','approved','denied','partial'));
ALTER TABLE insurance_pre_authorizations DROP CONSTRAINT IF EXISTS chk_preauth_amounts;
ALTER TABLE insurance_pre_authorizations ADD CONSTRAINT chk_preauth_amounts
 CHECK (requested_amount >= 0 AND approved_amount >= 0);
CREATE INDEX IF NOT EXISTS idx_preauth_tenant_id ON insurance_pre_authorizations (tenant_id);
CREATE INDEX IF NOT EXISTS idx_preauth_tenant_status ON insurance_pre_authorizations (tenant_id, auth_status);
ALTER TABLE insurance_pre_authorizations ENABLE ROW LEVEL SECURITY;
ALTER TABLE insurance_pre_authorizations FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_preauth_tenant_isolation ON insurance_pre_authorizations;
CREATE POLICY rls_preauth_tenant_isolation ON insurance_pre_authorizations
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== insurance_claim_lines (claim -> service line; links chargemaster medical_services) =====
CREATE TABLE IF NOT EXISTS insurance_claim_lines (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 claim_id INTEGER NOT NULL REFERENCES insurance_claims(id) ON DELETE CASCADE,
 service_id INTEGER REFERENCES medical_services(id) ON DELETE SET NULL,
 description TEXT DEFAULT '',
 quantity NUMERIC(10,2) DEFAULT 1,
 unit_price NUMERIC(14,2) DEFAULT 0,
 line_amount NUMERIC(14,2) DEFAULT 0,
 approved_amount NUMERIC(14,2) DEFAULT 0,
 created_at TIMESTAMP DEFAULT now()
);
ALTER TABLE insurance_claim_lines DROP CONSTRAINT IF EXISTS chk_claimline_amounts;
ALTER TABLE insurance_claim_lines ADD CONSTRAINT chk_claimline_amounts
 CHECK (quantity >= 0 AND unit_price >= 0 AND line_amount >= 0 AND approved_amount >= 0);
CREATE INDEX IF NOT EXISTS idx_claimline_tenant_id ON insurance_claim_lines (tenant_id);
CREATE INDEX IF NOT EXISTS idx_claimline_tenant_claim ON insurance_claim_lines (tenant_id, claim_id);
ALTER TABLE insurance_claim_lines ENABLE ROW LEVEL SECURITY;
ALTER TABLE insurance_claim_lines FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_claimline_tenant_isolation ON insurance_claim_lines;
CREATE POLICY rls_claimline_tenant_isolation ON insurance_claim_lines
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== insurance_claim_denials (denial reason + appeal lifecycle) =====
CREATE TABLE IF NOT EXISTS insurance_claim_denials (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 claim_id INTEGER NOT NULL REFERENCES insurance_claims(id) ON DELETE CASCADE,
 denial_reason TEXT DEFAULT '',

```

```

 denial_code TEXT DEFAULT '',
 appeal_status TEXT NOT NULL DEFAULT 'open',
 appeal_notes TEXT DEFAULT '',
 created_by INTEGER,
 created_at TIMESTAMP DEFAULT now()
);
ALTER TABLE insurance_claim_denials DROP CONSTRAINT IF EXISTS chk_denial_appeal_status;
ALTER TABLE insurance_claim_denials ADD CONSTRAINT chk_denial_appeal_status
 CHECK (appeal_status IN ('open','appealed','upheld','overturned','closed'));
CREATE INDEX IF NOT EXISTS idx_denial_tenant_id ON insurance_claim_denials (tenant_id);
CREATE INDEX IF NOT EXISTS idx_denial_tenant_claim ON insurance_claim_denials (tenant_id, claim_id);
ALTER TABLE insurance_claim_denials ENABLE ROW LEVEL SECURITY;
ALTER TABLE insurance_claim_denials FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_denial_tenant_isolation ON insurance_claim_denials;
CREATE POLICY rls_denial_tenant_isolation ON insurance_claim_denials
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

```

```

-- ===== insurance_payer_pricing (per-payer chargemaster price tiers) =====
CREATE TABLE IF NOT EXISTS insurance_payer_pricing (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 insurance_company_id INTEGER NOT NULL REFERENCES insurance_companies(id) ON DELETE CASCADE,
 service_id INTEGER NOT NULL REFERENCES medical_services(id) ON DELETE CASCADE,
 payer_price NUMERIC(14,2) NOT NULL DEFAULT 0,
 effective_from DATE DEFAULT CURRENT_DATE,
 is_active INTEGER DEFAULT 1,
 created_by INTEGER,
 created_at TIMESTAMP DEFAULT now()
);
ALTER TABLE insurance_payer_pricing DROP CONSTRAINT IF EXISTS chk_payerprice_nonneg;
ALTER TABLE insurance_payer_pricing ADD CONSTRAINT chk_payerprice_nonneg
 CHECK (payer_price >= 0);
CREATE INDEX IF NOT EXISTS idx_payerprice_tenant_id ON insurance_payer_pricing (tenant_id);
CREATE UNIQUE INDEX IF NOT EXISTS uq_payerprice_tenant_company_service
 ON insurance_payer_pricing (tenant_id, insurance_company_id, service_id);
ALTER TABLE insurance_payer_pricing ENABLE ROW LEVEL SECURITY;
ALTER TABLE insurance_payer_pricing FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_payerprice_tenant_isolation ON insurance_payer_pricing;
CREATE POLICY rls_payerprice_tenant_isolation ON insurance_payer_pricing
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

```

```

COMMIT;

```

```

=====
FILE: migrations\e11_03_nphies_lifecycle_tables_validate.sql
=====

```

```

-- e11_03_nphies_lifecycle_tables_validate.sql (run AFTER e11_03_nphies_lifecycle_tables_up.sql; read-only)

```

-- PASS = all 5 new tables exist, FORCE RLS + policy + tenant\_id NOT NULL FK -> tenants + parent FK + status C  
HECK.

SELECT

```
-- eligibility_checks
(SELECT relforcerowsecurity FROM pg_class WHERE relname='insurance_eligibility_checks') A
S el_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='insurance_eligibility_checks') A
S el_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='insurance_eligibility_checks'
 AND column_name='tenant_id' AND is_nullable='NO') A
S el_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_eligibility_checks'::regclass AND confrelid='tenants'::regclass AND contype='
f') AS el_tenant_fk, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_eligibility_checks'::regclass AND conname='chk_elig_status') A
S el_status_check, -- expect 1
-- pre_authorizations
(SELECT relforcerowsecurity FROM pg_class WHERE relname='insurance_pre_authorizations') A
S pa_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='insurance_pre_authorizations') A
S pa_policy, -- expect 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='insurance_pre_authorizations'
 AND column_name='tenant_id' AND is_nullable='NO') A
S pa_tenant_not_null, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_pre_authorizations'::regclass AND conname='chk_preauth_status') A
S pa_status_check, -- expect 1
-- claim_lines
(SELECT relforcerowsecurity FROM pg_class WHERE relname='insurance_claim_lines') A
S cl_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='insurance_claim_lines') A
S cl_policy, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_claim_lines'::regclass AND confrelid='insurance_claims'::regclass AND contype=
='f') AS cl_claim_fk, -- expect 1
-- claim_denials
(SELECT relforcerowsecurity FROM pg_class WHERE relname='insurance_claim_denials') A
S dn_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='insurance_claim_denials') A
S dn_policy, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_claim_denials'::regclass AND conname='chk_denial_appeal_status') A
S dn_appeal_check, -- expect 1
-- payer_pricing
(SELECT relforcerowsecurity FROM pg_class WHERE relname='insurance_payer_pricing') A
S pp_force_rls, -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='insurance_payer_pricing') A
S pp_policy, -- expect 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conrelid='insurance_payer_pricing'::regclass AND confrelid='medical_services'::regclass AND conty
pe='f') AS pp_service_fk; -- expect 1
```

=====

FILE: migrations\e12\_001\_surgery\_or\_down.sql

=====

```
-- =====
-- Epic E12 ? Surgery / OR migration DOWN (reverse only E12 additions).
-- Drops ONLY the 5 NEW tables introduced by this epic. Does NOT touch any
-- pre-existing table (surgeries, operating_rooms, inventory_items, etc.).
-- =====
```

```
BEGIN;
DROP TABLE IF EXISTS or_consumption CASCADE;
DROP TABLE IF EXISTS operative_notes CASCADE;
DROP TABLE IF EXISTS pacu_records CASCADE;
DROP TABLE IF EXISTS who_surgical_checklist CASCADE;
DROP TABLE IF EXISTS or_slots CASCADE;
COMMIT;
```

=====

FILE: migrations\e12\_001\_surgery\_or\_up.sql

=====

```
-- =====
-- Epic E12 ? Surgery / Operating Room
-- Candidate migration (UP). Idempotent. DO NOT auto-execute in production.
-- Adds 5 new tables: or_slots, who_surgical_checklist, pacu_records,
-- operative_notes, or_consumption. All FORCE RLS + canonical tenant policy.
-- =====
```

```
BEGIN;
```

```
-- ---- OR scheduling slots (conflict detection backing table) ----
```

```
CREATE TABLE IF NOT EXISTS or_slots (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 surgery_id INTEGER NOT NULL REFERENCES surgeries(id) ON DELETE CASCADE,
 room_id INTEGER NOT NULL REFERENCES operating_rooms(id) ON DELETE CASCADE,
 surgeon_id INTEGER NOT NULL REFERENCES system_users(id),
 slot_date DATE NOT NULL,
 slot_start_time TIME NOT NULL,
 slot_end_time TIME NOT NULL,
 duration_minutes INTEGER NOT NULL DEFAULT 60,
 status TEXT NOT NULL DEFAULT 'Booked',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
ALTER TABLE or_slots ENABLE ROW LEVEL SECURITY;
ALTER TABLE or_slots FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_or_slots_tenant_isolation ON or_slots;
CREATE POLICY rls_or_slots_tenant_isolation ON or_slots
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```

CREATE INDEX IF NOT EXISTS idx_or_slots_tenant_facility ON or_slots (tenant_id, facility_id);
CREATE INDEX IF NOT EXISTS idx_or_slots_room_date ON or_slots (room_id, slot_date, status);
CREATE INDEX IF NOT EXISTS idx_or_slots_surgeon_date ON or_slots (surgeon_id, slot_date, status);

-- ---- WHO Safe Surgery Checklist ----
CREATE TABLE IF NOT EXISTS who_surgical_checklist (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 surgery_id INTEGER NOT NULL REFERENCES surgeries(id) ON DELETE CASCADE,
 patient_id INTEGER,
 sign_in_completed INTEGER DEFAULT 0,
 sign_in_completed_by TEXT DEFAULT '',
 sign_in_at TIMESTAMP,
 time_out_completed INTEGER DEFAULT 0,
 time_out_completed_by TEXT DEFAULT '',
 time_out_at TIMESTAMP,
 sign_out_completed INTEGER DEFAULT 0,
 sign_out_completed_by TEXT DEFAULT '',
 sign_out_at TIMESTAMP,
 state TEXT NOT NULL DEFAULT 'Not Started',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 UNIQUE (surgery_id)
);

ALTER TABLE who_surgical_checklist ENABLE ROW LEVEL SECURITY;
ALTER TABLE who_surgical_checklist FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_who_surgical_checklist_tenant_isolation ON who_surgical_checklist;
CREATE POLICY rls_who_surgical_checklist_tenant_isolation ON who_surgical_checklist
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_who_checklist_tenant_surgery ON who_surgical_checklist (tenant_id, surgery_id);

-- ---- PACU (recovery) records ----
CREATE TABLE IF NOT EXISTS pacu_records (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 surgery_id INTEGER NOT NULL REFERENCES surgeries(id) ON DELETE CASCADE,
 patient_id INTEGER,
 start_time TEXT DEFAULT '', -- L1: freeform text; pending migration to TIMESTAMP (kept TEXT to avoid breaking existing inserts)
 end_time TEXT DEFAULT '', -- L1: freeform text; pending migration to TIMESTAMP (kept TEXT to avoid breaking existing inserts)
 pain_score INTEGER DEFAULT 0,
 bp TEXT DEFAULT '',
 hr TEXT DEFAULT '',
 spo2 TEXT DEFAULT '',
 temp TEXT DEFAULT '',
 aldrete_score INTEGER,
 discharge_status TEXT DEFAULT 'In Recovery',
 recovery_nurse TEXT DEFAULT '',
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 UNIQUE (surgery_id)

```

```

);
ALTER TABLE pacu_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE pacu_records FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_pacu_records_tenant_isolation ON pacu_records;
CREATE POLICY rls_pacu_records_tenant_isolation ON pacu_records
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_pacu_records_tenant_surgery ON pacu_records (tenant_id, surgery_id);

-- ---- Operative notes (surgery report) ----
CREATE TABLE IF NOT EXISTS operative_notes (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 surgery_id INTEGER NOT NULL REFERENCES surgeries(id) ON DELETE CASCADE,
 patient_id INTEGER,
 procedure_description TEXT DEFAULT '',
 findings TEXT DEFAULT '',
 complications TEXT DEFAULT '',
 blood_loss_final INTEGER DEFAULT 0,
 counts_verified TEXT DEFAULT 'Incomplete',
 specimen TEXT DEFAULT '',
 surgeon_signature TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 UNIQUE (surgery_id)
);
ALTER TABLE operative_notes ENABLE ROW LEVEL SECURITY;
ALTER TABLE operative_notes FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_operative_notes_tenant_isolation ON operative_notes;
CREATE POLICY rls_operative_notes_tenant_isolation ON operative_notes
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_operative_notes_tenant_surgery ON operative_notes (tenant_id, surgery_id);

-- ---- OR consumption (links to inventory_items) ----
CREATE TABLE IF NOT EXISTS or_consumption (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 surgery_id INTEGER NOT NULL REFERENCES surgeries(id) ON DELETE CASCADE,
 item_id INTEGER NOT NULL,
 qty_used INTEGER NOT NULL DEFAULT 0,
 batch_id INTEGER,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
ALTER TABLE or_consumption ENABLE ROW LEVEL SECURITY;
ALTER TABLE or_consumption FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_or_consumption_tenant_isolation ON or_consumption;
CREATE POLICY rls_or_consumption_tenant_isolation ON or_consumption
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_or_consumption_tenant_surgery ON or_consumption (tenant_id, surgery_id);
CREATE INDEX IF NOT EXISTS idx_or_consumption_item ON or_consumption (item_id);

```



COMMIT;

=====

FILE: migrations\e12\_001\_surgery\_or\_validate.sql

=====

-- =====

-- Epic E12 ? Surgery / OR migration VALIDATE.

-- Asserts: FORCE RLS enabled, canonical tenant policy present, tenant\_id NOT NULL,

-- and FK to tenants(id) exists for each new table. Raises an exception on failure.

-- =====

DO \$\$

DECLARE

tbls TEXT[] := ARRAY['or\_slots', 'who\_surgical\_checklist', 'pacu\_records', 'operative\_notes', 'or\_consumption'];

t TEXT;

BEGIN

FOREACH t IN ARRAY tbls LOOP

-- table exists

IF to\_regclass('public.' || t) IS NULL THEN

RAISE EXCEPTION 'E12 validate: table % missing', t;

END IF;

-- FORCE RLS enabled (relforcerowsecurity)

IF NOT EXISTS (SELECT 1 FROM pg\_class WHERE relname = t AND relrowsecurity AND relforcerowsecurity) TH

EN

RAISE EXCEPTION 'E12 validate: FORCE RLS not enabled on %', t;

END IF;

-- canonical tenant isolation policy present

IF NOT EXISTS (SELECT 1 FROM pg\_policies WHERE tablename = t AND policyname = 'rls\_' || t || '\_tenant\_isolation') THEN

RAISE EXCEPTION 'E12 validate: tenant isolation policy missing on %', t;

END IF;

-- tenant\_id is NOT NULL

IF EXISTS (SELECT 1 FROM information\_schema.columns WHERE table\_name = t AND column\_name = 'tenant\_id' AND is\_nullable = 'YES') THEN

RAISE EXCEPTION 'E12 validate: tenant\_id is nullable on %', t;

END IF;

-- FK from tenant\_id -> tenants(id)

IF NOT EXISTS (

SELECT 1 FROM pg\_constraint c

JOIN pg\_class rel ON rel.oid = c.conrelid

WHERE rel.relname = t AND c.contype = 'f'

AND c.confrelid = 'tenants'::regclass

) THEN

RAISE EXCEPTION 'E12 validate: FK to tenants(id) missing on %', t;

END IF;

END LOOP;

-- entity FK: each table must reference surgeries(id)

FOREACH t IN ARRAY tbls LOOP

IF NOT EXISTS (

SELECT 1 FROM pg\_constraint c

JOIN pg\_class rel ON rel.oid = c.conrelid

```

 WHERE rel.relname = t AND c.contype = 'f'
 AND c.confrelid = 'surgeries'::regclass
) THEN
 RAISE EXCEPTION 'E12 validate: FK to surgeries(id) missing on %', t;
 END IF;
END LOOP;
RAISE NOTICE 'E12 validate: OK ? all 5 tables have FORCE RLS + canonical policy + tenant_id NOT NULL + FK(
tenants) + FK(surgeries)';
END $$;

```

```

=====
FILE: migrations\e14_ob_maternity_down.sql
=====

```

```

-- =====
-- Epic E14 ? OB / Maternity migration (DOWN) [CANDIDATE ? DO NOT EXECUTE here]
-- Reverses ONLY the additions made by *_up.sql. Drops the NEW obgyn_* tables in
-- child->parent order. NEVER drops any pre-existing table (patients/tenants/etc.).
-- Idempotent (IF EXISTS). Policies/indexes are removed implicitly with the tables.
-- =====

```

```

BEGIN;

```

```

-- children first (FK dependents), then parents
DROP TABLE IF EXISTS obgyn_neonatal CASCADE;
DROP TABLE IF EXISTS obgyn_nst CASCADE;
DROP TABLE IF EXISTS obgyn_partogram CASCADE;
DROP TABLE IF EXISTS obgyn_ultrasounds CASCADE;
DROP TABLE IF EXISTS obgyn_antenatal_visits CASCADE;
DROP TABLE IF EXISTS obgyn_deliveries CASCADE;
DROP TABLE IF EXISTS obgyn_lab_panels CASCADE;
DROP TABLE IF EXISTS obgyn_pregnancies CASCADE;

```

```

COMMIT;

```

```

=====
FILE: migrations\e14_ob_maternity_up.sql
=====

```

```

-- =====
-- Epic E14 ? OB / Maternity migration (UP) [CANDIDATE ? DO NOT EXECUTE here]
-- Idempotent. Creates NEW obgyn_* tables only; FORCE RLS + canonical tenant policy.
-- tenant_id NOT NULL REFERENCES tenants(id), entity FK to patients / parent rows.
-- Authority fields (edd, living_children, apgar_*, gestational_age, efw_percentile,
-- risk/alert flags) are written by the server-side engine ? never trusted from client.
-- Run order: this file -> *_validate.sql (assert) -> (rollback via *_down.sql).
-- =====

```

```

BEGIN;

```

```

-- 1) pregnancies -----
CREATE TABLE IF NOT EXISTS obgyn_pregnancies (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 patient_id INTEGER NOT NULL REFERENCES patients(id),
 patient_name TEXT DEFAULT '',
 lmp DATE,
 edd DATE, -- server-computed (Naegele); may be NULL if LMP absent
 gravida INTEGER DEFAULT 1,
 para INTEGER DEFAULT 0,
 abortions INTEGER DEFAULT 0,
 living_children INTEGER DEFAULT 0, -- server-derived
 blood_group TEXT DEFAULT '',
 rh_factor TEXT DEFAULT '',
 risk_level TEXT DEFAULT 'Low',
 pre_pregnancy_weight REAL DEFAULT 0,
 height REAL DEFAULT 0,
 allergies TEXT DEFAULT '',
 chronic_conditions TEXT DEFAULT '',
 previous_cs INTEGER DEFAULT 0,
 previous_complications TEXT DEFAULT '',
 husband_name TEXT DEFAULT '',
 husband_blood_group TEXT DEFAULT '',
 attending_doctor TEXT DEFAULT '',
 status TEXT DEFAULT 'Active', -- Active | Delivered | Miscarriage | Ectopic | Terminated
 delivery_date TIMESTAMP,
 delivery_type TEXT DEFAULT '',
 outcome TEXT DEFAULT '',
 created_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

-- 2) antenatal_visits -----
CREATE TABLE IF NOT EXISTS obgyn_antenatal_visits (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 pregnancy_id INTEGER NOT NULL REFERENCES obgyn_pregnancies(id),
 patient_id INTEGER NOT NULL REFERENCES patients(id),
 visit_number INTEGER DEFAULT 1,
 gestational_age TEXT DEFAULT '', -- server-derived from LMP
 weight REAL DEFAULT 0,
 weight_gain REAL DEFAULT 0,
 blood_pressure TEXT DEFAULT '',
 systolic INTEGER DEFAULT 0,
 diastolic INTEGER DEFAULT 0,
 fundal_height REAL DEFAULT 0,
 fetal_heart_rate INTEGER DEFAULT 0,
 fetal_presentation TEXT DEFAULT '',
 fetal_movement TEXT DEFAULT 'Active',
 edema TEXT DEFAULT 'None',
 proteinuria TEXT DEFAULT 'Negative',
 glucose_urine TEXT DEFAULT 'Negative',
 hemoglobin REAL DEFAULT 0,

```

```

complaints TEXT DEFAULT '',
examination_notes TEXT DEFAULT '',
plan TEXT DEFAULT '',
next_visit DATE,
doctor TEXT DEFAULT '',
risk_flags TEXT DEFAULT '', -- server-classified
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3) partogram -----
CREATE TABLE IF NOT EXISTS obgyn_partogram (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 pregnancy_id INTEGER NOT NULL REFERENCES obgyn_pregnancies(id),
 patient_id INTEGER NOT NULL REFERENCES patients(id),
 cervical_dilation INTEGER DEFAULT 0,
 cervical_effacement INTEGER DEFAULT 0,
 descent_station INTEGER DEFAULT 0,
 contractions_per_10min INTEGER DEFAULT 0,
 contraction_duration INTEGER DEFAULT 0,
 contraction_intensity TEXT DEFAULT '',
 fetal_heart_rate_baseline INTEGER DEFAULT 0,
 fetal_heart_rate_variability TEXT DEFAULT '',
 decelerations TEXT DEFAULT 'None',
 molding TEXT DEFAULT 'None',
 caput_succedaneum TEXT DEFAULT 'None',
 meconium INTEGER DEFAULT 0,
 amniotic_fluid TEXT DEFAULT '',
 maternal_bp TEXT DEFAULT '',
 maternal_hr INTEGER DEFAULT 0,
 maternal_temp REAL DEFAULT 0,
 oxytocin_units REAL DEFAULT 0,
 notes TEXT DEFAULT '',
 alert_flags TEXT DEFAULT '', -- server-classified
 recorded_by TEXT DEFAULT '',
 recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4) ultrasounds -----
CREATE TABLE IF NOT EXISTS obgyn_ultrasounds (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 pregnancy_id INTEGER NOT NULL REFERENCES obgyn_pregnancies(id),
 patient_id INTEGER NOT NULL REFERENCES patients(id),
 scan_type TEXT DEFAULT 'Routine',
 gestational_age TEXT DEFAULT '', -- server-derived from biometry
 scan_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 bpd REAL DEFAULT 0,
 hc REAL DEFAULT 0,
 ac REAL DEFAULT 0,
 fl REAL DEFAULT 0,
 efw REAL DEFAULT 0,
 efw_percentile TEXT DEFAULT '', -- server-computed band

```

```

 amniotic_fluid_index REAL DEFAULT 0,
 placenta_location TEXT DEFAULT '',
 placenta_grade TEXT DEFAULT '',
 fetal_heart_rate INTEGER DEFAULT 0,
 fetal_presentation TEXT DEFAULT '',
 fetal_gender TEXT DEFAULT 'Not determined',
 number_of_fetuses INTEGER DEFAULT 1,
 cervical_length REAL DEFAULT 0,
 anomalies TEXT DEFAULT '',
 findings TEXT DEFAULT '',
 impression TEXT DEFAULT '',
 performed_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 5) deliveries -----
CREATE TABLE IF NOT EXISTS obgyn_deliveries (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 pregnancy_id INTEGER NOT NULL REFERENCES obgyn_pregnancies(id),
 patient_id INTEGER NOT NULL REFERENCES patients(id),
 admission_id INTEGER,
 delivery_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 gestational_age_at_delivery TEXT DEFAULT '',
 delivery_type TEXT DEFAULT 'NVD',
 delivery_method TEXT DEFAULT '',
 indication_for_cs TEXT DEFAULT '',
 anesthesia_type TEXT DEFAULT '',
 labor_duration_hours REAL DEFAULT 0,
 episiotomy INTEGER DEFAULT 0,
 perineal_tear TEXT DEFAULT 'None',
 blood_loss_ml INTEGER DEFAULT 0,
 placenta_delivery TEXT DEFAULT 'Complete',
 complications TEXT DEFAULT '',
 attending_doctor TEXT DEFAULT '',
 assisting_nurse TEXT DEFAULT '',
 anesthetist TEXT DEFAULT '',
 pediatrician TEXT DEFAULT '',
 notes TEXT DEFAULT '',
 apgar_1min INTEGER DEFAULT 0, -- server-computed
 apgar_5min INTEGER DEFAULT 0, -- server-computed
 baby_weight INTEGER DEFAULT 0,
 baby_length REAL DEFAULT 0,
 baby_head_circumference REAL DEFAULT 0,
 baby_gender TEXT DEFAULT '',
 baby_status TEXT DEFAULT 'Alive',
 baby_anomalies TEXT DEFAULT '',
 nicu_admission INTEGER DEFAULT 0,
 nicu_reason TEXT DEFAULT '',
 breastfeeding_initiated INTEGER DEFAULT 0,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

-- 6) neonatal -----

```
CREATE TABLE IF NOT EXISTS obgyn_neonatal (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 delivery_id INTEGER NOT NULL REFERENCES obgyn_deliveries(id),
 baby_patient_id INTEGER REFERENCES patients(id),
 apgar_1min INTEGER DEFAULT 0, -- server-computed
 apgar_5min INTEGER DEFAULT 0, -- server-computed
 apgar_10min INTEGER DEFAULT 0, -- server-computed
 birth_weight_grams INTEGER DEFAULT 0,
 length_cm REAL DEFAULT 0,
 head_circumference_cm REAL DEFAULT 0,
 blood_group TEXT DEFAULT '',
 coombs_test TEXT DEFAULT 'Not Done',
 resuscitation_needed INTEGER DEFAULT 0,
 resuscitation_type TEXT DEFAULT '',
 birth_injury TEXT DEFAULT '',
 jaundice_onset TEXT DEFAULT '',
 phototherapy_needed INTEGER DEFAULT 0,
 hypoglycemia INTEGER DEFAULT 0,
 hypothermia INTEGER DEFAULT 0,
 congenital_abnormalities TEXT DEFAULT '',
 feeding_type TEXT DEFAULT 'Breast',
 feeding_established INTEGER DEFAULT 0,
 discharge_destination TEXT DEFAULT 'Home',
 discharge_status TEXT DEFAULT 'Healthy',
 follow_up_plan TEXT DEFAULT '',
 recorded_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- 7) NST -----

```
CREATE TABLE IF NOT EXISTS obgyn_nst (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 pregnancy_id INTEGER NOT NULL REFERENCES obgyn_pregnancies(id),
 patient_id INTEGER NOT NULL REFERENCES patients(id),
 duration_minutes INTEGER DEFAULT 20,
 baseline_fhr INTEGER DEFAULT 0,
 variability TEXT DEFAULT '',
 accelerations INTEGER DEFAULT 0,
 decelerations TEXT DEFAULT 'None',
 contractions INTEGER DEFAULT 0,
 result TEXT DEFAULT 'Reactive',
 interpretation TEXT DEFAULT '',
 action_taken TEXT DEFAULT '',
 performed_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- 8) lab panels catalog -----

```
CREATE TABLE IF NOT EXISTS obgyn_lab_panels (
 id SERIAL PRIMARY KEY,
```

```

tenant_id INTEGER NOT NULL REFERENCES tenants(id),
panel_name TEXT DEFAULT '',
panel_name_ar TEXT DEFAULT '',
trimester TEXT DEFAULT '',
tests TEXT DEFAULT '',
is_active INTEGER DEFAULT 1
);

-- Update old rows with missing tenant_id to use default tenant (1)
UPDATE obgyn_pregnancies SET tenant_id = 1 WHERE tenant_id IS NULL;
UPDATE obgyn_deliveries SET tenant_id = 1 WHERE tenant_id IS NULL;

-- Enforce NOT NULL on tenant_id columns
ALTER TABLE obgyn_pregnancies ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE obgyn_deliveries ALTER COLUMN tenant_id SET NOT NULL;

-- Add Foreign Key constraints for tenant_id and patient_id
ALTER TABLE obgyn_pregnancies DROP CONSTRAINT IF EXISTS fk_obgyn_pregnancies_tenant;
ALTER TABLE obgyn_pregnancies ADD CONSTRAINT fk_obgyn_pregnancies_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;

ALTER TABLE obgyn_pregnancies DROP CONSTRAINT IF EXISTS fk_obgyn_pregnancies_patient;
ALTER TABLE obgyn_pregnancies ADD CONSTRAINT fk_obgyn_pregnancies_patient FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE;

ALTER TABLE obgyn_deliveries DROP CONSTRAINT IF EXISTS fk_obgyn_deliveries_tenant;
ALTER TABLE obgyn_deliveries ADD CONSTRAINT fk_obgyn_deliveries_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;

-- ===== Indexes (RLS + tenant filtering) =====
CREATE INDEX IF NOT EXISTS idx_obgyn_pregnancies_tenant_patient ON obgyn_pregnancies(tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_obgyn_pregnancies_tenant_status ON obgyn_pregnancies(tenant_id, status);
CREATE INDEX IF NOT EXISTS idx_obgyn_antenatal_tenant_pregnancy ON obgyn_antenatal_visits(tenant_id, pregnancy_id);
CREATE INDEX IF NOT EXISTS idx_obgyn_partogram_tenant_pregnancy ON obgyn_partogram(tenant_id, pregnancy_id);
CREATE INDEX IF NOT EXISTS idx_obgyn_ultrasounds_tenant_pregnancy ON obgyn_ultrasounds(tenant_id, pregnancy_id);
);
CREATE INDEX IF NOT EXISTS idx_obgyn_deliveries_tenant_pregnancy ON obgyn_deliveries(tenant_id, pregnancy_id);
CREATE INDEX IF NOT EXISTS idx_obgyn_neonatal_tenant_delivery ON obgyn_neonatal(tenant_id, delivery_id);
CREATE INDEX IF NOT EXISTS idx_obgyn_nst_tenant_pregnancy ON obgyn_nst(tenant_id, pregnancy_id);
CREATE INDEX IF NOT EXISTS idx_obgyn_lab_panels_tenant ON obgyn_lab_panels(tenant_id);

-- ===== FORCE Row-Level Security + canonical tenant policy =====
DO $$
DECLARE t TEXT;
BEGIN
 FOR t IN SELECT unnest(ARRAY[
 'obgyn_pregnancies', 'obgyn_antenatal_visits', 'obgyn_partogram', 'obgyn_ultrasounds',
 'obgyn_deliveries', 'obgyn_neonatal', 'obgyn_nst', 'obgyn_lab_panels'])
 LOOP
 EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
 EXECUTE format('ALTER TABLE %I FORCE ROW LEVEL SECURITY', t);
 EXECUTE format('DROP POLICY IF EXISTS %I ON %I', t || '_tenant_isolation', t);
 EXECUTE format(

```

```

 'CREATE POLICY %I ON %I USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')):
:integer) WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), ''))::integer)',
 t || '_tenant_isolation', t);
 END LOOP;
END $$;

COMMIT;

```

```

=====
FILE: migrations\e14_ob_maternity_validate.sql
=====

```

```

-- =====
-- Epic E14 ? OB / Maternity migration (VALIDATE) [CANDIDATE ? DO NOT EXECUTE here]
-- Asserts: all 8 obgyn_* tables exist with FORCE RLS + canonical isolation policy,
-- tenant_id is NOT NULL and FK -> tenants(id), the entity FK exists, and there are
-- no orphan / null-tenant rows. RAISE EXCEPTION (fail-closed) on any violation.
-- =====

DO $$
DECLARE
 t TEXT;
 tbls TEXT[] := ARRAY['obgyn_pregnancies', 'obgyn_antenatal_visits', 'obgyn_partogram',
 'obgyn_ultrasounds', 'obgyn_deliveries', 'obgyn_neonatal', 'obgyn_nst', 'obgyn_lab_panels'];
 n INTEGER;
BEGIN
 FOREACH t IN ARRAY tbls LOOP
 -- table exists
 IF NOT EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name = t) THEN
 RAISE EXCEPTION 'E14 validate: missing table %', t;
 END IF;

 -- FORCE RLS enabled
 SELECT COUNT(*) INTO n FROM pg_class WHERE relname = t AND relrowsecurity AND relforcerowsecurity;
 IF n <> 1 THEN
 RAISE EXCEPTION 'E14 validate: FORCE RLS not enabled on %', t;
 END IF;

 -- canonical isolation policy present
 IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE tablename = t AND policyname = t || '_tenant_isolation'
) THEN
 RAISE EXCEPTION 'E14 validate: tenant isolation policy missing on %', t;
 END IF;

 -- tenant_id NOT NULL
 IF EXISTS (SELECT 1 FROM information_schema.columns
 WHERE table_name = t AND column_name = 'tenant_id' AND is_nullable = 'YES') THEN
 RAISE EXCEPTION 'E14 validate: tenant_id is nullable on %', t;
 END IF;

 -- tenant_id FK -> tenants
 IF NOT EXISTS (

```



```

SELECT 1 FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name = ccu.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY' AND tc.table_name = t
 AND kcu.column_name = 'tenant_id' AND ccu.table_name = 'tenants') THEN
 RAISE EXCEPTION 'E14 validate: tenant_id FK -> tenants missing on %', t;
END IF;
END LOOP;

-- entity FK presence (parent linkage) per table
IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu ON tc.constraint_name = kcu.constraint_name
WHERE tc.constraint_type='FOREIGN KEY' AND tc.table_name='obgyn_pregnancies' AND kcu.column_name='patient_id') THEN
 RAISE EXCEPTION 'E14 validate: obgyn_pregnancies.patient_id FK missing';
END IF;
IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu ON tc.constraint_name = kcu.constraint_name
WHERE tc.constraint_type='FOREIGN KEY' AND tc.table_name='obgyn_neonatal' AND kcu.column_name='delivery_id') THEN
 RAISE EXCEPTION 'E14 validate: obgyn_neonatal.delivery_id FK missing';
END IF;

-- no null-tenant rows anywhere
FOREACH t IN ARRAY tbls LOOP
 EXECUTE format('SELECT COUNT(*) FROM %I WHERE tenant_id IS NULL', t) INTO n;
 IF n > 0 THEN RAISE EXCEPTION 'E14 validate: % has % null-tenant rows', t, n; END IF;
END LOOP;

-- no orphan antenatal/delivery/neonatal rows
SELECT COUNT(*) INTO n FROM obgyn_antenatal_visits a
 WHERE NOT EXISTS (SELECT 1 FROM obgyn_pregnancies p WHERE p.id = a.pregnancy_id);
IF n > 0 THEN RAISE EXCEPTION 'E14 validate: % orphan antenatal_visits', n; END IF;
SELECT COUNT(*) INTO n FROM obgyn_neonatal nn
 WHERE NOT EXISTS (SELECT 1 FROM obgyn_deliveries d WHERE d.id = nn.delivery_id);
IF n > 0 THEN RAISE EXCEPTION 'E14 validate: % orphan neonatal', n; END IF;
-- no orphan partogram/ultrasound/nst/delivery rows (pregnancy linkage)
SELECT COUNT(*) INTO n FROM obgyn_partogram pt
 WHERE NOT EXISTS (SELECT 1 FROM obgyn_pregnancies p WHERE p.id = pt.pregnancy_id);
IF n > 0 THEN RAISE EXCEPTION 'E14 validate: % orphan partogram', n; END IF;
SELECT COUNT(*) INTO n FROM obgyn_ultrasounds us
 WHERE NOT EXISTS (SELECT 1 FROM obgyn_pregnancies p WHERE p.id = us.pregnancy_id);
IF n > 0 THEN RAISE EXCEPTION 'E14 validate: % orphan ultrasounds', n; END IF;
SELECT COUNT(*) INTO n FROM obgyn_nst ns
 WHERE NOT EXISTS (SELECT 1 FROM obgyn_pregnancies p WHERE p.id = ns.pregnancy_id);
IF n > 0 THEN RAISE EXCEPTION 'E14 validate: % orphan nst', n; END IF;
SELECT COUNT(*) INTO n FROM obgyn_deliveries dv
 WHERE NOT EXISTS (SELECT 1 FROM obgyn_pregnancies p WHERE p.id = dv.pregnancy_id);
IF n > 0 THEN RAISE EXCEPTION 'E14 validate: % orphan deliveries', n; END IF;

RAISE NOTICE 'E14 validate: OK ? 8 obgyn_* tables, FORCE RLS + canonical policy, tenant_id NOT NULL + FK,
entity FKs, no orphans/null-tenant rows.';
END $$;

```

=====

FILE: migrations\e15\_pathology\_01\_specimens\_blocks\_slides\_reports\_down.sql

=====

```
-- =====
-- E15 PATHOLOGY ? DOWN (reverses ONLY this migration's additions).
-- Drops the NEW tables in child->parent order. Never touches pre-existing
-- tables (pathology_cases, pathology_specimens, patients, visits, tenants).
-- =====
BEGIN;
```

```
DROP POLICY IF EXISTS path_reports_tenant_isolation ON path_reports;
DROP POLICY IF EXISTS path_slides_tenant_isolation ON path_slides;
DROP POLICY IF EXISTS path_blocks_tenant_isolation ON path_blocks;
DROP POLICY IF EXISTS path_specimens_tenant_isolation ON path_specimens;
```

```
DROP TABLE IF EXISTS path_reports CASCADE;
DROP TABLE IF EXISTS path_slides CASCADE;
DROP TABLE IF EXISTS path_blocks CASCADE;
DROP TABLE IF EXISTS path_specimens CASCADE;
```

```
COMMIT;
```

=====

FILE: migrations\e15\_pathology\_01\_specimens\_blocks\_slides\_reports\_up.sql

=====

```
-- =====
-- E15 PATHOLOGY ? UP (candidate; idempotent; FORCE RLS + canonical tenant policy)
-- Creates the specimen -> block -> slide hierarchy + structured signed reports.
-- Every table: tenant_id NOT NULL REFERENCES tenants(id), FORCE RLS, canonical
-- policy tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer
-- DO NOT execute manually. Reviewed/applied via the controlled migration runner.
-- =====
BEGIN;
```

```
-- -----
-- 1. path_specimens (accessioned specimen; unique accession per tenant)
-- -----
```

```
CREATE TABLE IF NOT EXISTS path_specimens (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 facility_id INTEGER,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 visit_id INTEGER,
 accession_number TEXT NOT NULL,
 specimen_type TEXT,
 site TEXT,
 clinical_details TEXT,
 priority TEXT NOT NULL DEFAULT 'routine'
```

```

 CHECK (priority IN ('routine','urgent','stat')),
state TEXT NOT NULL DEFAULT 'Received'
 CHECK (state IN ('Received','Grossing','Processing','Reported','SignedOut')),
received_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
blocks_count INTEGER NOT NULL DEFAULT 0,
created_by INTEGER,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT uq_path_specimen_accession UNIQUE (tenant_id, accession_number)
);

ALTER TABLE path_specimens ENABLE ROW LEVEL SECURITY;
ALTER TABLE path_specimens FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS path_specimens_tenant_isolation ON path_specimens;
CREATE POLICY path_specimens_tenant_isolation ON path_specimens
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_path_specimens_tenant ON path_specimens (tenant_id, patient_id);

-- -----
-- 2. path_blocks (cassettes embedded from a specimen)
-- -----

CREATE TABLE IF NOT EXISTS path_blocks (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 specimen_id INTEGER NOT NULL REFERENCES path_specimens(id) ON DELETE CASCADE,
 block_no TEXT NOT NULL,
 cassette_label TEXT,
 embedding_type TEXT DEFAULT 'paraffin',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

ALTER TABLE path_blocks ENABLE ROW LEVEL SECURITY;
ALTER TABLE path_blocks FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS path_blocks_tenant_isolation ON path_blocks;
CREATE POLICY path_blocks_tenant_isolation ON path_blocks
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_path_blocks_specimen ON path_blocks (tenant_id, specimen_id);

-- -----
-- 3. path_slides (stained slides cut from a block)
-- -----

CREATE TABLE IF NOT EXISTS path_slides (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 block_id INTEGER NOT NULL REFERENCES path_blocks(id) ON DELETE CASCADE,
 specimen_id INTEGER NOT NULL REFERENCES path_specimens(id) ON DELETE CASCADE,
 slide_no TEXT NOT NULL,
 stain_type TEXT DEFAULT 'H&E',
 cut_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

ALTER TABLE path_slides ENABLE ROW LEVEL SECURITY;
ALTER TABLE path_slides FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS path_slides_tenant_isolation ON path_slides;

```

```

CREATE POLICY path_slides_tenant_isolation ON path_slides
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_path_slides_block ON path_slides (tenant_id, block_id);

-- -----
-- 4. path_reports (structured report; SNOMED; state machine; sign-out)
-- -----

CREATE TABLE IF NOT EXISTS path_reports (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id),
 facility_id INTEGER,
 specimen_id INTEGER NOT NULL REFERENCES path_specimens(id) ON DELETE CASCADE,
 pathologist_id INTEGER,
 gross_text TEXT,
 micro_text TEXT,
 diagnosis TEXT,
 snomed_codes JSONB DEFAULT '[]'::jsonb,
 icd10_codes JSONB DEFAULT '[]'::jsonb,
 malignancy_flag BOOLEAN NOT NULL DEFAULT FALSE,
 critical_flag BOOLEAN NOT NULL DEFAULT FALSE,
 state TEXT NOT NULL DEFAULT 'Received'
 CHECK (state IN ('Received', 'Grossing', 'Processing', 'Reported', 'SignedOut')),
 signed_at TIMESTAMP,
 signed_by INTEGER,
 addendum_count INTEGER NOT NULL DEFAULT 0,
 addenda JSONB DEFAULT '[]'::jsonb,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT uq_path_report_specimen UNIQUE (tenant_id, specimen_id)
);

ALTER TABLE path_reports ENABLE ROW LEVEL SECURITY;
ALTER TABLE path_reports FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS path_reports_tenant_isolation ON path_reports;
CREATE POLICY path_reports_tenant_isolation ON path_reports
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_path_reports_specimen ON path_reports (tenant_id, specimen_id);

COMMIT;

=====
FILE: migrations\e15_pathology_01_specimens_blocks_slides_reports_validate.sql
=====

-- =====
-- E15 PATHOLOGY ? VALIDATE (asserts: tables exist, FORCE RLS on, canonical
-- policy present, tenant_id NOT NULL + FK to tenants, entity FK present).
-- Raises EXCEPTION on any failure so the controlled runner aborts.
-- =====

DO $$
DECLARE

```

```

t TEXT;
tbls TEXT[] := ARRAY['path_specimens','path_blocks','path_slides','path_reports'];
BEGIN
 FOREACH t IN ARRAY tbls LOOP
 -- table exists
 IF NOT EXISTS (SELECT 1 FROM information_schema.tables
 WHERE table_name = t AND table_schema = 'public') THEN
 RAISE EXCEPTION 'E15 validate: table % missing', t;
 END IF;

 -- RLS enabled AND forced
 IF NOT EXISTS (SELECT 1 FROM pg_class c JOIN pg_namespace n ON n.oid=c.relnamespace
 WHERE c.relname = t AND n.nspname='public'
 AND c.relrowsecurity = TRUE AND c.relforcerowsecurity = TRUE) THEN
 RAISE EXCEPTION 'E15 validate: FORCE ROW LEVEL SECURITY not set on %', t;
 END IF;

 -- canonical tenant-isolation policy present
 IF NOT EXISTS (SELECT 1 FROM pg_policies
 WHERE tablename = t AND policyname = t || '_tenant_isolation') THEN
 RAISE EXCEPTION 'E15 validate: tenant isolation policy missing on %', t;
 END IF;

 -- tenant_id present and NOT NULL
 IF NOT EXISTS (SELECT 1 FROM information_schema.columns
 WHERE table_name = t AND column_name = 'tenant_id'
 AND is_nullable = 'NO') THEN
 RAISE EXCEPTION 'E15 validate: tenant_id NOT NULL missing on %', t;
 END IF;

 -- tenant_id FK -> tenants
 IF NOT EXISTS (
 SELECT 1 FROM information_schema.table_constraints tc
 JOIN information_schema.key_column_usage kcu ON tc.constraint_name = kcu.constraint_name
 JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name = ccu.constraint_name
 WHERE tc.constraint_type = 'FOREIGN KEY' AND tc.table_name = t
 AND kcu.column_name = 'tenant_id' AND ccu.table_name = 'tenants'
) THEN
 RAISE EXCEPTION 'E15 validate: tenant_id FK -> tenants missing on %', t;
 END IF;
 END LOOP;

 -- entity FK chain: specimen->patients, block->specimen, slide->block, report->specimen
 IF NOT EXISTS (
 SELECT 1 FROM information_schema.table_constraints tc
 JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name = ccu.constraint_name
 WHERE tc.constraint_type='FOREIGN KEY' AND tc.table_name='path_specimens' AND ccu.table_name='patients'
) THEN RAISE EXCEPTION 'E15 validate: path_specimens.patient_id FK -> patients missing'; END IF;

 IF NOT EXISTS (
 SELECT 1 FROM information_schema.table_constraints tc
 JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name = ccu.constraint_name
 WHERE tc.constraint_type='FOREIGN KEY' AND tc.table_name='path_blocks' AND ccu.table_name='path_specim

```

```

ens'
) THEN RAISE EXCEPTION 'E15 validate: path_blocks.specimen_id FK -> path_specimens missing'; END IF;

IF NOT EXISTS (
 SELECT 1 FROM information_schema.table_constraints tc
 JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name = ccu.constraint_name
 WHERE tc.constraint_type='FOREIGN KEY' AND tc.table_name='path_slides' AND ccu.table_name='path_blocks
'
) THEN RAISE EXCEPTION 'E15 validate: path_slides.block_id FK -> path_blocks missing'; END IF;

IF NOT EXISTS (
 SELECT 1 FROM information_schema.table_constraints tc
 JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name = ccu.constraint_name
 WHERE tc.constraint_type='FOREIGN KEY' AND tc.table_name='path_reports' AND ccu.table_name='path_speci
mens'
) THEN RAISE EXCEPTION 'E15 validate: path_reports.specimen_id FK -> path_specimens missing'; END IF;

-- unique accession per tenant
IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname = 'uq_path_specimen_accession') THEN
 RAISE EXCEPTION 'E15 validate: UNIQUE(tenant_id, accession_number) missing on path_specimens';
END IF;

RAISE NOTICE 'E15 validate: OK ? all pathology tables, FORCE RLS, policies, FKs, NOT NULL verified.';
END $$;

```

=====

FILE: migrations\e16\_01\_inventory\_cssd\_rls\_down.sql

=====

```

-- e16_01_inventory_cssd_rls_down.sql (rollback of e16_01 up)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ??????: ??? ?????? ?????? ?????? ?????? (?? ?????? ??? ??????) ? ??? ???? ?????????? ???;
-- ????? ????????? + ??? FORCE RLS + ????? ?????/????? tenant_id ??????. ?? ????? ???????
-- ??? ????? tenant_id ??? (?? ?????? ??? ?????/??????? ??? ??????). idempotent (IF EXISTS).
BEGIN;

```

```

-- inventory_items
DROP POLICY IF EXISTS rls_inventory_items_tenant_isolation ON inventory_items;
ALTER TABLE inventory_items NO FORCE ROW LEVEL SECURITY;
ALTER TABLE inventory_items DISABLE ROW LEVEL SECURITY;
ALTER TABLE inventory_items DROP CONSTRAINT IF EXISTS fk_inventory_items_tenant;
DROP INDEX IF EXISTS idx_inventory_items_tenant_id;

```

```

-- cssd_instrument_sets
DROP POLICY IF EXISTS rls_cssd_instrument_sets_tenant_isolation ON cssd_instrument_sets;
ALTER TABLE cssd_instrument_sets NO FORCE ROW LEVEL SECURITY;
ALTER TABLE cssd_instrument_sets DISABLE ROW LEVEL SECURITY;
ALTER TABLE cssd_instrument_sets DROP CONSTRAINT IF EXISTS fk_cssd_instrument_sets_tenant;
DROP INDEX IF EXISTS idx_cssd_instrument_sets_tenant_id;

```

```

-- cssd_sterilization_cycles

```

```

DROP POLICY IF EXISTS rls_cssd_cycles_tenant_isolation ON cssd_sterilization_cycles;
ALTER TABLE cssd_sterilization_cycles NO FORCE ROW LEVEL SECURITY;
ALTER TABLE cssd_sterilization_cycles DISABLE ROW LEVEL SECURITY;
ALTER TABLE cssd_sterilization_cycles DROP CONSTRAINT IF EXISTS fk_cssd_cycles_tenant;
DROP INDEX IF EXISTS idx_cssd_cycles_tenant_id;

```

```

-- cssd_load_items
DROP POLICY IF EXISTS rls_cssd_load_items_tenant_isolation ON cssd_load_items;
ALTER TABLE cssd_load_items NO FORCE ROW LEVEL SECURITY;
ALTER TABLE cssd_load_items DISABLE ROW LEVEL SECURITY;
ALTER TABLE cssd_load_items DROP CONSTRAINT IF EXISTS fk_cssd_load_items_tenant;
DROP INDEX IF EXISTS idx_cssd_load_items_tenant_id;

```

```

COMMIT;

```

```

=====
FILE: migrations\e16_01_inventory_cssd_rls_up.sql
=====

```

```

-- =====
-- e16_01_inventory_cssd_rls_up.sql
-- E16 INVENTORY / SUPPLY CHAIN + CSSD ? FORCE RLS + tenant_id on the four EXISTING
-- inventory/CSSD tables, plus the CSSD biological-indicator (BI) gate columns.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ???? ????? ? ??? ??????? ??????? (inventory_items, cssd_instrument_sets,
-- cssd_sterilization_cycles, cssd_load_items) ????? ?????? ??? ??? tenant_id ??? FORCE RLS.
-- ??? tenant_id NOT NULL REFERENCES tenants(id) + facility_id + ?????? ????????? ??????
-- ????? ?????? ?????? ????????? ?????? (BI) ??? ?????? ????????? ??? ?????? ?????? fail-CLOSED.
--
-- ?????? ?????? ?????? => ADD COLUMN IF NOT EXISTS + backfill dev (tenant_id=1) + SET NOT NULL
-- + DROP/ADD CONSTRAINT IF EXISTS + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS. idempotent.
-- down ??? ????????? ??? (?? ?????? ??? ????????? ?????????).
-- ?????: UPDATE ... SET tenant_id=1 WHERE tenant_id IS NULL ??? dev ? ?????? ??? ?????? ??? ?????? ?????????
???.
-- =====

```

```

BEGIN;

```

```

-- ===== inventory_items (canonical item master for batches/movements) =====
ALTER TABLE inventory_items ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
ALTER TABLE inventory_items ADD COLUMN IF NOT EXISTS facility_id INTEGER;
ALTER TABLE inventory_items ADD COLUMN IF NOT EXISTS reorder_point INTEGER DEFAULT 0;
ALTER TABLE inventory_items ADD COLUMN IF NOT EXISTS created_at TIMESTAMP DEFAULT now();
ALTER TABLE inventory_items ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP DEFAULT now();
UPDATE inventory_items SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE inventory_items ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE inventory_items DROP CONSTRAINT IF EXISTS fk_inventory_items_tenant;
ALTER TABLE inventory_items ADD CONSTRAINT fk_inventory_items_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_inventory_items_tenant_id ON inventory_items (tenant_id);
ALTER TABLE inventory_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE inventory_items FORCE ROW LEVEL SECURITY;

```

```

DROP POLICY IF EXISTS rls_inventory_items_tenant_isolation ON inventory_items;
CREATE POLICY rls_inventory_items_tenant_isolation ON inventory_items
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== cssd_instrument_sets =====
ALTER TABLE cssd_instrument_sets ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
ALTER TABLE cssd_instrument_sets ADD COLUMN IF NOT EXISTS facility_id INTEGER;
ALTER TABLE cssd_instrument_sets ADD COLUMN IF NOT EXISTS created_at TIMESTAMP DEFAULT now();
UPDATE cssd_instrument_sets SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE cssd_instrument_sets ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE cssd_instrument_sets DROP CONSTRAINT IF EXISTS fk_cssd_instrument_sets_tenant;
ALTER TABLE cssd_instrument_sets ADD CONSTRAINT fk_cssd_instrument_sets_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_cssd_instrument_sets_tenant_id ON cssd_instrument_sets (tenant_id);
ALTER TABLE cssd_instrument_sets ENABLE ROW LEVEL SECURITY;
ALTER TABLE cssd_instrument_sets FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_cssd_instrument_sets_tenant_isolation ON cssd_instrument_sets;
CREATE POLICY rls_cssd_instrument_sets_tenant_isolation ON cssd_instrument_sets
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== cssd_sterilization_cycles (+ BI gate columns) =====
ALTER TABLE cssd_sterilization_cycles ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
ALTER TABLE cssd_sterilization_cycles ADD COLUMN IF NOT EXISTS facility_id INTEGER;
ALTER TABLE cssd_sterilization_cycles ADD COLUMN IF NOT EXISTS bi_indicator_lot TEXT DEFAULT '';
ALTER TABLE cssd_sterilization_cycles ADD COLUMN IF NOT EXISTS bi_result_recorded_at TIMESTAMP;
ALTER TABLE cssd_sterilization_cycles ADD COLUMN IF NOT EXISTS bi_result_by TEXT DEFAULT '';
ALTER TABLE cssd_sterilization_cycles ADD COLUMN IF NOT EXISTS released_for_issue INTEGER DEFAULT 0;
UPDATE cssd_sterilization_cycles SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE cssd_sterilization_cycles ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE cssd_sterilization_cycles DROP CONSTRAINT IF EXISTS fk_cssd_cycles_tenant;
ALTER TABLE cssd_sterilization_cycles ADD CONSTRAINT fk_cssd_cycles_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
CREATE INDEX IF NOT EXISTS idx_cssd_cycles_tenant_id ON cssd_sterilization_cycles (tenant_id);
ALTER TABLE cssd_sterilization_cycles ENABLE ROW LEVEL SECURITY;
ALTER TABLE cssd_sterilization_cycles FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_cssd_cycles_tenant_isolation ON cssd_sterilization_cycles;
CREATE POLICY rls_cssd_cycles_tenant_isolation ON cssd_sterilization_cycles
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== cssd_load_items =====
ALTER TABLE cssd_load_items ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
ALTER TABLE cssd_load_items ADD COLUMN IF NOT EXISTS facility_id INTEGER;
ALTER TABLE cssd_load_items ADD COLUMN IF NOT EXISTS quantity INTEGER DEFAULT 1;
UPDATE cssd_load_items SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE cssd_load_items ALTER COLUMN tenant_id SET NOT NULL;
ALTER TABLE cssd_load_items DROP CONSTRAINT IF EXISTS fk_cssd_load_items_tenant;
ALTER TABLE cssd_load_items ADD CONSTRAINT fk_cssd_load_items_tenant
 FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;

```



```

CREATE INDEX IF NOT EXISTS idx_cssd_load_items_tenant_id ON cssd_load_items (tenant_id);
ALTER TABLE cssd_load_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE cssd_load_items FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_cssd_load_items_tenant_isolation ON cssd_load_items;
CREATE POLICY rls_cssd_load_items_tenant_isolation ON cssd_load_items
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

=====

FILE: migrations\e16\_01\_inventory\_cssd\_qls\_validate.sql

=====

```

-- e16_01_inventory_cssd_qls_validate.sql (run AFTER e16_01 up; read-only)
-- PASS = each of the four tables has tenant_id NOT NULL + FK->tenants + FORCE RLS + isolation policy.
SELECT
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='inventory_items') AS inv_i
tems_force_qls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='inventory_items'
 AND policynane='rls_inventory_items_tenant_isolation') AS inv_
items_policy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='inventory_items'
 AND column_name='tenant_id' AND is_nullable='NO') AS inv_
items_tenant_not_null, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='inventory_items'::regclass
 AND confrelid='tenants'::regclass AND contype='f') AS inv_
items_tenant_fk, -- 1
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='cssd_instrument_sets') AS sets
_force_qls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='cssd_instrument_sets'
 AND policynane='rls_cssd_instrument_sets_tenant_isolation') AS sets
_policy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='cssd_instrument_sets'
 AND column_name='tenant_id' AND is_nullable='NO') AS sets
_tenant_not_null, -- 1
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='cssd_sterilization_cycles') AS cycl
es_force_qls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='cssd_sterilization_cycles'
 AND policynane='rls_cssd_cycles_tenant_isolation') AS cycl
es_policy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='cssd_sterilization_cycles'
 AND column_name='released_for_issue') AS cycl
es_bi_gate_col, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='cssd_sterilization_cycles'
 AND column_name='tenant_id' AND is_nullable='NO') AS cycl
es_tenant_not_null, -- 1
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='cssd_load_items') AS load
_force_qls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='cssd_load_items'
 AND policynane='rls_cssd_load_items_tenant_isolation') AS load

```

```

_policy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='cssd_load_items'
 AND column_name='tenant_id' AND is_nullable='NO')
 AS load
_tenant_not_null, -- 1
 -- tenant-FK existence assertions (I3: match ADD CONSTRAINT statements in up script)
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='cssd_instrument_sets'::regclass
 AND confrelid='tenants'::regclass AND contype='f')
 AS sets
_tenant_fk, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='cssd_sterilization_cycles'::regclass
 AND confrelid='tenants'::regclass AND contype='f')
 AS cycl
es_tenant_fk, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='cssd_load_items'::regclass
 AND confrelid='tenants'::regclass AND contype='f')
 AS load
_tenant_fk; -- 1

```

=====

FILE: migrations\e16\_02\_supply\_chain\_down.sql

=====

```

-- e16_02_supply_chain_down.sql (rollback of e16_02 up)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ?? ??? ??????? ?????? ??????? e16_02 ? ??????? ?????? ?????? ?????????? ???FK
-- (??????? ? ???????) ??? DROP TABLE ?????? ???????/??????/????? ??????. idempotent (IF EXISTS).
BEGIN;

```

```

DROP TABLE IF EXISTS inventory_stock_counts CASCADE;
DROP TABLE IF EXISTS goods_receipt_items CASCADE;
DROP TABLE IF EXISTS goods_receipts CASCADE;
DROP TABLE IF EXISTS purchase_order_items CASCADE;
DROP TABLE IF EXISTS purchase_orders CASCADE;
DROP TABLE IF EXISTS inventory_movements CASCADE;
DROP TABLE IF EXISTS inventory_batches CASCADE;

```

```

COMMIT;

```

=====

FILE: migrations\e16\_02\_supply\_chain\_up.sql

=====

```

-- =====
-- e16_02_supply_chain_up.sql
-- E16 ? NEW supply-chain tables: batches (lot/expiry/FEF0), movements ledger
-- (transactional issue/receive/adjust/transfer), purchase orders + PO items,
-- goods-receipt notes (GRN) + GRN items, and periodic stock counts.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ????? ?????? ?????? => tenant_id INTEGER NOT NULL REFERENCES tenants(id) ??? ??????? + FORCE RLS
-- ??????? ????????? + FK ?????? ????? (inventory_items / purchase_orders / goods_receipts).
-- NOT ??????? ??? bootstrap ?? db_postgres.js (????? ??????? ???).

```

```

-- idempotent: CREATE TABLE/INDEX IF NOT EXISTS + DROP/CREATE POLICY IF EXISTS. BEGIN;?COMMIT;
--
-- ???? ???? ??? ????? DB (???? ?? ????? ?? ???? ??????):
-- - inventory_batches.qty_on_hand >= 0 => ?????? ?? ???? ??????? ?????? ??? ?? ???? ??????.
-- - ????? ??????? qty_delta <> 0 ?????? ??? ?????? ??????.
-- =====
BEGIN;

-- ===== inventory_batches: lot/expiry per item (FEFO source of truth) =====
CREATE TABLE IF NOT EXISTS inventory_batches (
 id SERIAL PRIMARY KEY,
 item_id INTEGER NOT NULL REFERENCES inventory_items(id) ON DELETE CASCADE,
 lot_number TEXT DEFAULT '',
 expiry_date DATE,
 qty_received INTEGER NOT NULL DEFAULT 0,
 qty_on_hand INTEGER NOT NULL DEFAULT 0,
 unit_cost REAL DEFAULT 0,
 received_at TIMESTAMP DEFAULT now(),
 status TEXT NOT NULL DEFAULT 'active',
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 created_at TIMESTAMP DEFAULT now(),
 CONSTRAINT chk_inv_batches_qty_on_hand_nonneg CHECK (qty_on_hand >= 0),
 CONSTRAINT chk_inv_batches_status CHECK (status IN ('active','quarantine','expired','depleted'))
);
CREATE INDEX IF NOT EXISTS idx_inv_batches_tenant_id ON inventory_batches (tenant_id);
CREATE INDEX IF NOT EXISTS idx_inv_batches_fefo ON inventory_batches (tenant_id, item_id, expiry_date);
ALTER TABLE inventory_batches ENABLE ROW LEVEL SECURITY;
ALTER TABLE inventory_batches FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_inv_batches_tenant_isolation ON inventory_batches;
CREATE POLICY rls_inv_batches_tenant_isolation ON inventory_batches
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== inventory_movements: append-only transactional stock ledger =====
CREATE TABLE IF NOT EXISTS inventory_movements (
 id SERIAL PRIMARY KEY,
 item_id INTEGER NOT NULL REFERENCES inventory_items(id) ON DELETE CASCADE,
 batch_id INTEGER REFERENCES inventory_batches(id) ON DELETE SET NULL,
 movement_type TEXT NOT NULL,
 qty_delta INTEGER NOT NULL,
 balance_after INTEGER,
 ref_table TEXT DEFAULT '',
 ref_id INTEGER,
 reason TEXT DEFAULT '',
 created_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT now(),
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 CONSTRAINT chk_inv_mov_type CHECK (movement_type IN
 ('receive','issue','adjust_in','adjust_out','transfer_in','transfer_out')),
 CONSTRAINT chk_inv_mov_qty_nonzero CHECK (qty_delta <> 0)
);

```

```

CREATE INDEX IF NOT EXISTS idx_inv_mov_tenant_id ON inventory_movements (tenant_id);
CREATE INDEX IF NOT EXISTS idx_inv_mov_item ON inventory_movements (tenant_id, item_id, created_at);
ALTER TABLE inventory_movements ENABLE ROW LEVEL SECURITY;
ALTER TABLE inventory_movements FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_inv_mov_tenant_isolation ON inventory_movements;
CREATE POLICY rls_inv_mov_tenant_isolation ON inventory_movements
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== purchase_orders: draft -> approved -> partially_received -> received | cancelled =====
CREATE TABLE IF NOT EXISTS purchase_orders (
 id SERIAL PRIMARY KEY,
 po_number TEXT DEFAULT '',
 supplier_id INTEGER,
 supplier_name TEXT DEFAULT '',
 status TEXT NOT NULL DEFAULT 'draft',
 total_amount REAL DEFAULT 0,
 notes TEXT DEFAULT '',
 created_by TEXT DEFAULT '',
 approved_by TEXT DEFAULT '',
 approved_at TIMESTAMP,
 created_at TIMESTAMP DEFAULT now(),
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 CONSTRAINT chk_po_status CHECK (status IN
 ('draft', 'approved', 'partially_received', 'received', 'cancelled'))
);
CREATE INDEX IF NOT EXISTS idx_po_tenant_id ON purchase_orders (tenant_id);
ALTER TABLE purchase_orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE purchase_orders FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_po_tenant_isolation ON purchase_orders;
CREATE POLICY rls_po_tenant_isolation ON purchase_orders
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== purchase_order_items =====
CREATE TABLE IF NOT EXISTS purchase_order_items (
 id SERIAL PRIMARY KEY,
 po_id INTEGER NOT NULL REFERENCES purchase_orders(id) ON DELETE CASCADE,
 item_id INTEGER NOT NULL REFERENCES inventory_items(id) ON DELETE CASCADE,
 qty_ordered INTEGER NOT NULL DEFAULT 0,
 qty_received INTEGER NOT NULL DEFAULT 0,
 unit_cost REAL DEFAULT 0,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 CONSTRAINT chk_po_items_qty_ordered_pos CHECK (qty_ordered > 0),
 CONSTRAINT chk_po_items_qty_received_nonneg CHECK (qty_received >= 0)
);
CREATE INDEX IF NOT EXISTS idx_po_items_tenant_id ON purchase_order_items (tenant_id);
CREATE INDEX IF NOT EXISTS idx_po_items_po ON purchase_order_items (tenant_id, po_id);
ALTER TABLE purchase_order_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE purchase_order_items FORCE ROW LEVEL SECURITY;

```

```

DROP POLICY IF EXISTS rls_po_items_tenant_isolation ON purchase_order_items;
CREATE POLICY rls_po_items_tenant_isolation ON purchase_order_items
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== goods_receipts (GRN header): one receipt event against a PO =====
CREATE TABLE IF NOT EXISTS goods_receipts (
 id SERIAL PRIMARY KEY,
 grn_number TEXT DEFAULT '',
 po_id INTEGER NOT NULL REFERENCES purchase_orders(id) ON DELETE CASCADE,
 received_by TEXT DEFAULT '',
 received_at TIMESTAMP DEFAULT now(),
 notes TEXT DEFAULT '',
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER
);
CREATE INDEX IF NOT EXISTS idx_grn_tenant_id ON goods_receipts (tenant_id);
CREATE INDEX IF NOT EXISTS idx_grn_po ON goods_receipts (tenant_id, po_id);
ALTER TABLE goods_receipts ENABLE ROW LEVEL SECURITY;
ALTER TABLE goods_receipts FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_grn_tenant_isolation ON goods_receipts;
CREATE POLICY rls_grn_tenant_isolation ON goods_receipts
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== goods_receipt_items: per-line received qty (creates a batch + movement) =====
CREATE TABLE IF NOT EXISTS goods_receipt_items (
 id SERIAL PRIMARY KEY,
 grn_id INTEGER NOT NULL REFERENCES goods_receipts(id) ON DELETE CASCADE,
 po_item_id INTEGER REFERENCES purchase_order_items(id) ON DELETE SET NULL,
 item_id INTEGER NOT NULL REFERENCES inventory_items(id) ON DELETE CASCADE,
 batch_id INTEGER REFERENCES inventory_batches(id) ON DELETE SET NULL,
 qty_received INTEGER NOT NULL DEFAULT 0,
 lot_number TEXT DEFAULT '',
 expiry_date DATE,
 unit_cost REAL DEFAULT 0,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 CONSTRAINT chk_grn_items_qty_pos CHECK (qty_received > 0)
);
CREATE INDEX IF NOT EXISTS idx_grn_items_tenant_id ON goods_receipt_items (tenant_id);
CREATE INDEX IF NOT EXISTS idx_grn_items_grn ON goods_receipt_items (tenant_id, grn_id);
ALTER TABLE goods_receipt_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE goods_receipt_items FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_grn_items_tenant_isolation ON goods_receipt_items;
CREATE POLICY rls_grn_items_tenant_isolation ON goods_receipt_items
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== inventory_stock_counts: periodic count / reconciliation =====
CREATE TABLE IF NOT EXISTS inventory_stock_counts (

```

```

id SERIAL PRIMARY KEY,
item_id INTEGER NOT NULL REFERENCES inventory_items(id) ON DELETE CASCADE,
batch_id INTEGER REFERENCES inventory_batches(id) ON DELETE SET NULL,
system_qty INTEGER NOT NULL DEFAULT 0,
counted_qty INTEGER NOT NULL DEFAULT 0,
difference INTEGER NOT NULL DEFAULT 0,
reconciled INTEGER NOT NULL DEFAULT 0,
counted_by TEXT DEFAULT '',
count_date TIMESTAMP DEFAULT now(),
notes TEXT DEFAULT '',
tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
facility_id INTEGER
);
CREATE INDEX IF NOT EXISTS idx_stock_counts_tenant_id ON inventory_stock_counts (tenant_id);
ALTER TABLE inventory_stock_counts ENABLE ROW LEVEL SECURITY;
ALTER TABLE inventory_stock_counts FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_stock_counts_tenant_isolation ON inventory_stock_counts;
CREATE POLICY rls_stock_counts_tenant_isolation ON inventory_stock_counts
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

```

=====
FILE: migrations\e16_02_supply_chain_validate.sql
=====

```

```

-- e16_02_supply_chain_validate.sql (run AFTER e16_02 up; read-only)
-- PASS = all seven NEW tables exist with tenant_id NOT NULL + FK->tenants + FORCE RLS + isolation policy,
-- plus the non-negative-stock + movement-type guard constraints.
SELECT
 -- inventory_batches
 (SELECT count(*)::int FROM information_schema.tables WHERE table_name='inventory_batches') AS bat
ches_exists, -- 1
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='inventory_batches') AS bat
ches_force_rls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='inventory_batches'
 AND policyname='rls_inv_batches_tenant_isolation') AS bat
ches_policy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='inventory_batches'
 AND column_name='tenant_id' AND is_nullable='NO') AS bat
ches_tenant_nn, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='inventory_batches'::regclass
 AND confrelid='tenants'::regclass AND contype='f') AS bat
ches_tenant_fk, -- 1
 (SELECT count(*)::int FROM pg_constraint
 WHERE conname='chk_inv_batches_qty_on_hand_nonneg') AS bat
ches_nonneg_chk, -- 1
 -- inventory_movements
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='inventory_movements') AS mov
_force_rls, -- t

```

```

 (SELECT count(*) FROM pg_policies WHERE tablename='inventory_movements'
 AND policyname='rls_inv_mov_tenant_isolation')
mov_policy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='inventory_movements'
 AND column_name='tenant_id' AND is_nullable='NO')
mov_tenant_nn, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conname='chk_inv_mov_type')
mov_type_chk, -- 1
 -- purchase_orders + items
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='purchase_orders')
mov_force_rls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='purchase_orders'
 AND policyname='rls_po_tenant_isolation')
mov_policy, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conname='chk_po_status')
mov_status_chk, -- 1
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='purchase_order_items')
mov_items_force_rls, -- t
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='purchase_order_items'::regclass
 AND confrelid='purchase_orders'::regclass AND contype='f')
mov_items_po_fk, -- 1
 -- goods_receipts + items
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='goods_receipts')
mov_force_rls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='goods_receipts'
 AND policyname='rls_grn_tenant_isolation')
mov_policy, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='goods_receipts'::regclass
 AND confrelid='purchase_orders'::regclass AND contype='f')
mov_po_fk, -- 1
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='goods_receipt_items')
mov_items_force_rls, -- t
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='goods_receipt_items'
 AND column_name='tenant_id' AND is_nullable='NO')
mov_items_tenant_nn, -- 1
 -- inventory_stock_counts
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='inventory_stock_counts')
mov_nts_force_rls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='inventory_stock_counts'
 AND policyname='rls_stock_counts_tenant_isolation')
mov_nts_policy; -- 1

```

=====

FILE: migrations\e16\_03\_cssd\_trays\_down.sql

=====

```

-- e16_03_cssd_trays_down.sql (rollback of e16_03 up)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- cssd_trays ???? ???? ?????? ??? ?????? ? ?????? ????? (????? ???????/??????/????? ?????). idempotent.
BEGIN;

```

```

DROP TABLE IF EXISTS cssd_trays CASCADE;
DROP POLICY IF EXISTS rls_cssd_trays_tenant_isolation ON cssd_trays;

COMMIT;

```

```

=====
FILE: migrations\e16_03_cssd_trays_up.sql
=====

```

```

-- =====
-- e16_03_cssd_trays_up.sql
-- E16 CSSD ? NEW table: cssd_trays (instrument trays processed through a sterilization
-- cycle; a tray may only reach 'sterile'/'issued' after its cycle's biological indicator
-- PASSED ? enforced fail-CLOSED in server.js; this table records the lifecycle + audit).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ???? ???? ???? => tenant_id INTEGER NOT NULL REFERENCES tenants(id) ??? ?????? + FORCE RLS
-- ?????? ???????? + FK ?????? ???????? ???? ???????. NOT ?????? ??? bootstrap ?? db_postgres.js.
-- idempotent: CREATE TABLE/INDEX IF NOT EXISTS + DROP/CREATE POLICY IF EXISTS. BEGIN;?COMMIT;
--
-- ??? ???? ??????: packed -> in_cycle -> sterile -> issued | quarantine | recalled.
-- ???????? ??? sterile/issued ?????? ?????? ??? ??? ?????? BI = pass (?? ???????? ??????).
-- =====
BEGIN;

```

```

CREATE TABLE IF NOT EXISTS cssd_trays (
 id SERIAL PRIMARY KEY,
 tray_code TEXT DEFAULT '',
 set_id INTEGER REFERENCES cssd_instrument_sets(id) ON DELETE SET NULL,
 cycle_id INTEGER REFERENCES cssd_sterilization_cycles(id) ON DELETE SET NULL,
 department TEXT DEFAULT '',
 status TEXT NOT NULL DEFAULT 'packed',
 sterilized_at TIMESTAMP,
 issued_to TEXT DEFAULT '',
 issued_at TIMESTAMP,
 used_in_surgery_id INTEGER,
 notes TEXT DEFAULT '',
 created_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT now(),
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 CONSTRAINT chk_cssd_trays_status CHECK (status IN
 ('packed','in_cycle','sterile','issued','quarantine','recalled'))
);

CREATE INDEX IF NOT EXISTS idx_cssd_trays_tenant_id ON cssd_trays (tenant_id);
CREATE INDEX IF NOT EXISTS idx_cssd_trays_cycle ON cssd_trays (tenant_id, cycle_id);
ALTER TABLE cssd_trays ENABLE ROW LEVEL SECURITY;
ALTER TABLE cssd_trays FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_cssd_trays_tenant_isolation ON cssd_trays;
CREATE POLICY rls_cssd_trays_tenant_isolation ON cssd_trays
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)

```



```

 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: migrations\e16_03_cssd_trays_validate.sql
=====

-- e16_03_cssd_trays_validate.sql (run AFTER e16_03 up; read-only)
-- PASS = cssd_trays exists with tenant_id NOT NULL FK->tenants + FORCE RLS + isolation policy
-- + cycle FK + status state-machine check + index.
SELECT
 (SELECT count(*)::int FROM information_schema.tables WHERE table_name='cssd_trays') AS tabl
e_exists, -- 1
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='cssd_trays') AS forc
e_rls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='cssd_trays'
 AND policyname='rls_cssd_trays_tenant_isolation') AS poli
cy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='cssd_trays'
 AND column_name='tenant_id' AND is_nullable='NO') AS tena
nt_not_null, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='cssd_trays'::regclass
 AND confrelid='tenants'::regclass AND contype='f') AS tena
nt_fk, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='cssd_trays'::regclass
 AND confrelid='cssd_sterilization_cycles'::regclass AND contype='f') AS cycl
e_fk, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conname='chk_cssd_trays_status') AS stat
us_chk, -- 1
 (SELECT count(*) FROM pg_indexes WHERE tablename='cssd_trays'
 AND indexname='idx_cssd_trays_tenant_id') AS idx;
 -- 1

=====
FILE: migrations\e17_001_quality_capa_down.sql
=====

-- =====
-- Epic E17 ? Quality / CAPA / Risk migration DOWN (reverse only E17 additions).
-- Drops ONLY the 2 NEW tables and removes ONLY the columns added by the up
-- migration. Does NOT drop the pre-existing quality_incidents table.
-- =====
BEGIN;
DROP TABLE IF EXISTS quality_risk_register CASCADE;
DROP TABLE IF EXISTS quality_capa CASCADE;

ALTER TABLE quality_incidents DROP COLUMN IF EXISTS workflow_state;
ALTER TABLE quality_incidents DROP COLUMN IF EXISTS visit_id;
ALTER TABLE quality_incidents DROP COLUMN IF EXISTS encounter_id;

```

```

ALTER TABLE quality_incidents DROP COLUMN IF EXISTS confidential;
ALTER TABLE quality_incidents DROP COLUMN IF EXISTS near_miss;
ALTER TABLE quality_incidents DROP COLUMN IF EXISTS harm_level;
COMMIT;

```

```

=====
FILE: migrations\e17_001_quality_capa_up.sql
=====

```

```

-- =====
-- Epic E17 ? Quality / Incidents / CAPA + Risk Register
-- Candidate migration (UP). Idempotent. DO NOT auto-execute in production.
-- Adds 2 new tables (quality_capa, quality_risk_register) with FORCE RLS +
-- canonical tenant policy, and REVERSIBLY extends the pre-existing
-- quality_incidents table with E17 incident-management columns
-- (harm_level, near_miss, confidential, encounter_id, visit_id, capa state).
-- The down migration drops the 2 NEW tables and removes ONLY the added columns;
-- it never drops quality_incidents itself.
-- =====

BEGIN;

-- ---- Extend pre-existing quality_incidents (additions are reversed by down) ----
ALTER TABLE quality_incidents ADD COLUMN IF NOT EXISTS harm_level TEXT NOT NULL DEFAULT 'None';
ALTER TABLE quality_incidents ADD COLUMN IF NOT EXISTS near_miss INTEGER NOT NULL DEFAULT 0;
ALTER TABLE quality_incidents ADD COLUMN IF NOT EXISTS confidential INTEGER NOT NULL DEFAULT 0;
ALTER TABLE quality_incidents ADD COLUMN IF NOT EXISTS encounter_id INTEGER;
ALTER TABLE quality_incidents ADD COLUMN IF NOT EXISTS visit_id INTEGER;
ALTER TABLE quality_incidents ADD COLUMN IF NOT EXISTS workflow_state TEXT NOT NULL DEFAULT 'Open';
CREATE INDEX IF NOT EXISTS idx_quality_incidents_tenant ON quality_incidents (tenant_id);
CREATE INDEX IF NOT EXISTS idx_quality_incidents_tenant_state ON quality_incidents (tenant_id, workflow_state)
;

-- ---- CAPA: corrective / preventive actions with owner, due date, state machine ----
CREATE TABLE IF NOT EXISTS quality_capa (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 incident_id INTEGER NOT NULL REFERENCES quality_incidents(id) ON DELETE CASCADE,
 capa_type TEXT NOT NULL DEFAULT 'Corrective', -- Corrective | Preventive
 title TEXT NOT NULL DEFAULT '',
 description TEXT NOT NULL DEFAULT '',
 root_cause TEXT DEFAULT '',
 owner_user_id INTEGER,
 owner_name TEXT DEFAULT '',
 due_date DATE,
 status TEXT NOT NULL DEFAULT 'Pending', -- Pending | InProgress | Completed | Verified | Cancel
 completion_notes TEXT DEFAULT '',
 completion_date DATE,
 verified_by TEXT DEFAULT '',
 created_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)

```

```

);
ALTER TABLE quality_capa ENABLE ROW LEVEL SECURITY;
ALTER TABLE quality_capa FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_quality_capa_tenant_isolation ON quality_capa;
CREATE POLICY rls_quality_capa_tenant_isolation ON quality_capa
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_quality_capa_tenant_facility ON quality_capa (tenant_id, facility_id);
CREATE INDEX IF NOT EXISTS idx_quality_capa_incident ON quality_capa (incident_id);
CREATE INDEX IF NOT EXISTS idx_quality_capa_tenant_status ON quality_capa (tenant_id, status);

-- ---- Risk register: indicators / targets / trend, optionally linked to incident ----
CREATE TABLE IF NOT EXISTS quality_risk_register (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 incident_id INTEGER REFERENCES quality_incidents(id) ON DELETE SET NULL,
 risk_title TEXT NOT NULL DEFAULT '',
 category TEXT DEFAULT '',
 likelihood INTEGER NOT NULL DEFAULT 1, -- 1..5
 impact INTEGER NOT NULL DEFAULT 1, -- 1..5
 risk_score INTEGER NOT NULL DEFAULT 1, -- likelihood*impact (server-computed)
 risk_level TEXT NOT NULL DEFAULT 'Low', -- Low | Medium | High | Extreme (server-computed)
 control_measure TEXT DEFAULT '',
 residual_likelihood INTEGER,
 residual_impact INTEGER,
 residual_score INTEGER,
 owner_name TEXT DEFAULT '',
 review_date DATE,
 status TEXT NOT NULL DEFAULT 'Open', -- Open | Mitigating | Closed
 created_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
ALTER TABLE quality_risk_register ENABLE ROW LEVEL SECURITY;
ALTER TABLE quality_risk_register FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_quality_risk_register_tenant_isolation ON quality_risk_register;
CREATE POLICY rls_quality_risk_register_tenant_isolation ON quality_risk_register
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_quality_risk_tenant_facility ON quality_risk_register (tenant_id, facility_id);
CREATE INDEX IF NOT EXISTS idx_quality_risk_tenant_status ON quality_risk_register (tenant_id, status);

COMMIT;

```

```

=====
FILE: migrations\e17_001_quality_capa_validate.sql
=====

```

```

-- =====
-- Epic E17 ? Quality / CAPA / Risk migration VALIDATE.
-- Asserts FORCE RLS + canonical policy + tenant_id NOT NULL + FK(tenants) on the
-- 2 new tables, entity FK to quality_incidents, and the E17 incident columns.

```

```

-- Raises an exception on failure.
-- =====
DO $$
DECLARE
 tbls TEXT[] := ARRAY['quality_capa', 'quality_risk_register'];
 t TEXT;
 cols TEXT[] := ARRAY['harm_level', 'near_miss', 'confidential', 'encounter_id', 'visit_id', 'workflow_state'];
 c TEXT;
BEGIN
 FOREACH t IN ARRAY tbls LOOP
 IF to_regclass('public.' || t) IS NULL THEN
 RAISE EXCEPTION 'E17 validate: table % missing', t;
 END IF;
 IF NOT EXISTS (SELECT 1 FROM pg_class WHERE relname = t AND relrowsecurity AND relforcerowsecurity) TH
EN
 RAISE EXCEPTION 'E17 validate: FORCE RLS not enabled on %', t;
 END IF;
 IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE tablename = t AND policyname = 'rls_' || t || '_tenant_
isolation') THEN
 RAISE EXCEPTION 'E17 validate: tenant isolation policy missing on %', t;
 END IF;
 IF EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = t AND column_name = 'tenant_id'
AND is_nullable = 'YES') THEN
 RAISE EXCEPTION 'E17 validate: tenant_id is nullable on %', t;
 END IF;
 IF NOT EXISTS (
 SELECT 1 FROM pg_constraint c2
 JOIN pg_class rel ON rel.oid = c2.conrelid
 WHERE rel.relname = t AND c2.contype = 'f' AND c2.confrelid = 'tenants'::regclass
) THEN
 RAISE EXCEPTION 'E17 validate: FK to tenants(id) missing on %', t;
 END IF;
 -- entity FK: each new table must reference quality_incidents(id)
 IF NOT EXISTS (
 SELECT 1 FROM pg_constraint c2
 JOIN pg_class rel ON rel.oid = c2.conrelid
 WHERE rel.relname = t AND c2.contype = 'f' AND c2.confrelid = 'quality_incidents'::regclass
) THEN
 RAISE EXCEPTION 'E17 validate: FK to quality_incidents(id) missing on %', t;
 END IF;
 END LOOP;

 -- E17 incident-management columns present on quality_incidents
 FOREACH c IN ARRAY cols LOOP
 IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'quality_incidents' AND col
umn_name = c) THEN
 RAISE EXCEPTION 'E17 validate: quality_incidents.% missing', c;
 END IF;
 END LOOP;

 RAISE NOTICE 'E17 validate: OK ? quality_capa + quality_risk_register FORCE RLS + canonical policy + tenan
t_id NOT NULL + FK(tenants) + FK(quality_incidents); incident columns present';
END $$;

```

=====

FILE: migrations\e17\_002\_infection\_down.sql

=====

```
-- =====
-- Epic E17 ? Infection Control migration DOWN (reverse only E17 additions).
-- Drops ONLY the 2 NEW tables introduced by this epic. Does NOT touch any
-- pre-existing table (patients, infection_surveillance, etc.).
-- =====
```

```
BEGIN;
DROP TABLE IF EXISTS ams_flags CASCADE;
DROP TABLE IF EXISTS hai_isolation CASCADE;
COMMIT;
```

=====

FILE: migrations\e17\_002\_infection\_up.sql

=====

```
-- =====
-- Epic E17 ? Infection Control: HAI isolation tracking + antimicrobial stewardship
-- Candidate migration (UP). Idempotent. DO NOT auto-execute in production.
-- Adds 2 new tables (hai_isolation, ams_flags) with FORCE RLS + canonical tenant
-- policy. Both reference patients(id). The down migration drops only these tables.
-- =====
```

```
BEGIN;
```

```
-- ---- HAI isolation tracking (precaution start/stop per patient) ----
```

```
CREATE TABLE IF NOT EXISTS hai_isolation (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 patient_name TEXT DEFAULT '',
 surveillance_id INTEGER, -- optional link to infection_surveillance
 precaution_type TEXT NOT NULL DEFAULT 'contact', -- standard|contact|droplet|airborne|protective
 hai_category TEXT DEFAULT '', -- CLABSI|CAUTI|VAP|SSI|... (empty = community)
 organism TEXT DEFAULT '',
 ward TEXT DEFAULT '',
 bed TEXT DEFAULT '',
 effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 resolved_at TIMESTAMP,
 status TEXT NOT NULL DEFAULT 'Active', -- Active | Resolved | Discontinued
 notes TEXT DEFAULT '',
 created_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
ALTER TABLE hai_isolation ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE hai_isolation FORCE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_hai_isolation_tenant_isolation ON hai_isolation;
```

```
CREATE POLICY rls_hai_isolation_tenant_isolation ON hai_isolation
```

```

 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_hai_isolation_tenant_facility ON hai_isolation (tenant_id, facility_id);
CREATE INDEX IF NOT EXISTS idx_hai_isolation_patient ON hai_isolation (patient_id);
CREATE INDEX IF NOT EXISTS idx_hai_isolation_tenant_status ON hai_isolation (tenant_id, status);

-- ---- Antimicrobial stewardship flags (review of antibiotic orders) ----
CREATE TABLE IF NOT EXISTS ams_flags (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 patient_name TEXT DEFAULT '',
 antibiotic TEXT NOT NULL DEFAULT '',
 flag_reason TEXT NOT NULL DEFAULT '', -- restricted|de-escalation|duration|duplicate|IV-to-P
0
 severity TEXT NOT NULL DEFAULT 'Advisory', -- Advisory | Action Required | Critical
 flagged_by TEXT DEFAULT '',
 reviewed_by TEXT DEFAULT '',
 reviewed_at TIMESTAMP,
 review_outcome TEXT DEFAULT '', -- approved|modified|stopped
 status TEXT NOT NULL DEFAULT 'Open', -- Open | Reviewed | Closed
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
ALTER TABLE ams_flags ENABLE ROW LEVEL SECURITY;
ALTER TABLE ams_flags FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_ams_flags_tenant_isolation ON ams_flags;
CREATE POLICY rls_ams_flags_tenant_isolation ON ams_flags
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
CREATE INDEX IF NOT EXISTS idx_ams_flags_tenant_facility ON ams_flags (tenant_id, facility_id);
CREATE INDEX IF NOT EXISTS idx_ams_flags_patient ON ams_flags (patient_id);
CREATE INDEX IF NOT EXISTS idx_ams_flags_tenant_status ON ams_flags (tenant_id, status);

COMMIT;

=====
FILE: migrations\e17_002_infection_validate.sql
=====

-- =====
-- Epic E17 ? Infection Control migration VALIDATE.
-- Asserts FORCE RLS + canonical policy + tenant_id NOT NULL + FK(tenants) and
-- entity FK to patients(id) on hai_isolation and ams_flags.
-- Raises an exception on failure.
-- =====

DO $$
DECLARE
 tbls TEXT[] := ARRAY['hai_isolation', 'ams_flags'];
 t TEXT;
BEGIN

```

```

FOREACH t IN ARRAY tbls LOOP
 IF to_regclass('public.' || t) IS NULL THEN
 RAISE EXCEPTION 'E17 validate: table % missing', t;
 END IF;
 IF NOT EXISTS (SELECT 1 FROM pg_class WHERE relname = t AND relrowsecurity AND relforcerowsecurity) TH
EN
 RAISE EXCEPTION 'E17 validate: FORCE RLS not enabled on %', t;
 END IF;
 IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE tablename = t AND policyname = 'rls_' || t || '_tenant_
isolation') THEN
 RAISE EXCEPTION 'E17 validate: tenant isolation policy missing on %', t;
 END IF;
 IF EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = t AND column_name = 'tenant_id'
AND is_nullable = 'YES') THEN
 RAISE EXCEPTION 'E17 validate: tenant_id is nullable on %', t;
 END IF;
 IF NOT EXISTS (
 SELECT 1 FROM pg_constraint c2
 JOIN pg_class rel ON rel.oid = c2.conrelid
 WHERE rel.relname = t AND c2.contype = 'f' AND c2.confrelid = 'tenants'::regclass
) THEN
 RAISE EXCEPTION 'E17 validate: FK to tenants(id) missing on %', t;
 END IF;
 IF NOT EXISTS (
 SELECT 1 FROM pg_constraint c2
 JOIN pg_class rel ON rel.oid = c2.conrelid
 WHERE rel.relname = t AND c2.contype = 'f' AND c2.confrelid = 'patients'::regclass
) THEN
 RAISE EXCEPTION 'E17 validate: FK to patients(id) missing on %', t;
 END IF;
END LOOP;
RAISE NOTICE 'E17 validate: OK ? hai_isolation + ams_flags FORCE RLS + canonical policy + tenant_id NOT NU
LL + FK(tenants) + FK(patients)';
END $$;

```

```

=====
FILE: migrations\e18_01_hr_workforce_down.sql
=====

```

```

-- e18_01_hr_workforce_down.sql (rollback of e18_01 up)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ?? ??? ??????? ?????? ??????? e18_01 ? ??????? ??? (???????? ??? ?????? ??? ??? FK ?????).
-- ?? ??? ??????? ????????? ?????? (hr_employees/hr_leaves/hr_attendance/hr_salaries) ???????.
-- ?? DROP TABLE ?????? ???????/??????/????? ??????. idempotent (IF EXISTS).
BEGIN;

DROP TABLE IF EXISTS hr_competencies CASCADE;
DROP TABLE IF EXISTS hr_payroll_slips CASCADE;
DROP TABLE IF EXISTS hr_leave_requests CASCADE;
DROP TABLE IF EXISTS hr_shifts CASCADE;
DROP TABLE IF EXISTS hr_licenses CASCADE;

```

COMMIT;

=====

FILE: migrations\e18\_01\_hr\_workforce\_up.sql

=====

-- =====

-- e18\_01\_hr\_workforce\_up.sql

-- E18 ? HR / Workforce: NEW tables only

-- hr\_licenses (SCFHS / professional-license tracking + server-side expiry alerts)

-- hr\_shifts (shift roster / scheduling, overlap-checked server-side)

-- hr\_leave\_requests (leave state machine: requested -> approved | denied | cancelled)

-- hr\_payroll\_slips (computed DRAFT payroll slips; GL posting GATED OFF by flag)

-- hr\_competencies (CME hours / SCFHS competency compliance)

-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).

--

-- ????? ????? ????? => tenant\_id INTEGER NOT NULL REFERENCES tenants(id) ??? ?????? + FORCE RLS

-- ?????? ?????? + FK ?????? (hr\_employees). NOT ?????? ??? bootstrap ?? db\_postgres.js.

-- idempotent: CREATE TABLE/INDEX IF NOT EXISTS + DROP/CREATE POLICY IF EXISTS. BEGIN;?COMMIT;

--

-- ??????: hr\_employees/hr\_leaves/hr\_attendance/hr\_salaries ?????? ?????? ?????? (bootstrap) ???

-- ?????? ???. ?? ???? E18 ?????? ?????? ??? ?????? ?????? ???.

-- =====

BEGIN;

-- ===== hr\_licenses: SCFHS / professional / document license tracking =====

CREATE TABLE IF NOT EXISTS hr\_licenses (

id SERIAL PRIMARY KEY,

employee\_id INTEGER NOT NULL REFERENCES hr\_employees(id) ON DELETE CASCADE,

license\_type TEXT NOT NULL DEFAULT 'SCFHS',

license\_number TEXT DEFAULT '',

authority TEXT DEFAULT 'SCFHS',

issue\_date DATE,

expiry\_date DATE,

alert\_days INTEGER NOT NULL DEFAULT 30,

status TEXT NOT NULL DEFAULT 'active',

notes TEXT DEFAULT '',

created\_by TEXT DEFAULT '',

created\_at TIMESTAMP DEFAULT now(),

tenant\_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,

facility\_id INTEGER,

CONSTRAINT chk\_hr\_lic\_status CHECK (status IN ('active','suspended','revoked','expired')),

CONSTRAINT chk\_hr\_lic\_alert\_nonneg CHECK (alert\_days >= 0)

);

CREATE INDEX IF NOT EXISTS idx\_hr\_lic\_tenant\_id ON hr\_licenses (tenant\_id);

CREATE INDEX IF NOT EXISTS idx\_hr\_lic\_emp ON hr\_licenses (tenant\_id, employee\_id);

CREATE INDEX IF NOT EXISTS idx\_hr\_lic\_expiry ON hr\_licenses (tenant\_id, expiry\_date);

ALTER TABLE hr\_licenses ENABLE ROW LEVEL SECURITY;

ALTER TABLE hr\_licenses FORCE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS rls\_hr\_lic\_tenant\_isolation ON hr\_licenses;

CREATE POLICY rls\_hr\_lic\_tenant\_isolation ON hr\_licenses



```

FOR ALL
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== hr_shifts: shift roster / scheduling =====
CREATE TABLE IF NOT EXISTS hr_shifts (
 id SERIAL PRIMARY KEY,
 employee_id INTEGER NOT NULL REFERENCES hr_employees(id) ON DELETE CASCADE,
 shift_date DATE NOT NULL,
 shift_name TEXT DEFAULT '',
 start_time TEXT NOT NULL DEFAULT '',
 end_time TEXT NOT NULL DEFAULT '',
 department TEXT DEFAULT '',
 status TEXT NOT NULL DEFAULT 'scheduled',
 notes TEXT DEFAULT '',
 created_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT now(),
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 branch_id INTEGER,
 CONSTRAINT chk_hr_shift_status CHECK (status IN ('scheduled','completed','cancelled'))
);
CREATE INDEX IF NOT EXISTS idx_hr_shift_tenant_id ON hr_shifts (tenant_id);
CREATE INDEX IF NOT EXISTS idx_hr_shift_emp_date ON hr_shifts (tenant_id, employee_id, shift_date);
ALTER TABLE hr_shifts ENABLE ROW LEVEL SECURITY;
ALTER TABLE hr_shifts FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_hr_shift_tenant_isolation ON hr_shifts;
CREATE POLICY rls_hr_shift_tenant_isolation ON hr_shifts
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== hr_leave_requests: state machine requested -> approved | denied | cancelled =====
CREATE TABLE IF NOT EXISTS hr_leave_requests (
 id SERIAL PRIMARY KEY,
 employee_id INTEGER NOT NULL REFERENCES hr_employees(id) ON DELETE CASCADE,
 leave_type TEXT NOT NULL DEFAULT 'Annual',
 start_date DATE NOT NULL,
 end_date DATE NOT NULL,
 days INTEGER NOT NULL DEFAULT 0,
 status TEXT NOT NULL DEFAULT 'requested',
 reason TEXT DEFAULT '',
 requested_by TEXT DEFAULT '',
 approved_by TEXT DEFAULT '',
 approved_at TIMESTAMP,
 denial_reason TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT now(),
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 CONSTRAINT chk_hr_leave_status CHECK (status IN ('requested','approved','denied','cancelled')),
 CONSTRAINT chk_hr_leave_days_pos CHECK (days > 0)
);
CREATE INDEX IF NOT EXISTS idx_hr_leave_req_tenant_id ON hr_leave_requests (tenant_id);
CREATE INDEX IF NOT EXISTS idx_hr_leave_req_emp ON hr_leave_requests (tenant_id, employee_id);

```

```

ALTER TABLE hr_leave_requests ENABLE ROW LEVEL SECURITY;
ALTER TABLE hr_leave_requests FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_hr_leave_req_tenant_isolation ON hr_leave_requests;
CREATE POLICY rls_hr_leave_req_tenant_isolation ON hr_leave_requests
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== hr_payroll_slips: computed DRAFT slips. GL posting GATED OFF by flag =====
CREATE TABLE IF NOT EXISTS hr_payroll_slips (
 id SERIAL PRIMARY KEY,
 employee_id INTEGER NOT NULL REFERENCES hr_employees(id) ON DELETE CASCADE,
 pay_month TEXT NOT NULL DEFAULT '',
 basic REAL NOT NULL DEFAULT 0,
 housing_allowance REAL NOT NULL DEFAULT 0,
 transport_allowance REAL NOT NULL DEFAULT 0,
 other_allowances REAL NOT NULL DEFAULT 0,
 gross_earnings REAL NOT NULL DEFAULT 0,
 gosi_deduction REAL NOT NULL DEFAULT 0,
 advances_deducted REAL NOT NULL DEFAULT 0,
 other_deductions REAL NOT NULL DEFAULT 0,
 total_deductions REAL NOT NULL DEFAULT 0,
 net_salary REAL NOT NULL DEFAULT 0,
 status TEXT NOT NULL DEFAULT 'draft',
 posted INTEGER NOT NULL DEFAULT 0,
 posted_at TIMESTAMP,
 journal_entry_id INTEGER,
 created_by TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT now(),
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 CONSTRAINT chk_hr_slip_status CHECK (status IN ('draft','approved','posted','cancelled')),
 CONSTRAINT chk_hr_slip_net_nonneg CHECK (net_salary >= 0),
 CONSTRAINT chk_hr_slip_posted_flag CHECK (posted IN (0,1)),
 CONSTRAINT uq_hr_slip_emp_month UNIQUE (tenant_id, employee_id, pay_month)
);

CREATE INDEX IF NOT EXISTS idx_hr_slip_tenant_id ON hr_payroll_slips (tenant_id);
CREATE INDEX IF NOT EXISTS idx_hr_slip_emp_month ON hr_payroll_slips (tenant_id, employee_id, pay_month);
ALTER TABLE hr_payroll_slips ENABLE ROW LEVEL SECURITY;
ALTER TABLE hr_payroll_slips FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_hr_slip_tenant_isolation ON hr_payroll_slips;
CREATE POLICY rls_hr_slip_tenant_isolation ON hr_payroll_slips
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ===== hr_competencies: CME hours / SCFHS competency compliance =====
CREATE TABLE IF NOT EXISTS hr_competencies (
 id SERIAL PRIMARY KEY,
 employee_id INTEGER NOT NULL REFERENCES hr_employees(id) ON DELETE CASCADE,
 competency_name TEXT NOT NULL DEFAULT '',
 cme_hours REAL NOT NULL DEFAULT 0,
 required_hours REAL NOT NULL DEFAULT 0,
 period_start DATE,

```

```

period_end DATE,
status TEXT NOT NULL DEFAULT 'in_progress',
notes TEXT DEFAULT '',
created_by TEXT DEFAULT '',
created_at TIMESTAMP DEFAULT now(),
tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
facility_id INTEGER,
CONSTRAINT chk_hr_comp_status CHECK (status IN ('in_progress','compliant','non_compliant')),
CONSTRAINT chk_hr_comp_hours_nonneg CHECK (cme_hours >= 0 AND required_hours >= 0)
);

CREATE INDEX IF NOT EXISTS idx_hr_comp_tenant_id ON hr_competencies (tenant_id);
CREATE INDEX IF NOT EXISTS idx_hr_comp_emp ON hr_competencies (tenant_id, employee_id);
ALTER TABLE hr_competencies ENABLE ROW LEVEL SECURITY;
ALTER TABLE hr_competencies FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_hr_comp_tenant_isolation ON hr_competencies;
CREATE POLICY rls_hr_comp_tenant_isolation ON hr_competencies
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

=====

FILE: migrations\e18\_01\_hr\_workforce\_validate.sql

=====

```

-- e18_01_hr_workforce_validate.sql (run AFTER e18_01 up; read-only)
-- PASS = all five NEW tables exist with tenant_id NOT NULL + FK->tenants + FK->hr_employees
-- + FORCE RLS + canonical isolation policy + key CHECK constraints.
SELECT
 -- hr_licenses
 (SELECT count(*)::int FROM information_schema.tables WHERE table_name='hr_licenses') AS lic_exist
s, -- 1
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='hr_licenses') AS lic_force
_rls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='hr_licenses'
 AND policyname='rls_hr_lic_tenant_isolation') AS lic_polic
y, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='hr_licenses'
 AND column_name='tenant_id' AND is_nullable='NO') AS lic_tenan
t_nn, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='hr_licenses'::regclass
 AND confrelid='tenants'::regclass AND contype='f') AS lic_tenan
t_fk, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='hr_licenses'::regclass
 AND confrelid='hr_employees'::regclass AND contype='f') AS lic_emp_f
k, -- 1
 -- hr_shifts
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='hr_shifts') AS shift_for
ce_rls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='hr_shifts'
 AND policyname='rls_hr_shift_tenant_isolation') AS shift_pol

```

```

icy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='hr_shifts'
 AND column_name='tenant_id' AND is_nullable='NO')
ant_nn, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conname='chk_hr_shift_status')
tus_chk, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='hr_shifts'::regclass
 AND confrelid='hr_employees'::regclass AND contype='f')
_fk, -- 1 (L3)
 -- hr_leave_requests
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='hr_leave_requests')
ce_rls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='hr_leave_requests'
 AND policyname='rls_hr_leave_req_tenant_isolation')
icy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='hr_leave_requests'
 AND column_name='tenant_id' AND is_nullable='NO')
ant_nn, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conname='chk_hr_leave_status')
tus_chk, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='hr_leave_requests'::regclass
 AND confrelid='hr_employees'::regclass AND contype='f')
_fk, -- 1
 -- hr_payroll_slips
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='hr_payroll_slips')
e_rls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='hr_payroll_slips'
 AND policyname='rls_hr_slip_tenant_isolation')
cy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='hr_payroll_slips'
 AND column_name='tenant_id' AND is_nullable='NO')
nt_nn, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conname='chk_hr_slip_net_nonneg')
chk, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conname='uq_hr_slip_emp_month')
 -- 1
 -- hr_competencies
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='hr_competencies')
e_rls, -- t
 (SELECT count(*) FROM pg_policies WHERE tablename='hr_competencies'
 AND policyname='rls_hr_comp_tenant_isolation')
cy, -- 1
 (SELECT count(*)::int FROM information_schema.columns WHERE table_name='hr_competencies'
 AND column_name='tenant_id' AND is_nullable='NO')
nt_nn, -- 1
 (SELECT count(*)::int FROM pg_constraint WHERE conrelid='hr_competencies'::regclass
 AND confrelid='hr_employees'::regclass AND contype='f')
fk; -- 1 (L3)

```

```

=====
FILE: migrations\e1_01_problems_down.sql
=====

```

```

-- e1_01_problems_down.sql (rollback of e1_01_problems_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ????? ????? ?? ???? problems. idempotent (IF EXISTS).
BEGIN;

DROP POLICY IF EXISTS rls_problems_tenant_isolation ON problems;
ALTER TABLE IF EXISTS problems DROP CONSTRAINT IF EXISTS chk_problems_status;
DROP TABLE IF EXISTS problems;

COMMIT;

=====
FILE: migrations\e1_01_problems_up.sql
=====

-- =====
-- e1_01_problems_up.sql
-- E1 DOCTOR STATION ? Problem List (coded diagnoses).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? ??????? ?? ?? ?? (medical_records.icd10_codes / medical_records_coding.primary_icd10)
-- ??? ????? ????? ??????? (problems). ????? ??? patient_id ??? ??? ??????? encounter_ref ??? ?? NULL
-- (?? ??? encounter_ref ??????? visit_lifecycle) ??????? orders.encounter_id.
-- ??? ??? 150 ????? FORCE RLS: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS
-- + ????? ??? tenant_id + ??? tenant_id.
--
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS
-- + DROP/ADD CONSTRAINT IF EXISTS ??? status CHECK (????? ??????? ???????).
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS problems (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 patient_id INTEGER NOT NULL,
 encounter_ref INTEGER,
 s.encounter_id)
 icd10 TEXT,
 snomed TEXT,
 description TEXT NOT NULL,
 status TEXT NOT NULL DEFAULT 'active',
 onset_date DATE,
 recorded_by INTEGER,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT chk_problems_status CHECK (status IN ('active', 'resolved'))
);

-- idempotent: ensure the status CHECK exists even when the table pre-dates this migration
-- (CREATE TABLE IF NOT EXISTS skips the inline constraint on an existing table). DROP then ADD.
ALTER TABLE problems DROP CONSTRAINT IF EXISTS chk_problems_status;

```

```
ALTER TABLE problems ADD CONSTRAINT chk_problems_status CHECK (status IN ('active', 'resolved'));
```

```
CREATE INDEX IF NOT EXISTS idx_problems_tenant_id ON problems (tenant_id);
```

```
CREATE INDEX IF NOT EXISTS idx_problems_patient_id ON problems (patient_id);
```

```
CREATE INDEX IF NOT EXISTS idx_problems_encounter_ref ON problems (encounter_ref);
```

```
ALTER TABLE problems ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE problems FORCE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_problems_tenant_isolation ON problems;
```

```
CREATE POLICY rls_problems_tenant_isolation ON problems
```

```
FOR ALL
```

```
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
```

```
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e1_01_problems_validate.sql
```

```
=====
```

```
-- e1_01_problems_validate.sql (run AFTER e1_01_problems_up.sql; read-only)
```

```
-- PASS = problems exists with tenant_id FK + FORCE RLS + isolation policy + indexes + status CHECK.
```

```
SELECT
```

```
(SELECT count(*) FROM information_schema.tables WHERE table_name='problems') AS problems_exists,
```

```
-- expect 1
```

```
(SELECT count(*) FROM information_schema.columns
```

```
WHERE table_name='problems'
```

```
AND column_name IN ('tenant_id','patient_id','encounter_ref','icd10','snomed','description','status','onset_date','recorded_by')) AS problems_cols, -- expect 9
```

```
(SELECT relforcerowsecurity FROM pg_class WHERE relname='problems') AS problems_force_rls,
```

```
-- expect t
```

```
(SELECT count(*) FROM pg_policies WHERE tablename='problems' AND policyname='rls_problems_tenant_isolation')
```

```
AS problems_policy, -- expect 1
```

```
(SELECT count(*) FROM pg_constraint WHERE conname='chk_problems_status') AS problems_status_check,
```

```
-- expect 1
```

```
(SELECT count(*) FROM pg_indexes WHERE tablename='problems' AND indexname='idx_problems_tenant_id') AS problems_idx; -- expect 1
```

```
-- ----- tenant_id is NOT NULL -----
```

```
SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
```

```
WHERE table_name='problems' AND column_name='tenant_id' AND is_nullable='NO';
```

```
-- expect 1
```

```
-- ----- FK to tenants(id) -----
```

```
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
```

```
WHERE conrelid='problems'::regclass AND confrelid='tenants'::regclass AND contype='f';
```

```
-- expect 1
```

```
=====
```

FILE: migrations\e1\_02\_clinical\_notes\_down.sql

=====

```
-- e1_02_clinical_notes_down.sql (rollback of e1_02_clinical_notes_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ????? ????? ?? ???? clinical_notes. idempotent (IF EXISTS).
```

BEGIN;

```
DROP POLICY IF EXISTS rls_clinical_notes_tenant_isolation ON clinical_notes;
ALTER TABLE IF EXISTS clinical_notes DROP CONSTRAINT IF EXISTS chk_clinical_notes_type;
ALTER TABLE IF EXISTS clinical_notes DROP CONSTRAINT IF EXISTS chk_clinical_notes_status;
DROP TABLE IF EXISTS clinical_notes;
```

COMMIT;

=====

FILE: migrations\e1\_02\_clinical\_notes\_up.sql

=====

-- =====

```
-- e1_02_clinical_notes_up.sql
-- E1 DOCTOR STATION ? Clinical Notes (SOAP, sign+lock).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ?????? ?????? ?????? SOAP (subjective/objective/assessment/plan) ?????? ??
-- medical_records ???????. ?????? ??? patient_id? encounter_ref ???? ??? NULL (????? orders.encounter_id).
-- ?????/??? ????? ??? medical_records (emr_status/signed_by/signed_at/integrity_hash) ???????? emr_amendment
S.
-- ??? ???? ??? 150 ????? FORCE RLS: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS
-- + ????? ??? tenant_id + ???? tenant_id.
--
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS
-- + DROP/ADD CONSTRAINT IF EXISTS ??? CHECKS (????? ???????? ????????).
-- =====
```

BEGIN;

```
CREATE TABLE IF NOT EXISTS clinical_notes (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 patient_id INTEGER NOT NULL,
 encounter_ref INTEGER, -- nullable: no encounters table yet (mirrors order
s.encounter_id)
 type TEXT NOT NULL DEFAULT 'SOAP',
 subjective TEXT,
 objective TEXT,
 assessment TEXT,
 plan TEXT,
 author_id INTEGER,
 emr_status TEXT NOT NULL DEFAULT 'draft', -- draft | locked (mirrors medical_records sign/loc
k)
 signed_by_user_id INTEGER,
```

```

signed_at TIMESTAMP,
locked_at TIMESTAMP,
integrity_hash TEXT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT chk_clinical_notes_type CHECK (type IN ('SOAP')),
CONSTRAINT chk_clinical_notes_status CHECK (emr_status IN ('draft', 'locked'))
);

-- idempotent: ensure CHECKs exist even when the table pre-dates this migration. DROP then ADD.
ALTER TABLE clinical_notes DROP CONSTRAINT IF EXISTS chk_clinical_notes_type;
ALTER TABLE clinical_notes ADD CONSTRAINT chk_clinical_notes_type CHECK (type IN ('SOAP'));
ALTER TABLE clinical_notes DROP CONSTRAINT IF EXISTS chk_clinical_notes_status;
ALTER TABLE clinical_notes ADD CONSTRAINT chk_clinical_notes_status CHECK (emr_status IN ('draft', 'locked'));

CREATE INDEX IF NOT EXISTS idx_clinical_notes_tenant_id ON clinical_notes (tenant_id);
CREATE INDEX IF NOT EXISTS idx_clinical_notes_patient_id ON clinical_notes (patient_id);
CREATE INDEX IF NOT EXISTS idx_clinical_notes_encounter_ref ON clinical_notes (encounter_ref);

ALTER TABLE clinical_notes ENABLE ROW LEVEL SECURITY;
ALTER TABLE clinical_notes FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_clinical_notes_tenant_isolation ON clinical_notes;
CREATE POLICY rls_clinical_notes_tenant_isolation ON clinical_notes
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

```

=====
FILE: migrations\e1_02_clinical_notes_validate.sql
=====

```

```

-- e1_02_clinical_notes_validate.sql (run AFTER e1_02_clinical_notes_up.sql; read-only)
-- PASS = clinical_notes exists with tenant_id FK + FORCE RLS + isolation policy + indexes + CHECKs + sign/lock cols.

```

```

SELECT
 (SELECT count(*) FROM information_schema.tables WHERE table_name='clinical_notes') AS notes_exists,
 -- expect 1
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='clinical_notes'
 AND column_name IN ('tenant_id','patient_id','encounter_ref','type','subjective','objective','assessment','plan','author_id','signed_at')) AS notes_cols, -- expect 10
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='clinical_notes'
 AND column_name IN ('emr_status','signed_by_user_id','locked_at','integrity_hash')) AS notes_lock_cols,
 -- expect 4
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='clinical_notes') AS notes_force_rls,
 -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='clinical_notes' AND policyname='rls_clinical_notes_tenant_isolation') AS notes_policy, -- expect 1
 (SELECT count(*) FROM pg_constraint WHERE conname='chk_clinical_notes_type') AS notes_type_check,

```



```

-- expect 1
(SELECT count(*) FROM pg_constraint WHERE conname='chk_clinical_notes_status') AS notes_status_check,
-- expect 1
(SELECT count(*) FROM pg_indexes WHERE tablename='clinical_notes' AND indexname='idx_clinical_notes_tenant_id') AS notes_idx; -- expect 1

-- ----- tenant_id is NOT NULL -----
SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
WHERE table_name='clinical_notes' AND column_name='tenant_id' AND is_nullable='NO';
-- expect 1

-- ----- FK to tenants(id) -----
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
WHERE conrelid='clinical_notes'::regclass AND confrelid='tenants'::regclass AND contype='f';
-- expect 1

=====
FILE: migrations\e21_001_cardiology_specialties_down.sql
=====

-- Migration: e21_001_cardiology_specialties_down.sql
-- Description: Rollback specialized tables for Cardiology department.

BEGIN;

DROP TABLE IF EXISTS cardio_meds_tracking;
DROP TABLE IF EXISTS cardio_cath_logs;
DROP TABLE IF EXISTS cardio_echo_reports;
DROP TABLE IF EXISTS cardio_patient_profiles;

COMMIT;

=====
FILE: migrations\e21_001_cardiology_specialties_up.sql
=====

-- Migration: e21_001_cardiology_specialties_up.sql
-- Description: Create specialized tables for Cardiology department with RLS support.

BEGIN;

-- 1. Cardiology Patient Profiles (Specialized clinical data)
CREATE TABLE IF NOT EXISTS cardio_patient_profiles (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL,
 tenant_id UUID NOT NULL,
 risk_score DECIMAL(5,2),
 chronic_conditions JSONB DEFAULT '[]',
 last_screening_date DATE,
 is_high_risk BOOLEAN DEFAULT FALSE,

```

```

 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_patient FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

-- 2. Echocardiogram Reports (Echo)
CREATE TABLE IF NOT EXISTS cardio_echo_reports (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL,
 tenant_id UUID NOT NULL,
 ef_percentage DECIMAL(5,2), -- Ejection Fraction
 valve_status JSONB DEFAULT '{}', -- Mitral, Aortic, etc.
 wall_motion_abnormality TEXT,
 chamber_sizes JSONB DEFAULT '{}',
 report_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 physician_id UUID,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_patient_echo FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

-- 3. Cardiac Catheterization Logs (Cath Lab)
CREATE TABLE IF NOT EXISTS cardio_cath_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL,
 tenant_id UUID NOT NULL,
 procedure_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 vessel_name VARCHAR(100),
 stenosis_percentage DECIMAL(5,2),
 stent_type VARCHAR(100),
 stent_diameter DECIMAL(5,2),
 pressure_gradient DECIMAL(5,2),
 complications TEXT,
 physician_id UUID,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_patient_cath FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

-- 4. Cardiology Medication Tracking (High-precision)
CREATE TABLE IF NOT EXISTS cardio_meds_tracking (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL,
 tenant_id UUID NOT NULL,
 drug_name VARCHAR(255) NOT NULL,
 dosage VARCHAR(100),
 frequency VARCHAR(100),
 start_date DATE,
 end_date DATE,
 effect_observed TEXT,
 is_active BOOLEAN DEFAULT TRUE,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_patient_meds FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

-- Enable RLS for all new tables

```

```

ALTER TABLE cardio_patient_profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE cardio_echo_reports ENABLE ROW LEVEL SECURITY;
ALTER TABLE cardio_cath_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE cardio_meds_tracking ENABLE ROW LEVEL SECURITY;

-- Create RLS Policies (Standard tenant isolation)
CREATE POLICY tenant_isolation_policy ON cardio_patient_profiles USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
CREATE POLICY tenant_isolation_policy ON cardio_echo_reports USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
CREATE POLICY tenant_isolation_policy ON cardio_cath_logs USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
CREATE POLICY tenant_isolation_policy ON cardio_meds_tracking USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

```

```

COMMIT;

```

```

=====
FILE: migrations\e21_02_smart_templates_up.sql
=====

```

```

-- e21_02_smart_templates_up.sql
-- Migration to add clinical_smart_templates table for Dot Phrases (Phase F2)

```

```

CREATE TABLE IF NOT EXISTS clinical_smart_templates (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 doctor_id INTEGER NOT NULL REFERENCES system_users(id) ON DELETE CASCADE,
 shortcut VARCHAR(50) NOT NULL,
 template_text TEXT NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT unique_doctor_shortcut UNIQUE (tenant_id, doctor_id, shortcut)
);

```

```

-- Enable Row Level Security
ALTER TABLE clinical_smart_templates ENABLE ROW LEVEL SECURITY;
ALTER TABLE clinical_smart_templates FORCE ROW LEVEL SECURITY;

```

```

-- Create Tenant isolation policy
DROP POLICY IF EXISTS rls_clinical_smart_templates ON clinical_smart_templates;
CREATE POLICY rls_clinical_smart_templates ON clinical_smart_templates
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

```

```

-- Grant privileges to nama_medical_app
GRANT ALL PRIVILEGES ON TABLE clinical_smart_templates TO nama_medical_app;

```

```

=====
FILE: migrations\e21_03_icu_bundles_up.sql
=====

```

```

-- e21_03_icu_bundles_up.sql
-- Migration to add icu_prevention_bundles table for Infection Control (Phase F3)

CREATE TABLE IF NOT EXISTS icu_prevention_bundles (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 admission_id INTEGER NOT NULL REFERENCES admissions(id) ON DELETE CASCADE,
 bundle_type VARCHAR(50) NOT NULL, -- 'VAP' | 'CLABSI' | 'CAUTI'
 audit_date DATE NOT NULL,
 checked_items JSONB NOT NULL, -- e.g. {"head_elevation": true, "dvt_prophylaxis": false}
 compliance_rate DECIMAL(5,2) NOT NULL, -- e.g. 80.00
 non_compliance_reason TEXT, -- mandatory if compliance_rate < 100
 recorded_by INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT unique_admission_bundle_date UNIQUE (tenant_id, admission_id, bundle_type, audit_date),
 CONSTRAINT chk_bundle_type CHECK (bundle_type IN ('VAP', 'CLABSI', 'CAUTI'))
);

-- Enable Row Level Security
ALTER TABLE icu_prevention_bundles ENABLE ROW LEVEL SECURITY;
ALTER TABLE icu_prevention_bundles FORCE ROW LEVEL SECURITY;

-- Create Tenant isolation policy
DROP POLICY IF EXISTS rls_icu_prevention_bundles ON icu_prevention_bundles;
CREATE POLICY rls_icu_prevention_bundles ON icu_prevention_bundles
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Grant privileges to nama_medical_app
GRANT ALL PRIVILEGES ON TABLE icu_prevention_bundles TO nama_medical_app;

=====
FILE: migrations\e21_clinical_specialties_up.sql
=====

-- e21_clinical_specialties_up.sql
-- Migrations for Metadata-Driven EMR & Clinical Subspecialties

-- 1. Clinical Departments (supports 100+ specialties)
CREATE TABLE IF NOT EXISTS clinical_departments (
 id SERIAL PRIMARY KEY,
 code VARCHAR(50) UNIQUE NOT NULL, -- e.g., 'CARDIOLOGY', 'PEDIATRICS_NICU'
 name_en VARCHAR(150) NOT NULL,
 name_ar VARCHAR(150) NOT NULL,
 category VARCHAR(100) NOT NULL, -- Internal Medicine, Surgical, Pediatrics, etc.
 is_active BOOLEAN DEFAULT TRUE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Clinical Templates for each specialty (form structures in JSONB)

```

```

CREATE TABLE IF NOT EXISTS clinical_templates (
 id SERIAL PRIMARY KEY,
 department_id INTEGER NOT NULL REFERENCES clinical_departments(id) ON DELETE CASCADE,
 template_name_en VARCHAR(150) NOT NULL,
 template_name_ar VARCHAR(150) NOT NULL,
 form_structure JSONB NOT NULL, -- Describes fields: { fields: [...] }
 is_active BOOLEAN DEFAULT TRUE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Patient Clinical Records (locked and signed EMR records)
CREATE TABLE IF NOT EXISTS patient_clinical_records (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 encounter_id INTEGER, -- Visit / admission ID
 template_id INTEGER NOT NULL REFERENCES clinical_templates(id),
 recorded_values JSONB NOT NULL, -- Form input values: { 'bp': 120, 'hr': 80 }
 doctor_id INTEGER NOT NULL REFERENCES system_users(id),
 is_locked BOOLEAN DEFAULT FALSE,
 signature VARCHAR(256),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL
);

-- 4. Clinical Knowledge Vectors (for RAG/LangChain)
-- Using REAL[] as a highly portable vector fallback to ensure out-of-the-box compatibility
CREATE TABLE IF NOT EXISTS clinical_knowledge_vectors (
 id SERIAL PRIMARY KEY,
 department_id INTEGER REFERENCES clinical_departments(id) ON DELETE CASCADE,
 content_chunk TEXT NOT NULL,
 embedding REAL[], -- 1536-dimensional OpenAI embeddings array
 metadata JSONB,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexing for fast search and tenant isolation
CREATE INDEX IF NOT EXISTS idx_patient_clinical_records_patient ON patient_clinical_records(patient_id);
CREATE INDEX IF NOT EXISTS idx_patient_clinical_records_tenant ON patient_clinical_records(tenant_id);
CREATE INDEX IF NOT EXISTS idx_clinical_templates_dept ON clinical_templates(department_id);

=====
FILE: migrations\e22_01_operational_money_numeric_down.sql
=====

-- e22_01_operational_money_numeric_down.sql
-- Reverse of e22_01: restore operational money columns to REAL (double precision).
-- NOTE: down-migration is provided for reversibility discipline; reverting NUMERIC->REAL re-introduces
-- floating drift and is NOT recommended except for emergency rollback. OWNER-RUN ONLY.

BEGIN;

DO $$

```

```

DECLARE
 t record;
 targets text[][] := ARRAY[
 ['invoices','total'], ['invoices','amount'], ['invoices','vat_amount'],
 ['pharmacy_sales','total_amount'], ['pharmacy_sales','discount'],
 ['pharmacy_sales','insurance_coverage'], ['pharmacy_sales','patient_share'],
 ['finance_vouchers','amount'],
 ['hr_salaries','basic'], ['hr_salaries','allowances'], ['hr_salaries','deductions'],
 ['hr_salaries','advances_deducted'], ['hr_salaries','net_salary']
];
 i int;
BEGIN
 FOR i IN 1 .. array_length(targets, 1) LOOP
 FOR t IN
 SELECT table_name, column_name
 FROM information_schema.columns
 WHERE table_schema = 'public'
 AND table_name = targets[i][1]
 AND column_name = targets[i][2]
 AND data_type = 'numeric'
 LOOP
 EXECUTE format(
 'ALTER TABLE public.%I ALTER COLUMN %I TYPE REAL USING %I::real',
 t.table_name, t.column_name, t.column_name
);
 END LOOP;
 END LOOP;
END $$;

COMMIT;

```

```

=====
FILE: migrations\e22_01_operational_money_numeric_up.sql
=====

```

```

-- e22_01_operational_money_numeric_up.sql
-- REMEDIATION (GATE4-M1): convert operational money columns from REAL (floating point) to NUMERIC(14,2).
-- Context: the GL (finance_journal_lines) and insurance_claims were already migrated to NUMERIC, but
-- operational money columns (invoices, pharmacy_sales, finance_vouchers, hr_salaries) remained REAL,
-- which causes rounding drift on SUM()/aggregation in financial reports.
-- SAFETY: idempotent + defensive ? only alters a column that (a) exists and (b) is still a floating type.
-- Values are ROUND()ed to 2dp during the cast (no silent truncation). Wrapped in a single transaction.
-- OWNER-RUN ONLY. Verify against production schema before executing.

```

```

BEGIN;

DO $$
DECLARE
 t record;
 targets text[][] := ARRAY[
 ['invoices','total'], ['invoices','amount'], ['invoices','vat_amount'],
 ['pharmacy_sales','total_amount'], ['pharmacy_sales','discount'],

```

```

 ['pharmacy_sales','insurance_coverage'], ['pharmacy_sales','patient_share'],
 ['finance_vouchers','amount'],
 ['hr_salaries','basic'], ['hr_salaries','allowances'], ['hr_salaries','deductions'],
 ['hr_salaries','advances_deducted'], ['hr_salaries','net_salary']
];
 i int;
BEGIN
 FOR i IN 1 .. array_length(targets, 1) LOOP
 FOR t IN
 SELECT table_name, column_name, data_type
 FROM information_schema.columns
 WHERE table_schema = 'public'
 AND table_name = targets[i][1]
 AND column_name = targets[i][2]
 AND data_type IN ('real', 'double precision')
 LOOP
 EXECUTE format(
 'ALTER TABLE public.%I ALTER COLUMN %I TYPE NUMERIC(14,2) USING ROUND(%I::numeric, 2)',
 t.table_name, t.column_name, t.column_name
);
 RAISE NOTICE 'Converted %.% -> NUMERIC(14,2)', t.table_name, t.column_name;
 END LOOP;
 END LOOP;
END $$;

COMMIT;

```

```

=====
FILE: migrations\e22_01_operational_money_numeric_validate.sql
=====

```

```

-- e22_01_operational_money_numeric_validate.sql
-- Validation for e22_01: assert NO targeted operational money column remains a floating type.
-- Returns the count of still-floating columns; expected = 0 after the up-migration.
-- OWNER-RUN ONLY (read-only SELECT, safe to run anytime).

```

```

SELECT count(*) AS still_floating_money_columns
FROM information_schema.columns
WHERE table_schema = 'public'
 AND data_type IN ('real', 'double precision')
 AND (table_name, column_name) IN (
 ('invoices','total'), ('invoices','amount'), ('invoices','vat_amount'),
 ('pharmacy_sales','total_amount'), ('pharmacy_sales','discount'),
 ('pharmacy_sales','insurance_coverage'), ('pharmacy_sales','patient_share'),
 ('finance_vouchers','amount'),
 ('hr_salaries','basic'), ('hr_salaries','allowances'), ('hr_salaries','deductions'),
 ('hr_salaries','advances_deducted'), ('hr_salaries','net_salary')
);
-- PASS criterion: still_floating_money_columns = 0

```

=====

FILE: migrations\e23\_01\_idempotency\_keys\_down.sql

=====

```
-- e23_01_idempotency_keys_down.sql ? rollback for e23_01.
-- Drops the idempotency store. Safe: the table holds only short-lived replay records, no business data.
BEGIN;
DROP TABLE IF EXISTS idempotency_keys;
COMMIT;
```

=====

FILE: migrations\e23\_01\_idempotency\_keys\_up.sql

=====

```
-- e23_01_idempotency_keys_up.sql
-- REMEDIATION (GATE12-H): financial idempotency store for ./idempotency.js.
-- Prevents duplicate money mutations (invoice/pay/refund/journal) on double-submit / retry / race by
-- keying on a client Idempotency-Key, scoped per tenant + route. Additive + idempotent (IF NOT EXISTS).
-- RLS tenant-isolated to match the system posture (FORCE + NULLIF(current_setting('app.tenant_id'))).
-- OWNER-RUN. Safe to run on a live DB: only creates a new table + grants (no change to existing data).

BEGIN;
```

```
CREATE TABLE IF NOT EXISTS idempotency_keys (
 id BIGSERIAL PRIMARY KEY,
 tenant_id INTEGER,
 idem_key TEXT NOT NULL,
 route TEXT NOT NULL,
 status TEXT NOT NULL DEFAULT 'in_progress', -- in_progress | completed
 response_status INTEGER,
 response_body JSONB,
 created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
 completed_at TIMESTAMPTZ
);
```

```
-- the race arbiter: one (tenant, key, route) at most
CREATE UNIQUE INDEX IF NOT EXISTS idempotency_keys_tenant_key_route
 ON idempotency_keys (tenant_id, idem_key, route);
```

```
-- supports TTL cleanup of stale rows
CREATE INDEX IF NOT EXISTS idempotency_keys_created_at ON idempotency_keys (created_at);
```

```
-- RLS: tenant isolation (same idiom as the rest of the schema)
ALTER TABLE idempotency_keys ENABLE ROW LEVEL SECURITY;
ALTER TABLE idempotency_keys FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_idempotency_keys_tenant_isolation ON idempotency_keys;
CREATE POLICY rls_idempotency_keys_tenant_isolation ON idempotency_keys
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
-- app role privileges (non-superuser, RLS still enforced)
GRANT SELECT, INSERT, UPDATE, DELETE ON idempotency_keys TO nama_medical_app;
```



```
GRANT USAGE, SELECT ON SEQUENCE idempotency_keys_id_seq TO nama_medical_app;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e23_01_idempotency_keys_validate.sql
```

```
=====
```

```
-- e23_01_idempotency_keys_validate.sql ? must print all_ok = t after a successful up migration.
```

```
SELECT
 (to_regclass('public.idempotency_keys') IS NOT NULL) AS table_exists,
 (SELECT relrowsecurity FROM pg_class WHERE oid = 'public.idempotency_keys'::regclass) AS rls_enabled,
 (SELECT relforcerowsecurity FROM pg_class WHERE oid = 'public.idempotency_keys'::regclass) AS rls_forced,
 EXISTS (SELECT 1 FROM pg_indexes WHERE indexname = 'idempotency_keys_tenant_key_route') AS unique_index,
 EXISTS (SELECT 1 FROM pg_policies WHERE tablename = 'idempotency_keys') AS policy_exists,
 (
 to_regclass('public.idempotency_keys') IS NOT NULL
 AND (SELECT relrowsecurity FROM pg_class WHERE oid = 'public.idempotency_keys'::regclass)
 AND (SELECT relforcerowsecurity FROM pg_class WHERE oid = 'public.idempotency_keys'::regclass)
 AND EXISTS (SELECT 1 FROM pg_indexes WHERE indexname = 'idempotency_keys_tenant_key_route')
 AND EXISTS (SELECT 1 FROM pg_policies WHERE tablename = 'idempotency_keys')
) AS all_ok;
```

```
=====
```

```
FILE: migrations\e24_tenants_control_center_candidate_down.sql
```

```
=====
```

```
-- e24_tenants_control_center_candidate_down.sql ? rollback for the e24 candidate.
```

```
-- Removes only the columns added by the candidate up (all new, no business data). Safe.
```

```
BEGIN;
ALTER TABLE tenants DROP COLUMN IF EXISTS suspended_reason;
ALTER TABLE tenants DROP COLUMN IF EXISTS suspended_at;
ALTER TABLE tenants DROP COLUMN IF EXISTS trial_ends_at;
ALTER TABLE tenants DROP COLUMN IF EXISTS last_activity_at;
COMMIT;
```

```
=====
```

```
FILE: migrations\e24_tenants_control_center_candidate_up.sql
```

```
=====
```

```
-- e24_tenants_control_center_candidate_up.sql
```

```
-- CANDIDATE ONLY ? NOT RUN in Batch 1 (the Tenant Control Center works without it; status+plan_type
```

```
-- already exist on `tenants`). Optional additive columns for a later batch. Apply on an ISOLATED restore
```

```
-- first (gate G9), with explicit owner approval. Additive only ? NO DROP / NO type change / NO data loss.
BEGIN;
ALTER TABLE tenants ADD COLUMN IF NOT EXISTS last_activity_at TIMESTAMPTZ;
ALTER TABLE tenants ADD COLUMN IF NOT EXISTS trial_ends_at TIMESTAMPTZ;
ALTER TABLE tenants ADD COLUMN IF NOT EXISTS suspended_at TIMESTAMPTZ;
ALTER TABLE tenants ADD COLUMN IF NOT EXISTS suspended_reason TEXT;
COMMIT;
```

```
=====
```

```
FILE: migrations\e24_tenants_control_center_candidate_validate.sql
```

```
=====
```

```
-- e24_tenants_control_center_candidate_validate.sql ? after the candidate up, all_ok must be t.
```

```
SELECT (
 EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name='tenants' AND column_name='last_activity_at')
AND EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name='tenants' AND column_name='trial_ends_at')
AND EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name='tenants' AND column_name='suspended_at')
AND EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name='tenants' AND column_name='suspended_reason')
) AS all_ok;
```

```
=====
```

```
FILE: migrations\e25_plans_pricing_candidate_down.sql
```

```
=====
```

```
-- e25_plans_pricing_candidate_down.sql
```

```
-- Rollback for e25 ? ONLY for an isolated test DB where e25_up was applied. These tables are NEW and
-- additive (Batch 3 created them); dropping them removes no pre-existing data. NEVER run on production
-- without explicit owner approval. Reverse dependency order.
```

```
BEGIN;
DROP TABLE IF EXISTS tenant_plan_assignments;
DROP TABLE IF EXISTS plan_entitlements;
DROP TABLE IF EXISTS plans;
COMMIT;
```

```
=====
```

```
FILE: migrations\e25_plans_pricing_candidate_up.sql
```

```
=====
```

```
-- e25_plans_pricing_candidate_up.sql
```

```
-- CANDIDATE ONLY ? NOT RUN on production in Batch 3. Apply on an ISOLATED restore first (gate G9),
-- with explicit owner approval. ADDITIVE ONLY ? NO DROP / NO type change / NO data rewrite / NO data loss.
-- Foundation for Jumanasoft SaaS plans/pricing/entitlements. Does NOT touch the existing `tenants` table
-- (tenants.plan_type / tenants.status remain as-is for backward compatibility).
```

```
BEGIN;
```

```

-- 1) Plan catalog (identity + pricing). Soft-disable via `active` (never hard-deleted).
CREATE TABLE IF NOT EXISTS plans (
 id SERIAL PRIMARY KEY,
 plan_key VARCHAR(50) NOT NULL UNIQUE,
 name_ar VARCHAR(120) NOT NULL,
 name_en VARCHAR(120) NOT NULL,
 description_ar TEXT NOT NULL DEFAULT '',
 description_en TEXT NOT NULL DEFAULT '',
 currency CHAR(3) NOT NULL,
 monthly_price NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (monthly_price >= 0),
 yearly_price NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (yearly_price >= 0),
 trial_days INTEGER NOT NULL DEFAULT 0 CHECK (trial_days >= 0 AND trial_days <= 365),
 active BOOLEAN NOT NULL DEFAULT true,
 sort_order INTEGER NOT NULL DEFAULT 0,
 created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
 updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
 CONSTRAINT plans_plan_key_fmt CHECK (plan_key ~ '^[a-z0-9_]{2,40}$')
);

CREATE INDEX IF NOT EXISTS idx_plans_active ON plans (active, sort_order);

-- 2) Entitlements / feature limits (1:1 with a plan). NULL limit = unlimited.
CREATE TABLE IF NOT EXISTS plan_entitlements (
 plan_id INTEGER PRIMARY KEY REFERENCES plans(id) ON DELETE CASCADE,
 max_users INTEGER CHECK (max_users IS NULL OR max_users >= 0),
 max_branches INTEGER CHECK (max_branches IS NULL OR max_branches >= 0),
 max_invoices_per_month INTEGER CHECK (max_invoices_per_month IS NULL OR max_invoices_per_month >= 0),
 modules_enabled TEXT NOT NULL DEFAULT '',
 support_level VARCHAR(20) NOT NULL DEFAULT 'standard',
 api_access BOOLEAN NOT NULL DEFAULT false,
 custom_domain BOOLEAN NOT NULL DEFAULT false
);

-- 3) Tenant -> plan assignment (historical; current = latest with effective_to IS NULL). No in-place mutate.
CREATE TABLE IF NOT EXISTS tenant_plan_assignments (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 plan_key VARCHAR(50) NOT NULL REFERENCES plans(plan_key),
 assignment_source VARCHAR(20) NOT NULL DEFAULT 'manual',
 assigned_by INTEGER,
 assigned_at TIMESTAMPTZ NOT NULL DEFAULT now(),
 effective_from TIMESTAMPTZ NOT NULL DEFAULT now(),
 effective_to TIMESTAMPTZ,
 CONSTRAINT tpa_source_chk CHECK (assignment_source IN ('manual', 'trial', 'migration'))
);

CREATE INDEX IF NOT EXISTS idx_tpa_tenant_current ON tenant_plan_assignments (tenant_id, effective_to, assigned_at DESC);

COMMIT;

=====
FILE: migrations\e25_plans_pricing_candidate_validate.sql

```

=====

```
-- e25_plans_pricing_candidate_validate.sql
-- Read-only validation for an ISOLATED test DB after e25_up. Returns all_ok = true iff the three tables,
-- key constraints, and indexes exist as designed. No writes.
SELECT
 (SELECT count(*) FROM information_schema.tables
 WHERE table_name IN ('plans','plan_entitlements','tenant_plan_assignments')) = 3
S tables_present,
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name = 'plans'
 AND column_name IN ('plan_key','currency','monthly_price','yearly_price','active','sort_order')) = 6
AS plans_cols_present,
 EXISTS (SELECT 1 FROM information_schema.table_constraints
 WHERE table_name='plans' AND constraint_name='plans_plan_key_fmt')
AS plan_key_check_present,
 EXISTS (SELECT 1 FROM information_schema.table_constraints
 WHERE table_name='tenant_plan_assignments' AND constraint_name='tpa_source_chk')
S source_check_present,
 EXISTS (SELECT 1 FROM pg_indexes WHERE indexname='idx_plans_active')
AS plans_index_present,
 EXISTS (SELECT 1 FROM pg_indexes WHERE indexname='idx_tpa_tenant_current')
AS tpa_index_present,
 (
 (SELECT count(*) FROM information_schema.tables
 WHERE table_name IN ('plans','plan_entitlements','tenant_plan_assignments')) = 3
 AND EXISTS (SELECT 1 FROM information_schema.table_constraints
 WHERE table_name='plans' AND constraint_name='plans_plan_key_fmt')
 AND EXISTS (SELECT 1 FROM information_schema.table_constraints
 WHERE table_name='tenant_plan_assignments' AND constraint_name='tpa_source_chk')
) AS all_ok;
```

=====

FILE: migrations\e26\_payment\_gateway\_ref\_down.sql

=====

```
-- Rollback Migration: Remove payment_gateway_ref column from invoices.
BEGIN;

DROP INDEX IF EXISTS idx_invoices_payment_gateway_ref;
ALTER TABLE invoices DROP COLUMN IF EXISTS payment_gateway_ref;

COMMIT;
```

=====

FILE: migrations\e26\_payment\_gateway\_ref\_up.sql

=====

```
-- Migration: Add payment_gateway_ref column to invoices for payment gateway integrations.
BEGIN;
```

```
ALTER TABLE invoices ADD COLUMN IF NOT EXISTS payment_gateway_ref VARCHAR(255);
CREATE INDEX IF NOT EXISTS idx_invoices_payment_gateway_ref ON invoices (payment_gateway_ref);

COMMIT;
```

```
=====
FILE: migrations\e26_payment_gateway_ref_validate.sql
=====
```

```
-- Validation: Verify that payment_gateway_ref column exists on invoices table.
DO $$
BEGIN
 IF NOT EXISTS (
 SELECT 1
 FROM information_schema.columns
 WHERE table_name = 'invoices'
 AND column_name = 'payment_gateway_ref'
) THEN
 RAISE EXCEPTION 'Validation failed: payment_gateway_ref column does not exist on invoices table';
 END IF;

 RAISE NOTICE 'Validation passed: payment_gateway_ref column exists on invoices table';
END $$;
```

```
=====
FILE: migrations\e27_pharmacy_queue_columns_down.sql
=====
```

```
-- Rollback Migration: Remove custom columns from pharmacy_prescriptions_queue.
BEGIN;

ALTER TABLE pharmacy_prescriptions_queue DROP COLUMN IF EXISTS medication_name;
ALTER TABLE pharmacy_prescriptions_queue DROP COLUMN IF EXISTS dosage;
ALTER TABLE pharmacy_prescriptions_queue DROP COLUMN IF EXISTS quantity_per_day;
ALTER TABLE pharmacy_prescriptions_queue DROP COLUMN IF EXISTS frequency;
ALTER TABLE pharmacy_prescriptions_queue DROP COLUMN IF EXISTS duration;
ALTER TABLE pharmacy_prescriptions_queue DROP COLUMN IF EXISTS price;
ALTER TABLE pharmacy_prescriptions_queue DROP COLUMN IF EXISTS payment_method;

COMMIT;
```

```
=====
FILE: migrations\e27_pharmacy_queue_columns_up.sql
=====
```

```
-- Migration: Add missing columns to pharmacy_prescriptions_queue for CPOE and pharmacy billing.
BEGIN;
```

```

ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS medication_name TEXT DEFAULT '';
ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS dosage TEXT DEFAULT '';
ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS quantity_per_day TEXT DEFAULT '1';
ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS frequency TEXT DEFAULT '';
ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS duration TEXT DEFAULT '';
ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS price NUMERIC(10, 2) DEFAULT 0;
ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS payment_method TEXT DEFAULT '';
ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS tenant_id INTEGER DEFAULT 1;
ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS branch_id INTEGER DEFAULT 1;

-- If tenant_id was missing and we need RLS (like on staging)
ALTER TABLE pharmacy_prescriptions_queue ALTER COLUMN tenant_id SET NOT NULL;
CREATE INDEX IF NOT EXISTS idx_ph_rx_queue_tenant ON pharmacy_prescriptions_queue (tenant_id, branch_id);

COMMIT;

=====
FILE: migrations\e27_pharmacy_queue_columns_validate.sql
=====

-- Validation: Verify that all required columns exist on pharmacy_prescriptions_queue.
DO $$
DECLARE
 col RECORD;
 required_cols TEXT[] := ARRAY['medication_name', 'dosage', 'quantity_per_day', 'frequency', 'duration', 'p
rice', 'payment_method', 'tenant_id', 'branch_id'];
 c TEXT;
BEGIN
 FOREACH c IN ARRAY required_cols LOOP
 IF NOT EXISTS (
 SELECT 1
 FROM information_schema.columns
 WHERE table_name = 'pharmacy_prescriptions_queue'
 AND column_name = c
) THEN
 RAISE EXCEPTION 'Validation failed: Column % does not exist on pharmacy_prescriptions_queue table'
, c;
 END IF;
 END LOOP;

 RAISE NOTICE 'Validation passed: All required columns exist on pharmacy_prescriptions_queue table';
END $$;

=====
FILE: migrations\e28_patients_schema_reconciliation_down.sql
=====

-- Rollback Migration: Remove custom clinical/admin columns from patients.
BEGIN;

```

```
DROP INDEX IF EXISTS idx_patients_mrn;
```

```
ALTER TABLE patients DROP COLUMN IF EXISTS mrn;
ALTER TABLE patients DROP COLUMN IF EXISTS allergies;
ALTER TABLE patients DROP COLUMN IF EXISTS chronic_diseases;
ALTER TABLE patients DROP COLUMN IF EXISTS emergency_contact_name;
ALTER TABLE patients DROP COLUMN IF EXISTS emergency_contact_phone;
ALTER TABLE patients DROP COLUMN IF EXISTS address;
ALTER TABLE patients DROP COLUMN IF EXISTS insurance_company;
ALTER TABLE patients DROP COLUMN IF EXISTS insurance_policy_number;
ALTER TABLE patients DROP COLUMN IF EXISTS insurance_class;
```

```
COMMIT;
```

```
=====
FILE: migrations\e28_patients_schema_reconciliation_up.sql
=====
```

```
-- Migration: Add missing clinical and administrative columns to patients table.
BEGIN;
```

```
ALTER TABLE patients ADD COLUMN IF NOT EXISTS mrn TEXT DEFAULT '';
ALTER TABLE patients ADD COLUMN IF NOT EXISTS allergies TEXT DEFAULT '';
ALTER TABLE patients ADD COLUMN IF NOT EXISTS chronic_diseases TEXT DEFAULT '';
ALTER TABLE patients ADD COLUMN IF NOT EXISTS emergency_contact_name TEXT DEFAULT '';
ALTER TABLE patients ADD COLUMN IF NOT EXISTS emergency_contact_phone TEXT DEFAULT '';
ALTER TABLE patients ADD COLUMN IF NOT EXISTS address TEXT DEFAULT '';
ALTER TABLE patients ADD COLUMN IF NOT EXISTS insurance_company TEXT DEFAULT '';
ALTER TABLE patients ADD COLUMN IF NOT EXISTS insurance_policy_number TEXT DEFAULT '';
ALTER TABLE patients ADD COLUMN IF NOT EXISTS insurance_class TEXT DEFAULT '';
```

```
-- Add index on MRN for fast lookups
CREATE INDEX IF NOT EXISTS idx_patients_mrn ON patients (mrn);
```

```
COMMIT;
```

```
=====
FILE: migrations\e28_patients_schema_reconciliation_validate.sql
=====
```

```
-- Validation: Verify that all required columns exist on patients.
```

```
DO $$
DECLARE
 col RECORD;
 required_cols TEXT[] := ARRAY['mrn', 'allergies', 'chronic_diseases', 'emergency_contact_name', 'emergency_contact_phone', 'address', 'insurance_company', 'insurance_policy_number', 'insurance_class'];
 c TEXT;
BEGIN
 FOREACH c IN ARRAY required_cols LOOP
```

```

 IF NOT EXISTS (
 SELECT 1
 FROM information_schema.columns
 WHERE table_name = 'patients'
 AND column_name = c
) THEN
 RAISE EXCEPTION 'Validation failed: Column % does not exist on patients table', c;
 END IF;
 END LOOP;

 RAISE NOTICE 'Validation passed: All required columns exist on patients table';
END $$;

```

```

=====
FILE: migrations\e29_prescriptions_rls_reconciliation_down.sql
=====

```

```

-- Rollback Migration: Remove RLS and custom columns from prescriptions.
BEGIN;

DROP INDEX IF EXISTS idx_prescriptions_tenant_facility;
DROP POLICY IF EXISTS rls_prescriptions_tenant_isolation ON prescriptions;
ALTER TABLE prescriptions DISABLE ROW LEVEL SECURITY;

ALTER TABLE prescriptions DROP COLUMN IF EXISTS tenant_id;
ALTER TABLE prescriptions DROP COLUMN IF EXISTS facility_id;

COMMIT;

```

```

=====
FILE: migrations\e29_prescriptions_rls_reconciliation_up.sql
=====

```

```

-- Migration: Reconcile prescriptions table columns and RLS with production.
BEGIN;

ALTER TABLE prescriptions ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
ALTER TABLE prescriptions ADD COLUMN IF NOT EXISTS facility_id INTEGER;

-- Backfill tenant_id
UPDATE prescriptions SET tenant_id = 1 WHERE tenant_id IS NULL;
ALTER TABLE prescriptions ALTER COLUMN tenant_id SET NOT NULL;

-- Enable RLS
ALTER TABLE prescriptions ENABLE ROW LEVEL SECURITY;
ALTER TABLE prescriptions FORCE ROW LEVEL SECURITY;

-- Create RLS policy
DROP POLICY IF EXISTS rls_prescriptions_tenant_isolation ON prescriptions;
CREATE POLICY rls_prescriptions_tenant_isolation ON prescriptions

```



```

FOR ALL
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create index
CREATE INDEX IF NOT EXISTS idx_prescriptions_tenant_facility ON prescriptions (tenant_id, facility_id, patient_id);

COMMIT;

=====
FILE: migrations\e29_prescriptions_rls_reconciliation_validate.sql
=====

-- Validation: Verify that prescriptions has tenant_id, facility_id, and RLS enabled.
DO $$
BEGIN
 IF NOT EXISTS (
 SELECT 1
 FROM information_schema.columns
 WHERE table_name = 'prescriptions'
 AND column_name = 'tenant_id'
) THEN
 RAISE EXCEPTION 'Validation failed: tenant_id column does not exist on prescriptions table';
 END IF;

 IF NOT EXISTS (
 SELECT 1
 FROM pg_policies
 WHERE tablename = 'prescriptions'
 AND policyname = 'rls_prescriptions_tenant_isolation'
) THEN
 RAISE EXCEPTION 'Validation failed: rls_prescriptions_tenant_isolation policy does not exist on prescriptions table';
 END IF;

 RAISE NOTICE 'Validation passed: prescriptions table RLS is fully reconciled';
END $$;

=====
FILE: migrations\e2_01_coding_down.sql
=====

-- e2_01_coding_down.sql (rollback of e2_01_coding_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ?????? ?????? ?? ????? coding. idempotent (IF EXISTS). ?? ????? medical_records_coding ??????.
BEGIN;

DROP POLICY IF EXISTS rls_coding_tenant_isolation ON coding;
ALTER TABLE IF EXISTS coding DROP CONSTRAINT IF EXISTS chk_coding_code_system;

```

```

-- idempotent inverse of the up's CREATE INDEX IF NOT EXISTS (explicit; DROP TABLE also removes them)
DROP INDEX IF EXISTS idx_coding_tenant_id;
DROP INDEX IF EXISTS idx_coding_patient_id;
DROP INDEX IF EXISTS idx_coding_encounter_ref;
DROP TABLE IF EXISTS coding;

COMMIT;

=====
FILE: migrations\e2_01_coding_up.sql
=====

-- =====
-- e2_01_coding_up.sql
-- E2 MEDICAL RECORDS / HIM ? Clinical Coding (ICD10 / SNOMED / CPT).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ????? ????? (coding) ??? ????? ICD10/SNOMED/CPT ?????/????
-- ??? ??? ?????? ????? ?? medical_records_coding (???? ?? ??? tenant_id ??? RLS).
-- ????? ??? patient_id ??? ????? ??????? encounter_ref ????? ??? NULL (?? ????? ????? encounters?
-- ?????? visit_lifecycle) ?????? orders.encounter_id / E1 problems.encounter_ref.
--
-- ??? ????? ??? FORCE RLS ??????: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS
-- + ????? ??? tenant_id + ??? tenant_id. fail-closed: NULLIF(current_setting('app.tenant_id')).
--
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP/ADD CONSTRAINT IF EXISTS + DROP POLICY IF EXISTS
-- + CREATE INDEX IF NOT EXISTS. ?? ????? medical_records_coding ?????? (????) ? ????? ?????? ???.
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS coding (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 patient_id INTEGER NOT NULL,
 encounter_ref INTEGER,
 s.encounter_id)
 code_system TEXT NOT NULL DEFAULT 'ICD10',
 code TEXT NOT NULL,
 description TEXT DEFAULT '',
 coder_id INTEGER,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT chk_coding_code_system CHECK (code_system IN ('ICD10', 'SNOMED', 'CPT'))
);

-- idempotent: ensure the code_system CHECK exists even when the table pre-dates this migration
-- (CREATE TABLE IF NOT EXISTS skips the inline constraint on an existing table). DROP then ADD.
ALTER TABLE coding DROP CONSTRAINT IF EXISTS chk_coding_code_system;
ALTER TABLE coding ADD CONSTRAINT chk_coding_code_system CHECK (code_system IN ('ICD10', 'SNOMED', 'CPT'));

CREATE INDEX IF NOT EXISTS idx_coding_tenant_id ON coding (tenant_id);
CREATE INDEX IF NOT EXISTS idx_coding_patient_id ON coding (patient_id);

```

```
CREATE INDEX IF NOT EXISTS idx_coding_encounter_ref ON coding (encounter_ref);
```

```
ALTER TABLE coding ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE coding FORCE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS rls_coding_tenant_isolation ON coding;
```

```
CREATE POLICY rls_coding_tenant_isolation ON coding
```

```
 FOR ALL
```

```
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
```

```
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
COMMIT;
```

```
=====
FILE: migrations\e2_01_coding_validate.sql
=====
```

```
-- e2_01_coding_validate.sql (run AFTER e2_01_coding_up.sql; read-only)
```

```
-- PASS = coding exists with tenant_id FK NOT NULL + FORCE RLS + isolation policy + indexes + code_system CHECK.

K.
```

```
SELECT
```

```
 (SELECT count(*) FROM information_schema.tables WHERE table_name='coding') AS coding_exists,
```

```
 -- expect 1
```

```
 (SELECT count(*) FROM information_schema.columns
```

```
 WHERE table_name='coding'
```

```
 AND column_name IN ('tenant_id','facility_id','patient_id','encounter_ref','code_system','code','description','coder_id','created_at')) AS coding_cols, -- expect 9
```

```
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='coding') AS coding_force_rls,
```

```
 -- expect t
```

```
 (SELECT count(*) FROM pg_policies WHERE tablename='coding' AND policyname='rls_coding_tenant_isolation') AS coding_policy, -- expect 1
```

```
 (SELECT count(*) FROM pg_constraint WHERE conname='chk_coding_code_system') AS coding_codesys_check,
```

```
 -- expect 1
```

```
 (SELECT count(*) FROM pg_indexes WHERE tablename='coding' AND indexname='idx_coding_tenant_id') AS coding_idx; -- expect 1
```

```
-- ----- tenant_id is NOT NULL -----
```

```
SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
```

```
 WHERE table_name='coding' AND column_name='tenant_id' AND is_nullable='NO';
```

```
 -- expect 1
```

```
-- ----- FK to tenants(id) -----
```

```
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
```

```
 WHERE conrelid='coding'::regclass AND confrelid='tenants'::regclass AND contype='f';
```

```
 -- expect 1
```

```
=====
FILE: migrations\e2_02_roi_down.sql
=====
```

```
-- e2_02_roi_down.sql (rollback of e2_02_roi_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ??? ???? ???? ?? ???? roi_requests. idempotent (IF EXISTS).
BEGIN;

DROP POLICY IF EXISTS rls_roi_requests_tenant_isolation ON roi_requests;
ALTER TABLE IF EXISTS roi_requests DROP CONSTRAINT IF EXISTS chk_roi_status;
-- idempotent inverse of the up's CREATE INDEX IF NOT EXISTS (explicit; DROP TABLE also removes them)
DROP INDEX IF EXISTS idx_roi_requests_tenant_id;
DROP INDEX IF EXISTS idx_roi_requests_patient_id;
DROP INDEX IF EXISTS idx_roi_requests_status;
DROP TABLE IF EXISTS roi_requests;

COMMIT;
```

```
=====
FILE: migrations\e2_02_roi_up.sql
=====
```

```
-- =====
-- e2_02_roi_up.sql
-- E2 MEDICAL RECORDS / HIM ? Release of Information (ROI) requests.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? ????? ??????? ?? ????????? (???? ???? ???? ???? ???? ???? ???? ???? ???? ????).
-- ??? ?? ?????? medical_records_requests (file-room) ?? ?? ??? ?????? ?????? ?????? + RLS.
-- ????? ?? patient_id? ?????? status ?????? pending/approved/released/denied.
--
-- ??? ??? ???? FORCE RLS ??????: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS
-- + ????? ?? tenant_id + ??? tenant_id. fail-closed.
--
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP/ADD CONSTRAINT IF EXISTS + DROP POLICY IF EXISTS
-- + CREATE INDEX IF NOT EXISTS.
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS roi_requests (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 patient_id INTEGER NOT NULL,
 requester TEXT NOT NULL DEFAULT '',
 purpose TEXT DEFAULT '',
 status TEXT NOT NULL DEFAULT 'pending',
 requested_by INTEGER,
 approved_by INTEGER,
 released_at TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT chk_roi_status CHECK (status IN ('pending', 'approved', 'released', 'denied'))
);

-- idempotent: ensure the status CHECK exists even when the table pre-dates this migration.
```

```

ALTER TABLE roi_requests DROP CONSTRAINT IF EXISTS chk_roi_status;
ALTER TABLE roi_requests ADD CONSTRAINT chk_roi_status CHECK (status IN ('pending', 'approved', 'released', 'denied'));

CREATE INDEX IF NOT EXISTS idx_roi_requests_tenant_id ON roi_requests (tenant_id);
CREATE INDEX IF NOT EXISTS idx_roi_requests_patient_id ON roi_requests (patient_id);
CREATE INDEX IF NOT EXISTS idx_roi_requests_status ON roi_requests (status);

ALTER TABLE roi_requests ENABLE ROW LEVEL SECURITY;
ALTER TABLE roi_requests FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_roi_requests_tenant_isolation ON roi_requests;
CREATE POLICY rls_roi_requests_tenant_isolation ON roi_requests
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

```

=====
FILE: migrations\e2_02_roi_validate.sql
=====

```

```

-- e2_02_roi_validate.sql (run AFTER e2_02_roi_up.sql; read-only)
-- PASS = roi_requests exists with tenant_id FK NOT NULL + FORCE RLS + isolation policy + indexes + status CHECK.

```

```

SELECT
 (SELECT count(*) FROM information_schema.tables WHERE table_name='roi_requests') AS roi_exists,
 -- expect 1
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='roi_requests'
 AND column_name IN ('tenant_id','facility_id','patient_id','requester','purpose','status','requested_by',
 'approved_by','released_at','created_at')) AS roi_cols, -- expect 10
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='roi_requests') AS roi_force_rls,
 -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='roi_requests' AND policyname='rls_roi_requests_tenant_isolation') AS roi_policy, -- expect 1
 (SELECT count(*) FROM pg_constraint WHERE conname='chk_roi_status') AS roi_status_check,
 -- expect 1
 (SELECT count(*) FROM pg_indexes WHERE tablename='roi_requests' AND indexname='idx_roi_requests_tenant_id')
AS roi_idx; -- expect 1

-- ----- tenant_id is NOT NULL -----
SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
 WHERE table_name='roi_requests' AND column_name='tenant_id' AND is_nullable='NO';
 -- expect 1

-- ----- FK to tenants(id) -----
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
 WHERE conrelid='roi_requests'::regclass AND confrelid='tenants'::regclass AND contype='f';
 -- expect 1

```

=====

FILE: migrations\e2\_03\_record\_access\_down.sql

=====

```
-- e2_03_record_access_down.sql (rollback of e2_03_record_access_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ????? ????? ?? ???? record_access_log. idempotent (IF EXISTS).
```

BEGIN;

```
DROP POLICY IF EXISTS rls_record_access_log_tenant_isolation ON record_access_log;
ALTER TABLE IF EXISTS record_access_log DROP CONSTRAINT IF EXISTS chk_record_access_type;
-- idempotent inverse of the up's CREATE INDEX IF NOT EXISTS (explicit; DROP TABLE also removes them)
DROP INDEX IF EXISTS idx_record_access_log_tenant_id;
DROP INDEX IF EXISTS idx_record_access_log_patient_id;
DROP INDEX IF EXISTS idx_record_access_log_at;
DROP TABLE IF EXISTS record_access_log;
```

COMMIT;

=====

FILE: migrations\e2\_03\_record\_access\_up.sql

=====

```
-- =====
-- e2_03_record_access_up.sql
-- E2 MEDICAL RECORDS / HIM ? Record Access Log + Break-Glass.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ???? PHI ??? ???? ????? ????? (?? ??? / ?? ???? / ??? / ?? ??????) ? ??? ??? ?????
-- HIM ????????? ????? ?? audit_trail ??????. access_type ???? normal ? break_glass?
-- break-glass ?????? reason ????????? ???? + ???? ????? BREAK_GLASS ?? audit_trail.
-- net-new (?? ?????) ? ???? ??? ???? ??? FORCE RLS ??????.
--
-- tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS + ????? ??? tenant_id + ?????. fail-closed.
--
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP/ADD CONSTRAINT IF EXISTS + DROP POLICY IF EXISTS
-- + CREATE INDEX IF NOT EXISTS.
```

-- =====

BEGIN;

```
CREATE TABLE IF NOT EXISTS record_access_log (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 patient_id INTEGER NOT NULL,
 accessor_id INTEGER,
 access_type TEXT NOT NULL DEFAULT 'normal',
 reason TEXT DEFAULT '',
 at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT chk_record_access_type CHECK (access_type IN ('normal', 'break_glass'))
```

```

);

-- idempotent: ensure the access_type CHECK exists even when the table pre-dates this migration.
ALTER TABLE record_access_log DROP CONSTRAINT IF EXISTS chk_record_access_type;
ALTER TABLE record_access_log ADD CONSTRAINT chk_record_access_type CHECK (access_type IN ('normal', 'break_glass'));

CREATE INDEX IF NOT EXISTS idx_record_access_log_tenant_id ON record_access_log (tenant_id);
CREATE INDEX IF NOT EXISTS idx_record_access_log_patient_id ON record_access_log (patient_id);
CREATE INDEX IF NOT EXISTS idx_record_access_log_at ON record_access_log (at);

ALTER TABLE record_access_log ENABLE ROW LEVEL SECURITY;
ALTER TABLE record_access_log FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_record_access_log_tenant_isolation ON record_access_log;
CREATE POLICY rls_record_access_log_tenant_isolation ON record_access_log
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: migrations\e2_03_record_access_validate.sql
=====

-- e2_03_record_access_validate.sql (run AFTER e2_03_record_access_up.sql; read-only)
-- PASS = record_access_log exists with tenant_id FK NOT NULL + FORCE RLS + isolation policy + indexes + access_type CHECK.

SELECT
 (SELECT count(*) FROM information_schema.tables WHERE table_name='record_access_log') AS ral_exists,
 -- expect 1
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='record_access_log'
 AND column_name IN ('tenant_id','facility_id','patient_id','accessor_id','access_type','reason','at'))
AS ral_cols, -- expect 7
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='record_access_log') AS ral_force_rls,
 -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='record_access_log' AND policyname='rls_record_access_log_tenant_isolation') AS ral_policy, -- expect 1
 (SELECT count(*) FROM pg_constraint WHERE conname='chk_record_access_type') AS ral_type_check,
 -- expect 1
 (SELECT count(*) FROM pg_indexes WHERE tablename='record_access_log' AND indexname='idx_record_access_log_tenant_id') AS ral_idx; -- expect 1

-- ----- tenant_id is NOT NULL -----
SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
 WHERE table_name='record_access_log' AND column_name='tenant_id' AND is_nullable='NO';
 -- expect 1

-- ----- FK to tenants(id) -----
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint

```

```
WHERE conrelid='record_access_log'::regclass AND confrelid='tenants'::regclass AND contype='f';
-- expect 1
```

=====

FILE: migrations\e30\_cardiology\_module\_down.sql

=====

```
-- Rollback Migration: Drop cardiology tables and RLS policies.
```

```
BEGIN;
```

```
DROP INDEX IF EXISTS idx_ecg_records_patient;
```

```
DROP INDEX IF EXISTS idx_cardiology_procedures_patient;
```

```
DROP POLICY IF EXISTS rls_ecg_records_tenant_isolation ON ecg_records;
```

```
DROP POLICY IF EXISTS rls_cardiology_procedures_tenant_isolation ON cardiology_procedures;
```

```
DROP TABLE IF EXISTS ecg_records;
```

```
DROP TABLE IF EXISTS cardiology_procedures;
```

```
COMMIT;
```

=====

FILE: migrations\e30\_cardiology\_module\_up.sql

=====

```
-- Migration: Create cardiology procedures and ECG records tables with RLS.
```

```
BEGIN;
```

```
-- Create cardiology_procedures table
```

```
CREATE TABLE IF NOT EXISTS cardiology_procedures (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 procedure_type TEXT NOT NULL, -- e.g. 'Catheterization', 'Echocardiography', 'Stress Test'
 findings TEXT DEFAULT '',
 recommendations TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

```
-- Create ecg_records table
```

```
CREATE TABLE IF NOT EXISTS ecg_records (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 leads_data JSONB NOT NULL, -- ECG voltage samples for leads (I, II, III, aVR, aVL, aVF, V1-V6)
 heart_rate INTEGER,
 interpretation TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```



```

 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);

-- Enable Row-Level Security (RLS) on both tables
ALTER TABLE cardiology_procedures ENABLE ROW LEVEL SECURITY;
ALTER TABLE cardiology_procedures FORCE ROW LEVEL SECURITY;

ALTER TABLE ecg_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE ecg_records FORCE ROW LEVEL SECURITY;

-- Create RLS policies for tenant isolation
DROP POLICY IF EXISTS rls_cardiology_procedures_tenant_isolation ON cardiology_procedures;
CREATE POLICY rls_cardiology_procedures_tenant_isolation ON cardiology_procedures
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

DROP POLICY IF EXISTS rls_ecg_records_tenant_isolation ON ecg_records;
CREATE POLICY rls_ecg_records_tenant_isolation ON ecg_records
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create indexes for fast lookups
CREATE INDEX IF NOT EXISTS idx_cardiology_procedures_patient ON cardiology_procedures (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_ecg_records_patient ON ecg_records (tenant_id, patient_id);

-- Grant privileges to the application roles (Staging & Production)
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE cardiology_procedures TO jumanasoft_staging_user';
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE ecg_records TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE cardiology_procedures_id_seq TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE ecg_records_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE cardiology_procedures TO nama_medical_app';
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE ecg_records TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE cardiology_procedures_id_seq TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE ecg_records_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;

```

```

=====
FILE: migrations\e30_cardiology_module_validate.sql
=====

```

```

-- Validation: Verify that cardiology tables exist and RLS is enabled.
DO $$
BEGIN
 IF NOT EXISTS (
 SELECT 1
 FROM information_schema.tables
 WHERE table_name = 'cardiology_procedures'
) THEN
 RAISE EXCEPTION 'Validation failed: table cardiology_procedures does not exist';
 END IF;

 IF NOT EXISTS (
 SELECT 1
 FROM information_schema.tables
 WHERE table_name = 'ecg_records'
) THEN
 RAISE EXCEPTION 'Validation failed: table ecg_records does not exist';
 END IF;

 IF NOT EXISTS (
 SELECT 1
 FROM pg_policies
 WHERE tablename = 'cardiology_procedures'
 AND policyname = 'rls_cardiology_procedures_tenant_isolation'
) THEN
 RAISE EXCEPTION 'Validation failed: rls_cardiology_procedures_tenant_isolation policy does not exist';
 END IF;

 IF NOT EXISTS (
 SELECT 1
 FROM pg_policies
 WHERE tablename = 'ecg_records'
 AND policyname = 'rls_ecg_records_tenant_isolation'
) THEN
 RAISE EXCEPTION 'Validation failed: rls_ecg_records_tenant_isolation policy does not exist';
 END IF;

 RAISE NOTICE 'Validation passed: Cardiology module tables and RLS are fully set up';
END $$;

```

```

=====
FILE: migrations\e31_gastro_module_down.sql
=====

```

```

-- Rollback: Drop gastroenterology tables and RLS policies.
BEGIN;

DROP TABLE IF EXISTS biopsy_samples CASCADE;
DROP TABLE IF EXISTS endoscopy_reports CASCADE;

COMMIT;

```

```
=====
FILE: migrations\e31_gastro_module_up.sql
=====
```

```
-- Migration: Create gastroenterology tables with RLS and conditional grants.
BEGIN;
```

```
-- Create endoscopy_reports table
```

```
CREATE TABLE IF NOT EXISTS endoscopy_reports (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 endoscopy_type TEXT NOT NULL, -- e.g. 'Gastroscopy', 'Colonoscopy', 'ERCP'
 indications TEXT DEFAULT '',
 findings TEXT DEFAULT '',
 complications TEXT DEFAULT '',
 recommendations TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

```
-- Create biopsy_samples table
```

```
CREATE TABLE IF NOT EXISTS biopsy_samples (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 specimen_source TEXT NOT NULL, -- e.g. 'Gastric Antrum', 'Sigmoid Colon'
 clinical_notes TEXT DEFAULT '',
 status TEXT DEFAULT 'Pending', -- 'Pending', 'Resulted'
 result_findings TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

```
-- Enable Row-Level Security (RLS)
```

```
ALTER TABLE endoscopy_reports ENABLE ROW LEVEL SECURITY;
ALTER TABLE endoscopy_reports FORCE ROW LEVEL SECURITY;
```

```
ALTER TABLE biopsy_samples ENABLE ROW LEVEL SECURITY;
ALTER TABLE biopsy_samples FORCE ROW LEVEL SECURITY;
```

```
-- Create RLS policies
```

```
DROP POLICY IF EXISTS rls_endoscopy_reports_tenant_isolation ON endoscopy_reports;
CREATE POLICY rls_endoscopy_reports_tenant_isolation ON endoscopy_reports
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
DROP POLICY IF EXISTS rls_biopsy_samples_tenant_isolation ON biopsy_samples;
```

```

CREATE POLICY rls_biopsy_samples_tenant_isolation ON biopsy_samples
FOR ALL
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create indexes
CREATE INDEX IF NOT EXISTS idx_endoscopy_reports_patient ON endoscopy_reports (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_biopsy_samples_patient ON biopsy_samples (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE endoscopy_reports TO jumanasoft_staging_user';
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE biopsy_samples TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE endoscopy_reports_id_seq TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE biopsy_samples_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE endoscopy_reports TO nama_medical_app';
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE biopsy_samples TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE endoscopy_reports_id_seq TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE biopsy_samples_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;

```

```

=====
FILE: migrations\e31_gastro_module_validate.sql
=====

```

```

-- Validation: Verify gastroenterology tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check endoscopy_reports table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'endoscopy_reports'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table endoscopy_reports does not exist';
 END IF;

 -- 2. Check biopsy_samples table
 SELECT EXISTS (

```

```

 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'biopsy_samples'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table biopsy_samples does not exist';
 END IF;

-- 3. Check RLS on endoscopy_reports
SELECT relrowsecurity INTO v_rls_enabled
FROM pg_class WHERE relname = 'endoscopy_reports';
IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on endoscopy_reports';
END IF;

-- 4. Check RLS on biopsy_samples
SELECT relrowsecurity INTO v_rls_enabled
FROM pg_class WHERE relname = 'biopsy_samples';
IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on biopsy_samples';
END IF;

-- 5. Check indexes
SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'endoscopy_reports' AND indexname = 'idx_endoscopy_reports
_patient'
) INTO v_index_exists;
IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_endoscopy_reports_patient does not exist';
END IF;

SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'biopsy_samples' AND indexname = 'idx_biopsy_samples_patie
nt'
) INTO v_index_exists;
IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_biopsy_samples_patient does not exist';
END IF;

RAISE NOTICE 'Validation passed: Gastroenterology module tables and RLS are fully set up';
END;
$$;

```

```

=====
FILE: migrations\e32_endocrine_module_down.sql
=====

```

```

-- Rollback: Drop endocrinology tables and RLS policies.
BEGIN;

DROP TABLE IF EXISTS insulin_regimens CASCADE;

```

```
DROP TABLE IF EXISTS diabetes_glucose_logs CASCADE;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e32_endocrine_module_up.sql
```

```
=====
```

```
-- Migration: Create endocrinology and diabetes tables with RLS and conditional grants.
```

```
BEGIN;
```

```
-- Create diabetes_glucose_logs table
```

```
CREATE TABLE IF NOT EXISTS diabetes_glucose_logs (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 glucose_value NUMERIC(5,2) NOT NULL, -- level in mg/dL or mmol/L
 log_type TEXT NOT NULL, -- e.g. 'Fasting', 'Postprandial', 'Random'
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

```
-- Create insulin_regimens table
```

```
CREATE TABLE IF NOT EXISTS insulin_regimens (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 insulin_type TEXT NOT NULL, -- e.g. 'Rapid-acting', 'Long-acting', 'Premixed'
 dosage TEXT NOT NULL, -- e.g. '10 units'
 is_active BOOLEAN DEFAULT TRUE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

```
-- Enable Row-Level Security (RLS)
```

```
ALTER TABLE diabetes_glucose_logs ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE diabetes_glucose_logs FORCE ROW LEVEL SECURITY;
```

```
ALTER TABLE insulin_regimens ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE insulin_regimens FORCE ROW LEVEL SECURITY;
```

```
-- Create RLS policies
```

```
DROP POLICY IF EXISTS rls_diabetes_glucose_logs_tenant_isolation ON diabetes_glucose_logs;
```

```
CREATE POLICY rls_diabetes_glucose_logs_tenant_isolation ON diabetes_glucose_logs
```

```
FOR ALL
```

```
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
```

```
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
DROP POLICY IF EXISTS rls_insulin_regimens_tenant_isolation ON insulin_regimens;
```

```

CREATE POLICY rls_insulin_regimens_tenant_isolation ON insulin_regimens
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create indexes
CREATE INDEX IF NOT EXISTS idx_diabetes_glucose_logs_patient ON diabetes_glucose_logs (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_insulin_regimens_patient ON insulin_regimens (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE diabetes_glucose_logs TO jumanasoft_staging_user';
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE insulin_regimens TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE diabetes_glucose_logs_id_seq TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE insulin_regimens_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE diabetes_glucose_logs TO nama_medical_app';
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE insulin_regimens TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE diabetes_glucose_logs_id_seq TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE insulin_regimens_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;

=====
FILE: migrations\e32_endocrine_module_validate.sql
=====

-- Validation: Verify endocrinology tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check diabetes_glucose_logs table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'diabetes_glucose_logs'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table diabetes_glucose_logs does not exist';
 END IF;

 -- 2. Check insulin_regimens table
 SELECT EXISTS (

```

```

 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'insulin_regimens'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table insulin_regimens does not exist';
 END IF;

-- 3. Check RLS on diabetes_glucose_logs
SELECT relrowsecurity INTO v_rls_enabled
FROM pg_class WHERE relname = 'diabetes_glucose_logs';
IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on diabetes_glucose_logs';
END IF;

-- 4. Check RLS on insulin_regimens
SELECT relrowsecurity INTO v_rls_enabled
FROM pg_class WHERE relname = 'insulin_regimens';
IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on insulin_regimens';
END IF;

-- 5. Check indexes
SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'diabetes_glucose_logs' AND indexname = 'idx_diabetes_glucose_logs_patient'
) INTO v_index_exists;
IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_diabetes_glucose_logs_patient does not exist';
END IF;

SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'insulin_regimens' AND indexname = 'idx_insulin_regimens_patient'
) INTO v_index_exists;
IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_insulin_regimens_patient does not exist';
END IF;

RAISE NOTICE 'Validation passed: Endocrinology module tables and RLS are fully set up';
END;
$$;

```

```

=====
FILE: migrations\e33_nephrology_module_down.sql
=====

```

```

-- Rollback: Drop nephrology tables and RLS policies.
BEGIN;

DROP TABLE IF EXISTS dialysis_sessions CASCADE;

```



```
COMMIT;
```

```
=====
FILE: migrations\e33_nephrology_module_up.sql
=====
```

```
-- Migration: Create nephrology and dialysis tables with RLS and conditional grants.
BEGIN;
```

```
-- Create dialysis_sessions table
```

```
CREATE TABLE IF NOT EXISTS dialysis_sessions (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 session_date DATE NOT NULL DEFAULT CURRENT_DATE,
 weight_pre NUMERIC(5,2), -- in kg
 weight_post NUMERIC(5,2), -- in kg
 blood_flow_rate INTEGER, -- in mL/min
 ultrafiltration_volume NUMERIC(4,2), -- in L
 duration_hours NUMERIC(3,1), -- in hours
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

```
-- Enable Row-Level Security (RLS)
```

```
ALTER TABLE dialysis_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE dialysis_sessions FORCE ROW LEVEL SECURITY;
```

```
-- Create RLS policy
```

```
DROP POLICY IF EXISTS rls_dialysis_sessions_tenant_isolation ON dialysis_sessions;
CREATE POLICY rls_dialysis_sessions_tenant_isolation ON dialysis_sessions
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
-- Create index
```

```
CREATE INDEX IF NOT EXISTS idx_dialysis_sessions_patient ON dialysis_sessions (tenant_id, patient_id);
```

```
-- Grant privileges conditionally
```

```
DO $$
```

```
BEGIN
```

```
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE dialysis_sessions TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE dialysis_sessions_id_seq TO jumanasoft_staging_user';
 END IF;
```

```
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE dialysis_sessions TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE dialysis_sessions_id_seq TO nama_medical_app';
 END IF;
```

```

 END IF;
END
$$;

COMMIT;

=====
FILE: migrations\e33_nephrology_module_validate.sql
=====

-- Validation: Verify nephrology tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check dialysis_sessions table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'dialysis_sessions'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table dialysis_sessions does not exist';
 END IF;

 -- 2. Check RLS on dialysis_sessions
 SELECT relrowsecurity INTO v_rls_enabled
 FROM pg_class WHERE relname = 'dialysis_sessions';
 IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on dialysis_sessions';
 END IF;

 -- 3. Check index
 SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'dialysis_sessions' AND indexname = 'idx_dialysis_sessions_patient'
) INTO v_index_exists;
 IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_dialysis_sessions_patient does not exist';
 END IF;

 RAISE NOTICE 'Validation passed: Nephrology module tables and RLS are fully set up';
END;
$$;

=====
FILE: migrations\e34_pulmonology_module_down.sql
=====

```

```

-- Rollback: Drop pulmonology tables and RLS policies.
BEGIN;

DROP TABLE IF EXISTS pulmonary_function_tests CASCADE;

COMMIT;

=====
FILE: migrations\e34_pulmonology_module_up.sql
=====

-- Migration: Create pulmonology and PFT tables with RLS and conditional grants.
BEGIN;

-- Create pulmonary_function_tests table
CREATE TABLE IF NOT EXISTS pulmonary_function_tests (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 test_date DATE NOT NULL DEFAULT CURRENT_DATE,
 fev1 NUMERIC(4,2), -- in liters
 fvc NUMERIC(4,2), -- in liters
 fev1_fvc_ratio NUMERIC(5,2), -- in %
 pef NUMERIC(5,1), -- in L/min
 interpretation TEXT DEFAULT '', -- Normal, Obstructive, Restrictive, Mixed
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);

-- Enable Row-Level Security (RLS)
ALTER TABLE pulmonary_function_tests ENABLE ROW LEVEL SECURITY;
ALTER TABLE pulmonary_function_tests FORCE ROW LEVEL SECURITY;

-- Create RLS policy
DROP POLICY IF EXISTS rls_pulmonary_function_tests_tenant_isolation ON pulmonary_function_tests;
CREATE POLICY rls_pulmonary_function_tests_tenant_isolation ON pulmonary_function_tests
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create index
CREATE INDEX IF NOT EXISTS idx_pulmonary_function_tests_patient ON pulmonary_function_tests (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE pulmonary_function_tests TO jumanasoft_staging_user';
 END IF;
END;

```

```

EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE pulmonary_function_tests_id_seq TO jumanasoft_staging_user';
END IF;

IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE pulmonary_function_tests TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE pulmonary_function_tests_id_seq TO nama_medical_app';
END IF;

END
$$;

COMMIT;

=====
FILE: migrations\e34_pulmonology_module_validate.sql
=====

-- Validation: Verify pulmonology tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check pulmonary_function_tests table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'pulmonary_function_tests'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table pulmonary_function_tests does not exist';
 END IF;

 -- 2. Check RLS on pulmonary_function_tests
 SELECT relrowsecurity INTO v_rls_enabled
 FROM pg_class WHERE relname = 'pulmonary_function_tests';
 IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on pulmonary_function_tests';
 END IF;

 -- 3. Check index
 SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'pulmonary_function_tests' AND indexname = 'idx_pulmonary_
function_tests_patient'
) INTO v_index_exists;
 IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_pulmonary_function_tests_patient does not exist';
 END IF;

 RAISE NOTICE 'Validation passed: Pulmonology module tables and RLS are fully set up';
END;
$$;

```

=====

FILE: migrations\e35\_rheumatology\_module\_down.sql

=====

-- Rollback: Drop rheumatology tables and RLS policies.

BEGIN;

DROP TABLE IF EXISTS joint\_assessments CASCADE;

COMMIT;

=====

FILE: migrations\e35\_rheumatology\_module\_up.sql

=====

-- Migration: Create rheumatology and joint assessment tables with RLS and conditional grants.

BEGIN;

-- Create joint\_assessments table

```
CREATE TABLE IF NOT EXISTS joint_assessments (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 assessment_date DATE NOT NULL DEFAULT CURRENT_DATE,
 tender_joint_count INTEGER, -- TJC (0-28)
 swollen_joint_count INTEGER, -- SJC (0-28)
 vas_pain INTEGER, -- 0 to 100 mm
 das28_score NUMERIC(4,2), -- Calculated DAS28 score
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
```

);

-- Enable Row-Level Security (RLS)

ALTER TABLE joint\_assessments ENABLE ROW LEVEL SECURITY;

ALTER TABLE joint\_assessments FORCE ROW LEVEL SECURITY;

-- Create RLS policy

DROP POLICY IF EXISTS rls\_joint\_assessments\_tenant\_isolation ON joint\_assessments;

CREATE POLICY rls\_joint\_assessments\_tenant\_isolation ON joint\_assessments

FOR ALL

USING (tenant\_id = NULLIF(current\_setting('app.tenant\_id', true), '')::integer)

WITH CHECK (tenant\_id = NULLIF(current\_setting('app.tenant\_id', true), '')::integer);

-- Create index

CREATE INDEX IF NOT EXISTS idx\_joint\_assessments\_patient ON joint\_assessments (tenant\_id, patient\_id);

-- Grant privileges conditionally

```

DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE joint_assessments TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE joint_assessments_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE joint_assessments TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE joint_assessments_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;

=====
FILE: migrations\e35_rheumatology_module_validate.sql
=====

-- Validation: Verify rheumatology tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check joint_assessments table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'joint_assessments'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table joint_assessments does not exist';
 END IF;

 -- 2. Check RLS on joint_assessments
 SELECT relrowsecurity INTO v_rls_enabled
 FROM pg_class WHERE relname = 'joint_assessments';
 IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on joint_assessments';
 END IF;

 -- 3. Check index
 SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'joint_assessments' AND indexname = 'idx_joint_assessments_patient'
) INTO v_index_exists;
 IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_joint_assessments_patient does not exist';
 END IF;

```

```
 RAISE NOTICE 'Validation passed: Rheumatology module tables and RLS are fully set up';
END;
$$;
```

```
=====
FILE: migrations\e36_neurology_module_down.sql
=====
```

```
-- Rollback: Drop neurology tables and RLS policies.
```

```
BEGIN;
```

```
DROP TABLE IF EXISTS neurology_assessments CASCADE;
```

```
COMMIT;
```

```
=====
FILE: migrations\e36_neurology_module_up.sql
=====
```

```
-- Migration: Create neurology and GCS/NIHSS assessment tables with RLS and conditional grants.
```

```
BEGIN;
```

```
-- Create neurology_assessments table
```

```
CREATE TABLE IF NOT EXISTS neurology_assessments (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 assessment_date DATE NOT NULL DEFAULT CURRENT_DATE,
 gcs_eye INTEGER CHECK (gcs_eye BETWEEN 1 AND 4),
 gcs_verbal INTEGER CHECK (gcs_verbal BETWEEN 1 AND 5),
 gcs_motor INTEGER CHECK (gcs_motor BETWEEN 1 AND 6),
 gcs_total_score INTEGER CHECK (gcs_total_score BETWEEN 3 AND 15),
 nihss_score INTEGER CHECK (nihss_score BETWEEN 0 AND 42),
 reflexes_status VARCHAR(100), -- e.g. Normal, Hyperreflexia, Hyporeflexia, Absent
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

```
-- Enable Row-Level Security (RLS)
```

```
ALTER TABLE neurology_assessments ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE neurology_assessments FORCE ROW LEVEL SECURITY;
```

```
-- Create RLS policy
```

```
DROP POLICY IF EXISTS rls_neurology_assessments_tenant_isolation ON neurology_assessments;
```

```
CREATE POLICY rls_neurology_assessments_tenant_isolation ON neurology_assessments
```

```
FOR ALL
```

```
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
```

```

WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create index
CREATE INDEX IF NOT EXISTS idx_neurology_assessments_patient ON neurology_assessments (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE neurology_assessments TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE neurology_assessments_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE neurology_assessments TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE neurology_assessments_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;

```

```

=====
FILE: migrations\e36_neurology_module_validate.sql
=====

```

```

-- Validation: Verify neurology tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check neurology_assessments table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'neurology_assessments'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table neurology_assessments does not exist';
 END IF;

 -- 2. Check RLS on neurology_assessments
 SELECT relrowsecurity INTO v_rls_enabled
 FROM pg_class WHERE relname = 'neurology_assessments';
 IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on neurology_assessments';
 END IF;

 -- 3. Check index
 SELECT EXISTS (
 SELECT FROM pg_indexes

```



```

 WHERE schemaname = 'public' AND tablename = 'neurology_assessments' AND indexname = 'idx_neurology_assessments_patient'
) INTO v_index_exists;
 IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_neurology_assessments_patient does not exist';
 END IF;

 RAISE NOTICE 'Validation passed: Neurology module tables and RLS are fully set up';
END;
$$;

```

```

=====
FILE: migrations\e37_nephrology_module_down.sql
=====

```

```

-- Rollback: Drop nephrology tables and RLS policies.
BEGIN;

DROP TABLE IF EXISTS dialysis_sessions CASCADE;

COMMIT;

```

```

=====
FILE: migrations\e37_nephrology_module_up.sql
=====

```

```

-- Migration: Create/upgrade nephrology and dialysis session tables with RLS and conditional grants.
BEGIN;

```

```

-- Create dialysis_sessions table if it does not exist

```

```

CREATE TABLE IF NOT EXISTS dialysis_sessions (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 session_date DATE NOT NULL DEFAULT CURRENT_DATE,
 dialysis_type VARCHAR(50) NOT NULL DEFAULT 'Hemodialysis',
 duration_hours NUMERIC(4,2),
 ultrafiltration_target_liters NUMERIC(4,2),
 blood_flow_rate_ml_min INTEGER,
 dialysate_flow_rate_ml_min INTEGER,
 pre_weight_kg NUMERIC(5,2),
 post_weight_kg NUMERIC(5,2),
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);

```

```

-- Upgrade existing table if it already existed with older schema

```

```

ALTER TABLE dialysis_sessions ADD COLUMN IF NOT EXISTS dialysis_type VARCHAR(50) NOT NULL DEFAULT 'Hemodialysi

```

```

s';
ALTER TABLE dialysis_sessions ADD COLUMN IF NOT EXISTS ultrafiltration_target_liters NUMERIC(4,2);
ALTER TABLE dialysis_sessions ADD COLUMN IF NOT EXISTS blood_flow_rate_ml_min INTEGER;
ALTER TABLE dialysis_sessions ADD COLUMN IF NOT EXISTS dialysate_flow_rate_ml_min INTEGER;
ALTER TABLE dialysis_sessions ADD COLUMN IF NOT EXISTS pre_weight_kg NUMERIC(5,2);
ALTER TABLE dialysis_sessions ADD COLUMN IF NOT EXISTS post_weight_kg NUMERIC(5,2);

-- Copy legacy data to new columns for compatibility
UPDATE dialysis_sessions SET pre_weight_kg = weight_pre WHERE pre_weight_kg IS NULL AND weight_pre IS NOT NULL
;
UPDATE dialysis_sessions SET post_weight_kg = weight_post WHERE post_weight_kg IS NULL AND weight_post IS NOT
NULL;
UPDATE dialysis_sessions SET ultrafiltration_target_liters = ultrafiltration_volume WHERE ultrafiltration_targ
et_liters IS NULL AND ultrafiltration_volume IS NOT NULL;
UPDATE dialysis_sessions SET blood_flow_rate_ml_min = blood_flow_rate WHERE blood_flow_rate_ml_min IS NULL AND
blood_flow_rate IS NOT NULL;

-- Enable Row-Level Security (RLS)
ALTER TABLE dialysis_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE dialysis_sessions FORCE ROW LEVEL SECURITY;

-- Create RLS policy
DROP POLICY IF EXISTS rls_dialysis_sessions_tenant_isolation ON dialysis_sessions;
CREATE POLICY rls_dialysis_sessions_tenant_isolation ON dialysis_sessions
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create index
CREATE INDEX IF NOT EXISTS idx_dialysis_sessions_patient ON dialysis_sessions (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE dialysis_sessions TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE dialysis_sessions_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE dialysis_sessions TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE dialysis_sessions_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;

=====
FILE: migrations\e37_nephrology_module_validate.sql
=====

```

```

-- Validation: Verify nephrology tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check dialysis_sessions table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'dialysis_sessions'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table dialysis_sessions does not exist';
 END IF;

 -- 2. Check RLS on dialysis_sessions
 SELECT relrowsecurity INTO v_rls_enabled
 FROM pg_class WHERE relname = 'dialysis_sessions';
 IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on dialysis_sessions';
 END IF;

 -- 3. Check index
 SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'dialysis_sessions' AND indexname = 'idx_dialysis_sessions_patient'
) INTO v_index_exists;
 IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_dialysis_sessions_patient does not exist';
 END IF;

 RAISE NOTICE 'Validation passed: Nephrology module tables and RLS are fully set up';
END;
$$;

```

```

=====
FILE: migrations\e38_opthalmology_module_down.sql
=====

```

```

-- Rollback: Drop ophthalmology tables and RLS policies.
BEGIN;

DROP TABLE IF EXISTS eye_exams CASCADE;

COMMIT;

```

```

=====
FILE: migrations\e38_opthalmology_module_up.sql

```

=====

-- Migration: Create ophthalmology eye\_exams table with RLS and conditional grants.

BEGIN;

-- Create eye\_exams table

CREATE TABLE IF NOT EXISTS eye\_exams (

id SERIAL PRIMARY KEY,

patient\_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,

doctor\_id INTEGER REFERENCES system\_users(id) ON DELETE SET NULL,

exam\_date DATE NOT NULL DEFAULT CURRENT\_DATE,

-- Visual Acuity (Snellen or Decimal)

od\_va\_uncorrected VARCHAR(20), -- Right Eye Uncorrected Visual Acuity

os\_va\_uncorrected VARCHAR(20), -- Left Eye Uncorrected Visual Acuity

od\_va\_corrected VARCHAR(20), -- Right Eye Corrected Visual Acuity

os\_va\_corrected VARCHAR(20), -- Left Eye Corrected Visual Acuity

-- Intraocular Pressure (IOP) in mmHg

od\_iop NUMERIC(4,1), -- Right Eye IOP

os\_iop NUMERIC(4,1), -- Left Eye IOP

iop\_method VARCHAR(50) DEFAULT 'Goldmann', -- Method: Goldmann, Tonopen, Airpuff

-- Refraction Parameters

od\_sphere NUMERIC(4,2),

os\_sphere NUMERIC(4,2),

od\_cylinder NUMERIC(4,2),

os\_cylinder NUMERIC(4,2),

od\_axis INTEGER CHECK (od\_axis IS NULL OR (od\_axis >= 0 AND od\_axis <= 180)),

os\_axis INTEGER CHECK (os\_axis IS NULL OR (os\_axis >= 0 AND os\_axis <= 180)),

-- Slit Lamp & Fundoscopy Findings

slit\_lamp\_exam TEXT DEFAULT '',

fundoscopy\_exam TEXT DEFAULT '',

notes TEXT DEFAULT '',

created\_at TIMESTAMP DEFAULT CURRENT\_TIMESTAMP,

tenant\_id INTEGER NOT NULL,

facility\_id INTEGER

);

-- Enable Row-Level Security (RLS)

ALTER TABLE eye\_exams ENABLE ROW LEVEL SECURITY;

ALTER TABLE eye\_exams FORCE ROW LEVEL SECURITY;

-- Create RLS policy

DROP POLICY IF EXISTS rls\_eye\_exams\_tenant\_isolation ON eye\_exams;

CREATE POLICY rls\_eye\_exams\_tenant\_isolation ON eye\_exams

FOR ALL

USING (tenant\_id = NULLIF(current\_setting('app.tenant\_id', true), '')::integer)

WITH CHECK (tenant\_id = NULLIF(current\_setting('app.tenant\_id', true), '')::integer);

-- Create index

CREATE INDEX IF NOT EXISTS idx\_eye\_exams\_patient ON eye\_exams (tenant\_id, patient\_id);

```
-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE eye_exams TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE eye_exams_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE eye_exams TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE eye_exams_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;
```

```
=====
FILE: migrations\e38_ophthalmology_module_validate.sql
=====
```

```
-- Validation: Verify ophthalmology tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check eye_exams table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'eye_exams'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table eye_exams does not exist';
 END IF;

 -- 2. Check RLS on eye_exams
 SELECT relrowsecurity INTO v_rls_enabled
 FROM pg_class WHERE relname = 'eye_exams';
 IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on eye_exams';
 END IF;

 -- 3. Check index
 SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'eye_exams' AND indexname = 'idx_eye_exams_patient'
) INTO v_index_exists;
 IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_eye_exams_patient does not exist';
 END IF;
END;
```

```
END IF;

RAISE NOTICE 'Validation passed: Ophthalmology module tables and RLS are fully set up';
END;
$$;
```

```
=====
FILE: migrations\e39_ent_module_down.sql
=====
```

```
-- Rollback: Drop ent tables and RLS policies.
BEGIN;

DROP TABLE IF EXISTS audiogram_records CASCADE;

COMMIT;
```

```
=====
FILE: migrations\e39_ent_module_up.sql
=====
```

```
-- Migration: Create ent audiogram_records table with RLS and conditional grants.
BEGIN;
```

```
-- Create audiogram_records table
CREATE TABLE IF NOT EXISTS audiogram_records (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 test_date DATE NOT NULL DEFAULT CURRENT_DATE,
```

```
-- Right Ear Air Conduction (AC) Thresholds in dB
```

```
right_ac_250 INTEGER,
right_ac_500 INTEGER,
right_ac_1000 INTEGER,
right_ac_2000 INTEGER,
right_ac_4000 INTEGER,
right_ac_8000 INTEGER,
```

```
-- Left Ear Air Conduction (AC) Thresholds in dB
```

```
left_ac_250 INTEGER,
left_ac_500 INTEGER,
left_ac_1000 INTEGER,
left_ac_2000 INTEGER,
left_ac_4000 INTEGER,
left_ac_8000 INTEGER,
```

```
-- Bone Conduction (BC) Thresholds in dB
```

```
right_bc_250 INTEGER,
right_bc_500 INTEGER,
```

```

right_bc_1000 INTEGER,
right_bc_2000 INTEGER,
right_bc_4000 INTEGER,

left_bc_250 INTEGER,
left_bc_500 INTEGER,
left_bc_1000 INTEGER,
left_bc_2000 INTEGER,
left_bc_4000 INTEGER,

-- Speech Audiometry
right_srt INTEGER,
left_srt INTEGER,
right_sd_score INTEGER CHECK (right_sd_score IS NULL OR (right_sd_score >= 0 AND right_sd_score <= 100)),
left_sd_score INTEGER CHECK (left_sd_score IS NULL OR (left_sd_score >= 0 AND left_sd_score <= 100)),

-- Otoscopy & Tympanometry Findings
otoscopy_right TEXT DEFAULT '',
otoscopy_left TEXT DEFAULT '',
tympanometry_right VARCHAR(50),
tympanometry_left VARCHAR(50),

interpretation TEXT DEFAULT '',
notes TEXT DEFAULT '',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
tenant_id INTEGER NOT NULL,
facility_id INTEGER
);

-- Enable Row-Level Security (RLS)
ALTER TABLE audiogram_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE audiogram_records FORCE ROW LEVEL SECURITY;

-- Create RLS policy
DROP POLICY IF EXISTS rls_audiogram_records_tenant_isolation ON audiogram_records;
CREATE POLICY rls_audiogram_records_tenant_isolation ON audiogram_records
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create index
CREATE INDEX IF NOT EXISTS idx_audiogram_records_patient ON audiogram_records (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE audiogram_records TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE audiogram_records_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE audiogram_records TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE audiogram_records_id_seq TO nama_medical_app';
 END IF;
END;

```

```
END IF;

END

$$;
```

```
COMMIT;
```

```
=====
FILE: migrations\e39_ent_module_validate.sql
=====
```

```
-- Validation: Verify ent tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check audiogram_records table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'audiogram_records'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table audiogram_records does not exist';
 END IF;

 -- 2. Check RLS on audiogram_records
 SELECT relrowsecurity INTO v_rls_enabled
 FROM pg_class WHERE relname = 'audiogram_records';
 IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on audiogram_records';
 END IF;

 -- 3. Check index
 SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'audiogram_records' AND indexname = 'idx_audiogram_records_patient'
) INTO v_index_exists;
 IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_audiogram_records_patient does not exist';
 END IF;

 RAISE NOTICE 'Validation passed: ENT module tables and RLS are fully set up';
END;

$$;
```

```
=====
FILE: migrations\e3_01_lab_samples_down.sql
=====
```



```

-- e3_01_lab_samples_down.sql (rollback of e3_01_lab_samples_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ??????? ????????? (??? ???? ???? ????????? ??????) ?? ??????. idempotent (IF EXISTS).
BEGIN;

DROP POLICY IF EXISTS rls_lab_samples_tenant_isolation ON lab_samples;
DROP INDEX IF EXISTS uq_lab_samples_tenant_barcode;
DROP INDEX IF EXISTS idx_lab_samples_patient_id;
DROP INDEX IF EXISTS idx_lab_samples_lab_order_id;
DROP INDEX IF EXISTS idx_lab_samples_tenant_id;
ALTER TABLE IF EXISTS lab_samples DROP CONSTRAINT IF EXISTS chk_lab_samples_state;
DROP TABLE IF EXISTS lab_samples;

COMMIT;

=====
FILE: migrations\e3_01_lab_samples_up.sql
=====

-- =====
-- e3_01_lab_samples_up.sql
-- E3 LABORATORY / LIS ? specimen lifecycle (1 of group E3).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ???? ???? ???? ?????? ?????????: Collected -> Received -> InProcess
-- -> Reported (?? Rejected). (????? Verified ???? ???? ????????? lab_results.status ?? ??????.)
-- ?? ???? ???? ?????? ???? ???? ???? ????????? (lab_order_id
-- ????? lab_radiology_orders.id ?????? ???? ?????? E-X ?????? orders.id ?? ?????? 'lab').
-- ????? ???? patient_id ???? ???? ????????? ?? ?????? ????? encounters ????.
-- ??? ???? ??? 150 ?????? FORCE RLS: tenant_id NOT NULL REFERENCES tenants(id)
-- + ENABLE+FORCE RLS + ?????? ???? tenant_id + ?????? tenant_id.
-- barcode ???? ???? ?????? (UNIQUE (tenant_id, barcode)) ? ???? ?????? ?????????? ?????????? HL7.
--
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP/ADD CONSTRAINT IF EXISTS ??? CHECK
-- + CREATE [UNIQUE] INDEX IF NOT EXISTS + DROP POLICY IF EXISTS.
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS lab_samples (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 lab_order_id INTEGER,
 patient_id INTEGER,
 barcode TEXT NOT NULL,
 state TEXT NOT NULL DEFAULT 'Collected',
 collected_by INTEGER,
 collected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 received_by INTEGER,
 seq})
-- -> lab_radiology_orders.id (legacy) / orders.id
-- server-generated specimen barcode (LAB-{order}-{

```

```

received_at TIMESTAMP,
rejected_reason TEXT,
rejected_by INTEGER,
rejected_at TIMESTAMP,
notes TEXT DEFAULT '',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
-- state set limited to REACHABLE states only; 'Verified' was unreachable (no transition advances
-- a sample to 'Verified' ? verification is a RESULT concept, samples go ...InProgress->Reported).
CONSTRAINT chk_lab_samples_state CHECK (state IN ('Collected','Received','InProgress','Reported','Rejected'
))
);

```

```

-- idempotent: ensure the state CHECK exists even when the table pre-dates this migration.
ALTER TABLE lab_samples DROP CONSTRAINT IF EXISTS chk_lab_samples_state;
ALTER TABLE lab_samples ADD CONSTRAINT chk_lab_samples_state CHECK (state IN ('Collected','Received','InProgress','Reported','Rejected'));

```

```

CREATE INDEX IF NOT EXISTS idx_lab_samples_tenant_id ON lab_samples (tenant_id);
CREATE INDEX IF NOT EXISTS idx_lab_samples_lab_order_id ON lab_samples (lab_order_id);
CREATE INDEX IF NOT EXISTS idx_lab_samples_patient_id ON lab_samples (patient_id);
-- barcode unique PER tenant (not globally) so two tenants may reuse a serial namespace safely.
CREATE UNIQUE INDEX IF NOT EXISTS uq_lab_samples_tenant_barcode ON lab_samples (tenant_id, barcode);

```

```

ALTER TABLE lab_samples ENABLE ROW LEVEL SECURITY;
ALTER TABLE lab_samples FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_lab_samples_tenant_isolation ON lab_samples;
CREATE POLICY rls_lab_samples_tenant_isolation ON lab_samples
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

```

```

COMMIT;

```

```

=====
FILE: migrations\e3_01_lab_samples_validate.sql
=====

```

```

-- e3_01_lab_samples_validate.sql (run AFTER e3_01_lab_samples_up.sql; read-only)
-- PASS = lab_samples exists with tenant_id FK + FORCE RLS + isolation policy + indexes
-- + state CHECK + per-tenant unique barcode.

```

```

SELECT
 (SELECT count(*) FROM information_schema.tables WHERE table_name='lab_samples') AS lab_samples_exists,
 -- expect 1
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='lab_samples'
 AND column_name IN ('tenant_id','facility_id','lab_order_id','patient_id','barcode','state',
 'collected_by','collected_at','received_by','received_at','rejected_reason')) AS lab_samples_cols, -- expect 11
 (SELECT reforcerowsecurity FROM pg_class WHERE relname='lab_samples') AS lab_samples_force_rls,
 -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='lab_samples' AND policyname='rls_lab_samples_tenant_isola

```

```

tion') AS lab_samples_policy, -- expect 1
 (SELECT count(*) FROM pg_constraint WHERE conname='chk_lab_samples_state') AS lab_samples_state_check,
 -- expect 1
 (SELECT count(*) FROM pg_indexes WHERE tablename='lab_samples' AND indexname='idx_lab_samples_tenant_id') AS
lab_samples_tenant_idx, -- expect 1
 (SELECT count(*) FROM pg_indexes WHERE tablename='lab_samples' AND indexname='uq_lab_samples_tenant_barcode'
) AS lab_samples_barcode_uq; -- expect 1

-- ----- tenant_id is NOT NULL -----
SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
 WHERE table_name='lab_samples' AND column_name='tenant_id' AND is_nullable='NO';
 -- expect 1

-- ----- FK to tenants(id) -----
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
 WHERE conrelid='lab_samples'::regclass AND confrelid='tenants'::regclass AND contype='f';
 -- expect 1

=====
FILE: migrations\e3_02_lab_results_down.sql
=====

-- e3_02_lab_results_down.sql (rollback of e3_02_lab_results_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
--
-- ?????? ?????: lab_results ???? ???? ????? ?? db_postgres.js (???? order_id/test_id/
-- result_value/is_abnormal/notes). ???? ?? ???? ??????? ???? ?? ?? ????? up:
-- ??????? + ??????? ??????? + ???? CHECK/FK + ??????? ????????? ????????? ??????
-- ?????? ?????? ??? ???. idempotent (IF EXISTS).
BEGIN;

-- child table first (FK -> lab_results)
DROP POLICY IF EXISTS rls_lab_callbacks_tenant_isolation ON lab_critical_callbacks;
DROP INDEX IF EXISTS idx_lab_callbacks_result_id;
DROP INDEX IF EXISTS idx_lab_callbacks_tenant_id;
DROP TABLE IF EXISTS lab_critical_callbacks;

DROP POLICY IF EXISTS rls_lab_results_tenant_isolation ON lab_results;
-- (RLS itself left enabled is harmless without a policy, but we disable to fully restore prior state.)
ALTER TABLE IF EXISTS lab_results NO FORCE ROW LEVEL SECURITY;
ALTER TABLE IF EXISTS lab_results DISABLE ROW LEVEL SECURITY;

DROP INDEX IF EXISTS idx_lab_results_order_id;
DROP INDEX IF EXISTS idx_lab_results_lab_sample_id;
DROP INDEX IF EXISTS idx_lab_results_tenant_id;

ALTER TABLE IF EXISTS lab_results DROP CONSTRAINT IF EXISTS chk_lab_results_status;
ALTER TABLE IF EXISTS lab_results DROP CONSTRAINT IF EXISTS fk_lab_results_tenant;

ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS reported_at;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS reported;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS verified_at;

```

```

ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS verified_by;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS hold_reasons;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS status;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS is_critical;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS delta_pct;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS abnormal_flag;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS ref_high;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS ref_low;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS normal_range;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS unit;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS value;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS test_name;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS loinc;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS lab_sample_id;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS facility_id;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS tenant_id;
ALTER TABLE IF EXISTS lab_results DROP COLUMN IF EXISTS created_at;

```

```

COMMIT;

```

```

=====
FILE: migrations\e3_02_lab_results_up.sql
=====

```

```

-- =====
-- e3_02_lab_results_up.sql
-- E3 LABORATORY / LIS ? structured, LOINC-coded results (2 of group E3).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? lab_results ?? ?????? ?????? (order_id, test_id, result_value, is_abnormal,
-- notes) ??? ?????? ??????? ??????? LOINC ?? ?????? ?????? ?????? ?????? (verification).
-- ADDITIVE: ?????? ??? ??????? ??????? (?? ???) ?????? ??????? ??????? ???
-- ADD COLUMN IF NOT EXISTS ??? ?? ??? ??????? ??????? ?????? (CREATE TABLE IF NOT EXISTS
-- ?????? ??????? ??????? ??? ??? ????).
--
-- tenant_id: ????? ?? ????? NOT NULL ??? ????? ?? ??? ??? NULL (????? ???/? ???
-- ??????). ?? ????? ??? ?????? ??? tenant_id ??? ?????? NULLABLE (?? ????? ??????)
-- ??? ????? ???????) ? ?????? ??? backfill ????? ??? NOT NULL.
--
-- idempotent: CREATE TABLE IF NOT EXISTS + ADD COLUMN IF NOT EXISTS + DROP/ADD CONSTRAINT
-- IF EXISTS + CREATE INDEX IF NOT EXISTS + DROP POLICY IF EXISTS + DO-block guards.
-- =====
BEGIN;

-- Base table (created only if it does not already exist; legacy orphan keeps its columns).
CREATE TABLE IF NOT EXISTS lab_results (
 id SERIAL PRIMARY KEY,
 order_id INTEGER,
 test_id INTEGER,
 result_value TEXT DEFAULT '',
 is_abnormal INTEGER DEFAULT 0,
 notes TEXT DEFAULT ''

```

```

);

-- ---- additive structured columns (LIS) ----
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS facility_id INTEGER;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS lab_sample_id INTEGER;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS loinc TEXT;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS test_name TEXT DEFAULT '';
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS value TEXT DEFAULT '';
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS unit TEXT DEFAULT '';
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS normal_range TEXT DEFAULT '';
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS ref_low REAL;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS ref_high REAL;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS abnormal_flag TEXT; -- 'N'|'L'|'H'|'HH'|'LL'
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS delta_pct REAL;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS is_critical INTEGER DEFAULT 0;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS status TEXT DEFAULT 'pending'; -- pending|held|verified
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS hold_reasons TEXT DEFAULT ''; -- comma list from lis.autoVerify
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS verified_by INTEGER;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS verified_at TIMESTAMP;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS reported INTEGER DEFAULT 0; -- 1 once released to chart
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS reported_at TIMESTAMP;
ALTER TABLE lab_results ADD COLUMN IF NOT EXISTS created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;

-- status CHECK (DROP then ADD; idempotent and tolerant of pre-existing rows == default 'pending').
ALTER TABLE lab_results DROP CONSTRAINT IF EXISTS chk_lab_results_status;
ALTER TABLE lab_results ADD CONSTRAINT chk_lab_results_status CHECK (status IN ('pending','held','verified'));

-- FK to tenants(id) ? added only if not present yet (legacy orphan had no tenant_id).
DO $$
BEGIN
 IF NOT EXISTS (
 SELECT 1 FROM pg_constraint
 WHERE conrelid='lab_results'::regclass AND confrelid='tenants'::regclass AND contype='f'
) THEN
 ALTER TABLE lab_results
 ADD CONSTRAINT fk_lab_results_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
 END IF;
END $$;

-- Enforce tenant_id NOT NULL ONLY when safe (no NULL rows). FAIL-SAFE: never coerce / never guess.
DO $$
BEGIN
 IF NOT EXISTS (SELECT 1 FROM lab_results WHERE tenant_id IS NULL) THEN
 BEGIN
 ALTER TABLE lab_results ALTER COLUMN tenant_id SET NOT NULL;
 EXCEPTION WHEN others THEN
 RAISE NOTICE 'lab_results.tenant_id left NULLABLE (could not set NOT NULL): %', SQLERRM;
 END;
 ELSE
 RAISE NOTICE 'lab_results has NULL tenant_id rows; left NULLABLE pending backfill before NOT NULL.';
 END IF;
END $$;

```

```
END $$;
```

```
CREATE INDEX IF NOT EXISTS idx_lab_results_tenant_id ON lab_results (tenant_id);
CREATE INDEX IF NOT EXISTS idx_lab_results_lab_sample_id ON lab_results (lab_sample_id);
CREATE INDEX IF NOT EXISTS idx_lab_results_order_id ON lab_results (order_id);
```

```
ALTER TABLE lab_results ENABLE ROW LEVEL SECURITY;
ALTER TABLE lab_results FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_lab_results_tenant_isolation ON lab_results;
CREATE POLICY rls_lab_results_tenant_isolation ON lab_results
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
-- =====
```

```
-- lab_critical_callbacks ? documented critical-value call-back log (CLINICAL SAFETY).
```

```
-- A critical result CANNOT be released to 'Reported' until a call-back row exists
```

```
-- (who notified whom, and when) ? enforced server-side. Greenfield child table.
```

```
-- =====
```

```
CREATE TABLE IF NOT EXISTS lab_critical_callbacks (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 result_id INTEGER NOT NULL REFERENCES lab_results(id) ON DELETE CASCADE,
 notified_to TEXT NOT NULL, -- physician / responsible party notified
 notified_by INTEGER, -- staff user id who made the call-back
 notified_by_name TEXT DEFAULT '',
 notified_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 ack INTEGER DEFAULT 0, -- 1 once the recipient acknowledged read-back
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE INDEX IF NOT EXISTS idx_lab_callbacks_tenant_id ON lab_critical_callbacks (tenant_id);
CREATE INDEX IF NOT EXISTS idx_lab_callbacks_result_id ON lab_critical_callbacks (result_id);
```

```
ALTER TABLE lab_critical_callbacks ENABLE ROW LEVEL SECURITY;
ALTER TABLE lab_critical_callbacks FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_lab_callbacks_tenant_isolation ON lab_critical_callbacks;
CREATE POLICY rls_lab_callbacks_tenant_isolation ON lab_critical_callbacks
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e3_02_lab_results_validate.sql
```

```
=====
```

```
-- e3_02_lab_results_validate.sql (run AFTER e3_02_lab_results_up.sql; read-only)
```

```
-- PASS = lab_results has structured LIS columns + FORCE RLS + isolation policy + status CHECK
```

```

-- + tenant_id index + FK to tenants. (tenant_id NOT NULL is enforced only when the
-- table had no NULL rows; we report both the column nullability and the row count.)

SELECT
 (SELECT count(*) FROM information_schema.tables WHERE table_name='lab_results') AS lab_results_exists,
 -- expect 1
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='lab_results'
 AND column_name IN ('tenant_id','lab_sample_id','loinc','test_name','value','unit','normal_range',
 'ref_low','ref_high','abnormal_flag','delta_pct','is_critical','status',
 'verified_by','verified_at','reported')) AS lab_results_lis_cols,
 -- expect 16
 (SELECT reforcerowsecurity FROM pg_class WHERE relname='lab_results') AS lab_results_force_rls,
 -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='lab_results' AND policyname='rls_lab_results_tenant_isola
tion') AS lab_results_policy, -- expect 1
 (SELECT count(*) FROM pg_constraint WHERE conname='chk_lab_results_status') AS lab_results_status_check,
 -- expect 1
 (SELECT count(*) FROM pg_indexes WHERE tablename='lab_results' AND indexname='idx_lab_results_tenant_id') AS
lab_results_tenant_idx; -- expect 1

-- ----- FK to tenants(id) -----
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
 WHERE conrelid='lab_results'::regclass AND confrelid='tenants'::regclass AND contype='f';
 -- expect 1

-- ----- tenant_id nullability + NULL row count (NOT NULL only when 0 NULL rows) -----
SELECT
 (SELECT is_nullable FROM information_schema.columns WHERE table_name='lab_results' AND column_name='tenant_i
d') AS tenant_id_nullable, -- 'NO' when enforced
 (SELECT count(*) FROM lab_results WHERE tenant_id IS NULL) AS null_tenant_rows;
 -- expect 0 in fresh/empty

-- ----- lab_critical_callbacks (critical call-back log) -----
SELECT
 (SELECT count(*) FROM information_schema.tables WHERE table_name='lab_critical_callbacks') AS callbacks_exis
ts, -- expect 1
 (SELECT reforcerowsecurity FROM pg_class WHERE relname='lab_critical_callbacks') AS callbacks_force_rls,
 -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='lab_critical_callbacks' AND policyname='rls_lab_callbacks
_tenant_isolation') AS callbacks_policy, -- expect 1
 (SELECT count(*) FROM pg_indexes WHERE tablename='lab_critical_callbacks' AND indexname='idx_lab_callbacks_t
enant_id') AS callbacks_tenant_idx, -- expect 1
 (SELECT count(*) FROM information_schema.columns WHERE table_name='lab_critical_callbacks' AND column_name='
tenant_id' AND is_nullable='NO') AS callbacks_tenant_not_null; -- expect 1

=====
FILE: migrations\e3_03_lab_qc_down.sql
=====

-- e3_03_lab_qc_down.sql (rollback of e3_03_lab_qc_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.

```

```

-- ???? ?????? ???????? ?? ??????. idempotent (IF EXISTS).
BEGIN;

DROP POLICY IF EXISTS rls_lab_qc_tenant_isolation ON lab_qc;
DROP INDEX IF EXISTS idx_lab_qc_analyzer;
DROP INDEX IF EXISTS idx_lab_qc_tenant_id;
DROP TABLE IF EXISTS lab_qc;

COMMIT;

=====
FILE: migrations\e3_03_lab_qc_up.sql
=====

-- =====
-- e3_03_lab_qc_up.sql
-- E3 LABORATORY / LIS ? quality control (Levey-Jennings) (3 of group E3).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? ???? ??? ?????? ??? ?????/????/???? ?? ????? (target) ????????? ????????
-- (sd) ?????? ????????? (value)? ????? ??? Westgard (??? 1-3s) ??? ??????. westgard_flag
-- ????? ?? lis.qcFlag ?? ???? ??????.
-- ??? ???? ??? 150 ????? FORCE RLS: tenant_id NOT NULL REFERENCES tenants(id)
-- + ENABLE+FORCE RLS + ????? ??? tenant_id + ???? tenant_id.
--
-- idempotent: CREATE TABLE IF NOT EXISTS + CREATE INDEX IF NOT EXISTS + DROP POLICY IF EXISTS.
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS lab_qc (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 analyzer TEXT NOT NULL DEFAULT '',
 analyte TEXT NOT NULL DEFAULT '',
 level TEXT NOT NULL DEFAULT '',
 value REAL,
 target REAL,
 sd REAL,
 z REAL,
 westgard_flag TEXT DEFAULT '',
 d_qc_inputs'
 breach INTEGER DEFAULT 0,
 reagent_lot TEXT DEFAULT '',
 entered_by INTEGER,
 at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_lab_qc_tenant_id ON lab_qc (tenant_id);
CREATE INDEX IF NOT EXISTS idx_lab_qc_analyzer ON lab_qc (analyzer);

```



```

ALTER TABLE lab_qc ENABLE ROW LEVEL SECURITY;
ALTER TABLE lab_qc FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_lab_qc_tenant_isolation ON lab_qc;
CREATE POLICY rls_lab_qc_tenant_isolation ON lab_qc
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

```

=====
FILE: migrations\e3_03_lab_qc_validate.sql
=====

```

```

-- e3_03_lab_qc_validate.sql (run AFTER e3_03_lab_qc_up.sql; read-only)
-- PASS = lab_qc exists with tenant_id FK (NOT NULL) + FORCE RLS + isolation policy + tenant index.

```

```

SELECT
 (SELECT count(*) FROM information_schema.tables WHERE table_name='lab_qc') AS lab_qc_exists,
 -- expect 1
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='lab_qc'
 AND column_name IN ('tenant_id','analyzer','analyte','level','value','target','sd','z','westgard_flag',
'breach','at')) AS lab_qc_cols, -- expect 11
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='lab_qc') AS lab_qc_force_rls,
 -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='lab_qc' AND policyname='rls_lab_qc_tenant_isolation') AS
lab_qc_policy, -- expect 1
 (SELECT count(*) FROM pg_indexes WHERE tablename='lab_qc' AND indexname='idx_lab_qc_tenant_id') AS lab_qc_te
nant_idx; -- expect 1

-- ----- tenant_id is NOT NULL -----
SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
 WHERE table_name='lab_qc' AND column_name='tenant_id' AND is_nullable='NO';
 -- expect 1

-- ----- FK to tenants(id) -----
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
 WHERE conrelid='lab_qc'::regclass AND confrelid='tenants'::regclass AND contype='f';
 -- expect 1

```

```

=====
FILE: migrations\e40_plastic_burns_down.sql
=====

```

```

-- Rollback: Drop plastic_burns tables and RLS policies.
BEGIN;

DROP TABLE IF EXISTS burn_assessments CASCADE;
DROP TABLE IF EXISTS clinical_photos_meta CASCADE;

```

```
COMMIT;
```

```
=====
FILE: migrations\e40_plastic_burns_up.sql
=====
```

```
-- Migration: Create plastic_burns tables with RLS and conditional grants.
BEGIN;
```

```
-- Create burn_assessments table
```

```
CREATE TABLE IF NOT EXISTS burn_assessments (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 assessment_date DATE NOT NULL DEFAULT CURRENT_DATE,
 weight_kg DECIMAL NOT NULL,

 -- Body region burn percentages (Rule of Nines)
 head_percent DECIMAL DEFAULT 0,
 torso_front_percent DECIMAL DEFAULT 0,
 torso_back_percent DECIMAL DEFAULT 0,
 left_arm_percent DECIMAL DEFAULT 0,
 right_arm_percent DECIMAL DEFAULT 0,
 left_leg_percent DECIMAL DEFAULT 0,
 right_leg_percent DECIMAL DEFAULT 0,
 perineum_percent DECIMAL DEFAULT 0,

 tbsa_percent DECIMAL NOT NULL, -- Total Body Surface Area
 parkland_fluid_ml DECIMAL NOT NULL, -- Calculated 24h fluid requirement
 fluid_first_8h_ml DECIMAL NOT NULL,
 fluid_next_16h_ml DECIMAL NOT NULL,

 clinical_notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

```
-- Create clinical_photos_meta table
```

```
CREATE TABLE IF NOT EXISTS clinical_photos_meta (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 photo_date DATE NOT NULL DEFAULT CURRENT_DATE,
 body_region VARCHAR(100) NOT NULL,
 description TEXT DEFAULT '',
 is_confidential BOOLEAN DEFAULT TRUE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

```

-- Enable Row-Level Security (RLS)
ALTER TABLE burn_assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE burn_assessments FORCE ROW LEVEL SECURITY;

ALTER TABLE clinical_photos_meta ENABLE ROW LEVEL SECURITY;
ALTER TABLE clinical_photos_meta FORCE ROW LEVEL SECURITY;

-- Create RLS policies
DROP POLICY IF EXISTS rls_burn_assessments_tenant_isolation ON burn_assessments;
CREATE POLICY rls_burn_assessments_tenant_isolation ON burn_assessments
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

DROP POLICY IF EXISTS rls_clinical_photos_meta_tenant_isolation ON clinical_photos_meta;
CREATE POLICY rls_clinical_photos_meta_tenant_isolation ON clinical_photos_meta
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create indexes
CREATE INDEX IF NOT EXISTS idx_burn_assessments_patient ON burn_assessments (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_clinical_photos_meta_patient ON clinical_photos_meta (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE burn_assessments, clinical_photos_meta TO jumanasoft_staging_us
er';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE burn_assessments_id_seq, clinical_photos_meta_id_seq TO juman
asoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE burn_assessments, clinical_photos_meta TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE burn_assessments_id_seq, clinical_photos_meta_id_seq TO nama_
medical_app';
 END IF;
END
$$;

COMMIT;

=====
FILE: migrations\e40_plastic_burns_validate.sql
=====

-- Validation: Verify plastic_burns tables, columns, RLS policies, and indexes.
DO $$
DECLARE

```

```

v_table_exists BOOLEAN;
v_rls_enabled BOOLEAN;
v_index_exists BOOLEAN;
BEGIN
-- 1. Check burn_assessments table
SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'burn_assessments'
) INTO v_table_exists;
IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table burn_assessments does not exist';
END IF;

-- 2. Check clinical_photos_meta table
SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'clinical_photos_meta'
) INTO v_table_exists;
IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table clinical_photos_meta does not exist';
END IF;

-- 3. Check RLS on burn_assessments
SELECT relrowsecurity INTO v_rls_enabled
FROM pg_class WHERE relname = 'burn_assessments';
IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on burn_assessments';
END IF;

-- 4. Check RLS on clinical_photos_meta
SELECT relrowsecurity INTO v_rls_enabled
FROM pg_class WHERE relname = 'clinical_photos_meta';
IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on clinical_photos_meta';
END IF;

-- 5. Check index on burn_assessments
SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'burn_assessments' AND indexname = 'idx_burn_assessments_p
atient'
) INTO v_index_exists;
IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_burn_assessments_patient does not exist';
END IF;

-- 6. Check index on clinical_photos_meta
SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'clinical_photos_meta' AND indexname = 'idx_clinical_photo
s_meta_patient'
) INTO v_index_exists;
IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_clinical_photos_meta_patient does not exist';

```

```
END IF;

RAISE NOTICE 'Validation passed: Plastic & Burns module tables and RLS are fully set up';
END;
$$;
```

```
=====
FILE: migrations\e41_intensive_care_down.sql
=====
```

```
-- Rollback: Drop intensive_care tables and RLS policies.
BEGIN;
```

```
DROP TABLE IF EXISTS icu_assessments CASCADE;
```

```
COMMIT;
```

```
=====
FILE: migrations\e41_intensive_care_up.sql
=====
```

```
-- Migration: Create intensive_care tables with RLS and conditional grants.
BEGIN;
```

```
-- Create icu_assessments table
```

```
CREATE TABLE IF NOT EXISTS icu_assessments (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 assessment_date DATE NOT NULL DEFAULT CURRENT_DATE,
```

```
-- APACHE II physiological variable points
```

```
apache_temp INTEGER NOT NULL DEFAULT 0,
apache_map INTEGER NOT NULL DEFAULT 0,
apache_hr INTEGER NOT NULL DEFAULT 0,
apache_rr INTEGER NOT NULL DEFAULT 0,
apache_pao2 INTEGER NOT NULL DEFAULT 0,
apache_ph INTEGER NOT NULL DEFAULT 0,
apache_na INTEGER NOT NULL DEFAULT 0,
apache_k INTEGER NOT NULL DEFAULT 0,
apache_creatinine INTEGER NOT NULL DEFAULT 0,
apache_hct INTEGER NOT NULL DEFAULT 0,
apache_wbc INTEGER NOT NULL DEFAULT 0,
apache_gcs INTEGER NOT NULL DEFAULT 0,
```

```
-- APACHE II age & chronic health points
```

```
apache_age_points INTEGER NOT NULL DEFAULT 0,
apache_chronic_points INTEGER NOT NULL DEFAULT 0,
apache_ii_score INTEGER NOT NULL DEFAULT 0,
```

```

-- SOFA organ dysfunction points
sofa_pao2_fio2 INTEGER NOT NULL DEFAULT 0,
sofa_platelets INTEGER NOT NULL DEFAULT 0,
sofa_bilirubin INTEGER NOT NULL DEFAULT 0,
sofa_map_vasopressor INTEGER NOT NULL DEFAULT 0,
sofa_gcs INTEGER NOT NULL DEFAULT 0,
sofa_creatinine INTEGER NOT NULL DEFAULT 0,
sofa_score INTEGER NOT NULL DEFAULT 0,

clinical_notes TEXT DEFAULT '',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
tenant_id INTEGER NOT NULL,
facility_id INTEGER
);

-- Enable Row-Level Security (RLS)
ALTER TABLE icu_assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE icu_assessments FORCE ROW LEVEL SECURITY;

-- Create RLS policy
DROP POLICY IF EXISTS rls_icu_assessments_tenant_isolation ON icu_assessments;
CREATE POLICY rls_icu_assessments_tenant_isolation ON icu_assessments
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create index
CREATE INDEX IF NOT EXISTS idx_icu_assessments_patient ON icu_assessments (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE icu_assessments TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE icu_assessments_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE icu_assessments TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE icu_assessments_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;

=====
FILE: migrations\e41_intensive_care_validate.sql
=====

-- Validation: Verify intensive_care tables, columns, RLS policies, and indexes.
DO $$

```

```

DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check icu_assessments table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'icu_assessments'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table icu_assessments does not exist';
 END IF;

 -- 2. Check RLS on icu_assessments
 SELECT relrowsecurity INTO v_rls_enabled
 FROM pg_class WHERE relname = 'icu_assessments';
 IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on icu_assessments';
 END IF;

 -- 3. Check index on icu_assessments
 SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'icu_assessments' AND indexname = 'idx_icu_assessments_patient'
) INTO v_index_exists;
 IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_icu_assessments_patient does not exist';
 END IF;

 RAISE NOTICE 'Validation passed: Intensive Care module tables and RLS are fully set up';
END;
$$;

```

```

=====
FILE: migrations\e42_orthopedics_down.sql
=====

```

```

-- Rollback: Drop orthopedics tables and RLS policies.
BEGIN;

DROP TABLE IF EXISTS orthopedic_implants CASCADE;
DROP TABLE IF EXISTS joint_rom_assessments CASCADE;

COMMIT;

```

```

=====
FILE: migrations\e42_orthopedics_up.sql
=====

```

```

-- Migration: Create orthopedics tables with RLS and conditional grants.
BEGIN;

-- Create orthopedic_implants table
CREATE TABLE IF NOT EXISTS orthopedic_implants (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 implant_date DATE NOT NULL DEFAULT CURRENT_DATE,
 implant_type VARCHAR(100) NOT NULL,
 manufacturer VARCHAR(100) NOT NULL,
 model_name VARCHAR(100) DEFAULT '',
 serial_number VARCHAR(100) NOT NULL,
 size_dimension VARCHAR(50) DEFAULT '',
 batch_lot_number VARCHAR(100) DEFAULT '',
 clinical_notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);

-- Create joint_rom_assessments table
CREATE TABLE IF NOT EXISTS joint_rom_assessments (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 assessment_date DATE NOT NULL DEFAULT CURRENT_DATE,
 joint_name VARCHAR(50) NOT NULL,
 lateral_side VARCHAR(10) NOT NULL, -- Left, Right, Bilateral
 movement_type VARCHAR(50) NOT NULL, -- Flexion, Extension, Abduction, Adduction, etc.
 angle_degrees INTEGER NOT NULL,
 is_restricted BOOLEAN NOT NULL DEFAULT FALSE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);

-- Enable Row-Level Security (RLS)
ALTER TABLE orthopedic_implants ENABLE ROW LEVEL SECURITY;
ALTER TABLE orthopedic_implants FORCE ROW LEVEL SECURITY;

ALTER TABLE joint_rom_assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE joint_rom_assessments FORCE ROW LEVEL SECURITY;

-- Create RLS policies
DROP POLICY IF EXISTS rls_orthopedic_implants_tenant_isolation ON orthopedic_implants;
CREATE POLICY rls_orthopedic_implants_tenant_isolation ON orthopedic_implants
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

DROP POLICY IF EXISTS rls_joint_rom_assessments_tenant_isolation ON joint_rom_assessments;
CREATE POLICY rls_joint_rom_assessments_tenant_isolation ON joint_rom_assessments

```



```

FOR ALL
USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create indexes
CREATE INDEX IF NOT EXISTS idx_orthopedic_implants_patient ON orthopedic_implants (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_joint_rom_assessments_patient ON joint_rom_assessments (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE orthopedic_implants, joint_rom_assessments TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE orthopedic_implants_id_seq, joint_rom_assessments_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE orthopedic_implants, joint_rom_assessments TO nama_medical_app';
 ;
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE orthopedic_implants_id_seq, joint_rom_assessments_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;

```

```

=====
FILE: migrations\e42_orthopedics_validate.sql
=====

```

```

-- Validation: Verify orthopedics tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check orthopedic_implants table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'orthopedic_implants'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table orthopedic_implants does not exist';
 END IF;

 -- 2. Check joint_rom_assessments table
 SELECT EXISTS (
 SELECT FROM information_schema.tables

```

```

 WHERE table_schema = 'public' AND table_name = 'joint_rom_assessments'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table joint_rom_assessments does not exist';
 END IF;

-- 3. Check RLS on orthopedic_implants
SELECT relrowsecurity INTO v_rls_enabled
FROM pg_class WHERE relname = 'orthopedic_implants';
IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on orthopedic_implants';
END IF;

-- 4. Check RLS on joint_rom_assessments
SELECT relrowsecurity INTO v_rls_enabled
FROM pg_class WHERE relname = 'joint_rom_assessments';
IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on joint_rom_assessments';
END IF;

-- 5. Check index on orthopedic_implants
SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'orthopedic_implants' AND indexname = 'idx_orthopedic_impl
ants_patient'
) INTO v_index_exists;
IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_orthopedic_implants_patient does not exist';
END IF;

-- 6. Check index on joint_rom_assessments
SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'joint_rom_assessments' AND indexname = 'idx_joint_rom_ass
essments_patient'
) INTO v_index_exists;
IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_joint_rom_assessments_patient does not exist';
END IF;

RAISE NOTICE 'Validation passed: Orthopedics module tables and RLS are fully set up';
END;
$$;

```

```

=====
FILE: migrations\e43_ophthalmology_down.sql
=====

```

```

-- Rollback: Drop ophthalmology tables and RLS policies.
BEGIN;

DROP TABLE IF EXISTS eye_exams CASCADE;

```

```
COMMIT;
```

```
=====
FILE: migrations\e43_ophthalmology_up.sql
=====
```

```
-- Migration: Create ophthalmology tables with RLS and conditional grants.
BEGIN;
```

```
-- Create eye_exams table if not exists
```

```
CREATE TABLE IF NOT EXISTS eye_exams (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 exam_date DATE NOT NULL DEFAULT CURRENT_DATE,
 od_va_uncorrected VARCHAR(10) DEFAULT '',
 os_va_uncorrected VARCHAR(10) DEFAULT '',
 od_va_corrected VARCHAR(10) DEFAULT '',
 os_va_corrected VARCHAR(10) DEFAULT '',
 od_iop NUMERIC DEFAULT 15,
 os_iop NUMERIC DEFAULT 15,
 iop_method VARCHAR(50) DEFAULT '',
 od_sphere NUMERIC DEFAULT 0.00,
 os_sphere NUMERIC DEFAULT 0.00,
 od_cylinder NUMERIC DEFAULT 0.00,
 os_cylinder NUMERIC DEFAULT 0.00,
 od_axis INTEGER DEFAULT 0,
 os_axis INTEGER DEFAULT 0,
 slit_lamp_exam TEXT DEFAULT '',
 funduscopy_exam TEXT DEFAULT '',
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

```
-- Add od_add and os_add if they don't exist
```

```
ALTER TABLE eye_exams ADD COLUMN IF NOT EXISTS od_add NUMERIC DEFAULT 0.00;
ALTER TABLE eye_exams ADD COLUMN IF NOT EXISTS os_add NUMERIC DEFAULT 0.00;
```

```
-- Enable Row-Level Security (RLS)
```

```
ALTER TABLE eye_exams ENABLE ROW LEVEL SECURITY;
ALTER TABLE eye_exams FORCE ROW LEVEL SECURITY;
```

```
-- Create RLS policies
```

```
DROP POLICY IF EXISTS rls_eye_exams_tenant_isolation ON eye_exams;
CREATE POLICY rls_eye_exams_tenant_isolation ON eye_exams
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

```
-- Create indexes
CREATE INDEX IF NOT EXISTS idx_eye_exams_patient ON eye_exams (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE eye_exams TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE eye_exams_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE eye_exams TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE eye_exams_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;
```

```
=====
FILE: migrations\e43_ophthalmology_validate.sql
=====
```

```
-- Validation: Verify ophthalmology tables, columns, RLS policies, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
 v_column_exists BOOLEAN;
BEGIN
 -- 1. Check eye_exams table
 SELECT EXISTS (
 SELECT FROM information_schema.tables
 WHERE table_schema = 'public' AND table_name = 'eye_exams'
) INTO v_table_exists;
 IF NOT v_table_exists THEN
 RAISE EXCEPTION 'Validation failed: table eye_exams does not exist';
 END IF;

 -- 2. Check od_add column
 SELECT EXISTS (
 SELECT FROM information_schema.columns
 WHERE table_name = 'eye_exams' AND column_name = 'od_add'
) INTO v_column_exists;
 IF NOT v_column_exists THEN
 RAISE EXCEPTION 'Validation failed: column od_add does not exist';
 END IF;

 -- 3. Check os_add column
 SELECT EXISTS (
```

```

 SELECT FROM information_schema.columns
 WHERE table_name = 'eye_exams' AND column_name = 'os_add'
) INTO v_column_exists;
 IF NOT v_column_exists THEN
 RAISE EXCEPTION 'Validation failed: column os_add does not exist';
 END IF;

-- 4. Check RLS on eye_exams
SELECT relrowsecurity INTO v_rls_enabled
FROM pg_class WHERE relname = 'eye_exams';
IF NOT v_rls_enabled THEN
 RAISE EXCEPTION 'Validation failed: RLS is not enabled on eye_exams';
END IF;

-- 5. Check index on eye_exams
SELECT EXISTS (
 SELECT FROM pg_indexes
 WHERE schemaname = 'public' AND tablename = 'eye_exams' AND indexname = 'idx_eye_exams_patient'
) INTO v_index_exists;
IF NOT v_index_exists THEN
 RAISE EXCEPTION 'Validation failed: index idx_eye_exams_patient does not exist';
END IF;

RAISE NOTICE 'Validation passed: Ophthalmology module tables and RLS are fully set up';
END;
$$;

```

```

=====
FILE: migrations\e44_surgery_urology_down.sql
=====

```

```

-- Rollback: Drop surgical checklists, time logs, and urodynamic studies tables and RLS.
BEGIN;

```

```

DROP TABLE IF EXISTS surgical_checklists CASCADE;
DROP TABLE IF EXISTS surgical_time_logs CASCADE;
DROP TABLE IF EXISTS urodynamic_studies CASCADE;

```

```

COMMIT;

```

```

=====
FILE: migrations\e44_surgery_urology_up.sql
=====

```

```

-- Migration: Create surgical checklists, time logs, and urodynamic studies tables with RLS and conditional grants.
BEGIN;

```

```

-- 1. Create surgical_checklists table
CREATE TABLE IF NOT EXISTS surgical_checklists (

```

```

id SERIAL PRIMARY KEY,
patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
surgery_date DATE NOT NULL DEFAULT CURRENT_DATE,
procedure_name VARCHAR(255) NOT NULL,
sign_in_confirmed BOOLEAN DEFAULT FALSE,
time_out_confirmed BOOLEAN DEFAULT FALSE,
sign_out_confirmed BOOLEAN DEFAULT FALSE,
notes TEXT DEFAULT '',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
tenant_id INTEGER NOT NULL,
facility_id INTEGER
);

```

-- 2. Create surgical\_time\_logs table

```

CREATE TABLE IF NOT EXISTS surgical_time_logs (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 procedure_name VARCHAR(255) NOT NULL,
 anesthesia_start_time TIMESTAMP,
 incision_time TIMESTAMP,
 closure_time TIMESTAMP,
 anesthesia_end_time TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);

```

-- 3. Create urodynamic\_studies table

```

CREATE TABLE IF NOT EXISTS urodynamic_studies (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 study_date DATE NOT NULL DEFAULT CURRENT_DATE,
 max_flow_rate NUMERIC(5,2) DEFAULT 0.00,
 voided_volume NUMERIC(5,2) DEFAULT 0.00,
 post_void_residual NUMERIC(5,2) DEFAULT 0.00,
 detrusor_pressure NUMERIC(5,2) DEFAULT 0.00,
 interpretation TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);

```

-- Enable Row-Level Security (RLS) on all three tables

```

ALTER TABLE surgical_checklists ENABLE ROW LEVEL SECURITY;
ALTER TABLE surgical_checklists FORCE ROW LEVEL SECURITY;

```

```

ALTER TABLE surgical_time_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE surgical_time_logs FORCE ROW LEVEL SECURITY;

```

```

ALTER TABLE urodynamic_studies ENABLE ROW LEVEL SECURITY;
ALTER TABLE urodynamic_studies FORCE ROW LEVEL SECURITY;

```

```

-- Create RLS policies
DROP POLICY IF EXISTS rls_surgical_checklists_tenant_isolation ON surgical_checklists;
CREATE POLICY rls_surgical_checklists_tenant_isolation ON surgical_checklists
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

DROP POLICY IF EXISTS rls_surgical_time_logs_tenant_isolation ON surgical_time_logs;
CREATE POLICY rls_surgical_time_logs_tenant_isolation ON surgical_time_logs
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

DROP POLICY IF EXISTS rls_urodynamic_studies_tenant_isolation ON urodynamic_studies;
CREATE POLICY rls_urodynamic_studies_tenant_isolation ON urodynamic_studies
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create indexes
CREATE INDEX IF NOT EXISTS idx_surgical_checklists_patient ON surgical_checklists (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_surgical_time_logs_patient ON surgical_time_logs (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_urodynamic_studies_patient ON urodynamic_studies (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE surgical_checklists, surgical_time_logs, urodynamic_studies TO
jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE surgical_checklists_id_seq, surgical_time_logs_id_seq, urodyn
amic_studies_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE surgical_checklists, surgical_time_logs, urodynamic_studies TO
nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE surgical_checklists_id_seq, surgical_time_logs_id_seq, urodyn
amic_studies_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;

=====
FILE: migrations\e44_surgery_urology_validate.sql
=====

-- Validation: Verify surgical checklists, time logs, and urodynamic studies tables, RLS, and indexes.
DO $$

```

```

DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check surgical_checklists
 SELECT EXISTS (SELECT FROM information_schema.tables WHERE table_name = 'surgical_checklists') INTO v_table_exists;
 IF NOT v_table_exists THEN RAISE EXCEPTION 'Validation failed: table surgical_checklists does not exist';
END IF;

 SELECT relrowsecurity INTO v_rls_enabled FROM pg_class WHERE relname = 'surgical_checklists';
 IF NOT v_rls_enabled THEN RAISE EXCEPTION 'Validation failed: RLS is not enabled on surgical_checklists';
END IF;

 SELECT EXISTS (SELECT FROM pg_indexes WHERE indexname = 'idx_surgical_checklists_patient') INTO v_index_exists;
 IF NOT v_index_exists THEN RAISE EXCEPTION 'Validation failed: index idx_surgical_checklists_patient does not exist'; END IF;

 -- 2. Check surgical_time_logs
 SELECT EXISTS (SELECT FROM information_schema.tables WHERE table_name = 'surgical_time_logs') INTO v_table_exists;
 IF NOT v_table_exists THEN RAISE EXCEPTION 'Validation failed: table surgical_time_logs does not exist'; END IF;

 SELECT relrowsecurity INTO v_rls_enabled FROM pg_class WHERE relname = 'surgical_time_logs';
 IF NOT v_rls_enabled THEN RAISE EXCEPTION 'Validation failed: RLS is not enabled on surgical_time_logs'; END IF;

 SELECT EXISTS (SELECT FROM pg_indexes WHERE indexname = 'idx_surgical_time_logs_patient') INTO v_index_exists;
 IF NOT v_index_exists THEN RAISE EXCEPTION 'Validation failed: index idx_surgical_time_logs_patient does not exist'; END IF;

 -- 3. Check urodynamic_studies
 SELECT EXISTS (SELECT FROM information_schema.tables WHERE table_name = 'urodynamic_studies') INTO v_table_exists;
 IF NOT v_table_exists THEN RAISE EXCEPTION 'Validation failed: table urodynamic_studies does not exist'; END IF;

 SELECT relrowsecurity INTO v_rls_enabled FROM pg_class WHERE relname = 'urodynamic_studies';
 IF NOT v_rls_enabled THEN RAISE EXCEPTION 'Validation failed: RLS is not enabled on urodynamic_studies'; END IF;

 SELECT EXISTS (SELECT FROM pg_indexes WHERE indexname = 'idx_urodynamic_studies_patient') INTO v_index_exists;
 IF NOT v_index_exists THEN RAISE EXCEPTION 'Validation failed: index idx_urodynamic_studies_patient does not exist'; END IF;

 RAISE NOTICE 'Validation passed: General Surgery and Urology module tables and RLS are fully set up';
END;
$$;

```



=====

FILE: migrations\e45\_specialties\_next\_down.sql

=====

-- Rollback: Drop CPB logs, pain assessments, and pediatric growth records tables.

BEGIN;

DROP TABLE IF EXISTS cpb\_logs CASCADE;

DROP TABLE IF EXISTS pain\_assessments CASCADE;

DROP TABLE IF EXISTS pediatric\_growth\_records CASCADE;

COMMIT;

=====

FILE: migrations\e45\_specialties\_next\_up.sql

=====

-- Migration: Create CPB logs, pain assessments, and pediatric growth records tables with RLS and grants.

BEGIN;

-- 1. Create cpb\_logs table

```
CREATE TABLE IF NOT EXISTS cpb_logs (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 bypass_date DATE NOT NULL DEFAULT CURRENT_DATE,
 pump_time INTEGER DEFAULT 0,
 cross_clamp_time INTEGER DEFAULT 0,
 flow_rate NUMERIC(4,2) DEFAULT 0.00,
 min_temp NUMERIC(3,1) DEFAULT 37.0,
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);
```

-- 2. Create pain\_assessments table

```
CREATE TABLE IF NOT EXISTS pain_assessments (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 assessment_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 pain_score_vas INTEGER DEFAULT 0,
 pca_pump_used BOOLEAN DEFAULT FALSE,
 pca_demands INTEGER DEFAULT 0,
 pca_deliveries INTEGER DEFAULT 0,
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
```

```

);

-- 3. Create pediatric_growth_records table
CREATE TABLE IF NOT EXISTS pediatric_growth_records (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 record_date DATE NOT NULL DEFAULT CURRENT_DATE,
 apgar_1min INTEGER,
 apgar_5min INTEGER,
 weight_kg NUMERIC(4,2),
 height_cm NUMERIC(4,1),
 head_circ_cm NUMERIC(3,1),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);

-- Enable Row-Level Security (RLS) on all three tables
ALTER TABLE cpb_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE cpb_logs FORCE ROW LEVEL SECURITY;

ALTER TABLE pain_assessments ENABLE ROW LEVEL SECURITY;
ALTER TABLE pain_assessments FORCE ROW LEVEL SECURITY;

ALTER TABLE pediatric_growth_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE pediatric_growth_records FORCE ROW LEVEL SECURITY;

-- Create RLS policies
DROP POLICY IF EXISTS rls_cpb_logs_tenant_isolation ON cpb_logs;
CREATE POLICY rls_cpb_logs_tenant_isolation ON cpb_logs
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

DROP POLICY IF EXISTS rls_pain_assessments_tenant_isolation ON pain_assessments;
CREATE POLICY rls_pain_assessments_tenant_isolation ON pain_assessments
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

DROP POLICY IF EXISTS rls_pediatric_growth_records_tenant_isolation ON pediatric_growth_records;
CREATE POLICY rls_pediatric_growth_records_tenant_isolation ON pediatric_growth_records
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create indexes
CREATE INDEX IF NOT EXISTS idx_cpb_logs_patient ON cpb_logs (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_pain_assessments_patient ON pain_assessments (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_pediatric_growth_records_patient ON pediatric_growth_records (tenant_id, patient_id);

-- Grant privileges conditionally

```

```

DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE cpb_logs, pain_assessments, pediatric_growth_records TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE cpb_logs_id_seq, pain_assessments_id_seq, pediatric_growth_records_id_seq TO jumanasoft_staging_user';
 END IF;

 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE cpb_logs, pain_assessments, pediatric_growth_records TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE cpb_logs_id_seq, pain_assessments_id_seq, pediatric_growth_records_id_seq TO nama_medical_app';
 END IF;
END
$$;

COMMIT;

```

```

=====
FILE: migrations\e45_specialties_next_validate.sql
=====

```

```

-- Validation: Verify CPB logs, pain assessments, and pediatric growth records tables, RLS, and indexes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check cpb_logs
 SELECT EXISTS (SELECT FROM information_schema.tables WHERE table_name = 'cpb_logs') INTO v_table_exists;
 IF NOT v_table_exists THEN RAISE EXCEPTION 'Validation failed: table cpb_logs does not exist'; END IF;

 SELECT relrowsecurity INTO v_rls_enabled FROM pg_class WHERE relname = 'cpb_logs';
 IF NOT v_rls_enabled THEN RAISE EXCEPTION 'Validation failed: RLS is not enabled on cpb_logs'; END IF;

 SELECT EXISTS (SELECT FROM pg_indexes WHERE indexname = 'idx_cpb_logs_patient') INTO v_index_exists;
 IF NOT v_index_exists THEN RAISE EXCEPTION 'Validation failed: index idx_cpb_logs_patient does not exist';
END IF;

 -- 2. Check pain_assessments
 SELECT EXISTS (SELECT FROM information_schema.tables WHERE table_name = 'pain_assessments') INTO v_table_exists;
 IF NOT v_table_exists THEN RAISE EXCEPTION 'Validation failed: table pain_assessments does not exist'; END IF;

 SELECT relrowsecurity INTO v_rls_enabled FROM pg_class WHERE relname = 'pain_assessments';
 IF NOT v_rls_enabled THEN RAISE EXCEPTION 'Validation failed: RLS is not enabled on pain_assessments'; END IF;

```

```

SELECT EXISTS (SELECT FROM pg_indexes WHERE indexname = 'idx_pain_assessments_patient') INTO v_index_exists;

IF NOT v_index_exists THEN RAISE EXCEPTION 'Validation failed: index idx_pain_assessments_patient does not exist'; END IF;

-- 3. Check pediatric_growth_records
SELECT EXISTS (SELECT FROM information_schema.tables WHERE table_name = 'pediatric_growth_records') INTO v_table_exists;

IF NOT v_table_exists THEN RAISE EXCEPTION 'Validation failed: table pediatric_growth_records does not exist'; END IF;

SELECT relrowsecurity INTO v_rls_enabled FROM pg_class WHERE relname = 'pediatric_growth_records';

IF NOT v_rls_enabled THEN RAISE EXCEPTION 'Validation failed: RLS is not enabled on pediatric_growth_records'; END IF;

SELECT EXISTS (SELECT FROM pg_indexes WHERE indexname = 'idx_pediatric_growth_records_patient') INTO v_index_exists;

IF NOT v_index_exists THEN RAISE EXCEPTION 'Validation failed: index idx_pediatric_growth_records_patient does not exist'; END IF;

RAISE NOTICE 'Validation passed: G16, G19, and G20 module tables and RLS are fully set up';

END;
$$;

```

```

=====
FILE: migrations\e46_specialties_next2_down.sql
=====

```

```

-- Rollback: Drop OBGYN pregnancies, psychiatric evaluations, and dermatology lesions tables.
BEGIN;

```

```

DROP TABLE IF EXISTS obgyn_pregnancies CASCADE;
DROP TABLE IF EXISTS psychiatric_evaluations CASCADE;
DROP TABLE IF EXISTS dermatology_lesions CASCADE;

```

```

COMMIT;

```

```

=====
FILE: migrations\e46_specialties_next2_up.sql
=====

```

```

-- Migration: Create OBGYN pregnancies, psychiatric evaluations, and dermatology lesions tables with RLS and grants.
BEGIN;

```

```

-- 1. Create obgyn_pregnancies table
CREATE TABLE IF NOT EXISTS obgyn_pregnancies (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,

```

```

 gravida INTEGER DEFAULT 0,
 para INTEGER DEFAULT 0,
 abortions INTEGER DEFAULT 0,
 living INTEGER DEFAULT 0,
 lmp_date DATE,
 edd_date DATE,
 gestational_weeks INTEGER,
 notes TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);

-- Ensure all columns exist in case the table was pre-existing
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS doctor_id INTEGER REFERENCES system_users(id) ON DELETE
SET NULL;
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS gravida INTEGER DEFAULT 0;
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS para INTEGER DEFAULT 0;
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS abortions INTEGER DEFAULT 0;
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS living INTEGER DEFAULT 0;
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS lmp_date DATE;
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS edd_date DATE;
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS gestational_weeks INTEGER;
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS notes TEXT DEFAULT '';
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS tenant_id INTEGER DEFAULT 1;
ALTER TABLE obgyn_pregnancies ADD COLUMN IF NOT EXISTS facility_id INTEGER;

-- 2. Create psychiatric_evaluations table
CREATE TABLE IF NOT EXISTS psychiatric_evaluations (
 id SERIAL PRIMARY KEY,
 patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
 evaluation_date DATE NOT NULL DEFAULT CURRENT_DATE,
 mse_appearance TEXT DEFAULT '',
 mse_behavior TEXT DEFAULT '',
 mse_speech TEXT DEFAULT '',
 mse_mood TEXT DEFAULT '',
 mse_affect TEXT DEFAULT '',
 mse_thought_process TEXT DEFAULT '',
 mse_thought_content TEXT DEFAULT '',
 mse_perception TEXT DEFAULT '',
 mse_cognition TEXT DEFAULT '',
 mse_insight TEXT DEFAULT '',
 mse_judgment TEXT DEFAULT '',
 diagnostic_summary TEXT DEFAULT '',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER
);

-- 3. Create dermatology_lesions table
CREATE TABLE IF NOT EXISTS dermatology_lesions (
 id SERIAL PRIMARY KEY,

```

```

patient_id INTEGER NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
doctor_id INTEGER REFERENCES system_users(id) ON DELETE SET NULL,
exam_date DATE NOT NULL DEFAULT CURRENT_DATE,
body_site VARCHAR(100) DEFAULT '',
lesion_type VARCHAR(100) DEFAULT '',
color VARCHAR(50) DEFAULT '',
size_mm NUMERIC(4,1) DEFAULT 0.0,
distribution VARCHAR(100) DEFAULT '',
biopsy_taken BOOLEAN DEFAULT FALSE,
notes TEXT DEFAULT '',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
tenant_id INTEGER NOT NULL,
facility_id INTEGER
);

-- Enable Row-Level Security (RLS) on all three tables
ALTER TABLE obgyn_pregnancies ENABLE ROW LEVEL SECURITY;
ALTER TABLE obgyn_pregnancies FORCE ROW LEVEL SECURITY;

ALTER TABLE psychiatric_evaluations ENABLE ROW LEVEL SECURITY;
ALTER TABLE psychiatric_evaluations FORCE ROW LEVEL SECURITY;

ALTER TABLE dermatology_lesions ENABLE ROW LEVEL SECURITY;
ALTER TABLE dermatology_lesions FORCE ROW LEVEL SECURITY;

-- Create RLS policies
DROP POLICY IF EXISTS rls_obgyn_pregnancies_tenant_isolation ON obgyn_pregnancies;
CREATE POLICY rls_obgyn_pregnancies_tenant_isolation ON obgyn_pregnancies
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

DROP POLICY IF EXISTS rls_psychiatric_evaluations_tenant_isolation ON psychiatric_evaluations;
CREATE POLICY rls_psychiatric_evaluations_tenant_isolation ON psychiatric_evaluations
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

DROP POLICY IF EXISTS rls_dermatology_lesions_tenant_isolation ON dermatology_lesions;
CREATE POLICY rls_dermatology_lesions_tenant_isolation ON dermatology_lesions
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- Create indexes
CREATE INDEX IF NOT EXISTS idx_obgyn_pregnancies_patient ON obgyn_pregnancies (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_psychiatric_evaluations_patient ON psychiatric_evaluations (tenant_id, patient_id);
CREATE INDEX IF NOT EXISTS idx_dermatology_lesions_patient ON dermatology_lesions (tenant_id, patient_id);

-- Grant privileges conditionally
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN

```

```

EXECUTE 'GRANT ALL PRIVILEGES ON TABLE obgyn_pregnancies, psychiatric_evaluations, dermatology_lesions
TO jumanasoft_staging_user';

EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE obgyn_pregnancies_id_seq, psychiatric_evaluations_id_seq, der
matology_lesions_id_seq TO jumanasoft_staging_user';

END IF;

IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
EXECUTE 'GRANT ALL PRIVILEGES ON TABLE obgyn_pregnancies, psychiatric_evaluations, dermatology_lesions
TO nama_medical_app';
EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE obgyn_pregnancies_id_seq, psychiatric_evaluations_id_seq, der
matology_lesions_id_seq TO nama_medical_app';
END IF;

END
$$;

COMMIT;

```

```

=====
FILE: migrations\e46_specialties_next2_validate.sql
=====

```

```

-- Validation: Verify OBGYN pregnancies, psychiatric evaluations, and dermatology lesions tables, RLS, and ind
exes.
DO $$
DECLARE
 v_table_exists BOOLEAN;
 v_rls_enabled BOOLEAN;
 v_index_exists BOOLEAN;
BEGIN
 -- 1. Check obgyn_pregnancies
 SELECT EXISTS (SELECT FROM information_schema.tables WHERE table_name = 'obgyn_pregnancies') INTO v_table_
exists;
 IF NOT v_table_exists THEN RAISE EXCEPTION 'Validation failed: table obgyn_pregnancies does not exist'; EN
D IF;

 SELECT relrowsecurity INTO v_rls_enabled FROM pg_class WHERE relname = 'obgyn_pregnancies';
 IF NOT v_rls_enabled THEN RAISE EXCEPTION 'Validation failed: RLS is not enabled on obgyn_pregnancies'; EN
D IF;

 SELECT EXISTS (SELECT FROM pg_indexes WHERE indexname = 'idx_obgyn_pregnancies_patient') INTO v_index_exis
ts;
 IF NOT v_index_exists THEN RAISE EXCEPTION 'Validation failed: index idx_obgyn_pregnancies_patient does no
t exist'; END IF;

 -- 2. Check psychiatric_evaluations
 SELECT EXISTS (SELECT FROM information_schema.tables WHERE table_name = 'psychiatric_evaluations') INTO v_
table_exists;
 IF NOT v_table_exists THEN RAISE EXCEPTION 'Validation failed: table psychiatric_evaluations does not exis
t'; END IF;

 SELECT relrowsecurity INTO v_rls_enabled FROM pg_class WHERE relname = 'psychiatric_evaluations';
 IF NOT v_rls_enabled THEN RAISE EXCEPTION 'Validation failed: RLS is not enabled on psychiatric_evaluation

```

```
s'; END IF;
```

```
SELECT EXISTS (SELECT FROM pg_indexes WHERE indexname = 'idx_psychiatric_evaluations_patient') INTO v_index_exists;
```

```
IF NOT v_index_exists THEN RAISE EXCEPTION 'Validation failed: index idx_psychiatric_evaluations_patient does not exist'; END IF;
```

```
-- 3. Check dermatology_lesions
```

```
SELECT EXISTS (SELECT FROM information_schema.tables WHERE table_name = 'dermatology_lesions') INTO v_table_exists;
```

```
IF NOT v_table_exists THEN RAISE EXCEPTION 'Validation failed: table dermatology_lesions does not exist';
END IF;
```

```
SELECT relrowsecurity INTO v_rls_enabled FROM pg_class WHERE relname = 'dermatology_lesions';
```

```
IF NOT v_rls_enabled THEN RAISE EXCEPTION 'Validation failed: RLS is not enabled on dermatology_lesions';
END IF;
```

```
SELECT EXISTS (SELECT FROM pg_indexes WHERE indexname = 'idx_dermatology_lesions_patient') INTO v_index_exists;
```

```
IF NOT v_index_exists THEN RAISE EXCEPTION 'Validation failed: index idx_dermatology_lesions_patient does not exist'; END IF;
```

```
RAISE NOTICE 'Validation passed: G21, G24, and G25 module tables and RLS are fully set up';
END;
$;
```

```
=====
FILE: migrations\e47_billing_tables_candidate_down.sql
=====
```

```
-- Candidate Down Migration: Rollback SaaS Billing Tables
```

```
-- =====
```

```
-- SAFE-BY-DESIGN: Design-Only candidate. DO NOT execute in database.
```

```
-- =====
```

```
BEGIN;
```

```
-- Drop Policies First
```

```
DROP POLICY IF EXISTS tenant_billing_audit_policy ON saas_billing_audit_events;
```

```
DROP POLICY IF EXISTS tenant_payment_transactions_policy ON saas_billing_payment_transactions;
```

```
DROP POLICY IF EXISTS tenant_checkout_sessions_policy ON saas_billing_checkout_sessions;
```

```
DROP POLICY IF EXISTS tenant_billing_subscriptions_policy ON saas_billing_subscriptions;
```

```
DROP POLICY IF EXISTS tenant_billing_customers_policy ON saas_billing_customers;
```

```
-- Drop Tables cleanly
```

```
DROP TABLE IF EXISTS saas_billing_audit_events;
```

```
DROP TABLE IF EXISTS saas_billing_provider_accounts;
```

```
DROP TABLE IF EXISTS saas_billing_webhook_events;
```

```
DROP TABLE IF EXISTS saas_billing_payment_transactions;
```

```
DROP TABLE IF EXISTS saas_billing_checkout_sessions;
```

```
DROP TABLE IF EXISTS saas_billing_subscriptions;
```

```
DROP TABLE IF EXISTS saas_billing_customers;
```



COMMIT;

=====

FILE: migrations\e47\_billing\_tables\_candidate\_up.sql

=====

-- Candidate Up Migration: Create SaaS Billing Tables

-- =====

-- SAFE-BY-DESIGN: Design-Only candidate. DO NOT execute in database.

-- =====

BEGIN;

-- 1. saas\_billing\_customers

```
CREATE TABLE IF NOT EXISTS saas_billing_customers (
 id SERIAL PRIMARY KEY,
 tenant_id INT NOT NULL,
 provider VARCHAR(50) NOT NULL,
 provider_customer_id_hash VARCHAR(255) NOT NULL,
 billing_email_masked VARCHAR(255) NOT NULL,
 status VARCHAR(50) DEFAULT 'active',
 created_by INT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT uq_tenant_provider UNIQUE (tenant_id, provider)
);
```

CREATE INDEX IF NOT EXISTS idx\_saas\_billing\_customers\_tenant ON saas\_billing\_customers (tenant\_id);

-- Enable RLS

ALTER TABLE saas\_billing\_customers ENABLE ROW LEVEL SECURITY;

ALTER TABLE saas\_billing\_customers FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant\_billing\_customers\_policy ON saas\_billing\_customers

USING (tenant\_id = NULLIF(current\_setting('app.tenant\_id', true), '')::integer)

WITH CHECK (tenant\_id = NULLIF(current\_setting('app.tenant\_id', true), '')::integer);

-- 2. saas\_billing\_subscriptions

```
CREATE TABLE IF NOT EXISTS saas_billing_subscriptions (
 id SERIAL PRIMARY KEY,
 tenant_id INT NOT NULL,
 plan_key VARCHAR(50) NOT NULL,
 provider VARCHAR(50) NOT NULL,
 provider_subscription_id_hash VARCHAR(255),
 status VARCHAR(50) NOT NULL,
 current_period_start TIMESTAMP,
 current_period_end TIMESTAMP,
 cancel_at_period_end BOOLEAN DEFAULT FALSE,
 trial_start TIMESTAMP,
 trial_end TIMESTAMP,
 created_by INT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_saas_billing_subs_tenant ON saas_billing_subscriptions (tenant_id);

-- Enable RLS
ALTER TABLE saas_billing_subscriptions ENABLE ROW LEVEL SECURITY;
ALTER TABLE saas_billing_subscriptions FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_billing_subscriptions_policy ON saas_billing_subscriptions
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 3. saas_billing_checkout_sessions
CREATE TABLE IF NOT EXISTS saas_billing_checkout_sessions (
 id SERIAL PRIMARY KEY,
 tenant_id INT NOT NULL,
 plan_key VARCHAR(50) NOT NULL,
 provider VARCHAR(50) NOT NULL,
 idempotency_key VARCHAR(255) UNIQUE NOT NULL,
 provider_session_id_hash VARCHAR(255),
 status VARCHAR(50) NOT NULL,
 amount_minor INT NOT NULL,
 currency VARCHAR(10) NOT NULL DEFAULT 'SAR',
 expires_at TIMESTAMP NOT NULL,
 created_by INT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_saas_billing_checkout_tenant ON saas_billing_checkout_sessions (tenant_id);

-- Enable RLS
ALTER TABLE saas_billing_checkout_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE saas_billing_checkout_sessions FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_checkout_sessions_policy ON saas_billing_checkout_sessions
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 4. saas_billing_payment_transactions
CREATE TABLE IF NOT EXISTS saas_billing_payment_transactions (
 id SERIAL PRIMARY KEY,
 tenant_id INT NOT NULL,
 provider VARCHAR(50) NOT NULL,
 idempotency_key VARCHAR(255) UNIQUE NOT NULL,
 provider_transaction_id_hash VARCHAR(255),
 amount_minor INT NOT NULL,
 currency VARCHAR(10) NOT NULL DEFAULT 'SAR',
 status VARCHAR(50) NOT NULL,
 payment_method_type VARCHAR(50),
 failure_code VARCHAR(100),

```

```

failure_message_safe VARCHAR(255),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_saas_billing_tx_tenant ON saas_billing_payment_transactions (tenant_id);

-- Enable RLS
ALTER TABLE saas_billing_payment_transactions ENABLE ROW LEVEL SECURITY;
ALTER TABLE saas_billing_payment_transactions FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_payment_transactions_policy ON saas_billing_payment_transactions
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- 5. saas_billing_webhook_events
CREATE TABLE IF NOT EXISTS saas_billing_webhook_events (
 id SERIAL PRIMARY KEY,
 provider VARCHAR(50) NOT NULL,
 provider_event_id_hash VARCHAR(255) UNIQUE NOT NULL,
 event_type VARCHAR(100) NOT NULL,
 received_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 processed_at TIMESTAMP,
 processing_status VARCHAR(50) DEFAULT 'pending',
 signature_verified BOOLEAN DEFAULT FALSE,
 payload_hash VARCHAR(64) NOT NULL,
 payload_safe_jsonb JSONB,
 error_message_safe VARCHAR(255)
);

CREATE INDEX IF NOT EXISTS idx_saas_billing_webhook_prov_evt ON saas_billing_webhook_events (provider, provider_event_id_hash);

-- Webhook events do not have tenant_id column; protected at server-side access

-- 6. saas_billing_provider_accounts
CREATE TABLE IF NOT EXISTS saas_billing_provider_accounts (
 id SERIAL PRIMARY KEY,
 provider VARCHAR(50) UNIQUE NOT NULL,
 mode VARCHAR(20) DEFAULT 'sandbox',
 enabled BOOLEAN DEFAULT FALSE,
 public_label VARCHAR(100),
 capabilities_jsonb JSONB,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- General settings, no RLS, protected by Super Admin Server-Side guards

-- 7. saas_billing_audit_events
CREATE TABLE IF NOT EXISTS saas_billing_audit_events (

```

```

 id SERIAL PRIMARY KEY,
 tenant_id INT,
 actor_user_id INT NOT NULL,
 actor_role VARCHAR(50) NOT NULL,
 action VARCHAR(100) NOT NULL,
 target_type VARCHAR(100),
 target_id INT,
 safe_metadata_jsonb JSONB,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_saas_billing_audit_tenant ON saas_billing_audit_events (tenant_id);

-- Enable RLS
ALTER TABLE saas_billing_audit_events ENABLE ROW LEVEL SECURITY;
ALTER TABLE saas_billing_audit_events FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_billing_audit_policy ON saas_billing_audit_events
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

```

```

=====
FILE: migrations\e47_billing_tables_candidate_validate.sql
=====

```

```

-- e47_billing_tables_candidate_validate.sql
-- Read-only validation for an ISOLATED test DB after e47_up.
-- Returns all_ok = true iff the seven tables, key constraints, RLS, and indexes exist as designed.
-- Absolutely NO writes.
SELECT
 -- 1. Verify tables count
 (SELECT count(*) FROM information_schema.tables
 WHERE table_name IN (
 'saas_billing_customers',
 'saas_billing_subscriptions',
 'saas_billing_checkout_sessions',
 'saas_billing_payment_transactions',
 'saas_billing_webhook_events',
 'saas_billing_provider_accounts',
 'saas_billing_audit_events'
)) = 7 AS all_tables_present,

 -- 2. Verify tenant_id column in tenant-scoped tables
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name IN (
 'saas_billing_customers',
 'saas_billing_subscriptions',
 'saas_billing_checkout_sessions',
 'saas_billing_payment_transactions',
 'saas_billing_audit_events'
)) = 5 AS all_tenant_scoped_tables_have_tenant_id;

```

```

) AND column_name = 'tenant_id') = 5 AS tenant_id_present,

-- 3. Verify no credit card PAN or CVV or raw secret columns are added
(SELECT count(*) FROM information_schema.columns
 WHERE table_name LIKE 'saas_billing_%'
 AND column_name IN ('card_number', 'cvv', 'cvc', 'raw_secret', 'secret_key', 'private_key'))
= 0 AS no_sensitive_columns,

-- 4. Verify indexes
EXISTS (SELECT 1 FROM pg_indexes WHERE indexname='idx_saas_billing_customers_tenant') AS customer_index_pr
esent,
EXISTS (SELECT 1 FROM pg_indexes WHERE indexname='idx_saas_billing_subs_tenant') AS sub_index_present,
EXISTS (SELECT 1 FROM pg_indexes WHERE indexname='idx_saas_billing_checkout_tenant') AS checkout_index_pre
sent,
EXISTS (SELECT 1 FROM pg_indexes WHERE indexname='idx_saas_billing_tx_tenant') AS tx_index_present,
EXISTS (SELECT 1 FROM pg_indexes WHERE indexname='idx_saas_billing_audit_tenant') AS audit_index_present,

-- 5. Overall validation result
(
 (SELECT count(*) FROM information_schema.tables
 WHERE table_name IN (
 'saas_billing_customers',
 'saas_billing_subscriptions',
 'saas_billing_checkout_sessions',
 'saas_billing_payment_transactions',
 'saas_billing_webhook_events',
 'saas_billing_provider_accounts',
 'saas_billing_audit_events'
)) = 7
 AND
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name IN (
 'saas_billing_customers',
 'saas_billing_subscriptions',
 'saas_billing_checkout_sessions',
 'saas_billing_payment_transactions',
 'saas_billing_audit_events'
) AND column_name = 'tenant_id') = 5
 AND
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name LIKE 'saas_billing_%'
 AND column_name IN ('card_number', 'cvv', 'cvc', 'raw_secret', 'secret_key', 'private_key'))
) = 0
) AS all_ok;

```

```
=====
```

```
FILE: migrations\e48_ews_score_types_candidate_down.sql
```

```
=====
```

```

-- e48_ews_score_types_candidate_down.sql ? rollback for the e48 candidate.
-- Restores the original 4-type CHECK. WILL FAIL (by design) if any rows already
-- use the new EWS types ? delete/migrate those rows deliberately first; the

```

```
-- constraint must never be dropped silently over live clinical rows.
```

```
BEGIN;
```

```
ALTER TABLE nursing_scores DROP CONSTRAINT IF EXISTS chk_nursing_score_type;
```

```
ALTER TABLE nursing_scores ADD CONSTRAINT chk_nursing_score_type
 CHECK (score_type IN ('morse', 'braden', 'news', 'pain'));
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e48_ews_score_types_candidate_up.sql
```

```
=====
```

```
-- e48_ews_score_types_candidate_up.sql ? CANDIDATE, NOT RUN (requires owner approval).
```

```
-- Gate 2: extend nursing_scores.score_type to persist server-computed EWS results
```

```
-- (MEWS / PEWS / qSOFA / SIRS / sepsis screen) alongside morse/braden/news/pain.
```

```
-- Additive-only: widens a CHECK constraint; no data change, no new table.
```

```
-- Rollback: e48_ews_score_types_candidate_down.sql restores the original CHECK
```

```
-- (safe only while no rows carry the new types).
```

```
BEGIN;
```

```
ALTER TABLE nursing_scores DROP CONSTRAINT IF EXISTS chk_nursing_score_type;
```

```
ALTER TABLE nursing_scores ADD CONSTRAINT chk_nursing_score_type
 CHECK (score_type IN ('morse', 'braden', 'news', 'pain', 'mews', 'pews', 'qsofa', 'sirs', 'sepsis_screen')
);
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e48_ews_score_types_candidate_validate.sql
```

```
=====
```

```
-- e48_ews_score_types_candidate_validate.sql ? post-apply validation for the e48 candidate.
```

```
DO $$
```

```
DECLARE
```

```
 def TEXT;
```

```
BEGIN
```

```
 SELECT pg_get_constraintdef(oid) INTO def
```

```
 FROM pg_constraint
```

```
 WHERE conname = 'chk_nursing_score_type' AND conrelid = 'nursing_scores'::regclass;
```

```
 IF def IS NULL THEN
```

```
 RAISE EXCEPTION 'Validation failed: chk_nursing_score_type missing on nursing_scores';
```

```
 END IF;
```

```
 IF def NOT LIKE '%mews%' OR def NOT LIKE '%pews%' OR def NOT LIKE '%qsofa%'
```

```
 OR def NOT LIKE '%sirs%' OR def NOT LIKE '%sepsis_screen%' THEN
```

```
 RAISE EXCEPTION 'Validation failed: EWS score types not present in CHECK: %', def;
```

```

END IF;

RAISE NOTICE 'e48 validate OK: %', def;
END $$;

=====
FILE: migrations\e49_result_acknowledgements_candidate_down.sql
=====

-- e49_result_acknowledgements_candidate_down.sql ? rollback for the e49 candidate.
-- Dropping this table erases the acknowledgement audit trail ? only acceptable while
-- the feature has never been live. If any rows exist, archive them first.
BEGIN;
DROP TABLE IF EXISTS result_acknowledgements;
COMMIT;

=====
FILE: migrations\e49_result_acknowledgements_candidate_up.sql
=====

-- e49_result_acknowledgements_candidate_up.sql ? CANDIDATE, NOT RUN (requires owner approval).
-- Gate 3: physician acknowledgement of verified abnormal/critical results (order?result
-- closed loop, provider side). One row per acknowledging clinician per result.
-- The /api/results/*/acknowledge endpoints feature-detect this table and return
-- 503 RESULT_ACK_PENDING_DDL until it exists. Additive-only; no existing data touched.

BEGIN;

CREATE TABLE IF NOT EXISTS result_acknowledgements (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 facility_id INTEGER,
 result_type TEXT NOT NULL,
 result_id INTEGER NOT NULL,
 patient_id INTEGER NOT NULL REFERENCES patients(id),
 ack_level TEXT NOT NULL,
 acknowledged_by INTEGER NOT NULL,
 acknowledged_by_name TEXT DEFAULT '',
 note TEXT DEFAULT '',
 acknowledged_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT chk_result_ack_type CHECK (result_type IN ('lab', 'rad')),
 CONSTRAINT chk_result_ack_level CHECK (ack_level IN ('critical', 'abnormal', 'unknown')),
 CONSTRAINT uq_result_ack UNIQUE (tenant_id, result_type, result_id, acknowledged_by)
);

ALTER TABLE result_acknowledgements ENABLE ROW LEVEL SECURITY;
ALTER TABLE result_acknowledgements FORCE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS rls_result_ack_tenant_isolation ON result_acknowledgements;
CREATE POLICY rls_result_ack_tenant_isolation ON result_acknowledgements

```

```

 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

CREATE INDEX IF NOT EXISTS idx_result_ack_tenant_result ON result_acknowledgements (tenant_id, result_type, result_id);
CREATE INDEX IF NOT EXISTS idx_result_ack_tenant_patient ON result_acknowledgements (tenant_id, patient_id);

DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE result_acknowledgements TO nama_medical_app';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE result_acknowledgements_id_seq TO nama_medical_app';
 END IF;
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE result_acknowledgements TO jumanasoft_staging_user';
 EXECUTE 'GRANT USAGE, SELECT ON SEQUENCE result_acknowledgements_id_seq TO jumanasoft_staging_user';
 END IF;
END $$;

COMMIT;

```

```

=====
FILE: migrations\e49_result_acknowledgements_candidate_validate.sql
=====

```

```

-- e49_result_acknowledgements_candidate_validate.sql ? post-apply validation.
DO $$
DECLARE
 rls_forced BOOLEAN;
BEGIN
 IF to_regclass('public.result_acknowledgements') IS NULL THEN
 RAISE EXCEPTION 'Validation failed: result_acknowledgements does not exist';
 END IF;

 SELECT relforcerowsecurity INTO rls_forced FROM pg_class WHERE relname = 'result_acknowledgements';
 IF NOT rls_forced THEN
 RAISE EXCEPTION 'Validation failed: FORCE RLS not enabled on result_acknowledgements';
 END IF;

 IF NOT EXISTS (
 SELECT 1 FROM pg_policies
 WHERE schemaname = 'public' AND tablename = 'result_acknowledgements'
 AND policyname = 'rls_result_ack_tenant_isolation'
) THEN
 RAISE EXCEPTION 'Validation failed: tenant isolation policy missing';
 END IF;

 IF NOT EXISTS (
 SELECT 1 FROM pg_constraint
 WHERE conname = 'uq_result_ack' AND conrelid = 'result_acknowledgements'::regclass
) THEN
 RAISE EXCEPTION 'Validation failed: uq_result_ack unique constraint missing';
 END IF;
END $$;

```



```

END IF;

RAISE NOTICE 'e49 validate OK';
END $$;

=====
FILE: migrations\e4_01_rad_worklist_up_validate_down.js
=====

/**
 * e4_01_rad_worklist_up_validate_down.js
 * =====
 * E4 ? RIS WORKLIST (candidate migration; NOT auto-run; gated; idempotent)
 *
 * Adds `rad_exams`: the RIS worklist state machine row that hangs off an existing
 * radiology order in `lab_radiology_orders`. State machine:
 * Scheduled -> Arrived -> InProgress -> Completed -> Reported
 *
 * Posture (matches the 150 FORCE-RLS tables already enforced in this codebase):
 * - tenant_id INTEGER NOT NULL
 * - ENABLE + FORCE ROW LEVEL SECURITY
 * - single canonical FOR ALL policy keyed on current_setting('app.tenant_id')
 * - (tenant_id, state) composite index
 *
 * NOTE: No DB exec here. Caller invokes up/validate/down with a pg pool/client.
 * =====
 */
'use strict';

const TABLE = 'rad_exams';
const POLICY = 'rad_exams_tenant_isolation';
const VALID_STATES = ['Scheduled', 'Arrived', 'InProgress', 'Completed', 'Reported'];

async function up(db) {
 // 1) table (additive, idempotent)
 await db.query(`
 CREATE TABLE IF NOT EXISTS ${TABLE} (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER,
 rad_order_id INTEGER NOT NULL, -- FK -> lab_radiology_orders.id (is_radiology=1)
 patient_id INTEGER,
 modality TEXT DEFAULT '', -- XR/CT/MRI/US/MG/...
 exam_name TEXT DEFAULT '',
 accession TEXT DEFAULT '', -- accession number (worklist key for MWL)
 state TEXT NOT NULL DEFAULT 'Scheduled',
 scheduled_at TIMESTAMP,
 arrived_at TIMESTAMP,
 started_at TIMESTAMP,
 completed_at TIMESTAMP,
 reported_at TIMESTAMP,
 assigned_tech INTEGER,

```

```

 notes TEXT DEFAULT '',
 created_by INTEGER,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
`);

// CHECK constraint on the state machine (idempotent: add only if absent)
await db.query(`
 DO $$
 BEGIN
 IF NOT EXISTS (
 SELECT 1 FROM pg_constraint WHERE conname = 'rad_exams_state_chk'
) THEN
 ALTER TABLE ${TABLE} ADD CONSTRAINT rad_exams_state_chk
 CHECK (state IN ('Scheduled','Arrived','InProgress','Completed','Reported'));
 END IF;
 END $$;
`);

// 2) RLS: enable + force (idempotent ? ALTER is no-op if already set)
await db.query(`ALTER TABLE ${TABLE} ENABLE ROW LEVEL SECURITY`);
await db.query(`ALTER TABLE ${TABLE} FORCE ROW LEVEL SECURITY`);

// 3) canonical tenant-isolation policy (drop+create = idempotent)
await db.query(`DROP POLICY IF EXISTS ${POLICY} ON ${TABLE}`);
await db.query(`
 CREATE POLICY ${POLICY} ON ${TABLE}
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
`);

// 4) tenant-scoped indexes
await db.query(`CREATE INDEX IF NOT EXISTS idx_rad_exams_tenant_state ON ${TABLE} (tenant_id, state)`);
await db.query(`CREATE INDEX IF NOT EXISTS idx_rad_exams_tenant_order ON ${TABLE} (tenant_id, rad_order_id)`);
await db.query(`CREATE INDEX IF NOT EXISTS idx_rad_exams_tenant_accession ON ${TABLE} (tenant_id, accession)`);

return { ok: true, table: TABLE };
}

async function validate(db) {
 const checks = {};
 const t = await db.query(`SELECT to_regclass('public.${TABLE}') AS reg`);
 checks.tableExists = !!t.rows[0].reg;

 const col = await db.query(
 `SELECT is_nullable FROM information_schema.columns WHERE table_name=$1 AND column_name='tenant_id',
 [TABLE]
 `);
 checks.tenantIdNotNull = col.rows.length === 1 && col.rows[0].is_nullable === 'NO';

```

```

const rls = await db.query(
 `SELECT relrowsecurity, relforcerowsecurity FROM pg_class WHERE relname=$1`, [TABLE]
);
checks.rlsEnabled = rls.rows.length === 1 && rls.rows[0].relrowsecurity === true;
checks.rlsForced = rls.rows.length === 1 && rls.rows[0].relforcerowsecurity === true;

const pol = await db.query(`SELECT polname FROM pg_policy WHERE polname=$1`, [POLICY]);
checks.policyExists = pol.rows.length === 1;

const idx = await db.query(
 `SELECT indexname FROM pg_indexes WHERE tablename=$1 AND indexname='idx_rad_exams_tenant_state'`, [TABLE]
);
checks.tenantIndexExists = idx.rows.length === 1;

checks.ok = Object.values(checks).every(Boolean);
return checks;
}

```

```

async function down(db) {
 // reverse order: policy -> disable RLS -> own indexes -> constraint -> table
 await db.query(`DROP POLICY IF EXISTS ${POLICY} ON ${TABLE}`);
 await db.query(`ALTER TABLE IF EXISTS ${TABLE} NO FORCE ROW LEVEL SECURITY`);
 await db.query(`ALTER TABLE IF EXISTS ${TABLE} DISABLE ROW LEVEL SECURITY`);
 await db.query(`DROP INDEX IF EXISTS idx_rad_exams_tenant_state`);
 await db.query(`DROP INDEX IF EXISTS idx_rad_exams_tenant_order`);
 await db.query(`DROP INDEX IF EXISTS idx_rad_exams_tenant_accession`);
 await db.query(`ALTER TABLE IF EXISTS ${TABLE} DROP CONSTRAINT IF EXISTS rad_exams_state_chk`);
 await db.query(`DROP TABLE IF EXISTS ${TABLE}`);
 return { ok: true, dropped: TABLE };
}

```

```

module.exports = { up, validate, down, TABLE, POLICY, VALID_STATES };

```

```

=====

```

```

FILE: migrations\e4_02_dicom_studies_up_validate_down.js

```

```

=====

```

```

/**
 * e4_02_dicom_studies_up_validate_down.js
 * =====
 * E4 ? DICOM STUDIES METADATA (candidate migration; NOT auto-run; gated; idempotent)
 *
 * Adds `dicom_studies`: PACS/DICOM study+series METADATA ONLY.
 * - NO image bytes are stored here. Image/object bytes (incl. .dcm) are served
 * EXCLUSIVELY through the existing guarded /api/phi-files/:id endpoint
 * (auth + explicit tenant predicate + path-traversal deny + nosniff + sandbox CSP).
 * - `stored_ref` is a soft reference to a phi_files row (the encrypted-at-rest object),
 * never a public path.
 * - This table is GATED for real PACS integration (RAD_MWL_ENABLED); by default the
 * app only registers metadata captured manually ? NO external PACS connection.
 *

```

```

* Posture: tenant_id NOT NULL + FORCE RLS canonical policy + (tenant_id, ...) index.
* =====
*/
'use strict';

const TABLE = 'dicom_studies';
const POLICY = 'dicom_studies_tenant_isolation';

async function up(db) {
 await db.query(`
 CREATE TABLE IF NOT EXISTS ${TABLE} (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER,
 rad_exam_id INTEGER, -- FK -> rad_exams.id (worklist row)
 rad_order_id INTEGER, -- FK -> lab_radiology_orders.id (convenience)
 patient_id INTEGER,
 study_uid TEXT DEFAULT '', -- DICOM StudyInstanceUID (metadata only)
 accession TEXT DEFAULT '', -- AccessionNumber (worklist key)
 modality TEXT DEFAULT '', -- DICOM Modality (CT/MR/CR/US/MG/...)
 study_desc TEXT DEFAULT '',
 series_count INTEGER DEFAULT 0,
 instance_count INTEGER DEFAULT 0,
 stored_ref INTEGER, -- soft ref -> phi_files.id (guarded object); NOT a
public path
 source TEXT DEFAULT 'manual', -- 'manual' | 'mwl' (gated) ? never an open PACS pu
ll
 study_at TIMESTAMP,
 created_by INTEGER,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
 `);

 await db.query(`ALTER TABLE ${TABLE} ENABLE ROW LEVEL SECURITY`);
 await db.query(`ALTER TABLE ${TABLE} FORCE ROW LEVEL SECURITY`);

 await db.query(`DROP POLICY IF EXISTS ${POLICY} ON ${TABLE}`);
 await db.query(`
 CREATE POLICY ${POLICY} ON ${TABLE}
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 `);

 await db.query(`CREATE INDEX IF NOT EXISTS idx_dicom_studies_tenant_exam ON ${TABLE} (tenant_id, rad_exam_
id)`);
 await db.query(`CREATE INDEX IF NOT EXISTS idx_dicom_studies_tenant_accession ON ${TABLE} (tenant_id, acce
ssion)`);
 await db.query(`CREATE INDEX IF NOT EXISTS idx_dicom_studies_tenant_uid ON ${TABLE} (tenant_id, study_uid)
`);

 return { ok: true, table: TABLE };
}

```

```

async function validate(db) {
 const checks = {};
 const t = await db.query(`SELECT to_regclass('public.${TABLE}') AS reg`);
 checks.tableExists = !!t.rows[0].reg;

 const col = await db.query(
 `SELECT is_nullable FROM information_schema.columns WHERE table_name=$1 AND column_name='tenant_id`,
 [TABLE]
);
 checks.tenantIdNotNull = col.rows.length === 1 && col.rows[0].is_nullable === 'NO';

 const rls = await db.query(
 `SELECT relrowsecurity, relforcerowsecurity FROM pg_class WHERE relname=$1`, [TABLE]
);
 checks.rlsEnabled = rls.rows.length === 1 && rls.rows[0].relrowsecurity === true;
 checks.rlsForced = rls.rows.length === 1 && rls.rows[0].relforcerowsecurity === true;

 const pol = await db.query(`SELECT polname FROM pg_policy WHERE polname=$1`, [POLICY]);
 checks.policyExists = pol.rows.length === 1;

 const idx = await db.query(
 `SELECT indexname FROM pg_indexes WHERE tablename=$1 AND indexname='idx_dicom_studies_tenant_accession`,
 [TABLE]
);
 checks.tenantIndexExists = idx.rows.length === 1;

 checks.ok = Object.values(checks).every(Boolean);
 return checks;
}

async function down(db) {
 await db.query(`DROP POLICY IF EXISTS ${POLICY} ON ${TABLE}`);
 await db.query(`ALTER TABLE IF EXISTS ${TABLE} NO FORCE ROW LEVEL SECURITY`);
 await db.query(`ALTER TABLE IF EXISTS ${TABLE} DISABLE ROW LEVEL SECURITY`);
 await db.query(`DROP INDEX IF EXISTS idx_dicom_studies_tenant_exam`);
 await db.query(`DROP INDEX IF EXISTS idx_dicom_studies_tenant_accession`);
 await db.query(`DROP INDEX IF EXISTS idx_dicom_studies_tenant_uid`);
 await db.query(`DROP TABLE IF EXISTS ${TABLE}`);
 return { ok: true, dropped: TABLE };
}

module.exports = { up, validate, down, TABLE, POLICY };

```

```

=====
FILE: migrations\e4_03_rad_reports_up_validate_down.js
=====

```

```

/**
 * e4_03_rad_reports_up_validate_down.js
 * =====
 * E4 ? STRUCTURED RADIOLOGY REPORTING + CRITICAL + PRIOR COMPARE
 * (candidate migration; NOT auto-run; gated; idempotent)

```

```

*
* Adds `rad_reports`: a structured radiology report with templated findings
* (e.g. BI-RADS) stored as `structured_json`, an `impression`, a critical-finding
* flag, signing fields, and an addendum chain (`addendum_of`).
*
* CRITICAL-RESULT FAIL-CLOSED RULE (enforced in the server, schema-supported here):
* A report flagged `is_critical = TRUE` CANNOT be signed (`signed_at`/`signed_by`)
* until a critical notification has been documented & tracked by
* `critical_notified_at` (and a `notifications` row of type 'critical' + audit).
* The server refuses the sign transition when is_critical AND NOT critical_notified.
*
* PRIOR-COMPARE RULE: the prior-study list returns only SIGNED reports for the same
* patient + modality within the tenant (the E3 Issue-1 lesson: signed priors only).
*
* Posture: tenant_id NOT NULL + FORCE RLS canonical policy + (tenant_id, ...) index.
* =====
*/
'use strict';

const TABLE = 'rad_reports';
const POLICY = 'rad_reports_tenant_isolation';

async function up(db) {
 await db.query(`
 CREATE TABLE IF NOT EXISTS ${TABLE} (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL,
 facility_id INTEGER,
 rad_exam_id INTEGER, -- FK -> rad_exams.id
 rad_order_id INTEGER, -- FK -> lab_radiology_orders.id
 patient_id INTEGER,
 modality TEXT DEFAULT '',
 template TEXT DEFAULT '', -- e.g. 'BI-RADS', 'generic'
 structured_json TEXT DEFAULT '', -- JSON string of templated findings
 findings TEXT DEFAULT '',
 impression TEXT DEFAULT '',
 birads TEXT DEFAULT '', -- BI-RADS category when applicable
 is_critical BOOLEAN DEFAULT FALSE,
 critical_notified_at TIMESTAMP, -- set when critical notification documented
 critical_notify_ref INTEGER, -- soft ref -> notifications.id
 status TEXT NOT NULL DEFAULT 'Draft', -- Draft | Signed | Added
 radiologist_id INTEGER,
 signed_by INTEGER,
 signed_at TIMESTAMP,
 addendum_of INTEGER, -- self-ref -> rad_reports.id (prior signed report)
 prior_study_id INTEGER, -- ref -> rad_reports.id used as comparison prior
 created_by INTEGER,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
 `);

 await db.query(`
 DO $$

```

```

BEGIN
 IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname = 'rad_reports_status_chk') THEN
 ALTER TABLE ${TABLE} ADD CONSTRAINT rad_reports_status_chk
 CHECK (status IN ('Draft', 'Signed', 'Addended'));
 END IF;
END $$;
`);

await db.query(`ALTER TABLE ${TABLE} ENABLE ROW LEVEL SECURITY`);
await db.query(`ALTER TABLE ${TABLE} FORCE ROW LEVEL SECURITY`);

await db.query(`DROP POLICY IF EXISTS ${POLICY} ON ${TABLE}`);
await db.query(`
 CREATE POLICY ${POLICY} ON ${TABLE}
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
`);

await db.query(`CREATE INDEX IF NOT EXISTS idx_rad_reports_tenant_exam ON ${TABLE} (tenant_id, rad_exam_id
)`);
await db.query(`CREATE INDEX IF NOT EXISTS idx_rad_reports_tenant_patient_modality ON ${TABLE} (tenant_id,
patient_id, modality)`);
await db.query(`CREATE INDEX IF NOT EXISTS idx_rad_reports_tenant_status ON ${TABLE} (tenant_id, status)`);
;

return { ok: true, table: TABLE };
}

async function validate(db) {
 const checks = {};
 const t = await db.query(`SELECT to_regclass('public.${TABLE}') AS reg`);
 checks.tableExists = !!t.rows[0].reg;

 const col = await db.query(
 `SELECT is_nullable FROM information_schema.columns WHERE table_name=$1 AND column_name='tenant_id`,
[TABLE]
);
 checks.tenantIdNotNull = col.rows.length === 1 && col.rows[0].is_nullable === 'NO';

 const crit = await db.query(
 `SELECT column_name FROM information_schema.columns WHERE table_name=$1 AND column_name='critical_noti
fied_at`, [TABLE]
);
 checks.criticalNotifyColumn = crit.rows.length === 1;

 const rls = await db.query(
 `SELECT relrowsecurity, relforcerowsecurity FROM pg_class WHERE relname=$1`, [TABLE]
);
 checks.rlsEnabled = rls.rows.length === 1 && rls.rows[0].relrowsecurity === true;
 checks.rlsForced = rls.rows.length === 1 && rls.rows[0].relforcerowsecurity === true;

 const pol = await db.query(`SELECT polname FROM pg_policy WHERE polname=$1`, [POLICY]);
 checks.policyExists = pol.rows.length === 1;

```

```

 const idx = await db.query(
 `SELECT indexname FROM pg_indexes WHERE tablename=$1 AND indexname='idx_rad_reports_tenant_patient_modality'`, [TABLE]
);
 checks.priorCompareIndexExists = idx.rows.length === 1;

 checks.ok = Object.values(checks).every(Boolean);
 return checks;
 }

```

```

async function down(db) {
 await db.query(`DROP POLICY IF EXISTS ${POLICY} ON ${TABLE}`);
 await db.query(`ALTER TABLE IF EXISTS ${TABLE} NO FORCE ROW LEVEL SECURITY`);
 await db.query(`ALTER TABLE IF EXISTS ${TABLE} DISABLE ROW LEVEL SECURITY`);
 await db.query(`DROP INDEX IF EXISTS idx_rad_reports_tenant_exam`);
 await db.query(`DROP INDEX IF EXISTS idx_rad_reports_tenant_patient_modality`);
 await db.query(`DROP INDEX IF EXISTS idx_rad_reports_tenant_status`);
 await db.query(`ALTER TABLE IF EXISTS ${TABLE} DROP CONSTRAINT IF EXISTS rad_reports_status_chk`);
 await db.query(`DROP TABLE IF EXISTS ${TABLE}`);
 return { ok: true, dropped: TABLE };
}

```

```

module.exports = { up, validate, down, TABLE, POLICY };

```

```

=====

```

```

FILE: migrations\e50a_cardiology_specialized_down.sql

```

```

=====

```

```

-- Down migration for General Cardiology Specialized Data
-- Target: namaweb/migrations/e50_cardiology_specialized_down.sql

```

```

DROP TABLE IF EXISTS cardiology_echo;
DROP TABLE IF EXISTS cardiology_exams;

```

```

=====

```

```

FILE: migrations\e50a_cardiology_specialized_up.sql

```

```

=====

```

```

-- Migration for General Cardiology Specialized Data
-- Target: namaweb/migrations/e50_cardiology_specialized_up.sql

```

```

CREATE TABLE IF NOT EXISTS cardiology_exams (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 encounter_id UUID NOT NULL,
 tenant_id UUID NOT NULL,
 heart_sounds TEXT,
 jvp_height DECIMAL(4,2),
 edema_grade INTEGER CHECK (edema_grade BETWEEN 0 AND 4),

```



```

 carotid_bruit BOOLEAN DEFAULT FALSE,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS cardiology_echo (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 encounter_id UUID NOT NULL,
 tenant_id UUID NOT NULL,
 ef_percent DECIMAL(5,2),
 lv_dimension DECIMAL(5,2),
 mitral_regurgitation_grade INTEGER CHECK (mitral_regurgitation_grade BETWEEN 0 AND 4),
 aortic_stenosis_area DECIMAL(5,2),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE cardiology_exams ENABLE ROW LEVEL SECURITY;
ALTER TABLE cardiology_echo ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON cardiology_exams
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON cardiology_echo
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

=====
FILE: migrations\e50_cardiology_extensions_down.sql
=====

-- Migration: e50_cardiology_extensions_down.sql
-- Description: Reverses the cardiology extensions migration.

BEGIN;

DROP TABLE IF EXISTS nuclear_imaging_results;
DROP TABLE IF EXISTS ep_mapping_data;
DROP TABLE IF EXISTS cardiology_procedures;

COMMIT;

=====
FILE: migrations\e50_cardiology_extensions_up.sql
=====

-- Migration: e50_cardiology_extensions_up.sql
-- Description: Adds specialized tables for Interventional, EP, and Nuclear Cardiology.
-- Compliance: PDPL / RLS Enabled

```

```
BEGIN;
```

```
-- 1. Interventional Cardiology (Cath Lab)
```

```
CREATE TABLE IF NOT EXISTS cardiology_procedures (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 procedure_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 operator_id UUID NOT NULL,
 procedure_type VARCHAR(100), -- e.g., PCI, TAVI, Angiography
 d2b_time_minutes INTEGER, -- Door-to-Balloon time for STEMI
 contrast_volume_ml INTEGER,
 vessel_target VARCHAR(255),
 stenosis_percentage INTEGER,
 stent_type VARCHAR(100),
 outcome VARCHAR(255),
 notes TEXT,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
 CONSTRAINT fk_patient FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
-- 2. Electrophysiology (EP) Mapping
```

```
CREATE TABLE IF NOT EXISTS ep_mapping_data (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 procedure_id UUID NOT NULL,
 electrode_site VARCHAR(100),
 voltage_mv FLOAT,
 timing_ms INTEGER,
 ablation_energy_joules FLOAT,
 is_successful BOOLEAN DEFAULT FALSE,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_tenant_ep FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
 CONSTRAINT fk_proc_ep FOREIGN KEY (procedure_id) REFERENCES cardiology_procedures(id) ON DELETE CASCADE
);
```

```
-- 3. Nuclear Cardiology Results
```

```
CREATE TABLE IF NOT EXISTS nuclear_imaging_results (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 isotope_used VARCHAR(100),
 perfusion_score INTEGER,
 ejection_fraction_pct INTEGER,
 is_reversible_ischemia BOOLEAN,
 imaging_report_path TEXT, -- Path to phi_vault/
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_tenant_nuc FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
 CONSTRAINT fk_patient_nuc FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```

-- Enable Row Level Security (RLS)
ALTER TABLE cardiology_procedures ENABLE ROW LEVEL SECURITY;
ALTER TABLE ep_mapping_data ENABLE ROW LEVEL SECURITY;
ALTER TABLE nuclear_imaging_results ENABLE ROW LEVEL SECURITY;

-- Create RLS Policies (Golden Access Rule: Tenant Isolation)
CREATE POLICY tenant_isolation_policy ON cardiology_procedures
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON ep_mapping_data
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON nuclear_imaging_results
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

COMMIT;

=====
FILE: migrations\e50_clinical_qls_candidate_down.sql
=====

-- e50_clinical_qls_candidate_down.sql ? rollback for the e50 candidate.
-- Removes the RLS policies and the added constraints/columns. Dropping RLS RE-OPENS
-- cross-tenant access to PHI (patient_clinical_records) ? only run deliberately in a
-- controlled window. tenant_id columns are dropped last (data-losing on those columns).

BEGIN;

-- 4. restore global unique on clinical_departments.code
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_constraint WHERE conrelid = 'clinical_departments'::regclass AND conname = 'uq
_clinical_dept_tenant_code') THEN
 ALTER TABLE clinical_departments DROP CONSTRAINT uq_clinical_dept_tenant_code;
 END IF;
 IF NOT EXISTS (
 SELECT 1 FROM pg_constraint c
 WHERE c.conrelid = 'clinical_departments'::regclass AND c.contype = 'u'
 AND pg_get_constraintdef(c.oid) = 'UNIQUE (code)'
) THEN
 ALTER TABLE clinical_departments ADD CONSTRAINT clinical_departments_code_key UNIQUE (code);
 END IF;
END $$;

-- 3. clinical_knowledge_vectors
DO $$
BEGIN
 IF to_regclass('public.clinical_knowledge_vectors') IS NOT NULL THEN
 DROP POLICY IF EXISTS rls_clinical_knowledge_vectors ON clinical_knowledge_vectors;
 ALTER TABLE clinical_knowledge_vectors NO FORCE ROW LEVEL SECURITY;
 ALTER TABLE clinical_knowledge_vectors DISABLE ROW LEVEL SECURITY;
 ALTER TABLE clinical_knowledge_vectors DROP COLUMN IF EXISTS tenant_id;

```

```

END IF;
END $$;

-- 2. clinical_templates
DROP POLICY IF EXISTS rls_clinical_templates ON clinical_templates;
ALTER TABLE clinical_templates NO FORCE ROW LEVEL SECURITY;
ALTER TABLE clinical_templates DISABLE ROW LEVEL SECURITY;
ALTER TABLE clinical_templates DROP CONSTRAINT IF EXISTS fk_ct_tenant;
ALTER TABLE clinical_templates DROP COLUMN IF EXISTS tenant_id;

-- 1. patient_clinical_records
DROP POLICY IF EXISTS rls_patient_clinical_records ON patient_clinical_records;
ALTER TABLE patient_clinical_records NO FORCE ROW LEVEL SECURITY;
ALTER TABLE patient_clinical_records DISABLE ROW LEVEL SECURITY;
ALTER TABLE patient_clinical_records DROP CONSTRAINT IF EXISTS fk_pcr_tenant;

COMMIT;

=====
FILE: migrations\e50_clinical_rls_candidate_up.sql
=====

-- e50_clinical_rls_candidate_up.sql ? CANDIDATE, NOT RUN (requires owner DDL approval).
-- Gate 5 (Wave 2 isolation): close the remaining e21 clinical-metadata isolation gaps.
--
-- Findings (from Gate 0 + Gate 5 code read):
-- 1. patient_clinical_records HOLDS PHI (recorded_values JSONB) but has NO RLS and no FK
-- to tenants ? app-layer WHERE tenant_id=$ is the only guard (no DB backstop).
-- 2. clinical_templates has NO tenant_id and NO RLS (isolation only transitive via the
-- RLS-protected clinical_departments; direct dept_id reads were unguarded until the
-- Gate 5 code fix added a JOIN).
-- 3. clinical_knowledge_vectors (RAG chunks) has NO tenant_id and NO RLS.
-- 4. clinical_departments.code is GLOBALLY UNIQUE, so `ON CONFLICT (code)` (server.js
-- POST /api/clinical/departments) collides across tenants ? two tenants cannot both
-- own 'CARDIOLOGY'. Correct constraint is UNIQUE (tenant_id, code).
--
-- This migration is idempotent and additive. It BACKFILLS tenant_id where the column is
-- newly added: clinical_templates/knowledge_vectors inherit tenant_id from their parent
-- department (which is already tenant-scoped). Rows whose parent tenant is unresolved are
-- left NULL and the NOT NULL / FK steps are guarded so the migration fails loudly rather
-- than silently mis-assigning PHI/metadata to tenant 1.
--
-- ORDER OF OPERATIONS matters: run this DDL FIRST, THEN flip server.js
-- POST /api/clinical/departments to `ON CONFLICT (tenant_id, code)` (coupled code change,
-- documented in the Gate 5 report ? do NOT flip the code before this lands).

BEGIN;

-- ----- 1. patient_clinical_records: FK + FORCE RLS (PHI) -----
DO $$
BEGIN
 IF to_regclass('public.patient_clinical_records') IS NOT NULL THEN

```

```

-- FK to tenants (only if every tenant_id already resolves ? else the ADD fails loudly)
IF NOT EXISTS (
 SELECT 1 FROM information_schema.table_constraints
 WHERE table_name = 'patient_clinical_records' AND constraint_type = 'FOREIGN KEY'
 AND constraint_name = 'fk_pcr_tenant'
) THEN
 ALTER TABLE patient_clinical_records
 ADD CONSTRAINT fk_pcr_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
END IF;

ALTER TABLE patient_clinical_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE patient_clinical_records FORCE ROW LEVEL SECURITY;
EXECUTE 'DROP POLICY IF EXISTS rls_patient_clinical_records ON patient_clinical_records';
EXECUTE 'CREATE POLICY rls_patient_clinical_records ON patient_clinical_records
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), ''))::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), ''))::integer)';
END IF;
END $$;

-- ----- 2. clinical_templates: add tenant_id (backfill from dept) + FORCE RLS -----
ALTER TABLE clinical_templates ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
UPDATE clinical_templates t
 SET tenant_id = d.tenant_id
 FROM clinical_departments d
 WHERE d.id = t.department_id AND t.tenant_id IS NULL;
-- fail loudly if any template could not inherit a tenant (orphan department)
DO $$
DECLARE orphan_count INTEGER;
BEGIN
 SELECT count(*) INTO orphan_count FROM clinical_templates WHERE tenant_id IS NULL;
 IF orphan_count > 0 THEN
 RAISE EXCEPTION 'e50: % clinical_templates rows have unresolved tenant_id ? resolve before enforcing',
 orphan_count;
 END IF;
END $$;
ALTER TABLE clinical_templates ALTER COLUMN tenant_id SET NOT NULL;
DO $$
BEGIN
 IF NOT EXISTS (
 SELECT 1 FROM information_schema.table_constraints
 WHERE table_name = 'clinical_templates' AND constraint_name = 'fk_ct_tenant'
) THEN
 ALTER TABLE clinical_templates
 ADD CONSTRAINT fk_ct_tenant FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE;
 END IF;
END $$;
ALTER TABLE clinical_templates ENABLE ROW LEVEL SECURITY;
ALTER TABLE clinical_templates FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_clinical_templates ON clinical_templates;
CREATE POLICY rls_clinical_templates ON clinical_templates
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), ''))::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), ''))::integer);
CREATE INDEX IF NOT EXISTS idx_clinical_templates_tenant ON clinical_templates (tenant_id);

```

```

-- ----- 3. clinical_knowledge_vectors: add tenant_id (backfill from dept) + FORCE RLS -----
DO $$
BEGIN
 IF to_regclass('public.clinical_knowledge_vectors') IS NOT NULL THEN
 ALTER TABLE clinical_knowledge_vectors ADD COLUMN IF NOT EXISTS tenant_id INTEGER;
 UPDATE clinical_knowledge_vectors v
 SET tenant_id = d.tenant_id
 FROM clinical_departments d
 WHERE d.id = v.department_id AND v.tenant_id IS NULL;
 -- department_id is nullable here; global (dept-less) chunks stay NULL and are left
 -- readable to all tenants ONLY if a permissive policy is chosen. We instead require a
 -- tenant: fail loudly on NULL so no cross-tenant RAG leakage is silently allowed.
 IF EXISTS (SELECT 1 FROM clinical_knowledge_vectors WHERE tenant_id IS NULL) THEN
 RAISE EXCEPTION 'e50: clinical_knowledge_vectors has rows with unresolved tenant_id (dept-less chu
nks) ? assign a tenant or add a deliberate shared-catalog policy before enforcing';
 END IF;
 ALTER TABLE clinical_knowledge_vectors ALTER COLUMN tenant_id SET NOT NULL;
 ALTER TABLE clinical_knowledge_vectors ENABLE ROW LEVEL SECURITY;
 ALTER TABLE clinical_knowledge_vectors FORCE ROW LEVEL SECURITY;
 DROP POLICY IF EXISTS rls_clinical_knowledge_vectors ON clinical_knowledge_vectors;
 CREATE POLICY rls_clinical_knowledge_vectors ON clinical_knowledge_vectors
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
 CREATE INDEX IF NOT EXISTS idx_ckv_tenant ON clinical_knowledge_vectors (tenant_id);
 END IF;
END $$;

-- ----- 4. clinical_departments: per-tenant uniqueness on code -----
-- Replace the global UNIQUE(code) with UNIQUE(tenant_id, code). The old constraint name is
-- Postgres-generated (clinical_departments_code_key); drop defensively by discovery.
DO $$
DECLARE v_conname TEXT;
BEGIN
 SELECT c.conname INTO v_conname
 FROM pg_constraint c
 WHERE c.conrelid = 'clinical_departments'::regclass AND c.contype = 'u'
 AND pg_get_constraintdef(c.oid) = 'UNIQUE (code)';
 IF v_conname IS NOT NULL THEN
 EXECUTE format('ALTER TABLE clinical_departments DROP CONSTRAINT %I', v_conname);
 END IF;
 IF NOT EXISTS (
 SELECT 1 FROM pg_constraint
 WHERE conrelid = 'clinical_departments'::regclass AND conname = 'uq_clinical_dept_tenant_code'
) THEN
 ALTER TABLE clinical_departments ADD CONSTRAINT uq_clinical_dept_tenant_code UNIQUE (tenant_id, code);
 END IF;
END $$;

-- grants (mirror the repo convention; only if the app roles exist)
DO $$
BEGIN
 IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'nama_medical_app') THEN
 IF to_regclass('public.patient_clinical_records') IS NOT NULL THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE patient_clinical_records TO nama_medical_app';
 END IF;
 END IF;
END $$;

```

```

END IF;
IF to_regclass('public.clinical_templates') IS NOT NULL THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE clinical_templates TO nama_medical_app';
END IF;
END IF;
IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'jumanasoft_staging_user') THEN
 IF to_regclass('public.patient_clinical_records') IS NOT NULL THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE patient_clinical_records TO jumanasoft_staging_user';
 END IF;
 IF to_regclass('public.clinical_templates') IS NOT NULL THEN
 EXECUTE 'GRANT ALL PRIVILEGES ON TABLE clinical_templates TO jumanasoft_staging_user';
 END IF;
END IF;
END $$;

```

```

COMMIT;

```

```

=====
FILE: migrations\e50_clinical_rls_candidate_validate.sql
=====

```

```

-- e50_clinical_rls_candidate_validate.sql ? post-apply validation for the e50 candidate.

```

```

DO $$
DECLARE
 forced BOOLEAN;
BEGIN
 -- 1. patient_clinical_records FORCE RLS + policy + FK
 SELECT relforcerowsecurity INTO forced FROM pg_class WHERE relname = 'patient_clinical_records';
 IF forced IS NOT TRUE THEN
 RAISE EXCEPTION 'e50 validate: FORCE RLS not enabled on patient_clinical_records';
 END IF;
 IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE tablename = 'patient_clinical_records' AND policyname = 'rls_patient_clinical_records') THEN
 RAISE EXCEPTION 'e50 validate: policy missing on patient_clinical_records';
 END IF;
 IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE table_name = 'patient_clinical_records' AND constraint_name = 'fk_pcr_tenant') THEN
 RAISE EXCEPTION 'e50 validate: FK fk_pcr_tenant missing';
 END IF;

 -- 2. clinical_templates FORCE RLS + tenant_id NOT NULL
 SELECT relforcerowsecurity INTO forced FROM pg_class WHERE relname = 'clinical_templates';
 IF forced IS NOT TRUE THEN
 RAISE EXCEPTION 'e50 validate: FORCE RLS not enabled on clinical_templates';
 END IF;
 IF EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'clinical_templates' AND column_name = 'tenant_id' AND is_nullable = 'YES') THEN
 RAISE EXCEPTION 'e50 validate: clinical_templates.tenant_id must be NOT NULL';
 END IF;

 -- 4. clinical_departments per-tenant unique
 IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conrelid = 'clinical_departments'::regclass AND conname =

```

```

'uq_clinical_dept_tenant_code') THEN
 RAISE EXCEPTION 'e50 validate: uq_clinical_dept_tenant_code missing';
END IF;
IF EXISTS (
 SELECT 1 FROM pg_constraint c
 WHERE c.conrelid = 'clinical_departments'::regclass AND c.contype = 'u'
 AND pg_get_constraintdef(c.oid) = 'UNIQUE (code)'
) THEN
 RAISE EXCEPTION 'e50 validate: global UNIQUE(code) still present ? should be per-tenant';
END IF;

RAISE NOTICE 'e50 validate OK';
END $$;

```

```

=====
FILE: migrations\e51_cath_lab_specialized_down.sql
=====

```

```

-- Down migration for Interventional Cardiology (Cath Lab)
-- Target: namaweb/migrations/e51_cath_lab_specialized_down.sql

```

```

DROP TABLE IF EXISTS stent_registry;
DROP TABLE IF EXISTS cath_lab_procedures;

```

```

=====
FILE: migrations\e51_cath_lab_specialized_up.sql
=====

```

```

-- Migration for Interventional Cardiology (Cath Lab)
-- Target: namaweb/migrations/e51_cath_lab_specialized_up.sql

```

```

CREATE TABLE IF NOT EXISTS cath_lab_procedures (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 encounter_id UUID NOT NULL,
 tenant_id UUID NOT NULL,
 operator_id UUID NOT NULL REFERENCES system_users(id),
 access_site TEXT CHECK (access_site IN ('Radial', 'Femoral', 'Brachial', 'Other')),
 fluoroscopy_time_min DECIMAL(5,2),
 contrast_volume_ml DECIMAL(6,2),
 procedure_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 status TEXT CHECK (status IN ('Scheduled', 'In-Progress', 'Completed', 'Cancelled')),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE IF NOT EXISTS stent_registry (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 procedure_id UUID NOT NULL REFERENCES cath_lab_procedures(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,

```



```

vessel_segment TEXT NOT NULL, -- e.g., LAD, LCx, RCA
stent_brand TEXT,
diameter_mm DECIMAL(4,2),
length_mm DECIMAL(4,2),
material TEXT, -- e.g., DES, BMS
pressure_post_dilation DECIMAL(5,2),
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

```
-- Enable RLS
```

```
ALTER TABLE cath_lab_procedures ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE stent_registry ENABLE ROW LEVEL SECURITY;
```

```

CREATE POLICY tenant_isolation_policy ON cath_lab_procedures
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

```

```

CREATE POLICY tenant_isolation_policy ON stent_registry
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

```

```

=====
FILE: migrations\e52_ep_specialized_down.sql
=====

```

```

-- Down migration for Electrophysiology (EP) Specialized Data
-- Target: namaweb/migrations/e52_ep_specialized_down.sql

```

```

DROP TABLE IF EXISTS ep_device_registry;
DROP TABLE IF EXISTS ep_ablation_logs;

```

```

=====
FILE: migrations\e52_ep_specialized_up.sql
=====

```

```

-- Migration for Electrophysiology (EP) Specialized Data
-- Target: namaweb/migrations/e52_ep_specialized_up.sql

```

```

CREATE TABLE IF NOT EXISTS ep_ablation_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 procedure_id UUID NOT NULL, -- Linked to cath_lab_procedures or general encounter
 tenant_id UUID NOT NULL,
 site_name TEXT NOT NULL, -- e.g., 'Pulmonary Vein', 'AV Node'
 energy_joules DECIMAL(5,2),
 duration_sec DECIMAL(5,2),
 modality TEXT CHECK (modality IN ('RF', 'Cryo', 'Laser')),
 success_indicator BOOLEAN DEFAULT FALSE,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE IF NOT EXISTS ep_device_registry (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 device_type TEXT CHECK (device_type IN ('Pacemaker', 'ICD', 'CRT', 'Other')),
 model TEXT,
 lead_position TEXT,
 sensitivity DECIMAL(5,2),
 output_voltage DECIMAL(5,2),
 implantation_date DATE,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE ep_ablation_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE ep_device_registry ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON ep_ablation_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON ep_device_registry
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

=====
FILE: migrations\e53_nuclear_cardiology_down.sql
=====

-- Down migration for Nuclear Cardiology Specialized Data
-- Target: namaweb/migrations/e53_nuclear_cardiology_down.sql

DROP TABLE IF EXISTS nuclear_perfusion_results;
DROP TABLE IF EXISTS nuclear_cardiology_scans;

=====
FILE: migrations\e53_nuclear_cardiology_up.sql
=====

-- Migration for Nuclear Cardiology Specialized Data
-- Target: namaweb/migrations/e53_nuclear_cardiology_up.sql

CREATE TABLE IF NOT EXISTS nuclear_cardiology_scans (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 encounter_id UUID NOT NULL,
 tenant_id UUID NOT NULL,
 tracer_used TEXT, -- e.g., 'Technetium-99m Sestamibi'
 dose_mci DECIMAL(6,2),
 scan_type TEXT CHECK (scan_type IN ('SPECT', 'PET', 'Hybrid')),
 stress_type TEXT CHECK (stress_type IN ('Exercise', 'Pharmacological', 'None')),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

```
CREATE TABLE IF NOT EXISTS nuclear_perfusion_results (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 scan_id UUID NOT NULL REFERENCES nuclear_cardiology_scans(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 wall_motion_abnormality TEXT,
 perfusion_defect_location TEXT,
 viability_status TEXT CHECK (viability_status IN ('Viable', 'Non-viable', 'Indeterminate')),
 ejection_fraction_nuclear DECIMAL(5,2),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Enable RLS
```

```
ALTER TABLE nuclear_cardiology_scans ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE nuclear_perfusion_results ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation_policy ON nuclear_cardiology_scans
 USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
CREATE POLICY tenant_isolation_policy ON nuclear_perfusion_results
 USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
=====
```

```
FILE: migrations\e54_preventive_cardio_ob_down.sql
```

```
=====
```

```
-- Down migration for Preventive Cardiology & Cardio-Obstetrics
```

```
-- Target: namaweb/migrations/e54_preventive_cardio_ob_down.sql
```

```
DROP TABLE IF EXISTS cardio_obstetrics_records;
```

```
DROP TABLE IF EXISTS preventive_cardio_profiles;
```

```
=====
```

```
FILE: migrations\e54_preventive_cardio_ob_up.sql
```

```
=====
```

```
-- Migration for Preventive Cardiology & Cardio-Obstetrics
```

```
-- Target: namaweb/migrations/e54_preventive_cardio_ob_up.sql
```

```
CREATE TABLE IF NOT EXISTS preventive_cardio_profiles (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 smoking_status TEXT CHECK (smoking_status IN ('Never', 'Former', 'Current')),
 alcohol_intake TEXT,
 activity_level TEXT,
 family_history_score INTEGER,
 target_ldl DECIMAL(5,2),
 target_bp_systolic INTEGER,
 target_bp_diastolic INTEGER,
```

```

 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS cardio_obstetrics_records (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 pregnancy_stage TEXT, -- e.g., 'First Trimester', 'Second Trimester', 'Third Trimester'
 cardiac_condition TEXT,
 fetal_impact_score INTEGER,
 delivery_mode_recommendation TEXT,
 maternal_risk_level TEXT CHECK (risk_level IN ('Low', 'Moderate', 'High', 'Critical')),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE preventive_cardio_profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE cardio_obstetrics_records ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON preventive_cardio_profiles
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON cardio_obstetrics_records
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

=====
FILE: migrations\e55_vascular_ahf_down.sql
=====

-- Down migration for Peripheral Vascular & Advanced Heart Failure
-- Target: namaweb/migrations/e55_vascular_ahf_down.sql

DROP TABLE IF EXISTS vad_registry;
DROP TABLE IF EXISTS ahf_monitoring;
DROP TABLE IF EXISTS vascular_abi_logs;

=====
FILE: migrations\e55_vascular_ahf_up.sql
=====

-- Migration for Peripheral Vascular & Advanced Heart Failure
-- Target: namaweb/migrations/e55_vascular_ahf_up.sql

CREATE TABLE IF NOT EXISTS vascular_abi_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 left_ankle_sys DECIMAL(5,2),

```

```

 right_ankle_sys DECIMAL(5,2),
 left_arm_sys DECIMAL(5,2),
 right_arm_sys DECIMAL(5,2),
 calculated_abi_l DECIMAL(4,2),
 calculated_abi_r DECIMAL(4,2),
 rutherford_class INTEGER CHECK (rutherford_class BETWEEN 0 AND 6),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS ahf_monitoring (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 bnp_level DECIMAL(10,2),
 weight_kg DECIMAL(5,2),
 nyha_class INTEGER CHECK (nyha_class BETWEEN 1 AND 4),
 lvef_percent DECIMAL(5,2),
 fluid_balance_liters DECIMAL(5,2),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS vad_registry (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 device_model TEXT,
 flow_rate DECIMAL(5,2),
 speed_rpm INTEGER,
 power_watts DECIMAL(5,2),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE vascular_abi_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE ahf_monitoring ENABLE ROW LEVEL SECURITY;
ALTER TABLE vad_registry ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON vascular_abi_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON ahf_monitoring
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON vad_registry
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

=====
FILE: migrations\e56_pulmonology_extensions_down.sql
=====

-- Migration: e56_pulmonology_extensions_down.sql
-- Description: Reverses the pulmonology extensions migration.

```

```
BEGIN;
```

```
DROP TABLE IF EXISTS bronchoscopy_reports;
```

```
DROP TABLE IF EXISTS sleep_study_results;
```

```
DROP TABLE IF EXISTS pulmonary_function_tests;
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e56_pulmonology_extensions_up.sql
```

```
=====
```

```
-- Migration: e56_pulmonology_extensions_up.sql
```

```
-- Description: Adds specialized tables for Pulmonology, Sleep Medicine, and Bronchoscopy.
```

```
-- Compliance: PDPL / RLS Enabled
```

```
BEGIN;
```

```
-- 1. Pulmonary Function Tests (PFT)
```

```
CREATE TABLE IF NOT EXISTS pulmonary_function_tests (
```

```
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
 tenant_id UUID NOT NULL,
```

```
 patient_id UUID NOT NULL,
```

```
 test_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
 fev1_liters FLOAT,
```

```
 fvc_liters FLOAT,
```

```
 fev1_fvc_ratio FLOAT,
```

```
 peak_flow_l_sec FLOAT,
```

```
 interpretation TEXT,
```

```
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
 CONSTRAINT fk_tenant_pft FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
```

```
 CONSTRAINT fk_patient_pft FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
```

```
);
```

```
-- 2. Sleep Study Results (Polysomnography)
```

```
CREATE TABLE IF NOT EXISTS sleep_study_results (
```

```
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
 tenant_id UUID NOT NULL,
```

```
 patient_id UUID NOT NULL,
```

```
 study_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
 ahi_index FLOAT, -- Apnea-Hypopnea Index
```

```
 odi_index FLOAT, -- Oxygen Desaturation Index
```

```
 arousal_index FLOAT,
```

```
 cpap_pressure_cmh2o FLOAT,
```

```
 sleep_efficiency_pct FLOAT,
```

```
 notes TEXT,
```

```
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
 CONSTRAINT fk_tenant_sleep FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
```

```
 CONSTRAINT fk_patient_sleep FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
```

```
);
```

```
-- 3. Bronchoscopy Reports
CREATE TABLE IF NOT EXISTS bronchoscopy_reports (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 procedure_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 sedation_type VARCHAR(100),
 site_sampled VARCHAR(255),
 biopsy_result TEXT,
 complications TEXT,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_tenant_branch FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
 CONSTRAINT fk_patient_branch FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
-- Enable Row Level Security (RLS)
ALTER TABLE pulmonary_function_tests ENABLE ROW LEVEL SECURITY;
ALTER TABLE sleep_study_results ENABLE ROW LEVEL SECURITY;
ALTER TABLE bronchoscopy_reports ENABLE ROW LEVEL SECURITY;
```

```
-- Create RLS Policies (Golden Access Rule: Tenant Isolation)
CREATE POLICY tenant_isolation_policy ON pulmonary_function_tests
 USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
CREATE POLICY tenant_isolation_policy ON sleep_study_results
 USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
CREATE POLICY tenant_isolation_policy ON bronchoscopy_reports
 USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
COMMIT;
```

```
=====
FILE: migrations\e56_respiratory_specialized_down.sql
=====
```

```
-- Down migration for Pulmonology & Respiratory Specialized Data
-- Target: namaweb/migrations/e56_respiratory_specialized_down.sql
```

```
DROP TABLE IF EXISTS respiratory_oxygen_logs;
DROP TABLE IF EXISTS respiratory_pft_logs;
```

```
=====
FILE: migrations\e56_respiratory_specialized_up.sql
=====
```

```
-- Migration for Pulmonology & Respiratory Specialized Data
-- Target: namaweb/migrations/e56_respiratory_specialized_up.sql
```

```
CREATE TABLE IF NOT EXISTS respiratory_pft_logs (
```

```

 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 encounter_id UUID NOT NULL,
 fev1_predicted DECIMAL(5,2),
 fev1_actual DECIMAL(5,2),
 fvc_predicted DECIMAL(5,2),
 fvc_actual DECIMAL(5,2),
 ratio DECIMAL(4,2),
 dlco_percent DECIMAL(5,2),
 gold_stage INTEGER CHECK (gold_stage BETWEEN 1 AND 4),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS respiratory_oxygen_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 spo2_level INTEGER CHECK (spo2_level BETWEEN 0 AND 100),
 oxygen_flow_rate DECIMAL(4,1),
 delivery_method TEXT CHECK (delivery_method IN ('Nasal Cannula', 'Venturi Mask', 'BIPAP', 'CPAP', 'Intubated')),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE respiratory_pft_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE respiratory_oxygen_logs ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON respiratory_pft_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON respiratory_oxygen_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

=====
FILE: migrations\e57_gastro_extensions_down.sql
=====

-- Migration: e57_gastro_extensions_down.sql
-- Description: Reverses the gastroenterology extensions migration.

BEGIN;

DROP TABLE IF EXISTS gi_motility_studies;
DROP TABLE IF EXISTS hepatology_metrics;
DROP TABLE IF EXISTS gastro_endoscopy_reports;

COMMIT;

```



```

=====
FILE: migrations\e57_gastro_extensions_up.sql
=====

-- Migration: e57_gastro_extensions_up.sql
-- Description: Adds specialized tables for Gastroenterology, Hepatology, and Advanced Endoscopy.
-- Compliance: PDPL / RLS Enabled

BEGIN;

-- 1. Advanced Endoscopy Reports (EUS, ERCP, Enteroscopy)
CREATE TABLE IF NOT EXISTS gastro_endoscopy_reports (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 procedure_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 procedure_type VARCHAR(100), -- e.g., EUS, ERCP, Colonoscopy
 findings TEXT,
 biopsy_taken BOOLEAN DEFAULT FALSE,
 biopsy_site VARCHAR(255),
 outcome VARCHAR(255),
 sedation_type VARCHAR(100),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_tenant_gastro FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
 CONSTRAINT fk_patient_gastro FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

-- 2. Hepatology Metrics (Liver Function & Scoring)
CREATE TABLE IF NOT EXISTS hepatology_metrics (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 record_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 bilirubin_mgdl FLOAT,
 albumin_gdl FLOAT,
 inr_value FLOAT,
 creatinine_mgdl FLOAT,
 meld_score FLOAT,
 child_pugh_score INTEGER,
 ascites_grade INTEGER,
 encephalopathy_grade INTEGER,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_tenant_hep FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
 CONSTRAINT fk_patient_hep FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

-- 3. GI Motility Studies
CREATE TABLE IF NOT EXISTS gi_motility_studies (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 study_type VARCHAR(100), -- e.g., Manometry, Gastric Emptying
 results_summary TEXT,
 transit_time_hours FLOAT,

```

```
conclusion TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tenant_motility FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
CONSTRAINT fk_patient_motility FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

```
-- Enable Row Level Security (RLS)
```

```
ALTER TABLE gastro_endoscopy_reports ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE hepatology_metrics ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE gi_motility_studies ENABLE ROW LEVEL SECURITY;
```

```
-- Create RLS Policies (Golden Access Rule: Tenant Isolation)
```

```
CREATE POLICY tenant_isolation_policy ON gastro_endoscopy_reports
 USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
CREATE POLICY tenant_isolation_policy ON hepatology_metrics
 USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
CREATE POLICY tenant_isolation_policy ON gi_motility_studies
 USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

```
COMMIT;
```

```
=====
```

```
FILE: migrations\e57_respiratory_allergy_sleep_down.sql
```

```
=====
```

```
-- Down migration for Respiratory Allergy & Sleep Medicine
```

```
-- Target: namaweb/migrations/e57_respiratory_allergy_sleep_down.sql
```

```
DROP TABLE IF EXISTS respiratory_cpap_logs;
```

```
DROP TABLE IF EXISTS respiratory_sleep_psg;
```

```
DROP TABLE IF EXISTS respiratory_biologicals;
```

```
DROP TABLE IF EXISTS respiratory_allergy_tests;
```

```
=====
```

```
FILE: migrations\e57_respiratory_allergy_sleep_up.sql
```

```
=====
```

```
-- Migration for Respiratory Allergy & Sleep Medicine
```

```
-- Target: namaweb/migrations/e57_respiratory_allergy_sleep_up.sql
```

```
CREATE TABLE IF NOT EXISTS respiratory_allergy_tests (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 allergen_name TEXT NOT NULL,
 wheal_size_mm DECIMAL(4,2),
 result TEXT CHECK (result IN ('Positive', 'Negative', 'Equivocal')),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
);

CREATE TABLE IF NOT EXISTS respiratory_biologicals (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 drug_name TEXT NOT NULL,
 dose TEXT,
 frequency TEXT,
 response_score INTEGER CHECK (response_score BETWEEN 0 AND 10),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS respiratory_sleep_psg (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 ahi DECIMAL(5,2),
 rdi DECIMAL(5,2),
 lowest_spo2 INTEGER CHECK (lowest_spo2 BETWEEN 0 AND 100),
 sleep_efficiency_percent DECIMAL(5,2),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS respiratory_cpap_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 usage_hours DECIMAL(4,2),
 leak_rate DECIMAL(5,2),
 pressure_cmh2o DECIMAL(4,1),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Enable RLS
ALTER TABLE respiratory_allergy_tests ENABLE ROW LEVEL SECURITY;
ALTER TABLE respiratory_biologicals ENABLE ROW LEVEL SECURITY;
ALTER TABLE respiratory_sleep_psg ENABLE ROW LEVEL SECURITY;
ALTER TABLE respiratory_cpap_logs ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation_policy ON respiratory_allergy_tests
 USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
CREATE POLICY tenant_isolation_policy ON respiratory_biologicals
 USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
CREATE POLICY tenant_isolation_policy ON respiratory_sleep_psg
 USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
CREATE POLICY tenant_isolation_policy ON respiratory_cpap_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

=====

FILE: migrations\e58\_respiratory\_advanced\_down.sql

=====

-- Down migration for Respiratory Advanced  
-- Target: namaweb/migrations/e58\_respiratory\_advanced\_down.sql

DROP TABLE IF EXISTS la\_respiratory\_home\_ox\_logs;  
DROP TABLE IF EXISTS respiratory\_home\_ox\_setup;  
DROP TABLE IF EXISTS respiratory\_biopsy\_samples;  
DROP TABLE IF EXISTS respiratory\_bronchoscopy\_logs;  
DROP TABLE IF EXISTS respiratory\_abg\_logs;  
DROP TABLE IF EXISTS respiratory\_vent\_logs;

=====

FILE: migrations\e58\_respiratory\_advanced\_up.sql

=====

-- Migration for Respiratory Care, Bronchoscopy, and Home Oxygen  
-- Target: namaweb/migrations/e58\_respiratory\_advanced\_up.sql

CREATE TABLE IF NOT EXISTS respiratory\_vent\_logs (  
 id UUID PRIMARY KEY DEFAULT gen\_random\_uuid(),  
 patient\_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,  
 tenant\_id UUID NOT NULL,  
 mode TEXT, -- e.g., 'AC', 'SIMV', 'PSV'  
 peep DECIMAL(4,1),  
 fio2 DECIMAL(4,2),  
 tidal\_volume INTEGER,  
 respiratory\_rate INTEGER,  
 created\_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT\_TIMESTAMP  
);

CREATE TABLE IF NOT EXISTS respiratory\_abg\_logs (  
 id UUID PRIMARY KEY DEFAULT gen\_random\_uuid(),  
 patient\_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,  
 tenant\_id UUID NOT NULL,  
 ph DECIMAL(4,3),  
 po2 DECIMAL(5,2),  
 pco2 DECIMAL(5,2),  
 hco3 DECIMAL(5,2),  
 spo2 INTEGER,  
 created\_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT\_TIMESTAMP  
);

CREATE TABLE IF NOT EXISTS respiratory\_bronchoscopy\_logs (  
 id UUID PRIMARY KEY DEFAULT gen\_random\_uuid(),  
 patient\_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,  
 tenant\_id UUID NOT NULL,  
 procedure\_type TEXT CHECK (procedure\_type IN ('Diagnostic', 'Therapeutic')),  
 sedation\_type TEXT,  
 duration\_min INTEGER,

```

 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS respiratory_biopsy_samples (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 procedure_id UUID NOT NULL REFERENCES respiratory_bronchoscopy_logs(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 site_location TEXT,
 sample_type TEXT, -- e.g., 'BAL', 'Forceps'
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS respiratory_home_ox_setup (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 device_type TEXT CHECK (device_type IN ('Concentrator', 'Cylinder')),
 initial_flow_rate DECIMAL(4,1),
 start_date DATE,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS la_respiratory_home_ox_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 setup_id UUID NOT NULL REFERENCES respiratory_home_ox_setup(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 spo2_level INTEGER,
 current_flow_rate DECIMAL(4,1),
 compliance_hours DECIMAL(4,2),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE respiratory_vent_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE respiratory_abg_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE respiratory_bronchoscopy_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE respiratory_biopsy_samples ENABLE ROW LEVEL SECURITY;
ALTER TABLE respiratory_home_ox_setup ENABLE ROW LEVEL SECURITY;
ALTER TABLE la_respiratory_home_ox_logs ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON respiratory_vent_logs USING (tenant_id = current_setting('app.current_tenant')::uuid);
CREATE POLICY tenant_isolation_policy ON respiratory_abg_logs USING (tenant_id = current_setting('app.current_tenant')::uuid);
CREATE POLICY tenant_isolation_policy ON respiratory_bronchoscopy_logs USING (tenant_id = current_setting('app.current_tenant')::uuid);
CREATE POLICY tenant_isolation_policy ON respiratory_biopsy_samples USING (tenant_id = current_setting('app.current_tenant')::uuid);
CREATE POLICY tenant_isolation_policy ON respiratory_home_ox_setup USING (tenant_id = current_setting('app.current_tenant')::uuid);
CREATE POLICY tenant_isolation_policy ON la_respiratory_home_ox_logs USING (tenant_id = current_setting('app.current_tenant')::uuid);

```

=====

FILE: migrations\e59\_gastro\_hepatology\_down.sql

=====

-- Down migration for Gastroenterology & Hepatology Specialized Data  
-- Target: namaweb/migrations/e59\_gastro\_hepatology\_down.sql

DROP TABLE IF EXISTS gastro\_nutrition\_plans;  
DROP TABLE IF EXISTS gastro\_liver\_metrics;  
DROP TABLE IF EXISTS gastro\_endoscopy\_logs;

=====

FILE: migrations\e59\_gastro\_hepatology\_up.sql

=====

-- Migration for Gastroenterology & Hepatology Specialized Data  
-- Target: namaweb/migrations/e59\_gastro\_hepatology\_up.sql

```
CREATE TABLE IF NOT EXISTS gastro_endoscopy_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 encounter_id UUID NOT NULL,
 procedure_type TEXT CHECK (procedure_type IN ('EGD', 'Colonoscopy', 'ERCP', 'EUS', 'Enteroscopy')),
 boston_scale_right INTEGER CHECK (boston_scale_right BETWEEN 0 AND 3),
 boston_scale_left INTEGER CHECK (boston_scale_left BETWEEN 0 AND 3),
 findings TEXT,
 biopsy_taken BOOLEAN DEFAULT FALSE,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS gastro_liver_metrics (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 bilirubin DECIMAL(5,2),
 albumin DECIMAL(5,2),
 creatinine DECIMAL(5,2),
 ascites_grade INTEGER CHECK (ascites_grade BETWEEN 0 AND 3),
 encephalopathy_grade INTEGER CHECK (encephalopathy_grade BETWEEN 0 AND 4),
 meld_score DECIMAL(4,2),
 child_pugh_class TEXT CHECK (child_pugh_class IN ('A', 'B', 'C')),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS gastro_nutrition_plans (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 calorie_target INTEGER,
 protein_target DECIMAL(5,2),
```

```

route TEXT CHECK (route IN ('Oral', 'Enteral', 'Parenteral')),
special_requirements TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

```
-- Enable RLS
```

```

ALTER TABLE gastro_endoscopy_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE gastro_liver_metrics ENABLE ROW LEVEL SECURITY;
ALTER TABLE gastro_nutrition_plans ENABLE ROW LEVEL SECURITY;

```

```

CREATE POLICY tenant_isolation_policy ON gastro_endoscopy_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

```

```

CREATE POLICY tenant_isolation_policy ON gastro_liver_metrics
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

```

```

CREATE POLICY tenant_isolation_policy ON gastro_nutrition_plans
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

```

```

=====
FILE: migrations\e5_01_drug_batches_down.sql
=====

```

```

-- e5_01_drug_batches_down.sql (rollback of e5_01_drug_batches_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ?????? ???????? ?????? ?? ???? drug_batches. idempotent (IF EXISTS).
BEGIN;

```

```

DROP POLICY IF EXISTS rls_drug_batches_tenant_isolation ON drug_batches;
DROP INDEX IF EXISTS idx_drug_batches_fefo;
DROP INDEX IF EXISTS idx_drug_batches_tenant_id;
ALTER TABLE IF EXISTS drug_batches DROP CONSTRAINT IF EXISTS chk_drug_batches_qty;
DROP TABLE IF EXISTS drug_batches;

COMMIT;

```

```

=====
FILE: migrations\e5_01_drug_batches_up.sql
=====

```

```

-- =====
-- e5_01_drug_batches_up.sql
-- E5 PHARMACY ? Per-lot drug batches for FEFO (First-Expired-First-Out) dispensing.
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ????? ????? ???????? ?? ???? ???? (pharmacy_drug_catalog.stock_qty + expiry_date ??? ????)
-- ??? ?????/??????? ??? ???? (drug_batches) ?????? ?????? DATE ????? ??? ????? ??? ???? ??????
-- ?? ???? ???? ??? ?????? ???????? ?????? (FEFO) ??? ???? ?????? ?? ???? ??????.
-- ??? ???? ??? FORCE RLS: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS
-- + ????? ??? tenant_id + ???? tenant_id + ???? (drug_id, expiry_date) ?????? FEFO.

```

```

--
-- idempotent: CREATE TABLE IF NOT EXISTS + DROP POLICY IF EXISTS + CREATE INDEX IF NOT EXISTS
-- + DROP/ADD CONSTRAINT IF EXISTS ?????? (????? ??????? ???????).
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS drug_batches (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 branch_id INTEGER,
 drug_id INTEGER NOT NULL, -- -> pharmacy_drug_catalog.id (no hard FK: catalog
managed out-of-band)
 drug_name TEXT, -- denormalized for logs/audit when catalog row abs
ent
 lot TEXT, -- batch / lot number (supplier)
 expiry_date DATE NOT NULL, -- real DATE per lot (FEFO key); never dispense whe
n < CURRENT_DATE
 qty_received INTEGER NOT NULL DEFAULT 0,
 qty_on_hand INTEGER NOT NULL DEFAULT 0, -- decremented FEFO on dispense; never below 0
 cost_price NUMERIC(12,2) DEFAULT 0,
 supplier_id INTEGER,
 received_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT chk_drug_batches_qty CHECK (qty_on_hand >= 0 AND qty_received >= 0)
);

-- idempotent: ensure the qty CHECK exists even when the table pre-dates this migration
ALTER TABLE drug_batches DROP CONSTRAINT IF EXISTS chk_drug_batches_qty;
ALTER TABLE drug_batches ADD CONSTRAINT chk_drug_batches_qty CHECK (qty_on_hand >= 0 AND qty_received >= 0);

CREATE INDEX IF NOT EXISTS idx_drug_batches_tenant_id ON drug_batches (tenant_id);
-- FEFO selection index: per tenant + drug, ordered by earliest expiry first.
CREATE INDEX IF NOT EXISTS idx_drug_batches_fefo ON drug_batches (tenant_id, drug_id, expiry_date);

ALTER TABLE drug_batches ENABLE ROW LEVEL SECURITY;
ALTER TABLE drug_batches FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_drug_batches_tenant_isolation ON drug_batches;
CREATE POLICY rls_drug_batches_tenant_isolation ON drug_batches
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

COMMIT;

=====
FILE: migrations\e5_01_drug_batches_validate.sql
=====

-- e5_01_drug_batches_validate.sql (run AFTER e5_01_drug_batches_up.sql; read-only)
-- PASS = drug_batches exists with tenant_id FK + NOT NULL + FORCE RLS + isolation policy + FEFO indexes + qty
CHECK.

```



```

SELECT
 (SELECT count(*) FROM information_schema.tables WHERE table_name='drug_batches') AS drug_batches_exists,
 -- expect 1
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='drug_batches'
 AND column_name IN ('tenant_id','branch_id','drug_id','drug_name','lot','expiry_date','qty_received','q
ty_on_hand','cost_price','supplier_id','received_at')) AS drug_batches_cols, -- expect 11
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='drug_batches') AS drug_batches_force_rls,
 -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='drug_batches' AND policyname='rls_drug_batches_tenant_iso
lation') AS drug_batches_policy, -- expect 1
 (SELECT count(*) FROM pg_constraint WHERE conname='chk_drug_batches_qty') AS drug_batches_qty_check,
 -- expect 1
 (SELECT count(*) FROM pg_indexes WHERE tablename='drug_batches' AND indexname='idx_drug_batches_tenant_id')
AS drug_batches_tenant_idx, -- expect 1
 (SELECT count(*) FROM pg_indexes WHERE tablename='drug_batches' AND indexname='idx_drug_batches_fefo') AS dr
ug_batches_fefo_idx; -- expect 1

-- ----- tenant_id is NOT NULL ; expiry_date is NOT NULL -----
SELECT
 (SELECT count(*)::int FROM information_schema.columns
 WHERE table_name='drug_batches' AND column_name='tenant_id' AND is_nullable='NO') AS tenant_id_not_null,
 -- expect 1
 (SELECT count(*)::int FROM information_schema.columns
 WHERE table_name='drug_batches' AND column_name='expiry_date' AND is_nullable='NO') AS expiry_date_not_nu
ll; -- expect 1

-- ----- FK to tenants(id) -----
SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
 WHERE conrelid='drug_batches'::regclass AND confrelid='tenants'::regclass AND contype='f';
 -- expect 1

=====
FILE: migrations\e5_02_pharmacy_dispense_down.sql
=====

-- e5_02_pharmacy_dispense_down.sql (rollback of e5_02_pharmacy_dispense_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ???? ?????? ???????? ??????? ?? ???? pharmacy_dispense? ??????? ?? ????? ?????? ??? ????????.
-- idempotent (IF EXISTS). ??????: ?? ???? pharmacy_prescriptions_queue ???? (????? ???? ??????).
BEGIN;

DROP POLICY IF EXISTS rls_pharmacy_dispense_tenant_isolation ON pharmacy_dispense;
DROP INDEX IF EXISTS idx_pharmacy_dispense_patient;
DROP INDEX IF EXISTS idx_pharmacy_dispense_prescription;
DROP INDEX IF EXISTS idx_pharmacy_dispense_tenant_id;
ALTER TABLE IF EXISTS pharmacy_dispense DROP CONSTRAINT IF EXISTS chk_pharmacy_dispense_status;
ALTER TABLE IF EXISTS pharmacy_dispense DROP CONSTRAINT IF EXISTS chk_pharmacy_dispense_qty;
DROP TABLE IF EXISTS pharmacy_dispense;

-- roll back the additive verification columns (only the columns this migration added)
DO $$do$

```

```

BEGIN
 IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name='pharmacy_prescriptions_queue') THEN
 ALTER TABLE pharmacy_prescriptions_queue DROP COLUMN IF EXISTS verified_at;
 ALTER TABLE pharmacy_prescriptions_queue DROP COLUMN IF EXISTS verified_by;
 END IF;
END
do;

COMMIT;

```

```

=====
FILE: migrations\e5_02_pharmacy_dispense_up.sql
=====

```

```

-- =====
-- e5_02_pharmacy_dispense_up.sql
-- E5 PHARMACY ? Pharmacist verification + dispense ledger (per-batch FEFO line).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ??? ????? (pharmacy_dispense) ??? ????? ????? (prescription_id) + ????? + ?????
-- ???????? (drug_batch_id ?? drug_batches) + ????? + ?? ??? (verified_by) + ?? ??? (dispensed_by).
-- ??? ?????: ??? ????? Pending -> Verified -> Dispensed (?? Rejected). ????? ?? ??? ??? ??????.
-- ??? ??? FORCE RLS: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS + ??? + ???.
--
-- ??? ??? ??? ????? (idempotent) ????? ????? ????? ??? pharmacy_prescriptions_queue:
-- verified_by / verified_at (????? ????? ?????? route_level_ddl_batch_c? ????? ?? ?? ???).
--
-- idempotent: CREATE TABLE/INDEX IF NOT EXISTS + DROP POLICY IF EXISTS + ADD COLUMN IF NOT EXISTS
-- + DROP/ADD CONSTRAINT IF EXISTS.
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS pharmacy_dispense (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 branch_id INTEGER,
 prescription_id INTEGER, -- -> pharmacy_prescriptions_queue.id (queue item d
ispensed)
 patient_id INTEGER,
 drug_id INTEGER, -- -> pharmacy_drug_catalog.id
 drug_batch_id INTEGER REFERENCES drug_batches(id), -- the FEFO batch the qty was taken from
 drug_name TEXT,
 qty INTEGER NOT NULL DEFAULT 0,
 verified_by INTEGER, -- pharmacist who passed the CDS verify checkpoint
 verified_at TIMESTAMP,
 dispensed_by INTEGER,
 dispensed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 status TEXT NOT NULL DEFAULT 'Dispensed',
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT chk_pharmacy_dispense_qty CHECK (qty >= 0),
 CONSTRAINT chk_pharmacy_dispense_status CHECK (status IN ('Dispensed','Reversed'))
);

```

```

ALTER TABLE pharmacy_dispense DROP CONSTRAINT IF EXISTS chk_pharmacy_dispense_qty;
ALTER TABLE pharmacy_dispense ADD CONSTRAINT chk_pharmacy_dispense_qty CHECK (qty >= 0);
ALTER TABLE pharmacy_dispense DROP CONSTRAINT IF EXISTS chk_pharmacy_dispense_status;
ALTER TABLE pharmacy_dispense ADD CONSTRAINT chk_pharmacy_dispense_status CHECK (status IN ('Dispensed','Reversed'));

CREATE INDEX IF NOT EXISTS idx_pharmacy_dispense_tenant_id ON pharmacy_dispense (tenant_id);
CREATE INDEX IF NOT EXISTS idx_pharmacy_dispense_prescription ON pharmacy_dispense (tenant_id, prescription_id);
CREATE INDEX IF NOT EXISTS idx_pharmacy_dispense_patient ON pharmacy_dispense (tenant_id, patient_id);

ALTER TABLE pharmacy_dispense ENABLE ROW LEVEL SECURITY;
ALTER TABLE pharmacy_dispense FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_pharmacy_dispense_tenant_isolation ON pharmacy_dispense;
CREATE POLICY rls_pharmacy_dispense_tenant_isolation ON pharmacy_dispense
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);

-- ----- verification-state columns on the existing queue (table provisioned out-of-band) -----
-- Guarded so this migration is safe even if pharmacy_prescriptions_queue is not yet present.
DO do
BEGIN
 IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name='pharmacy_prescriptions_queue') THEN
 ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS verified_by INTEGER;
 ALTER TABLE pharmacy_prescriptions_queue ADD COLUMN IF NOT EXISTS verified_at TIMESTAMP;
 END IF;
END
do;

COMMIT;

=====
FILE: migrations\e5_02_pharmacy_dispense_validate.sql
=====

-- e5_02_pharmacy_dispense_validate.sql (run AFTER e5_02_pharmacy_dispense_up.sql; read-only)
-- PASS = pharmacy_dispense exists with tenant_id FK + NOT NULL + FORCE RLS + isolation policy + indexes + checks.

SELECT
 (SELECT count(*) FROM information_schema.tables WHERE table_name='pharmacy_dispense') AS dispense_exists,
 -- expect 1
 (SELECT count(*) FROM information_schema.columns
 WHERE table_name='pharmacy_dispense'
 AND column_name IN ('tenant_id','prescription_id','patient_id','drug_id','drug_batch_id','qty','verified_by','dispensed_by','dispensed_at','status')) AS dispense_cols, -- expect 10
 (SELECT relforcerowsecurity FROM pg_class WHERE relname='pharmacy_dispense') AS dispense_force_rls,
 -- expect t
 (SELECT count(*) FROM pg_policies WHERE tablename='pharmacy_dispense' AND policyname='rls_pharmacy_dispense_tenant_isolation') AS dispense_policy, -- expect 1

```

```

(SELECT count(*) FROM pg_constraint WHERE conname='chk_pharmacy_dispense_qty') AS dispense_qty_check,
 -- expect 1
(SELECT count(*) FROM pg_constraint WHERE conname='chk_pharmacy_dispense_status') AS dispense_status_check,
 -- expect 1
(SELECT count(*) FROM pg_indexes WHERE tablename='pharmacy_dispense' AND indexname='idx_pharmacy_dispense_tenant_id') AS dispense_tenant_idx; -- expect 1

```

```

-- ----- tenant_id is NOT NULL -----

```

```

SELECT count(*)::int AS tenant_id_not_null FROM information_schema.columns
 WHERE table_name='pharmacy_dispense' AND column_name='tenant_id' AND is_nullable='NO';
 -- expect 1

```

```

-- ----- FK to tenants(id) -----

```

```

SELECT count(*)::int AS fk_to_tenants FROM pg_constraint
 WHERE conrelid='pharmacy_dispense'::regclass AND confrelid='tenants'::regclass AND contype='f';
 -- expect 1

```

```

-- ----- verification columns present on the queue -----

```

```

SELECT count(*)::int AS queue_verify_cols FROM information_schema.columns
 WHERE table_name='pharmacy_prescriptions_queue' AND column_name IN ('verified_by','verified_at');
 -- expect 2 (if queue table present)

```

```

=====

```

```

FILE: migrations\e5_03_controlled_log_down.sql

```

```

=====

```

```

-- e5_03_controlled_log_down.sql (rollback of e5_03_controlled_log_up.sql)
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL.
-- ??? ???? ???? ???? ? ???? controlled_drug_log ???? ? ???? ??????.
-- idempotent (IF EXISTS). ?? ???? pharmacy_drug_catalog ???? (???? ???? ?????).
BEGIN;

```

```

DROP POLICY IF EXISTS rls_controlled_drug_log_tenant_isolation ON controlled_drug_log;
DROP INDEX IF EXISTS idx_controlled_log_drug;
DROP INDEX IF EXISTS idx_controlled_log_tenant_id;
ALTER TABLE IF EXISTS controlled_drug_log DROP CONSTRAINT IF EXISTS chk_controlled_log_qty;
DROP TABLE IF EXISTS controlled_drug_log;

```

```

DO do
BEGIN
 IF EXISTS (SELECT 1 FROM information_schema.tables WHERE table_name='pharmacy_drug_catalog') THEN
 ALTER TABLE pharmacy_drug_catalog DROP COLUMN IF EXISTS schedule_class;
 ALTER TABLE pharmacy_drug_catalog DROP COLUMN IF EXISTS is_controlled;
 END IF;
END
do;

COMMIT;

```

```

=====

```

FILE: migrations\e5\_03\_controlled\_log\_up.sql

```
=====

-- =====
-- e5_03_controlled_log_up.sql
-- E5 PHARMACY ? Controlled / high-alert drug register (dual-record, witnessed).
-- CANDIDATE ONLY ? DO NOT EXECUTE ON PRODUCTION WITHOUT EXPLICIT DDL APPROVAL (DB gate).
--
-- ?????: ??? ??? ????? ??????? ???????/???????? (controlled_drug_log): ?? ??? ????? ????? ???
-- ????? ??? ?????? + ?????? + ?????? ???/??? + ?? ??? (dispensed_by) + ?????? (witnessed_by) + ??????.
-- ????? FAIL-CLOSED: ?? ????? ??? ?????? ??? witnessed_by (????? ???? ??????? ?? ?????? ??? ? NULL
-- ??? ????? ?? ????? ??? ?????? ??????? ??? ????? ?????? ??? ?? ?????? ??? ?????).
-- ???? (idempotent) ???? is_controlled / schedule_class ??? pharmacy_drug_catalog.
-- ??? ???? ??? FORCE RLS: tenant_id NOT NULL REFERENCES tenants(id) + ENABLE+FORCE RLS + ??? + ???.
--
-- idempotent: CREATE TABLE/INDEX IF NOT EXISTS + DROP POLICY IF EXISTS + ADD COLUMN IF NOT EXISTS.
-- =====
BEGIN;

CREATE TABLE IF NOT EXISTS controlled_drug_log (
 id SERIAL PRIMARY KEY,
 tenant_id INTEGER NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
 branch_id INTEGER,
 drug_id INTEGER,
 drug_name TEXT,
 drug_batch_id INTEGER,
 dispense_id INTEGER, -- -> pharmacy_dispense.id
 prescription_id INTEGER,
 patient_id INTEGER,
 qty INTEGER NOT NULL DEFAULT 0,
 balance_before INTEGER,
 balance_after INTEGER,
 schedule_class TEXT, -- e.g. 'CDII'/'CDIII'/'high-alert'
 dispensed_by INTEGER, -- pharmacist
 witnessed_by INTEGER, -- second witness (route requires this for controll
ed)
 at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT chk_controlled_log_qty CHECK (qty >= 0)
);

ALTER TABLE controlled_drug_log DROP CONSTRAINT IF EXISTS chk_controlled_log_qty;
ALTER TABLE controlled_drug_log ADD CONSTRAINT chk_controlled_log_qty CHECK (qty >= 0);

CREATE INDEX IF NOT EXISTS idx_controlled_log_tenant_id ON controlled_drug_log (tenant_id);
CREATE INDEX IF NOT EXISTS idx_controlled_log_drug ON controlled_drug_log (tenant_id, drug_id, at);

ALTER TABLE controlled_drug_log ENABLE ROW LEVEL SECURITY;
ALTER TABLE controlled_drug_log FORCE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS rls_controlled_drug_log_tenant_isolation ON controlled_drug_log;
CREATE POLICY rls_controlled_drug_log_tenant_isolation ON controlled_drug_log
 FOR ALL
 USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer)
 WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::integer);
```

-- ----- controlled flags on the catalog (table managed out-of-band; add only what is missing) -----

DO \$do\$

BEGIN

IF EXISTS (SELECT 1 FROM information\_schema.tables WHERE table\_name='pharmacy\_drug\_catalog') THEN

ALTER TABLE pharmacy\_drug\_catalog ADD COLUMN IF NOT EXISTS is\_controlled INTEGER DEFAULT 0;

ALTER TABLE pharmacy\_drug\_catalog ADD COLUMN IF NOT EXISTS schedule\_class TEXT;

END IF;

END

\$do\$;

COMMIT;

=====

FILE: migrations\e5\_03\_controlled\_log\_validate.sql

=====

-- e5\_03\_controlled\_log\_validate.sql (run AFTER e5\_03\_controlled\_log\_up.sql; read-only)

-- PASS = controlled\_drug\_log exists with tenant\_id FK + NOT NULL + FORCE RLS + isolation policy + indexes + qty CHECK.

SELECT

(SELECT count(\*) FROM information\_schema.tables WHERE table\_name='controlled\_drug\_log') AS log\_exists,

-- expect 1

(SELECT count(\*) FROM information\_schema.columns

WHERE table\_name='controlled\_drug\_log'

AND column\_name IN ('tenant\_id','drug\_id','drug\_name','dispense\_id','prescription\_id','patient\_id','qty','balance\_before','balance\_after','schedule\_class','dispensed\_by','witnessed\_by','at')) AS log\_cols, -- expect 13

(SELECT relforcerowsecurity FROM pg\_class WHERE relname='controlled\_drug\_log') AS log\_force\_rls,

-- expect t

(SELECT count(\*) FROM pg\_policies WHERE tablename='controlled\_drug\_log' AND policyname='rls\_controlled\_drug\_log\_tenant\_isolation') AS log\_policy, -- expect 1

(SELECT count(\*) FROM pg\_constraint WHERE conname='chk\_controlled\_log\_qty') AS log\_qty\_check,

-- expect 1

(SELECT count(\*) FROM pg\_indexes WHERE tablename='controlled\_drug\_log' AND indexname='idx\_controlled\_log\_tenant\_id') AS log\_tenant\_idx; -- expect 1

-- ----- tenant\_id is NOT NULL -----

SELECT count(\*)::int AS tenant\_id\_not\_null FROM information\_schema.columns

WHERE table\_name='controlled\_drug\_log' AND column\_name='tenant\_id' AND is\_nullable='NO';

-- expect 1

-- ----- FK to tenants(id) -----

SELECT count(\*)::int AS fk\_to\_tenants FROM pg\_constraint

WHERE conrelid='controlled\_drug\_log'::regclass AND confrelid='tenants'::regclass AND contype='f';

-- expect 1

-- ----- controlled flags present on the catalog -----

SELECT count(\*)::int AS catalog\_controlled\_cols FROM information\_schema.columns

WHERE table\_name='pharmacy\_drug\_catalog' AND column\_name IN ('is\_controlled','schedule\_class');

-- expect 2 (if catalog present)

=====

FILE: migrations\e60\_nephrology\_dialysis\_down.sql

=====

-- Down migration for Nephrology & Dialysis Specialized Data  
-- Target: namaweb/migrations\e60\_nephrology\_dialysis\_down.sql

DROP TABLE IF EXISTS renal\_transplant\_logs;  
DROP TABLE IF EXISTS dialysis\_sessions;  
DROP TABLE IF EXISTS nephrology\_gfr\_logs;

=====

FILE: migrations\e60\_nephrology\_dialysis\_up.sql

=====

-- Migration for Nephrology & Dialysis Specialized Data  
-- Target: namaweb/migrations/e60\_nephrology\_dialysis\_up.sql

```
CREATE TABLE IF NOT EXISTS nephrology_gfr_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 creatinine DECIMAL(5,2),
 age INTEGER,
 sex TEXT CHECK (sex IN ('Male', 'Female')),
 calculated_egfr DECIMAL(5,2),
 stage INTEGER CHECK (stage BETWEEN 1 AND 5),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS dialysis_sessions (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 session_date DATE DEFAULT CURRENT_DATE,
 pre_weight DECIMAL(5,2),
 post_weight DECIMAL(5,2),
 uf_volume DECIMAL(5,2),
 blood_flow_rate INTEGER,
 la_duration_min INTEGER,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS renal_transplant_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 donor_type TEXT CHECK (donor_type IN ('Living', 'Deceased')),
 transplant_date DATE,
```

```

 immunosuppressant_regimen TEXT,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE nephrology_gfr_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE dialysis_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE renal_transplant_logs ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON nephrology_gfr_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON dialysis_sessions
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON renal_transplant_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

=====
FILE: migrations\e60_nephrology_extensions_down.sql
=====

-- Migration: e60_nephrology_extensions_down.sql
-- Description: Reverses the nephrology extensions migration.

BEGIN;

DROP TABLE IF EXISTS nephrology_labs;
DROP TABLE IF EXISTS renal_transplant_records;
DROP TABLE IF EXISTS dialysis_sessions;

COMMIT;

=====
FILE: migrations\e60_nephrology_extensions_up.sql
=====

-- Migration: e60_nephrology_extensions_up.sql
-- Description: Adds specialized tables for Nephrology, Dialysis, and Renal Transplantation.
-- Compliance: PDPL / RLS Enabled

BEGIN;

-- 1. Dialysis Sessions (HD, PD, Home Dialysis)
CREATE TABLE IF NOT EXISTS dialysis_sessions (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 session_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 session_type VARCHAR(50), -- e.g., Hemodialysis, Peritoneal, Home

```



```

pre_weight_kg FLOAT,
post_weight_kg FLOAT,
uf_volume_liters FLOAT,
ktv_value FLOAT,
urr_percentage FLOAT,
blood_flow_qb INTEGER,
dialysate_flow_qd INTEGER,
notes TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tenant_neph FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
CONSTRAINT fk_patient_neph FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

```

## -- 2. Renal Transplantation Records

```

CREATE TABLE IF NOT EXISTS renal_transplant_records (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 donor_id UUID,
 transplant_date DATE,
 hla_match_score FLOAT,
 immunosuppressant_regimen TEXT,
 graft_function_status VARCHAR(100),
 last_biopsy_date DATE,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_tenant_trans FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
 CONSTRAINT fk_patient_trans FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

```

## -- 3. Specialized Nephrology Labs

```

CREATE TABLE IF NOT EXISTS nephrology_labs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 sample_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 creatinine_mgdl FLOAT,
 urea_mgdl FLOAT,
 potassium_meq_l FLOAT,
 phosphorus_mgdl FLOAT,
 calcium_mgdl FLOAT,
 calculated_gfr FLOAT,
 gfr_formula_used VARCHAR(50), -- e.g., CKD-EPI, MDRD
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_tenant_lab FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
 CONSTRAINT fk_patient_lab FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);

```

## -- Enable Row Level Security (RLS)

```

ALTER TABLE dialysis_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE renal_transplant_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE nephrology_labs ENABLE ROW LEVEL SECURITY;

```

## -- Create RLS Policies (Golden Access Rule: Tenant Isolation)

```

CREATE POLICY tenant_isolation_policy ON dialysis_sessions

```

```

 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON renal_transplant_records
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON nephrology_labs
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

COMMIT;

```

```

=====
FILE: migrations\e61_oncology_hematology_down.sql
=====

```

```

-- Down migration for Hematology & Oncology Specialized Data
-- Target: namaweb/migrations/e61_oncology_hematology_down.sql

```

```

DROP TABLE IF EXISTS bmt_registry;
DROP TABLE IF EXISTS oncology_toxicity_logs;
DROP TABLE IF EXISTS oncology_chemo_cycles;

```

```

=====
FILE: migrations\e61_oncology_hematology_up.sql
=====

```

```

-- Migration for Hematology & Oncology Specialized Data
-- Target: namaweb/migrations/e61_oncology_hematology_up.sql

```

```

CREATE TABLE IF NOT EXISTS oncology_chemo_cycles (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 protocol_name TEXT NOT NULL,
 cycle_number INTEGER NOT NULL,
 day_number INTEGER NOT NULL,
 drug_name TEXT NOT NULL,
 dose_mg_m2 DECIMAL(10,2),
 actual_dose_mg DECIMAL(10,2),
 admin_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 admin_status TEXT CHECK (admin_status IN ('Planned', 'Administered', 'Delayed', 'Cancelled')),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE IF NOT EXISTS oncology_toxicity_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 cycle_id UUID NOT NULL REFERENCES oncology_chemo_cycles(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 organ_system TEXT NOT NULL, -- e.g., 'Hematologic', 'Gastrointestinal', 'Neurologic'
 ctcae_grade INTEGER CHECK (ctcae_grade BETWEEN 1 AND 5),
 onset_date DATE,

```

```

 resolution_date DATE,
 intervention TEXT,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS bmt_registry (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 stem_cell_source TEXT CHECK (stem_cell_source IN ('Autologous', 'Allogeneic', 'Haploidentical')),
 conditioning_regimen TEXT,
 infusion_date DATE,
 engraftment_date DATE,
 gvhd_grade INTEGER CHECK (gvhd_grade BETWEEN 0 AND 4),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Enable RLS
ALTER TABLE oncology_chemo_cycles ENABLE ROW LEVEL SECURITY;
ALTER TABLE oncology_toxicity_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE bmt_registry ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON oncology_chemo_cycles
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON oncology_toxicity_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON bmt_registry
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

=====
FILE: migrations\e62_endocrine_extensions_down.sql
=====

-- Migration: e62_endocrine_extensions_down.sql
-- Description: Reverses the endocrine extensions migration.

BEGIN;

DROP TABLE IF EXISTS thyroid_metrics;
DROP TABLE IF EXISTS diabetes_logs;

COMMIT;

=====
FILE: migrations\e62_endocrine_extensions_up.sql
=====

-- Migration: e62_endocrine_extensions_up.sql

```

-- Description: Adds specialized tables for Endocrinology and Diabetes.  
-- Compliance: PDPL / RLS Enabled

BEGIN;

-- 1. Diabetes Glucose Logs

```
CREATE TABLE IF NOT EXISTS diabetes_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 record_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 glucose_value_mgdl FLOAT,
 hba1c_percentage FLOAT,
 insulin_dose_units FLOAT,
 meal_type VARCHAR(50), -- Fasting, Post-Prandial, etc.
 notes TEXT,
 CONSTRAINT fk_tenant_dia FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
 CONSTRAINT fk_patient_dia FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

-- 2. Thyroid & Endocrine Metrics

```
CREATE TABLE IF NOT EXISTS thyroid_metrics (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 tenant_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 record_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 tsh_level FLOAT,
 t4_free FLOAT,
 t3_total FLOAT,
 nodule_size_mm FLOAT,
 hormone_imbalance_type VARCHAR(100),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_tenant_thy FOREIGN KEY (tenant_id) REFERENCES tenants(id) ON DELETE CASCADE,
 CONSTRAINT fk_patient_thy FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

-- Enable Row Level Security (RLS)

```
ALTER TABLE diabetes_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE thyroid_metrics ENABLE ROW LEVEL SECURITY;
```

-- Create RLS Policies (Golden Access Rule: Tenant Isolation)

```
CREATE POLICY tenant_isolation_policy ON diabetes_logs
 USING (tenant_id = current_setting('app.current_tenant')::UUID);

CREATE POLICY tenant_isolation_policy ON thyroid_metrics
 USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

COMMIT;

```
=====
FILE: migrations\e62_endocrinology_diabetes_down.sql
=====
```

```
-- Down migration for Endocrinology & Diabetes Specialized Data
-- Target: namaweb/migrations/e62_endocrinology_diabetes_down.sql
```

```
DROP TABLE IF EXISTS endocrine_thyroid_logs;
DROP TABLE IF EXISTS diabetes_complications;
DROP TABLE IF EXISTS endocrine_glucose_logs;
```

```
=====
FILE: migrations\e62_endocrinology_diabetes_up.sql
=====
```

```
-- Migration for Endocrinology & Diabetes Specialized Data
-- Target: namaweb/migrations/e62_endocrinology_diabetes_up.sql
```

```
CREATE TABLE IF NOT EXISTS endocrine_glucose_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 value DECIMAL(5,2),
 timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
 context TEXT CHECK (context IN ('Fasting', 'Post-prandial', 'Bedtime', 'Random')),
 insulin_dose DECIMAL(5,2),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS diabetes_complications (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 complication_type TEXT CHECK (complication_type IN ('Retinopathy', 'Nephropathy', 'Neuropathy', 'Foot Ulcer')),
 severity_grade INTEGER CHECK (severity_grade BETWEEN 0 AND 4),
 last_screening_date DATE,
 notes TEXT,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS endocrine_thyroid_logs (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 patient_id UUID NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
 tenant_id UUID NOT NULL,
 tsh DECIMAL(5,2),
 t3 DECIMAL(5,2),
 t4 DECIMAL(5,2),
 thyroid_volume DECIMAL(5,2),
 nodule_size DECIMAL(5,2),
 created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Enable RLS
ALTER TABLE endocrine_glucose_logs ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE diabetes_complications ENABLE ROW LEVEL SECURITY;
ALTER TABLE endocrine_thyroid_logs ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON endocrine_glucose_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON diabetes_complications
 USING (tenant_id = current_setting('app.current_tenant')::uuid);

CREATE POLICY tenant_isolation_policy ON endocrine_thyroid_logs
 USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

=====

FILE: migrations\e63\_rheumatology\_immunology\_down.sql

=====

```
-- Down migration for Rheumatology & Immunology Specialized Data
-- Target: namaweb/migrations/e63_rheumatology_immunology_down.sql
```

```
DROP TABLE IF EXISTS rheum_biologic_tracking;
DROP TABLE IF EXISTS rheum_activity_scores;
DROP TABLE IF EXISTS rheum_joint_logs;
```